Blue Book Edition 14

TECHNICAL REPORT

Companion Specification for Energy Metering

COSEM Interface Classes

and

OBIS Object Identification System

DLMS UA 1000-1 Ed. 14

31 August 2020

DLMS User Association

© Copyright 1997-2020 DLMS User Association

dlms™
device
language
message
specification

CONTENTS

Foreword 15
List of main technical changes in Edition 13 17
List of main technical changes in Edition 14 18
Introduction 19
1 Scope 20
2 Referenced documents 21
3 Terms, definitions and abbreviated terms 24
4 The COSEM interface classes 39
4.1 Basic principles 39
4.1.1 General 39
4.1.2 Referencing methods 40
4.1.3 Reserved base_names for special COSEM objects 40
4.1.4 Class description notation 40
4.1.5 Common data types 43
4.1.6 Data formats 44
4.1.7 The COSEM server model 48
4.1.8 COSEM logical device 49
4.1.9 Information security 50
4.2 Overview of the COSEM interface classes 51
4.3 Interface classes for parameters and measurement data 56
4.3.1 Data (class_id = 1, version = 0) 56
4.3.2 Register (class_id = 3, version = 0) 57
4.3.3 Extended register (class_id = 4, version = 0) 62
4.3.4 Demand register (class_id = 5, version = 0) 63
4.3.5 Register activation (class_id = 6, version = 0) 66
4.3.6 Profile generic (class_id = 7, version = 1) 68
4.3.7 Utility tables (class_id = 26, version = 0) 73
4.3.8 Register table (class_id = 61, version = 0) 74
4.3.9 Status mapping (class_id = 63, version = 0) 76
4.3.10 Compact data (class_id = 62, version = 1) 77
4.4 Interface classes for access control and management 87
4.4.1 Overview 87
4.4.2 Client user identification 87
4.4.3 Association SN (class_id = 12, version 4) 88
4.4.4 Association LN (class_id = 15, version 3) 92
4.4.5 SAP assignment (class_id = 17, version = 0) 98
4.4.6 Image transfer (class_id = 18, version = 0) 99
4.4.7 Security setup (class_id = 64, version = 1) 106
4.4.8 Push interface class (class_id = 40, version = 2) 112
4.4.9 COSEM data protection (class_id = 30, version = 0) 123
4.4.10 Function control (class_id = 122, version = 0) 141
4.4.11 Array manager (class_id = 123, version = 0) 142
4.4.12 Communication port protection (class_id = 124, version = 0) 148
4.5 Interface classes for time- and event bound control 151

© Copyright 1997-2020 DLMS User Association

4.5.1 Clock (class_id = 8, version = 0) 151
4.5.2 Script table (class_id = 9, version = 0) 153
4.5.3 Schedule (class_id = 10, version = 0) 155
4.5.4 Special days table (class_id = 11, version = 0) 158
4.5.5 Activity calendar (class_id = 20, version = 0) 159
4.5.6 Register monitor (class_id = 21, version = 0) 162
4.5.7 Single action schedule (class_id = 22, version = 0) 163
4.5.8 Disconnect control (class_id = 70, version = 0) 164
4.5.9 Limiter (class_id = 71, version = 0) 167
4.5.10 Parameter monitor (class_id = 65, version = 1) 169
4.5.11 Sensor manager interface (class_id = 67, version = 0) 173
4.5.12 Arbitrator (class_id = 68, version = 0) 176
4.5.13 Modelling examples: tariffication and billing 181
4.6 Payment metering related interface classes 182
4.6.1 Overview of the COSEM accounting model 182
4.6.2 Account (class_id = 111, version = 0) 185
4.6.3 Credit interface (class_id = 112, version = 0) 194
4.6.4 Charge (class_id = 113, version = 0) 205
4.6.5 Token gateway (class_id = 115, version = 0) 211
4.7 Interface classes for setting up data exchange via local ports and modems 213
4.7.1 IEC local port setup (class_id = 19, version = 1) 213
4.7.2 IEC HDLC setup (class_id = 23, version = 1) 214
4.7.3 IEC twisted pair (1) setup (class_id = 24, version = 1) 216
4.7.4 Modem configuration (class_id = 27, version = 1) 219
4.7.5 Auto answer (class_id = 28, version = 2) 220
4.7.6 Auto connect (class_id = 29, version = 2) 223
4.7.7 GPRS modem setup (class_id = 45, version = 0) 225
4.7.8 GSM diagnostic (class_id = 47, version = 2) 226
4.7.9 LTE monitoring (class_id = 151, version = 1) 229
4.8 Interface classes for setting up data exchange via M-Bus 233
4.8.1 Overview 233
4.8.2 M-Bus slave port setup (class_id = 25, version = 0) 233
4.8.3 M-Bus client (class_id = 72, version = 1) 234
4.8.4 Wireless Mode Q channel (class_id = 73, version = 1) 239
4.8.5 M-Bus master port setup (class_id = 74, version = 0) 239
4.8.6 DLMS/COSEM server M-Bus port setup (class_id = 76, version = 0) 240
4.8.7 M-Bus diagnostic (class_id = 77, version = 0) 242
4.9 Interface classes for setting up data exchange over the Internet 245
4.9.1 TCP-UDP setup (class_id = 41, version = 0) 245
4.9.2 IPv4 setup (class_id = 42, version = 0) 246
4.9.3 IPv6 setup (class_id = 48, version = 0) 249
4.9.4 MAC address setup (class_id = 43, version = 0) 251
4.9.5 PPP setup (class_id = 44, version = 0) 252
4.9.6 SMTP setup (class_id = 46, version = 0) 256
4.9.7 NTP setup (class_id = 100, version = 0) 257
4.10 Interface classes for setting up data exchange using S-FSK PLC 259

© Copyright 1997-2020 DLMS User Association

4.10.1 General 259
4.10.2 Overview 259
4.10.3 S-FSK Phy&MAC set-up (class_id = 50, version = 1) 262
4.10.4 S-FSK Active initiator (class_id = 51, version = 0) 266
4.10.5 S-FSK MAC synchronization timeouts (class_id = 52, version = 0) 267
4.10.6 S-FSK MAC counters (class_id = 53, version = 0) 269
4.10.7 IEC 61334-4-32 LLC setup (class_id = 55, version = 1) 272
4.10.8 S-FSK Reporting system list (class_id = 56, version = 0) 273
4.11 Interface classes for setting up the LLC layer for ISO/IEC 8802-2 274
4.11.1 General 274
4.11.2 ISO/IEC 8802-2 LLC Type 1 setup (class_id = 57, version = 0) 274
4.11.3 ISO/IEC 8802-2 LLC Type 2 setup (class_id = 58, version = 0) 274
4.11.4 ISO/IEC 8802-2 LLC Type 3 setup (class_id = 59, version = 0) 276
4.12 Interface classes for setting up and managing DLMS/COSEM narrowband OFDM PLC profile for PRIME networks 279
4.12.1 Overview 279
4.12.2 Mapping of PRIME NB OFDM PLC PIB attributes to COSEM IC attributes 280
4.12.3 61334-4-32 LLC SSCS setup (class_id = 80, version = 0) 282
4.12.4 PRIME NB OFDM PLC Physical layer parameters 282
4.12.5 PRIME NB OFDM PLC Physical layer counters (class_id = 81, version = 0) 282
4.12.6 PRIME NB OFDM PLC MAC setup (class_id = 82, version = 0) 283
4.12.7 PRIME NB OFDM PLC MAC functional parameters (class_id = 83 version = 0) 285
4.12.8 PRIME NB OFDM PLC MAC counters (class_id = 84, version = 0) 286
4.12.9 PRIME NB OFDM PLC MAC network administration data (class_id = 85, version = 0) 287
4.12.10 PRIME NB OFDM PLC MAC address setup (class_id = 43, version = 0) 290
4.12.11 PRIME NB OFDM PLC Application identification (class_id = 86, version = 0) 290
4.13 Interface classes for setting up and managing the DLMS/COSEM narrowband OFDM PLC profile for G3-PLC networks 291
4.13.1 Overview 291
4.13.2 Mapping of G3-PLC IB attributes to COSEM IC attributes 291
4.13.3 G3-PLC MAC layer counters (class_id = 90, version = 1) 293
4.13.4 G3-PLC MAC setup (class_id = 91, version = 2) 294
4.13.5 G3-PLC 6LoWPAN adaptation layer setup (class_id = 92, version = 2) 307
4.14 Interface classes for setting up and managing DLMS/COSEM HS-PLC ISO/IEC 12139-1 neighbourhood networks 313
4.14.1 Overview 313
4.14.2 HS-PLC ISO/IEC 12139-1 MAC setup (class_id = 140, version = 0) 313
4.14.3 HS-PLC ISO/IEC 12139-1 CPAS setup (class_id = 141, version = 0) 314
4.14.4 HS-PLC ISO/IEC 12139-1 IP SSAS setup (class_id = 142, version = 0) 315
4.14.5 HS-PLC ISO/IEC 12139-1 HDLC SSAS setup (class_id = 143, version = 0) 315
4.15 ZigBee® setup classes 317
4.15.1 Overview 317

© Copyright 1997-2020 DLMS User Association

4.15.2 ZigBee® SAS startup (class_id = 101, version = 0) 318
4.15.3 ZigBee® SAS join (class_id = 102, version = 0) 320
4.15.4 ZigBee® SAS APS fragmentation (class_id = 103, version = 0) 321
4.15.5 ZigBee® network control (class_id = 104, version = 0) 322
4.15.6 ZigBee® tunnel setup (class_id = 105, version = 0) 328
4.16 Interface classes for setting up and managing the DLMS/COSEM profile for LPWAN networks 330
4.16.1 General 330
4.16.2 Generic interface classes 330
4.16.3 LPWAN specific interface classes 336
4.17 Interface classes for setting up and managing the DLMS/COSEM profile for Wi-SUN networks 342
4.17.1 Wi-SUN setup (class_id = 95, version 0) 342
4.17.2 Wi-SUN diagnostic (class_id = 96, version 0) 346
4.17.3 RPL diagnostic (class_id = 97, version 0) 349
4.17.4 MPL diagnostic (class_id = 98, version 0) 351
4.18 Interface classes for setting up and managing the DLMS/COSEM profile for ISO/IEC 14908 PLC networks 353
4.18.1 General 353
4.18.2 ISO/IEC 14908 identification (class_id = 130, version = 0) 353
4.18.3 ISO/IEC 14908 protocol setup (class_id = 131, version = 0) 354
4.18.4 ISO/IEC 14908 protocol status (class_id = 132, version = 0) 354
4.18.5 ISO/IEC 14908 diagnostic (class_id = 133, version = 0) 356
5 Maintenance of the interface classes 359
5.1 General 359
5.1.1 New versions of interface classes 359
5.1.2 New interface classes 359
5.1.3 Removal of interface classes 359
5.2 Previous versions of interface classes – general 359
5.3 Previous versions of interface classes for parameters and measurement data 359
5.3.1 Profile generic (class_id = 7, version = 0) 359
5.3.2 Compact data (class_id = 62, version = 0) 363
5.4 Previous versions of interface classes for access control and management 366
5.4.1 Association SN (class_id = 12, version = 0) 366
5.4.2 Association SN (class_id = 12, version = 1) 368
5.4.3 Association SN (class_id = 12, version = 2) 370
5.4.4 Association SN (class_id = 12, version = 3) 373
5.4.5 Association LN (class_id = 15, version = 0) 377
5.4.6 Association LN (class_id = 15, version = 1) 381
5.4.7 Association LN (class_id = 15, version = 2) 387
5.4.8 Security setup (class_id = 64, version = 0) 392
5.4.9 Push Setup (class_id = 40, version = 0) 394
5.4.10 Push Setup (class_id = 40, version = 1) 399
5.5 Previous versions of interface classes for time- and event- bound control 407
5.5.1 Parameter monitor (class_id = 65, version = 0) 407
5.6 Previous versions of payment metering related interface classes 408

© Copyright 1997-2020 DLMS User Association

5.7 Previous versions of interface classes for setting up data exchange via local ports and modems 408
5.7.1 IEC local port setup (class_id = 19, version = 0) 408
5.7.2 IEC HDLC setup, (class_id = 23, version = 0) 410
5.7.3 IEC twisted pair (1) setup (class_id = 24, version = 0) 411
5.7.4 PSTN modem configuration (class_id = 27, version = 0) 413
5.7.5 Auto answer (class_id = 28, version = 0) 415
5.7.6 PSTN auto dial (class_id = 29, version = 0) 416
5.7.7 Auto connect (class_id = 29, version = 1) 417
5.7.8 GSM diagnostic (class_id = 47, version = 0) 419
5.7.9 GSM diagnostic (class_id = 47, version = 1) 421
5.7.10 LTE Monitoring (class_id = 155, version = 0) 424
5.8 Previous versions of interface classes for setting up data exchange via M-Bus 425
5.8.1 M-Bus client (class_id = 72, version = 0) 425
5.9 Previous versions of interface classes for setting up data exchange over the internet 430
5.10 Previous versions of interface classes for data exchange using S-FSK PLC 430
5.10.1 S-FSK Phy&MAC setup (class_id = 50, version = 0) 430
5.10.2 S-FSK IEC 61334-4-32 LLC setup (class_id = 55, version = 0) 433
5.11 Previous versions of interface classes for setting up the LLC layer for ISO/IEC 8802-2 434
5.12 Previous versions of interface classes for setting up and managing DLMS/COSEM narrowband OFDM PLC profile for PRIME networks 434
5.13 Previous versions of interface classes for setting up and managing DLMS/COSEM narrowband OFDM PLC profile for G3-PLC networks 434
5.13.1 Mapping of G3-PLC IB attributes to COSEM IC attributes (Original version) 434
5.13.2 G3 NB OFDM PLC MAC layer counters (class_id = 90, version = 0) 436
5.13.3 G3 NB OFDM PLC MAC setup (class_id = 91, version = 0) 437
5.13.4 G3 NB OFDM PLC 6LoWPAN adaptation layer setup (class_id = 92, version = 0) 440
5.13.5 Mapping of G3-PLC PIB attributes to COSEM IC attributes (previous version) 445
5.13.6 G3-PLC MAC setup (class_id = 91, version = 1) 447
5.13.7 G3-PLC 6LoWPAN adaptation layer setup (class_id = 92, version = 1) 453
5.14 Previous versions of interface classes for setting up and managing DLMS/COSEM HS-PLC ISO/IEC 12139-1 neighbourhood networks 459
5.15 Previous versions of ZigBee® setup classes 459
6 Relation to OBIS 460
6.1 General 460
6.2 Abstract COSEM objects 460
6.2.1 Use of value group C 460
6.2.2 Data of historical billing periods 462
6.2.3 Billing period values / reset counter entries 463
6.2.4 Other abstract general purpose OBIS codes 463
6.2.5 Clock objects (class_id = 1, 8) 464
6.2.6 Modem configuration and related objects 464

© Copyright 1997-2020 DLMS User Association

6.2.7 Script table objects (class_id = 9) 465
6.2.8 Special days table objects (class_id = 11) 466
6.2.9 Schedule objects (class_id = 10) 466
6.2.10 Activity calendar objects (class_id = 20) 466
6.2.11 Register activation objects (class_id = 6) 467
6.2.12 Single action schedule objects (class_id = 22) 467
6.2.13 Register monitor and alarm monitor objects (class_id = 21) 467
6.2.14 Parameter monitor objects (class_id = 65) 468
6.2.15 Limiter objects (class_id = 71) 468
6.2.16 Array manager objects (class_id = 123) 468
6.2.17 Payment metering related objects 468
6.2.18 IEC local port setup objects (class_id = 19) 469
6.2.19 Standard readout profile objects (class_id = 7) 469
6.2.20 IEC HDLC setup objects (class_id = 23) 470
6.2.21 IEC twisted pair (1) setup objects (class_id = 24 etc.) 470
6.2.22 Objects related to data exchange over M-Bus 470
6.2.23 Objects to set up data exchange over the Internet 471
6.2.24 Objects to set up Push Setup (class_id = 40) 473
6.2.25 Objects for setting up data exchange using S-FSK PLC 473
6.2.26 Objects for setting up the ISO/IEC 8802-2 LLC layer 474
6.2.27 Objects for data exchange using narrowband OFDM PLC for PRIME networks 474
6.2.28 Objects for data exchange using narrow-band OFDM PLC for G3-PLC networks 475
6.2.29 ZigBee® setup objects 476
6.2.30 Objects for setting up and managing data exchange using ISO/IEC 14908 PLC networks 476
6.2.31 Objects for data exchange using HS-PLC ISO/IEC 12139-1 ISO/EC 12139-1 networks 476
6.2.32 Objects for data exchange using Wi-SUN networks 477
6.2.33 Association objects (class_id = 12, 15) 477
6.2.34 SAP assignment object (class_id = 17) 477
6.2.35 COSEM logical device name object 478
6.2.36 Information security related objects (class_id = 64 etc.) 478
6.2.37 Image transfer objects (class_id = 18) 479
6.2.38 Function control objects (class_id = 122) 479
6.2.39 Communication port protection objects (class_id = 124) 479
6.2.40 Utility table objects (class_id = 26) 479
6.2.41 Compact data objects (class_id = 62) 480
6.2.42 Device ID objects 480
6.2.43 Metering point ID objects 480
6.2.44 Parameter changes and calibration objects 481
6.2.45 I/O control signal objects 481
6.2.46 Disconnect control objects (class_id = 70) 481
6.2.47 Arbitrator objects (class_id = 68) 481
6.2.48 Status of internal control signals objects 482
6.2.49 Internal operating status objects 482

© Copyright 1997-2020 DLMS User Association

6.2.50 Battery entries objects 482
6.2.51 Power failure monitoring objects 483
6.2.52 Operating time objects 483
6.2.53 Environment related parameters objects 483
6.2.54 Status register objects 484
6.2.55 Event code objects 484
6.2.56 Communication port log parameter objects 484
6.2.57 Consumer message objects 484
6.2.58 Currently active tariff objects 485
6.2.59 Event counter objects 485
6.2.60 Profile entry digital signature objects 485
6.2.61 Profile entry counter objects 485
6.2.62 Meter tamper event related objects 486
6.2.63 Error register objects 486
6.2.64 Alarm register, Alarm filter and Alarm descriptor objects 487
6.2.65 General list objects 487
6.2.66 Event log objects (class_id = 7) 488
6.2.67 Inactive objects 488
6.3 Electricity related COSEM objects 489
6.3.1 Value group D definitions 489
6.3.2 ID numbers – Electricity 489
6.3.3 Billing period values / reset counter entries 489
6.3.4 Other electricity related general purpose objects 490
6.3.5 Measurement algorithm 491
6.3.6 Metering point ID (electricity related) 492
6.3.7 Electricity related status objects 493
6.3.8 List objects – Electricity (class_id = 7) 493
6.3.9 Threshold values 494
6.3.10 Register monitor objects (class_id = 21) 494
6.4 HCA related COSEM objects 496
6.4.1 General 496
6.4.2 ID numbers – HCA 496
6.4.3 Billing period values / reset counter entries - HCA 496
6.4.4 General purpose objects – HCA 497
6.4.5 Measured Values – HCA 497
6.4.6 Error register objects – HCA 499
6.4.7 List objects – HCA 499
6.4.8 Data profile objects – HCA 499
6.5 Thermal energy meter related COSEM objects 500
6.5.1 General 500
6.5.2 ID numbers – Thermal energy meter 500
6.5.3 Billing period values / reset counter entries - Thermal energy meter 500
6.5.4 General purpose objects – Thermal energy meter 501
6.5.5 Measured values - Thermal energy meter 501
6.5.6 Error register objects – Thermal energy meter 503
6.5.7 List objects – Thermal energy meter 504

© Copyright 1997-2020 DLMS User Association

6.5.8 Data profile objects – Thermal energy meter 504
6.6 Gas related COSEM objects 505
6.6.1 General 505
6.6.2 ID numbers – Gas 505
6.6.3 Billing period values / reset counter entries – Gas 505
6.6.4 Other general purpose objects – Gas 505
6.6.5 Internal operating status objects – Gas 507
6.6.6 Measured values – Gas 508
6.6.7 Conversion related factors and coefficients – Gas 509
6.6.8 Calculation methods – Gas 510
6.6.9 Natural gas analysis 510
6.6.10 List objects – Gas 511
6.7 Water meter related COSEM objects 512
6.7.1 General 512
6.7.2 ID numbers – water meter 512
6.7.3 Billing period values / reset counter entries – water meter 512
6.7.4 General purpose objects – water meter 513
6.7.5 Measured values – water meter 513
6.7.6 Error register objects – water meter 515
6.7.7 List objects – water meter 515
6.7.8 Data profile objects – water meter 515
6.7.9 Coding of OBIS identifications 516
7 COSEM Object Identification System (OBIS) 517
7.1 Scope 517
7.2 OBIS code structure 517
7.2.1 Value groups and their use 517
7.2.2 Manufacturer specific codes 517
7.2.3 Reserved ranges 518
7.2.4 Summary of rules for manufacturer, utility, consortia and country specific codes 518
7.2.5 Standard object codes 519
7.3 Value group definitions – overview 519
7.3.1 Value group A 519
7.3.2 Value group B 519
7.3.3 Value group C 520
7.3.4 Value group D 521
7.3.5 Value group E 523
7.3.6 Value group F 523
7.4 Abstract objects (Value group A = 0) 524
7.4.1 General and service entry objects – Abstract 524
7.4.2 Error registers, alarm registers / filters / descriptor objects – Abstract 528
7.4.3 List objects – Abstract 529
7.4.4 Register table objects – Abstract 529
7.4.5 Data profile objects – Abstract 529
7.5 Electricity (Value group A = 1) 530
7.5.1 Value group C codes – Electricity 530

© Copyright 1997-2020 DLMS User Association

7.5.2 Value group D codes – Electricity 532
7.5.3 Value group E codes – Electricity 535
7.5.4 Value group F codes – Electricity 540
7.5.5 OBIS codes – Electricity 541
7.6 Heat Cost Allocators (Value group A = 4) 547
7.6.1 General 547
7.6.2 Value group C codes – HCA 547
7.6.3 Value group D codes – HCA 548
7.6.4 Value group E codes – HCA 548
7.6.5 OBIS codes – HCA 549
7.7 Thermal energy (Value group A = 5 or A = 6) 552
7.7.1 General 552
7.7.2 Value group C codes – Thermal energy 552
7.7.3 Value group D codes – Thermal energy 553
7.7.4 Value group E codes – Thermal energy 554
7.7.5 OBIS codes – Thermal energy 554
7.8 Gas (Value group A = 7) 559
7.8.1 General introduction to gas measurement 559
7.8.2 Value group C codes – Gas 566
7.8.3 Value group D codes – Gas 568
7.8.4 Value group E codes – Gas 580
7.8.5 Value group F codes – Gas 583
7.8.6 OBIS codes – Gas 583
7.9 Water (Value group A = 8 and A = 9) 591
7.9.1 General 591
7.9.2 Value group C codes – Water 591
7.9.3 Value group D codes – Water 592
7.9.4 Value group E codes – Water 592
7.9.5 OBIS codes – Water 593
7.10 Other media (Value group A= 15) 595
7.10.1 General 595
7.10.2 Value group C codes – Other media 595
7.10.3 Value group E codes – Other media 595
7.10.4 Value group D codes – Other media 595
7.10.5 Value group F codes – Other media 595
7.11 Code presentation 595
7.11.1 Reduced ID codes (e.g. for IEC 62056-21) 595
7.11.2 Display 596
7.11.3 Special handling of value group F 596
7.11.4 COSEM 597
Annex A (Informative) Additional information on Auto answer and Auto connect ICs 598
Annex B (Informative) Additional information to M-Bus client (class_id = 72, version 1) 600
Annex C (Informative) Additional information on IPv6 setup (class_id = 48, version = 0) 602
C.1 General 602
C.2 IPv6 addressing 602
C.3 IPv6 header format 603

© Copyright 1997-2020 DLMS User Association

C.4 IPv6 header extensions 604
C.4.1 Overview 604
C.4.2 Hop-by-Hop options 605
C.4.3 Destination options 605
C.4.4 Routing options 605
C.4.5 Fragment options 606
C.4.6 Security options 606
Annex D (Informative) Overview of the narrow-band OFDM PLC technology for PRIME networks 607
Annex E (informative) Overview of the narrow-band OFDM PLC technology for G3-PLC networks 608
Annex F (informative) Bibliography 609
Index 611

Figure 1 – The three steps approach of DLMS/COSEM: Modelling – Messaging – Transporting 20
Figure 2 – The meaning of the definitions concerning the Image 25
Figure 3 – An interface class and its instances 39
Figure 4 – The COSEM server model 48
Figure 5 – Combined metering device 49
Figure 6 – Overview of the interface classes – Part 1 51
Figure 7 – Overview of the interface classes – Part 2 52
Figure 8 – Overview of the interface classes – Part 3 53
Figure 9 – The time attributes when measuring sliding demand 63
Figure 10 – The attributes in the case of block demand 63
Figure 11 – The attributes in the case of sliding demand (number of periods = 3) 64
Figure 12 – Image transfer process flow chart 104
Figure 13 – COSEM model of push operation 113
Figure 14 – Push windows, delays and retries 115
Figure 15 – COSEM model of data protection 125
Figure 16 – Example: Read *protection_buffer* attribute 127
Figure 17 – Example of managing an array 143
Figure 18 – The generalized time concept 151
Figure 19 – State diagram of the Disconnect control IC 164
Figure 20 – Definition of upper and lower thresholds 176
Figure 21 – COSEM tariffication model (example) 181
Figure 22 – COSEM billing model (example) 182
Figure 23 – Outline Account model 184
Figure 24 – Diagram of attribute relationships 185
Figure 25 – Credit States when priority >0 195
Figure 26 – Operation of current_credit_status flags 198
Figure 27 – Interaction of *current_credit_amount* and *available_credit* with Token “Credit” and Emergency “Credit” 204

© Copyright 1997-2020 DLMS User Association

Figure 28 – Object model of DLMS/COSEM servers 259
Figure 29 – Object model of DLMS/COSEM servers 279
Figure 30 – Example of a ZigBee® network 318
Figure 31 – Push windows and delays 394
Figure 32 – Push windows and delays 399
Figure 33 – Data of historical billing periods – example with modulo 12, VZ = 5 462
Figure 34 – Quadrant definitions for active and reactive power 532
Figure 35 – Model of the line and the transformer for calculation of loss quantities 537
Figure 36 – Residential gas metering installation 559
Figure 37 – Industrial gas metering installation (single stream) 560
Figure 38 – City gate or border crossing installation (multi stream) 561
Figure 39 – Data flow of volume conversion and energy calculation 563
Figure 40 – Reduced ID code presentation 596
Figure A. 1 – Network connectivity example for a GSM/GPRS network 598
Figure B. 1 – Encryption key status diagram 600
Figure C. 1 – IPv6 address formats 602
Figure C. 2 – IPv6 header format 603
Figure C. 3 – Traffic class parameter format 604

Table 1 – Reserved base_names for SN referencing 40
Table 2 – Common data types 43
Table 3 – List of interface classes by class_id 53
Table 4 – Enumerated values for physical units 58
Table 5 – Examples for scaler_unit 62
Table 6 – Encoding of selective access parameters with data_index 81
Table 7 – Example daily billing data captured to *compact_buffer* 82
Table 8 – “Compact data“ object attributes – Daily billing data example 83
Table 10 – Example diagnostic and alarm data captured to *compact_buffer* 84
Table 11 – “Compact data“ object attributes – Diagnostic and Alarm data example 84
Table 12 – Example diagnostic and alarm data read from “Profile generic” *buffer* 84
Table 13 – Example logbook data entries in “Profile generic” *buffer* 85
Table 14 – Example logbook data captured to *compact_buffer* 85
Table 15 – “Compact data“ object attributes – Logbook data example 85
Table 16 – Example logbook data read from “Profile generic” *buffer* 86
Table 17 – Key information required to establish data protection keys 135
Table 18 – Protection parameters of *protection_parameters_get* attribute 136
Table 19 – Protection parameters of *protection_parameters_set* attribute 137
Table 20 – Protection parameters of *get_protected_attributes* method 138
Table 21 – Protection parameters of *set_protected_attributes* method 139
Table 22 – Protection parameters of *invoke_protected_method* method 140
Table 23 – Schedule 155

© Copyright 1997-2020 DLMS User Association

Table 24 – Special days table 155
Table 25 – Disconnect control IC – states and state transitions 165
Table 26 – Explicit presentation of threshold value arrays 176
Table 27 – Explicit presentation of action_sets 176
Table 28 – Credit states 195
Table 29 – Credit state transitions 196
Table 30 – Fatal error register 218
Table 31 – Mapping IEC 61334-4-512:2001 MIB variables to COSEM IC attributes / methods 260
Table 32 – MAC addresses in the S-FSK profile 265
Table 33 – Mapping of PRIME NB OFDM PLC PIB attributes to COSEM IC attributes 280
Table 34 – Mapping of G3-PLC IB attributes specified in ITU-T G.9903:2017 to COSEM IC attributes 291
Table 35 – Use of ZigBee® setup COSEM interface classes 318
Table 36 – C/D Rule 1 334
Table 37 – Encoding of selective access parameters with data_index 398
Table 38 – ADS address elements 413
Table 39 – Mapping of G3-PLC IB attributes specified in ITU-T G.9903:2013 Amd. 1 to COSEM IC attributes 434
Table 40 – Mapping of G3-PLC IB attributes to COSEM IC attributes 445
Table 41 – Use of value group C for abstract objects in the COSEM context 461
Table 42 – Representation of various values by appropriate ICs 489
Table 43 – Measuring algorithms – enumerated values 491
Table 44 – Threshold objects, electricity 494
Table 45 – Register monitor objects, electricity 495
Table 46 – Digital / Analogue output configurations – enumerated values 506
Table 47 – Indexes and index differences 508
Table 48 – Flow rate 509
Table 49 – Process values 509
Table 50 – Conversion related factors and coefficients 510
Table 51 – Calculation methods 510
Table 52 – Natural gas analysis 511
Table 53 – OBIS code structure and use of value groups 517
Table 54 – Rules for manufacturer, utility, consortia and country specific codes 518
Table 55 – Value group A codes 519
Table 56 – Value group B codes 520
Table 57 – Value group C codes – Abstract objects 520
Table 58 – Value group D codes – Consortia specific identifiers 521
Table 59 – Value group D codes – Country specific identifiers 521
Table 60 – OBIS codes for general and service entry objects 524
Table 61 – OBIS codes for error registers, alarm registers and alarm filters – Abstract 528

© Copyright 1997-2020 DLMS User Association

Table 62 – OBIS codes for list objects – Abstract 529
Table 63 – OBIS codes for Register table objects – Abstract 529
Table 64 – OBIS codes for data profile objects – Abstract 529
Table 65 – Value group C codes – Electricity 530
Table 66 – Value group D codes – Electricity 532
Table 67 – Value group E codes – Electricity – Tariff rates 535
Table 68 – Value group E codes – Electricity – Harmonics 535
Table 69 – Value group E codes – Electricity – Extended phase angle measurement 536
Table 70 – Value group E codes – Electricity – Transformer and line losses 537
Table 71 – Value group E codes – Electricity – UNIPEDE voltage dips 540
Table 72 – OBIS codes for general and service entry objects – Electricity 541
Table 73 – OBIS codes for error register objects – Electricity 545
Table 74 – OBIS codes for list objects – Electricity 545
Table 75 – OBIS codes for data profile objects – Electricity 545
Table 76 – OBIS codes for Register table objects – Electricity 546
Table 77 – Value group C codes – HCA 547
Table 78 – Value group D codes – HCA 548
Table 79 – Value group E codes – HCA 548
Table 80 – OBIS codes for general and service entry objects – HCA 549
Table 81 – OBIS codes for error register objects – HCA 550
Table 82 – OBIS codes for list objects – HCA 550
Table 83 – OBIS codes for data profile objects – HCA 550
Table 84 – OBIS codes for HCA related objects (examples) 551
Table 85 – Value group C codes – Thermal energy 552
Table 86 – Value group D codes – Thermal energy 553
Table 87 – Value group E codes – Thermal Energy – Tariff rates 554
Table 88 – OBIS codes for general and service entry objects – Thermal energy 554
Table 89 – OBIS codes for error register objects – Thermal energy 556
Table 90 – OBIS codes for list objects – Thermal Energy Meters 557
Table 91 – OBIS codes for data profile objects – Thermal energy 557
Table 92 – OBIS codes for Thermal energy related objects (examples) 557
Table 93 – OBIS codes of the main objects in the gas conversion process data flow 564
Table 94 – Value group C codes – Gas 566
Table 95 – Value group D codes – Gas – Indexes and index differences 569
Table 96 – Value group D codes – Gas – Flow rate 573
Table 97 – Value group D codes – Gas – Process values 575
Table 98 – Value group D codes – Gas – Conversion related factors and coefficients 578
Table 99 – Value group D codes – Gas – Natural gas analysis values 579
Table 100 – Value group E codes – Gas – Indexes and index differences – Tariff rates 580
Table 101 – Value group E codes – Gas – Conversion related factors and coefficients 581
Table 102 – Value group E codes – Gas – Calculation methods 582

© Copyright 1997-2020 DLMS User Association

Table 103 – Value group E codes – Gas – Natural gas analysis values – Averages 582
Table 104 – OBIS codes for general and service entry objects – Gas 583
Table 105 – OBIS codes for error register objects – Gas 590
Table 106 – OBIS codes for list objects – Gas 590
Table 107 – OBIS codes for data profile objects – Gas 591
Table 108 – Value group C codes – Water 591
Table 109 – Value group D codes – Water 592
Table 110 – Value group E codes – Water 592
Table 111 – OBIS codes for general and service entry objects – Water 593
Table 112 – OBIS codes for error register objects – Water 594
Table 113 – OBIS codes for list objects – Water Meters 594
Table 114 – OBIS codes for data profile objects – Water 594
Table 115 – OBIS codes for water related objects (examples) 594
Table 116 – Value group C codes – Other media 595
Table 117 – Example of display code replacement 596
Table 118 – Value group F – Billing periods 597
Table B. 1 – Encryption key is preset in the slave and cannot be changed 601
Table B. 2 – Encryption key is preset in the slave and new key is set after installation 601
Table B. 3 – Encryption key is not preset in the slave, but can be set, case a) 601
Table B. 4 – Encryption key is not preset in the slave, but can be set, case b) 601
Table C. 1 – IPv6 header vs. IPv6 IC 604
Table C. 2 – Optional IPv6 header extensions vs. IPv6 IC 605

© Copyright 1997-2020 DLMS User Association

Foreword

Copyright

© Copyright 1997-2020 DLMS User Association

This document is confidential. It may not be copied, nor handed over to persons outside the standardization environment.

The copyright is enforced by national and international law. The "Berne Convention for the Protection of Literary and Artistic Works", which is signed by 176 countries world-wide, and other treaties apply.

Liability

DLMS User Association Publications have the form of recommendations for international use. While all reasonable efforts are made to ensure that the technical content of DLMS User Association Publications is accurate, the DLMS User Association cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.

No liability shall attach to DLMS User Association or its directors, employees, servants or agents including individual experts and members of its technical committees for any personal injury, property damage or other damage of any nature whatsoever, whether direct or indirect, or for costs (including legal fees) and expenses arising out of the publication, use of, or reliance upon, this DLMS User Association Publication or any other DLMS User Association Publications.

Intellectual Property Rights

The DLMS User Association (DLMS UA) draws attention to the fact that it is claimed that compliance with this document may involve the use of a patent concerning the Image transfer procedure.

The DLMS UA takes no position concerning the evidence, validity and scope of this patent right.

The holder of this patent right has assured the DLMS UA that he/she is willing to negotiate licenses either free of charge or under reasonable and non-discriminatory terms and conditions with applicants throughout the world. In this respect, the statement of the holder of this patent right is registered with the DLMS UA. Information may be obtained from Itron, Inc., Liberty Lake, Washington, USA.

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights other than those identified above. The DLMS UA shall not be held responsible for identifying any or all such patent rights.

The DLMS UA maintains on-line databases of patents relevant to their standards. Users are encouraged to consult the databases for the most up to date information concerning patents.

Acknowledgement

The actual document has been established by the WG Maintenance of the DLMS UA.

Clauses 4.4.7 and 4.4.9 are based on parts of NIST documents. Reprinted courtesy of the National Institute of Standards and Technology, Technology Administration, U.S. Department of Commerce. Not copyrightable in the United States.

© Copyright 1997-2020 DLMS User Association

Status of standardisation

The contents of this edition is the basis of future revisions to:

- IEC 62056-6-1, Electricity Metering Data Exchange – The DLMS/COSEM suite – Part 6-1: Object identification system (OBIS); and
- IEC 62056-6-2, Electricity Metering Data Exchange – The DLMS/COSEM suite – Part 6-2: COSEM interface classes.

Revision history

Version	Date	Author	Comment
Release 1	01 April 1998	DLMS UA	Initial version
First Edition	12 Nov. 1998	DLMS UA	Considering comments received after R1
Second Edition	03 May 1999	DLMS UA	Major rework, classes and associations added
Third Edition	29 Feb. 2000	DLMS UA	Major rework, adapted to CDVs of IEC TC13, OBIS added
Fourth Edition	25 March 2001	DLMS UA	Considering comments to CDVs by IEC National Committees
Fifth Edition	05 March 2002	DLMS UA	Content adapted to IEC International Standards
Sixth Edition	16th August 2004	DLMS UA	Content adapted to draft IEC 62056-61 Edition 2, draft IEC 62056-62 (CD versions) and EN 13757-1:2002
Seventh Edition	12th September 2005	DLMS UA	Content adapted to 13/1341/CDV, draft IEC 62056-61 Edition 2, 13/1342/CDV, draft IEC 62056-62 Edition 2 and comments received on these drafts, as well as on EN 13757-1:2002
Eighth Edition	14th September 2007	DLMS UA	Document restructured, editorial errors in Ed. 7.0 corrected.
Ninth Edition	9th February 2009	DLMS UA	New elements for smart metering and advanced gas volume and energy conversion added.
Tenth Edition	26th August 2010	DLMS UA	Brought in line with Green Book. New interface classes and new versions of interface classes added. New OBIS codes added.
Eleventh Edition	26th August 2013	DLMS UA	New interface classes and new versions of existing interface classes added. New OBIS codes added.
Twelfth Edition	10th September 2014	DLMS UA	New interface classes and new versions of existing interface classes added in support of Security management, Payment Metering, Arbitrator, Compact data, Data protection, Push setup, G3-PLC, M-Bus,New OBIS codes added.
Twelfth Edition Corrigendum 1	21st December 2015	DLMS UA	Technical and editorial corrections
Edition 12.1	21st December 2015	DLMS UA	Consolidated edition including the Corrigendum 1.
Edition 12.2	21st December 2016	DLMS UA	New or updated interface classes for Function Control, Array Manager, NTP setup, Compact data,GSM-related and HS-PLC networks added. New OBIS codes added
Edition 13	8th May 2019	DLMS UA	See below
Edition 14	31st Aug 2020	DLMS UA	See below

© Copyright 1997-2020 DLMS User Association

List of main technical changes in Edition 13

Item	Clause	Description
1.	4.1.8.4	Addition of firmware – related objects to the mandatory list of objects to ensure that product certifications by the DLMS UA have improved traceability.
2.	4.3.2	Addition of non-SI units, footnote for converted volume, formula to describe Force (Newtons) and alternate pressure units (g/cm^2, atm) to Table 4.
3.	4.4.12	Addition of Communication Port Protection IC (class_id = 124, version = 0).
4.	4.4.8	Push Setup update to version 1.
5.	4.13	Update to the G3-PLC setup interface classes: including the G3-PLC MAC setup (class_id = 91) and the G3-PLC 6LoWPAN adaptation layer setup (class_id = 92) to be in line with IUT-T 9903:2017.
6.	6	Generic clause added to permit Data IC (class_id = 1) to be replaced with Register IC (class_id = 3) or Extended register IC (class_id = 4) in many applications, and related notes removed.
7.	6.4	Heat Cost Allocator relation to OBIS.
8.	6.5	Thermal energy meter relation to OBIS.
9.	6.7	Water meter relation to OBIS.
10.	7.3.4.3	Addition of Mexico.
11.	7.5.1	Additional value group C codes for reactive power inductive and capacitative, and line-to-line voltage.
12.	7.5.2.1	Additional value group D codes for average values for recording intervals 1 & 2.
13.	7.6.4	Heat Cost Allocator OBIS codes updates.
14.	7.7.4	Thermal energy meter OBIS codes updates.
15.	7.9.4	Water meter OBIS codes updates..

© Copyright 1997-2020 DLMS User Association

List of main technical changes in Edition 14

Item	Clause	Description
1.	3.8	Abbreviation list updated
2.	4.4.4	Note removed from the description of secret attribute. (Also removed from 5.4.6, 5.4.7)
3.	4.4.8.2	Push IC – new version 2
4.	4.4.10	Parameter monitor V1
5.	4.7.8	GSM Diagnostic now V2, V1 moved to section 5
6.	4.7.9	LTE Monitoring now V1, V0 moved to section 5
7.	4.16	LPWAN interface classes added
8.	4.17	Wi-SUN interface classes added
9.	4.18	New section for ISO/IEC 14908 PLC networks
10.	5.4.10	Push IC version 1 moved to here
11.	5.5.1	Parameter monitor V0
12.	6.1.6.3	New section describing delta data types
13.	6.2.5	New clock objects added
14.	6.2.30	New section for ISO/IEC 14908 networks
15.	6.2.61	New section for profile entry counter objects + update to table 56
16.	6.2.32	Wi-SUN objects added
17.	Table 2	Delta data types added
18.	Table 60	Updated to show VZ values
19.	Table 72	Updated to show VZ values

© Copyright 1997-2020 DLMS User Association

Introduction

Object modelling and data identification

Driven by the business needs of the energy market participants – generally in a liberalized, competitive environment – and by the desire to manage natural resources efficiently and to involve the consumers, the utility meter became part of an integrated metering, control and billing system. The meter is not any more a simple data recording device but it relies critically on communication capabilities. Ease of system integration, interoperability and data security are important requirements.

COSEM, the *Companion Specification for Energy Metering*, addresses these challenges by looking at the utility meter as part of a complex measurement and control system. The meter has to be able to convey measurement results from the metering points to the business processes which use them. It also has to be able to provide information to the consumer and manage consumption and eventually local generation.

COSEM achieves this by using *object modelling* techniques to model all functions of the meter, without making any assumptions about which functions need to be supported, how those functions are implemented and how the data are transported. The formal specification of COSEM interface classes forms a major part of COSEM.

To process and manage the information it is necessary to uniquely identify all data items in a manufacturer-independent way. The definition of OBIS, the *Object Identification System* is another essential part of COSEM. It is based on DIN 43863-3:1997, *Electricity meters – Part 3: Tariff metering device as additional equipment for electricity meters – EDIS – Energy Data Identification System.* The set of OBIS codes has been considerably extended over the years to meet new needs.

COSEM models the utility meter as a *server* application – see 4.1.7 – used by *client* applications that retrieve data from, provide control information to, and instigate known actions within the meter via controlled access to the COSEM objects. The *clients* act as agents for third parties i.e. the business processes of energy market participants.

The standardized COSEM interface classes form an extensible library. Manufacturers use elements of this library to design their products that meet a wide variety of requirements.

The server offers means to retrieve the functions supported, i.e. the COSEM objects instantiated. The objects can be organized to *logical devices and application associations* and to provide specific access rights to various clients.

The concept of the standardized interface class library provides different users and manufacturers with a maximum of diversity while ensuring interoperability.

© Copyright 1997-2020 DLMS User Association

1 Scope

The DLMS/COSEM specification specifies a data model and communication protocols for data exchange with metering equipment. It follows a three-step approach as illustrated in Figure 1.

Step 1, Modelling: This covers the data model of metering equipment as well as rules for data identification. The data model provides a view of the functionality of the meter, as it is available at its interface(s). It uses generic building blocks to model this functionality. The model does not cover internal, implementation-specific issues.

Step 2, Messaging: This covers the communication services and protocols for mapping the elements of the data model to application protocol data units (APDU).

Step 3, Transporting: This covers the services and protocols for the transportation of the messages through the communication channel.

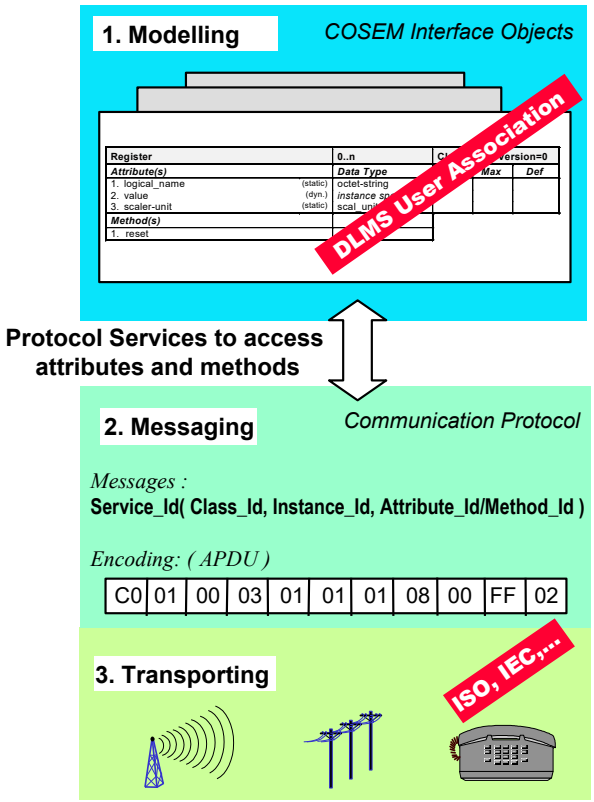


Figure 1 – The three steps approach of DLMS/COSEM: Modelling – Messaging – Transporting

Step 1 is specified in this document. It specifies the COSEM interface classes (ICs), the OBIS object identification system, and the use of interface objects for modelling the various functions of the metering equipment.

Step 2 and 3 are specified in the Green Book, DLMS UA 1000-2 It specifies communication profiles for various communication media and the protocol layers of these communication profiles. The top layer in any profile is the DLMS/COSEM application layer. It provides services to establish a logical connection between the client and the server(s). It also provides the xDLMS messaging services to access attributes and methods of the COSEM interface objects. The lower, communication profile specific protocol layers transport the information.

Rules for conformance testing are specified in the “Yellow Book”, DLMS UA 1001-1 "DLMS/COSEM Conformance Test Process".

Terms are explained in the “White book” DLMS UA 1002, "COSEM Glossary of Terms".

© Copyright 1997-2020 DLMS User Association

2 Referenced documents

Reference	Title
DLMS UA 1000-2 Ed. 10	*DLMS/COSEM Architecture and Protocols, the "Green Book" Edition 10*
DLMS UA 1001-1	*DLMS/COSEM Conformance test and certification process, the "Yellow Book"*
DLMS UA 1002	*COSEM Glossary of terms, the "White Book"*
IEC TR 61000-2-8:2002	*Electromagnetic compatibility (EMC) – Part 2-8: Environment - Voltage dips and short interruptions on public electric power supply systems with statistical measurement results*
IEC 61334-4-32:1996	*Distribution automation using distribution line carrier systems – Part 4: Data communication protocols – Section 32: Data link layer – Logical link control (LLC)*
IEC 61334-4-41:1996	*Distribution automation using distribution line carrier systems – Part 4: Data communication protocols – Section 41: Application protocols – Distribution line message specification*
IEC 61334-4-511:2000	*Distribution automation using distribution line carrier systems – Part 4-511: Data communication protocols – Systems management – CIASE protocol*
IEC 61334-4-512:2001	*Distribution automation using distribution line carrier systems – Part 4-512: Data communication protocols – System management using profile 61334-5-1 – Management Information Base (MIB)*
IEC 61334-5-1:2001	*Distribution automation using distribution line carrier systems – Part 5-1: Lower layer profiles – The spread frequency shift keying (S-FSK) profile*
IEC 62053-23:2003	Electricity metering equipment (a.c.) – Particular requirements – Part 23: Static meters for reactive energy (classes 2 and 3)
IEC TR 62055-21:2005	*Technical report Electricity metering - Payment systems- Part 21: Framework for standardization. 2005-8*
IEC 62056-21:2002	*Electricity metering – Data exchange for meter reading, tariff and load control – Part 21: Direct local data exchange*
IEC 62056-31:1999	*Electricity metering – Data exchange for meter reading, tariff and load control – Part 31: Using local area networks on twisted pair with carrier signalling* NOTE This Edition is referenced in the interface class "IEC twisted pair (1) setup" (class_id: 24, version: 0)
IEC 62056-3-1:2013	*Electricity metering data exchange – The DLMS/COSEM suite – Part 3-1: Use of local area networks on twisted pair with carrier signalling* NOTE This Edition is referenced in the interface class "IEC twisted pair (1) setup" (class_id: 24, version: 1)
IEC 62056-8-6: 2017	*Electricity metering data exchange – The DLMS/COSEM suite – Part 8-6: High speed PLC ISO/IEC 12139-1 profile for neighbourhood networks*
IEC 62056-8-8:2020	*Electricity metering data exchange - The DLMS/COSEM suite - Part 8-8: Communication profile for ISO/IEC 14908 series networks*
ISO/IEC 8802-2:1998	*IEEE Standard for Information technology – Telecommunications and information exchange between systems – Local and metropolitan area networks – Specific requirements – Part 2: Logical Link Control*
ISO/IEC 12139-1:2009	*Information technology —Telecommunications and information exchange between systems — Powerline communication (PLC) — High speed PLC medium access control (MAC) and physical layer (PHY) — Part 1: General requirements*
ISO/IEC 14908-1:2012	*Interconnection of information technology equipment – Control network protocol Part Protocol stack*
ISO/IEC/IEEE 60559:2011	*Information technology – Microprocessor Systems – Floating-Point arithmetic*
ISO 4217	*Codes for the representation of currencies and funds*
ITU-T E.212 (05.2008)	*SERIES E: OVERALL NETWORK OPERATION,TELEPHONE SERVICE, SERVICE OPERATION AND HUMAN FACTORS - International operation – Maritime mobile service and public land mobile service - The international identification plan for public networks and subscriptions*

© Copyright 1997-2020 DLMS User Association

Reference	Title
3GPP TS 24.008 V13.7.0 (2016-10)	*Technical Specification Digital cellular telecommunications system (Phase 2+) (GSM); Universal Mobile Telecommunications System (UMTS); LTE; Mobile radio interface Layer 3 specification; Core network protocols; Stage 3*
3GPP TS 24.301 V13.4.0 (2016-01)	*Technical Specification Group Core Network and Terminals; Non-Access-Stratum (NAS) protocol for Evolved Packet System (EPS); Stage 3*
3GPP TS 24.301 V13.11.0 (2018-01)	*Technical Specification Group Core Network and Terminals; Non-Access-Stratum (NAS) protocol for Evolved Packet System (EPS);Stage 3*
3GPP TS 27.007 V16.1.0 (2019-06)	*Technical Specification Group Core Network and Terminals; AT command set for User Equipment (UE)*
3GPP TS 36.101 V15.4.0 (2019-01)	*Technical Specification Group Radio Access Network; Evolved Universal Terrestrial Radio Access (E-UTRA); User Equipment (UE) radio transmission and reception*
3GPP TS 36.133 V13.11.0 (2018-03)	*Technical Specification LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Requirements for support of radio resource management*
3GPP TS 36.133 V14.4.0 (2017-06)	*Technical Specification LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Requirements for support of radio resource management*
3GPP TS 36.213 V15.5.0 (2019-05)	*Technical Specification Group Radio Access Network; Evolved Universal Terrestrial Radio Access (E-UTRA); Physical layer procedures*
3GPP TS 36.304 V13.8.0 (2018-01)	*Technical Specification Group Radio Access Network; Evolved Universal Terrestrial Radio Access (E-UTRA); User Equipment (UE) procedures in idle mode*
3GPP TS 36.321 V15.5.0 (2019-05)	*Technical Specification Group Radio Access Network; Evolved Universal Terrestrial Radio Access (E-UTRA); Medium Access Control (MAC) protocol specification*
3GPP TS 36.331 V15.5.1 (2019-05)	*Technical Specification Group Radio Access Network; Evolved Universal Terrestrial Radio Access (E-UTRA); Radio Resource Control (RRC); Protocol specification*
ITU-T G.9903 Amd. 1:2013	*SERIES G: TRANSMISSION SYSTEMS AND MEDIA, DIGITAL SYSTEMS AND NETWORKS – Access networks – In premises networks – Narrow-band orthogonal frequency division multiplexing power line communication transceivers for G3-PLC networks* NOTE This Recommendation is referenced in version 0 of the G3-PLC setup classes.
ITU-T G.9903:2014	*SERIES G: TRANSMISSION SYSTEMS AND MEDIA, DIGITAL SYSTEMS AND NETWORKS – Access networks – In premises networks –* *Narrow-band orthogonal frequency division multiplexing power line communication transceivers for G3-PLC networks* NOTE This Recommendation is referenced in version 1 of the G3-PLC setup classes.
ITU-T G.9903:2017	*SERIES G: TRANSMISSION SYSTEMS AND MEDIA, DIGITAL SYSTEMS AND NETWORKS - Access networks – In premises networks - Narrowband orthogonal frequency division multiplexing power line communication transceivers for G3-PLC networks* NOTE This Recommendation is referenced in current version of the G3-PLC setup classes.
ITU-T G.9904:2012	*SERIES G: TRANSMISSION SYSTEMS AND MEDIA, DIGITAL SYSTEMS AND NETWORKS – Access networks – In premises networks – Narrow-band orthogonal frequency division multiplexing power line communication transceivers for PRIME networks*
EN 834:1994	*Heat cost allocators for the determination of the consumption of room heating radiators – Appliances with electrical energy supply*
EN 1434-1:2015	*Heat meters – Part 1: General requirements*
EN 1434-2:2015	*Heat meters – Part 2: Constructional requirements*
EN 13757-1:2014	*Communication system for meters – Part 1: Data exchange*
EN 13757-2	*Communication systems for meters and remote reading of meters – Part 2: Physical and link layer*
EN 13757-3:2004	*Communication systems for and remote reading of meters – Part 3: Dedicated application layer* NOTE This standard is referenced in the “M-Bus client setup” interface class version 0.
EN 13757-3:2018	*Communication systems for meters – Part 3: Dedicated application layer* NOTE This standard is referenced in the M-Bus client setup interface class version 1.

© Copyright 1997-2020 DLMS User Association

Reference	Title
EN 13757-4:2013	*Communication system for and remote reading of meters – Part 4: Wireless meter (Radio meter reading for operation in SRD bands)*
EN 13757-5:2015	*Communication systems for meters – Part 5: Wireless M-Bus relaying*
EN 13757-7:2018	*Communication system for meters – Part 7: Transport and security services*
IEEE 802.15.4: 2006 also available as ISO/IEC/IEEE 8802-15-4 Ed 1.0	*IEEE 802.15.4-2006 Standard for Information technology – Telecommunications and information exchange between systems – Local and metropolitan area networks – Specific requirements – Part 15.4: Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (WPANs) – September 2006.*
ETSI GSM 05.08	*Digital cellular telecommunications system (Phase 2+); Radio subsystem link control*
ANSI C12.19:2012	*American National Standard For Utility Industry End Device Data Tables*
ZigBee® 053474	*ZigBee® Specification.* The specification can be downloaded free of charge from https://www.zigbee.org/zigbee-for-developers/zigbee-pro/
[FANSPEC]	*Wi-SUN Alliance: Field Area Network Working Group (FANWG):Technical Profile Specification:Field Area Network:Version 1v26.*
[PHYSPEC]	*Wi-SUN Alliance: PHY Working Group (PHYWG) Wi-SUN PHY Specification Revision 1V02*
LoRaWAN 1.0.3	LoRaWAN® Specification v1.0.3 *https://lora-alliance.org/resource-hub/lorawanr-specification-v103*
The following RFCs are available online from the Internet Engineering Task Force (IETF): ***http://www.ietf.org/rfc/std-index.txt, http://www.ietf.org/rfc/***	
IETF STD 51	*The Point-to-Point Protocol (PPP), 1994. (Also RFC 1661, RFC 1662)*
RFC 768	*User Datagram Protocol*
RFC 791	*Internet Protocol (Also: IETF STD 0005),* 1981
RFC 793	*Transmission Control Protco*
RFC 1144	*Compressing TCP/IP Headers for Low-Speed Serial Links,* 1990
RFC 1213	*Management Information Base for Network Management of TCP/IP-based internets: MIB-II*
RFC 1332	*The PPP Internet Protocol Control Protocol (IPCP),* 1992, Updated by: RFC 3241. Obsoletes: RFC 1172
RFC 1570	*PPP LCP Extensions, 1994*
IETF STD 51 / RFC 1661	*The Point-to-Point Protocol (PPP) (Also: IETF STD 0051),* 1994, Updated by: RFC 2153, Obsoletes: RFC 1548
IETF STD 51 / RFC 1662	*PPP in HDLC-like Framing, (Also: IETF STD 0051),* 1994, Obsoletes: RFC 1549
RFC 1994	*PPP Challenge Handshake Authentication Protocol (CHAP),* 1996. Obsoletes: RFC
RFC 2433	*PPP CHAP Extension,* 1998
RFC 2460	*Internet Protocol, Version 6*
RFC 2474	*Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers,* 1998
RFC 2507	*IP Header Compression,* 1999
RFC 2508	*Compressing IP/UDP/RTP Headers for Low-Speed Serial Links,* 1999
RFC 2759	*Microsoft PPP CHAP Extensions, Version 2,* 2000
RFC 2986	PKCS #10 v1.7: *Certification Request Syntax Standard*
RFC 3095	*RObust Header Compression (ROHC): Framework and four profiles: RTP, UDP, ESP, and uncompressed,* 2001
RFC 3241	*Robust Header Compression (ROHC) over PPP,* 2002. Updates: RFC1332

© Copyright 1997-2020 DLMS User Association

Reference	Title
RFC 3315	*Dynamic Host Configuration Protocol for IPv6 (DHCPv6)*
RFC 3513	*Internet Protocol Version 6 (IPv6) Addressing Architecture,* 2003
RFC 3544	*IP Header Compression over PPP,* 2003
RFC 3748	*Extensible Authentication Protocol (EAP),* 2004
RFC 4291	*IP Version 6 Addressing Architecture*
RFC 4443,	*Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification*
RFC 4944	*Transmission of IPv6 Packets over IEEE 802.15.4 Networks*
RFC 4861	*Neighbor Discovery for IP version 6 (IPv6), 2007*
RFC 4944	*Internet Engineering Task Force (IETF). RFC 4944: Transmission of IPv6 Packets over IEEE 802.15.4 Networks [online]. Edited by G. Montenegro, N. Kushalnagar and D. Culler. September 2007*
RFC 5216	*The EAP-TLS Authentication Protocol*
RFC 5280	*Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile, 2008*
RFC 5905	*Network Time Protocol Version 4: Protocol and Algorithms Specification, 2010.*
RFC 6206	*The Trickle Algorithm*
RFC 6282	*Internet Engineering Task Force (IETF). RFC 6282: Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks [online]. Edited by J. Hui, Ed. September 2011*
RFC 6550	*RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks*
RFC 6775	*Neighbor Discovery Optimization for IPv6 over Low-Power Wireless Personal Area Networks (6LoWPANs), 2012*
RFC 7731	*Multicast Protocol for Low-Power and Lossy Networks (MPL)*
RFC 7774	*Multicast Protocol for Low-Power and Lossy Networks (MPL) Parameter Configuration Option for DHCPv6*
	Point-to-Point (PPP) Protocol Field Assignments. Online database. Available from: http://www.iana.org/assignments/ppp-numbers/ppp-numbers.xhtml

3 Terms, definitions and abbreviated terms

For the purposes of this document, the following terms and definitions apply.

ISO and IEC maintain terminological databases for use in standardization at the following addresses:

- IEC Electropedia: available at http://www.electropedia.org/
- ISO Online browsing platform: available at http://www.iso.org/obp

3.1 Terms and definitions related to the Image transfer process (see 4.4.6)

3.1.1
Image
binary data of a specified size

Note 1 to entry: An Image can be seen as a container. It may consist of one or multiple elements (image_to_activate) which are transferred, verified and activated together.

3.1.2
ImageSize
size of the whole Image to be transferred

© Copyright 1997-2020 DLMS User Association

Note 1 to entry: ImageSize is expressed in octets.

3.1.3
ImageBlock
part of the Image of size ImageBlockSize

Note 1 to entry: The Image is transferred in ImageBlocks. Each block is identified by its ImageBlockNumber.

3.1.4
ImageBlockSize
size of ImageBlock expressed in octets

3.1.5
ImageBlockNumber
identifier of an ImageBlock. ImageBlocks are numbered sequentially, starting from 0

The meaning of the definitions above is illustrated in Figure 2.

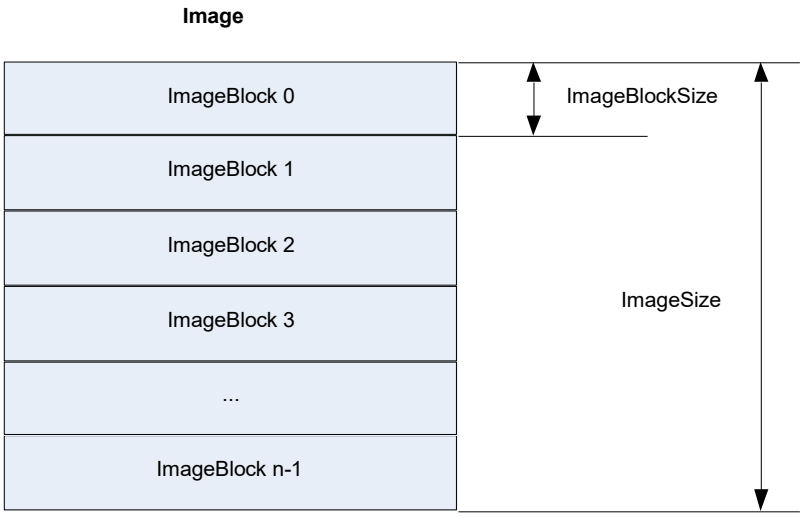


Figure 2 – The meaning of the definitions concerning the Image

3.2 Terms and definitions related to the S-FSK PLC setup ICs (see 4.10)

3.2.1
initiator
user-element of a client System Management Application Entity (SMAE)

Note 1 to entry: The initiator uses the CIASE and xDLMS ASE and is identified by its system title.

[SOURCE: IEC 61334-4-511:2000, 3.8.1]

3.2.2
active initiator
initiator which issues or has last issued a CIASE Register request when the server is in the unconfigured state

[SOURCE: IEC 61334-4-511:2000, 3.9.1]

3.2.3
new system
server system which is in the unconfigured state: its MAC address equals "NEW-address"

[SOURCE: IEC 61334-4-511:2000, 3.9.3]

© Copyright 1997-2020 DLMS User Association

3.2.4
new system title
system-title of a new system

Note 1 to entry: This is the system title of a system, which is in the new state.

[SOURCE: IEC 61334-4-511:2000 3.9.4]

3.2.5
registered system
server system which has an individual valid MAC address (therefore, different from "NEW Address", see IEC 61334-5-1:2001: Medium Access Control)

[SOURCE: IEC 61334-4-511:2000 3.9.5]

3.2.6
reporting system
server system which issues a DiscoverReport

[SOURCE: IEC 61334-4-511:2000, 3.9.6 modified to correct an error in IEC 61334-4-511.]

3.2.7
sub-slot
time needed to transmit two bytes by the physical layer

Note 1 to entry: Timeslots are divided to sub-slots in the RepeaterCall mode of the physical layer.

3.2.8
timeslot
time needed to transmit a physical frame

Note 1 to entry: As specified in IEC 61334-5-1, 3.3.1, a physical frame comprises 2 bytes preamble, 2 bytes start subframe delimiter, 38 bytes PSDU and 3 bytes pause.

3.3 Terms and definitions related to the PRIME NB OFDM PLC setup ICs (see 4.12)

Definitions related to the physical layer

3.3.1
base node
the master node, which controls and manages the resources of a subnetwork

[SOURCE: ITU-T G.9904:2012, 3.2.1]

3.3.2
beacon slot
the location of the beacon PDU within a frame

[SOURCE: ITU-T G.9904:2012, 3.2.2]

3.3.3
node
any one element of a subnetwork, which is able to transmit to and receive from other subnetwork elements

[SOURCE: ITU-T G.9904:2012, 3.2.9]

3.3.4
registration
the process by which a service node is accepted as member of the subnetwork and allocated a LNID

© Copyright 1997-2020 DLMS User Association

[SOURCE: ITU-T G.9904:2012, 3.2.12]

3.3.5
service node
any one node of a subnetwork, which is not a base node

[SOURCE: ITU-T G.9904:2012, 3.2.13]

3.3.6
subnetwork
a set of elements that can communicate by complying with this specification and share a single base node

[SOURCE: ITU-T G.9904:2012, 3.2.15]

Definitions related to the MAC layer

3.3.7
disconnected state <of a service node>
this is the initial functional state for all service nodes. When disconnected, a service node is not able to communicate data or switch other nodes' data; its main function is to search for a subnetwork within its reach and try to register on it

[SOURCE: ITU-T G.9904:2012, 8.1]

3.3.8
terminal state <of a service node>
when in this functional state a service node is able to establish connections and communicate data, but it is not able to switch other nodes' data

[SOURCE: ITU-T G.9904:2012, 8.1]

3.3.9
switch state <of a service node>
when in this functional state a service node is able to perform all Terminal functions. Additionally, it is able to forward data to and from other nodes in the same subnetwork. It is a branch point on the tree structure

[SOURCE: ITU-T G.9904:2012, 8.1]

3.3.10
promotion
the process by which a service node is qualified to switch (repeat, forward) data traffic from other nodes and act as a branch point on the subnetwork tree structure. A successful promotion represents the transition between Terminal and Switch state. When a service node is in the Disconnected state, it cannot directly transition to Switch state

[SOURCE: ITU-T G.9904:2012, 8.1]

3.3.11
demotion
the process by which a service node ceases to be a branch point on the subnetwork tree structure. A successful demotion represents the transition between Switch and Terminal state

[SOURCE: ITU-T G.9904:2012, 8.1]

3.4 Terms and definitions related to the ISO/IEC 14908 setup ICs (see 4.18)

3.4.1
domain
logical network that is a unit for addressing

© Copyright 1997-2020 DLMS User Association

Note 1 to entry: Subnet (see below) and node addresses are assigned by the administrator responsible for the domain, and they have meaning only in the context of that domain.

Note 2 to entry: All nodes must be in the same domain to be able to address each other.

3.4.2
node
abstraction for a physical node that represents the highest degree of address resolvability on a network

Note to entry : A node is identified (addressed) within a subnet by its (logical) node identifier. A physical node may belong to more than one subnet; when it does, it is assigned one (logical) node number for each subnet to which it belongs. A physical node may belong to at most two subnets; these subnets must be in different domains. A node may also be identified (absolutely) within a network by its Unique_Node_ID.

3.4.3
subnet
set of nodes accessible through the same link layer protocol

3.4.4
transaction
sequence of messages that are correlated together

3.5 Terms and definitions related to ZigBee® (see 4.15)

NOTE Terms marked with * are from the ZigBee® Specification.

3.5.1
CAD
Consumer Access Device; a ZigBee® gateway device that acts like an IHD within the ZigBee® network, but has an additional connection to a different network (i.e. WiFi)

3.5.2
IHD
in Home Display; a device that has a screen for the displaying of Energy information to the consumer

3.5.3
install code
a Hashed (via MMO) Pre-Configured Linked Key (PCLK) that is provided to a Trust Center via out-of-band communications

Note to entry: A new device wishing to join the network would need to send this install code to the Trust Center, which would allow the Trust Center to execute the joining process, using this install code as part of the security information

3.5.4
link key *
this is a key that is shared exclusively between two, and only two, peer application-layer entities within a PAN

3.5.5
MAC address/IEEE address
these are used synonymously to represent the EUI-64 code allocated to the ZigBee® Radio

3.5.6
ZigBee®
ZigBee® is a specification for a suite of high level communication protocols used to create personal area networks built from small, low-power digital radios. ZigBee® is based on an IEEE 802.15 standard. Though low-powered, ZigBee® devices often transmit data over longer distances by passing data through intermediate devices to reach more distant ones, creating a mesh network

© Copyright 1997-2020 DLMS User Association

3.5.7
ZigBee® client
this is similar to the role of the DLMS/COSEM Client

Note to entry: For a greater understanding of the interaction between the client and server the ZigBee® PRO specification should be read.

3.5.8
ZigBee® coordinator *
an IEEE 802.15.4-2003 PAN coordinator that is the principal controller of an IEEE 802.15.4-2003-based network that is responsible for network formation. The PAN coordinator must be a full function device (FFD)

3.5.9
ZigBee® cluster
a set of message types related to a certain device function (e.g. metering, ballast control)

3.5.10
ZigBee® mirror
a device which echoes data being published by a battery operated ZigBee® device, allowing other network actors to obtain data while the battery operated device is unavailable due to power saving

3.5.11
ZigBee® PRO
an alternative name for the ZigBee® 2007 protocol

3.5.12
ZigBee® router *
an IEEE 802.15.4-2003 FFD participating in a ZigBee® network, which is not the ZigBee® coordinator but may act as an IEEE 802.15.4-2003 coordinator within its personal operating space, that is capable of routing messages between devices and supporting associations

3.5.13
ZigBee® server
this is similar to the role of the DLMS/COSEM Server

Note 1 to entry: For a greater understanding of the interaction between the client and server the ZigBee® PRO specification should be read.

3.5.14
ZigBee® Trust Center *
the device trusted by devices within a ZigBee® network to distribute keys for the purpose of network and end-to-end application configuration management

3.6 Terms and definitions related to Payment metering ICs (see 4.6)

3.6.1
account
statement of the credits and charges of an individual with reference to a contractual relationship between the said individual and another party; in this case a utility service provider

3.6.2
available
(credit), total value that may be decremented by charges without further action

3.6.3
charge
representation of a financial liability on an account

© Copyright 1997-2020 DLMS User Association

Note 1 to entry: Within this specification charges are modelled in the form of “Charge” objects that define the amount due, collection mechanism, collection periodicity, collection amount and other relevant variables.

Note 2 to entry: There may be one or more instalments payable and their size may be determined explicitly or in terms of a rate of payment per unit of time or of consumption.

Note 3 to entry: Charges may also be levied as a fixed amount per vend.

3.6.4
credit mode
mode of operation of a meter in a payment system that does not require payment for the consumption in advance

3.6.5
collect
take payment of an instalment of a charge, accounting for the collection amount determined by the *unit_charge_active* attribute of the “Charge” object

3.6.6
commodity
utility product delivered to a consumer at a service point on their premises under a contract of supply such as electricity, gas, water, and heat

3.6.7
enabled
when used in the context of “Credit” or “Charge” types; means that the “Credit” or “Charge” type appears in the *credit_reference_list* or *charge_reference_list* respectively of the “Account” object

3.6.8
emergency credit
amount of credit administered in a payment metering system working in prepayment mode, representing a short term loan to the consumer

Note to entry: This is a feature of some payment metering systems in which the consumer is able to obtain a limited amount of credit as a short-term loan, often mediated locally by the prepayment unit itself. The word “emergency” indicates urgent need rather than disaster.

3.6.9
Enterprise Resource Planning (ERP) system
Back Office System
computer system carrying out the business processing of an organisation (such as an energy supplier), as distinct from the communications system. See also Head End System

3.6.10
friendly credit
period of time with a configurable start and end point, where the meter will not disconnect supply regardless of the status of the *available_credit*. Also known as non-disconnect period

Note to entry: This function is used in circumstances where it would be inconvenient to obtain needed credit (for example, at night or in the case of a frail elderly consumer).

3.6.11
Head End System (HES)
computer system, connected by a communications network to a population of intelligent devices, whose job is the control and coordination of information flows to and from those devices, typically on behalf of a separate ERP (“back office”) system

3.6.12
Home Area Network (HAN)
communications network constructed with the principal aim of connecting devices in one premises

© Copyright 1997-2020 DLMS User Association

3.6.13
in use
state of a “Credit” object that, at the point of query, has a positive *current_credit_amount* and that Credit is being consumed by some active Charges represented by “Charge” objects

Note to entry: When the current_credit_amount reaches zero, the credit status becomes exhausted.

3.6.14
load limiting
mode of operation of some payment metering systems (not necessarily in prepayment mode) in which the consumer is able to draw on a supply provided they do not exceed a configured level of demand

Note to entry: The implied purpose is for management of the consumer’s finances: where demand is subject to limitation for the benefit of the generation or distribution system the term “load management” is more often used.

3.6.15
local communications
mechanism of communicating with the meter over some media, within the vicinity of that meter such as over a HAN or optical port

3.6.16
manual entry
entering of a token to the payment metering installation via means of a manual process

3.6.17
managed payment mode
specialisation of credit mode that allows operation of an Account, Credit and possibly Charges in a meter where the payment for the service is received by the utility after the service has been consumed

Note 1 to entry: When in managed payment mode tokens are not normally used, however the credit is adjusted using the methods in the “Credit” object.

Note 2 to entry: The meter is allowed to go into an allowable amount of debt before being credited from the client in line with a received cash payment by the utility.

Note 3 to entry: In this example cash is used as a generic term for a real life payment of currency to the utility which could be executed as legal tender, automated electronic transfer etc.

3.6.18
payment metering installation
set of payment metering equipment installed and ready for use at a consumer’s premise

Note to entry: This includes mounting the equipment as appropriate, and where a multi-device installation is involved, the connection of each unit of equipment as appropriate. It also includes the connection of utility supply network to each supply interface, the connection of the consumer’s load interface, and the commissioning of the equipment into an operational state as a payment metering installation.

3.6.19
prepayment mode
mode of operation of a meter in a payment system, whereby the consumer pays for service in advance of consumption

3.6.20
post-payment
method of operation of a payment system whereby a consumer may consume service before paying for it

Note 1 to entry: This term can be used interchangeably with the term Credit mode when used in the context of operational modes.

Note 2 to entry: This term is usually used in conjunction with a system description whereas Credit mode is used when referring to the operational mode of a meter or account.

© Copyright 1997-2020 DLMS User Association

3.6.21
remote communications
transportation of a token or other message from a client to a server running a payment metering application process via some form of WAN and access network

Note to entry: This could be point to point, mesh radio, fibre optic connection etc. and may travel through multiple devices and over multiple protocols before reaching the meter.

3.6.22
repayable
credit_types such as emergency credit where an amount added to *current_credit_amount* of a "Credit" object has to be repaid before the Credit is selectable again

3.6.23
reserved credit
amount of credit that is held in reserve in the account of a payment meter, for use at a later time, at the discretion of the consumer

Note to entry: The mechanism for reserving this credit may be subject to agreement between the utility supplier and the consumer. For example a proportion of every token may be added to the reserve Credit or the supplier might give the consumer an allowance every month, but these arrangements will be project specific.

3.6.24
selectable
specific state of a "Credit" object where the consumer's immediate confirmation is needed before it can be brought into use

Note to entry: For example, Emergency Credit has the nature of a short-term loan and should therefore only be deployed with the consumer's agreement. The term refers respectively to the need to get agreement and to the fact of having received agreement. Only a "Credit" made (1) Selectable can be (2) Selected / Invoked. Not all Credits need to be selected by an external trigger, as in most cases the meter application automatically performs this action.

3.6.25
selected/invoked
specific state of a "Credit" object where the value of *current_credit_amount* is included in the calculation of *available_credit* in the related "Account"

Note to entry: This is the state of a Credit before becoming In use and is considered in the available_credit attribute of the "Account", but is not yet being consumed by any Charge (due to a higher priority Credit being In use.

3.6.26
service
provision of a commodity (such as water, electricity, gas or heat)

3.6.27
social credit
credit that is given free of payment for reasons such as the relief of poverty

Note 1 to entry: Typically such a credit is given at fixed times (e.g. monthly) in limited amounts. This particular type of credit could also be consumption based, such that that the consumer must keep consumption below a limiting threshold in order to use the social credit. This could be controlled by the consumer being disconnected if the limit is breached.

Note 2 to entry: Social credits are modelled of "Credit" objects of type emergency_credit, time_based_credit, consumption_based_credit.

3.6.28
temporary debt
transient liability to the meter that accrues when *Charge*s are collected at a time when all credits are exhausted

Note to entry: This temporary debt amount is accumulated in amount_to_clear in the "Account" object.

3.6.29
token
self-contained package of data related to the purchase of credit or to other system functions, embodied in a token carrier (q.v.). The token forms a link between source and destination of

© Copyright 1997-2020 DLMS User Association

the transaction. The token contents may reflect money, energy, time, etc., in harmony with the currency declared in the meter

Note to entry: Defined in IEC TR 62055-21:2005 as ““<Equipment-related definition>[sic] information content including an instruction issued on a token carrier by a vending or management system that is capable of subsequent transfer to and acceptance by a specific payment meter, or one of a group of meters, with appropriate security”.

3.6.30
token carrier
means of transferring a token from one system element to another, typically in material “physical” or electronic “virtual” form

Note to entry: In a general sense, the token refers to the instruction and information being transferred, while the token carrier refers to the physical device being used to carry the instruction and information, or to the communications medium in the case of a virtual token carrier.

3.6.31
token carrier interface
interface between the token carrier and the payment metering installation

Note 1 to entry: For example, it may be a keypad for numeric tokens, or a physical token carrier acceptor, or a communications connection to a local or remote machine for a virtual token carrier interface.

Note 2 to entry: The token carrier interface may also be used to pass additional information to or from the payment meter, such as for the purposes of payment system management.

3.6.32
top-up credit token
credit purchased by the consumer and capable of being delivered in the form of a token (as well as by other means) in a physical or virtual token carrier

3.6.33
vend
operation or transaction resulting in the available credit held on a payment meter to be increased by use of a credit token

Note to entry: Vend would normally relate to a transaction in conjunction with a vending system at a point of sale, resulting in the creation of a token that can be transported by means of a physical or virtual token carrier.

3.7 Terms and definitions related to the Arbitrator IC (see 4.5.12)

3.7.1
action
operation that can be requested locally or remotely from the server

3.7.2
actor
entity requesting an action

Note to entry: It can be the local application process or a client.

3.7.3
arbitrator
function modelled in COSEM that can determine, based on pre-configured rules, which action is carried out when multiple actors request potentially conflicting actions to control the same resource

© Copyright 1997-2020 DLMS User Association

3.8 Abbreviated terms

Abbreviation	Explanation
3GPP	3rd Generation Partnership Project
6LoWPAN	IPv6 over Low-Power Wireless Personal Area Network
AA	Application Association
ABP	Activation by Personalisation
AARE	A-Associate Response – an APDU of the ACSE
AARQ	A-Associate Request – an APDU of the ACSE
ACSE	Association Control Service Element
ADD	Automated Device Discovery
APDU	Application layer protocol data unit
ADP	Primary Station Address
ADS	Secondary Station Address
AGA	American Gas Association
AGA 8	Method for calculation of compressibility (Gas metering)
AGC	Automatic Gain Control
AL	Application layer
AP	Application process
APDU	Application Protocol Data Unit
APS	Application Support Sublayer (ZigBee® term)
ARFCN	Absolute Radio Frequency Channel Number
ASE	Application Service Element
ATM	Automated Topology Management
A-XDR	Adapted Extended Data Representation (IEC 61334-6)
base_name	The short_name corresponding to the first attribute (“logical_name”) of a COSEM object
BCD	Binary Coded Decimal
BER	Bit Error Rate
CBCP	CallBack Control Protocol (PPP)
CC	Current Credit (S-FSK PLC profile)
C/D	Compression and Decompression
CDMA	Code Division Multiple Access
CENELEC	European Committee for Electrotechnical Standardization
CHAP	Challenge Handshake Authentication Protocol
CIASE	Configuration Initiation Application Service Element (S-FSK PLC profile)
class_id	Interface class identification code
CLI	Calling Line Identity
COSEM	Companion Specification for Energy Metering
COSEM object	An instance of a COSEM interface class
CPAS	Common Part Adaptation Sublayer
CRC	Cyclic Redundancy Check
CSD	Circuit Switched Data
CSAP	Client Service Access Point

© Copyright 1997-2020 DLMS User Association

Abbreviation	Explanation
CSMA	Carrier Sense Multiple Access
CtoS	Client to Server challenge
CU	Currently Unused
DC	Delta credit (S-FSK PLC profile)
DC	Data concentrator
DevAddr	(Related to LoRaWAN) A 32-bit non-unique identifier assigned to an end-device statically or dynamically after a Join Procedure (depending on the activation mode)
DHCP	Dynamic Host Configuration Protocol
DIB	Data Information Block (M-Bus)
DIF	Data Information Field (M-Bus)
DL	Data Link
DLMS	Device Language Message Specification
DLMS UA	DLMS User Association
DLPDU	Data Link Protocol Data Unit
DNS	Domain Name Server
DSCP	Differentiated Services Code Point
DSSID	Direct Switch ID
EAP	Extensible Authentication Protocol
EARFCN	E-UTRA Absolute Radio Frequency Channel Number
EDGE	Enhanced Data rates for GSM Evolution
EMC	Emergency Credit (Payment metering)
ERP	Enterprise Resource Planning
EUI-48	48-bit Extended Unique Identifier
EUI-64	64-bit Extended Unique Identifier
E-UTRA	Evolved UMTS Terrestrial Radio Access
FCC	Federal Communications Commission
FFD	Full-Function Device
FIFO	First-In-First-Out
F/R	Fragmentation and Reassembly
FTP	File Transfer Protocol
GCM	Galois/Counter Mode, an algorithm for authenticated encryption with associated data
GMT	Greenwich Mean Time. Replaced by Coordinated Universal Time (UTC).
GPRS	General Packet Radio Service
GPS	Global Positioning System
GSM	Global System for Mobile Communications
HAN	Home Area Network
HART	Highway Addressable Remote Transducer; see http://www.hartcomm.org/ (in relation with the Sensor manager interface class)
HDLC	High-level Data Link Control
HES	Head End System
HHT	Hand Held Terminal
HLS	High Level Security Authentication

© Copyright 1997-2020 DLMS User Association

Abbreviation	Explanation
HSDPA	High-Speed Downlink Packet Access
HS-PLC	High-Speed Power Line Carrier
IANA	Internet Assigned Numbers Authority
IB	Information Base
IC	Interface Class (COSEM)
IC	Initial credit (S-FSK PLC profile)
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
IPCP	Internet Protocol Control Protocol
IPv4	Internet Protocol version 4
IPv6	Internet Protocol version 6
ISO	International Organization for Standardization
ISP	Internet Service Provider
IT	Information Technology
ITU-T	International Telecommunication Union – Telecommunication
KEK	Key Encryption Key
LA	Local Area
LAC	Local Area Code
LAN	Local Area Network
LCID	Local Connection Identifier
LCP	Link Control Protocol
LDN	Logical Device Name
LBD	(6)LoWPAN Bootstrapping Device
LLC	Logical Link Control (sublayer)
LLS	Low Level Security
LN	Logical Name
LNID	Local Node Identifier
LOADng	6LoWPAN Ad Hoc On-Demand Distance Vector Routing Next Generation (LOADng)
LQI	Link Quality Indicator (ZigBee ® term)
LSB	Least Significant Bit
LSID	Local Switch Identifier
LTE	Long Term Evolution (Wireless communication)
m	mandatory
M2M	Machine to Machine
MAC	Medium Access Control
M-Bus	Meter Bus
MCC	Mobile Country Code
MD5	Message Digest Algorithm 5
MIB	Management Information Base (S-FSK PLC profile)
MID	Measuring Instruments Directive 2004/22/EC of the European Parliament and of the Council

© Copyright 1997-2020 DLMS User Association

Abbreviation	Explanation
MMO	Matyas-Meyer-Oseas hash (ZigBee ® term)
MNC	Mobile Network Code
MPAN	(UK term) Meter Point Access Number – reference of the location of the Electricity meter on the electricity distribution network.
MPDU	MAC Protocol Data Unit
MSB	Most Significant Bit
MSDU	MAC Service Data Unit
MT	Mobile Termination
NB	Narrow-band
ND	Neighbour Discovery
NRSRP	Narrowband Reference Signal Received Power
NRSRQ	Narrowband Reference Signal Received Quality
NTP	Network Time Protocol
o	optional
OBIS	Object Identification System
OFDM	Orthogonal Frequency Division Multiplexing
OTA	Over the Air – Refers to Firmware Upgrade using ZigBee ®
PAN	Personal Area Network (Term used in relation to G3-PLC [1]) and ZigBee ®
Pad	Padding
PAP	Password Authentication Protocol
PCLK	Pre-Configured Link Key (ZigBee® term)
PDU	Protocol Data Unit
PhL, PHY	Physical Layer
PIB	PLC Information Base
PID	Program Identifier
PIN	Personal Identity Number
PLC	Power Line Carrier
PLMN	Public Land Mobile Network
PNPDU	Promotion Needed PDU
POS	Personal Operating Space (ZigBee ®)
POS	Point Of Sale (Payment metering)
PPDU	Physical Protocol Data Unit
PPP	Point-to-Point Protocol
PSTN	Public Switched Telephone Network
QoS	Quality of Service
RB	Radio Band
REJ PDU	Reject Protocol Data Unit
RFC	Request for Comments; a document published by the Internet Engineering Task Force
RFD	Reduced Function Device
ROHC	Robust Header Compression
RREP	Route Reply
RREQ	Route Request

© Copyright 1997-2020 DLMS User Association

Abbreviation	Explanation
RRER	Route Error
RSRP	Reference Signal Received Power
RSRQ	Reference Signal Received Quality
RSSI	Received Signal Strength Indication (ZigBee® term)
SAP	Service Access Point
SAS	Startup Attribute Set (ZigBee® term)
SCP	Shared Contention Period
SDU	Service Data Unit
SE	Smart Energy
SEP	Smart Energy Profile (ZigBee® term)
S-FSK	Spread – Frequency Shift Keying
SGERG88	Method for calculation of compressibility (Gas metering)
SHA	Secure Hash Algorithm
SI	International System of Units (*Système International d'Unités)*
SID	Switch identifier
SMS	Short Message Service
SMTP	Simple Mail Transfer Protocol
SN	Short Name
SNA	Subnetwork Address
SSAS	Service Specific Adaptation Sublayer
SSCS	Service Specific Convergence Layer
StoC	Server to Client Challenge
TAB	In the case of the EURIDIS profiles without DLMS and without DLMS/COSEM: data code. In the case of profiles using DLMS or DLMS/COSEM: value at which the equipment is programmed for Discovery
TABi	List of TAB field
TCC	Transmission Control Code (IPv4)
TCP	Transmission Control Protocol
TFTP	Trivial File Transfer Protocol
TOU	Time of use
TTL	Time To Live
UDP	User Datagram Protocol
UMTS	Universal Mobile Telecommunications System
UNC	Unconfigured (S-FSK PLC profile)
UTC	Coordinated Universal Time
VIB	Value Information Block (M-Bus)
VIF	Value Information Field (M-Bus)
VZ	Billing period counter (Form *Vorwertzähler* in German, see DIN 43863-3)
wake-up	trigger the meter to connect to the communication network to be available to a client (e.g. HES)
WAN	Wide Area Network
wM-Bus	Wireless M-Bus

© Copyright 1997-2020 DLMS User Association

Abbreviation	Explanation
ZTC	ZigBee® Trust Center
[1)] In the case of the G3-PLC technology, PAN may be defined as PLC Area Network.	

4 The COSEM interface classes

4.1 Basic principles

4.1.1 General

This Clause 4 describes the basic principles on which the COSEM interface classes (ICs) are built. It also gives a short overview on how interface objects – instantiations of the ICs – are used for communication purposes. Data collection systems and metering equipment from different vendors, following these specifications, can exchange data in an interoperable way.

For specification purposes, this document uses the technique of object modelling.

An object is a collection of attributes and methods. Attributes represent the characteristics of an object. The value of an attribute may affect the behaviour of an object. The first attribute of any object is the *logical_name*. It is one part of the identification of the object. An object may offer a number of methods to either examine or modify the values of the attributes.

Objects that share common characteristics are generalized as an interface class, identified with a class_id. Within a specific IC, the common characteristics (attributes and methods) are described once for all objects. Instantiations of ICs are called COSEM interface objects.

Manufacturers may add proprietary methods and attributes to any object; see 4.1.2.

Figure 3 illustrates these terms by means of an example:

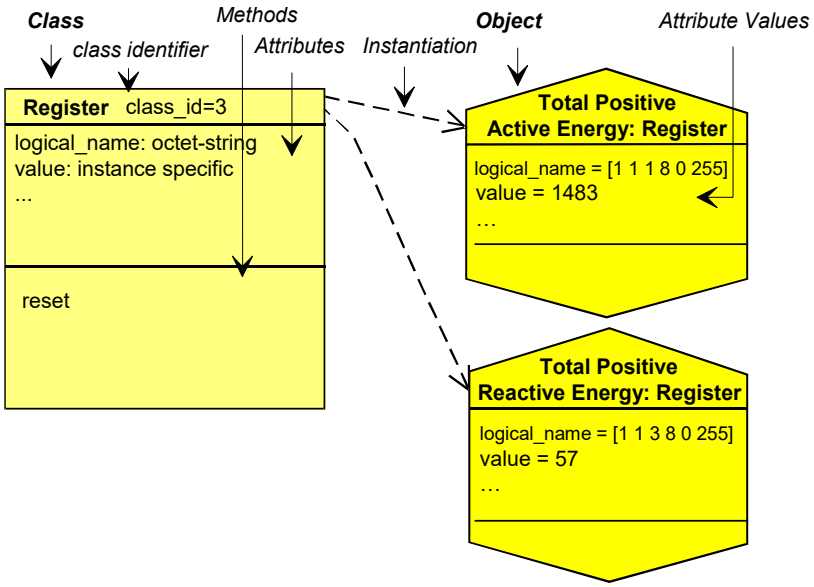


Figure 3 – An interface class and its instances

The IC “Register” is formed by combining the features necessary to model the behaviour of a generic register (containing measured or static information) as seen from the client (data collection system, hand held terminal). The contents of the register are identified by the attribute *logical_name*. The *logical_name* contains an OBIS identifier (see Clause 7). The actual (dynamic) content of the register is carried by its *value* attribute.

Defining a specific meter means defining several specific objects. In the example of Figure 3, the meter contains two registers; i.e. two specific instances of the IC “Register” are instantiated.

© Copyright 1997-2020 DLMS User Association

Through the instantiation, one COSEM object becomes a “total, positive, active energy register” whereas the other becomes a “total, positive, reactive energy register”.

NOTE The COSEM interface objects (instances of COSEM ICs) represent the behaviour of the meter as seen from the “outside”. Therefore, modifying the value of an attribute – for example resetting the *value* attribute of a register – is always initiated from the outside. Internally initiated changes of the attributes – for example updating the *value* attribute of a register – are not described in this model.

4.1.2 Referencing methods

Attributes and methods of COSEM objects can be referenced in two different ways:

Using logical names (LN referencing): In this case, the attributes and methods are referenced via the identifier of the COSEM object instance to which they belong.

The reference for an attribute is: class_id, value of the *logical_name* attribute, attribute_index.

The reference for a method is: class_id, value of the *logical_name* attribute, method_index, where:

- attribute_index is used as the identifier of the attribute required. Attribute indexes are specified in the definition of each IC. They are positive numbers starting with 1. Proprietary attributes may be added: these shall be identified with negative numbers;
- method_index is used as the identifier of the method required. Method indexes are specified in the definition of each IC. They are positive numbers starting with 1. Proprietary methods may be added: these shall be identified with negative numbers.

Using short names (SN referencing): This kind of referencing is intended for use in simple devices. In this case, each attribute and method of a COSEM object is identified with a 13-bit integer. The syntax for the short name is the same as the syntax of the name of a DLMS named variable. See IEC 61334-4-41:1996 and DLMS UA 1000-2 Ed. 10, 9.5.

4.1.3 Reserved base_names for special COSEM objects

In order to facilitate access to devices using SN referencing, some short_names are reserved as base_names for special COSEM objects. The range for reserved base_names is from 0xFA00 to 0xFFF8. The following specific base_names are defined, see Table 1.

Table 1 – Reserved base_names for SN referencing

Base_name (objectName)	COSEM object
0xFA00	Current Association SN
0xFB00	Script table (instantiation: Broadcast “Script table”)
0xFC00	SAP assignment
0xFD00	“Data” or “Register” object containing the “COSEM logical device name” in the attribute “value”

4.1.4 Class description notation

This subclause describes the notation used to define the ICs.

A short text describes the functionality and application of the IC. A table gives an overview of the IC including the class name, the attributes, and the methods. Each attribute and method shall be described in detail. The template is shown below.

© Copyright 1997-2020 DLMS User Association

Class name		**Cardinality**	**class_id, version**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. ...	(...)	...				x + 0x...
3. ...	(...)	...				x + 0x...
Specific methods (if required)		***m/o***				
1.		...				x + 0x...
2.		...				x + 0x...
3.		...				x + 0x...

Class name	Describes the interface class (e.g. “Register”, “Clock”, “Profile generic”...). NOTE 1 Interface classes names are mentioned in quotation marks.
Cardinality	Specifies the number of instances of the IC within a logical device (see 4.1.8). *value* The IC shall be instantiated exactly “value” times. *Min...max.* The IC shall be instantiated at least “min.” times and at most “max.” times. If min. is zero (0) then the IC is optional, otherwise (min. > 0) “min.” instantiations of the IC are mandatory.
class_id	Identification code of the IC (range 0 to 65 535). The class_id of each object is retrieved together with the logical name by reading the *object_list* attribute of an “Association LN” / ”Association SN” object. - class_id-s from 0 to 8 191 are reserved to be specified by the DLMS UA. - class_id-s from 8 192 to 32 767 are reserved for manufacturer specific ICs. - class_id-s from 32 768 to 65 535 are reserved for user group specific ICs. The DLMS UA reserves the right to assign ranges to individual manufacturers or user groups.
version	Identification code of the version of the IC. The version of each object is retrieved together with the class_id and the logical name by reading the *object_list* attribute of an “Association LN” / ”Association SN” object. **Within one logical device, all instances of a certain IC shall be of the same version.** Version numbers are to be allocated by the DLMS User Association.
Attributes	Specifies the attributes that belong to the IC. *(dyn.)* Classifies an attribute that carries a process value, which is updated by the meter itself. *(static)* Classifies an attribute, which is not updated by the meter itself (for example configuration data). There are some attributes which may be either static or dynamic depending on the application. In these cases this property is not indicated. NOTE 2 Attribute names use the underscore notation. When mentioned in the text they are in *italic*. Example: *logical_name*
logical_ name	octet-string It is always the first attribute of an IC. It identifies the instantiation (COSEM object) of this IC. The value of the *logical_name* conforms to OBIS; see Clauses 6 and 7.
Data type	Defines the data type of an attribute; see 4.1.5.

© Copyright 1997-2020 DLMS User Association

Min.	Specifies if the attribute has a minimum value. *X* The attribute has a minimum value. <*empty*> The attribute has no minimum value.
Max.	Defines if the attribute has a maximum value. *X* The attribute has a maximum value. <*empty*> The attribute has no maximum value.
Def.	Specifies if the attribute has a default value. This is the value of the attribute after reset. *X* The attribute has a default value. <*empty*> The default value is not defined by the IC specification.
Short name	When Short Name (SN) referencing is used, each attribute and method of object instances has to be mapped to short names. The base_name x of each object instance is the DLMS named variable the logical name attribute is mapped to. It is selected in the implementation phase. The IC definition specifies the offsets for the other attributes and for the methods.
Specific methods	Provides a list of the specific methods that belong to the object. Method Name () The method has to be described in the subsection “Method description”. NOTE 3 Method names use the underscore notation. When mentioned in the text they are in *italic*. Example: *add_object.*
m/o	Defines if the method is mandatory or optional. *m (mandatory)* The method is mandatory. *o (optional)* The method is optional.

Attribute description

Describes each attribute with its data type (if the data type is not simple), its data format and its properties (minimum, maximum and default values).

Method description

Describes each method and the invoked behaviour of the COSEM object(s) instantiated.

NOTE Services for accessing attributes or methods by the protocol are specified in DLMS UA 1000-2 Ed. 10, Clause 9.

Selective access

The xDLMS attribute-related services typically reference the entire attribute. However, for certain attributes selective access to just a part of the attribute may be provided. The part of the attribute is identified by specific selective access parameters. These are defined as part of the attribute specification.

Selective access is available with the following interface class attributes and methods:

- “Profile generic” objects, buffer attribute;
- “Association SN” objects, object_list and access_rights_list attribute;
- “Association LN” objects, object_list attribute;
- “Compact data”, objects, compact_buffer attribute;
- “Push” objects, push_object_list attribute;
- “Data protection” objects, protection_object_list attribute get_protected_attributes method and set_protected_attributes method.

© Copyright 1997-2020 DLMS User Association

4.1.5 Common data types

Table 2 contains the list of data types usable for attributes of COSEM objects.

Table 2 – Common data types

Type description	Tag [a]	Definition	Value range
-- simple data types			
null-data	[0]		
boolean	[3]	boolean	TRUE or FALSE
bit-string	[4]	An ordered sequence of boolean values	
double-long	[5]	Integer32	-2 147 483 648... 2 147 483 647
double-long-unsigned	[6]	Unsigned32	0...4 294 967 295
	[7]	Tag of the "floating-point" type in IEC 61334-4-41:1996, not usable in DLMS/COSEM. See tags [23] and [24]	
octet-string	[9]	An ordered sequence of octets (8 bit bytes)	
visible-string	[10]	An ordered sequence of ASCII characters	
	[11]	Tag of the "time" type in IEC 61334-4-41:1996, not usable in DLMS/COSEM. See tag [27]	
utf8-string	[12]	An ordered sequence of characters encoded as UTF-8	
bcd	[13]	binary coded decimal	
integer	[15]	Integer8	-128...127
long	[16]	Integer16	-32 768...32 767
unsigned	[17]	Unsigned8	0...255
long-unsigned	[18]	Unsigned16	0...65 535
long64	[20]	Integer64	$-2^{63}...2^{63}-1$
long64-unsigned	[21]	Unsigned64	$0...2^{64}-1$
enum	[22]	The elements of the enumeration type are defined in the *Attribute description* or *Method description* section of a COSEM IC specification.	0...255
float32	[23]	OCTET STRING (SIZE(4))	For formatting, see 4.1.6.2.
float64	[24]	OCTET STRING (SIZE(8))	
date-time[b]	[25]	OCTET STRING SIZE(12))	For formatting, see 4.1.6.1.
date	[26]	OCTET STRING (SIZE(5))	
time	[27]	OCTET STRING (SIZE(4))	
delta-integer	[28]	Integer8	-128...127
delta-long	[29]	Integer16	-32 768...32 767
delta-double-long	[30]	Integer32	-2 147 483 648... 2 147 483 647
delta-unsigned	[31]	Unsigned8	0...255
delta-long-unsigned	[32]	Unsigned16	0...65 535
delta-double-long-unsigned	[33]	Unsigned32	0...4 294 967 295
-- complex data types			
array	[1]	The elements of the array are defined in the *Attribute* or *Method description* section of a COSEM IC specification.	
structure	[2]	The elements of the structure are defined in the *Attribute or Method description* section of a COSEM IC specification.	
Compact array	[19]	Provides an alternative, compact encoding of complex data.	

© Copyright 1997-2020 DLMS User Association

Type description	Tag [a]	Definition	Value range
-- CHOICE		For some COSEM interface objects attributes, the data type may be chosen at instantiation, in the implementation phase of the COSEM server. The server always shall send back the data type and the value of each attribute, so that together with the logical name an unambiguous interpretation is ensured. The list of possible data types is defined in the “Attribute description” section of a COSEM IC specification.	
[a] The tags are as defined in DLMS UA 1000-2 Ed. 10, 9.5.			
[b] The type-description for date-time has been harmonised as date-time throughout the document.			

4.1.6 Data formats

4.1.6.1 Date and time formats

Date and time information may be represented using the data type *octet-string*.

NOTE 1 In this case the encoding includes the tag of the data type *octet-string*, the length of the octet-string and the elements of *date*, *time* and /or *date-time* as applicable.

Date and time information may be also represented using the data types *date, time* and *date-time*.

NOTE 2 In these cases, the encoding includes only the tag of the data types date, time or date-time as applicable and the elements of date, time or date-time.

NOTE 3 The (SIZE ( )) specifications are applicable only when date, time or date time are represented by the data types date, time or date-time.

date	OCTET STRING (SIZE(5))
	{
	year highbyte,
	year lowbyte,
	month,
	day of month,
	day of week
	}
	Where:
	year: interpreted as long-unsigned
	range 0…big
	0xFFFF = not specified
	year highbyte and year lowbyte represent the 2 bytes of the long-unsigned
	
	month: interpreted as unsigned
	range 1…12, 0xFD, 0xFE, 0xFF
	1 is January
	0xFD = daylight_savings_end
	0xFE = daylight_savings_begin
	0xFF = not specified
	
	dayOfMonth: interpreted as unsigned
	range 1…31, 0xFD, 0xFE, 0xFF
	0xFD = 2nd last day of month
	0xFE = last day of month
	0xE0 to 0xFC = reserved
	0xFF = not specified
	
	dayOfWeek: interpreted as unsigned
	range 1…7, 0xFF
	1 is Monday
	0xFF = not specified
	
	For repetitive dates, the unused parts shall be set to “not specified”.

© Copyright 1997-2020 DLMS User Association

	For countries not using the Gregorian calendar, Month 1 is the starting month of the calendar and the range of dayOfMonth may be different.

The elements dayOfMonth and dayOfWeek shall be interpreted together:

- if last dayOfMonth is specified (0xFE) and dayOfWeek is wildcard, this specifies the last calendar day of the month;
- if last dayOfMonth is specified (0xFE) and an explicit dayOfWeek is specified (for example 7, Sunday) then it is the last occurrence of the weekday specified in the month, i.e. the last Sunday;
- if the year is not specified (0xFFFF), and dayOfMonth and dayOfWeek are both explicitly specified, this shall be interpreted as the dayOfWeek on, or following dayOfMonth;
- if the year and month are specified, and both the dayOfMonth and dayOfWeek are explicitly specified but the values are not consistent it shall be considered as an error.

Examples:

1) year = 0xFFFF, month =0x FF, dayOfMonth = 0xFE, dayofWeek = 0xFF: last day of the month in every year and month;
2) year = 0xFFFF, month =0x FF, dayOfMonth = 0xFE, dayofWeek = 0x07: last Sunday in every year and month;
3) year = 0xFFFF, month = 0x03, dayOfMonth = 0xFE, dayofWeek = 0x07: last Sunday in March in every year;
4) year = 0xFFFF, month = 0x03, dayOfMonth = 0x01, dayofWeek = 0x07: first Sunday in March in every year;
5) year = 0xFFFF, month = 0x03, dayOfMonth = 0x16, dayofWeek = 0x05: fourth Friday in March in every year;
6) year = 0xFFFF, month = 0x0A, dayOfMonth = 0x16, dayofWeek = 0x07: fourth Sunday in October in every year;
7) year = 0x07DE, month = 0x08, dayOfMonth = 0x13, (2014.08.13, Wednesday) dayofWeek = 0x02 (Tuesday): error, as the dayOfMonth and dayOfWeek in the given year and month do not match.

time	OCTET STRING (SIZE(4))
	{
	hour,
	minute,
	second,
	hundredths
	}
	Where:
	hour: interpreted as unsigned range0…23, 0xFF,
	minute: interpreted as unsigned range0…59, 0xFF,
	second: interpreted as unsigned range0…59, 0xFF,
	hundredths: interpreted as unsigned range0…99, 0xFF
	
	For hour, minute, second and hundredths: 0xFF = not specified.
	
	For repetitive times the unused parts shall be set to “not specified”.

date-time	OCTET STRING (SIZE(12)) { year highbyte, year lowbyte, month, day of month, day of week, hour, minute, second, hundredths of second, deviation highbyte,

© Copyright 1997-2020 DLMS User Association

	deviation lowbyte, clock status } The elements of *date* and *time* are encoded as defined above. Some may be set to “not specified” as defined above. In addition: deviation: interpreted as long range -720…+720 in minutes of local time to UTC 0x8000 = not specified Deviation highbyte and deviation lowbyte represent the 2 bytes of the long. Clock_status interpreted as unsigned. The bits are defined as follows: bit 0 (LSB): invalid [a] value, bit 1: doubtful [b] value, bit 2: different clock base [c], bit 3: invalid clock status [d], bit 4: reserved, bit 5: reserved, bit 6: reserved, bit 7 (MSB): daylight saving active [e] 0xFF = not specified

[a] Time could not be recovered after an incident. Detailed conditions are manufacturer specific (for example after the power to the clock has been interrupted). For a valid status, bit 0 shall not be set if bit 1 is set.

[b] Time could be recovered after an incident but the value cannot be guaranteed. Detailed conditions are manufacturer specific. For a valid status, bit 1 shall not be set if bit 0 is set.

[c] Bit is set if the basic timing information for the clock at the actual moment is taken from a timing source different from the source specified in clock_base.

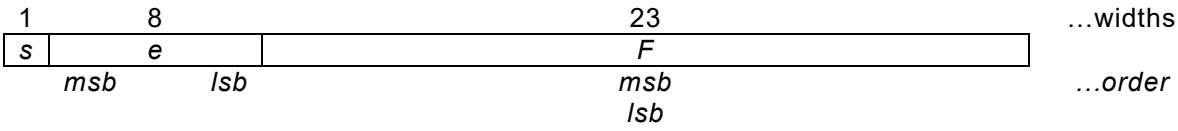
[d] This bit indicates that at least one bit of the clock status is invalid. Some bits may be correct. The exact meaning shall be explained in the manufacturer’s documentation.

[e] Flag set to true: the transmitted time contains the daylight saving deviation (summer time).
Flag set to false: the transmitted time does not contain daylight saving deviation (normal time).

4.1.6.2 Floating point number formats

Floating point number formats are defined in ISO/IEC/IEEE 60559:2011.

The single format is:



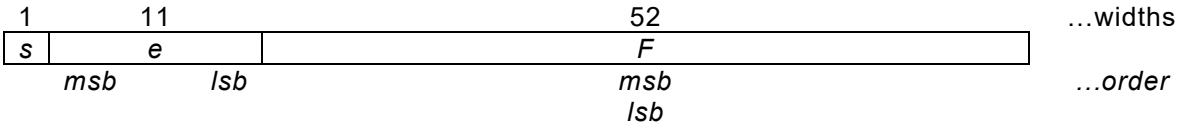
Where:
- *s* is the sign bit;
- *e* is the exponent; it is 8 bits wide and the exponent bias is +127;
- *f* is the fraction, it is 23 bits.

With this, the value is (if 0 < e < 255):

$$v = (-1)^s \cdot 2^{e-127} \cdot (1.f)$$

The double format is:

© Copyright 1997-2020 DLMS User Association



Where:
- *s* is the sign bit;
- *e* is the exponent; it is 11 bits wide and the exponent bias is +1 023;
- *f* is the fraction, it is 52 bits.

With this, the value is (if 0 < e < 2 047):

$$v = (-1)^s \cdot 2^{e-1023} \cdot (1.f)$$

For details, see ISO/IEC/IEEE 60559:2011.

Floating-point numbers shall be represented as a fixed length octet-string, containing the 4 bytes (float32) of the single format or the 8 bytes (float64) of the double format floating-point number as specified above, most significant byte first.

EXAMPLE 1 The decimal value “1” represented in single floating-point format is:

Bit 31 Sign bit **0** 0: + 1: -	Bits 30-23 Exponent field: **01111111** Decimal value of exponent field and exponent: **127**-127 = 0	Bits 22-0 Significand 1.**00000000000000000000000** Decimal value of the significand: 1.0000000

NOTE The significand is the binary number 1 followed by the radix point followed by the binary bits of the fraction.

The encoding, including the tag of the data type is (all values are hexadecimal): 17 3F 80 00 00.

EXAMPLE 2 The decimal value “1” represented in double floating-point format is:

Bit 63 Sign bit **0** 0: + 1: -	Bits 62-52 Exponent field: **01111111111** Decimal value of exponent field and exponent: **1023**-1023 = 0	Bits 51-0 Significand 1.**0000000000000000000000000000000000000000000000000000** Decimal value of the significand: 1.0000000000000000

The encoding, including the tag of the data type is (all values are hexadecimal): 18 3F F0 00 00 00 00 00 00.

EXAMPLE 3 The decimal value “62056” represented in single floating-point format is:

Bit 31 Sign bit **0** 0: + 1: -	Bits 30-23 Exponent field: **10001110** Decimal value of exponent field and exponent: **142**-127 = 15	Bits 22-0 Significand 1.**11100100110100000000000** Decimal value of the significand: 1.8937988

The encoding, including the tag of the data type is (all values are hexadecimal): 17 47 72 68 00.

EXAMPLE 4 The decimal value “62056” represented in double floating-point format is:

Bit 63 Sign bit **0** 0: + 1: -	Bits 62-52 Exponent field: **10000001110** Decimal value of exponent field and exponent: **1038**-1023 = 15	Bits 51-0 Significand 1.**1110010011010000000000000000000000000000000000000000** Decimal value of the significand: 1.8937988281250000

The encoding, including the tag of the data type is (all values are hexadecimal): 18 40 EE 4D 00 00 00 00 00.

© Copyright 1997-2020 DLMS User Association

4.1.6.3 Null- data and delta-value encoding

Null-data encoding and delta-value encoding are two methods to improve efficiency when array or arrays of structures have to be transferred, for example Profile generic IC buffer attribute or Register table IC table_cell_values attribute.

With null-data encoding, the value may be replaced by “null-data” if it can be unambiguously recovered from the previous value (for example for time: if it can be calculated from the previous value and capture_period; or for a value: if it is equal to the previous value).

With delta-value encoding the original value may be replaced by a delta-value as follows:

- the first value shall be encoded using an appropriate integer or unsigned data type;
- subsequent values shall be encoded the same way, or they may be replaced by a delta value encoded using an appropriate delta data type.

The two methods can be used individually or combined and they can be used with compression to further improve efficiency.

4.1.7 The COSEM server model

The COSEM server is structured into three hierarchical levels as shown in Figure 4:

Level 1: Physical device
Level 2: Logical device
Level 3: Accessible COSEM objects

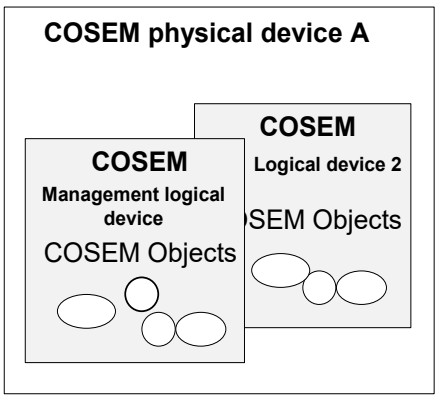


Figure 4 – The COSEM server model

The example in Figure 5 shows how a combined metering device can be structured using the COSEM server model.

© Copyright 1997-2020 DLMS User Association

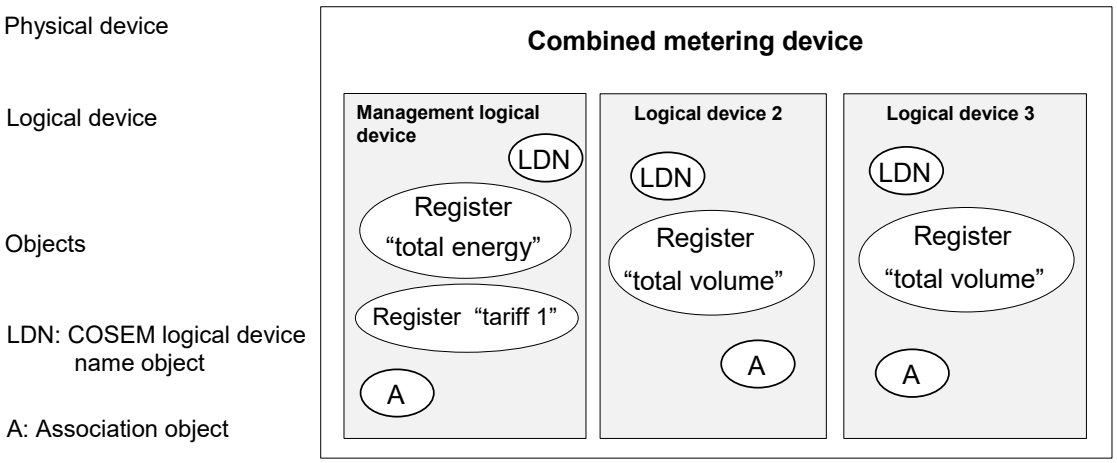


Figure 5 – Combined metering device

4.1.8 COSEM logical device

4.1.8.1 General

A COSEM logical device contains a set of COSEM objects. Each physical device shall contain a "Management logical device".

The addressing of COSEM logical devices shall be provided by the addressing scheme of the lower layers of the protocol stack used.

4.1.8.2 COSEM logical device name (LDN)

Each COSEM logical device can be identified by its unique COSEM LDN. The LDN is defined as a string of up to 16 octets. For the data types see 4.4.5 and 6.2.35.

The first three octets shall carry the manufacturer identifier [1]. The manufacturer shall ensure that the LDN, starting with the three octets identifying the manufacturer and followed by up to 13 octets, is unique for each and every LD manufactured.

4.1.8.3 The "association view" of the logical device

In order to access COSEM objects in the server, an application association (AA) shall first be established with a client. AAs identify the partners and characterize the context within which the associated server and client applications will communicate. The major parts of this context are:

- the application context;
- in the case of explicitly established AAs, the authentication mechanism;
- the xDLMS context.

AAs are modelled by special COSEM objects:

- instances of the IC "Association SN" – see 4.4.3 – are used with short name referencing;
- instances of the IC "Association LN" – see 4.4.4 – are used with logical name referencing.

Depending on the AA established between the client and the server, different access rights may be granted by the server. Access rights concern a set of COSEM objects – the visible objects – that can be accessed ('seen') within the given AA. In addition, access to attributes and methods of these COSEM objects may also be restricted within the AA (for example a certain

[1] Administered by the DLMS User Association, in cooperation with the FLAG Association.

© Copyright 1997-2020 DLMS User Association

type of client can only read a particular attribute of a COSEM object, but cannot write it). Access right may also stipulate required cryptographic protection.

The list of the visible COSEM objects – the “association view” – can be obtained by the client by reading the *object_list* attribute of the appropriate association object.

4.1.8.4 Mandatory contents of a COSEM logical device

The following objects shall be present in each COSEM logical device. They shall be accessible for GET/Read in all AAs with this logical device:

- COSEM logical device name object;
- current “Association” (LN or SN) object.

If the “SAP Assignment” object is present, then the COSEM logical device name object does not have to be present.

For identifying the firmware the following objects are mandatory:

- an active firmware identifier object that holds the identifier of the currently active firmware;
- an active firmware signature object that holds the digital signature of the currently active firmware.

  Note : The digital signature algorithm is not specified here.

If a Logical Device has multiple firmwares then an active firmware identifier object and an active firmware signature object shall be present for each firmware.

The following objects may be optionally present:

- one or more active firmware version object(s) that hold(s) the version of the currently active firmware.

4.1.8.5 Management logical device

As specified in 4.1.8.1, the management logical device is a mandatory element of any physical device. It has a reserved address. It shall support an AA to a public client with the lowest level security (no security) authentication. Its role is to support revealing the internal structure of the physical device and to support notification of events in the server.

In addition to the “Association” object modelling the AA with the public client, the management logical device shall contain a “SAP assignment” object, giving its Service Access Point (SAP) and the SAPs of all other logical devices within the physical device. The SAP assignment object shall be readable at least by the public client.

If there is only one logical device within the physical device, the “SAP assignment” object may be omitted.

4.1.9 Information security

DLMS/COSEM provides several information security features for accessing and transporting data:

- data access security controls access rights to the data held by a DLMS/COSEM server;
- data transport security allows the sending party to apply cryptographic protection to the xDLMS APDUs sent. This requires ciphered APDUs. The receiving party can check or remove this protection;
- COSEM data security allows protecting COSEM attribute values, as well as method invocation and return parameters.

On the COSEM object level, information security is supported / managed by the following objects:

© Copyright 1997-2020 DLMS User Association

- “Association SN”, see 4.4.3;
- “Association LN”, see 4.4.4;
- “Security setup”, see 4.4.7; and
- “Data protection”, see 4.4.9.

Information security on the DLMS/COSEM Application layer level is described in DLMS UA 1000-2 Ed. 10, 9.2.

4.2 Overview of the COSEM interface classes

The ICs defined currently and the relations between them are shown in Figure 6, Figure 7 and Figure 8.

NOTE 1 The IC “base” itself is not specified explicitly. It contains only one attribute *logical_name*.

NOTE 2 In the description of the “Demand register”, “Clock” and “Profile generic” ICs, the 2nd attributes are labelled differently from that of the 2nd attribute of the “Data” IC, namely *current_average_value*, *time* and *buffer* vs. *value*. This is to emphasize the specific nature of the *value*.

NOTE 3 On these Figures the interface classes are presented in each group by increasing class_id. In the clauses specifying the various groups of interface classes, the new interface classes are put at the end of the relevant clause.

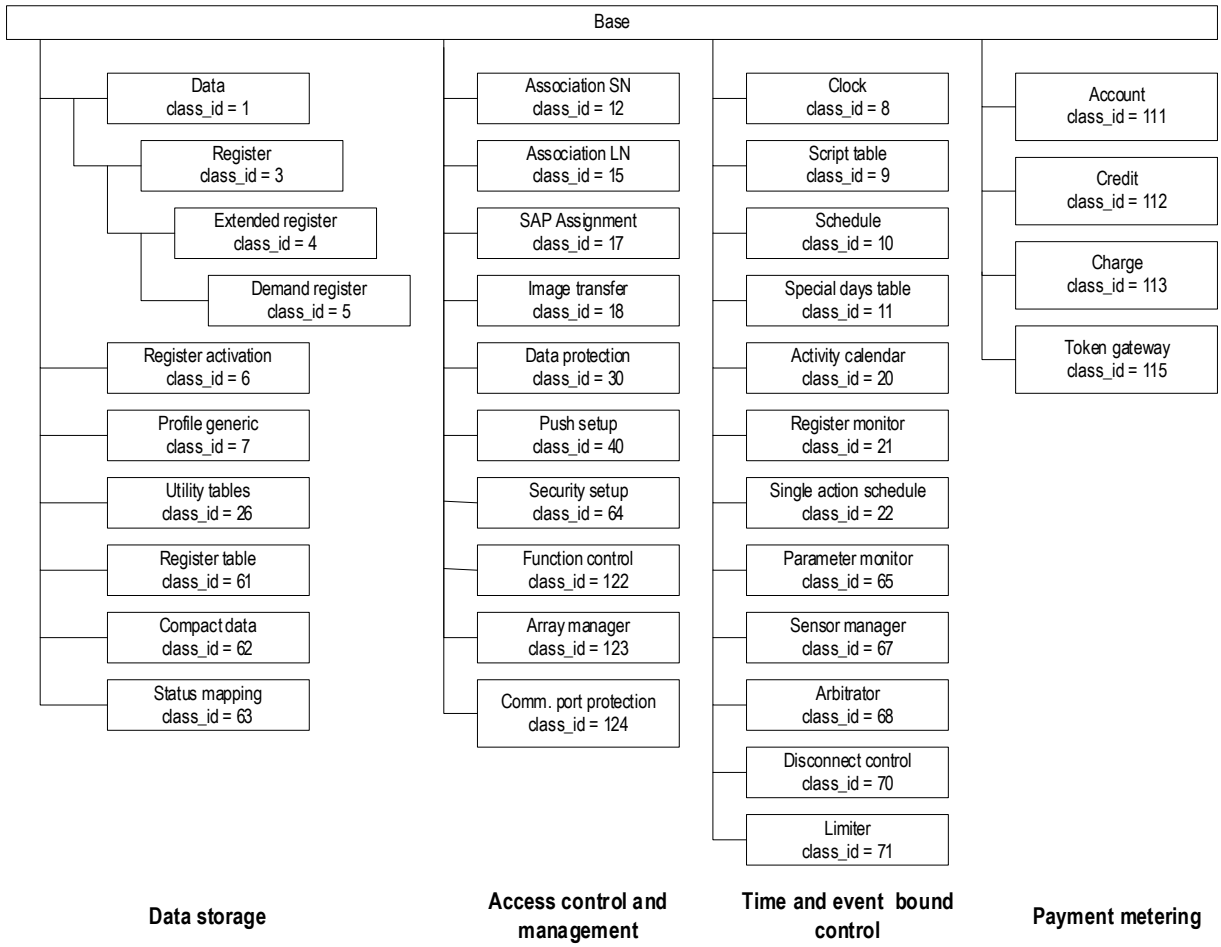


Figure 6 – Overview of the interface classes – Part 1

© Copyright 1997-2020 DLMS User Association

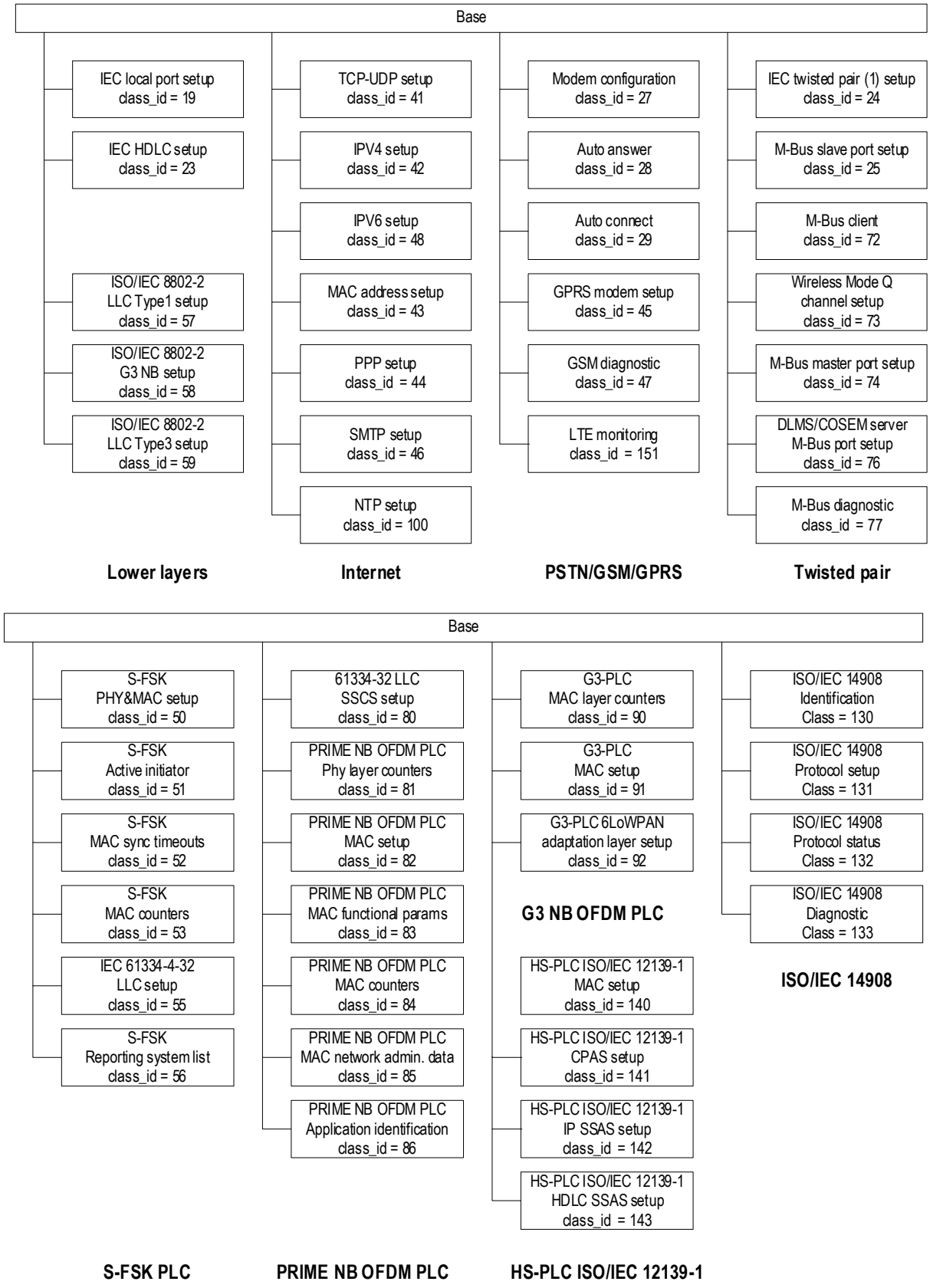


Figure 7 – Overview of the interface classes – Part 2

© Copyright 1997-2020 DLMS User Association

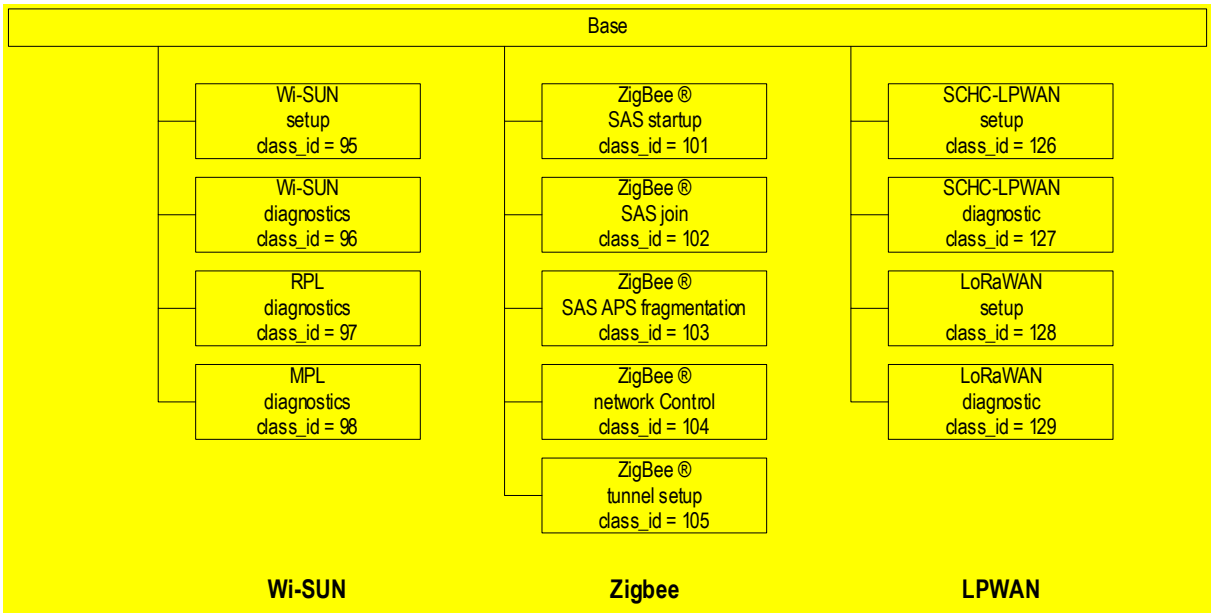


Figure 8 – Overview of the interface classes – Part 3

Table 3 lists the interface classes by class_id.

Table 3 – List of interface classes by class_id

Interface class name	class_id	version(s)	Clause
Data	1	0	4.3.1
Register	3	0	4.3.2
Extended register	4	0	4.3.3
Demand register	5	0	4.3.4
Register activation	6	0	4.3.5
Profile generic	7	1 0	4.3.6 5.3.1
Clock	8	0	4.5.1
Script table	9	0	4.5.2
Schedule	10	0	4.5.3
Special days table	11	0	4.5.4
Association SN	12	4 3 2 1 0	4.4.3 5.4.4 5.4.3 5.4.2 5.4.1
Association LN	15	3 2 1 0	4.4.4 5.4.7 5.4.6 5.4.5
SAP Assignment	17	0	4.4.5

© Copyright 1997-2020 DLMS User Association

Interface class name	class_id	version(s)	Clause
Image transfer	18	0	4.4.6
IEC local port setup	19	1 0	4.7.1 5.7.1
Activity calendar	20	0	4.5.5
Register monitor	21	0	4.5.6
Single action schedule	22	0	4.5.7
IEC HDLC setup	23	1 0	4.7.2 5.7.2
IEC twisted pair (1) setup	24	1 0	4.7.3 5.7.3
M-BUS slave port setup	25	0	4.8.2
Utility tables	26	0	4.3.7
Modem configuration PSTN modem configuration	27	1 0	4.7.4 5.7.4
Auto answer Note The use of version 1 is not allowed.	28	2 1 0	4.7.5 – 5.7.5
Auto connect PSTN Auto dial	29	2 1 0	4.7.6 5.7.7 5.7.6
Data protection	30	0	4.4.9
Push setup	40	2 1 0	4.4.8 5.4.10 5.4.9
TCP-UDP setup	41	0	4.9.1
IPv4 setup	42	0	4.9.2
MAC address setup (Ethernet setup)	43	0	4.9.4, 4.12.10
PPP setup	44	0	4.9.5
GPRS modem setup	45	0	4.7.7
SMTP setup	46	0	4.9.6
GSM diagnostic	47	2 1 0	4.7.8 5.7.8 5.7.8
IPv6 setup	48	0	4.9.3
S-FSK Phy&MAC setup	50	1 0	4.10.3 5.10.1
S-FSK Active initiator	51	0	4.10.4
S-FSK MAC synchronization timeouts	52	0	4.10.5
S-FSK MAC counters	53	0	4.10.6
IEC 61334-4-32 LLC setup S-FSK IEC 61334-4-32 LLC setup	55	1 0	4.10.7 5.10.2
S-FSK Reporting system list	56	0	4.10.8
ISO/IEC 8802-2 LLC Type 1 setup	57	0	4.11.2
ISO/IEC 8802-2 LLC Type 2 setup	58	0	4.11.3

© Copyright 1997-2020 DLMS User Association

Interface class name	class_id	version(s)	Clause
ISO/IEC 8802-2 LLC Type 3 setup	59	0	4.11.4
Register table	61	0	4.3.8
Compact data	62	1 0	4.3.10 5.3.2
Status mapping	63	0	4.3.9
Security setup	64	1 0	4.4.7 5.4.8
Parameter monitor	65	1 0	4.5.10 5.5.1
Sensor manager	67	0	4.5.11
Arbitrator	68	0	4.5.12.2
Disconnect control	70	0	4.5.8
Limiter	71	0	4.5.9
M-Bus client	72	1 0	4.8.3 5.8.1
Wireless Mode Q channel	73	0	4.8.4
M-Bus master port setup	74	0	4.8.5
DLMS/COSEM server M-Bus port setup	76	0	4.8.6
M-Bus diagnostic	77	0	4.8.7
61334-4-32 LLC SSCS setup	80	0	4.12.3
PRIME NB OFDM PLC Physical layer counters	81	0	4.12.5
PRIME NB OFDM PLC MAC setup	82	0	4.12.6
PRIME NB OFDM PLC MAC functional parameters	83	0	4.12.7
PRIME NB OFDM PLC MAC counters	84	0	4.12.8
PRIME NB OFDM PLC MAC network administration data	85	0	4.12.9
PRIME NB OFDM PLC Application identification	86	0	4.12.11
G3-PLC MAC layer counters G3 NB OFDM PLC MAC layer counters	90	1 0	4.13.3 5.13.2
G3-PLC MAC setup G3-PLC MAC setup G3 NB OFDM PLC MAC setup	91	2 1 0	4.13.4 5.13.6 5.13.3
G3-PLC 6LoWPAN adaptation layer setup G3-PLC 6LoWPAN adaptation layer setup G3 NB OFDM PLC 6LoWPAN adaptation layer setup	92	2 1 0	4.13.5 5.13.7 5.13.4
Wi-SUN setup	95	0	4.17.1
Wi-SUN diagnostic	96	0	4.17.2
RPL diagnostic	97	0	4.17.3
MPL diagnostic	98	0	4.17.4
NTP Setup	100	0	4.9.7
ZigBee® SAS startup	101	0	4.15.2
ZigBee® SAS join	102	0	4.15.3
ZigBee® SAS APS fragmentation	103	0	4.15.4
ZigBee® network control	104	0	4.15.5

© Copyright 1997-2020 DLMS User Association

Interface class name	class_id	version(s)	Clause
ZigBee® tunnel setup	105	0	4.15.6
Account	111	0	4.6.2
Credit	112	0	4.6.3.4
Charge	113	0	4.6.4
Token gateway	115	0	4.6.5
Function control	122	0	4.4.10
Array manager	123	0	4.4.11
Communication port protection	124	0	4.4.12
SCHC-LPWAN setup	126	0	4.16.2.1
SCHC-LPWAN diagnostic	127	0	4.16.2.2
LoRaWAN setup	128	0	4.16.3.2.2
LoRaWAN diagnostic	129	0	4.16.3.3
ISO/IEC14908 Identification	130	0	4.18.1
ISO/IEC 14908 Protocol setup	131	0	4.18.3
ISO/IEC 14908 protocol status	132	1	4.18.4
ISO/IEC 14908 diagnostic	133	1	4.18.5
HS-PLC ISO/IEC 12139-1 MAC setup	140	0	4.14.2
HS-PLC ISO/IEC 12139-1 CPAS setup	141	0	4.14.3
HS-PLC ISO/IEC 12139-1 IP SSAS setup	142	0	4.14.4
HS-PLC ISO/IEC 12139-1 HDLC SSAS setup	143	0	4.14.5
LTE monitoring	151	1 0	4.7.9 5.7.10

4.3 Interface classes for parameters and measurement data

4.3.1 Data (class_id = 1, version = 0)

This IC allows modelling various data, such as configuration data and parameters. The data are identified by the attribute *logical_name*.

Data	0...n	class_id = 1, version = 0			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. value	CHOICE				x + 0x08
Specific methods	**m/o**				

Attribute description	
logical_name	Identifies the “Data” object instance. See Clauses 6 and 7.
value	Contains the data. CHOICE { -- simple data types null-data [0], boolean [3], bit-string [4], double-long [5], The data type depends on the instantiation defined by the *logical_name* and possibly from the manufacturer. It has to be chosen so, that together with the *logical_name*, an unambiguous interpretation is possible. Any simple and complex data

© Copyright 1997-2020 DLMS User Association

```
        double-long-unsigned        [6],
        octet-string                [9],
        visible-string              [10],
        utf8-string                 [12],
        bcd                         [13],
        integer                     [15],
        long                        [16],
        unsigned                    [17],
        long-unsigned               [18],
        long64                      [20],
        long64-unsigned             [21],
        enum                        [22],
        float32                     [23],
        float64                     [24],
        date-time                   [25],
        date                        [26],
        time                        [27],
        delta-integer               [28],
        delta-long                  [29],
        delta-double-long           [30],
        delta-unsigned              [31],
        delta-long-unsigned         [32],
        delta-double-long-unsigned  [33],

        -- complex data types
        array                       [1],
        structure                   [2],
        compact-array               [19]
}
```

types listed in 4.1.5 can be used, unless the choice is restricted in Clause 6.

4.3.2 Register (class_id = 3, version = 0)

This IC allows modelling a process or a status value with its associated scaler and unit. “Register” objects know the nature of the process or status value. It is identified by the attribute *logical_name*.

Register	**0...n**	**class_id = 3, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. value	CHOICE				x + 0x08
3. scaler_unit (static)	scal_unit_type				x + 0x10
Specific methods	**m/o**				
1. reset (data)	o				x + 0x28

Attribute description	
logical_name	Identifies the “Register” object instance. See Clauses 6 and 7.
value	Contains the current process or status value.

```
CHOICE
{
        -- simple data types
```

The data type of the value depends on the instantiation defined by *logical_name* and possibly on the

© Copyright 1997-2020 DLMS User Association

null-data	[0],	choice of the manufacturer. It has to be chosen so that, together with the *logical_name*, an unambiguous interpretation of the value is possible. When, instead of a “Data” object, a “Register” object is used, (with the *scaler_unit* attribute not used or with scaler = 0, unit = 255) then the data types allowed for the *value* attribute of the “Data” IC are allowed.
bit-string	[4],	
double-long	[5],	
double-long-unsigned	[6],	
octet-string	[9],	
visible-string	[10],	
utf8-string	[12],	
integer	[15],	
long	[16],	
unsigned	[17],	
long-unsigned	[18],	
long64	[20],	
long64-unsigned	[21],	
enum	[22],	
float32	[23],	
float64	[24].	

}

scaler_unit

Provides information on the unit and the scaler of the value.

scal_unit_type ::= structure

{
scaler,
unit
}

scaler: integer

This is the exponent (to the base of 10) of the multiplication factor.

REMARK If the value is not numerical, then the scaler shall be set to 0.

unit: enum

Enumeration defining the physical unit; for details see Table 4 below.

Method description

reset (data)

Forces a reset of the object. By invoking this method, the value is set to the default value. The default value is an instance specific constant.

data ::= integer (0)

Table 4 – Enumerated values for physical units

unit ::= enum	Unit	Quantity	Unit name	SI definition (comment)
		SI units		
(1)	a	time	year	
(2)	mo	time	month	
(3)	wk	time	week	7*24*60*60 s
(4)	d	time	day	24*60*60 s
(5)	h	time	hour	60*60 s
(6)	min	time	minute	60 s
(7)	s	time (t)	second	s
(8)	°	(phase) angle	degree	rad*180/π
(9)	°C	temperature (T)	degree-celsius	K-273.15
(10)	currency	(local) currency		
(11)	m	length (l)	metre	m
(12)	m/s	speed (v)	metre per second	ms^{-1}

© Copyright 1997-2020 DLMS User Association

unit ::= enum	Unit	Quantity	Unit name	SI definition (comment)
(13)	m^3	volume (V) r_V , meter constant or pulse value (volume)	cubic metre	m^3
(14)	m^3	corrected volume [a]	cubic metre	m^3
(15)	m^3/h	volume flux	cubic metre per hour	$m^3 s^{-1}/(60*60)$
(16)	m^3/h	corrected volume flux [a]	cubic metre per hour	$m^3 s^{-1}/(60*60)$
(17)	m^3/d	volume flux	cubic metre per day	$m^3 s^{-1}/(24*60*60)$
(18)	m^3/d	corrected volume flux [a]	cubic metre per day	$m^3 s^{-1}/(60*60)$
(19)	l	volume	litre	10^{-3} m^3
(20)	kg	mass (m)	kilogram	
(21)	N	force (F)	newton	$N = kg \cdot m \cdot s^{-2}$
(22)	Nm	energy	newton meter	J = Nm = Ws
(23)	Pa	pressure (p)	pascal	N/m^2
(24)	bar	pressure (p)	bar	10^5 Nm^{-2}
(25)	J	energy	joule	J = Nm = Ws
(26)	J/h	thermal power, rate of change	joule per hour	$Js^{-1}/(60*60)$
(27)	W	active power (P)	watt	$W = Js^{-1}$
(28)	VA	apparent power (S)	volt-ampere	
(29)	var	reactive power (Q)	var	
(30)	Wh	active energy r_W , active energy meter constant or pulse value	watt-hour	Ws*(60*60)
(31)	VAh	apparent energy r_S , apparent energy meter constant or pulse value	volt-ampere-hour	VAs*(60*60)
(32)	varh	reactive energy r_B , reactive energy meter constant or pulse value	var-hour	var s *(60*60s
(33)	A	current (I)	ampere	A
(34)	C	electrical charge (Q)	coulomb	C = As
(35)	V	voltage (U)	volt	V
(36)	V/m	electric field strength (E)	volt per metre	Vm^{-1}
(37)	F	capacitance (C)	farad	$CV^{-1} = AsV^{-1}$
(38)	Ω	resistance (R)	ohm	$\Omega = VA^{-1}$
(39)	$\Omega m^2/m$	resistivity (ρ)		Ωm
(40)	Wb	magnetic flux (Φ)	weber	Wb = Vs
(41)	T	magnetic flux density (B)	tesla	Wbm^{-2}
(42)	A/m	magnetic field strength (H)	ampere per metre	Am^{-1}
(43)	H	inductance (L)	henry	$H = WbA^{-1}$
(44)	Hz	frequency (f, ω)	hertz	s^{-1}
(45)	1/(Wh)	R_W , active energy meter constant or pulse value		
(46)	1/(varh)	R_B , reactive energy meter constant or pulse value		
(47)	1/(VAh)	R_S , apparent energy meter constant or pulse value		
(48)	V^2h	volt-squared hour, r_{U2h} , volt-squared hour meter constant or pulse value	volt-squared-hours	$V^2 s^{-1}$ /(60*60)
(49)	A^2h	ampere-squared hour, r_{I2h} , ampere-squared hour meter constant or pulse value	ampere-squared-hours	A^2 s^{-1} /(60*60)
(50)	kg/s	mass flux	kilogram per second	$kg\ s^{-1}$
(51)	S, mho	conductance	siemens	Ω^{-1}
(52)	K	temperature (T)	kelvin	

© Copyright 1997-2020 DLMS User Association

unit ::= enum	Unit	Quantity	Unit name	SI definition (comment)
(53)	$1/(V^2h)$	R_{U2h} , volt-squared hour meter constant or pulse value		
(54)	$1/(A^2h)$	R_{I2h} , ampere-squared hour meter constant or pulse value		
(55)	$1/m^3$	R_V , meter constant or pulse value (volume)		
(56)		percentage	%	
(57)	Ah	ampere-hours	Ampere-hour	
(58),(59)		reserved		
(60)	Wh/m^3	energy per volume	$3,6*10^3$ J/m^3	
(61)	J/m^3	calorific value, wobbe		
(62)	Mol %	molar fraction of gas composition	mole percent	(Basic gas composition unit)
(63)	g/m^3	mass density, quantity of material		(Gas analysis, accompanying elements)
(64)	Pa s	dynamic viscosity	pascal second	(Characteristic of gas stream)
(65)	J/kg	specific energy NOTE The amount of energy per unit of mass of a substance	Joule / kilogram	$m^2 \cdot kg \cdot s^{-2} / kg = m^2 \cdot s^{-2}$
(66)	g/cm^2	pressure	gram per square centimeter	98,066 5 Pa
(67)	atm	pressure	atmosphere	$101,325*10^3$ Pa
(68),(69)		reserved		
(70)	dBm	signal strength, dB milliwatt (e.g. of GSM radio systems)		
(71)	dbµV	signal strength, dB microvolt		
(72)	dB	logarithmic unit that expresses the ratio between two values of a physical quantity		
(73)..(127)		reserved		
		Non – SI Units		
(128)	in	length (l)	inch	
(129)	ft	length (l)	foot	
(130)	lb	mass (m)	pound	
(131)	°F	temperature	degree Fahrenheit	
(132)	°R	temperature	degree Rankine	
(133)	sq. in	area	square inch	
(134)	sq ft	area	square foot	
(135)	ac	area	acre	
(136)	cu in	volume	cubic inch	
(137)	cu ft	volume	cubic foot	
(138)	ac ft	volume	acre-foot	
(139)	gal (imp)	volume	gallon (imperial)	
(140)	gal (US)	volume	gallon (US)	
(141)	lbf	force	pound force	
(142)	psi	pressure (p)	Pound force per square inch	
(143)	lb/cu ft	density	pound per cubic foot	

© Copyright 1997-2020 DLMS User Association

unit ::= enum	Unit	Quantity	Unit name	SI definition (comment)
(144)	lb/(ft .s)	dynamic viscosity	pound per (foot . second)	
(145)	sq ft/s	kinematic viscosity	square foot per second	
(146)	Btu	energy	British thermal unit	
(147)	thm(EC)	energy	Therm(EU)	
(148)	thm(US)	energy	Therm(US)	
(149)	Btu/lb	calorific value of mass, enthalpy	British thermal unit per pound	
(150)	Btu/cu ft	calorific value of volume, wobbe	British thermal unit per cubic foot	
(151)	cu ft	volume (V) rv, meter constant or pulse value (volume)	cubic feet	
(152)	ft/s	speed (v)	foot per second	
(153)	cu ft/s	volume flux	cubic foot per second	
(154)	cu ft/min	volume flux	cubic foot per min	
(155)	cu ft/h	volume flux	cubic foot per hour	
(156)	cu ft/d	volume flux	cubic foot per day	
(157)	ac ft/s	volume flux	acre foot per second	
(158)	ac ft/min	volume flux	acre foot per min	
(159)	ac ft/h	volume flux	acre foot per hour	
(160)	ac ft/d	volume flux	acre foot per day	
(161)	gal (imp)	volume (V) rv, meter constant or pulse value (volume)	imperial gallon	
(162)	gal (imp) / s	volume flux	imperial gallon per second	
(163)	gal (imp) / min	volume flux	imperial gallon per min	
(164)	gal (imp) / h	volume flux	imperial gallon per hour	
(165)	gal (imp) / d	volume flux	imperial gallon per day	
(166)	gal (US)	volume (V) rv, meter constant or pulse value (volume)	US gallon	
(167)	gal (US) / s	volume flux	US gallon per second	
(168)	gal (US) / min	volume flux	US gallon per min	
(169)	gal (US) / h	volume flux	US gallon per hour	
(170)	gal (US) / d	volume flux	US gallon per day	
(171)	Btu/s	energy flow, heat, power, change rate	British thermal unit per second	
(172)	Btu/min	energy flow, heat, power, change rate	British thermal unit per minute	
(173)	Btu/h	energy flow, heat, power, change rate	British thermal unit per hour	
(174)	Btu/d	energy flow, heat, power, change rate	British thermal unit per day	
(175) .. (252)		reserved		
(253)		extended table of units		

© Copyright 1997-2020 DLMS User Association

unit ::= enum	Unit	Quantity	Unit name	SI definition (comment)
(254)	other	other unit		
(255)	count	no unit, unitless, count		
[a] Usage of these units (16 & 18) is deprecated as OBIS codes specify a corrected volume (flux). This is to avoid contradiction of units associated with selected OBIS codes.				

Some examples are shown in Table 5 below.

Table 5 – Examples for scaler_unit

Value	Scaler	Unit	Data
263788	-3	m^3	263,788 m^3
593	3	Wh	593 kWh
3467	-1	V	346,7
3467	0	V	3467 V
3467	1	V	34 670 V

4.3.3 Extended register (class_id = 4, version = 0)

This IC allows modelling a process value with its associated scaler, unit, status and capture time information. “Extended Register” IC objects know the nature of the process value. It is described by the attribute *logical_name*.

Extended register		**0...n**	**class_id = 4, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. value	(dyn.)	CHOICE				x + 0x08
3. scaler_unit	(static)	scal_unit_type				x + 0x10
4. status	(dyn.)	CHOICE				x + 0x18
5. capture_time	(dyn.)	octet-string				x + 0x20
Specific methods		***m/o***				
1. reset (data)		o				x + 0x38

Attribute description	
logical_name	Identifies the “Extended register” object instance. See 6.3.1.
value	See the specification of the “Register” IC in 4.3.2.
scaler_unit	See the specification of the “Register” IC in 4.3.2.
status	Provides “Extended Register” IC specific status information. The meaning of the elements of the status shall be provided for each object instance. CHOICE { -- simple data types null-data [0], bit-string [4], double-long-unsigned [6], The data type and the encoding depend on the instantiation and possibly on the choice of the manufacturer. For the interpretation, extra information from the manufacturer may be necessary.

© Copyright 1997-2020 DLMS User Association

	octet-string [9], visible-string [10], utf8-string [12], unsigned [17], long-unsigned [18], long64-unsigned [21]. }
	Def. Depending on the status type definition.
capture_time	Provides an “Extended Register” IC specific date and time information showing when the value of the attribute *value* has been captured. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
Method description	
reset (data)	This method forces a reset of the object. By invoking this method, the attribute *value* is set to the default value. The default value is an instance specific constant. The attribute *capture_time* is set to the time of the reset execution. data ::= integer (0)

4.3.4 Demand register (class_id = 5, version = 0)

This IC allows modelling a demand value with its associated scaler, unit, status and time information. A “Demand register” object measures and computes a *current_average_value* periodically and it stores a *last_average_value.* The time interval *T* over which the demand is calculated is defined by specifying *number_of_periods* and *period*. See Figure 8.

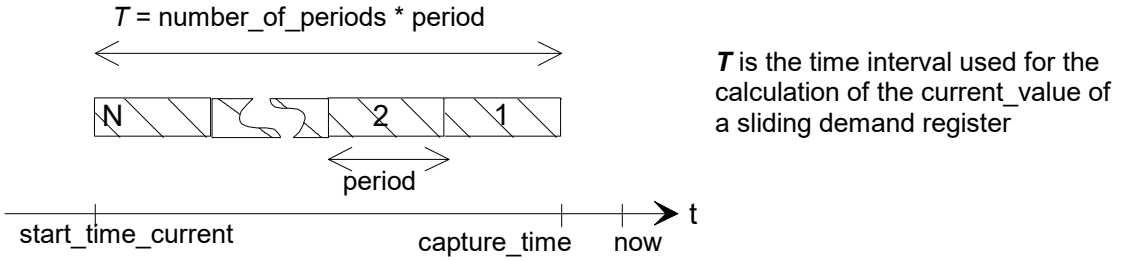


Figure 9 – The time attributes when measuring sliding demand

The demand register delivers two types of demand: *current_average_value* and *last_average_value* (see Figure 9 and Figure 10).

“Demand register” objects know the nature the of process value, which is described by the attribute *logical_name*.

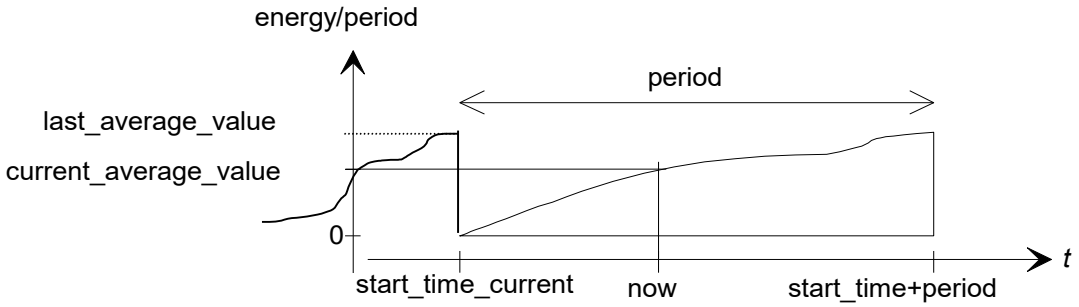


Figure 10 – The attributes in the case of block demand

© Copyright 1997-2020 DLMS User Association

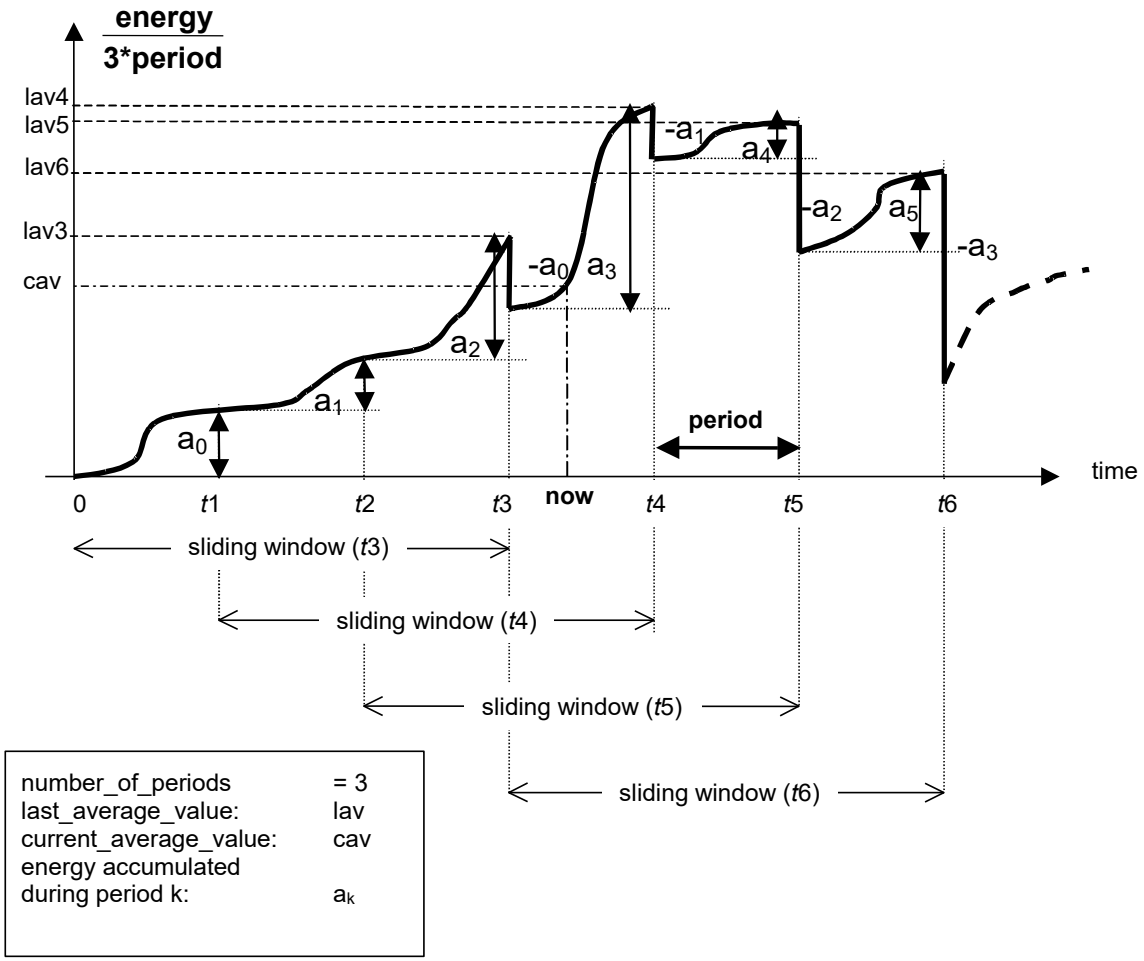


Figure 11 – The attributes in the case of sliding demand (number of periods = 3)

Demand register		**0...n**	**class_id = 5, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. current_average_value	(dyn.)	CHOICE			0	x + 0x08
3. last_average_value	(dyn.)	CHOICE			0	x + 0x10
4. scaler_unit	(static)	scal_unit_type				x + 0x18
5. status	(dyn.)	CHOICE				x + 0x20
6. capture_time	(dyn.)	octet-string				x + 0x28
7. start_time_current	(dyn.)	octet-string				x + 0x30
8. period	(static)	double-long-unsigned	1			x + 0x38
9. number_of_periods	(static)	long-unsigned	1		1	x + 0x40
Specific methods		***m/o***				
1. reset (data)		o				x + 0x48
2. next_period (data)		o				x + 0x50

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “Demand register” object instance. See 6.3.1, 6.4.5, 6.5.5, 6.6.6 and 6.7.5.
current_average_value	Provides the current value (running demand) of the energy accumulated since *start_time*, divided by *number_of_periods*period*. The data type of the value depends on the instantiation defined by *logical_name* and possibly on the choice of the manufacturer. The type has to be chosen so that – together with the *logical_name* – unambiguous interpretation of the value is possible. If a quantity other than energy is measured, other calculation methods may apply (for example for calculating average values of voltage or current). CHOICE For data types, see “Register” IC in 4.3.2, value attribute.
last_average_value	Provides the value of the energy accumulated (over the last *number_of_periods*period*) divided by *number_of_periods*period*. The energy of the current (not terminated) period is not considered by the calculation. If a quantity other than energy is measured, other calculation methods may apply (for example for calculating average values of voltage or current). CHOICE For data types, see “Register” IC in 4.3.2, value attribute.
scaler_unit	See the specification of “Register” IC in 4.3.2, scaler_unit attribute.
status	Provides “Demand register” specific status information. The data type and the encoding depend on the instantiation and possibly on the choice of the manufacturer. For the interpretation, extra information from the manufacturer may be necessary. CHOICE For data types, see “Extended register” IC in 4.3.3, status attribute. *Def.* Depending on the status type definition.
capture_time	Provides the date and time when the *last_average_value* has been calculated. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
start_time_current	Provides the date and time when the measurement of the *current_average_value* has been started. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
period	Period is the interval between two successive updates of the *last_average_value*. (*number_of_periods*period* is the denominator for the calculation of the demand). double-long-unsigned Measuring period in seconds The behaviour of the meter after writing a new value to this attribute shall be specified by the manufacturer.

© Copyright 1997-2020 DLMS User Association

number_of_ periods	The number of periods used to calculate the *last_average_value.* number_of_periods >= 1 - *number_of_periods* > 1 indicates that the *last_average_value* represents “sliding demand”, - *number_of_periods* = 1 indicates that the *last_average_value* represents "block demand". The behaviour of the meter after writing a new value to this attribute shall be specified by the manufacturer.
Method description	
reset (data)	This method forces a reset of the object. Activating this method provokes the following actions: - the current period is terminated; - the *current_average_value* and the *last_average_value* are set to their default values; - the *capture_time* and the *start_time_current* are set to the time of the execution of *reset (data).* data ::= integer (0)
next_period (data)	This method is used to trigger the regular termination (and restart) of a period. Closes (terminates) the current measuring period. Updates *capture_time* and *start_time* and copies *current_average_value* to *last_average_value*, sets *current_average_value* to its default value. Starts the next measuring period. REMARK The old *last_average_value* (and *capture_time*) can be read during the time “period”. The old *current_average_value* is not available any more at the interface. data ::= integer (0)

4.3.5 Register activation (class_id = 6, version = 0)

This IC allows modelling the handling of different tariffication structures. To each “Register activation” object, groups of “Register”, “Extended register” or “Demand register” objects, modelling different kind of quantities (for example active energy, active demand, reactive energy, etc.) are assigned. Subgroups of these registers, defined by the *activation masks* define different tariff structures (for example day tariff, night tariff). One of these activation masks, the *active_mask*, defines which subset of the registers, assigned to the “Register activation” object instance is active. Registers not included in the *register_assignment* attribute of any “Register activation” object are always enabled by default.

Register activation		**0...n**	**class_id = 6, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. register_assignment	(static)	array				x + 0x08
3. mask_list	(static)	array				x + 0x10
4. active_mask	(dyn.)	octet-string				x+ 0x18
Specific methods		***m/o***				
1. add_register (data)		o				x + 0x30
2. add_mask (data)		o				x + 0x38
3. delete_mask (data)		o				x + 0x40

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “Register activation” object instance. See 6.2.11.

register_ assignment	Specifies an ordered list of COSEM objects assigned to the “Register activation” object. The list may contain different kinds of COSEM objects, for example instances of “Register”, “Extended register” or “Demand register”.
	array object_definition
	
	object_definition ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}

mask_list	Specifies a list of register activation masks. Each entry (mask) is identified by its mask_name and contains an array of indices referring to the registers assigned to the mask (the first object in register_assignment is referenced by index 1, the second object by index 2,…).
	array register_act_mask
	
	register_act_mask ::= structure
	{
	mask_name: octet-string,
	index_list: index_array
	}
	mask_name has to be uniquely defined within the object
	index_array ::= array unsigned

active_mask	Defines the currently active mask. The mask is defined by its mask_name (see mask_list). The *active_mask* defines the registers currently enabled; all other registers listed in the *register_assignment* are disabled.
Method description	

add_register (data)	Adds one more registers to the attribute *register_assignment*. The new register is added at the end of the array; i.e. the newly added register has the highest index. The indices of the existing registers are not modified.
	data ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}

add_mask (data)	Adds another mask to the attribute *mask_list*. If there exists already a mask with the same name, the existing mask will be overwritten by the new mask. data ::= register_act_mask (see above)
delete_mask (data)	Deletes a mask from the attribute *mask_list*. The mask is defined by its mask name. data ::= octet-string (mask_name)

© Copyright 1997-2020 DLMS User Association

4.3.6 Profile generic (class_id = 7, version = 1)

This IC provides a generalized concept allowing to store, sort and access data groups or data series, called *capture_objects*. Capture objects are appropriate attributes or elements of (an) attribute(s) of COSEM objects. The capture objects are collected periodically or occasionally.

A profile has a *buffer* to store the captured data. To retrieve only a part of the buffer, either a value range or an entry range may be specified, asking to retrieve all entries that fall within the range specified.

The list of *capture objects* defines the values to be stored in the *buffer* (using auto capture or the method *capture*). The list is defined statically to ensure homogenous buffer entries (all entries have the same size and structure). If the list of capture objects is modified, the *buffer* is cleared. If the buffer is captured by other “Profile generic” objects, their *buffer* is cleared as well, to guarantee the homogeneity of their *buffer* entries.

The *buffer* may be defined as sorted by one of the *capture objects*, e.g. the clock, or the entries are stacked in a “last in first out” order. For example, it is very easy to build a “maximum demand register” with a one entry deep sorted profile capturing and sorted by a “Demand register” *last_average_value* attribute. It is just as simple to define a profile retaining the three largest values of some period.

The size of profile data is determined by three parameters:

a) the number of entries filled (*entries_in_use*). This will be zero after clearing the profile;

b) the maximum number of entries to retain (*profile_entries*). If all entries are filled and a *capture* () request occurs, the least important entry (according to the requested sorting method) will get lost. This maximum number of entries may be specified. Upon changing it, the buffer will be adjusted;

c) the physical limit for the buffer. This limit typically depends on the objects to capture. The object will reject an attempt of setting the maximum number of entries that is larger than physically possible.

Profile generic		**0...n**	**class_id = 7, version = 1**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. buffer	(dyn.)	compact-array or array				x + 0x08
3. capture_objects	(static)	array				x + 0x10
4. capture_period	(static)	double-long-unsigned				x + 0x18
5. sort_method	(static)	enum				x + 0x20
6. sort_object	(static)	capture_object_definition				x + 0x28
7. entries_in_use	(dyn.)	double-long-unsigned	0		0	x + 0x30
8. profile_entries	(static)	double-long-unsigned	1		1	x + 0x38
Specific methods		***m/o***				
1. reset (data)		o				x + 0x58
2. capture (data)		o				x + 0x60
3. *reserved from previous versions*		o				
4. *reserved from previous versions*		o				

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “Profile generic” object instance. For examples, see 6.2.19, 6.2.21, 6.2.42, etc.
buffer	Contains a sequence of entries. Each entry contains values of the captured objects.

compact-array or array entry

```
entry ::= structure
{
    CHOICE
    {
    -- simple data types
        null-data                   [0],
        boolean                     [3],
        bit-string                  [4],
        double-long                 [5],
        double-long-unsigned        [6],
        octet-string                [9],
        visible-string              [10],
        utf8-string                 [12],
        bcd                         [13],
        integer                     [15],
        long                        [16],
        unsigned                    [17],
        long-unsigned               [18],
        long64                      [20],
        long64-unsigned             [21],
        enum                        [22],
        float32                     [23],
        float64                     [24],
        date-time                   [25],
        date                        [26],
        time                        [27],
        delta-integer               [28],
        delta-long                  [29],
        delta-double-long           [30],
        delta-unsigned              [31],
        delta-long-unsigned         [32],
        delta-double-long-unsigned  [33],

        -- complex data types
        array                 [1],
        structure             [2],
        compact-array         [19]
    }
}
```

The number and the order of the elements of the structure holding the entries is the same as in the definition of the *capture_objects*. The *buffer* is filled by auto captures or by subsequent calls of the method *capture*. The sequence of the entries within the array is ordered according to the *sort_method* specified.

Default: The *buffer* is empty after reset.

REMARK 1 Reading the entire *buffer* delivers only those entries, which are “in use”.

© Copyright 1997-2020 DLMS User Association

REMARK 2 The value of a captured object may be replaced by "null-data" if it can be unambiguously recovered from the previous value (for example for time: if it can be calculated from the previous value and *capture_period*; or for a value: if it is equal to the previous value).

selective access (see 4.1.4) to the attribute buffer may be available (optional). The selective access parameters are as defined below.

capture_objects

Specifies the list of capture objects that are assigned to the object instance.

Upon a call of the *capture* (data) method or automatically in defined intervals, the selected attributes are copied into the *buffer* of the profile.

array capture_object_definition

capture_object_definition ::= structure
{
class_id: long-unsigned,
logical_name: octet-string,
attribute_index: integer,
data_index: long-unsigned
}

- where attribute_index is a pointer to the attribute within the object identified by its class_id and *logical_name*. Attribute_index 1 refers to the 1st attribute (i.e. the *logical_name*), attribute_index 2 to the 2nd, etc.); attribute_index 0 refers to all public attributes;
- where data_index is a pointer selecting a specific element of the attribute. The first element in the attribute structure is identified by data_index 1. If the attribute is not a structure, then the data_index has no meaning. If the capture object is the buffer of a profile, then the data_index identifies the captured object of the buffer (i.e. the column) of the inner profile;

  data_index 0: references the whole attribute.

capture_period

>= 1: Automatic capturing assumed. Specifies the capturing period in seconds.

0: No automatic capturing; capturing is triggered externally or capture events occur asynchronously.

sort_method

If the profile is unsorted, it works as a "first in first out" buffer (it is hence sorted by capturing, and not necessarily by the time maintained in the "Clock" object). If the *buffer* is full, the next call to *capture* () will push out the first (oldest) entry of the *buffer* to make space for the new entry.

If the profile is sorted, a call to *capture* () will store the new entry at the appropriate position in the buffer, moving all following entries and probably losing the least interesting entry. If the new entry would enter the *buffer* after the last entry and if the *buffer* is already full, the new entry will not be retained at all.

enum: (1) fifo (first in first out),
(2) lifo (last in first out),
(3) largest,
(4) smallest,
(5) nearest_to_zero,
(6) furthest_from_zero

Def. fifo

sort_object

If the profile is sorted, this attribute specifies the register or clock that the ordering is based upon.

capture_object_definition — See above.

Def. — no object to sort by (only possible with sort_method fifo or lifo)

© Copyright 1997-2020 DLMS User Association

NOTE 1 If the sort_method is FIFO or LIFO, then all elements of the capture_object_definition specifying the sort object can be zero.

entries_in_use	Counts the number of entries stored in the buffer. After a call of the *reset* () method, the buffer does not contain any entries, and this value is zero. Upon each subsequent call of the *capture* () method, this value will be incremented up to the maximum number of entries that will get stored (see *profile_entries*).	
	double-long-unsigned	0…profile_entries
	Def.	0
profile_entries	Specifies how many entries shall be retained in the buffer.	
	double-long-unsigned	1…(limited by physical size)
	Def.	1

Parameters for selective access to the buffer attribute

Access selector	Access parameter	Comment
1	range_descriptor	Only buffer elements corresponding to the range_descriptor shall be returned in the response.
2	entry_descriptor	Only buffer elements corresponding to the entry_descriptor shall be returned in the response.

range_descriptor ::= structure

{

restricting_object:	capture_object_definition	Defines the capture_object restricting the range of entries to be retrieved. Only simple data types are allowed.
from_value:		Oldest or smallest entry to retrieve
	CHOICE	
	{ -- simple data types	
	double-long [5],	
	double-long-unsigned [6],	
	octet-string [9],	
	visible-string [10],	
	utf8-string [12],	
	integer [15],	
	long [16],	
	unsigned [17],	
	long-unsigned [18],	
	long64 [20],	
	long64-unsigned [21],	
	float32 [23],	
	float64 [24],	
	date-time [25],	
	date [26],	
	time [27]	
	}	
to_value:	CHOICE	Newest or largest entry to retrieve
	{see above}	

© Copyright 1997-2020 DLMS User Association

selected_values:	array capture_object_definition	List of columns to retrieve. If the array is empty (has no entries), all captured data are returned. Otherwise, only the columns specified in the array are returned. The type *capture_object_definition* is specified above (*capture_objects*).

}

entry_descriptor ::= structure

{

from_entry:	double-long-unsigned	first entry to retrieve,
to_entry:	double-long-unsigned	last entry to retrieve to_entry == 0: highest possible entry,
from_selected_value:	long-unsigned	index of first value to retrieve,
to_selected_value:	long-unsigned	index of last value to retrieve to_selected_value == 0: highest possible selected_value

}

NOTE 2 from_entry and to_entry identify the lines, from_selected_value to_selected_value identify the columns of the buffer to be retrieved.

NOTE 3 Numbering of entries and selected values starts from 1.

Method description	
reset (data)	Clears the *buffer*. It has no valid entries afterwards; *entries_in_use* is zero after this call. This call does not trigger any additional operations on the capture objects. Specifically, it does not reset any attributes captured. data ::= integer (0)
capture (data)	Copies the values of the objects to capture into the *buffer* by reading each capture object. Depending on the *sort_method* and the actual state of the *buffer* this produces a new entry or a replacement for the less significant entry. As long as not all entries are already used, the *entries_in_use* attribute will be incremented. This call does not trigger any additional operations within the capture objects such as *capture* () or *reset* (). Note, that if more than one attribute of an object need to be captured, they have to be defined one by one on the list of *capture_objects*. If the attribute_index = 0, all attributes are captured. data ::= integer (0)

Behaviour of the object after modification of certain attributes

Any modification of one of the *capture_objects* describing the static structure of the *buffer* will automatically call a *reset* () and this call will propagate to all other profiles capturing this profile.

If writing to *profile_entries* is attempted with a value too large for the buffer, it will be rejected.

Restrictions

When defining the *capture_objects*, circular reference to the profile shall be avoided.

Profile used to define a subset of preferred readout values

© Copyright 1997-2020 DLMS User Association

By setting *profile_entries* to 1, a “Profile generic” object can be used to define a set of preferred readout values. See also 6.2.19. Setting *capture_period* to 1 ensures that the values are updated every second.

4.3.7 Utility tables (class_id = 26, version = 0)

This IC allows encapsulating ANSI C12.19 table data. Each “table” is represented by an instance of this IC, identified by its *logical name*.

Utility tables		**0...n**	**class_id = 26, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. table_ID	(static)	long-unsigned				x + 0x08
3. length		double-long-unsigned				x + 0x10
4. buffer		octet-string				x + 0x18
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “Utility tables” object instance. See 6.2.40.
table_ID	Table number. This table number is as specified in the ANSI standard and may be either a standard table or a manufacturer’s table.
length	Number of octets in table buffer.
buffer	Contents of the table. *Selective access* (see 4.1.4) to the attribute *buffer* may be available (optional). The selective access parameters are defined below.

Parameters for selective access to the buffer attribute

Access selector	Parameter	Comment
1	offset_access	Access to table by offset and count using offset_selector for parameter data.
2	index_access	Access to table by element id and number of elements using index_selector for parameter data.

offset_selector ::= structure
{

Offset:	double-long-unsigned	Offset in octets to the start of access area, relative to the start of the table.
Count:	long-unsigned	Number of octets requested or transferred

}

index_selector ::= structure
{

Index:	array long-unsigned	Sequence of indices to identify elements within the table’s hierarchy.
Count:	long-unsigned	Number of elements requested or transferred. Values of count greater than 1 return up to that many elements. A value of zero,

© Copyright 1997-2020 DLMS User Association

when given in the context of a request, refers to the entire sub-tree of the hierarchy starting at the selection point.

}

4.3.8 Register table (class_id = 61, version = 0)

This IC allows to group homogenous entries, identical attributes of multiple objects, which are all instances of the same IC, and in their *logical_name* (OBIS code) the value in value groups A to D and F is identical. The possible values in value group E are defined in Clause 7 in a tabular form: the table header defines the common part of the OBIS code and each table cell defines one possible value of value group E. A “Register table” object may capture attributes of some or all of those objects.

NOTE 1 Some examples are the “Extended phase angle measurement” table, see Table 69 or the “UNIPEDE voltage dip quantities” table, see Table 71.

NOTE 2 If more complex functionality is needed, the “Profile generic” IC can be used.

Register table		**0...n**	**class_id = 61, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. table_cell_values	(dyn.)	compact-array or array				x + 0x08
3. table_cell_definition	(static)	structure				x + 0x10
4. scaler_unit	(static)	scaler_unit_type				x + 0x18
Specific methods		***m/o***				
1. reset (data)		o				x + 0x28
2. capture (data)		o				x + 0x30

Attribute description

logical_ name

Identifies the “Register table” object instance.

When the format of the *logical_name* is A.B.C.D.255.F; the values A to D and F define the common part of the logical name of the objects, the attributes of which are captured. Only one attribute of the objects concerned can be captured (for example the *value* attribute).

When the format of the *logical_name* is A.B.98.10.x.255, several instances of the “Register table” IC can be used to capture different attributes of the objects concerned. The value group E numbers the instances. See 7.4.4.

© Copyright 1997-2020 DLMS User Association

table_cell_values

Holds the value of the attributes captured, as they would be returned to a GET or Read .request to the individual attributes.

compact array or array table_cell_entry

table_cell_entry ::= CHOICE

```
{
    -- simple data types
    null-data                [0],
    bit-string               [4],
    double-long              [5],
    double-long-unsigned     [6],
    octet-string             [9],
    visible-string           [10],
    utf8-string              [12],
    bcd                      [13],
    integer                  [15],
    long                     [16],
    unsigned                 [17],
    long-unsigned            [18],
    long64                   [20],
    long64-unsigned          [21],
    float32                  [23],
    float64                  [24],

    -- complex data types
    structure                [2]
}
```

If the captured attribute is attribute_0, redundant values may be replaced by “null-data”, if their value can be unambiguously recovered (for example *scaler_unit*).

table_cell_definition

Specifies the list of attributes captured in the register table.

structure

```
{
    class_id:          long-unsigned,
    logical_name:      octet-string,
    group_E_values:    array unsigned,
    attribute_index:   integer
}
```

where:

- class_id defines the common class_id of the objects the attributes of which are captured;
- logical_name contains the common logical name of the objects, with E = 255 (wildcard);
- group_E_values contain the list of cell identifiers, of type *unsigned*, as defined in the respective table of Clause 7;
- attribute_index is a pointer to the attribute within the object. attribute_index 0 refers to all public attributes.

If the *logical_name* of the “Register table” object is in the format A.B.C.D.255.F and the defined attribute of all objects identified in the respective table in Clause 7 are captured, then attribute 3 may not be accessible. In this case:

- the class_id shall be 1 “Data”, 3 “Register” or 4 “Extended register”;
- the *logical_name* of the objects to be captured is defined by the *logical_name* of the “Register table” object and the respective table in Clause 7.
- the attribute index shall be 2 (value).

© Copyright 1997-2020 DLMS User Association

scaler_unit	See the description of “Register” IC. In the case when “value” attributes of “Register” or “Extended register” objects are captured, the *scaler_unit* shall be common for all objects and this attribute shall hold a copy. If other attributes or ICs are captured, the *scaler_unit* attribute has no meaning and shall be inaccessible.
Method description	
reset (data)	Clears the *table_cell_values*. It has no effect on the attributes captured. data ::= integer (0)
capture (data)	Copies the values of the attributes into the *table_cell_values*. If the attribute_index = 0, all attributes are captured.

Behaviour of the object after modification of the table_cell_definition attribute

Any modification to this attribute will automatically call the *reset* (data) method and this will propagate to all profiles capturing this object.

If writing to *table_cell_definition* is attempted with a value too large the buffer holding the *table_cell_values* attribute, it will be rejected.

4.3.9 Status mapping (class_id = 63, version = 0)

This IC allows modelling the mapping of bits in a status word to entries in a reference table.

Status mapping		**0...n**	**class_id = 63, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. status_word	(dyn.)	CHOICE				x + 0x08
3. mapping_table	(static)	structure				x + 0x10
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “Status mapping” object instances. See 6.2.45, 6.2.47, 6.2.54 and 6.3.7.

status_word	Contains the current value of the status word.
	CHOICE
	{
	bit-string [4],
	double-long-unsigned [6],
	octet-string [9],
	visible-string, [10],
	utf8-string [12],
	unsigned [17],
	long-unsigned [18],
	long64-unsigned [21]
	}
	The size of the *status_word* is n*8 bits, the maximum size is 65 536 bits.
	Manufacturers may choose any of the types listed above. However, the status word is always interpreted as a bit-string.

© Copyright 1997-2020 DLMS User Association

mapping_ table	Contains the mapping of the status_word to the positions in the reference table.
	structure
	{
	ref_table_id: unsigned,
	ref_table_mapping: CHOICE
	{
	long-unsigned [18],
	array long-unsigned [1]
	}
	}
	Where:
	– ref_table_id identifies the reference status table;
	– if the “long-unsigned” choice is taken, the value points to an entry in the reference table. This entry is mapped to the leading bit of the status word. The next entry is mapped to the next bit and so on. The last entry that is mapped to the trailing bit is determined by the length of the status word;
	– if the “array” choice is taken, the elements of the array point to entries in the reference status table. The order of the elements in the array corresponds to the position in the status word. The first element in the array maps the table entry referenced to the leading bit and the last element to the trailing bit.

4.3.10 Compact data (class_id = 62, version = 1)

4.3.10.1 Compact data interface class specification

NOTE This version 1 supports both relative and absolute selective access.

Instances of the “Compact data” IC allow capturing the values of COSEM object attributes as determined by the *capture_objects* attribute. Capturing can take place:

- on an external trigger (explicit capturing); or
- upon reading the compact_buffer attribute (implicit capturing)

as determined by the *capture_method* attribute.

The values are stored in the *compact_buffer* attribute as an octet-string.

The set of data types is identified by the *template_id* attribute. The data type of each attribute captured is held by the *template_description* attribute.

The client can reconstruct the data in the uncompacted form – i.e. including the COSEM attribute descriptor, the data type and the data values – using the *capture_objects*, *template_id* and *template_description* attributes.

© Copyright 1997-2020 DLMS User Association

Compact data		0...n	class_id = 62, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. compact_buffer	(dyn.)	octet-string				x + 0x08
3. capture_objects	(static)	array				x + 0x10
4. template_id	(static)	unsigned				x + 0x18
5. template_description	(dyn.)	octet-string				x + 0x20
6. capture_method	(static)	enum				x + 0x28
Specific methods		***m/o***				
1. reset (data)		o				x + 0x38
2. capture (data)		o				x + 0x40

Attribute description	
logical_name	Identifies the “Compact data” object instance. See 6.2.41.
compact_buffer	Contains the values of the attributes captured as an octet-string. When the data captured is of type octet-string, bit-string, visible-string, utf8-string or array the length is also included here.

capture_objects	Specifies the list of COSEM object attributes that are assigned to the “Compact data” object instance.
	The *template_id* attribute shall be the first element in the capture_objects array.
	Upon an explicit or implicit invocation of the *capture* (data) method the values of the selected attributes are captured into the *compact_buffer.*
	
	Two, mutually exclusive selective access mechanisms are available:
	– relative selective access, i.e. entries defined relative to current date or entry are returned: this mechanism is controlled by the data_index element; or
	– absolute selective access, i.e. entries in an explicitly defined date range or entry range are returned: entries selected by this mechanism are controlled by the *restriction_element* and the columns are controlled by the lower nibble of the MS byte in the *data_index*.
	
	array capture_object_definition
	capture_object_definition ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer,
	data_index: long-unsigned,
	restriction: restriction_element
	}
	
	Where:
	– attribute_index is a pointer to the attribute within the object, identified by class_id and logical_name: attribute_index 1 refers to the 1st attribute (i.e. the logical_name), attribute_index 2 to the 2nd attribute etc.; attribute_index 0 refers to all public attributes, noting DLMS UA 1000-2 Ed. 10, 9.1.4.3.7;

© Copyright 1997-2020 DLMS User Association

capture_objects (continued)

– data_index is a pointer selecting one or several specific elements of an attribute with a complex data type (structure or array):
  – if the data type of the attribute is simple, then data_index has no meaning;
  – if the data type of the attribute is a structure or an array – other than the *buffer* of a Profile generic object – then data_index points to one or several specific elements in the structure or array;
  – when the attribute is the *buffer* of a "Profile generic" object, the data_index carries selective access parameters relative to current date or entry;

data_index:	**MS-Byte**		**LS-Byte**
	Upper nibble	Lower nibble	

  – 0x0000 = identifies the whole attribute;
  – 0x0001 to 0x0FFF = identifies one element in the complex attribute. The first element in the complex attribute is identified by data_index 1;
  – 0x1000 to 0xFFFF = selective access to the array holding the *buffer* of a "Profile generic" object. The data_index selects entries within a number of last (recent) time periods, or a number of last (recent) entries, as well as the columns in the array.

The encoding is specified in Table 6;

When the attribute is the *buffer* of a "Profile generic" object, then restriction_element specifies selective access parameters in an explicitly defined date range or entry range.

```
restriction_element ::= structure
{
    restriction_type:  enum:
                           (0) none,
                           (1) restriction by date,
                           (2) restriction by entry
    restriction_value: CHOICE
    {
        null-data,                  // no restrictions apply
        restriction_by_date,
        restriction_by_entry
    }
}

restriction_by_date ::= structure
{
    from_date:         octet-string,
    to_date:           octet-string
}

restriction_by_entry ::= structure
{
    from_entry:        double-long-unsigned,
```


© Copyright 1997-2020 DLMS User Association

capture_objects (continued)	to_entry: double-long-unsigned } - restriction_element defines absolute selective access to a “Profile generic” buffer by date range (from_date to to_date) or by entries (from_entry to to_entry). To use this absolute selective access mechanism, data_index shall be: - MS Byte upper nibble set to 0x0; - MS Byte lower nibble 0x0 to 0xF in accordance with Table 6; - Lower byte set to 0x00 - restriction_element is composed of restriction_type and restriction_value: – for restriction by date range the restriction_type element holds (1) restriction by date and the restriction_value element holds restriction_by_date structure; – for restriction by entries the restriction_type element holds (2) restriction by entry and the restriction_value element holds restriction_by_entry structure; – otherwise, the restriction_type element holds (0) none and the restriction_value element holds null-data. This choice shall be taken also if relative selective access is to be used.
template_id	Contains the identifier of the template. It shall uniquely identify the instance of the “Compact data” IC and the *template_description.*
template_ description	Provides the data type of each attribute captured. It is an *octet-string* generated automatically by the server upon the programming of the *capture_objects* and it has the following structure: - the first octet is 0x02 (the tag of a structure); - this is followed by the number of elements in the structure – the same as the number of elements in the *capture_objects* array – encoded as a variable length integer; - this is followed by the data type of each attribute, in the same order as in the *capture_object* array: – in the case of attributes with simple data type, the data type is represented by a single octet, carrying the tag of the data type. In the case of *bit-string* [4], *octet-string* [9], *visible-string* [10], *utf8-strin*g [12] the length of the string is part of the data held in the *compact_buffer*; – in the case of an *array* [1], the data type is represented by a single octet 0x01. This is followed by the type description of the elements in the array. The number of elements in the array is part of the data held in the *compact_buffer*; – in the case of a *structure* [2], the data type is represented by a single octet 0x02, followed by the number of elements inside the structure followed by the tag of each element of the structure.

© Copyright 1997-2020 DLMS User Association

capture_method	Defines the way the *compact_buffer* is updated. enum (0) Capture upon invoking the *capture* (data) method. This may occur remotely or locally (explicit capturing), (1) Capture upon reading the *compact_buffer* attribute (implicit capturing).
Method description	
reset (data)	Clears the *compact_buffer*. After invoking this method the buffer holds an octet-string of 0 length until a new capture takes place. This call does not trigger any additional operations of the *capture_objects*. Specifically, it does not reset any captured attributes. data ::= integer(0)
capture (data)	Copies the values of the attributes into the *compact_buffer* by reading each capture object. This call does not trigger any additional operations within the *capture_objects* such as capture () or reset (). data ::= integer(0)

Table 6 – Encoding of selective access parameters with data_index

	MS-Byte Upper nibble : Selects the time intervals or entries
0xF	**Last complete number of months:** This is equal to a selective access to the profile *buffer* resulting in all entries of the last complete number of months and the first entry at midnight of the current month.
0xE	**Last complete number of days:** This is equal to a selective access to the profile *buffer* resulting in all entries of the last complete number of days and the first entry at midnight of today.
0xD	**Last complete number of hours:** This is equal to a selective access to the profile *buffer* resulting in all entries of the last complete number of hours and the first entry of the current hour.
0xC	**Last complete number of minutes:** This is equal to a selective access to the profile *buffer* resulting in all entries of the last complete number of minutes and the first entry of the current minute.
0xB	**Last number of seconds:** This is equal to a parameterised read access to the profile resulting in all entries of the last number of seconds.
0xA	**Last complete number of months including the current month:** This is the same as 0xF above but all entries up to now are retrieved.
0x9	**Last complete number of days including the current day:** This is the same as 0xE above but all entries up to now are retrieved.
0x8	**Last complete number of hours including the current hour:** This is the same as 0xD above but all entries up to now are retrieved.
0x7	**Last complete number of minutes including the current minute:** This is the same as 0xC above but all entries up to now are retrieved.
	Values 0x6 to 0x2 apply only for the Push interface class (class_id = 40, version = 2)
0x6	**Complete number of months after last confirmation:** This is equal to a selective access to the profile *buffer* resulting in all entries of the complete number of months after the last confirmed entry.
0x5	**Complete number of days after last confirmation:** This is equal to a selective access to the profile *buffer* resulting in all entries of the complete number of days after the last confirmed entry.
0x4	**Complete number of hours after last confirmation:** This is equal to a selective access to the profile *buffer* resulting in all entries of the complete number of hours after the last confirmed entry.
0x3	**Complete number of minutes after last confirmation:** This is equal to a selective access to the profile *buffer* resulting in all entries of the complete number of minutes after the last confirmed entry.

© Copyright 1997-2020 DLMS User Association

	MS-Byte Upper nibble : Selects the time intervals or entries
0x2*	**Number of entries after last confirmation**: This is equal to a selective access to the profile *buffer* resulting in number of entries after the last confirmed entry.
0x1	Last number of entries
0x0	Whole attribute or a single element in an attribute with complex data type is selected (see above)
	MS-Byte Lower nibble != 0: Defines the number of columns selected
0x0	All columns
0x1 to 0xF	Number of columns, starting from column 1
0x00..0xFF	LS-Byte: – when entries in time intervals are selected, specifies the number of time intervals, value 0x00 defines all time periods ; – when entries are selected, specifies the number of entries, value 0x00 defines all entries.
Example 1)	0xE401: The entries for the last complete day are selected. The first 4 columns are included.
Example 2)	0xA300: The entries for the current month are selected. The first 3 columns are included.
Example 3)	0x800C: The entries for the last complete 12 hours are selected. All columns are included.
Example 4)	0x1080: The last 128 entries are selected. All columns are included.
Example 5)	0x4608: The entries for the last complete 8 hours after the last confirmed entry are selected. The first 6 columns are included.

Behaviour of the object after modification of certain attributes:

Any modification of the *capture_objects* shall reset the *compact_buffer* and automatically update the *template_description*.

Restrictions

When defining the *capture_objects* attribute, circular references shall be avoided.

4.3.10.2 Examples for using compact data

4.3.10.2.1 Example Daily billing data

Table 7 shows the daily billing data that are captured – together with the mandatory *template_id* – to the *compact_buffer* attribute of a “Compact data” object.

Table 7 – Example daily billing data captured to *compact_buffer*

Data	class _id	Logical name	attribute_ id	data_ index	Size (bytes)	Type	Value
1	2	3	4	5	6	7	8
Template Id	62	0-0:66.0.0.255	4	0	1	unsigned	0
Unix time	1	0-0:1.1.0.255	2	0	4	double-long-unsigned	1374573317
Operating status	1	0-0:96.5.0.255	2	0	1	unsigned	0x29
Error register	1	0-0:97.97.0.255	2	0	1	unsigned	0x18
Total index	3	7-0:13.83.1.255	2	0	4	double-long-unsigned	6422483
Index F1	3	7-0:13.83.1.255	2	0	4	double-long-unsigned	865234
Index F2	3	7-0:13.83.1.255	2	0	4	double-long-unsigned	1234567
Index F3	3	7-0:13.83.1.255	2	0	4	double-long-unsigned	2345678
Activity calendar name	20	0-0:13.0.0.255	2	0	6	octet-string	“ABCDEF”
Event counter	1	0-0:96.15.1.255	2	0	2	long-unsigned	7890

© Copyright 1997-2020 DLMS User Association

Table 8 shows the attributes of the “Compact data“ object.

Table 8 – “Compact data“ object attributes – Daily billing data example

capture_objects (array)	For the elements of the array, see columns 2, 3, 4 and 5 of Table 7.
template_id (unsigned)	0
template_description (octet-string)	-- For the data types, see column 7 of Table 7. 02 0A 11 06 11 11 06 06 06 06 09 12
compact_buffer (octet-string) 32 bytes)	-- For the values see column 8 of Table 6. 00 51EE5305 29 18 0061FFD3 000D33D2 0012D687 0023CACE 06414243444546 1ED2

For comparison, the A-XDR encoding of the same data as if they were accessed using a GET-WITH-LIST service is shown in Table 9. Only the encoding of the result (SEQUENCE OF Get-Data-Result) is shown.

Table 9 – Example daily billing data read using GET-WITH LIST

Encoding	Explanation	Length
09	**SEQUENCE** of 9 elements	1
00 06 51EE5305	double-long-unsigned	6
00 11 29	unsigned	3
00 11 18	unsigned	3
00 06 0061FFD3	double-long-unsigned	6
00 06 000D33D2	double-long-unsigned	6
00 06 0012D687	double-long-unsigned	6
00 06 0023CACE	double-long-unsigned	6
00 09 06 414243444546	octet-string of length 6	9
00 12 1ED2	long-unsigned	4
	Total	50 bytes
NOTE The leading 00-s in each element are there to indicate the CHOICE “Data” in Get-Data-Result.		

4.3.10.2.2 Diagnostic and Alarm data

Table 10 shows the diagnostic and alarm data that are captured – together with the mandatory *template_id* – to the *compact_buffer* attribute of a “Compact data” object.

© Copyright 1997-2020 DLMS User Association

Table 10 – Example diagnostic and alarm data captured to *compact_buffer*

Data	class _id	Logical name	attribute _id	data_ index	Size (bytes)	Type	Value
1	2	3	4	5	6	7	8
Template Id	62	0-0:66.0.0.255	4	0	1	unsigned	1
Current Diagnostic	3	7-0:96.5.1.255	2	0	2	long-unsigned	0x4200
Daily Diagnostic	3	7-1:96.5.1.255	2	0	2	long-unsigned	0x4108
Billing Period Diagnostic	1	7-2:96.5.1.255	2	0	2	long-unsigned	0x4308
Synchronization event counter	1	0-0:96.15.2.255	2	0	2	long-unsigned	763
Metrological firmware version	1	7-0:0.2.1.255	2	0	8	octet-string	“ABCDEFGH”
Metrological event counter	1	0-0:96.15.1.255	2	0	2	long-unsigned	1532
Non-metrological firmware version	1	7-1:0.2.1.255	2	0	8	octet-string	“DEFGHIJK”

Table 11 shows the attributes of the “Compact data“ object.

Table 11 – “Compact data“ object attributes – Diagnostic and Alarm data example

capture_objects (array)	For the elements of the array, see columns 2, 3, 4 and 5 of Table 10.
template_id (unsigned)	1
template_description (octet-string)	-- For the data types, see column 7 of Table 10. 02 08 11 12 12 12 12 09 12 09
compact_buffer (octet-string) 29 bytes	-- For the values, see column 8 of Table 10. 01 4200 4108 4308 02FB 08414243444546 4748 05FC 084445464748494A4B

For comparison, the A-XDR encoding of the data as if they were read from the *buffer* attribute of a “Profile generic” object is shown in Table 12 (only the Data is shown).

Table 12 – Example diagnostic and alarm data read from “Profile generic” *buffer*

Encoding	Explanation	Length
01 01	array of one element	2
02 07	structure of 7 elements	2
12 4200	long-unsigned	3
12 4108	long-unsigned	3
12 4308	long-unsigned	3
12 02FB	long-unsigned	3
09 08 4142434445464748	octet-string of length 8	10
12 05FC	long-unsigned	3
09 08 4445464748494A4B	octet-string of length 8	10
	Total	39 bytes

© Copyright 1997-2020 DLMS User Association

4.3.10.2.3 Logbook reading

In this example, the data to be compacted is the *buffer* attribute of a Logbook held by a “Profile generic” object capturing 2 elements, as shown in Table 13.

Table 13 – Example logbook data entries in “Profile generic” *buffer*

Data	class _id	Logical name	Attribute _id	data_ index	Size (bytes)	Type	Value
1	2	3	4	5	6	7	8
Unix time	1	0-0:1.1.0.255	2	0	4	double-long-unsigned	See Note
Event code	1	0-0:96.11.2.255	2	0	1	unsigned	
NOTE: For this example, the following values are assumed: - UNIX timestamp: 1374573317D, - status: 0x29, - for simplicity of the example, the values are the same for all entries, - there are 50 entries in the buffer.							

Table 14 shows the data to be captured by the “Compact data” object.

Table 14 – Example logbook data captured to *compact_buffer*

Data	class _id	Logical name	attribute _id	Data index	Size (bytes)	Type	Value
1	2	3	4	5	6	7	8
Template Id	62	0-0:66.0.0.255	4	0	1	unsigned	2
Logbook buffer [1]	7	0-0:99.98.0.255	2	0	dyn. [2]	array of structure	2
[1] See Table 12. [2] The size is dynamic and depends on the number of entries captured.							

Table 15 shows the attributes of the “Compact data” object.

Table 15 – “Compact data“ object attributes – Logbook data example

capture_objects (array)	For the elements of the array, see columns 2, 3, 4 and 5 of Table 13. The capture object has only 2 elements: the Template_id and the *buffer* attribute of the Logbook.
template_id (unsigned)	2
template-description (octet-string)	-- For the data types, see column 7 of Table 14. 02 02 11 01 02 02 06 11 Meaning: 02 02 – a structure of 2 elements 11 – first element is an unsigned 01 02 02– second element is an array of structure with two elements in the structure 06 first one is a double-long-unsigned 11 second one is an unsigned
compact_buffer (octet-string)	-- For the values, see column 8 of Table 14. 02 -- value of the template-id 32 -- number of the elements in the array and 50*5 = 250 bytes (for 50 elements in the log book) 252 bytes in total

© Copyright 1997-2020 DLMS User Association

For comparison, the A-XDR encoding of the same data when read from the *buffer* attribute of a “Profile generic” object is shown in Table 16.

Table 16 – Example logbook data read from “Profile generic” *buffer*

Encoding	Explanation	Length
01 32	array of 50 elements	2
02 02	structure of 2 elements	2
06 51EE5305	double-long-unsigned	5
11 29	unsigned	2
02 02	structure of 2 elements	2
06 51EE5305	double-long-unsigned	5
11 29	unsigned	2
02 02	structure of 2 elements	2
06 51EE5305	double-long-unsigned	5
11 29	unsigned	2
….	…	…
	Total	**452 bytes**

© Copyright 1997-2020 DLMS User Association

4.4 Interface classes for access control and management

4.4.1 Overview

Interface classes in this category model the logical structure of the DLMS/COSEM server, allow configuring and managing access to its resources, updating the firmware and managing security:

- the “Association SN” class – see 4.4.3 – and the “Association LN” class – see 4.4.4 – model AAs. Their instances, the Association objects provide the list of objects accessible in each AA, manage and control access rights to their attributes and methods. They also manage the authentication of the communicating partners;
- the “SAP Assignment” class – see 4.4.5 – models the logical structure of the server;
- the “Image transfer” class – see 4.4.6 – models the firmware update process;
- the “Security setup” class – see 4.4.7 – models the elements of the security context. “Security setup” objects are referenced from the “Association” objects and allow configuring security suites and security policies and managing security material;
- the “Push setup” class – see 4.4.8 – models the push operation of the server;
- the “Data protection” class – see 4.4.9 – specifies the necessary elements to apply cryptographic protection to COSEM object attribute values as well as to method invocation and return parameters;
- the “Function control” class – see 4.4.10 – allows enabling and disabling functions in the server;
- the “Array manager” class – see 4.4.11 – allows managing attributes of type array of other interface objects;
- the “Communication port protection” class – see 4.4.12 – allows protection of ports against unauthorised attempts at communication.

4.4.2 Client user identification

This feature enables the server to distinguish between different users from the client side and to log their activities accessing the meter but it is not applicable to pre-established AAs.

Each AA established between a client and a server can be used by several users on the client side. The properties of the AA are configured in the server, using the “Association” and the “Security setup” objects. All users of an AA on the client side use these same properties.

The security keys are known by the client and the server but they need not be known by the users of the client.

The list of users – identified by their user_id and user_name – is known both by the client and the server. In the server it is held by the *user_list* attribute of the “Association” objects.

NOTE 1 The way a client authenticates a user to log into a client system is outside the scope of this specification.

During AA establishment, the user_id – belonging to the user_name – is carried by the calling-AE-invocation-id field of the AARQ APDU.

NOTE 2 For this reason, this feature is not available with pre-established AAs.

If the user_id provided is on the *user_list*, the AA can be established – provided that all other conditions are met – and the *current_user* attribute is updated. The value of this attribute can be logged.

If the server does not “know” the user, the AA shall not be established. The server may silently discard the request to establish the AA or it may send back an appropriate error message.

The user identification process is optional: if the *user_list* is empty – i.e. it is an array of 0 elements – the function is disabled.

© Copyright 1997-2020 DLMS User Association

4.4.3 Association SN (class_id = 12, version 4)

COSEM logical devices able to establish AAs within a COSEM context using SN referencing, model the AAs using instances of the “Association SN” IC. A COSEM logical device may have one instance of this IC for each AA the device is able to support.

The **short_name** of the current “Association SN” object itself is fixed within the COSEM context. See 4.1.3.

Association SN	**0...n**	**class_id = 12, version = 4**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	objlist_type				x + 0x08
3. access_rights_list (static)	access_rights_type				x + 0x10
4. security_setup_reference (static)	octet-string				x + 0x18
5. user_list (static)	array				x + 0x20
6. current_user	structure				x + 0x28
Specific methods	***m/o***				
1. *reserved from previous versions*	o				
2. *reserved from previous versions*	o				
3. read_by_logicalname (data)	o				x + 0x30
4. *reserved from previous versions*	o				
5. change_secret (data)	o				x + 0x40
6. *reserved from previous versions*	o				
7. *reserved from previous versions*					
8. reply_to_HLS_authentication (data)	o				x + 0x58
9. add_user (data)	o				x + 0x60
10. remove_user (data)	o				x + 0x68

Attribute description	
logical_name	Identifies the “Association SN” object instance. See 6.2.33.

object_list	Contains the list of all objects with their base_name (short_name), class_id, version and *logical_name*. The base_name is the DLMS objectName of the first attribute (*logical_name).*
	objlist_type ::= array objlist_element
	
	objlist_element ::= structure
	{
	base_name: long,
	class_id: long-unsigned,
	version: unsigned,
	logical_name: octet-string
	}
	selective access (see 4.1.4) to the attribute *object_list* may be available. The access selector values and their parameters are as defined below.

© Copyright 1997-2020 DLMS User Association

access_rights_list

Contains the access rights to attributes and methods.

The link between the *object_list* and the *access_rights_list* is the base_name, present in both the objlist_element structure and the access_right_element structure. Therefore, the base_names on the two lists shall be the same. The number – and preferably, the order – of the elements in the array of objlist_element and the array of access_right_element shall also be the same.

```
access_rights_type ::= array        access_rights_element

access_rights_element ::= structure
{
    base_name:                  long,
    attribute_access:           attribute_access_descriptor,
    method_access:              method_access_descriptor
}

attribute_access_descriptor ::= array      attribute_access_item

attribute_access_item ::= structure
{
    attribute_id:               integer,
    access_mode:                enum
```

When the enum value is interpreted as an unsigned8, the meaning of each bit is as shown below:

```
                {
                    Bit     attribute access_mode

                    (0)     read-access,
                    (1)     write-access,
                    (2)     authenticated request,
                    (3)     encrypted request,
                    (4)     digitally signed request,
                    (5)     authenticated response,
                    (6)     encrypted response,
                    (7)     digitally signed response
                }

    access_selectors:       CHOICE
        {
        null-data           [0],
        array integer       [1]
        }
}

method_access_descriptor ::= array              method_access_item
method_access_item ::= structure
{
    method_id:              integer,
    access_mode:            enum
```


© Copyright 1997-2020 DLMS User Association

access_ rights_list (continued)	When the enum value is interpreted as an unsigned8, the meaning of each bit is as shown below:
	{
	Bit method access_mode
	
	(0) access,
	(1) not-used,
	(2) authenticated request,
	(3) encrypted request,
	(4) digitally signed request,
	(5) authenticated response,
	(6) encrypted response,
	(7) digitally signed response
	}
	}
	selective access (see 4.1.4) to the attribute *access_rights_list* may be available (optional). The access selector values and their parameters are as defined below.

security_ setup_ reference	References the “Security setup” object by its *logical_name*. The referenced object manages security for a given “Association SN” object instance.

user_list	Contains the list of users allowed to use the AA managed by the given instance of the “Association SN” IC.
	array user_list_entry
	
	user_list_entry ::= structure
	{
	user_id: unsigned,
	user_name visible-string
	}
	Where:
	- user_id is the identifier of the user (this value is carried by the calling-AE-invocation-id field of the AARQ);
	- user_name is the name of the user.
	If the user_list attribute is empty – i.e. it is an array of 0 elements – any user can use the AA, i.e. the calling-AE-invocation-id field of the AARQ is ignored.
	If the user_list attribute is not empty then only the users in the list can establish the AA, i.e. the calling-AE-invocation-id field of the AARQ shall be present and its value shall match one of user_ids in the user_list or else, the AA is not established.

current_user	Holds the identifier of the current user. current_user ::= user_list_entry (see above) If the user_list is empty, then current_user shall be a structure {user_id: unsigned 0, user_name: visible string of 0 elements}

© Copyright 1997-2020 DLMS User Association

Parameters for selective access to the *object_list* and *access_rights_list* attribute

Access selector value	Parameter	Available with attribute	Comment
1	class_id: long-unsigned	2	Delivers the subset of the object_list for a specific class_id. For the response: data ::= objlist_type

2	structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
		2	Delivers the entry of the object_list for a specific class_id and *logical_name*.
			For the response: data ::= objlist_element

3	base_name: long	2, 3	In the case of attribute 2, delivers the entry of the object_list for a specific base_name. For the response: data ::= objlist_element In the case of attribute 3, delivers the entry of the access_rights_list for a specific base_name. For the response: data ::=access_rights_element

Method description	

read_by_ logicalname (data)	Reads attributes for selected objects. The objects are specified by their class_id and their *logical_name*. With this method, the parameterized access feature can also be used.
	data ::= array attribute_identification
	
	attribute_identification ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}
	where attribute_index is a pointer (i.e. offset) to the attribute within the object.
	attribute_index 0 delivers all attributes; attribute_index 1 delivers the first attribute (i.e. *logical_name*), etc.).
	For the response: data is according to the type of the attribute.
	NOTE 1 If at least one attribute has no read access right under the current association, then a *read_by_logicalname* () to attribute index 0 reveals the error message "scope-of-access-violated", see DLMS UA 1000-2 Ed. 10, 9.5.

change_secret (data)	Changes the LLS or HLS secret (for example password). data ::= octet-string new secret NOTE 2 The structure of the "new secret" depends on the security mechanism implemented. The "new secret" may contain additional check bits and it may be encrypted.
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers to the server the result of the secret processing by the client of the server's challenge to the client, f(StoC), as the data service parameter of the Read.request primitive invoked with parameterised access. data ::= octet-string client's response to the challenge
reply_to_HLS_ authentication (data)	If the authentication is accepted, then the response (Read.confirm primitive) contains Result == OK and the result of the secret processing by the server of

© Copyright 1997-2020 DLMS User Association

(continued)	the client's challenge to the server, f(CtoS) in the data service parameter of the Read.response service. data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.
add_user (data)	Adds a user to the user_list. data ::= user_list_entry (see above)
remove_user (data)	Removes a user from the user_list. data ::= user_list_entry (see above)

4.4.4 Association LN (class_id = 15, version 3)

COSEM logical devices able to establish AAs within a COSEM context using LN referencing, model the AAs through instances of the "Association LN" IC. A COSEM logical device has one instance of this IC for each AA the device is able to support.

Association LN	**0...MaxNbofAss.**	**class_id = 15, version = 3**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	object_list_type				x + 0x08
3. associated_partners_id	associated_partners_type				x + 0x10
4. application_context_name	context_name_type				x + 0x18
5. xDLMS_context_info	xDLMS_context_type				x + 0x20
6. authentication_mechanism_name	mechanism_name_type				x + 0x28
7. secret	octet-string				x + 0x30
8. association_status	enum				x + 0x38
9. security_setup_reference (static)	octet-string				x + 0x40
10. user_list (static)	array				x + 0x48
11. current_user	structure				x + 0x50
Specific methods	***m/o***				
1. reply_to_HLS_authentication (data)	o				x + 0x60
2. change_HLS_secret (data)	o				x + 0x68
3. add_object (data)	o				x + 0x70
4. remove_object (data)	o				x + 0x78
5. add_user (data)	o				x + 0x80
6. remove_user (data)	o				x + 0x88

Attribute description	
logical_name	Identifies the "Association LN" object instance. See 6.2.33.

© Copyright 1997-2020 DLMS User Association

object_list

Contains the list of visible COSEM objects with their class_id, version, *logical_name* and the access rights to their attributes and methods within the given AA.

```
object_list_type ::= array              object_list_element
object_list_element ::= structure
{
        class_id:               long-unsigned,
        version:                unsigned,
        logical_name:           octet-string,
        access_rights:          access_right
}

access_right ::= structure
{
        attribute_access:       attribute_access_descriptor,
        method_access:          method_access_descriptor
}

attribute_access_descriptor ::= array       attribute_access_item

attribute_access_item ::=       structure
{
        attribute_id:           integer,
        access_mode:            enum

        When the enum value is interpreted as an unsigned8, the meaning of each
        bit is as shown below:
                        {
                                Bit     attribute access_mode

                                (0)     read-access,
                                (1)     write-access,
                                (2)     authenticated request,
                                (3)     encrypted request,
                                (4)     digitally signed request,
                                (5)     authenticated response,
                                (6)     encrypted response,
                                (7)     digitally signed response
                        }

        access_selectors:       CHOICE
        {
                null-data       [0],
                array integer   [1]
        }
}
```


© Copyright 1997-2020 DLMS User Association

object_list (continued)

method_access_descriptor ::= array method_access_item

method_access_item ::= structure
{
method_id: integer,
access_mode: enum

When the enum value is interpreted as an unsigned8, the meaning of each bit is as shown below:

{
Bit method access_mode

(0) access,
(1) not-used,
(2) authenticated request,
(3) encrypted request,
(4) digitally signed request,
(5) authenticated response,
(6) encrypted response,
(7) digitally signed response
}
}

Where:

- the attribute_access_descriptor and the method_access_descriptor elements always contain all implemented attributes or methods;
- access_selectors contain a list of the supported selector values.

selective access (see 4.1.4) to the attribute *object_list* may be available (optional). The selective access parameters are as defined below.

associated_ partners_ id

Contains the identifiers of the COSEM client and server (logical device) APs within the physical devices hosting these APs, which belong to the AA modelled by the “Association LN” object.

associated_partners_type ::= structure
{
client_SAP: integer,
server_SAP: long-unsigned
}

The range for the client_SAP is 0…0x7F.

The range for the server_SAP is 0x0000…0x3FFF.

The SAPs shall be in the range allowed by the data type and the media.

© Copyright 1997-2020 DLMS User Association

application_ context_name

In the COSEM environment, it is intended that an application context pre-exists and is referenced by its name during the establishment of an AA. This attribute contains the name of the application context for that AA.

```
context_name_type ::=        CHOICE
{
        context_name_structure          [2],
        octet-string                    [9]
}
```

The application context name is specified as OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.2.

When the context_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.

```
context_name_structure ::= structure
{
        joint_iso_ctt_element:              unsigned,
        country_element:                    unsigned,
        country_name_element:               long-unsigned,
        identified_organization_element:    unsigned,
        DLMS_UA_element:                    unsigned,
        application_context_element:        unsigned,
        context_id_element:                 unsigned
}
```

Example 1: In the case of context_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 01 11 01 (all values are hexadecimal).

When the context_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.

Example 2: In the case of context_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 01 01 (all values are hexadecimal).

xDLMS_ context_info

Contains all the necessary information on the xDLMS context for the given AA.

```
xDLMS_context_type ::= structure
{
        conformance:                bit-string,
        max_receive_pdu_size:       long-unsigned,
        max_send_pdu_size:          long-unsigned,
        dlms_version_number:        unsigned,
        quality_of_service:         integer,
        cyphering_info:             octet-string
}
```

Where:

- the conformance element contains the xDLMS conformance block supported by the server. The length of the bit-string is 24 bits;
- the max_receive_pdu_size element contains the maximum length for an xDLMS APDU, expressed in bytes that the client may send, negotiated during the application association process and limited by the server-max-receive-pdu-size parameter of the xDLMS initiateResponse APDU;
- the max_send_pdu_size element in an active AA contains the maximum length for an xDLMS APDU, expressed in bytes that the server may send. It is limited by the client-max-receive-pdu-size parameter of the xDLMS initiateRequest APDU.

© Copyright 1997-2020 DLMS User Association

xDLMS_ context_info (continued)	- the dlms_version_number element contains the DLMS version number supported by the server; - the quality_of_service element is not used; - the cyphering_info element – in an active AA – contains the dedicated key parameter of the xDLMS initiateRequest APDU. See DLMS UA 1000-2 Ed. 10, 9.5.

authentication_ mechanism_ name	Contains the name of the authentication mechanism for the AA.
	mechanism_name_type ::= CHOICE
	{
	mechanism_name_structure [2],
	octet-string [9]
	}
	The authentication mechanism name is specified as an OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.3.
	
	When the mechanism_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.
	
	mechanism_name_structure ::= structure
	{
	joint_iso_ctt_element: unsigned,
	country_element: unsigned,
	country_name_element: long-unsigned,
	identified_organization_element: unsigned,
	DLMS_UA_element: unsigned,
	authentication_mechanism_name_element: unsigned,
	mechanism_id_element: unsigned
	}
	
	Example 3: In the case of mechanism_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 02 11 01 (all values are hexadecimal):
	
	When the mechanism_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.
	
	EXAMPLE 4: In the case of mechanism_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 02 01 (all values are hexadecimal).
	
	No mechanism_name is required when no authentication is used.

secret	Contains the secret for the LLS or HLS authentication process.
association_ status	Indicates the current status of the association, which is modelled by the object. enum: (0) non-associated, (1) association-pending, (2) associated
security_setup _reference	References a “Security setup” object by its logical name. The referenced object manages security for a given “Association LN” object instance.

© Copyright 1997-2020 DLMS User Association

user_list	Contains the list of users allowed to use the AA managed by the given instance of the "Association LN" IC.
	array user_list_entry
	
	user_list_entry ::= structure
	{
	user_id: unsigned,
	user_name: visible-string
	}
	Where:
	- user_id is the identifier of the user (this value is carried by the calling-AE-invocation-id field of the AARQ);
	- user_name is the name of the user.
	If the *user_list* attribute is empty – i.e. it is an array of 0 elements – any user can use the AA, i.e. the calling-AE-invocation-id field of the AARQ is ignored.
	If the *user_list* attribute is not empty then only the users in the list can establish the AA, i.e. the calling-AE-invocation-id field of the AARQ shall be present and its value shall match one of user_ids in the *user_list* or else the AA is not established.

current_user	Holds the identifier of the current user. current_user ::= user_list_entry (see above) If the *user_list* is empty, then current_user shall be a structure {user_id: unsigned 0, user_name: visible string of 0 elements}

Parameters for selective access to the *object_list* attribute

- If no selective access is requested, (no Access_Selection_Parameters parameter is present in the GET.request (.indication) service primitive for the object_list attribute) the corresponding .response (.confirmation) service shall contain all object_list_elements of the object_list attribute.
- When selective access is requested to the object_list attribute (the Access_Selection_Parameter parameter is present), the response shall contain a 'filtered' list of object_list_elements, as follows:

Access selector	Access parameter	Comment
1	NULL	All information excluding the access_rights shall be included in the response.
2	class_list	Access by class_id. In this case, only those object_list_elements of the *object_list* shall be included in the response, which have a class_id equal to one of the class_id-s of the class_list. No access_right information is included. class_list ::= array class_id class_id: long-unsigned

3	object_id_list	Access by object. The full information record of object instances on the object_id_list shall be returned.
		object_id_list ::= array object_id
		object_id ::= structure
		{
		class_id: long-unsigned,
		logical_name: octet-string
		}

© Copyright 1997-2020 DLMS User Association

Access selector	Access parameter	Comment
4	object_id	The full information record of the required COSEM object instance shall be returned. object_id ::= structure See above.

Method description	
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers to the server the result of the secret processing by the client of the server's challenge to the client, f(StoC), as the *data* service parameter of the ACTION.request primitive invoked. data ::= octet-string client's response to the challenge If the authentication is accepted, then the response (ACTION.confirm primitive) contains Result == OK and the result of the secret processing by the server of the client's challenge to the server, f(CtoS) in the *data* service parameter of the response service. data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.
change_HLS_secret (data)	Changes the HLS secret (for example encryption key). data ::= octet-string new HLS secret The structure of the "new secret" depends on the security mechanism implemented. The "new secret" may contain additional check bits and it may be encrypted.
add_object (data)	Adds the referenced object to the *object_list*. data ::= object_list_element (see above)
remove_object (data)	Removes the referenced object from the *object_list*. data ::= object_list_element (see above)
add_user (data)	Adds a user to the *user_list*. data ::= user_list_entry (see above)
remove_user (data)	Removes a user from the *user_list*. data ::= user_list_entry (see above)

4.4.5 SAP assignment (class_id = 17, version = 0)

This IC allows modelling the logical structure of physical devices, by providing information on the assignment of the logical devices to their SAP-s. See DLMS UA 1000-2 Ed. 10, Clause 10.

SAP assignment	**0...1**	**class_id = 17, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. SAP_assignment_list (static)	asslist_type			0	x + 0x08
Specific methods	***m/o***				
1. connect_logical_device (data)	o				x + 0x20

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “SAP assignment” objects instance. See 6.2.34.

SAP_ assignment_ list	Contains the list of all logical devices and their SAP addresses within the physical device.
	asslist_type ::= array asslist_element
	
	asslist_element ::= structure
	{
	SAP: long-unsigned,
	logical_device_name: CHOICE
	{
	octet-string [9],
	visible-string [10],
	utf8-string [12]
	}
	}
	REMARK: The actual addressing is performed by the supporting communication layers.

Method description	
connect_logical_ device (data)	Connects a logical device to a SAP. Connecting to SAP 0 will disconnect the device. More than one device cannot be connected to one SAP (exception SAP 0). data ::= asslist_element

4.4.6 Image transfer (class_id = 18, version = 0)

4.4.6.1 General

Instances of the Image transfer IC model the process of transferring binary files, called Images to COSEM servers.

NOTE This specification includes some improvements and precisions to the text. The main changes are:

- The description of the Image transfer process is given as an example only. The text and the flow chart are updated;
- Data exchange between the client and a conceptual Image server is out of Scope;
- A clear distinction is made between the Image transferred and the Images to activate;
- Steps 1 and 6 are optional now;
- Size of the *image_transferred_blocks_status* bit-string may be dynamic;
- It is specified now that setting the value of the *image_transfer_enabled* attribute to FALSE disables the image transfer process;
- It is specified now that re-initiating the image transfer process resets the whole process;
- Some precisions have been added to the effect of invoking the methods;

4.4.6.2 The steps of the image transfer process

The Image transfer usually takes place in several steps:

- Step 1: (Optional): Get ImageBlockSize;
- Step 2: Client initiates Image transfer;
- Step 3: Client transfers ImageBlocks;
- Step 4: Client checks completeness of the Image;
- Step 5: Server verifies the Image (Initiated by the client or on its own);

© Copyright 1997-2020 DLMS User Association

- Step 6 (Optional): Client checks the information on the images to activate;
- Step 7: Server activates the Image(s) (Initiated by the client or on its own).

For an example with more detailed explanations, see 4.4.6.4.

4.4.6.3 Image transfer interface class specification

Image transfer		**0…n**	**class_id = 18, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. image_block_size	(static)	double-long-unsigned				x + 0x08
3. image_transferred_blocks_status	(dyn.)	bit-string				x + 0x10
4. image_first_not_transferred_block_number	(dyn.)	double-long-unsigned				x + 0x18
5. image_transfer_enabled	(static)	boolean				x + 0x20
6. image_transfer_status	(dyn.)	enum				x + 0x28
7. image_to_activate_info	(dyn.)	array				x + 0x30
Specific methods		***m/o***				
1. image_transfer_initiate (data)		m				x + 0x40
2. image_block_transfer (data)		m				x + 0x48
3. image_verify (data)		m				x + 0x50
4. image_activate (data)		m				x + 0x58

Attribute description	
logical_name	Identifies the “Image transfer” object instance. See 6.2.37.
image_block_size	Holds the ImageBlockSize, expressed in octets, which can be handled by the server. ImageBlockSize shall not exceed the ServerMaxReceivePduSize negotiated. NOTE 1 *image_block_size* is a property of the server.
image_transferred_blocks_status	Provides information about the transfer status of each ImageBlock. Each bit in the bit-string provides information about one individual ImageBlock: 0 = Not transferred, 1 = Transferred NOTE 2 The size of the attribute may be dynamic, i.e. it may grow upon reception of new ImageBlocks.
image_first_not_transferred_block_number	Provides the ImageBlockNumber of the first ImageBlock not transferred. Once the Image is complete, the value returned should be equal to or above the number of blocks calculated from the Image size and the ImageBlockSize.
image_transfer_enabled	Controls the enabling of the Image transfer process. boolean: FALSE = Disabled, TRUE = Enabled The image transfer methods can be invoked successfully only if the value of this attribute is TRUE. Setting the value of this this attribute to FALSE disables all methods (invoking them fails).

© Copyright 1997-2020 DLMS User Association

image_transfer_ status	Holds the status of the Image transfer process. enum: (0) Image transfer not initiated, (1) Image transfer initiated, (2) Image verification initiated, (3) Image verification successful, (4) Image verification failed, (5) Image activation initiated, (6) Image activation successful, (7) Image activation failed

image_to_activate_ info	Provides information on the Image(s) ready for activation. It is generated as the result of the Image verification process. The client may check this information before activating the Images.
	array image_to_activate_info_element
	
	image_to_activate_info_element ::= structure
	{
	image_to_activate_size: double-long-unsigned,
	image_to_activate_identification: octet-string,
	image_to_activate_signature: octet-string
	}
	Where:
	- image_to_activate_size is the size of the Image to be activated, expressed in octets;
	- image_to_activate_identification is the identification of the Image to be activated, and may contain information like manufacturer, device type, version information, etc.;
	- image_to_activate_signature is the signature of the Image to be activated.
	NOTE 3 The algorithm to generate the signature is out of the Scope of this specification.

Method description	

image_transfer_ initiate (data)	Initializes the Image transfer process.
	data ::= structure
	{
	image_identifier: octet-string,
	image_size: double-long-unsigned
	}
	Where:
	- image_identifier identifies the Image to be transferred;
	- image_size holds the ImageSize, expressed in octets.
	NOTE 4 The *image_identifier* identifies the Image to be transferred (container) but it is not necessarily linked to its content, i.e. the Images which will be activated. That information can be retrieved from the *image_to_activate* attribute after verification of the Image transferred.
	After a successful invocation of the method the *image_transfer_status* attribute is set to (1) and the *image_first_not_transferred_block_number* is set to 0. Any subsequent invocation of the method resets the whole Image transfer process and all ImageBlocks need to be transferred again.

image_block_ transfer (data)	Transfers one block of the Image to the server. data ::= structure {

© Copyright 1997-2020 DLMS User Association

	image_block_number: double-long-unsigned, image_block_value: octet-string } After a successful invocation of the method the corresponding bit in the *image_transferred_blocks_status* attribute is set to 1 and the *image_first_not_transferred_block_number* attribute is updated.
image_verify (data)	Verifies the integrity of the Image before activation. data ::= integer (0) The result of the invocation of this method may be success, temporary_failure or other_reason. If it is not success, then the result of the verification can be found by retrieving the value of the *image_transfer_status* attribute. In the case of success, the *image_to_activate_info* attribute holds the information about the images to activate. NOTE 5 xDLMS services Action-Result / Data-Access-Result codes are specified in DLMS UA 1000-2 Ed. 10, 9.5.
image_activate (data)	Activates the Image. data ::= integer (0) If the Image transferred has not been verified before, then this is done as part of the Image activation. The result of the invocation of this method may be success, temporary-failure or other-reason. If it is not success, then the result of the activation can be learned by retrieving the value of the image_transfer_status attribute. NOTE 6 xDLMS services Action-Result / Data-Access-Result codes are specified in DLMS UA 1000-2 Ed. 10, 9.5.

4.4.6.4 Example for a standard Image transfer process

In this subclause an example of a usual Image transfer process from a client prospective is given including a flow chart shown in Figure 11. Please note that it is not mandatory to follow the process; depending on the use case some steps may be omitted or others may be necessary.

Precondition: The Image transfer has to be enabled: *image_transfer_enabled* =TRUE.

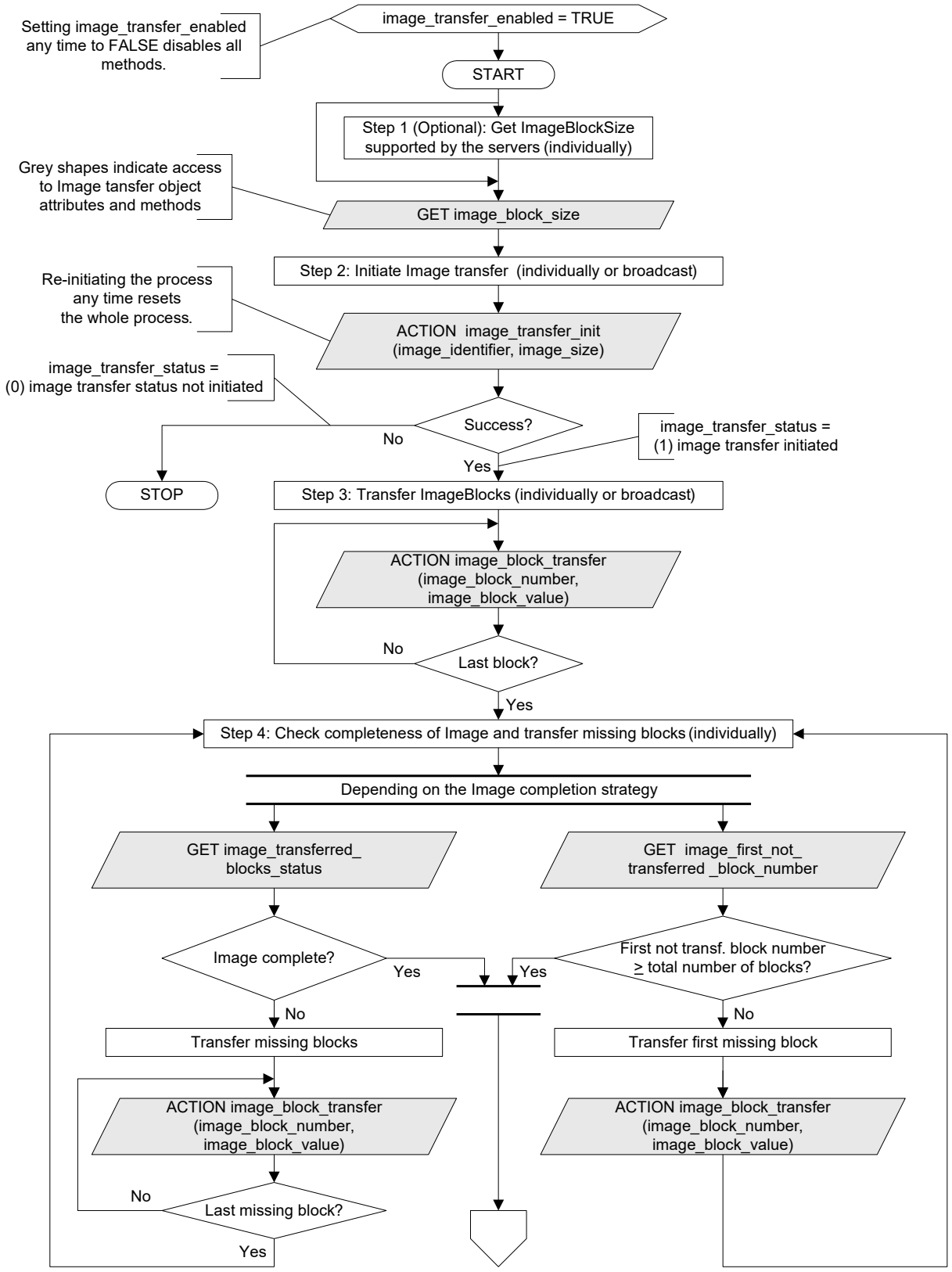
Setting *image_transfer_enabled* any time to FALSE disables all methods (invoking those methods fails). The value of the status attributes is not defined.

Step 1 (Optional): Get ImageBlockSize

If the client does not know the size of the image blocks the image transfer target server can handle, it shall read the *image_block_size* attribute of the relevant “Image transfer” object of each server the image has to be transferred to, before starting the process. The client can transfer then ImageBlocks of the right size.

If ImageBlocks are sent using broadcast to a group of COSEM servers the ImageBlockSize shall be the same in each member of the group.

© Copyright 1997-2020 DLMS User Association



© Copyright 1997-2020 DLMS User Association

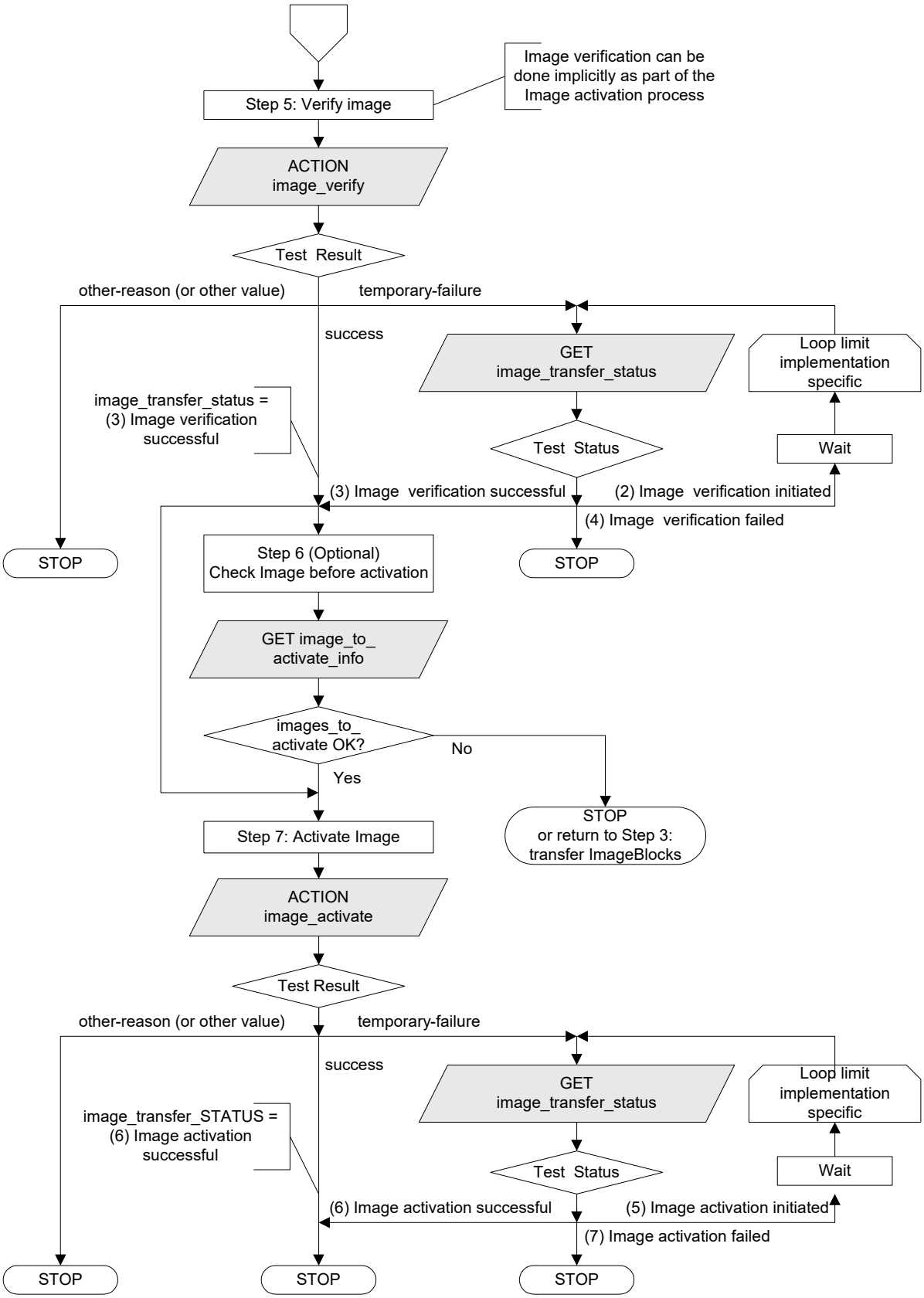


Figure 12 – Image transfer process flow chart

© Copyright 1997-2020 DLMS User Association

Step 2: Client initiates Image transfer

The client initiates the Image transfer process individually or using broadcast in all servers by invoking the *image_transfer_initiate* method. The method invocation parameter holds the identifier and the size of the Image to be transferred. The server shall make the memory space – necessary to accommodate the Image – available.

After a successful initiation, the value of the *image_transfer_status* attribute is (1). The *image_transferred_blocks_status* attribute shall be reset, the value of the *image_first_not_transferred_block_number* attribute shall be set to 0 and the value of the *image_to_activate_info* attribute should be reset. The image transfer process is initiated and the COSEM server is prepared to accept ImageBlocks.

Step 3: Client transfers ImageBlocks

The client transfers ImageBlocks to (a group of) server(s) by invoking the *image_block_transfer* method individually or using broadcast. The method invocation parameters include the ImageBlockNumber and one ImageBlock. ImageBlocks are accepted only by those COSEM servers, in which the Image transfer process has been successfully initiated. Other servers silently discard any ImageBlocks received.

Step 4: Client checks completeness of the Image

The client checks – with each server individually – the completeness of the Image transferred. If the Image is not complete, it transfers the ImageBlocks not (yet) transferred. This is an iterative process, continued until the whole Image is successfully transferred.

To identify and transfer the ImageBlocks not transferred, two mechanisms are available.

- the client may retrieve the status of each ImageBlock: either not transferred or transferred. This is performed by retrieving the value of the *image_transferred_blocks_status* attribute. The client transfers then the ImageBlocks not (yet) transferred;
- alternatively, the client may retrieve the ImageBlockNumber of the first block not transferred. This is performed by retrieving the value of the *image_first_not_transferred_block_number* attribute. The client transfers then this ImageBlock not (yet) transferred;
- after this, the client checks again he completeness of the Image.

NOTE 1 The two mechanisms can be freely combined.

Step 5: Server verifies the Image

The Image is verified by the server. This can be initiated by invoking the *image_verify* method by the client or it may be initiated also by the server. The result can be:

- success, if the verification could be completed;
- temporary-failure, if the verification has not been completed;
- other-reason, if the verification failed.

NOTE 2 The conditions of verifying the Image are out of the scope of this document.

Step 6 (Optional): Client checks the information on the images to activate

The result of the Image verification may be checked by the client by retrieving the value of the *image_transfer_status* attribute. The value of this attribute is updated as a result of the Image verification.

An Image transferred may hold one or more Images to be activated. For each Image to be activated the image_to_activate_info attribute holds the parameters: {image_to_activate_size, image_to_activate_identification, image_to_activate_signature}.

© Copyright 1997-2020 DLMS User Association

If this information is not what is expected, the client can restart transferring the image.

Otherwise it goes to the next step, activation of the Image

Step 7: Server activates the Image(s)

The Image is activated by the server. This can be initiated by invoking the *image_activate* method by the client or it may be initiated also by the server. If the activation is done without a previous verification, then verification is done implicitly as part of the activation. The result of the invocation of the *Image_activate* method can be:

- success, if the Image activation has been successfully started;
- temporary-failure, if the verification/activation has not been completed;
- other-reason, if the activation failed.

In the case of success, the server performs the activation of the new Image(s). During this process, it is not accessible. After the Image(s) has (have) been activated, the result may be checked by the client by retrieving the value of the *image_transfer_status* attribute or by reading the contents of the appropriate COSEM objects holding the identifier, version and digital signature of the active firmware.

4.4.7 Security setup (class_id = 64, version = 1)

Instances of the “Security setup” IC contain the necessary information on the security suite in use and the security policy applicable between the server and a client and/or third party identified by their respective system titles. They also provide methods to increase the level of security and to manage symmetric keys, asymmetric key pairs and Certificates.

Security setup		**0...n**	**class_id = 64, version = 1**			
Attributes		**Data type**	**Min.**	**Max.**	**Def.**	**Short name**
1. logical_name	(static)	octet-string				x
2. security_policy	(static)	enum				x + 0x08
3. security_suite	(static)	enum				x + 0x10
4. client_system_title	(dyn.)	octet-string				x + 0x18
5. server_system_title	(static)	octet-string				x + 0x20
6. certificates	(dyn.)	array				x + 0x28
Specific methods		m/o				
1. security_activate (data)		**o**				x + 0x30
2. key_transfer (data)		**o**				x + 0x38
3. key_agreement (data)		**o**				x + 0x48
4. generate_key_pair (data)		**o**				x + 0x50
5. generate_certificate_request (data)		**o**				x + 0x58
6. import_certificate (data)		**o**				x + 0x60
7. export_certificate (data)		**o**				x + 0x68
8. remove_certificate (data)		**o**				x + 0x70

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “Security setup” object instance. See 6.2.36.
security_policy	Enforces authentication and/or encryption and/or digital signature using the security algorithms available within the security suite. It applies independently for requests and responses. enum: When the enum value is interpreted as an unsigned, the meaning of each bit is as shown below: Bit Security policy 0 unused, shall be set to 0, 1 unused, shall be set to 0, 2 authenticated request, 3 encrypted request, 4 digitally signed request, 5 authenticated response, 6 encrypted response, 7 digitally signed response NOTE 1 With this, the value (0) means that no cryptographic protection is required, as in version 0 of the “Security setup” IC. The access rights may require stronger protection than what is required by the security policy.
security_suite	Specifies the security algorithms available. enum: (0) AES-GCM-128 authenticated encryption and AES-128 key wrap, (1) AES-GCM-128 authenticated encryption, ECDSA P-256 digital signature, ECDH P-256 key agreement, SHA-256 hash, V.44 compression and AES-128 key wrap, (2) AES-GCM-256 authenticated encryption, ECDSA P-384 digital signature, ECDH P-384 key agreement, SHA-384 hash, V.44 compression and AES-256 key wrap, (3) ... (15) reserved
client_system_ title	Carries the (current) client system title: - in the S-FSK PLC environment, the active initiator sends its system title using the CIASE protocol; NOTE 2 It is also held by the active_initiator attribute of the S-FSK Active initiator object; see 4.10.4. - during confirmed or unconfirmed AA establishment, it is carried by the calling-AP-title field of the AARQ APDU; If a client system title has already been sent during a registration process, like in the case of the S-FSK PLC profile, the client system title carried the AARQ APDU should be the same. Otherwise, the AA shall be rejected and appropriate diagnostic information should be sent; - in a pre-established AA, it can be written by the client using an unsecured service.

© Copyright 1997-2020 DLMS User Association

server_system_ title	Carries the server system title. - in the S-FSK PLC environment, the server sends its system title during the discover process, using the CIASE protocol; - during confirmed AA establishment, it is carried by the responding-AP-title field of the AARE APDU. This attribute shall be read only.

certificates	Carries information on the X.509 v3 Certificates available and stored in the server.
	array certificate_info
	
	certificate_info ::= structure
	{
	certificate_entity: enum:
	(0) server,
	(1) client or third party,
	(2) certification authority,
	(3) other
	certificate_type: enum:
	(0) digital signature,
	(1) key agreement,
	(2) TLS,
	(3) other
	serial_number: octet-string,
	issuer: octet-string,
	subject: octet-string,
	subject_alt_name: octet-string
	}
	For the elements serial_number, issuer, subject, subject_alt_name see DLMS UA 1000-2 Ed. 10, 9.2.6.4.
	The octet-strings that hold the serial_number, issuer, subject, and subject_alt_name shall include the tag, length and value (TLV) of the corresponding field of the Certificate.
	A server that uses public key cryptography stores zero or more of the following public key Certificates:
	
	For the server:
	- Digital Signature Certificate,
	- Static Key Agreement Certificate,
	- TLS Certificate.
	
	For each client and third party:
	- Digital Signature Certificate,
	- Static Key Agreement Certificate,
	- TLS Certificate.
	
	For Certification Authorities:
	- Certification Authority Certificate.

© Copyright 1997-2020 DLMS User Association

Methods description	
security_activate (data)	Activates and strengthens the security policy: data ::= enum, as specified at the *security_policy* attribute. Strengthening the security policy means that another kind of protection is added. Once a kind of protection is added, it cannot be removed using this method. If the method is invoked with a value indicating a security policy that is weaker than the value of the security_policy attribute, the method invocation fails. The new security policy applies as soon as the method invocation has been confirmed with success.

key_transfer (data)	Used to transfer one or more symmetric keys.
	
	The *data* parameter includes identification of the key(s) transported and the wrapped key(s) themselves.
	
	data ::= array key_transfer_data
	
	key_transfer_data ::= structure
	{
	key_id: enum:
	(0) global unicast encryption key (GUEK),
	(1) global broadcast encryption key (GBEK),
	(2) authentication key (GAK),
	(3) master key (KEK)
	
	key_wrapped: octet-string
	}
	
	NOTE 3 It is possible to transfer multiple keys by invoking the method with an array of *n key_transfer data*, or to invoke the method *n* times with an array of a single *key_transfer_data.*
	
	The key wrap algorithm is as specified by the security suite. In suites 0, 1, and 2 the AES key wrap algorithm is used.
	
	The KEK is the master key.
	If the unwrapping of the key(s) is successful, the result of the method invocation is success.
	If the unwrapping of the key(s) is not successful, the method invocation fails and an appropriate reason for the failure shall be sent back. The keys are not changed.
	The new keys are activated immediately after result of the method invocation is sent back with result = success. Notice that this rule equally applies to all keys, including the master key.

© Copyright 1997-2020 DLMS User Association

key_transfer (data) (continued)

NOTE 4 The APDUs carrying the method invocation service primitives are protected as stipulated by the *security_policy* and the access rights to the methods. The method invocation parameters can be additionally protected by invoking the method through a "Data protection" object. The required protection can be specified in project specific companion specifications.

This note equally applies to the *key_agreement (data)* method.

NOTE 5 If using the new keys fails, the client may try to recover from this situation by reverting to using the previous keys or repeating the key transfer process.

key_agreement (data)

Used to agree on one or more symmetric keys using the key agreement algorithm as specified by the security suite. In the case of suites 1 and 2 the ECDH key agreement algorithm is used with the Ephemeral Unified Model C(2e, 0s, ECC CDH) scheme.

The *data* parameter includes the identification of the key(s) and data used in the key agreement transaction.

data ::= array key_agreement_data

key_agreement_data ::= structure
{
  key_id: enum:
    (0) global unicast encryption key,
    (1) global broadcast encryption key,
    (2) authentication key,
    (3) master key (KEK)
  key_data: octet-string
}

NOTE 6 It is possible to agree on multiple keys by invoking the method with an array of *n key_agreement_data*, or to invoke the method *n* times with an array of a single *key_agreement_data*.

The element key_id identifies the key(s) to be agreed on.

The element *key_data* is the public key of ephemeral key pair right concatenated with digital signature, which is calculated over the key_id value and public key of ephemeral key pair value using the digital signature private key: $Q_{e,U,}$ ECDSA(key_id II $Q_{e,U}$).

If the server could verify the digital signature of *key_data*, compute the shared secret and derive the key, then the method invocation is successful and the server sends its own *key_data* to the client. Otherwise, the method invocation fails and a reason for the failure is sent back.

The new keys are activated immediately after result of the method invocation is sent back with result = success.

NOTE 7 If using the new keys fails, the client may try to recover from this situation by reverting to using the previous keys or repeating the key transfer process.

© Copyright 1997-2020 DLMS User Association

generate_key_pair (data)	Generates an asymmetric key pair as required by the security suite. The data parameter identifies the usage of the key pair to be generated. data ::= enum: (0) digital signature key pair, (1) key agreement key pair, (2) TLS key pair NOTE 8 There is maximum one key pair for digital signature, one key pair for key agreement and one key pair for TLS. NOTE 9 Key agreement key pair is the static key used with the One-Pass Diffie-Hellman C(1e, 1s, ECC CDH) scheme when the server takes the role of Party V or with the Static Unified Model C(0e, 2s, ECC CDH) scheme.
generate_certificate_request (data)	When this method is invoked, the server sends the Certificate Signing Request (CSR) data that is necessary for a CA to generate a Certificate for a server public key. The data parameter identifies the key pair for which the Certificate will be requested. data ::= enum: (0) digital signature key pair, (1) key agreement key pair, (2) TLS key pair NOTE 10 There is maximum one key pair for digital signature, one key pair for key agreement and one key pair for TLS. The method invocation response parameters include the data necessary to generate the Certificate. NOTE 11 It is the responsibility of the client to forward it to the Certification Authority. response data ::= octet-string, formatted as specified in PKCS #10 (RFC 2986). The CSR shall be signed by the private key belonging to the public key in the request. This acts as the “proof of possession“ of the private key.
import_certificate (data)	Imports an X.509 v3 Certificate of a public key. data ::= octet-string Data is formatted as DER encoded X.509 v3 (RFC 5280).
export_certificate (data)	Exports an X.509 v3 Certificate in the server. Certificate is identified with entity identification or the serial number of the Certificate.

© Copyright 1997-2020 DLMS User Association

export_certificate (data)
(continued)
	data ::= certificate_identification
	
	certificate_identification ::= structure
	{
	certificate_identification_type: enum:
	(0) certificate_identification_entity,
	(1) certificate_identification_serial
	
	certification_identification_options: CHOICE
	{
	certificate_identification_by_entity,
	certificate_identification_by_serial
	}
	}
	certificate_identification_by_entity ::= structure
	{
	certificate_entity: enum:
	(0) server,
	(1) client,
	(2) certification authority,
	(3) other
	certificate_type: enum:
	(0) digital signature,
	(1) key agreement,
	(2) TLS;
	(3) other
	
	system_title: octet-string
	}
	
	certificate_identification_by_serial ::= structure
	{
	serial_number: octet-string,
	issuer: octet-string
	}
	
	response data ::= octet-string
	response data octet-string is formatted as X.509 v3 DER format.
	
	If no matching Certificate is found, the response shall contain an appropriate error code.

remove_certificate (data)	Removes X.509 v3 Certificate in the server. data ::= certificate_identification See export_certificate for certificate_identification. NOTE 12 The Certificates of the server for digital signature, key agreement and TLS cannot be removed from the server. When update of these Certificates is needed the new key pair is generated, new Certificate Signing Request is generated and new certificate is imported.

4.4.8 Push interface class (class_id = 40, version = 2)

4.4.8.1 Overview

DLMS messages can be ‘pushed’ to a destination without being explicitly requested. The push is initiated by a trigger such as.:

© Copyright 1997-2020 DLMS User Association

- a scheduled time is reached;
- a value being locally monitored exceeds a threshold;
- a local event (e.g. power-up/down, push button pressed, meter cover opened).

The DLMS/COSEM push mechanism follows the publish/subscribe pattern.

Publishing is modelled in DLMS/COSEM by the *object_list* attribute of "Association" objects which provides the list of COSEM objects and their attributes accessible in a given AA. Subscription is modelled by writing the appropriate attributes of "Push setup" objects. The required data are sent using the xDLMS DataNotification service when the specified trigger is activated.

The COSEM model of the Push operation is shown in Figure 13.

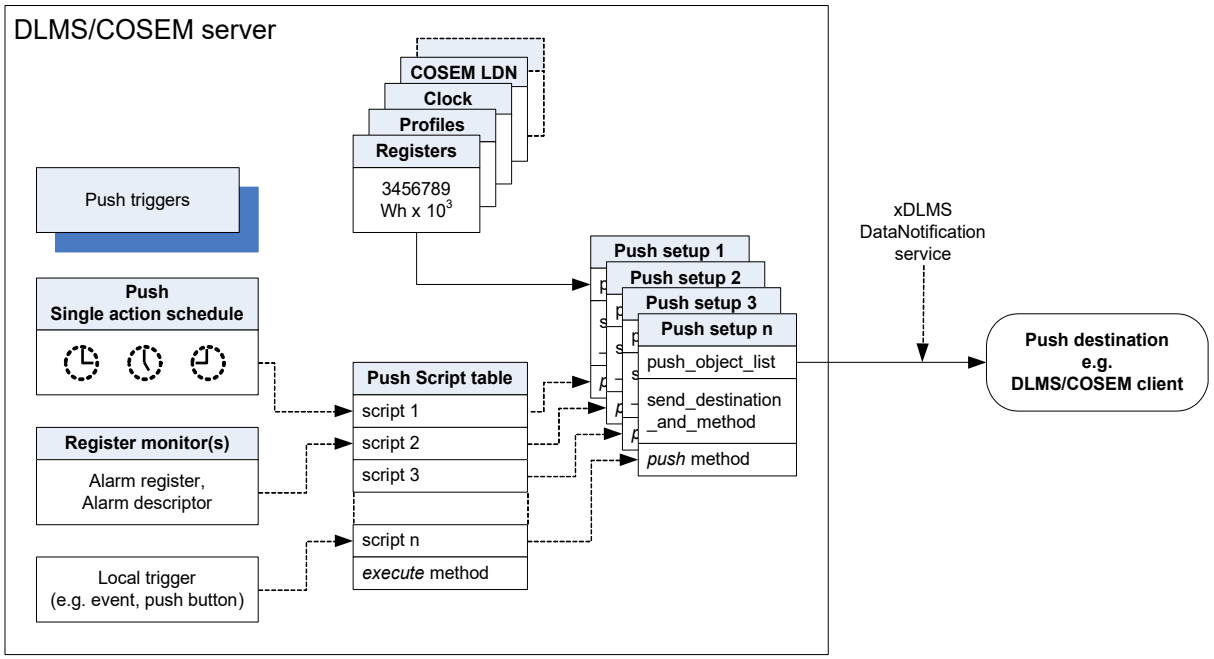


Figure 13 – COSEM model of push operation

The core element of modelling the push operation is the "Push setup" IC. The *push_object_list* attribute contains a list of references to COSEM object attributes to be pushed.

When push uses a gateway, then the version of the gateway protocol for end devices without WAN/NN knowledge is to be applied. This is described in the DLMS UA 1000-2 Ed. 10, 10.9.4.3.

The various triggers (e.g. schedulers, monitors, local triggers etc.) call a script entry in a Push "Script table" object (instances of the "Script table" IC) which invokes then the *push* method of the related "Push setup" object. The destination, the communication media, the protocol, the encoding, the timing as well as any retries of the push operation are determined by the other attributes of the "Push setup" object.

Each trigger can cause the data to be sent to a dedicated destination. Therefore, for every trigger or a group of triggers an individual "Push setup" object is available defining the content and the destination of the push message as well as the communication medium used.

For the purposes of pushing data when an alarm occurs, Alarm monitor objects (instances of the "Register monitor" IC) are available.

© Copyright 1997-2020 DLMS User Association

The alarms are held by *Alarm register* or *Alarm descriptor* objects, see 6.2.64.

NOTE 1 The structure of the *Alarm descriptor* as well as the alarm conditions need to be defined in a project specific companion specification.

Alarm descriptor objects are monitored by Alarm "Register monitor" objects, see 6.2.13. The *actions* attribute provides the link to the action_up and maybe the action_down scripts in the Push "Script table" object – see 6.2.7 – which invoke then the *push* method of the desired "Push setup" object. When an alarm occurs, the predefined set of data – that may include among other data the *Alarm register* and *Alarm descriptor* objects – are pushed.

The push data is sent – when the conditions for pushing are met – using the unsolicited, non-client/server type xDLMS service, the DataNotification service. When the data pushed is long, it can be sent in blocks.

The push process takes place within the application context of the AA which is referenced by the *push_client_SAP* attribute of the "Push setup" object. The security context is determined by the "Security setup" object referenced from the "Association" object. The necessary data protection parameters are defined in attribute *push_protection_parameters* which offers the same options as defined in the "Data Protection" IC.

All information necessary to process the data received by the client shall be either:

- retrieved by the client e.g. by reading the *push_object_list attribute*; or
- shall be part of the data pushed; or
- shall be predefined in the client application.

4.4.8.2 Push setup (class_id = 40, version = 2)

The Push setup IC contains a list of references to COSEM object attributes to be pushed. It also contains the push destination and method as well as the communication time windows and the handling of retries.

In version 1 the possibility of data protection was added offering the same options as defined in the Data protection IC.

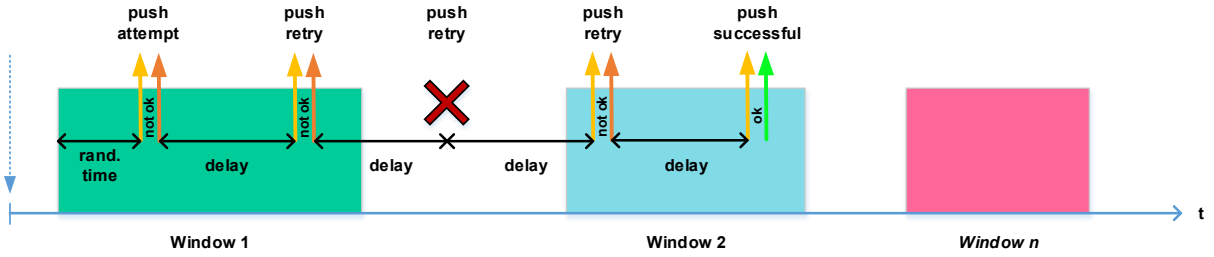
In version 2 six new possibilities are specified:

- a new entry selection mechanism allows selecting entries related to the last confirmed entry;
- a new column selection mechanism allows explicitly selecting columns of Profile generic object buffer attributes;
- the **repetition_delay** attribute now provides an exponential formula that allows increasing the repetition delay with each retry attempt;
- a new **push_operation_method** attribute allows specifying if the DataNotification.request service primitive is invoked with Service_Class == Unconfirmed or confirmed and how the push retry operation works;
- a new **confirmation_parameters** attribute allows limiting the time interval in which entries related to last confirmed entry can be selected;
- a new **last_confirmation_date_time** attribute holds the date_time when the DataNotification service was last confirmed.

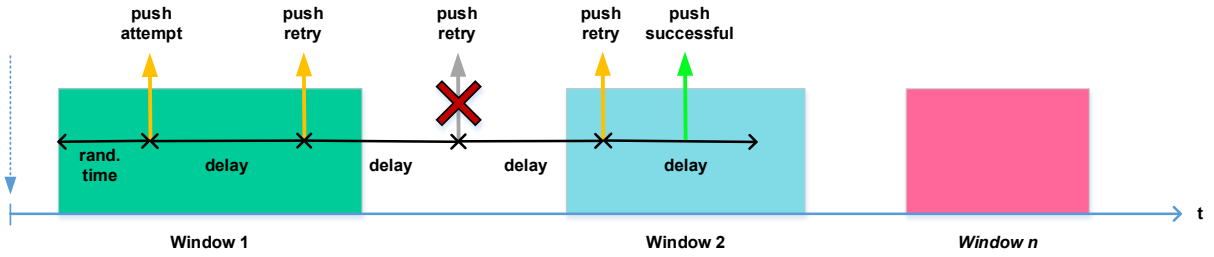
This version of the interface class is intended to facilitate the use of relative and absolute data selection. It would be common practice to use an instance of the Push setup IC with relative data selection for routine data push, and an instance of the Push setup IC with absolute data for special cases.

© Copyright 1997-2020 DLMS User Association

The push takes place upon invoking the push method, triggered by a Push Single action schedule object, by an Alarm Register monitor object, by a dedicated internal event or externally. After the push operation has been triggered, it is executed according to the settings made in the given Push setup object. Depending on the communication window settings, the push is executed immediately or as soon as a communication window becomes active, after a random delay. If the push was not successful, retries are made. Push retries may be made when supporting protocol layer failure is indicated or when confirmation is missing. Push operation with windows, delays and retries is shown in Figure 14.



Case a) Retry on supporting layer failure



Case b) Retry on missing confirmation

Figure 14 – Push windows, delays and retries

© Copyright 1997-2020 DLMS User Association

Push setup		0...n	class_id = 40, version = 2			
Attribute (s)		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. push_object_list	(static)	array				x + 0x08
3. send_destination_and_method	(static)	structure				x + 0x10
4. communication_window	(static)	array				x + 0x18
5. randomisation_start_interval	(static)	long-unsigned				x + 0x20
6. number_of_retries	(static)	unsigned				x + 0x28
7. repetition_delay	(static)	structure				x + 0x30
8. port_reference	(static)	octet-string				x + 0x38
9. push_client_SAP	(static)	integer				x + 0x40
10. push_protection_parameters	(static)	array				x + 0x48
11. push_operation_method	(static)	enum				x + 0x50
12. confirmation_parameters	(static)	structure				x + 0x58
13. last_confirmation_date_time	(dyn.)	date-time				x + 0x60
Specific methods		***m/o***				
1. push (data)		m				x + 0x68
2. reset (data)		o				x + 0x70

Attribute description

logical_name Identifies the “Push setup” object instance. See 6.2.24.

push_ object_list Defines the list of attributes to be pushed.

Upon invocation of the push (data) method the items are sent to the destination defined in the send_destination_and_method attribute.

```
array        push_object_definition
push_object_definition ::=     structure
{
                class_id:              long-unsigned,
                logical_name:          octet-string,
                attribute_index:       integer,
                data_index:            long-unsigned,
                restriction:           restriction_element,
                column:                column_element
}

      restriction_element::= structure
      {
            restriction_type:   enum:
                  (0) none,
                  (1) restriction by date,
                  (2) restriction by entry
            restriction_value: CHOICE
            {
```


© Copyright 1997-2020 DLMS User Association

```
                null-data, // no restrictions apply
                restriction_by_date,
                restriction_by_entry
            }
        }
        restriction_by_date ::= structure
        {
            from_date:  octet-string,
            to_date:    octet-string
        }
        restriction_by_entry ::= structure
        {
            from_entry: double-long-unsigned,
            to_entry:   double-long-unsigned
        }
        column_element ::= array capture_object_definition
```

Where:

- class_id, logical_name and attribute_index reference a COSEM object attribute. For referencing all attributes of an object, see DLMS UA 1000-2 , 9.1.4.3.7, attribute_0 referencing.

If the attribute is simple or if attribute_0 is referenced, then:

- data_index has no meaning and shall be set to 0x0000;
- restriction_element restriction_type shall be 0 (none) and restriction_element restriction_value shall be null-data;
- column_element shall be an array of 0 elements.

If the attribute is structure or an array – other than Profile generic object buffer – then:

- data_index points to one element in the structure or array and can take any value between 0x0001 and 0x0FFF;
- restriction_element restriction_type shall be 0 (none) and restriction_element restriction_value shall be null-data;
- column_element shall be an array of 0 elements.

If the attribute is a Profile generic object buffer, then data_index, restriction and column define selective access parameters to select entries and columns.

There are three, mutually exclusive entry selection mechanisms available:

1) Relative selective access related to current date and time. Entries in last time intervals relative to current date and time or last entries as identified by data_index are selected;
2) Relative selective access related to last confirmed entry. Entries in next time intervals relative to last confirmed entry or next entries as identified by data_index are selected;
3) Absolute selective access. Entries in an explicitly defined date range or entry range as identified by restriction are selected.

© Copyright 1997-2020 DLMS User Association

There are two, mutually exclusive column selection mechanisms available:

a) a defined number of columns starting from column 1 are selected;

b) an explicitly defined list of columns are selected.

In the case of entry selection mechanism (1) and (2) the selective access parameters are defined by data_index interpreted as three different fields as shown below. The encoding is shown in Table 6.

data_index	MS-Byte		LS-Byte
	Upper nibble	Lower nibble	

Where:

– data_index MS-Byte Upper nibble defines how the entries are selected;

– data_index MS-Byte Lower nibble is used to specify the number of columns when applicable;

– data_index LS-Byte defines the number of entries or time intervals.

In the case of entry selection mechanism (1) and (2), restriction_element restriction_type shall be 0 and restriction_element restriction_value shall be null_data.

In the case of entry selection mechanism (3) the selective access parameters are defined by restriction_element. In this case Data_index MS_byte Upper_nibble shall be set to 0x0, and Data_index LS_byte shall be set to 0x00.

- for restriction by date range restriction_type holds (1) range by date and restriction_value holds restriction_by_date structure;
- for restriction by entries restriction_type holds (2) range by entry and restriction_value holds restriction_by_entry structure;
- if selective access is not required, restriction_type holds (0) none and restriction_value holds null-data. This choice shall be taken also if relative selective access is used.

In the case of column selection mechanism (a), Data_index MS_byte Lower_nibble defines the number of columns to be selected, starting from column 1. In this case, column_element shall be an empty array.

In the case of column selection mechanism (b), the columns to be selected are identified by column_element. In this case, Data_index MS_byte Lower_nibble shall be set to 0.

If relative selective access related to last confirmed entry is used, the last confirmed entry is updated when the DataNotification.confim service primitive is invoked with Result == CONFIRMED.

The DataNotification APDU carrying the push data is protected as stipulated by the security suite, security policy and the security material held by the “Security setup” object referenced from the Association SN / Association LN object within

© Copyright 1997-2020 DLMS User Association

the application context of the AA which is linked to the push_client_SAP attribute; as well as by the push_potection_parameters attribute. For attributes to which no access right is granted within this AA, or which cannot be accessed for any other reason, null-data should be returned.

NOTE 1 If the push_object_list array is empty, the push operation is disabled.

NOTE 2 The push_object_list attribute itself can be also pushed to identify the data pushed.

NOTE 3 Last confirmed entry is an internal variable maintained by the server AP.

send_ destination_ and_method

Contains the destination address and transport service (e.g. phone number, email address, IP address) where the data specified by the *push_object_list* has to be sent, as well as the sending method.

send_destination_and_method ::= structure

```
{
    transport_service:    transport_service_type,
    destination:          octet-string,
    message:              message_type,
}
```

Where:

- The transport_service element defines the type of service used to push the data:

  transport_service_type ::= enum:

  (0) TCP,
  (1) UDP,
  (2) reserved for FTP,
  (3) reserved for SMTP,
  (4) SMS,
  (5) HDLC,
  (6) M-Bus,
  (7) reserved for ZigBee ®,
  (8) DLMS Gateway
  (200…255) manufacturer specific

- the destination element contains the target address where the data has to be sent. The elements of the target address depend on the transport service used.

  Each “Push setup” object instance specifies a single destination. If it is required to push data to several destinations, several “Push setup” objects have to be instantiated.

- The message element identifies the encoding of the xDLMS APDU used.

  message_type::= enum:

  (0) A-XDR encoded xDLMS APDU,
  (1) XML encoded xDLMS APDU,
  (128…255) manufacturer specific

- All other transport_service_type and message_type values are reserved for future use.

communication_ window

Defines the time points when the communication window(s) for the push become(s) active (start_time) and inactive (end_time). See Figure 14.

array window_element

window_element ::= structure

{

© Copyright 1997-2020 DLMS User Association

	start_time: octet-string, end_time: octet-string } start_time and end_time are formatted as specified in clause 4.1.6.1 for date_time including wildcards. If the end of a communication window is reached when a push has already started the operation will be completed. If no communication windows are defined (array [0]) the push operation is always possible.
randomisation_ start_ interval	Defines a maximum delay, in seconds, of sending the first DataNotification APDU after invoking the push method. This is to avoid a situation where many servers may push simultaneously. It is used as follows: • the delay is random from 0 up to the maximum value. A value of 0 deactivates the mechanism; • if the push is invoked within an active communication window, the random delay is applied. It is not applied for retries; • if the push is invoked outside a communication window, then the random delay is applied when the communication windows opens; • if the random delay extends beyond the current communication window, then the DataNotification APDU will be sent in the next window if available; • if no communication windows are defined, then the random delay is applied whenever the push method is invoked including all retries.
number_of_ retries	Defines the maximum number of retries in the case of unsuccessful or skipped push attempts. After a successful push operation, no further push attempts are made until the push operation is triggered again. For further details on the retransmission operation see the **push_operation_method** attribute.

repetition_ delay	Defines the elements for calculating the time delay until the next attempt after an unsuccessful push. See Figure 14.
	
	repetition_delay ::= structure
	{
	repetition_delay_min: long-unsigned,
	repetition_delay_exponent: long-unsigned,
	repetition_delay_max long-unsigned
	}
	Where:
	• repetition_delay_min is the minimum delay, in seconds, until a next push attempt is started;
	• repetition_delay_exponent is used for calculating the next delay;
	• repetition_delay_max is the maximum delay, in seconds. Any calculated time delay will be capped by this value.
	
	The repetition delay is calculated using the following formula:
	
	$repetition_delay = repetition_delay_min \times (repetition_delay_exponent \times 0.01)^{n-1}$
	
	where n refers to the ordinal number of the push retries with n = 1 for the first retry etc.
	NOTE 4 The repetition delay itself is not influenced by the communication window. But a push retry only can be made if a communication window is active at that time. Otherwise it is handled like an unsuccessful push attempt.
	NOTE 5 The push data is not stored in an intermediate buffer. In the case of push retries, the current values of the attributes may change with every push retry attempt.

© Copyright 1997-2020 DLMS User Association

port_reference	Contains the logical name of a communication port setup object allowing the selection of a specific communication channel for the push based on the transport_service_type. This mainly applies in cases where several channels of the same type are supported. If this information is not available or not needed, the attribute may be left empty (octet-string [0]).
push_ client_SAP	Defines the client SAP where the push is directed to in the supporting layer of the DataNotification service. The push process takes place within the application context of the AA which is linked to the push_client_SAP attribute. The security context is determined by the Security setup object referenced in the related Association SN /LN object.

push_protection_ parameters	Specifies all protection parameters to be applied to the DataNotification APDU when the push data is sent. It offers the same options as the Data Protection IC but it is bound to the sending of the data defined in the push_object_list attribute.
	array protection_parameters_element
	
	protection_parameters_element ::= structure
	{
	protection_type: enum:
	(0) authentication,
	(1) encryption,
	(2) authentication and encryption,
	(3) digital signature
	
	protection_options: structure
	{
	transaction_id: octet-string,
	originator_system_title: octet-string,
	recipient_system_title: octet-string,
	other_information: octet-string,
	key_info: key_info_element
	}
	}
	Where:
	– transaction_id holds the identifier of the transaction;
	– originator_system_title holds the system title of the originator that applies the protection;
	– recipient_system_title holds the system title of the recipient which will check and remove the given protection;
	– other_information carries other information. Its contents may be specified in project specific companion specifications. An octet-string of length 0 indicates that this field is not used;
	– key_info holds the information necessary for the recipient to obtain the right key for checking and removing authentication and encryption. In the case of digital signature, key_info is not necessary and it shall be a structure of 0 elements.

© Copyright 1997-2020 DLMS User Association

The fields transaction-id….other-information are A-XDR encoded OCTET STRINGs. The length and the value of each field are included in the AAD when applicable.

```
key_info_element ::= structure
{
    key_info_type: enum:
                    (0) identified_key,
                    -- used with identified_key_info_options
                    (1) wrapped_key,
                    -- used with wrapped_key_info_options
                    (2) agreed_key
                    -- used with agreed_key_info_options
    key_info_options: CHOICE
    {
            identified_key_info_options,
            wrapped_key_info_options,
            agreed_key_info_options
    }
}
identified_key_info_options ::= enum:
                                (0) global_unicast_encryption_key,
                                (1) global_broadcast_encryption_key
wrapped_key_info_options ::= structure
{
    kek_id:     enum:
                    (0) master_key
    key_ciphered_data: octet-string
}
agreed_key_info_options ::= structure
{
    key_parameters: octet-string,
    key_ciphered_data: octet-string
}
```

This attribute is first written by the client. The server may need to fill in some additional elements.

The use of the various elements is the same as specified in Table 17 and Table 18 of the Data protection IC (class_id = 30, version = 0).

push_operation_method	Defines if the DataNotification.request service primitive is invoked with Service_Class == Unconfirmed or Confirmed and how the push retry operates. enum: (0) unconfirmed, retry on supporting protocol layer failure, (1) unconfirmed, retry on missing supporting protocol layer confirmation, (2) confirmed, retry on missing confirmation. In case (0), the repetition delay for the next retry attempt is started upon supporting layer failure reported through the DataNotification.confirm service primitive.

© Copyright 1997-2020 DLMS User Association

	In cases (1) and (2), the repetition delay for the next retry attempt is started when the DataNotification.request service primitive is invoked. In cases (0) and (1), the push operation is deemed as successful when supporting layer confirmation is reported by invoking DataNotification.confim service primitive with Result == CONFIRMED. In case (2), the push operation is deemed as successful when the server AL receives the data-notification-confirm APDU and invokes DataNotification.confirm service primitive with Service_Class == Confirmed and Result == CONFIRMED.

confirmation_ parameters	Defines the selection of entries defined by data_index to avoid pushing data from too far in the past. If entries have not yet been confirmed then all entries are selected. This attribute is applicable only for relative selective access related to last confirmation.
	
	confirmation_parameters ::= structure
	{
	confirmation_start_date: date-time,
	confirmation_interval: double-long-unsigned
	}
	
	Where:
	- confirmation_start_date is the starting date and time for selecting entries. Fields of date-time not specified are not used. If all fields are not specified the use of confirmation_start_date is disabled;
	- confirmation_interval is a time interval in seconds backwards from the current date and time. If confirmation_interval is set to zero, the use of confirmation_interval is disabled.

last_confirmation_ date_time	Holds the date and time when the AL most recently invoked the DataNotification.confirm service primitive with Result == CONFIRMED. *date-time* is formatted as specified in 4.1.6.1.
Method description	
push (data)	Activates the push process leading to an attempt to send a DataNotification APDU carrying the push data. data ::= integer(0)
reset (data)	Resets the push process to initial state. data ::= integer(0)

4.4.9 COSEM data protection (class_id = 30, version = 0)

4.4.9.1 Overview

Instances of this IC allow applying cryptographic protection on COSEM data i.e. on attribute values and method invocation and return parameters. This is achieved by accessing attributes and/or methods of other COSEM objects indirectly through instances of the “Data protection” interface class that provide the necessary mechanisms and parameters to apply / verify / remove protection on COSEM data.

NOTE 1 “Accessing” includes reading / writing / capturing / pushing COSEM object attributes or invoking methods.

© Copyright 1997-2020 DLMS User Association

NOTE 2 When attributes and methods of COSEM objects are accessed directly, protection can be provided by protecting the xDLMS APDUs as stipulated by the relevant security policy and the access rights.

NOTE 3 For definitions and abbreviations related to cryptographic security see DLMS UA 1000-2 Ed. 10, Clause 3.

Protection on COSEM data is aligned with and complements protection on xDLMS APDUs as defined in DLMS UA 1000-2 Ed. 10, Clause 9.

The use cases for COSEM data protection include, but are not limited to:

- reading or writing a pre-defined set of protected attribute values;
- storing a pre-defined set of protected attribute values in “Profile generic” objects for later retrieval;
- pushing a pre-defined set of protected attribute values;
- reading or writing selected attributes of other COSEM objects with protection;
- invoking a method of another COSEM object with protected method invocation and return parameters.

Protection may comprise any combination of authentication, encryption and digital signature and can be applied in a layered manner. The parties applying and removing the protection are the DLMS/COSEM server and another identified party, which may be a DLMS/COSEM client or a third party.

Applying data protection between a DLMS/COSEM server and a third party allows keeping critical / sensitive data confidential towards the client through which the third party accesses the server. Signing COSEM data by a third party supports non-repudiation.

For end-to-end protection between third parties and servers, see also DLMS UA 1000-2 Ed. 10, 4.7 and 9.2.2.5.

The protection parameters are always controlled by the client with some elements filled in by the server as appropriate.

The security suite is determined by the “Security setup” object referenced from the current “Association SN” / “Association LN” object.

Figure 14 shows the COSEM model of data protection and the relationship of a “Data protection” object with other COSEM objects.

For accessing attributes of other COSEM objects with protected data, there are two mechanisms available:

- reading or writing the *protection_buffer* attribute. The *protection_buffer* can be also captured in “Profile generic” objects or pushed using “Push setup” objects;
- invoking the get_protected_attributes / set_protected_attributes method.

For accessing a method of another COSEM object with protected data, the *invoke_protected_method* method is available.

APDUs carrying service invocations to access attributes and methods of “Data protection” objects are protected as stipulated by access rights to these attributes and methods, and by “Security setup” *security_suite* and *security_policy*.

© Copyright 1997-2020 DLMS User Association

The master key and Certificates – as required by the security suite – are held by "Security setup".

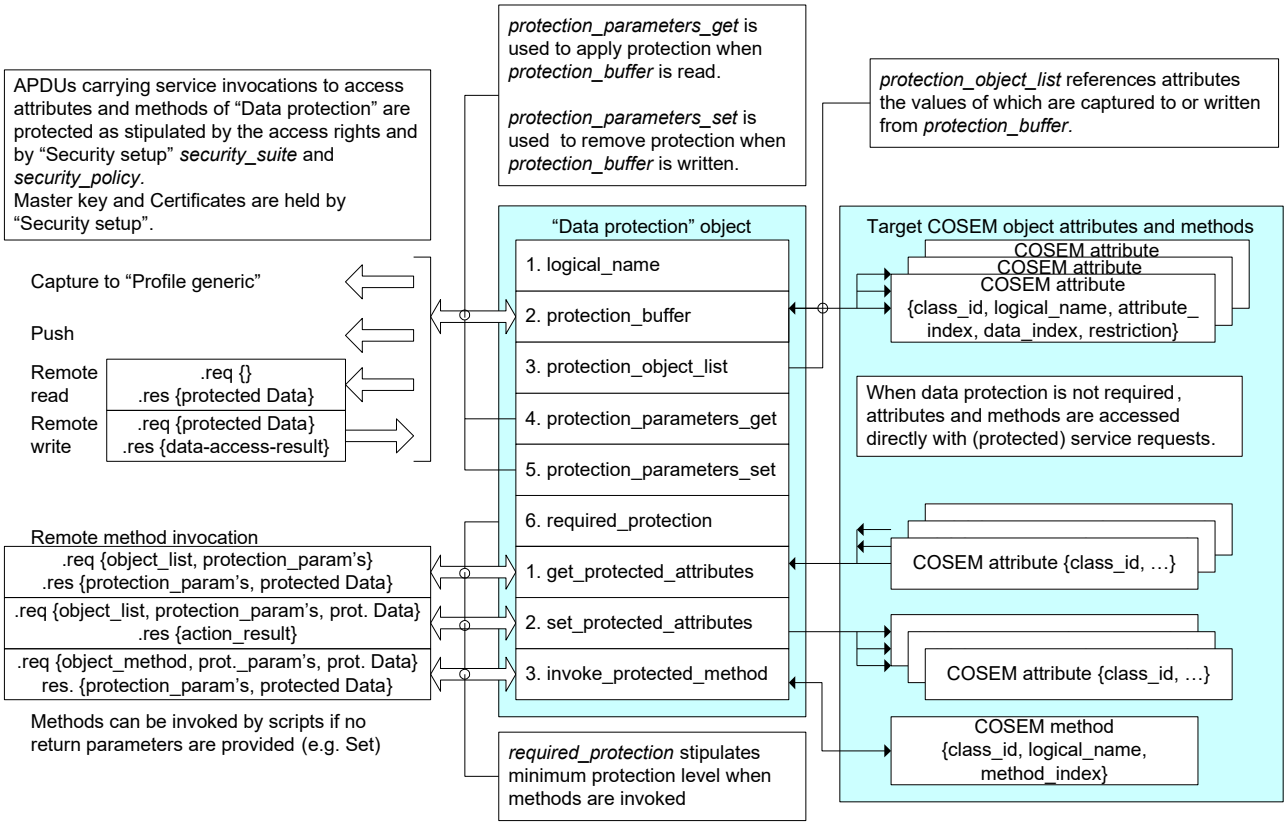


Figure 15 – COSEM model of data protection

Protection on COSEM data is applied and removed in the various cases as follows:

1) When the *protection_buffer* attribute is read / captured in a "Profile generic" object / pushed:
   - attributes determined by *protection_object_list* are captured;
   - protection according to *protection_parameters_get* is applied on the set of attributes and the result is put to *protection_buffer*;
   - the value of *protection_buffer* is returned / captured in the "Profile generic" *buffer* / pushed using "Push setup" objects.
2) When the *protection_buffer* is written:
   - protected Data are written to *protection_buffer;* and
   - protection according to *protection_parameters_set* is removed and the resulting attribute values are written to the attributes specified by *protection_object_list.*
3) When the *get_protected_attributes* method is invoked:
   - attributes determined by the object_list element of get_protected_attributes_request are captured;
   - protection according to the *required_protection* attribute and response protection_parameters is applied. If protection_parameters do not satisfy required_protection then the method invocation fails;
   - the protected attribute values are returned.

© Copyright 1997-2020 DLMS User Association

4) When the *set_protected_attributes* method is invoked:
   - protection on protected_attributes is verified and removed using the protection_parameters that must meet *required_protection*;
   - the resulting attribute values are put in the attributes specified by the object_list element;
5) When the *invoke_protected_method* method is invoked:
   - protection from protected method invocation parameters is removed using the protection parameters in the request that must meet *required_protection*;
   - the method specified by the object_method element of invoke_protected_method_request is invoked with this method invocation parameter;
   - on the return parameters, the protection using the response protection parameters that must meet *required_protection* is applied. If protection_parameters do not satisfy required_protection then the method invocation fails;
   - the protected method return parameters are returned.

Figure 15 shows, as an example, how protected Data in *protection_buffer* is constructed from the attributes determined by *protection_object_list* according to the *protection_parameters_get.* See also DLMS UA 1000-2 Ed. 10, Figure 81.

When the *protection_buffer* attribute is read the following steps are performed:

6) prerequisites: *protection_object_list*, *protection_parameters_get*, master key, key agreement and digital signature certificates as needed;
7) capture COSEM object attributes determined by *protection_object_list* and create Data, a structure containing the individual Data of the attributes captured;
8) protect Data according to *protection_parameters_get.*

   NOTE 4 In the example shown in Figure 15 two layers of protection are applied:
   - the first layer is a combination of compression / encryption / authentication as determined by the Security control byte SC, resulting (C)Data,
   - the second layer is digital signature applied to (C)Data.

9) put the protected data, of data type octet-string, into *protection_buffer*;
10) return the value of *protection_buffer*.

It may be necessary to read also *protection_parameters_get* to obtain the protection parameters to verify / remove protection by the recipient.

The invocation counter used when protection is applied / removed is related to the key used. When the protection is applied the corresponding invocation counter is incremented. When the key is changed the invocation counter shall be reset to 0.

© Copyright 1997-2020 DLMS User Association

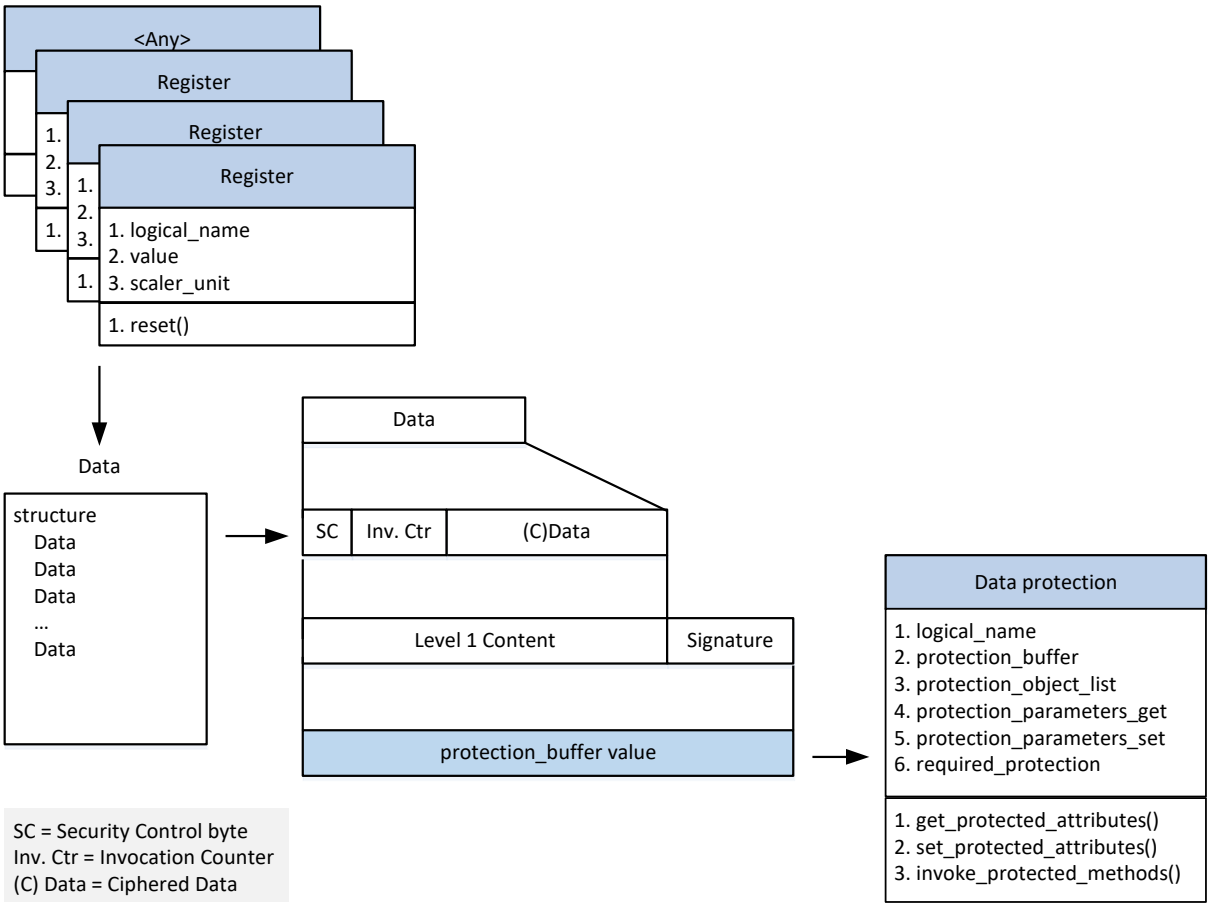


Figure 16 – Example: Read *protection_buffer* attribute

4.4.9.2 Data protection interface class specification (class_id = 30, version = 0)

Data protection		**0...n**	**class_id = 30, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. protection_buffer	(dyn.)	octet-string				x + 0x08
3. protection_object_list	(static)	array				x + 0x10
4. protection_parameters_get	(static)	array				x + 0x18
5. protection_parameters_set	(static)	array				x + 0x20
6. required_protection	(static)	enum				x + 0x28
Specific methods		***m/o***				
1. get_protected_attributes (data)		m				x + 0x30
2. set_protected_attributes (data)		m				x + 0x38
3. invoke_protected_method (data)		m				x + 0x40

Attribute description	
logical_name	Identifies the “Data protection” object instance. See 6.2.36.

© Copyright 1997-2020 DLMS User Association

protection_ buffer	Contains the protected Data. When read, the attributes determined by *protection_object_list* are captured then protection is applied according to *protection_parameters_get*. When written, the protected Data is put into the *protection_buffer*, then the protection is verified / removed according to *protection_parameters_set* and the attributes determined by *protection_object_list* are set.

protection_ object_list	Defines the list of attributes to be captured to *protection_buffer* when it is read or the list of attributes to be set when *protection_buffer* is written.
	Two, mutually exclusive selective access mechanisms are available:
	- relative selective access, i.e. entries defined relative to current date or entry are returned: this mechanism is controlled by the data_index element; or
	- absolute selective access, i.e. entries in an explicitly defined date range or entry range are returned: this mechanism is controlled by the restriction element: entries selected by this mechanism are controlled by the *restriction_element* and the columns are controlled by the lower nibble of the MS byte in the *data_index*.
	array object_definition
	object_definition ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer,
	data_index: long-unsigned,
	restriction: restriction_element
	}
	
	Where:
	– attribute_index is a pointer to the attribute within the object, identified by class_id and logical_name: attribute_index 1 refers to the 1st attribute (i.e. the *logical_name*), attribute_index 2 to the 2nd attribute etc.; attribute_index 0 refers to all public attributes, noting DLMS UA 1000-2 Ed. 10, 9.1.4.3.7;
	– data_index is a pointer selecting one or several specific elements of an attribute with a complex data type (structure or array):
	– if the data type of the attribute is simple, then data_index has no meaning;
	– if the data type of the attribute is a structure or an array – other than the *buffer* of a Profile generic object – then data_index points to one or several specific elements in the structure or array;
	– when the attribute is the *buffer* of a “Profile generic” object, the data_index carries selective access parameters relative to current date or entry.

© Copyright 1997-2020 DLMS User Association

protection_ object_list (continued)

data_index:	MS-Byte		LS-Byte
	Upper nibble	Lower nibble	

- 0x0000 = identifies the whole attribute;
- 0x0001 to 0x0FFF = identifies one element in the complex attribute. The first element in the complex attribute is identified by data_index 1;
- 0x1000 to 0xFFFF = selective access to the array holding the buffer of a “Profile generic” object. The data-index selects entries within a number of last (recent) time periods, or a number of last (recent) entries, as well as the columns in the array.

The encoding is specified in Table 6.

When the attribute is the buffer of a “Profile generic” object, then restriction_element specifies selective access parameters in an explicitly defined date range or entry range.

```
restriction_element ::=  structure
{
      restriction_type:  enum:
                              (0) none,
                              (1) restriction by date,
                              (2) restriction by entry
      restriction_value: CHOICE
      {
            null-data,                  // no restrictions apply
            restriction_by_date,
            restriction_by_entry
      }
}

restriction_by_date ::=  structure
{
      from_date:        octet-string,
      to_date:          octet-string
}

restriction_by_entry ::= structure
{
      from_entry:       double-long-unsigned,
      to_entry:         double-long-unsigned
}
```

- restriction_element defines absolute selective access to a “Profile generic” *buffer* by date range (from_date to to_date) or by entries (from_entry to to_entry). To use this absolute selective access mechanism, data_index shall be:
  - MS Byte upper nibble set to 0x0;
  - MS Byte lower nibble set to 0x0 to 0xF in accordance with Table 6;
  - Lower Byte upper nibble set to 0x00;

© Copyright 1997-2020 DLMS User Association

protection_object_list (continued)

- restriction_element is composed of restriction_type and restriction_value:
  - for restriction by date range the restriction_type element holds (1) restriction by date and the restriction_value element holds restriction_by_date structure;
  - for restriction by entries the restriction_type element holds (2) restriction by entry and the restriction_value element holds restriction_by_entry structure;
  - otherwise, the restriction_type element holds (0) none and the restriction_value element holds null-data. This choice shall be taken also if relative selective access is to be used.

protection_parameters_get

Contains all necessary parameters to specify the protection applied when the *protection_buffer* is read.

array protection_parameters_element

```
protection_parameters_element ::= structure
{
    protection_type:  enum:
                          (0) authentication,
                          (1) encryption,
                          (2) authentication and encryption,
                          (3) digital signature
    protection_options ::= structure
    {
        transaction_id:          octet-string,
        originator_system_title: octet-string,
        recipient_system_title:  octet-string,
        other_information:       octet-string,
        key_info:                key_info_element
    }
}
```

Where:

– transaction_id holds the identifier of the transaction;

– originator_system_title holds the system title of the originator that applies the protection;

– recipient_system_title holds the system title of the recipient which will check and remove the given protection;

– other_information carries other information. Its contents may be specified in project specific companion specifications. An octet-string of length 0 indicates that this field is not used;

– key_info holds the information necessary for the recipient to obtain the right key for checking and removing authentication and encryption. In the case of digital signature, key_info is not necessary and it shall be a structure of 0 elements.

© Copyright 1997-2020 DLMS User Association

protection_ parameters_get (continued)

The fields transaction-id ….other-information are A-XDR encoded OCTET STRINGs. The length and the value of each field are included in the AAD when applicable.

```
key_info_element ::= structure
{
        key_info_type:    enum:
                                (0) identified_key,
                                    -- used with identified_key_info_options
                                (1) wrapped_key,
                                    -- used with wrapped_key_info_options
                                (2) agreed_key
                                    -- used with agreed_key_info_options
              key_info_options: CHOICE
              {
                    identified_key_info_options,
                    wrapped_key_info_options,
                    agreed_key_info_options
              }
}
identified_key_info_options ::=     enum:
                                      (0) global_unicast_encryption_key,
                                      (1) global_broadcast_encryption_key

wrapped_key_info_options ::=      structure
{
        kek_id:                 enum:
                                      (0) master_key,
        key_ciphered_data:      octet-string
}
agreed_key_info_options ::= structure
{
        key_parameters:         octet-string,
        key_ciphered_data:      octet-string
}
```

This attribute is first written by the client. The server may need to fill in some additional elements, in which case this attribute has to be read back by the client – and if needed, forwarded to the third party – so that they can use these parameters to verify / remove protection.

For the use of the various elements see Table 17 and Table 18.

© Copyright 1997-2020 DLMS User Association

protection_ parameters_set	Contains all necessary parameters to verify / remove the protection applied when the *protection_buffer* is written. array protection_parameters_element protection_parameters_element: see the *protection_parameters_get* attribute. This attribute is written by the client and it is used by the server to verify and remove protection. For the use of the various elements see Table 17 and Table 19.
required_ protection	Stipulates the required protection on the attribute values / invocation and return parameters of methods accessed through the “Data protection” object. enum: When the enum value is interpreted as an unsigned, the meaning of each bit is as shown below: Bit Required protection 0 unused, shall be set to 0, 1 unused, shall be set to 0, 2 authenticated request, 3 encrypted request, 4 digitally signed request, 5 authenticated response, 6 encrypted response, 7 digitally signed response
Method description	

get_protected_ attributes (data)	Gets the value of the attributes specified by the object_list element, with the protection according to the protection_parameters in get_protected_ attributes_response applied.
	
	Method invocation parameters:
	
	data ::= get_protected_attributes_request
	get_protected_attributes_request ::= structure
	{
	object_list: array object_definition,
	protection_parameters: array protection_parameters_element,
	}
	Return parameters:
	
	data ::= get_protected_attributes_response

© Copyright 1997-2020 DLMS User Association

get_protected_ attributes (data) (continued)	get_protected_attributes_response ::= structure
	{
	protection_parameters: array protection_parameters_element,
	protected_attributes: octet-string
	}
	Where:
	- object_list: see the *protection_object_list* attribute;
	- protection_parameters ::= array protection_parameters_element.
	protection_parameters_element: see the *protection_parameters_get* attribute.
	The protection_parameters in get_protected_attributes_response are elaborated by copying the protection_parameters from get_protected_attributes_request except that the server may need to fill in some elements. The remote party uses the protection_parameters in get_protected_attributes_response to verify / remove protection on the protected_attributes returned.
	
	For the use of the various elements see Table 17 and Table 20.

set_protected_ attributes (data)	Sets the value of the attributes specified by the object_list element, after verifying and removing the protection on the protected_attributes element according to the protection_parameters element.
	
	Method invocation parameters:
	
	data ::= set_protected_attributes_request
	set_protected_attributes_request ::= structure
	{
	object_list: array object_definition,
	protection_parameters: array protection_parameters_element,
	protected_attributes: octet-string
	}
	Where:
	- object_list: see the *protection_object_list* attribute;
	- protection_parameters ::= array protection_parameters_element.
	
	protection_parameters_element: see the *protection_parameters_get* attribute.
	
	For the use of the various elements see Table 17 and Table 21.

© Copyright 1997-2020 DLMS User Association

invoke_ protected_ method (data)

Invokes the method specified by the object_method element, after verifying and removing the protection on protected_method_invocation_parameters according to the protection_parameters in invoke_protected_method_ request.

After invoking the method specified by the object_method element, the protection according to the protection_parameters in invoke_protected_ method_response – that must meet *required_protection* – is applied on the return parameters, and invoke_protected_method_response composed of protection_parameters and protected_method_return_parameters is returned.

Method invocation parameters:

data ::= invoke_protected_method_request

invoke_protected_method_request ::= structure
{
    object_method: object_method_definition,
    protection_parameters: array protection_parameters_element,
    protected_method_invocation_parameters: octet-string
}
object_method_definition ::= structure
{
    class_id: long-unsigned,
    logical_name: octet-string,
    method_index: integer,
}

Return parameters:

data ::= invoke_protected_method_response

invoke_protected_method_response ::= structure
{
    protection_parameters: array protection_parameters_element,
    protected_method_return_parameters: octet-string
}

protection_parameters_element: see the specification of the protection_parameters_get attribute.

If the method invoked does not provide return parameters then invoke_protected_method_response shall be a structure, with protection_parameters an array of zero elements and protected_method_return_parameters an octet-string of zero length.

The server uses the protection_parameters in invoke_protected_method_ request – that must meet *required_protection* – to verify and remove protection from protected_method_invocation_parameters.

© Copyright 1997-2020 DLMS User Association

invoke_ protected_ method (data) (continued)

The protection_parameters in invoke_protected_method_response are elaborated by copying the protection_parameters from invoke_protected_ method_request to invoke_protected_method_response except that the server may need to fill in some elements.

The protection_parameters in invoke_protected_method_response – that must meet *required_protection* – are applied then to protect data in protected_method_return_parameters.

For the use of the various elements see Table 17 and Table 22.

Table 17 – Key information required to establish data protection keys

Key information choices		Comment
key_info_options		
(0) **identified_key_info_options**	S	The EK is identified
(0) global_unicast_encryption_key	S	GUEK
(1) global_broadcast_encryption_key	S	GBEK
(1) **wrapped_key_info_options**	S	The EK is transported using key wrap
kek_id	M	
(0) master_key	M	Identifies the key used for wrapping the key_ciphered_data. 0 = Master Key (KEK)
key_ciphered_data	M	Randomly generated key wrapped with KEK.
(2) **agreed_key_info_options**	S	The key is agreed by the parties using either: - the One-Pass Diffie-Hellman C(1e, 1s, ECC CDH) scheme or - the Static Unified Model C(0e, 2s, ECC CDH) scheme
key_parameters	M	Identifier of the key agreement scheme: 0x01: C(1e, 1s, ECC CDH) 0x02: C(0e, 2s, ECC CDH) All other reserved.
key_ciphered_data	M	- In the case of the C(1e, 1s, ECC CDH) scheme: the public key Qu of the ephemeral key agreement key pair of party U, signed with the private digital signature key of party U. - In the case of the C(0e, 2s, ECC CDH) scheme: an octet-string of length zero. NOTE In the second case party U has to provide a nonce, $Nonce_U$. See DLMS UA 1000-2 Ed. 10, 9.2.3.4.6.4 and 9.2.3.4.6.5.
M: Mandatory (part of a structure) S: Selectable (part of a CHOICE)		

© Copyright 1997-2020 DLMS User Association

Table 18 – Protection parameters of *protection_parameters_get* attribute

protection_parameters_get		Written by the client	Filled in by the server
array protection_parameters_element	M	x	
protection_type	M	x	
(0) authentication	S	x	
(1) encryption	S	x	
(2) authentication and encryption	S	x	
(3) digital signature	S	x	
protection_options	M	x	
transaction_id	M	x	
originator_system_title (Shall carry the server system title)	M	x	
recipient_system_title (Shall carry the remote party system title)	M	x	
other_information	M	x	
key_info	M	x	
key_info_type	M	x	
(0) identified_key	S	x	
(1) wrapped_key	S	x	
(2) agreed_key	S	x	
key_info_options	M	x	
identified_key_options	S	x	
(0) global_unicast_encryption_key	S	x	
(1) global_broadcast_encryption_key	S	x	
wrapped_key_options	S	x	
kek_id	M	x	
(0) master_key	M	x	
key_ciphered_data	M	x	
agreed_key_options	S	x	
key_parameters	M	x	
key_ciphered_data	M		x
The *protection_parameters_get* attribute is written by the client, but when key_info_type is (2) agreed key, the key_ciphered_data of agreed_key_options is empty. When the One-pass Diffie-Hellman C(1e, 1s, ECC CDH) scheme is used, this element is filled by the server and the remote party has to read back *protection_parameters_get* in order to be able to verify and remove protection.			
M: Mandatory (part of a structure) S: Selectable (part of a CHOICE)			

© Copyright 1997-2020 DLMS User Association

Table 19 – Protection parameters of *protection_parameters_set* attribute

protection_parameters_set		Written by the client	Filled in by the server
array protection_parameters_element	M	x	
protection_type	M	x	
(0) authentication	S	x	
(1) encryption	S	x	
(2) authentication and encryption	S	x	
(3) digital signature	S	x	
protection_options	M	x	
transaction_id	M	x	
originator_system_title (Shall carry the remote party system title)	M	x	
recipient_system_title ((Shall carry the server systems title)	M	x	
other_information	M	x	
key_info	M	x	
key_info_type	M	x	
(0) identified_key	S	x	
(1) wrapped_key	S	x	
(2) agreed_key	S	x	
key_info_options	M	x	
identified_key_options	S	x	
(0) global_unicast_encryption_key	S	x	
(1) global_broadcast_encryption_key	S	x	
wrapped_key_options	S	x	
kek_id	M	x	
(0) master_key	M	x	
key_ciphered_data	M	x	
agreed_key_options	S	x	
key_parameters	M	x	
key_ciphered_data	M	x	
M: Mandatory (part of a structure) S: Selectable (part of a CHOICE)			

© Copyright 1997-2020 DLMS User Association

Table 20 – Protection parameters of *get_protected_attributes* method

Protection parameters	request	response
array protection_parameters_element	M	M (=)
protection_type	M	M (=)
(0) authentication	S	S (=)
(1) encryption	S	S (=)
(2) authentication and encryption	S	S (=)
(3) digital signature	S	S (=)
protection_options	M	M (=)
transaction_id	M	M (=)
originator_system_title	M	M[1]
recipient_system_title	M	M[2]
other_information	M	M (=)
key_info	M	M (=)
key_info_type	M	M (=)
(0) identified_key	S	S (=)
(1) wrapped_key	S	S (=)
(2) agreed_key	S	S (=)
key_info_options	M	M (=)
identified_key_options	S	S (=)
(0) global_unicast_encryption_key	S	S (=)
(1) global_broadcast_encryption_key	S	S (=)
wrapped_key_options	S	S (=)
kek_id	M	M (=)
(0) master_key	M	M (=)
key_ciphered_data	–	M [3]
agreed_key_options	S	S (=)
key_parameters	M	M (=)
key_ciphered_data	–	M [4]

Notice that protection parameters are taken from request and put into response. Protection parameters are only applied for protection of response.

[1] originator_system_title from request is put into recipient_system_title of response.

[2] recipient_system_title from request is put into originator_system_title of response.

[3] key_ciphered_data is filled by the server.

[4] When the One-pass Diffie-Hellman C(1e, 1s, ECC CDH) scheme is used, this element is filled by the server.

M: Mandatory (part of a structure)

S: Selectable (part of a CHOICE)

© Copyright 1997-2020 DLMS User Association

Table 21 – Protection parameters of *set_protected_attributes* method

Protection parameters	request	response
array protection_parameters_element	M	
protection_type	M	
(0) authentication	S	
(1) encryption	S	
(2) authentication and encryption	S	
(3) digital signature	S	
protection_options	M	
transaction_id	M	
originator_system_title	M	
recipient_system_title	M	
other_information	M	
key_info	M	
key_info_type	M	
(0) identified_key	S	
(1) wrapped_key	S	
(2) agreed_key	S	
key_info_options	M	
identified_key_options	S	
(0) global_unicast_encryption_key	S	
(1) global_broadcast_encryption_key	S	
wrapped_key_options	S	
kek_id	M	
(0) master_key	M	
key_ciphered_data	M	
agreed_key_options	S	
key_parameters	M	
key_ciphered_data	M	
Notice that protection parameters are taken from request and are not present in response. Protection parameters are only used for removing protection on the request.		
M: Mandatory (part of a structure) S: Selectable (part of a CHOICE)		

© Copyright 1997-2020 DLMS User Association

Table 22 – Protection parameters of *invoke_protected_method* method

Protection parameters	request	response
array protection_parameters_element	M	M(=)
protection_type	M	M(=)
(0) authentication	S	S(=)
(1) encryption	S	S(=)
(2) authentication and encryption	S	S(=)
(3) digital signature	S	S(=)
protection_options	M	M(=)
transaction_id	M	M(=)
originator_system_title	M	M[1]
recipient_system_title	M	M[2]
other_information	M	M(=)
key_info	M	M(=)
key_info_type	M	M(=)
(0) identified_key	S	S(=)
(1) wrapped_key	S	S(=)
(2) agreed_key	S	S(=)
key_info_options	M	M(=)
identified_key_options	S	S(=)
(0) global_unicast_encryption_key	S	S(=)
(1) global_broadcast_encryption_key	S	S(=)
wrapped_key_options	S	S(=)
kek_id	M	M(=)
(0) master_key	M	M(=)
key_ciphered_data	M	M[3]
agreed_key_options	S	S(=)
key_parameters	M	M(=)
key_ciphered_data	M	M[4]

Notice that protection parameters are taken from request and put into response. Protection parameters in the request are used to remove protection from request and protection parameters in the response are used to apply protection on response.

[1] originator_system_title from request is put into recipient_system_title of response.

[2] recipient_system_title from request is put into originator_system_title of response.

[3] key_ciphered_data is sent by the originator in the request and filled by the server for the response.

[4] When the One-pass Diffie-Hellman C(1e, 1s, ECC CDH) scheme is used, this element is filled by the server.

Legend:
M: Mandatory (part of a structure)
S: Selectable (part of a CHOICE)

© Copyright 1997-2020 DLMS User Association

4.4.10 Function control (class_id = 122, version = 0)

Instances of the IC “Function control” allow enabling and disabling functions in the server. Each function that can be enabled / disabled is identified by a name and is defined by a particular set of object identifiers referenced.

To allow enabling and disabling of functions controlled by time, “Single action schedule” and “Script table” objects are also specified.

Function control		**0...n**	**class_id = 122, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. activation_status	(dyn.)	array				x + 0x08
3. function_list	(static)	array				x + 0x10
Specific methods		***m/o***				
1. set_function_status (data)		m				x + 0x20
2. add_function (data)		o				x + 0x28
3. remove_function (data)		o				x + 0x30

Attribute description	
logical_name	Identifies the “Function control” object instance. See 6.2.38.

activation_status	Shows the current status of each functional block defined in the *function_list* attribute.
	activation_status ::= array function_status_type
	function_status_type ::= structure
	{
	function_name octet-string,
	function_status boolean
	}
	
	TRUE = function enabled,
	FALSE = function disabled.

function_list	Defines a list of functions which can be enabled or disabled by invoking the *set_function_status* method.
	
	function_list ::= array functional_block
	
	functional_block ::= structure
	{
	function_name: octet-string,
	function_specification: array function_definition
	}

© Copyright 1997-2020 DLMS User Association

function_list (continued)	function_definition ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
	The function_specification element contains a list of objects involved in this function. The objects are referenced by their class ids and logical names.
	In case of functions that cannot be linked to specific objects, class_id = 0 is used and the second element holds a descriptive name instead of a logical_name.
	The names of the functions and the behaviour of the server in case of enabling or disabling a function have to be defined in project specific companion specifications.
	Please also note that this attribute controls only the functions that are available in the server and that can be enabled or disabled. It is not possible to download new functions to the server by modifying this attribute or by invoking the *add_function* method.

Method description	
set_function_status (data)	Enables or disables one or more functions defined in attribute *function_list*. data ::= array function_status_type as defined in attribute *activation_status* above. The status of functions that are not listed is not modified.
add_function (data)	Adds a new function to the attribute *function_list*. If a function with the same name already exists, the existing function will be replaced by the new function. data ::= functional_block See attribute function_list above for further details.
remove_function (data)	Removes a function from the attribute *function_list.* data ::= octet-string, holding the function_name.

4.4.11 Array manager (class_id = 123, version = 0)

Instances of the “Array manager” IC allow managing attributes of type *array* of other interface objects, i.e.:

- retrieving the number of entries;
- selectively reading a range of entries;
- inserting a new entry or updating an existing entry;
- removing a range of entries.

Each instance allows managing several attributes of type *array* assigned to it.

An example of the application is shown in Figure 17.

© Copyright 1997-2020 DLMS User Association

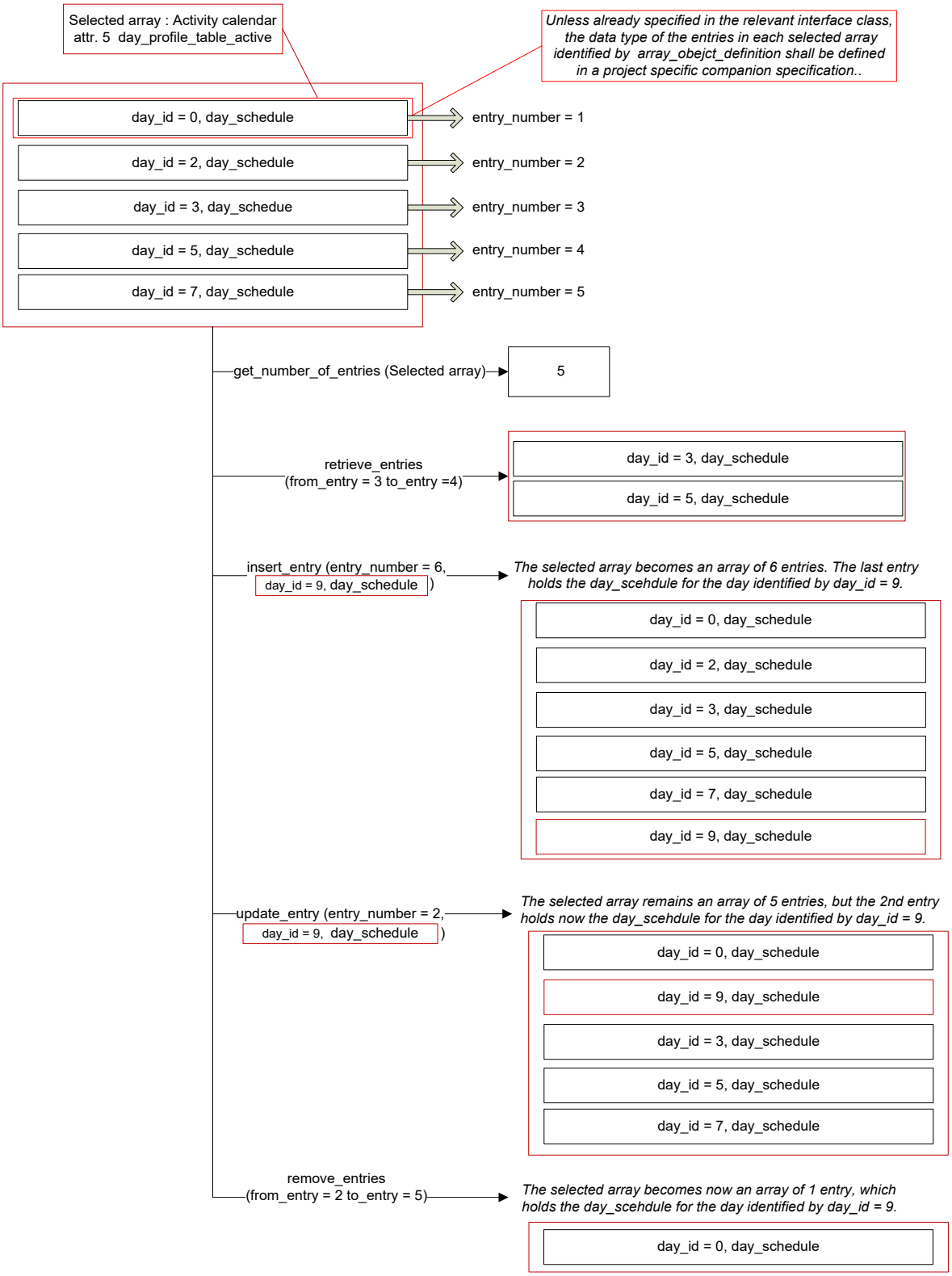


Figure 17 – Example of managing an array

© Copyright 1997-2020 DLMS User Association

Array manager	**0...n**	**class_id = 123, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. array_object_list (static)	array				x + 0x08
Specific methods	***m/o***				
1. retrieve_number_of_entries (data)	m				x + 0x20
2. retrieve_entries (data)	m				x + 0x28
3. insert_entry (data)	o				x + 0x30
4. update_entry (data)	o				x + 0x38
5. remove_entries (data)	o				x + 0x40

Attribute description

logical_name Identifies the “Array manager” object instance. See 6.2.16.

array_object_list Defines the list of attributes of type *array* that can be managed by an instance of this IC. Each “Array manager” object can manage 1 to n arrays.

```
array        array_object_list_definition
array_object_list_definition ::= structure
{
        array_object_id:                unsigned,
        array_object_list_element:      array_object_definition
}
array_object_definition ::= structure
{
        class_id:          long-unsigned,
        logical_name:      octet-string,
        attribute_index:   integer
}
```

Where attribute_index is a pointer to the attribute within the object identified by its class_id and logical_name. Attribute_index 1 refers to the first attribute (i.e. the logical_name), attribute_index 2 to the second attribute, etc.).

Each attribute identified by the array_object_definition shall be of type *array*.

An attribute of type *array* holds a number of entries that can be referenced by their entry number. The number 1 identifies the first entry and the number m identifies the last entry, where m is the number of entries of a selected array.

Unless already specified in the relevant interface class, the data type of the entries in each array identified by array_object_definition shall be specified in a project specific companion specification.

When the target array is accessed through an “Array manager” object, its access rights are observed.

© Copyright 1997-2020 DLMS User Association

Method description	
retrieve_number_ of_entries	Returns the number of entries in the array identified. The method invocation parameter identifies an array from the array_object_list by its array_object_id: data:= unsigned The return parameter holds the number of entries in the array identified.

retrieve_entries (data)	Returns a range of entries in one of the arrays of the array_object_list.
	The method invocation parameters identify an array from the array_object_list by its array_object_id and the entries to be retrieved:
	
	data ::= entries_to_retrieve
	
	entries_to_retrieve ::= structure
	{
	array_object_id: unsigned,
	list_of_entries: structure
	{
	from_entry: long-unsigned,
	to_entry: long-unsigned
	}
	}
	
	Where:
	- array_object_id identifies one of the arrays managed by an “Array manager” object;
	- list_of_entries identifies the entries to be retrieved.
	
	If the number of entries in the selected array is m:
	- if from_entry < last entry < m, then the entries in this range are retrieved;
	- if from_entry < last entry but last_entry > m, then the entries identified by from_entry up to the last entry are retrieved;
	- if from_entry < last_entry, but from_entry > m, then the method returns an array of 0 elements;
	- if from_entry > last_entry the method invocation fails.
	The return parameters hold an array of the entries selected:
	
	data ::= retrieved_entries:
	retrieved_entries: array entry
	entry ::= attribute specific
	
	The data type depends on the attribute of type *array* as specified in the IC specification or in the project specific companion specification.

© Copyright 1997-2020 DLMS User Association

insert_entry (data)

Inserts a new entry in a selected array.

```
data ::=            structure
{
        array_object_id:        unsigned,
        entry_to_insert:        structure
                {
                        entry_number:   long-unsigned,
                        entry:          attribute specific

                The data type of the entry depends on the attribute of type
                array as specified in the IC specification or in the project
                specific companion specification.
                }
}
```

Where:

- array_object_id identifies one of the arrays managed by an “Array manager” object;
- entry_to_insert identifies the entry_number and holds the entry itself to be inserted.

If entry_number = 0:

- if the selected array is already full, then the method invocation fails;
- else, the new entry is inserted at the beginning of the array: it becomes the first entry and all the other entries of the selected array are shifted up by one;

If entry_number > m (where m is the number of entries in the array) :

- if the selected array is already full, then the method invocation fails;
- else, the new entry is inserted at the end of the selected array: it becomes the last entry. The other entries are not shifted.

If an entry_number identifies an existing entry, it is inserted after the existing entry.

If the entries are shifted due to inserting a new entry then all the references to them have to be updated.

© Copyright 1997-2020 DLMS User Association

update_entry (data)

Updates an existing entry in a selected array.

```
data ::=structure
{
    array_object_id:        unsigned,
    entry_to_update::= structure
        {
        entry_number:   long-unsigned,
        entry:          attribute specific

        The data type of the entry depends on the attribute of type
        array as specified in the IC specification or in the project
        specific companion specification.
        }
}
```

Where:

- array_object_id identifies one of the arrays managed by an “Array manager” object;
- entry_to_update identifies the entry_number and holds the entry itself to be updated.

When an entry_number identifies an existing entry, it is overwritten.

In any other case, the method invocation fails.

remove_entries (data)

Removes a range of entries in one of the arrays of the array_object_list.

The method invocation parameters identify an array from the array_object_list by its array_object_id and the entries to be removed:

```
data ::= entries_to_remove

entries_to_remove ::= structure
{
    array_object_id:  unsigned;
    list_of_entries::= structure
    {
        from_entry:     long-unsigned,
        to_entry:       long-unsigned
    }
}
```


© Copyright 1997-2020 DLMS User Association

retrieve_entries (data) (continued)	Where: - array_object_id identifies one of the arrays managed by an “Array manager” object; - list_of_entries identifies the entries to be removed. If the number of entries in the selected array is m: - if from_entry < last entry< m, then the entries in this range are removed; - if from_entry < last entry but last_entry > m, then the entries identified by from_entry up to the last entry are removed; - if from_entry < last_entry, but from_entry > m, then no entries are removed; - if from_entry > last_entry the method invocation fails. If the entries are shifted due to removing entries, then all the references to them have to be updated.

4.4.12 Communication port protection (class_id = 124, version = 0)

Instances of the “Communication port protection” IC can be used to protect communication ports of DLMS/COSEM servers against possibly malicious communication attempts, in particular to prevent brute force attacks by reducing the possible number of attempts.

Each instance references a single communication port by its logical name (OBIS code). If an acceptable number of failed attempts is exceeded then the communication port is temporarily locked. The lockout time may increase with each failed attempt, until a maximum lockout time is reached.

A failed attempt is one that leads to discarding the APDU carrying a service request. The criteria for detecting a failed attempt are out of the Scope of this document.

The objects count both the number of failed attempts between two resets and the cumulative number of failed attempts.

It is possible to configure the communication ports such that they are locked to all attempts or unlocked to all attempts.

© Copyright 1997-2020 DLMS User Association

Communication port protection		0...n	class_id = 124, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	***Short name***
1. logical_name	(static)	octet-string				x
2. protection_mode	(static)	enum			1	x + 0x08
3. allowed_failed_attempts	(static)	long-unsigned				x + 0x10
4. initial_lockout_time	(static)	double-long-unsigned				x + 0x18
5. steepness_factor	(static)	unsigned			1	x + 0x20
6. max_lockout_time	(static)	double-long-unsigned				x + 0x28
7. port_reference	(static)	octet-string				x + 0x30
8. protection_status	(dyn.)	enum				x + 0x38
9. failed_attempts	(dyn.)	double-long-unsigned				x + 0x40
10. cumulative_failed_attempts	(dyn.)	double-long-unsigned				x + 0x48
Specific methods	***m/o***					
1. reset (data)	o	o				x + 0x50

Attribute description	
logical_name	Identifies the “Communication port protection” object instance. See 6.2.39.
protection_mode	Controls the protection mode. enum (0) locked, (1) locked_on_failed_attempts, (2) unlocked When set to locked, no communication can take place via the port referenced by attribute 7. When set to “locked_on_failed_attempts”, the port becomes temporarily locked when the number of failed communication attempts exceeds an allowed number. The length of the lockout period may increase with each failed attempt, as determined by attributes 5 and 6. When set to “unlocked” the lockout mechanism is disabled; communication through the port referenced by attribute 7 is always possible. When the port is locked no communication can take place, even valid APDUs are discarded.
allowed_failed_attempts	Holds the number of allowed failed communication attempts before the lockout mechanism is triggered.
initial_lockout_time	Holds the initial value of the lockout time, in seconds, after the first failed communication attempt when the value of allowed_failed_attempts is reached.
steepness_factor	Holds a factor that controls how the lockout time is increased with each failed attempt when allowed_failed_attempts is reached, until the max_lockout_time is reached. The current lockout time is calculated using the following formulae: $NCA = failed_attempts - allowed_failed_attempts$ $CLT = initial_lockout_time\ x\ \left(steepness_factor^{(NCA-1)}\right)$

© Copyright 1997-2020 DLMS User Association

steepness_factor (continued)

Where:

- NCA is number of counted failed attempts: that is: the number of attempts that count towards extending the lockout time,
- CLT is the current lockout time.

The table below shows example values with an initial lockout time of 60s.

NCA	steepness_factor		
	1	2	3
	CLT (in seconds)		
1	60	60	60
2	60	120	180
3	60	240	540
4	60	480	1620
5	60	960	4860

Once lockout occurs, even valid attempts are discarded until the current lockout time has expired.

max_lockout_time

Holds the maximum time, in seconds, for which the communication port can be locked, even if the number of failed attempts keeps increasing. The purpose of this attribute is to avoid a denial of service attack.

port_reference

Contains the logical name of an object that identifies the communication port being protected. That object may be a communication port setup object or any other suitably chosen object.

If this information is not available or not needed, the attribute may be left empty (octet-string [0]).

protection_status

Holds the current protection status of the communication port.

enum

(0) unlocked,

(1) temporarily_locked,

(2) locked

The status is unlocked if the protection_mode is unlocked or the number of failed attempts does not exceed the number of allowed_failed_attempts.

NOTE 1 The status becomes unlocked again when the current_lockout_time has elapsed.

The status is temporarily_locked if the number of failed communication attempts exceeds allowed_failed_attempts and the current_lockout_time has not yet expired.

The status is locked if the protection_mode is locked.

NOTE 2 When a port is locked, the server still may be accessed on another port.

failed_attempts

Holds the total number of failed attempts since the last reset, independent of the protection_mode and the triggering of the protection mechanism.

cumulative_failed_attempts

Holds the cumulative number of failed attempts, independent of the protection_mode and the triggering of the protection mechanism.

This attribute is never reset.

© Copyright 1997-2020 DLMS User Association

Method description	
reset (data)	Resets the variables of the lockout mechanism: - the value of failed_attempts is reset to 0; - the current lockout time is reset to 0; - the protection_status is set to “(0) unlocked” if the protection_mode is (1) locked_on_failed_attempts, and is not affected otherwise. data ::= integer (0)

4.5 Interface classes for time- and event bound control

4.5.1 Clock (class_id = 8, version = 0)

This IC models the device clock, managing all information related to date and time including deviations of the local time to a generalized time reference (UTC) due to time zones and daylight saving time schemes. The IC also offers various methods to adjust the clock.

The *date* information includes the elements year, month, day of month and day of week. The *time* information includes the elements hour, minutes, seconds, hundredths of seconds, and the deviation of the local time from UTC. The daylight saving time function modifies the deviation of local time to UTC depending on the attributes; see Figure 17. The start and end point of that function is normally set once. An internal algorithm calculates the real switch point depending on these settings.

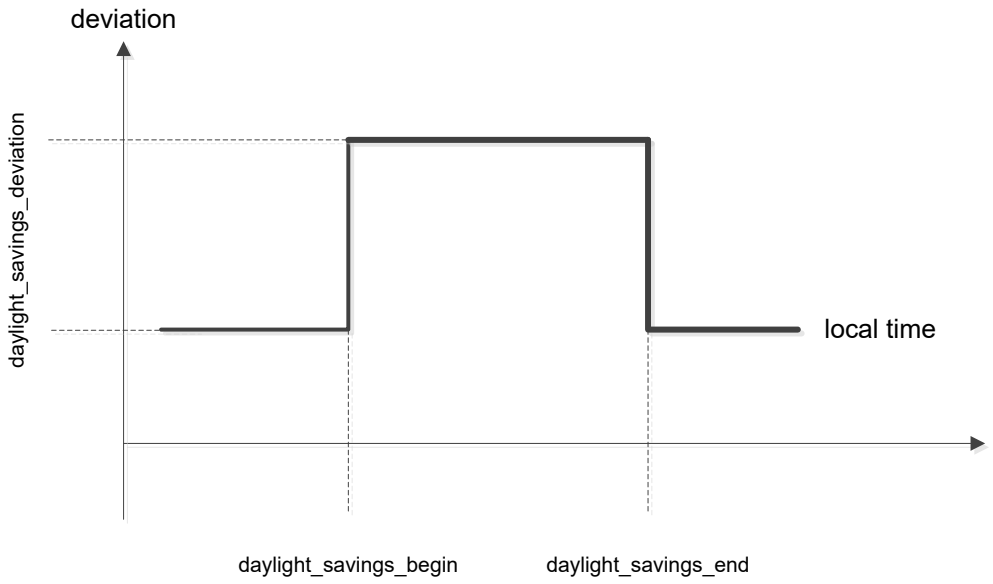


Figure 18 – The generalized time concept

© Copyright 1997-2020 DLMS User Association

Clock		0...n	class_id = 8, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. time	(dyn.)	octet-string				x + 0x08
3. time_zone	(static)	long	-720	+840		x + 0x10
4. status	(dyn.)	unsigned				x + 0x18
5. daylight_savings_begin	(static)	octet-string				x + 0x20
6. daylight_savings_end	(static)	octet-string				x + 0x28
7. daylight_savings_deviation	(static)	integer	-120	+120		x + 0x30
8. daylight_savings_enabled	(static)	boolean				x + 0x38
9. clock_base	(static)	enum				x + 0x40
Specific methods		***m/o***				
1. adjust_to_quarter (data)		o				x + 0x60
2. adjust_to_measuring_period (data)		o				x + 0x68
3. adjust_to_minute (data)		o				x + 0x70
4. adjust_to_preset_time (data)		o				x + 0x78
5. preset_adjusting_time (data)		o				x + 0x80
6. shift_time (data)		o				x + 0x88

Attribute description	
logical_name	Identifies the “Clock” object instance. See 6.2.5.
time	Contains the meter’s local date and time, its deviation to UTC and the status. octet-string, formatted as specified in 4.1.6.1 for *date-time.* When this attribute is set, all the fields in the octet-string representing *date-time* shall be evaluated and the local date and time in the meter shall be set according to the rules defined in 4.1.6.1. Only specified fields of the *date-time* are changed. EXAMPLE For setting the *date* without changing the *time*, all time-relevant octets in the octet string representing *date-time* shall be set to “not specified”.
time_zone	The deviation of local, normal time to UTC in minutes. The value depends on the geographical location of the meter.
status	The clock_status maintained by the meter. The clock_status element indicated in the *time* attribute is equal to the value of this attribute. unsigned, formatted as specified in 4.1.6.1 for *clock_status.*
daylight_savings_ begin	Defines the local switch date and time when the local time starts to deviate from the normal time. For generic definitions, wildcards are allowed. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
daylight_savings_ end	Defines the local switch date and time when the local time ends to deviate from the local normal time. octet-string, formatted as specified in 4.1.6.1 for *date-time.*

© Copyright 1997-2020 DLMS User Association

daylight_savings_ deviation	Contains the number of minutes by which the deviation in generalized time shall be corrected at daylight savings begin. integer: Deviation in the range of ± 120 min
daylight_savings_ enabled	boolean: TRUE = DST enabled, FALSE = DST disabled NOTE This is a parameter to enable and disable the daylight savings time feature. The current status is indicated in the status attribute.
clock_base	Defines where the basic timing information comes from. enum: (0) not defined, (1) internal crystal, (2) mains frequency 50 Hz, (3) mains frequency 60 Hz, (4) GPS (global positioning system), (5) radio controlled
Method description	
adjust_to_quarter (data)	Sets the meter's time to the nearest (+/-) quarter of an hour value (*:00, *:15, *:30, *:45). data ::= integer (0)
adjust_to_ measuring_period (data)	Sets the meter's time to the nearest (+/-) starting point of a measuring period. data ::= integer (0)
adjust_to_minute (data)	Sets the meter's time to the nearest minute. If second_counter < 30 s, second_counter is set to 0. If second_counter ≥ 30 s, second_counter is set to 0, and minute_counter and all depending clock values are incremented if necessary. data ::= integer (0)
adjust_to_preset_ time (data)	This method is used in conjunction with the preset_adjusting_time method. If the meter's time lies between validity_interval_start and validity_interval_end, then time is set to preset_time. data ::= integer (0)

preset_adjusting_ time (data)	Presets the time to a new value (preset_time) and defines a validity_interval within which the new time can be activated.
	data ::= structure
	{
	preset_time: octet-string,
	validity_interval_start: octet-string,
	validity_interval_end: octet-string
	}
	all octet-strings formatted as specified in 4.1.6.1 for *date-time.*

shift_time (data)	Shifts the time by *n* (-900 <= *n* <= 900) s. data ::= long

4.5.2 Script table (class_id = 9, version = 0)

This IC allows modelling the triggering of a series of actions by executing scripts using the *execute* (data) method.

© Copyright 1997-2020 DLMS User Association

“Script table” objects contain a table of script entries. Each entry consists of a script identifier and a series of action specifications. An action specification activates a method or modifies an attribute of a COSEM object within the logical device.

A certain script may be activated by other COSEM objects within the same logical device or from the outside.

If two scripts have to be executed at the same time instance, then the one with the smaller index is executed first.

Script table	**0...n**	**class_id = 9, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. scripts (static)	array				x + 0x08
Specific methods	***m/o***				
1. execute(data)	m				x + 0x20

Attribute description

logical_ name

Identifies the “Script table” object instance. See 6.2.7.

scripts

Specifies the different scripts, i.e. the lists of actions.

array script

```
script ::= structure
{
    script_identifier:      long-unsigned,
    actions:                array action_specification
}
```

The script_identifier 0 is reserved. If specified with an *execute* method, it results in a null script (no actions to perform).

```
action_specification ::= structure
{
    service_id:             enum,
    class_id:               long-unsigned,
    logical_name:           octet-string,
    index:                  integer,
    parameter:              service specific
}
```

Where:

- the service_id element defines which action to be applied to the referenced object:
  - (1) write attribute,
  - (2) execute specific method

© Copyright 1997-2020 DLMS User Association

scripts (continued)	- the index element defines (with service_id 1) which attribute of the selected object is affected; or (with service_id 2) which specific method is to be executed. The first attribute (logical_name) has index 1, the first specific method has index 1 as well. NOTE 1 The action_specification is limited to activate methods that do not produce any response (from the server to the client). NOTE 2 A "dummy" action specification with all elements 0 means that the action is not configured.
Method description	
execute (data)	Executes the script specified in parameter data. data ::= long-unsigned If data matches one of the script_identifiers in the script table, then the corresponding action_specification is executed.

4.5.3 Schedule (class_id = 10, version = 0)

This IC, together with the IC "Special days", allows modelling time- and date-driven activities within a device. Table 23 and Table 24 provide an overview and show the interactions between the two ICs.

Table 23 – Schedule

Index	enable	action (script)	Switch _time	validity_ window	exec_weekdays							exec_specdays					date range	
					Mo	Tu	We	Th	Fr	Sa	Su	S1	S2	…	S8	S9	begin_ date	end_ date
120	Yes	xxxx:yy	06:00	0xFFFF	*x*	*x*	*x*	*x*	*x*	*x*							xx-04-01	xx-09-30
121	Yes	xxxx:yy	22:00	15	*x*	*x*	*x*	*x*	*x*								xx-04-01	xx-09-30
122	Yes	xxxx:yy	12:00	0						*x*							xx-04-01	xx-09-30
200	No	xxxx:yy	06:30		*x*	*x*	*x*	*x*	*x*	*x*							xx-04-01	xx-09-30
201	No	xxxx:yy	21:30		*x*	*x*	*x*	*x*	*x*								xx-04-01	xx-09-30
202	No	xxxx:yy	11:00							*x*							xx-04-01	xx-09-30

Table 24 – Special days table

Index	special_day_date	day_id
12	xx-12-24	S1
33	xx-12-25	S3
77	97-03-31	S3

Schedule	**0…n**	**class_id = 10, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. entries (static)	array				x + 0x08
Specific methods	***m/o***				
1. enable/disable (data)	o				x + 0x20
2. insert (data)	o				x + 0x28
3. delete (data)	o				x + 0x30

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “Schedule” object instance. See 6.2.9.

entries	Specifies the scripts to be executed at given times. There is only one script that can be executed per entry.
	array schedule_table_entry
	
	schedule_table_entry ::= structure
	{
	index: long-unsigned,
	enable: boolean,
	script_logical_name: octet-string,
	script_selector: long-unsigned,
	switch_time: octet-string,
	validity_window: long-unsigned,
	exec_weekdays: bit-string,
	exec_specdays: bit-string,
	begin_date: octet-string,
	end_date: octet-string
	}
	
	Where:
	- script_logical_name: defines the logical name of the “Script table” object;
	- script_selector: defines the script_identifier of the script to be executed;
	- switch_time accepts wildcards to define repetitive entries. The format of the octet-string follows the rules set in 4.1.6.1 for time;
	- validity_window defines a period in minutes, in which an entry shall be processed after power fail. (time between defined switch_time and actual power_up) 0xFFFF: the script is processed any time;
	- exec_weekdays defines the days of the week on which the entry is valid;
	- exec_specdays perform the link to the IC “Special days table”, day_id;
	- begin_date and end_date define the date period in which the entry is valid (wildcards are allowed). The format follows the rules set in 4.1.6.1 for *date*.

Method description	

enable/disable (data)	Sets the disabled bit of range A entries to true and then enables the entries of range B.
	data ::= structure
	{
	firstIndexA: long-unsigned,
	lastIndexA: long-unsigned,
	firstIndexB: long-unsigned,
	lastIndexB: long-unsigned
	}

© Copyright 1997-2020 DLMS User Association

enable/ disable (data) (continued)	firstIndexA	first index of the range that is disabled,
	lastIndexA	last index of the range that is disabled,
	firstIndexB	first index of the range that is enabled,
	lastIndexB	last index of the range that is enabled,
	firstIndexA/B < lastIndexA/B:	all entries of the range A/B are disabled / enabled,
	firstIndexA/B = lastIndexA/B:	one entry is disabled/enabled,
	firstIndexA/B > lastIndexA/B:	nothing disabled/enabled,
	firstIndexA/B and lastIndexA/B > 9999:	no entry is disabled/enabled
insert (data)	Inserts a new entry in the table. If the index of the entry exists already, the existing entry is overwritten by the new entry. entry: schedule_table_entry data ::= corresponding to entry	

delete (data)	Deletes a range of entries in the table.
	data ::= structure
	{
	firstIndex: long-unsigned,
	lastIndex: long-unsigned
	}	

	firstIndex:	first index of the range that is deleted,
	lastIndex:	last index of the range that is deleted,
	firstIndex < lastIndex:	all entries of the range A/B are deleted,
	firstIndex = lastIndex:	one entry is deleted,
	firstIndex > lastIndex:	nothing deleted

Remarks concerning “inconsistencies” in the table entries:

If the same script should be executed several times at a specific time instance, then it is executed only once.

If different scripts should be executed at the same time instance, then the execution order is according to the “index”. The script with the lowest “index” is executed first.

Recovery after power failure

After a power failure, the whole schedule is processed to execute all the necessary scripts that would get lost during a power failure. For this, the entries that were not executed during the power failure have to be detected. Depending on the validity window they are executed in the correct order (as they would have been executed in normal operation).

Handling of time changes

There are four different "actions" of time changes:

a) time setting forward (by writing to the attribute *time* of the “Clock” object);
b) time setting backward (by writing to the attribute *time* of the “Clock” object);
c) time synchronization (using the method *adjust_to_quarter* of the “Clock” object);
d) daylight saving action.

© Copyright 1997-2020 DLMS User Association

All these four actions need a different handling executed by the schedule in interaction with the time setting activity.

Time setting forward

This is handled the same way as a power failure. All entries missed are executed depending on the validity_window. A (manufacturer specific defined) short time setting can be handled like time synchronization.

Time setting backward

This results in a repetition of those entries that are activated during the repeated time. A (manufacturer specific defined) short time setting can be handled like time synchronization.

Time synchronization

Time synchronization is used to correct small deviations between a master clock and the local clock. The algorithm is manufacturer specific. It shall guarantee that no entry of the schedule gets lost, or is executed twice. The validity_window attribute has no effect, because all entries have to be executed in normal operation.

Daylight saving

If the clock is put forward, then all scripts, which fall into the forwarding interval (and would therefore get lost) are executed. If the clock is put back, re-execution of the scripts, which fall into the backwarding interval, is suppressed.

4.5.4 Special days table (class_id = 11, version = 0)

This IC allows defining special dates. On such dates, a special switching behaviour overrides the normal one. The IC works in conjunction with the class "Schedule" or "Activity calendar". The linking data item is *day_id*.

Special days table	**0...n**	**class_id = 11, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. entries (static)	array				x + 0x08
Specific methods	***m/o***				
1. insert (data)	o				x + 0x10
2. delete (data)	o				x + 0x18

Attribute description	
logical_name	Identifies the “Special days table” object instance. See 6.2.8.

© Copyright 1997-2020 DLMS User Association

entries	Specifies a special day identifier for a given date. The date may have wildcards for repeating special days like Christmas.
	array spec_day_entry
	spec_day_entry ::= structure
	{
	index: long-unsigned,
	specialday_date: octet-string,
	day_id: unsigned
	}
	Where:
	- specialday_date formatting follows the rules set in 4.1.6.1 for *date*;
	- the range of the day_id shall match the length of the bit-string exec_specdays in the related object of IC “Schedule”.

Method description	
insert (data)	Inserts a new entry in the table. entry ::= spec_day_entry If a special day with the same index or with the same date as an already defined day is inserted, the old entry will be overwritten.
delete (data)	Deletes an entry in the table. index Index of the entry that shall be deleted. data ::= long-unsigned

4.5.5 Activity calendar (class_id = 20, version = 0)

This IC allows modelling the handling of various tariff structures in the meter. The IC provides a list of scheduled actions, following the classical way of calendar based schedules by defining seasons, weeks…

An “Activity calendar” object may coexist with the more general “Schedule” object and it can even overlap with it. If actions in a “Schedule” object are scheduled for the same activation time as in an “Activity calendar” object, the actions triggered by the “Schedule” object are executed first.

After a power failure, only the “last action” missed from the object “Activity calendar” is executed (delayed). This is to ensure proper tariffication after power up. If a “Schedule” object is present, then the missed “last action” of the “Activity calendar” shall be executed at the correct time within the sequence of actions requested by the “Schedule” object.

The “Activity calendar” object defines the activation of certain scripts, which can perform different activities inside the logical device. The interface to the IC “Script table” is the same as for the IC “Schedule” (see 4.5.3).

If an instance of the IC “Special days table” (see 4.5.4) is available, relevant entries there take precedence over the “Activity calendar” object driven selection of a day profile. The day profile referenced in the “Special days table” activates the day_schedule of the *day_profile_table* in the “Activity calendar” object by referencing through the day_id.

© Copyright 1997-2020 DLMS User Association

Activity calendar		**0...n**	**class_id = 20, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. calendar_name_active	(static)	octet-string				x + 0x08
3. season_profile_active	(static)	array				x + 0x10
4. week_profile_table_active	(static)	array				x + 0x18
5. day_profile_table_active	(static)	array				x + 0x20
6. calendar_name_passive	(static)	octet-string				x + 0x28
7. season_profile_passive	(static)	array				x + 0x30
8. week_profile_table_passive	(static)	array				x + 0x38
9. day_profile_table_passive	(static)	array				x + 0x40
10. activate_passive_calendar_time	(static)	octet-string				x + 0x48
Specific methods		***m/o***				
1. activate_passive_calendar (data)		o				x + 0x50

Attribute description	
	Attributes called ..._active are currently active. Attributes called ..._passive will be activated by the specific method activate_passive_calendar.
logical_name	Identifies the “Activity calendar” object instance. See 6.2.10.
calendar_name	Typically contains an identifier, which is descriptive to the set of scripts activated by the object.

season_profile	Contains a list of seasons defined by their starting date and a specific week_profile to be executed. The list is sorted according to season_start (in increasing order);
	array season
	
	season ::= structure
	{
	season_profile_name: octet-string,
	season_start: octet-string,
	week_name: octet-string
	}
	
	Where:
	- season_profile_name is a user defined name identifying the current season_profile;
	- season_start defines the starting time of the season, formatted as specified in 4.1.6.1 for *date-time*. In season_start, wildcards are allowed. If all fields are wildcards, the season will never start. When using wildcards, special care has to be taken to avoid conflicting parametrization, i.e. that the season_start of two different seasons is the same;
	NOTE The current season is terminated by the season_start of the next season.
	- week_name defines the week_profile active in this season.

© Copyright 1997-2020 DLMS User Association

week_profile_table

Contains an array of week_profiles to be used in the different seasons. For each week_profile, the day_profile for every day of a week is identified.

array week_profile

```
week_profile ::= structure
{
    week_profile_name:   octet-string,
    monday:              day_id,
    tuesday:             day_id,
    wednesday:           day_id,
    thursday:            day_id,
    friday:              day_id,
    saturday:            day_id,
    sunday:              day_id
}
```

day_id: unsigned

Where:

- week_profile_name is a user defined name identifying the current week_profile;
- Monday defines the day_profile valid on Monday;
- …

Sunday defines the day_profile valid on Sunday.

day_profile_table

Contains an array of day_profiles, identified by their day_id. For each day_profile, a list of scheduled actions is defined by a script to be executed and the corresponding activation time (start_time). The list is sorted according to start_time.

array day_profile

```
day_profile ::= structure
{
    day_id:          unsigned,
    day_schedule:    array day_profile_action
}

day_profile_action ::= structure
{
    start_time:          octet-string,
    script_logical_name: octet-string,
    script_selector:     long-unsigned
}
```

Where:

- day_id is a user defined identifier, identifying the current day_profile;
- start_time: defines the time when the script is to be executed (no wildcards); the format follows the rules set in 4.1.6.1 for *time*;
- script_logical_name: defines the *logical_name* of the “Script table” object;
- script_selector: defines the script_identifier of the script to be executed.

© Copyright 1997-2020 DLMS User Association

activate_ passive_ calendar_time	Defines the time when the object itself calls the specific method *activate_passive_calendar*. A definition with "not specified" notation in all fields of the attribute will deactivate this automatism. Partial "not specified" notation in just some fields of date and time are not allowed. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
Method description	
activate_ passive_ calendar (data)	This method copies all attributes called ..._passive to the corresponding attributes called ..._active. data ::= integer (0)

4.5.6 Register monitor (class_id = 21, version = 0)

This IC allows modelling the function of monitoring of values modelled by “Data”, “Register”, “Extended register” or “Demand register” objects. It allows specifying thresholds, the value monitored, and a set of scripts (see 4.5.2) that are executed when the value monitored crosses a threshold.

The IC “Register monitor” requires an instantiation of the IC “Script table” in the same logical device.

Register monitor		**0...n**	**class_id = 21, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. thresholds	(static)	array				x + 0x08
3. monitored_value	(static)	value_definition				x + 0x10
4. actions	(static)	array			0	x + 0x18
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “Register monitor” object instance. See 6.2.13 and 6.3.10.
thresholds	Provides the threshold values to which the attribute of the referenced register is compared. array threshold threshold: The threshold is of the same type as the monitored attribute of the referenced object.

monitored_ value	Defines which attribute of an object is to be monitored. Only values with simple data types are allowed.
	value_definition ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}

© Copyright 1997-2020 DLMS User Association

actions	Defines the scripts to be executed when the monitored attribute of the referenced object crosses the corresponding threshold. The attribute *actions* has exactly the same number of elements as the attribute *thresholds*. The ordering of the action_items corresponds to the ordering of the thresholds (see above).
	array action_set
	
	action_set ::= structure
	{
	action_up: action_item,
	action_down: action_item
	}
	
	Where:
	- action_up defines the action when the attribute value of the monitored register crosses the threshold in the upwards direction;
	- action_down defines the action when the attribute value of the monitored register crosses the threshold in the downwards direction.
	action_item ::= structure
	{
	script_logical_name: octet-string,
	script_selector: long-unsigned
	}

4.5.7 Single action schedule (class_id = 22, version = 0)

This IC allows modelling the execution of periodic actions within a meter. Such actions are not necessarily linked to tariffication (see “Activity calendar” or “Schedule”).

Single action schedule		**0...n**	**class_id = 22, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. executed_script	(static)	script				x + 0x08
3. type	(static)	enum				x + 0x10
4. execution_time	(static)	array				x + 0x18
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “Single action schedule” object instance. See 6.2.12.

executed_ script	Contains the logical name of the “Script table” object and the script selector of the script to be executed.
	script ::= structure
	{
	script_logical_name: octet-string,
	script_selector: long-unsigned
	}
	Script_logical_name and script_selector define the script to be executed.

© Copyright 1997-2020 DLMS User Association

type	enum: (1) size of execution_time = 1; wildcard in *date* allowed, (2) size of execution_time = *n*; all *time* values are the same, wildcards in *date* not allowed, (3) size of execution_time = *n*; all *time* values are the same, wildcards in *date* are allowed, (4) size of execution_time = *n*; *time* values may be different, wildcards in *date* not allowed, (5) size of execution_time = *n*; *time* values may be different, wildcards in *date* are allowed

execution_ time	Specifies the time and the date when the script is executed.
	array execution_time_date
	execution_time_date ::= structure
	{
	time: octet-string,
	date: octet-string
	}
	The two octet-strings contain *time* and *date*, in this order; *time* and *date* are formatted as specified in 4.1.6.1. Hundredths of seconds shall be zero.

4.5.8 Disconnect control (class_id = 70, version = 0)

Instances of the “Disconnect control” IC manage an internal or external disconnect unit of the meter (e.g. electricity breaker, gas valve) in order to connect or disconnect – partly or entirely – the premises of the consumer to / from the supply. The state diagram and the possible state transitions are shown in Figure 18.

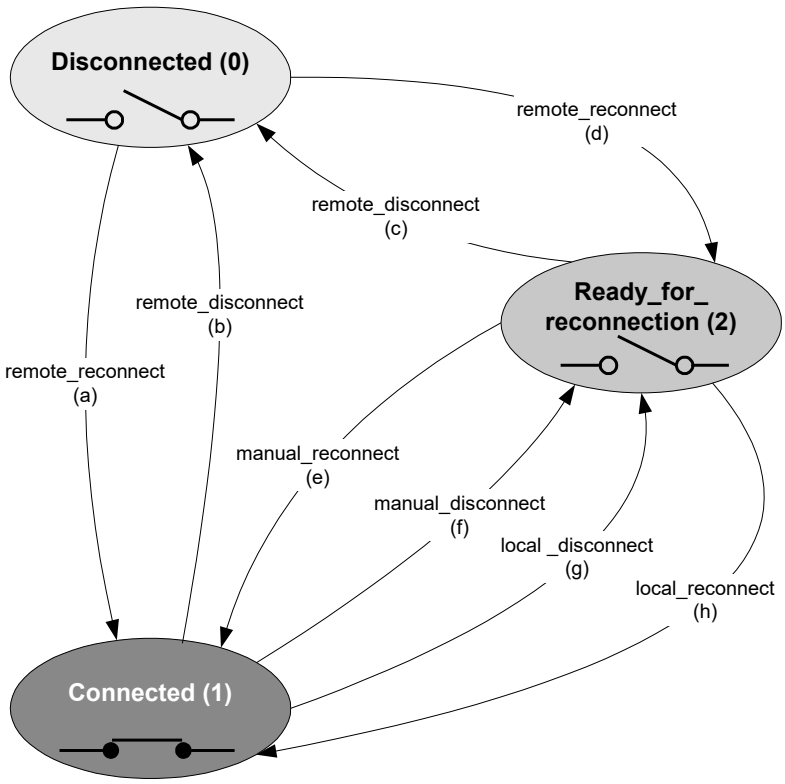


Figure 19 – State diagram of the Disconnect control IC

© Copyright 1997-2020 DLMS User Association

Disconnect and reconnect can be requested:

- Remotely, via a communication channel: remote_disconnect, remote_reconnect;
- Manually, using e.g. a push button: manual_disconnect, manual_reconnect;
- Locally, by a function of the meter, e.g. limiter, prepayment: local_disconnect, local_reconnect.

The states and state transitions of the Disconnect control IC are shown in Table 25. The possible state transitions depend on the control mode. The Disconnect control object doesn't feature a memory, i.e. any commands are executed immediately.

To define the behaviour of the disconnect control object for each trigger, the control mode shall be set.

Table 25 – Disconnect control IC – states and state transitions

States		
State number	**State name**	**State description**
0	Disconnected	The *output_state* is set to FALSE and the consumer is disconnected.
1	Connected	The *output_state* is set to TRUE and the consumer is connected.
2	Ready_for_reconnection	The *output_state* is set to FALSE and the consumer is disconnected.
State transitions		
Transition	**Transition name**	**State description**
a	remote_reconnect	Moves the “Disconnect control” object from the Disconnected (0) state directly to the Connected (1) state without manual intervention.
b	remote_disconnect	Moves the “Disconnect control” object from the Connected (1) state to the Disconnected (0) state.
c	remote_disconnect	Moves the “Disconnect control” object from the Ready_for_reconnection (2) state to the Disconnected (0) state.
d	remote_reconnect	Moves the “Disconnect control” object from the Disconnected (0) state to the Ready_for_reconnection (2) state. From this state, it is possible to move to the Connected (2) state via the manual_reconnect transition (e) or local_reconnect transition (h).
e	manual_reconnect	Moves the “Disconnect control” object from the Ready_for_connection (2) state to the Connected (1) state.
f	manual_disconnect	Moves the “Disconnect control” object from the Connected (1) state to the Ready_for_connection (2) state. From this state, it is possible to move back to the Connected (2) state via the manual_reconnect transition (e) or local_reconnect transition (h).
g	local_disconnect	Moves the “Disconnect control” object from the Connected (1) state to the Ready_for_connection (2) state. From this state, it is possible to move back to the Connected (2) state via the manual_reconnect transition (e) or local_reconnect transition (h). NOTE 1 Transitions f) and g) are essentially the same, but their trigger is different.
h	local_reconnect	Moves the “Disconnect control” object from the Ready_for_connection (2) state to the Connected (1) state NOTE 2 Transitions e) and h) are essentially the same, but their trigger is different.

© Copyright 1997-2020 DLMS User Association

Disconnect control		0…n	class_id = 70, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	***Short name***
1. logical_name	(static)	octet-string				x
2. output_state	(dyn.)	boolean				x + 0x08
3. control_state	(dyn.)	enum				x + 0x10
4. control_mode	(static)	enum				x + 0x18
Specific methods		**m/o**				
1. remote_disconnect (data)		m				x + 0x20
2. remote_reconnect (data)		m				x + 0x28

Attribute description

logical_name

Identifies the “Disconnect control” object instance. See 6.2.22 and 6.2.46.

output_state

Shows the actual physical state of the device connection the supply.

boolean: TRUE = (BOOLEAN 1) Connected,
FALSE = (BOOLEAN 0) Disconnected

NOTE 1 In electricity metering, the supply is connected when the disconnector device is closed.

NOTE 2 In gas and water metering, the supply is connected when the valve is open.

control_state

Shows the internal state of the disconnect control object.

enum: (0) Disconnected,
(1) Connected,
(2) Ready_for_reconnection

control_mode

Configures the behaviour of the disconnect control object for all triggers, i.e. the possible state transitions.

control _mode	Disconnection				Reconnection			
	Remote		Manual	Local	Remote		Manual	Local
enum:	(b)	(c)	(f)	(g)	(a)	(d)	(e)	(h)
(0)	–	–	–	–	–	–	–	–
(1)	x	x	x	x	–	x	x	–
(2)	x	x	x	x	x	–	x	–
(3)	x	x	–	x	–	x	x	–
(4)	x	x	–	x	x	–	x	–
(5)	x	x	x	x	–	x	x	x
(6)	x	x	–	x	–	x	x	x

NOTE 3 In Mode (0) the disconnect control object is always in 'connected' state.

NOTE 4 Local disconnection is always possible unless the corresponding trigger is inhibited.

Method description

remote_ disconnect (data)

Forces the “Disconnect control” object into 'disconnected' state if remote disconnection is enabled (control mode > 0).

data ::= integer (0)

© Copyright 1997-2020 DLMS User Association

remote_reconnect (data)	Forces the “Disconnect control” object into the 'ready_for_reconnection' state if a direct remote reconnection is disabled (*control_mode* = 1, 3, 5, 6). Forces the “Disconnect control” object into the 'connected' state if a direct remote reconnection is enabled (*control_mode* = 2, 4). data ::= integer (0)

4.5.9 Limiter (class_id = 71, version = 0)

Instances of the “Limiter” IC allow defining a set of actions that are executed when the value of a *value* attribute of a monitored object “Data”, “Register”, “Extended Register”, “Demand Register”, etc. crosses the threshold value for at least minimal duration time.

The threshold value can be normal or emergency threshold. The *emergency threshold* is activated via the *emergency_profile* defined by *emergency profile id*, *emergency activation time*, and *emergency duration*. The *emergency profile id* element is matched to an *emergency profile group id*: this mechanism enables the activation of the emergency threshold only for a specific emergency group.

Limiter		**0...n**	**class_id = 71, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	***Short name***
1. logical_name	(static)	octet-string				x
2. monitored_value	(static)	value_definition				x + 0x08
3. threshold_active	(dyn.)	threshold				x + 0x10
4. threshold_normal	(static)	threshold				x + 0x18
5. threshold_emergency	(static)	threshold				x + 0x20
6. min_over_threshold_duration	(static)	double-long-unsigned				x + 0x28
7. min_under_threshold_duration	(static)	double-long-unsigned				x + 0x30
8. emergency_profile	(static)	emergency_profile				x + 0x38
9. emergency_profile_group_id_list	(static)	array				x + 0x40
10. emergency_profile_active	(dyn.)	boolean				x + 0x48
11. actions	(static)	action				x + 0x50
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “Limiter” object instance. See 6.2.15.

monitored_value	Defines an attribute of an object to be monitored. Only attributes with simple data types are allowed.
	value_definition ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}

threshold_active	Provides the active threshold value to which the attribute monitored is compared. threshold: The threshold is of the same type as the attribute monitored

© Copyright 1997-2020 DLMS User Association

threshold_normal	Provides the threshold value to which the attribute monitored is compared when in normal operation. threshold: see above
threshold_ emergency	Provides the threshold value to which the attribute monitored is compared when an emergency profile is active. threshold: see above
min_over_ threshold_duration	Defines minimal over threshold duration in seconds required to execute the over threshold action.
min_under_ threshold_duration	Defines minimal under threshold duration in seconds required to execute the under threshold action.

emergency_profile	An *emergency_profile* is defined by three elements: emergency_profile_id, emergency_activation_time, emergency_duration.
	An emergency profile is activated if the emergency_profile_id element matches one of the elements on the emergency_profile_group_id_list, and time matches the emergency_activation_time and emergency_duration element:
	emergency_profile ::= structure
	{
	emergency_profile_id: long-unsigned,
	emergency_activation_time: octet-string,
	emergency_duration: double-long-unsigned
	}
	Where:
	- the emergency_activation_time element defines the date and time when the emergency_profile activated. The octet-string is encoded as specified in 4.1.6.1 for *date-time*,
	- the emergency_duration element defines the duration in seconds, for which the *emergency_profile* is activated.
	When an emergency profile is active, the *emergency_profile_active* attribute is set to TRUE.

emergency_ profile_group_id_ list	Defines a list of group id-s of the emergency profile. The emergency profile can be activated only if emergency_profile_id element of the *emergency_profile* attribute matches one of the elements on the *emergency_profile_group_id_list*: array emergency_profile_group_id emergency_profile_group_id ::= long-unsigned
emergency_ profile_active	Indicates that the emergency_profile is active.

© Copyright 1997-2020 DLMS User Association

actions	Defines the scripts to be executed when the monitored value crosses the threshold for minimal duration time.
	action ::= structure
	{
	action_over_threshold: action_item,
	action_under_threshold: action_item
	}
	Where:
	- action_over_threshold defines the action when the value of the attribute monitored crosses the threshold in upwards direction and remains over threshold for minimal over threshold duration time;
	- action_under_threshold defines the action when the value of the attribute monitored crosses the threshold in the downwards direction and remains under threshold for minimal under threshold duration time.
	
	action_item ::= structure
	{
	script_logical_name: octet-string,
	script_selector: long-unsigned
	}

4.5.10 Parameter monitor (class_id = 65, version = 1)

Instances of the “Parameter monitor” IC are used to monitor a list of COSEM object attributes that hold parameters.

If the value of an attribute in the parameter_list changes, the identifier and the value of that attribute is automatically captured to the changed_parameter attribute. The time when the change of the parameter occurred is captured in the capture_time attribute. These attributes may be captured by a “Profile generic” object. In this way, a log of all parameter changes can be built. For the OBIS code of the Parameter monitor log objects, see 7.4.5.

NOTE 1 In the case of simultaneous or quasi simultaneous parameter changes the order of capturing and logging the changed parameters has to be managed by the application.

Several Parameter monitor objects and corresponding Profile generic objects can be instantiated to manage a number of parameter groups. The link between the Parameter monitor object and the corresponding Profile generic object is via the capture_object attribute of the Profile generic object.

NOTE 2 As the various parameters may be of different type and length, the entries in the profile column holding the parameters will be also of different type and length. This can be managed by capturing different kind of parameters into different Parameter list Profile generic objects and parameter logs.

NOTE 3 The Profile generic object holding the parameter change log may capture other suitable object attributes, such as the time attribute of the Clock object and any other relevant values.

Use of the Parameter monitor IC for detecting configuration parameter changes

Each client that has access to the server has to be aware of the current parametrisation in order to be able to correctly exchange data with servers. The parameters may be grouped and each group may have a name.

Although the parametrisation may be known initially, it may change during the lifetime of the meter, for example it may be changed by another client such as a field service device.

© Copyright 1997-2020 DLMS User Association

The current configuration may always be retrieved by reading the configuration parameters at the beginning of the exchange. This is not efficient and in the case of push operation it is not practical.

A solution is needed that allows clients to verify if the configuration of the server is as expected or to detect if any change has occurred.

To achieve this, version 1 of the Parameter monitor IC specifies new attributes to hold:

- the parameter_list_name;
- the hash_algorithm_id;
- the parameter_value_digest; and
- the parameter values.

When a client configures the server, it can retrieve these attributes and share them with other clients.

When a client starts a data exchange with the server, it can read the parameter_value_digest attribute and compare it to the value it has stored. If they match, the configuration is as expected, if not, the client can read the parameter_values attribute to identify which one(s) have changed.

The attributes can also be captured into Profile generic object buffer attributes to assure the validity of the data captured. The resulting overhead can be minimized by the use of null-data encoding.

Parameter monitor	**0...n**	**class_id = 65, version = 1**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. changed_parameter	structure				x + 0x08
3. capture_time	date-time				x + 0x10
4. parameter_list	array				x + 0x18
5. parameter_list_name	octet-string				x + 0x20
6. hash_algorithm_id	enum				x + 0x28
7. parameter_value_digest	octet-string				x + 0x30
8. parameter_values	structure				x + 0x38
Specific methods	***m/o***				x + 0x40
1. add_parameter (data)	o				x + 0x48
2. delete_parameter (data)	o				x + 0x50

Attribute description	
logical_name	Identifies the Parameter monitor instance. See 6.2.14

© Copyright 1997-2020 DLMS User Association

changed_parameter	Holds the identifier and the value of the most recently changed parameter.
	structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer,
	attribute_value: CHOICE
	-- CHOICE as specified in the Data IC
	}

capture_time	Provides data and time information showing when the value of the changed_parameter attribute has been captured. Date-time is formatted as specified in 4.1.6.1.

parameter_list	Holds the list of parameters managed by a given instance of the Parameter monitor IC.
	parameter_list ::= array parameter_list_element
	
	parameter_list_element ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}
	NOTE 4 Parameters should be selected to avoid dynamically changing values, to ensure that the digest remains consistent. The Register monitor IC is intended for monitoring dynamically variable values.
	NOTE 5 The list of parameters monitored may be changed by using the add_parameter or delete_parameter methods or writing this attribute.

parameter_list_ name	Holds the name given to the list of parameter as defined by the parameter_list attribute.
hash_algorithm_id	enum (0) SHA-256, (1) SHA-384, (2) Last 16 bytes of SHA-256, (3) Last 8 bytes of SHA-256, (4) Last 4 bytes of SHA-256
parameter_value_ digest	Result of the digest calculation according to the algorithm specified by the hash_algorithm_id attribute. The digest is calculated on all the parameter list values as defined by attribute parameter_list in the order they are placed in the array. To calculate the digest, the whole set of parameter_list attribute values is first transformed into an octet-string. Rules for this transformation are defined DLMS UA 1000-2 , 9.2.3.4.3. Data conversion, for all the data types which are not an octet-string.

© Copyright 1997-2020 DLMS User Association

parameter_values	Holds a copy of the values of the attributes referenced in parameter_list attribute. It is a structure that holds the A-XDR encoded value of each attribute in the order of the parameter_list array.
Method description	
add_parameter (data)	Adds one parameter to the parameter_list attribute. data ::= parameter_list_element NOTE 6 A parameter can be logged as soon as it is added to the list. Adding an element to the list does not affect the buffer of the Profile generic object capturing the changed_parameter attribute.
delete_parameter (data)	Deletes one parameter from the parameter_list. data ::= parameter_list_element NOTE 7 When a parameter is deleted from the parameter list, its changes will not be logged any more. Removing an element from the list does not affect the buffer of the Profile generic object capturing the changed_parameter attribute.

© Copyright 1997-2020 DLMS User Association

4.5.11 Sensor manager interface (class_id = 67, version = 0)

4.5.11.1 General

Most measuring instruments under the scope of the Measuring Instruments Directive (MID) operate with dedicated sensors (transducers and transmitters) connected to the processing unit. These sensors have to be permanently supervised concerning their functioning and limits to fulfil the metrological requirements for subsequent calculation of monetary values.

In addition, the measured values have to be monitored. These values may be related to a physical quantity – raw values of voltage, current, resistance, frequency, digital output – provided by the sensor, and the measured quantities resulting from the processing of the information provided by the sensor.

It is necessary to monitor and often to log the relevant values in order to obtain diagnostic information that allows:

- the identification of the sensor device;
- the connection and the sealing status of the sensor;
- the configuration of the sensors;
- the monitoring of the operation of the sensors;
- the monitoring of the result of the processing.

The “Sensor manager” interface class allows managing detailed information related to a sensor by a single object.

For simpler sensors / devices, already existing COSEM objects – identifying the sensors, holding measurement values and monitoring those measurement values – can be used.

4.5.11.2 Sensor manager interface class specification (class_id = 67, version = 0)

Instances of the “Sensor manager” IC manage complex information related to sensors. They also allow monitoring the raw data and the processed value, derived by processing the raw-data using appropriate algorithms as required by the particular application. This IC includes a number of functions:

- nameplate data of the sensor and site information (attributes 2 to 6);
- an “Extended register” function for the *raw-value* (attributes 7 to 10);
- a “Register monitor” function for the *raw-value* (attributes 11-12);

NOTE 1 Not every raw data (e.g. the voltage output of a pressure sensor) has its own OBIS code / object. This is the reason to include raw data in the Sensor manager class.

- a “Register monitor” function for the *processed_value* (attributes 13 to 15).

NOTE 2 Not all “modules” are necessarily present. The attributes not used are possibly not implemented or not accessible.

© Copyright 1997-2020 DLMS User Association

Sensor manager		**0...n**	**class_id = 67, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. serial_number	(dyn,)	octet-string				x + 0x08
3. metrological_identification	(dyn.)	octet-string				x + 0x10
4. output_type	(dyn.)	enum				x + 0x18
5. adjustment_method	(dyn.)	octet-string				x + 0x20
6. sealing_method	(dyn.)	enum				x + 0x28
7. raw_value	(dyn.)	CHOICE				x + 0x30
8. scaler_unit	(dyn.)	structure				x + 0x38
9. status	(dyn.)	CHOICE				x + 0x40
10. capture_time	(dyn.)	date-time				x + 0x48
11. raw_value_thresholds	(dyn.)	array				x + 0x50
12. raw_value_actions	(dyn.)	array				x + 0x58
13. processed_value	(dyn.)	processed_value_ definition				x + 0x60
14. processed_value_thresholds	(dyn.)	array				x + 0x68
15. processed_value_actions	(dyn.)	array				x + 0x70
Specific methods		***m/o***				
1. reset (data)		*o*				x + 0x80

Attribute description	
logical_name	Identifies the “Sensor manager” object instance. See 6.4.4.
serial_number	Identifies the sensor (->site information). NOTE 3 For simple sensors, the serial number may be held by “Data” objects with appropriate OBIS codes if this is the only information required. See Gas related general purpose objects, Configuration objects in Table 104.
metrological_ identification	Describes metrologically relevant information of the sensor, e.g. metrological identifier, calibration date.
output_type	describes the physical output of the sensor (->site information) enum: (0) not specified, (1) 4–20 mA, (2) 0–20 mA (3) 0–5 V, (4) 0–10 V, (5) Pt100, (6) Pt500, (7) Pt1000, (8) HART, (128) manufacturer specific All other values are reserved.
adjustment_method	Describes the sensor adjustment method, e.g. by 3-Measuring-point-equation.

© Copyright 1997-2020 DLMS User Association

sealing_method	Type of seals applied to the sensor. enum: (0) none, (1) mechanical (e.g. wire or protective sticker), (2) electronic (e.g. contact), (3) software (e.g. password protection).
raw_value	Physical value from the sensor (e.g. voltage from a pressure to voltage converter). For the possible data type choices, see the “Register” IC in 4.3.2.
scaler_unit	The scaler and the unit of the *raw_value*. For the definition of *scaler_unit,* see the “Register” IC in 4.3.2.
status	Status of last *raw_value* captured (for the definition of status, see the “Extended register” IC. The status keeps information like: - sensor failure, - sensor activated / inactivated. The meaning of the elements of the *status* shall be provided for each “Sensor manager” object instance.
capture_time	Provides a “Sensor manager” object specific date and time information showing when the value of the attribute *raw_value* has been captured. *date-time* is formatted as specified in 4.1.6.1. For the possible data type choices, see the “Extended register” IC in 4.3.3.
raw_value_ thresholds	Provides the threshold values to which the *raw_value* attribute is compared. array threshold threshold: The threshold is of the same type as the raw-data.
raw_value_ actions	Defines the scripts to be executed when the *raw_value* crosses the corresponding threshold. The attribute *raw_value_actions* has exactly the same number of elements as the attribute *raw_value_thresholds*. The ordering of the *action_items* corresponds to the ordering of the thresholds. For the specification of actions, see the *Register monitor* IC in 4.5.6.

processed_value	References the attribute holding the processed value of the raw data provided by the sensor.
	processed_value_definition ::= structure
	
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}
	Note that more than one “Sensor manager” objects and processed value objects may belong to the same sensor.

processed_value_ thresholds	Provides the threshold values to which the processed value is compared. array threshold threshold: The threshold is of the same type as the value processed. (referenced by the *processed-value* attribute).

© Copyright 1997-2020 DLMS User Association

processed_value_ actions	Defines the scripts to be executed when the processed value crosses the corresponding threshold. The attribute *processed_value_actions* has exactly the same number of elements as the attribute *processed_value_thresholds*. The ordering of the *action_items* corresponds to the ordering of the thresholds. For the specification of actions, see the *Register monitor* IC in 4.5.6.
Method description	
reset (data)	This method resets the *raw_value* to the default value. The default value is an instance specific constant. The attribute *capture_time* is set to the time of the reset execution. data ::= integer (0)

4.5.11.3 Example for absolute pressure sensor

Figure 19 illustrates the definition of relevant upper and lower thresholds.

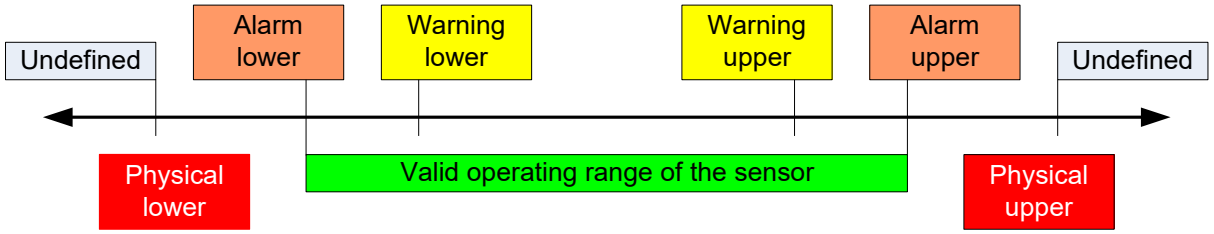


Figure 20 – Definition of upper and lower thresholds

Table 26 and Table 27 show examples of the various thresholds and the actions performed when the thresholds are crossed.

Table 26 – Explicit presentation of threshold value arrays

Threshold	Physical lower	Physical upper	Alarm lower	Alarm upper	Warning lower	Warning upper
Value	1,0	5,5	1,2	5,0	1,4	4,8
scaler_unit	1, Volt	1, Volt	1, bar	1, bar	1, bar	1, bar

Table 27 – Explicit presentation of action_sets

action_set	Physical lower	Physical upper	Alarm lower	Alarm upper	Warning lower	Warning upper
action_up	clr_phy_ alarm_bit	set_phy_ alarm_bit	clear_ alarm_bit	set_ alarm_bit	clear_ warn_bit	set_ warn_bit
action_down	set_phy_ alarm_bit	clr_phy_ alarm_bit	set_ alarm_bit	clear_ alarm_bit	set_ warn_bit	clear_ warn_bit

4.5.12 Arbitrator (class_id = 68, version = 0)

4.5.12.1 Overview

Instances of the “Arbitrator” IC allow determining, based on pre-configured rules comprising permissions and weightings, which action is carried out when multiple actors may request potentially conflicting actions to control the same resource. Generally, there is one “Arbitrator” object instantiated for each resource for which competing action requests need to be handled.

© Copyright 1997-2020 DLMS User Association

The “Arbitrator” IC allows:

- configuring the possible, potentially conflicting actions that can be requested;
- configuring the permissions for each actor to request the possible actions;
- configuring the weighting for each actor for each possible request.

NOTE 1 Examples for a resource are the supply control switch or a gas valve of the meter. Examples for possible actions are disconnect supply, enable reconnection, reconnect supply, prevent disconnection, prevent reconnection.

The actions that can be requested are held in the *actions* attribute as an array of script identifiers. The scripts – held by separate “Script table” objects – allow performing a wide range of functions. There may be actions designed to inhibit the execution of other actions: these are modelled as null-scripts, i.e. pointing to a script_identifier (0) of a “Script table” object.

NOTE 2 The “Arbitrator” objects do not contain the names of the actions, but their purpose and effect can be deduced by looking at the relevant “Script table” objects.

The permissions are held in the *permissions_table* attribute as an array of bit-strings: each element in the array represents one actor and each bit in the bit-string represents one action from the *actions* array.

The weightings are held in the *weightings_table* attribute as a two-dimensional array: each line represents one actor and there is one weight allocated to each possible action for that actor. For actions designed to inhibit other actions a very high weight may be allocated compared to the weight of the action that is to be inhibited.

Actions are requested by invoking the *request_action* method. The method invocation parameters contain:

- the identifier of the actor. This element, an unsigned number, points to a line in the *permissions_table, weightings_table* and *most_recent_requests_table* attributes;

  NOTE 3 Names of the actors may be specified in project specific companion specifications.

- the list of actions requested, in the form of a bit-string. Each bit corresponds to one element of the *actions* array: for the actions requested the bit is set to 1, for the actions not requested (inactions) the bit is set to 0. An actor may request none, one, several or all actions in a single request in one invocation. The reason to allow requesting multiple actions in a single request is to allow the actor to request an action, and at the same time to prevent another actor reversing that action.

  NOTE 4 For example, an actor may request disconnecting the supply and preventing another actor to reconnect it.

  NOTE 5 An earlier action request by an actor can be cleared by not requesting the same action (i.e. by requesting an inaction) in another invocation of the *request_action* method by the same actor. With this, a request for inhibiting an action is lifted.

The *most_recent_requests_table* attribute holds the list of the most recent request of each actor, in the form of an array of bit-strings: each element in the array represents the last request of an actor, and each bit in the bit-string represents one action / inaction requested.

When the *request_action* method is invoked by an actor the AP carries out the following activities:

- it checks the *permissions_table* attribute entry for the given actor to see if the actions requested are permitted or not;
- it updates the *most_recent_requests_table* attribute by setting or clearing the bit in the bit-string for that actor for each action requested that is also permitted (bit is set); or not requested / not permitted (bit is cleared;
- it applies the *weightings_table* for the *most_recent_requests_table:* for each bit set in the *most_recent_requests_table* the corresponding weight of each actor is applied;
- for each action, the weights are summed; and then

© Copyright 1997-2020 DLMS User Association

- if there is a unique highest total weight for an action, this value is written to the *last_outcome* attribute and the corresponding script is executed. If there is no highest unique total weight, nothing happens.

4.5.12.2 Arbitrator interface class specification (class_id = 68, version = 0)

Arbitrator		**0...n**	**class_id = 68, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. actions	(static)	array			all empty	x + 0x08
3. permissions_table	(static)	array			all bits cleared	x + 0x10
4. weightings_table	(static)	array			all zero	x + 0x18
5. most_recent_requests_table	(dyn.)	array			all bits cleared	x + 0x20
6. last_outcome	(dyn.)	unsigned	0	n	0	x + 0x28
Specific methods		**m/o**				
1. request_action (data)		m				x + 0x30
2. reset (data)		o				x + 0x38

Attribute description	
logical_name	Identifies the “Arbitrator” object instance. See 6.2.47.

actions	Defines the actions that can be requested.
	
	actions ::= array action_item
	action_item ::= structure
	{
	script_logical_name: octet-string,
	script_selector: long-unsigned
	}
	Where:
	– script_logical_name: defines the *logical_name* of the “Script table” object;
	– script_selector: defines the script_identifier of the script to be executed.
	Entries that are intended to inhibit other actions are represented by null-scripts, i.e. pointing to script_identifier (0) of a “Script table” object.

© Copyright 1997-2020 DLMS User Association

permissions_table	Contains the permissions for each actor to request actions. permissions_table ::= array actor_permissions actor_permissions ::= bit-string Each entry represents the permissions for one actor to request each action. Each bit in the bit-string corresponds to one action in the *actions* array. The length of the bit-string shall be the same as the number of elements in the *actions* array. The leading bit corresponds to the first element and the trailing bit corresponds to the last element in the *actions* array. The bits shall be set for actions allowed and cleared for actions not allowed. NOTE 1 These *actor_permissions* model business rules rather than acting as a form of access control.
weightings_ table	Holds the weight allocated for each actor and to each possible action of that actor. weightings_table ::= array actor_weighting_list actor_weighting_list ::= array actor_action_weight actor_action_weight ::= long-unsigned The number and order of elements in the *weightings_table* array shall be the same as in the *permissions_table* array. The number of elements in the actor_weighting_list array shall be the same as in the *actions* array. The first element shows the weight for the first action and the last element shows the weight of the last action. NOTE 2 It is preferred using powers of 2 as weights. Using different weights for different actors and actions helps to avoid situations when there is no unique outcome after evaluating the action request (in which case no action is performed).
most_recent_ requests_table	Holds the most recent requests of each actor. most_recent_requests_table ::= array most_recent_request most_recent_request ::= bit-string Each entry represents the most recent request of an actor. The number and order of elements in the *most_recent_requests_table* array shall be the same as in the *permissions_table* array. The length of the most_recent_request bit-string shall be the same as the number of elements in the *actions* array. The leading bit corresponds to the first element and the trailing bit corresponds to the last element in the *actions* array. For each action that has been requested and which is also permitted the bit is set. For each action that is not requested (inaction) or not allowed the bit is cleared.

© Copyright 1997-2020 DLMS User Association

last_outcome	Contains the outcome of the most recent request that has resulted a unique highest total weight for an action requested and therefore performed. The number identifies a bit in the request_action_list bit-string: the number 1 corresponds to the leading bit and consequently to the first element in the *actions* array.
Method description	

request_action (data)	Defines the actions that are requested by an actor.
	data ::= structure
	{
	request_actor: unsigned,
	request_action_list: bit-string
	}
	Where:
	– request_actor is an index into the corresponding arrays. The number 1 identifies the first entry in the *permissions_table*, the first entry of the *weightings_table* and the first entry in the *most_recent_requests_table* arrays. The number *n* identifies the highest entry in these arrays;
	– request_action_list identifies the action(s) to be performed. For each action requested the bit is set. For each action not requested (inaction) the bit is not set. The length of the bit-string shall be the same as the number of elements in the *actions* array. The leading bit corresponds to the first element and the trailing bit corresponds to the last element.
	Although several actions can be requested, only one action (modelled by a script) will be actually performed, provided that it is permitted for that actor and there is a single highest total outcome. As specified above, an action may be a null-script.

reset (data)	Clears the configurable fields of the object instance, that is: – clears all bits in the *permissions_table* attribute; – sets all values in the *weightings_table* attribute to zero; – clears all bits in the *most_recent_requests_table* attribute; – sets the *last_outcome* attribute to zero. NOTE 3 Following a reset it can be expected that every invocation of *request_action* will leave the *last_outcome* attribute equal to zero, and no action will be performed since the elements in the *permissions_table* attribute are all cleared. data ::= integer (0)

© Copyright 1997-2020 DLMS User Association

4.5.13 Modelling examples: tariffication and billing

Figure 21 shows an example of modelling tariff parametrization and management using COSEM objects.

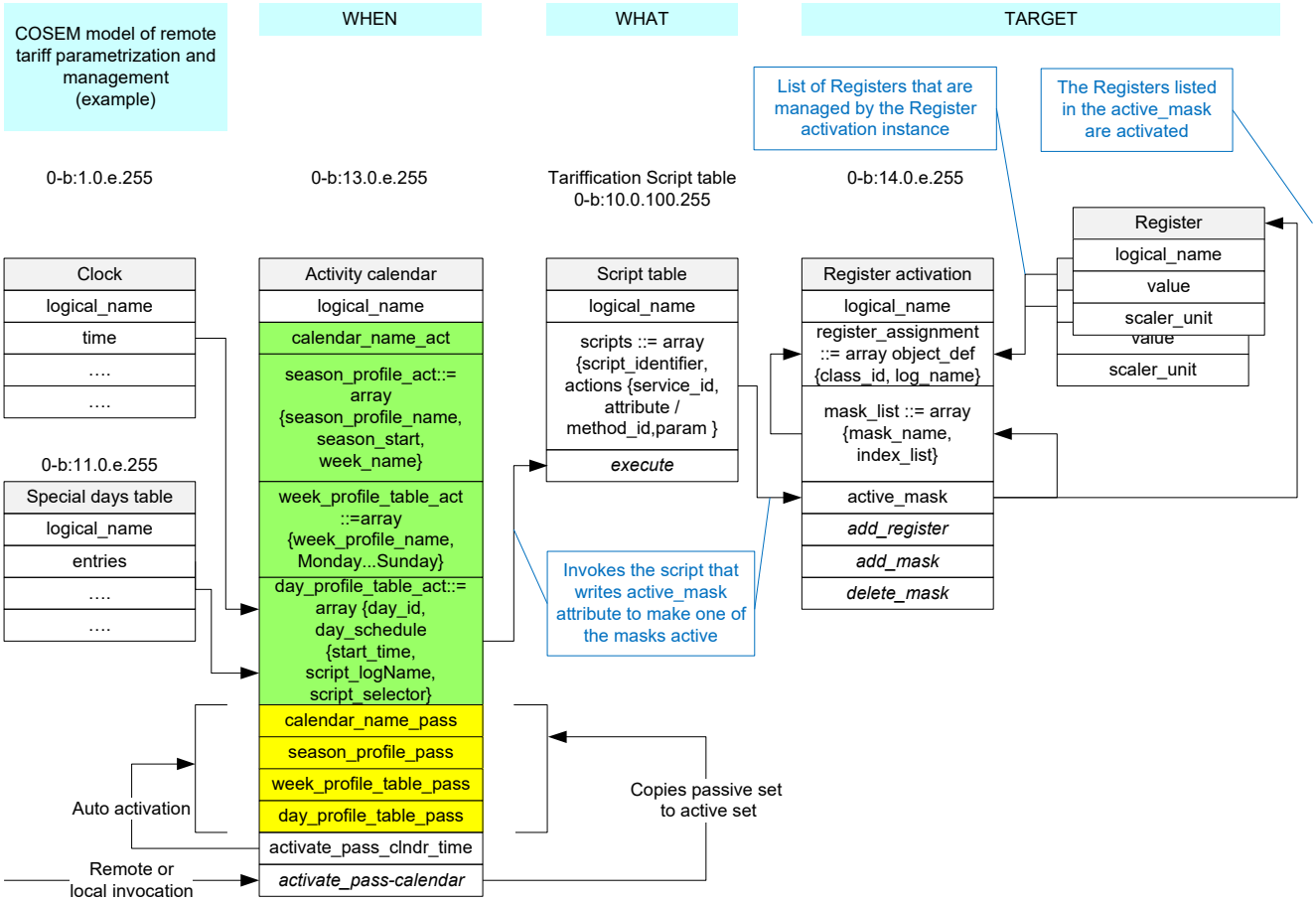


Figure 21 – COSEM tariffication model (example)

© Copyright 1997-2020 DLMS User Association

Figure 22 shows an example of modelling parametrization and management of billing using COSEM objects.

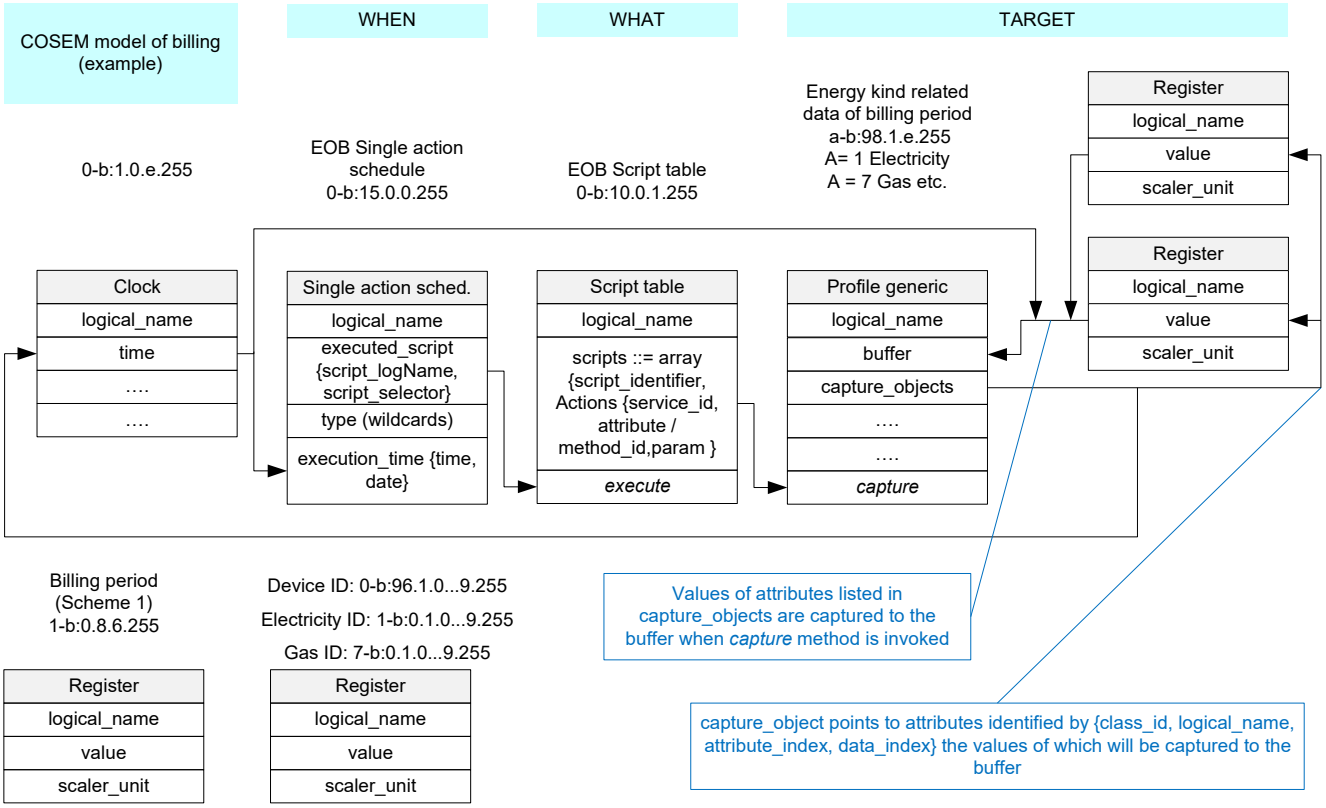


Figure 22 – COSEM billing model (example)

4.6 Payment metering related interface classes

4.6.1 Overview of the COSEM accounting model

The COSEM accounting model contains four interlinked interface classes: “Account”, “Credit”, “Charge” and the “Token gateway” IC. These classes are concerned with accounting for energy, not with delivery of that energy. The “Account” is linked to it’s associated “Credit”, “Charge” and “Token gateway” objects by use of the value group D and B field such that an “Account” with D=0 should be linked to a “Token gateway” with D=40 and have a “Credit” objects with D=10 and “Charge” objects with D=20. Whereas an “Account” with D=1 should have “token gateway” with D=41, “Credit” objects with D=11 and “Charge” objects with D=21 etc. Multiple “Token gateway”, “Credit” and “Charge” objects related to the same “Account” are identified using different values in the value group E field.

An “Account” object contains summary information and coordinates information pertaining to Credits and Charges. There is a single “Account” object per supply, for example, electricity import has one “Account” object, but a system that also has micro-generation could have a second “Account” to deal with the export of generated electricity; the second “Account” might or might not be accessible via the same Application Association (AA) as the first.

A “Credit” object contains detailed information about one source of funds. There is one or (usually) more “Credit” object(s) associated with an “Account”: for example, one object for token credit and one object for emergency credit. Both of these objects can receive credit amounts from tokens, but emergency credit can only receive credit amounts when it has been consumed (entirely or partially) and when *credit_configuration* has bit 2 (Requires the credit amount to be paid back) set.

© Copyright 1997-2020 DLMS User Association

There are several types of credit listed in IEC TR 62055-21:2005, and these are the types supported by the ”Credit” IC. There can be zero or more instances of each type of Credit.

- token_credit**:** Credit that is transferred to a meter operating in prepayment mode, normally in the form of Credit Tokens;

  NOTE 1 In a meter operating in credit mode or managed payment mode, a “Credit” object configured with type token_credit is used for recording the amount of credit used since last synchronised with a client.

  NOTE 2 The content of the token is not defined by this document and may be an amount of money or another quantity that can be accounted in a way that is equivalent to the currency used by the meter.

- reserved_credit**:** Credit that is held in reserve, which is released under specific conditions;
- emergency_credit**:** Accounting functions that deal with the calculation and transacting of credit that is released only under emergency situations. Usually the amount of emergency credit used is recovered from subsequently purchased credit token

  NOTE 3 Emergency above refers to a time when a consumer does not have any token credit, and not to any safety related situation.

- time_based_credit**:** Credit that is released on a scheduled time basis;
- consumption_based_credit**:** Credit that is released on the basis of a schedule of consumption levels. For example if a consumer keeps their consumption below a threshold, the system may release a predefined amount of credit.

A “Charge” object contains detailed information about one sink of expenditure, that is, one way in which credit is being used up. There can be one or (usually) more “Charge” objects associated with an “Account” for instance, one for energy usage, one for standing charge, and possibly one paying off a debt such as an installation charge.

There are several types of charge listed in IEC TR 62055-21:2005, but the following types, distinguished by the trigger for collection, are the ones most useful in the COSEM accounting model. There can be zero or more instances of each type of ”Charge” IC.

- consumption_based_collection**:** describes charges that are collected according to the amount of consumption that has occurred in a tariff. A price per unit is assigned to each tariff register of the energy consumed

  NOTE 4 Tariffs cannot be applied when currency is in time or energy units.

- time_based_collection**:** describes charges that are collected regularly according to the passage of time, independent of consumption in that period. This may be used to collect standing charges, or debt charge to be paid off over a period of time;
- payment_event_based_collection: describes charges that are collected from every top-up that is received, typically for debt repayment. These may be expressed as amount-based, where a fixed amount is taken from each top-up credit received (for example, the consumer pays £2 out of every vend regardless of the vend amount), or percentage_based_collection where a proportion of the amount of top-up credit received is taken (for example, with every vend the consumer pays 20% of the vend amount). Bit 0 (Percentage based collection) of the *charge_configuration* attribute of the “Charge” object specifies the method of *event_based_collection*. Figure 23 gives a general view of the account model.

© Copyright 1997-2020 DLMS User Association

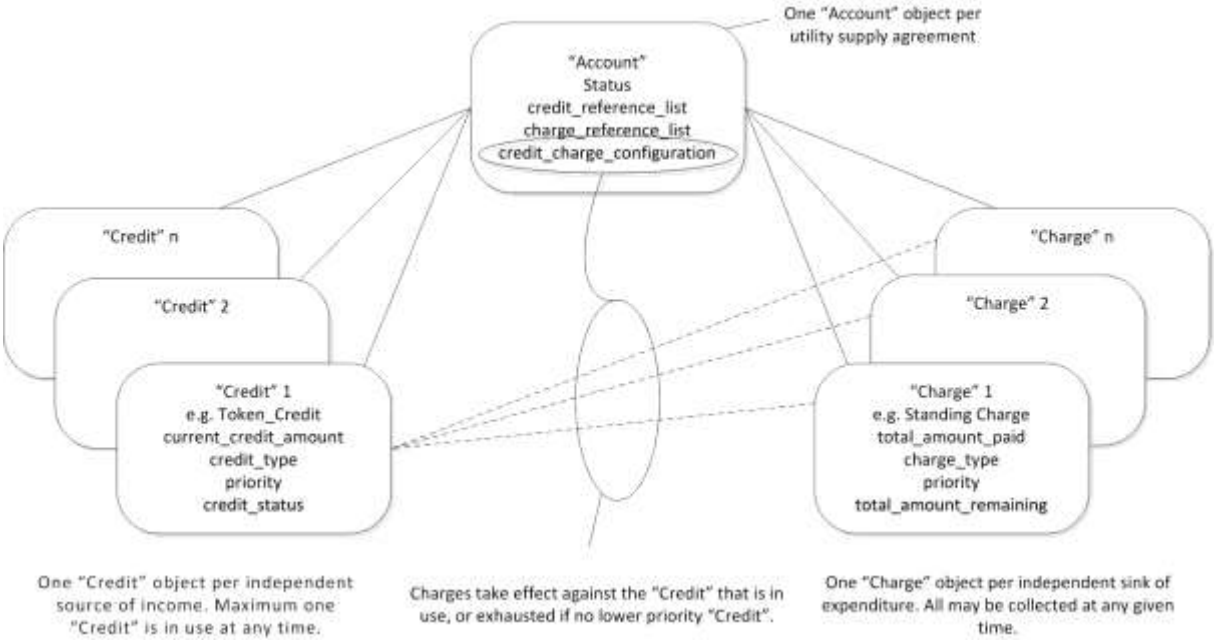


Figure 23 – Outline Account model

Figure 24 shows instances of “Account”, “Credit” and “Charge” interface classes with some of their attributes and the relationships between those attributes. In this example:

a) There is one “Account”, two “Credit” and two “Charge” objects configured;

b) “Credit” 1 is of type *token_credit* and the *low_credit_threshold* and *limit* attributes are configured to be 0;

c) Interaction between multiple classes is covered in this diagram. Detailed configuration of individual “Credit” and “Charge” objects is not shown.

© Copyright 1997-2020 DLMS User Association

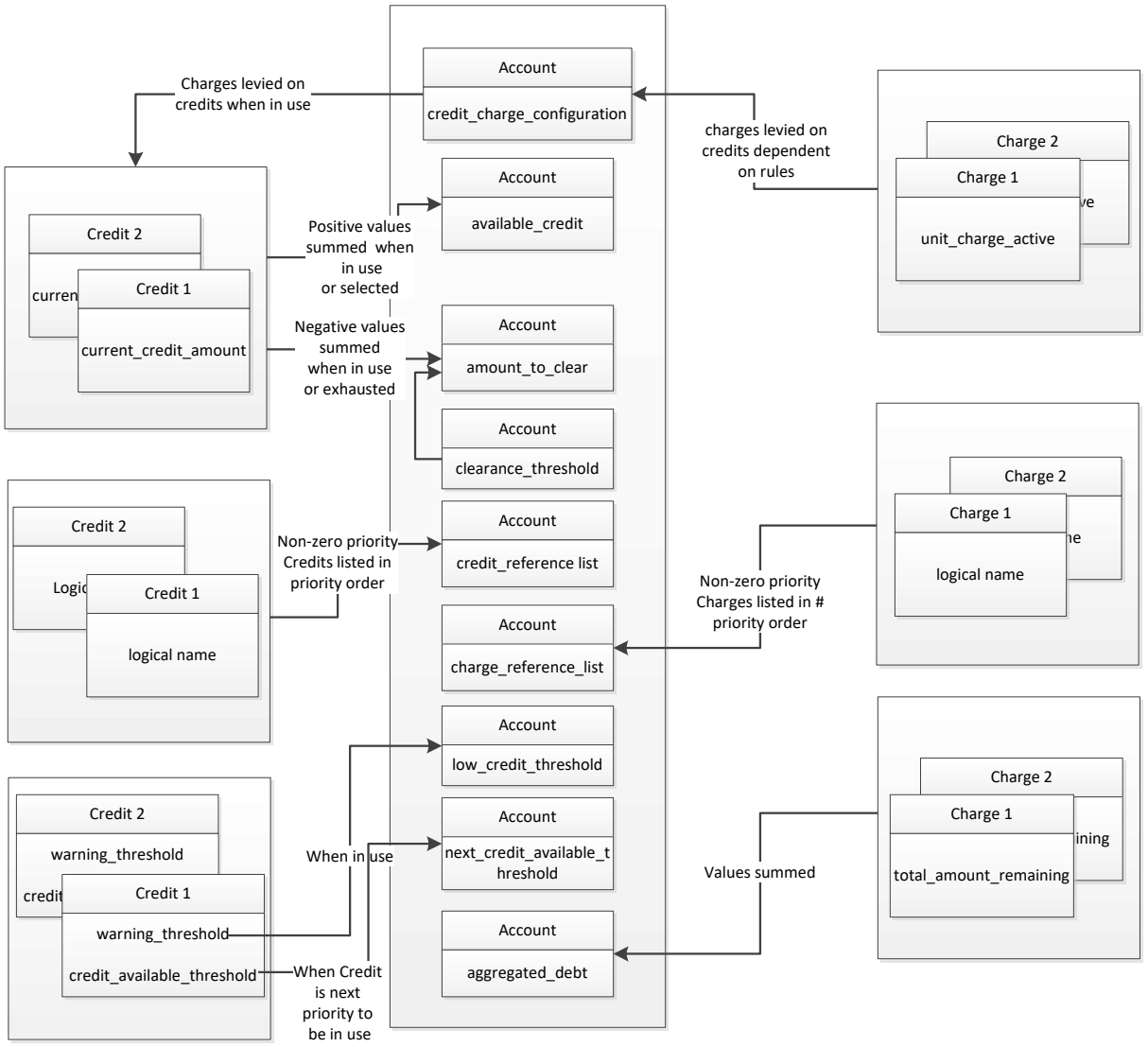


Figure 24 – Diagram of attribute relationships

4.6.2 Account (class_id = 111, version = 0)

Instances of the “Account” IC manage all the necessary elements related to the supervision of the “Credit” objects and the “Charge” objects referenced by a particular instance of the “Account” IC.

The operation of the payment metering function will be defined within the configuration of “Charge”, “Credit”, and “Account” objects and disconnection rules.

NOTE 1 Disconnection rules are either application specific or can be modelled using an instance of the “Arbitrator” IC, see 4.5.12.2.

If explicitly specified for a particular project, it is permissible for accounting to switch from credit to prepayment mode, or from prepayment to credit mode, once an accounting configuration has been correctly set up and the “Account” object has been activated. In the absence of any such specification the operating mode should remain fixed for any particular “Account” once it has been made active.

© Copyright 1997-2020 DLMS User Association

Account		**0...n**	**class_id = 111, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string			n/a	x
2. account_mode_and_status		structure			0, 0	x + 0x08
3. current_credit_in_use	(dyn.)	unsigned			0	x + 0x10
4. current_credit_status	(dyn.)	bit-string			All bits clear	x + 0x18
5. available_credit	(dyn.)	double-long			0	x + 0x20
6. amount_to_clear	(dyn.)	double-long			0	x + 0x28
7. clearance_threshold	(static)	double-long			0	x + 0x30
8. aggregated_debt	(dyn.)	double-long			0	x + 0x38
9. credit_reference_list	(static)	array			empty	x + 0x40
10. charge_reference_list	(static)	array			empty	x + 0x48
11. credit_charge_configuration	(static)	array			empty	x + 0x50
12. token_gateway_configuration	(static)	array			empty	x + 0x58
13. account_activation_time	(static)	octet-string			n/a	x + 0x60
14. account_closure_time	(static)	octet-string			n/a	x + 0x68
15. currency	(static)	structure			n/a	x + 0x70
16. low_credit_threshold	(dyn.)	double-long			0	x + 0x78
17. next_credit_available_threshold	(dyn.)	double-long			0	x + 0x80
18. max_provision	(static)	long-unsigned			0	x + 0x88
19. max_provision_period	(static)	double-long			0	x + 0x90
Specific methods		**m/o**				
1. activate_account (data)		o				x + 0x98
2. close_account (data)		o				x + 0xA0
3. reset_account (data)		o				x + 0xA8

Attribute description	
logical_name	Identifies the “Account” object instance. See 6.2.17.
account_mode_and_status	Defines the payment_mode, enumerated below, and the status of the “Account”, also enumerated. account_mode_and_status ::= structure { payment_mode: enum: (1) Credit mode, (2) Prepayment mode

© Copyright 1997-2020 DLMS User Association

account_mode_ and_status (continued)	account_status: enum: (1) New (inactive) account, (2) Account active, (3) Account closed } The payment_mode is an indication of a notional prepayment or credit/managed payment mode. NOTE 2 Payment_mode does not force some other actions in the meter, or cause a change of behaviour. The “Credit” and “Charge” objects associated with an “Account” object do not operate unless the “Account” object is in the active state. A New (inactive) “Account” object is passive and will become active at *account_activation_time* or invocation of the *activate_account* method.
current_credit_ in_use	This attribute is an index into the *credit_reference_list* indicating which “Credit” object is *In use.*
current_credit_ status	This attribute provides the status of the current “Credit” object *In use* and some information about the next priority “Credit” object. Bit 0 = in_credit, set when the *available_credit* is above zero, Bit 1= low_credit, set when the “Account” object’s *available_credit* level has passed below the “Account” object *low_credit_threshold*, NOTE 3 The *low_credit_theshold* attribute of the associated “Credit” object *In use* is echoed in that attribute. Bit 2 = next_credit_enabled, set when the “Account” object *credit_reference_list* contains at least one other “Credit” object with non-zero priority, regardless of credit level, Bit 3 = next_credit_selectable, set when the next item in the “Account” object *credit_reference_list* contains a lower priority “Credit” object with non-zero priority with a *credit_configuration* bit 1 (Requires confirmation) set such that it requires confirmation (for example “Emergency Credit” that is selectable but has not yet been selected), NOTE 4 The next “Credit” object becomes selectable when the immediately higher priority “Credit” object has reached the “Account” object’s *next_credit_available_threshold*. Bit 4 = next_credit_selected, set when the ”Account” object *credit_reference_list* contains a “Credit” object of lower priority than the current “Credit” object *In use*, and that lower priority credit object is in the *Selected/Invoked* state, NOTE 5 This is, for example, an “Emergency Credit” which has been selected but is not yet *In use*. Bit 5 = selectable_credit_in_use, set when the previously selected credit in the “Account” object *credit_reference_list* is now *In use*, Bit 6 = out_of_credit, set when the “Credit” object that was *In use* has been exhausted and no further credit is available without an action on the part of the consumer or other actor. Bit 7 = RESERVED

© Copyright 1997-2020 DLMS User Association

current_credit_ status (continued)	When the next priority ”Credit” object becomes the current “Credit” object then all bits have to update. NOTE 6 A “Credit” object becomes *Selectable* when it is the next priority “Credit” and the *next_credit_available_threshold* attribute (reflecting the *credit_available_threshold* attribute in the “Credit”) of the “Account” object is greater than or equal to the *available_credit* in the “Account” object. However if the *available_credit* becomes greater than the *next_credit_available_threshold* due to the selection of the “Credit” the status of the “Credit” remains (2) *Selected*. If the *available_credit* becomes greater than the *next_credit_available_threshold* due to a top up or method invocation to increase the *current_credit_amount* of a “Credit” objects that is (2) *Selected* or (3) *In use* then this will change the status of the “Credit” object to (0) *Enabled*.
available_credit	The *available_credit* attribute is the sum of the positive *current_credit_amount* values in the instances of the “Credit” class that: - are listed in the “Account” object *credit_reference_list;* - and have a “Credit” object *credit_status* (2) *Selected/Invoked* or (3) *In use*. This attribute holds the aggregate amount of credit available associated with the “Account*”* object. This value is positive or zero. NOTE 7 Negative values of “Credit“ object *current_credit_amount* attributes are summed in *amount_to_clear*. See also the vertical axis in Figure 26. double-long, scaled according to the *currency* attribute.
amount_to_clear	This value is the sum of: – all negative values of *current_credit_amount* in “Credit” objects that: – are listed in the “Account” object *credit_reference_list* and – have a “Credit” object *credit_status* (4) *Exhausted*, – have the *credit_configuration* bit 2 (Requires the credit amount to be paid back) is cleared, – the negative (Value * -1) of the amount of credit used from all “Credit” objects where the *credit_configuration* bit 2 (Requires the credit amount to be paid back) is set; and – the negative (Value * -1) of the value of the “Account” object *clearance_threshold* attribute; See also Figure 26. NOTE 8 ”Credit” objects where the *credit_configuration* bit 2 is set are referred to as “repayable” credits. Credit used is the difference between the *preset_credit_amount* and the *current_credit_amount* (thus a positive value) of a “Credit” object when the “Credit” is (3) *In use* or (4*) Exhausted*. In the case of non-repayable credits the credit used is not relevant. NOTE 9 The payment meter’s application process might have specific functional behaviour to allow a consumer to live in emergency credit or not. This functionality is not within the Scope of this Companion Specification This attribute is significant when the meter has accumulated a temporary debt, for instance, while an emergency credit is *In use*, and/or during a friendly credit period. This attribute contains the minimum credit that the meter will need to receive (via token receipts and method invocations on “Credit” objects) in order to clear negative credits and also make “Credits” marked as repayable (for example an emergency credit), enabled again after they have been in the (3) *In use*, (2) *Selected/Invoked* or (4) *Exhausted* state.

© Copyright 1997-2020 DLMS User Association

amount_to_clear (continued)	NOTE 10 For an explanation of the process of distributing top ups between “Credit” objects please refer to attribute 12 *token_gateway_configuration*. NOTE 11 Where a meter is being used in prepayment mode, *amount_to_clear* is usually the amount needed to reverse a disconnection. Amount to clear is shown on the vertical axis in Figure 26 as a negative value. double-long, scaled according to the *currency* attribute.
clearance_ threshold	This attribute is used in conjunction with the *amount_to_clear*, and is included in the description of that attribute. This attribute becomes relevant when a meter has consumed all credit sources and has “Credit” objects with *current_credit_amount* attributes that have values less than zero. This represents the value of credit that the *available_credit* attribute must reach in order to make repayable “Credits” enabled again. To achieve this, it is clear that enough credit has to be added to the meter to ensure that the current_credit_available attribute on all listed “Credit” objects is zero or greater. double-long, scaled according to the currency attribute.
aggregated_debt	This attribute is a simple sum of *total_amount_remaining of* all the “Charge” objects which are listed in the “Account” object *charge_reference_list,* where bit 1 (Continuous collection) of the *charge_configuration* is cleared. NOTE 12 This value is not interchangeable with *amount_to_clear*; it is provided to assist external accounting. double-long, scaled according to the *currency* attribute.
credit_ reference_list	This attribute is an array of logical names, identifying a collection of “Credit” objects that operate with this “Account” object. The elements in the array shall appear in priority order (priority 1 being first to priority n being last). Priority 0 credits shall NOT appear in this list, as by definition they are not enabled. credit_reference_list ::= array credit_reference credit_reference: octet-string
charge_ reference_list	This attribute is an array of logical names, identifying a set of “Charge” objects that operate with this “Account” object. The elements in the array shall appear in priority order (priority 1 being first to priority n being last). Priority 0 charges shall NOT appear in this list, as by definition they are not applied. charge_reference_list ::= array charge_reference charge_reference: octet-string

© Copyright 1997-2020 DLMS User Association

credit_charge_ configuration

This attribute maps out which Charges are to be collected from which Credits.

If the array has zero elements then it is assumed that any Charge may be collected from any Credit in accordance with the “Credit” objects *credit_status being (3) In use or (4) Exhausted.*

If there are entries in this array then they represent each Charge that can be collected from each Credit.

If there is a Credit from which no Charges are collected then that Credit is not consumed and will remain unchanged until a Charge is configured.

NOTE 13 Collection of a Charge will cease when reaching zero, in cases where the appropriate “Charge” object has bit 1 (continuous collection) of its *charge_configuration* cleared. This will not alter the “Account” *credit_charge_configuration.*

credit_charge_configuration ::= array
credit_charge_configuration_element

credit_charge_configuration_element ::= structure

{
credit_reference: octet-string,
charge_reference: octet-string,
collection_configuration: bit-string
}

Where *credit_reference* and *charge_reference* contain the *logical_name* of the relevant “Credit” and “Charge” object.

collection_configuration ::= bit-string

This element defines behaviour under specific conditions.

Bit 0 = Collect when supply disconnected. When set, the “Charge” referred to in *charge_reference* may be collected when supply is disconnected. When cleared, the “Charge” referred to in *charge_reference* shall not be collected when supply is disconnected.

Bit 1 = Collect in load limiting periods. When set, the “Charge” referred to in *charge_reference* may be collected when supply is in a load limiting period. When cleared, the “Charge” referred to in *charge_reference* shall not be collected when supply is in a load limiting period.

Bit 2 = Collect in friendly credit periods. When set, the “Charge” object referred to in *charge_reference* may be collected when supply is in a friendly credit period. When cleared, the “Charge” referred to in *charge_reference* shall not be collected when supply is in a friendly credit period.

NOTE 14 The meter application knows internally whether it is in a supply disconnected, load limiting period, or friendly credit period, and takes appropriate action based on the value of this attribute.

NOTE 15 It is implicit that charges are also collected at times when these specific conditions are not present.

© Copyright 1997-2020 DLMS User Association

token_gateway_ configuration

This attribute is designed to configure how a new top-up token from the “Token gateway” is to be apportioned, such that a configurable percentage of the token amount is distributed to each “Credit” object.

NOTE 16 The distribution of token top up amounts is also affected by the values of the *current_credit_amount*, *credit_type* and *credit_status* attributes of the “Credit” object.

If there are restrictions on how the credit token received through the “Token gateway“ object is distributed, then this attribute determines the minimum proportion of the credit token that is attributed to each “Credit” object referenced in the array.

The proportions in this array shall not add up to more than 100%. If the sum of the proportions is less than 100% then the remainder will be added to the “Credit” objects using the rules below for an empty *“token_gateway_configuration”* attribute.

NOTE 17 It is possible that some “Credits” will get more than their allocated proportion as a result of the process highlighted above. The credit token is always 100% distributed across the “Credit” objects.

NOTE 18 The connection between a meter’s one or more “Token gateway” objects and a specific “Account” object is not explicitly shown in the model. The management of multiple “Account” objects and multiple token gateways is the subject of project specific companion specifications. However in the normal case an “Account” object has one ”Token gateway” object and all tokens received follow the rules associated with the “Account” object with which it is associated (by application of the value group D field; see also 4.6.1).

```
token_gateway_configuration ::= array
                                token_gateway_configuration_element

token_gateway_configuration_element ::= structure
{
    credit_reference        octet-string,
    token_proportion        unsigned;
}
```

Where:

– credit_reference contains the *logical_name* of the relevant “Credit” object;

– token_proportion is scaled as one percent per integer step from 0 to 100.

If there are no entries in the array then it is assumed that there are no restrictions on processing the credit token within the meter’s payment application and credit should be applied first to the lowest priority “Credit” object with its *credit_status* set to *(3) In use or* (4) *Exhausted,* then apportioned to the other “Credit” objects in ascending order of the “Credit” object *priority* (starting with priority n and ending with priority 1).

If the “Credit” object requires a limited amount of top up (for example, an Emergency “Credit” object that is being repaid in full) then the credit used (*preset_credit_amount* less *current_credit_amount* before the top up) is taken from the Token amount and any surplus is applied to higher priority *Credit*s following the same rules. At the point when the credit used is repaid the “Credit” will move from (3) *In use* or (4) *Exhausted* to (0) *Enabled.*

See also Note 10.

© Copyright 1997-2020 DLMS User Association

account_ activation_time	Defines the time when the object itself invokes the specific method *activate_account*. A definition with "not specified" notation in all fields of the attribute will not be acted upon. Partial "not specified" notation in some fields of date and time are not allowed. When this time is in the past, this field indicates the time at which the “Account” object was activated. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
account_ closure_time	Defines the time when the object itself invokes the specific method *close_account*. A definition with "not specified" notation in all fields of the attribute will not be acted upon. Partial "not specified" notation in just some fields of date and time are not allowed. When this time is in the past, this field indicates the time at which the “Account” object was closed. octet-string, formatted as specified in 4.1.6.1 for *date-time.*

currency	Defines the “currency” unit used by all functions of an “Account” object. The *charge_per_unit* in the *unit_charge* attribute of “Charge” objects has an additional scaling factor that can be applied.
	The units could be currency or other units such as kWh, minutes; or other consumption units for different types of meters. When the units are monetary, then the name is expressed using the 3 character international symbol (GBP, USD, etc.) as defined in ISO 4217.
	
	currency ::= structure
	{
	currency_name: utf8-string,
	currency_scale: integer,
	currency_unit: enum
	}
	Where:
	currency_name: utf8-string
	as per ISO 4217
	OR
	min = minutes,
	hrs = hours,
	sec = seconds,
	kWh = kilowatt hours,
	Whr = Watt hours,
	mt3 = m^3,
	ft3 = ft^3,
	Jle = Joule
	OR
	Defined by project specific companion specification such that all characters used are lower case.
	
	The *currency_scale* indicates the exponent of a decimal multiple or submultiple of the base unit in which the relevant attribute values are expressed. Negative values indicate submultiples,
	NOTE 19 For example: if the currency is Euros and the values are expressed in tenths of one cent (one thousandth of a Euro) then the scaler will have value -3 (minus 3).

© Copyright 1997-2020 DLMS User Association

currency (continued)	currency_unit: enum: (0) time, (1) consumption, (2) monetary The *currency_unit* is an enumerated value that indicates the generic type of currency: – time (in minutes, seconds, hours etc.); – consumption (in kWhrs, Whrs, m^3, Whrs etc.); or monetary (GBP, USD, EUR etc.).
low_credit_ threshold	This attribute reflects the *warning_threshold* attribute of the "Credit" object currently *In use*. This threshold can be used to generate a warning to the consumer, for example. It has no other function. double-long, scaled according to the *currency* attribute. NOTE 20 The value of this attribute will change depending on which "Credit" object is *In use* at the time of the query.
next_credit_ available_ threshold	This threshold reflects the *credit_available_threshold* attribute of the next-priority "Credit" object (except when there is no next priority "Credit"; in which case this attribute has the value -2 147 483 648 (largest possible negative value)). When the *available_cre*dit attribute of the "Account" object becomes less than or equal to this value, the *credit_status* attribute of the next priority "Credit" object changes to: (1) *Selectable* in the case when the *credit_configuration* attribute of the "Credit" object concerned has the bit 1 (Requires confirmation) set; (2) *Selected/Invoked* in the case where the bit 1 (Requires confirmation) is cleared and the next_credit_available_threshold is greater than zero; (3) *In use* in the case where the bit 1 (Requires confirmation) is cleared and the next_credit_available_threshold is equal to zero. NOTE 21 The "next-priority" refers to the next "Credit", in priority order, that has not yet been selected. This attribute will change depending on which "Credit" is next in priority order. The *next_credit_available_threshold* will change depending on which "Credit" is *In use*. double-long, scaled according to the *currency* attribute.
max_provision	This attribute affects the operation of "Charge" objects that are configured with *charge_type* equal to (2) *payment_event_based_collection*. This attribute holds the limit amount scaled as defined in the *currency* attribute across all "Charge" objects with *charge_type* (2) *payment_event_based_collection* The limit is related to the time period defined in *max_provision_period.* NOTE 22 The combination of this attribute and *max_provision_period* are intended to provide a limit to the total value of collections from top-ups within, for example, one week. NOTE 23 If a consumer vends many times in a short period, it is possible that he pays more than is required by the utility into one of his instantiated "Charge" objects (due to the unpredictability of payment event based collection). long-unsigned, scaled according to the *currency* attribute.

© Copyright 1997-2020 DLMS User Association

max_provision_ period	This attribute holds the time period applicable for the *max_provision* attribute. This is specified in seconds. double-long
Method description	
activate_account (data)	This method allows the activation of the “Account”. The *account_status* element of *account_mode_and_status* will be set to (2) *Account active*. NOTE 24 The *account_activation_time* will be set to the time of invoking this method. data ::= integer (0)
close_account (data)	This method forces the closure of the Account. NOTE 25 This may be done pursuant to a change of supplier or change of tenancy or any other reason. The *account_status* element of *account_mode_and_status* will be set to (3) *Account closed*. The *account_closure_time* is set to the time of invoking this method. NOTE 26 When this method is invoked the Account will no longer be active. As such it’s attributes will cease to change along with the attributes of all “Credit” and “Charge” objects listed within the *credit_reference_list* and *charge_reference_list* until such time that the “Account” object becomes active again, or the “Credit” and “Charge” objects are referenced from another “Account” object. data ::= integer (0)
reset_account (data)	If the *account_status* of the attribute *account_mode_and_status* is not equal to (2) *Active account*, then this method sets the attributes of the “Account” to default values except where an attribute does not have a default value, in which case the behaviour shall be project specific. If the *account_status* element of the attribute *account_mode_and_status* is equal to 1 (New, inactive account), then this method shall have no effect. data ::= integer (0)

4.6.3 Credit interface (class_id = 112, version = 0)

4.6.3.1 General

Instances of the “Credit” IC allow the management of a credit that can be consumed by charges. There are several different credit types; each “Credit” object characterizes itself by the values of its attributes.

All “Credits” associated with one supply are listed in the *credit_reference_list* attribute of the “Account” object. “Credits” move between states by:

- top-ups;
- the adjustment of current_credit_amount by method invocation; or
- the decrement of credit by charges.

This is explained in the state diagram in 4.6.3.2. 4.6.1 lists “Credit” types as defined in IEC TR 62055-21:2005.

© Copyright 1997-2020 DLMS User Association

4.6.3.2 Credit states

The credit states only have meaning when *priority* is non-zero. They are shown in Table 28 and Figure 25. The state transitions are shown in Table 29.

Table 28 – Credit states

Priority	Credit state	Meaning
0	Any	The instance of the “Credit” object is inactive. NOTE When a “Credit” has a non-zero priority, but does not appear in any *credit_reference_list* then it has the same behaviour as if it had a zero priority.
>0	(0) Enabled	Reference to the “Credit” appears in the *credit_reference_list* of an active “Account” with a non-zero priority.
>0	(1) Selectable	The “Credit” requires some additional interaction before it can be *In use*. A credit is selectable only when it is the next priority credit, it is not exhausted and when bit 1 (Requires confirmation) of “Credit” *credit_configuration* is set.
>0	(2) Selected/Invoked	The “Credit” that was selectable has now been selected but it may not yet be *In use* (it could be that some of the higher priority ”Credit” is still being used i.e. in the case of EMC). Alternatively a “Credit” that was Enabled and did not require selection may arrive here directly if the higher priority credit has become *Exhausted*.
>0	(3) In use	The “Credit” is being used to pay *Charge*s within the meter.
>0	(4) Exhausted	The “Credit” has run out.

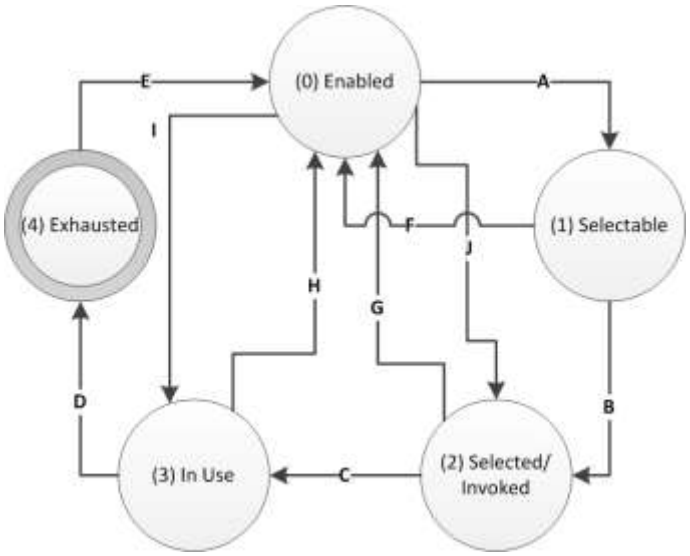


Figure 25 – Credit States when priority >0

© Copyright 1997-2020 DLMS User Association

Table 29 – Credit state transitions

Credit state	From	To	Conditions
A	(0) Enabled	(1) Selectable	A “Credit” object becomes selectable when it is the highest priority “Credit” object that is not exhausted and when the “Account” object *available_credit* reaches the “Account” object *next_credit_available_threshold*. NOTE 1 This only applies when the *credit_configuration* attribute of the “Credit” object concerned has bit 1 (Requires confirmation) set.
B	(1) Selectable	(2) Selected/ Invoked	The trigger to the transition is external to the COSEM domain such as a button push on the meter or some other stimulus. NOTE 2 This only applies when the *credit_configuration* attribute of the “Credit” object concerned has bit 1 (Requires confirmation) set.
C	(2) Selected/ Invoked	(3) In use	This transition occurs when the previous priority “Credit” object becomes exhausted. NOTE 3 At this point the “Account” object *next_credit_available_threshold* updates to reflect the *credit_available_threshold* of the next priority “Credit” object.
D	(3) In use	(4) Exhausted	This transition occurs when the *current_credit_amount attribute of the “Credit” object* reaches the level set in the *limit* attribute. NOTE 4 This may indirectly cause a disconnection of supply, in the case where there is no lower priority “”Credit” object to become *In use*. NOTE 5 At the point when the current “Credit” becomes exhausted, *credit_status* is set to(4) *Exhausted*.
E	(4) Exhausted	(0) Enabled	This transition occurs when the *current_credit_amount* becomes larger than the *limit* attribute., or – in the case of a repayable credits – the credit used has been paid back. NOTE 6 This “Credit” object has received some top-up amount from an incoming token or method invocation.
F	(1) Selectable	(0) Enabled	This transition occurs when the “Account” object *available_credit* becomes larger than the “Account” object *next_credit_available_threshold.* NOTE 7 This only applies when the *credit_configuration* attribute of the “Credit” object concerned has the bit 1 (Requires confirmation) set. NOTE 8 This is usually because the *current_credit_amount* attribute of a “Credit” object that is *In use* has been increased.
G	(2) Selected / Invoked	(0) Enabled	This transition occurs when the “Account” object *available_credit* becomes larger than the “Account” object *next_credit_available_threshold*. NOTE 9 This is usually because the *current_credit_amount* attribute of a “Credit” object that is *In use* has been increased.
H	(3) In use	(0) Enabled	This transition occurs when *the credit_status* of a higher priority “Credit” object becomes *In use*. NOTE 10 This is usually because the higher priority “Credit” object *current_credit_amount* has been increased. At this point the *next_credit_available_threshold* of the “Account” object will also change.
I	(0) Enabled	(3) In use	This transition occurs when the immediately higher priority “Credit” object becomes exhausted. NOTE 11 This only applies when the *credit_configuration* attribute of the “Credit” object concerned has the bit 1 (Requires confirmation) cleared and the *credit_available_threshold* of the “Credit” object is set to zero and it’s *current_credit_amount* is greater

© Copyright 1997-2020 DLMS User Association

Credit state	From	To	Conditions
			than the *limit* (a credit cannot be *In use* until the higher priority Credit is Exhausted).
J	(1) Enabled	(2) Selected/ Invoked	The transition occurs when the *available_credit* in the "Account" object becomes less than the *next_credit_available_threshold* on the "Account" object. NOTE 12 This only applies when the *credit_configuration* attribute of the "Credit" object concerned has the bit 1 (Requires confirmation before it can be selected or *In use*) cleared, and the *credit_available_threshold* of this "Credit" object is greater than zero.

4.6.3.3 Current credit status flags

The significance of the *current_credit_status* flags (this is held by the *current_credit_status* attribute of the "Account" object) is explained in Figure 25 below.

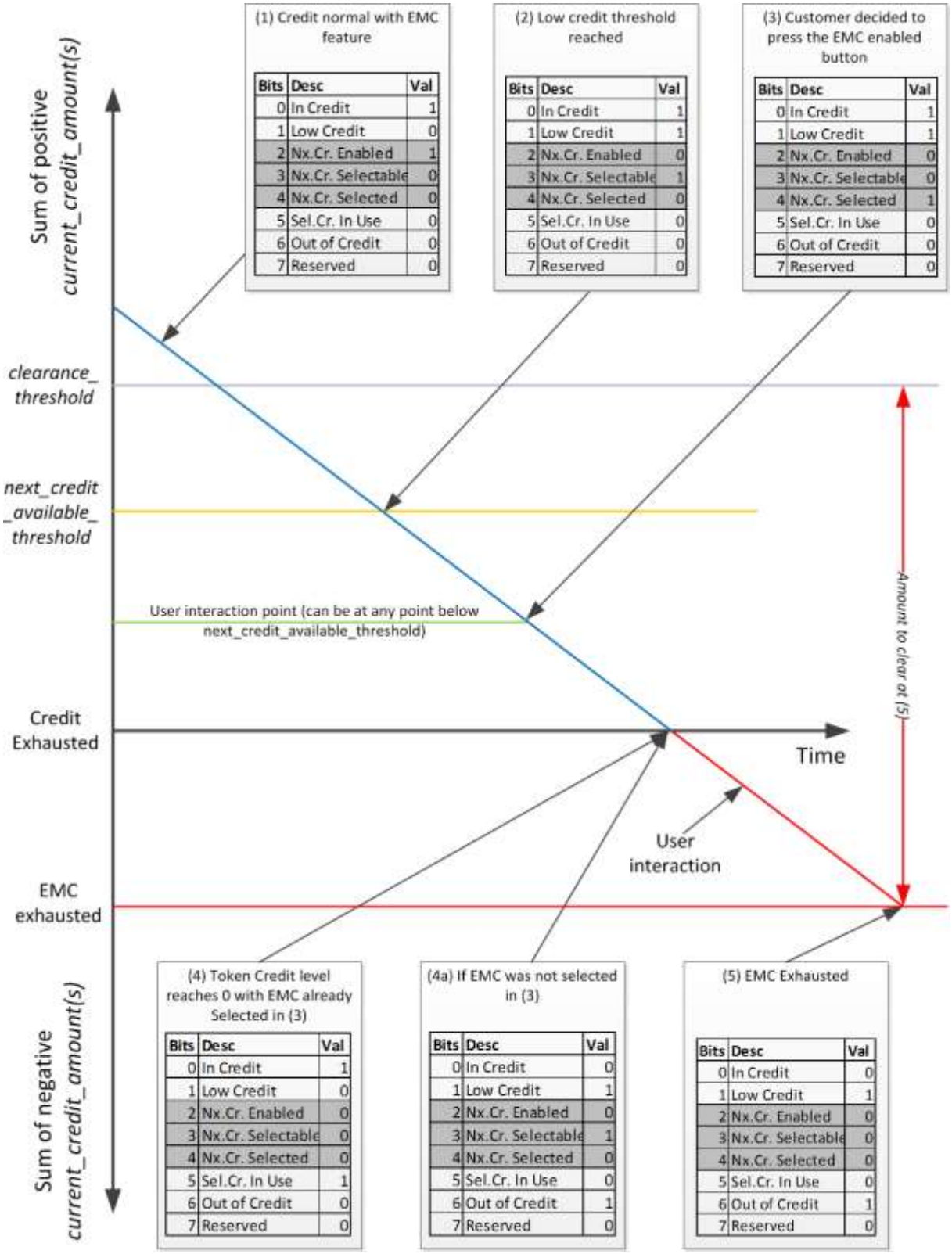
In Figure 26 an example is given for possible cases involving a payment metering system with two "Credit" objects: Token Credit and Emergency Credit.

NOTE This is one possible implementation. The type and number of credits used in an actual project are subject to project specific companion specifications.

There are two options for selection of selectable credits; either selection of a "Credit" whilst the current "Credit" is *In use* (case 4) or when the current "Credit" has been exhausted (case 4a).

In the case of credit being selected while current credit is *In use* (case 4), credit will start to be consumed from the next credit immediately after current credit has exhausted. In the case (4a) the current credit will become exhausted and potentially continue to be decremented by charges until the consumer takes action by selecting the selectable credit or adding a top up.

© Copyright 1997-2020 DLMS User Association



Bits	Desc	Val
0	In Credit	1
1	Low Credit	0
2	Nx.Cr. Enabled	1
3	Nx.Cr. Selectable	0
4	Nx.Cr. Selected	0
5	Sel.Cr. In Use	0
6	Out of Credit	0
7	Reserved	0

Bits	Desc	Val
0	In Credit	1
1	Low Credit	1
2	Nx.Cr. Enabled	0
3	Nx.Cr. Selectable	1
4	Nx.Cr. Selected	0
5	Sel.Cr. In Use	0
6	Out of Credit	0
7	Reserved	0

Bits	Desc	Val
0	In Credit	1
1	Low Credit	1
2	Nx.Cr. Enabled	0
3	Nx.Cr. Selectable	0
4	Nx.Cr. Selected	1
5	Sel.Cr. In Use	0
6	Out of Credit	0
7	Reserved	0

Bits	Desc	Val
0	In Credit	1
1	Low Credit	0
2	Nx.Cr. Enabled	0
3	Nx.Cr. Selectable	0
4	Nx.Cr. Selected	0
5	Sel.Cr. In Use	1
6	Out of Credit	0
7	Reserved	0

Bits	Desc	Val
0	In Credit	0
1	Low Credit	1
2	Nx.Cr. Enabled	0
3	Nx.Cr. Selectable	1
4	Nx.Cr. Selected	0
5	Sel.Cr. In Use	0
6	Out of Credit	1
7	Reserved	0

Bits	Desc	Val
0	In Credit	0
1	Low Credit	1
2	Nx.Cr. Enabled	0
3	Nx.Cr. Selectable	0
4	Nx.Cr. Selected	0
5	Sel.Cr. In Use	0
6	Out of Credit	1
7	Reserved	0

Figure 26 – Operation of current_credit_status flags

© Copyright 1997-2020 DLMS User Association

4.6.3.4 Credit (class_id = 112, version = 0)

Credit		0...n	class_id = 112, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. current_credit_amount	(dyn.)	double-long				x + 0x08
3. credit_type	(static)	enum				x + 0x10
4. priority	(static)	unsigned				x + 0x18
5. warning_threshold	(static)	double-long				x + 0x20
6. limit	(static)	double-long				x + 0x28
7. credit_configuration	(static)	bit-string				x + 0x30
8. credit_status	(dyn.)	enum				x + 0x38
The following attribute (9) applies to emergency, some time-based (could be reserved credit), and consumption based credits only.						
9. preset_credit_amount	(static)	double-long				x + 0x40
10. credit_available_threshold	(static)	double-long				x + 0x48
The following attribute applies to time-based, and consumption based credit only.						
11. period	(static)	date-time				x + 0x50
Specific methods		***m/o***				
1. update_amount (data)		o				x + 0x58
2. set_amount_to_value (data)		o				x + 0x60
The following method applies to emergency, and time-based consumption based credit only.						
3. invoke_credit (data)		o				x + 0x68

Attribute description	
logical_name	Identifies the “Credit” object instance. See 6.2.17.
current_credit_amount	Provides the credit value of this particular “Credit” object. (See Figure 27). NOTE 1 This value is increased and decreased by invoking the methods of this object, the action of top-ups, and the collection of charges. This value contributes to the *available_credit* attribute in the “Account” object from which the “Credit” object is referenced. double-long, scaled according to the *currency* attribute of the “Account” object.
credit_type	This is an enumeration which identifies the type of Credit that this object represents. The type indicates which attributes are expected to be processed for this “Credit” object, but the actual processing is directed by values of other attributes including the configuration flags. The types available are: enum: (0) token_credit, (1) reserved_credit, (2) emergency_credit, (3) time_based_credit, (4) consumption_based_credit

© Copyright 1997-2020 DLMS User Association

priority	Describes the activation priority of this “Credit” object. Value 1 is the highest priority and value 255 the lowest. Every “Credit” object shall have a different priority, except in the case of priority 0. NOTE 2 Behaviour will be undefined if there are multiple “Credit” objects configured with the same non-zero priority. A value of 0 indicates that the “Credit” object is never activated, although it can still be configured and its *current_credit_amount* can be adjusted by invoking its methods, but it cannot receive top ups. When a “Credit” object of priority zero is configured it shall not appear in the “Account” *credit_reference_list*, it will be not considered as part of the *available_credit* calculation and it shall not be decremented by charges. See the *credit_reference_list* attribute of the “Account” object for more information.
warning_ threshold	Holds a threshold value for *current_credit_amount.* When *current_credit_amount* is decremented to the value *of warning_threshold*, a warning is triggered for the consumer that credit is low. NOTE 3 The *low_credit_theshold* attribute of the associated “Account” object echoes this attribute of the currently *In use* “Credit” object. double-long, scaled according to the *currency* attribute of the “Account” object.
limit	This attribute holds a threshold value for *current_credit_amount*. When *current_credit_amount* is decremented to the value of *limit*, then *credit_status* becomes (4) *Exhausted*. NOTE 4 This is typically set to zero in a meter operating in prepayment mode. For a meter operating in credit mode the highest-priority “Credit” object would normally have a limit equal to the largest possible negative number. double-long, scaled according to the *currency* attribute of the “Account” object.
credit_ configuration	Allows configuring the behaviour of the “Credit” object. If **bit 0** is set then the credit item requires a visual indication on the meter display when it becomes selected. If **bit 1** is set, confirmation is required from the consumer before moving into this type of “Credit” (for example this is the case of an emergency credit object, where the consumer is required to press a button before the “Credit” is (2) Selected/Invoked) If **bit 2** is set then this indicates that this “Credit” amount requires repayment and consequently the credit used will contribute to the *amount_to_clear* attribute in the “Account” object that references the particular “Credit” object. If **bit 3** is set then the *current_credit_amount* can be cleared by an end of billing period action (using a script acting upon the *set_amount_to_value* method. It is not recommended to configure “Credits” to permit this unless the meter is operating in credit mode. If **bit 4** is set then this “Credit” object will be permitted to receive credit from tokens.

© Copyright 1997-2020 DLMS User Association

credit_ configuration (continued)	credit_configuration: bit-string: Bit 0 = Requires visual indication, Bit 1 = Requires confirmation before it can be selected/invoked, or Bit 2 = Requires the credit amount to be paid back, Bit 3 = Resettable, Bit 4 = Able to receive credit amounts from tokens
credit_status	Driven by the prepayment application to indicate the state that the “Credit” object is in. The states appear in Figure 24 above. Depending on the type of the “Credit” and its configuration, the value of this attribute is driven by the application based on the values of the *current_credit_amount*, *limit*, *preset_credit_amount* and *credit_available_threshold* attributes of the “Credit” objects and the *amount_to_clear* attribute of the “Account”. When the *amount_to_clear* attribute of the ‘Account” object referencing this “Credit” object transitions from a negative *value* to zero the EMC status shall be (0) *Enabled* again. NOTE 5 When a “Credit” object has a *credit_status* (4) *Exhausted* then this “Credit” cannot be invoked again until the “Credit” transitions to (0) *Enabled*. If this is the lowest priority “Credit” object, then charges may still be applied to this object. enum: (0) The instance of the credit class is in the state of *Enabled*, (1) The instance of the credit class is in the *Selectable* state, (2) The instance of the credit class is in the *Selected/Invoked* state, (3) The instance of the credit class is in the *In use* state and may be consumed by charges, (4) The *current_credit_amount* has been entirely consumed and the credit is considered *Exhausted*. For additional explanation see Table 28.
preset_credit_ amount	This attribute is a value that is set as an initial amount of credit that is available for the “Credit” object. The value is added to the *current_credit_amount* attribute: a) when the *credit_status* becomes (2), *Selected/Invoked* if *credit_configuration* bit 1 (Requires confirmation) is set, and the application confirmation occurs, or b) when the *credit_status* becomes (3) *In use* if *credit_configuration* bit 1 (Requires confirmation) is cleared unless the *credit_status* is (4) *Exhausted*, or c) when the method *invoke_credit* is invoked, if the *credit_configuration* bit 2 (Requires the credit amount to be paid back) is set, or d) when the date-time in *period* occurs which is implicit. When a “Credit” does not require a preset amount, the *preset_credit_amount* shall be 0 and the “Credit” can receive credit amounts from credit tokens or by invocation of the *update_amount* and *set_amount_to_value* methods. The value of *preset credit_amount* does not change unless a new value is written by the client.

© Copyright 1997-2020 DLMS User Association

preset_credit_ amount (continued)	In the case of “Credits” of type *emergency_credit*: - the *preset_credit_amount* attribute shall be used, - the “Credit” shall only be able to receive credit amounts from tokens when: - the status is (3) *In use or* (4) *Exhausted*; - some (or all) credit has been used; and - it is required to be paid back (*credit_configuration* has bit 2 (Requires the credit amount to be paid back) set. NOTE 6 When the *current_credit_amount* has reached the *limit* then the *credit_status* attribute will become (4) *Exhausted*. If the *credit_configuration* bit 2 (Requires the credit amount to be paid back) is set, then the credit used is the difference between *preset_credit_amount* and *current_credit_amount* (positive value), and it would be usual that it is repaid by the addition of a credit token or by means of invoking a method. After paying back the “Credit” the *current_credit_amount* will be zero but the *credit_status* will be (0) Enabled in the case of *amount_to_clear* = 0 or (4) Exhausted in the case where *amount_to_clear* is less than zero. If this attribute is set to zero there is no behaviour associated with it. double-long, scaled according to the *currency* attribute of the “Account” object.
credit_ available_ threshold	A threshold value related to the “Account” object *available_credit.* When the *available_credit* on the “Account” object decrements to the *credit_available_threshold* on the next-priority “Credit” object, the *credit_status* of that object changes to: e) 1 (Selectable) if the *credit_configuration* bit 1 (Requires confirmation) is set, or f) 2 (Selected/Invoked) if the *credit_configuration* bit 1 (Requires confirmation) is cleared. NOTE 7 This reflects whether the “Credit” requires confirmation before it is put into use. NOTE 8 A “Credit” will not be *In use* until exhaustion of a higher-priority “Credit”, but if selected before a higher priority “Credit” is exhausted the value of the *current_credit_amount* attribute will contribute to *available_credi*t in the associated “Account”. double-long, scaled according to the *currency* attribute of the “Account” object.
period	Where the *credit_type* = 3 (time_based_credit) or *credit_type* = 4 (consumption_based_credit), this attribute holds the time at which the *current_credit_amount* is set automatically to the *preset_credit_amount*. octet-string, formatted as specified in 4.1.6.1 for *date-time.* Wildcards are allowed. If all fields are wildcards, the *preset_credit_amount* will never be added.
Method description	
update_amount (data)	This method adjusts the value of the *current_credit_amount* attribute. Positive values adjust the *current_credit_amount* positively. Negative values are also permitted. data ::= double-long, scaled according to the *currency* attribute of the “Account” object.

© Copyright 1997-2020 DLMS User Association

set_amount_ to_value (data)	This method sets the value of the *current_credit_amount* attribute. The amount previously in the updated attribute shall be given as the response parameter. data ::= double-long, scaled according to the *currency* attribute of the "Account" object. Returns double-long, scaled according to the *currency* attribute of the "Account" object.
invoke_credit (data)	This method is invoked to bring this "Credit" object to *credit_status* (2)*Selected/Invoked* if it has a *credit_configuration* bit 1 is set (Requires confirmation), and the *credit_status* is (1) *Selectable*. The mechanism for selecting the "Credit" is not in the scope of COSEM and shall be specified by the implementer (e.g. button push, meter process, script etc.) data ::= integer (0)

4.6.3.5 Additional remarks

a) Although it is usual that tokens cause the incrementing of *current_credit_amount* and application of "Charge" objects cause decrement, these inputs are applied by means of signed addition and it could be possible to accept tokens or "Charge" objects with negative values.

b) Behaviour of the emergency_credit type is determined by the *preset_credit_amount* attribute and the 'Requires the credit amount to be paid back' option in *credit_configuration* attribute.

   When the "Credit" object is selected/invoked, the value of the *preset_credit_amount* attribute is added to the value of the *current_credit_amount* attribute (which is normally zero at this point in its lifecycle).

c) To replenish emergency credit and make its status (0) Enabled after being (3) *In use*, sufficient payment should be received to take the *available_credit* of the "Account" object up to at least the *clearance_threshold.*

   This will necessarily involve providing sufficient funds to repay the negative value of *current_credit_amount* of the "Credit" objects providing EMC function. To allow for the joint management of multiple "Credit" instances the credit used values for each "Credit" object are aggregated into the *amount_to_clear* together with the *clearance_threshold* attribute of the relevant "Account" object. When *amount_to_clear* reaches zero the status (4) *Exhausted* is by definition cleared on all associated "Credit" objects.

d) The "Account" object *token_gateway_configuration* attribute determines the distribution of credit to "Credit" objects.

Figure 27 shows interaction of the *current_credit_amount* attributes of two "Credit" Objects in a payment metering system. It can be seen that the Emergency Credit, when selected contributes to the *available_credit* but does not become *In use* until the Token "Credit" has become *Exhausted*. At the point where Emergency "Credit" is *In use* it starts contributing to the *amount_to_clear*. When the Emergency "Credit" is *In use* and contributing to *amount_to_clear*, the *amount_to_clear* also takes into consideration the clearance threshold. The clearance threshold is only important when Token "Credit" *current_credit_amount* is zero or below.

© Copyright 1997-2020 DLMS User Association

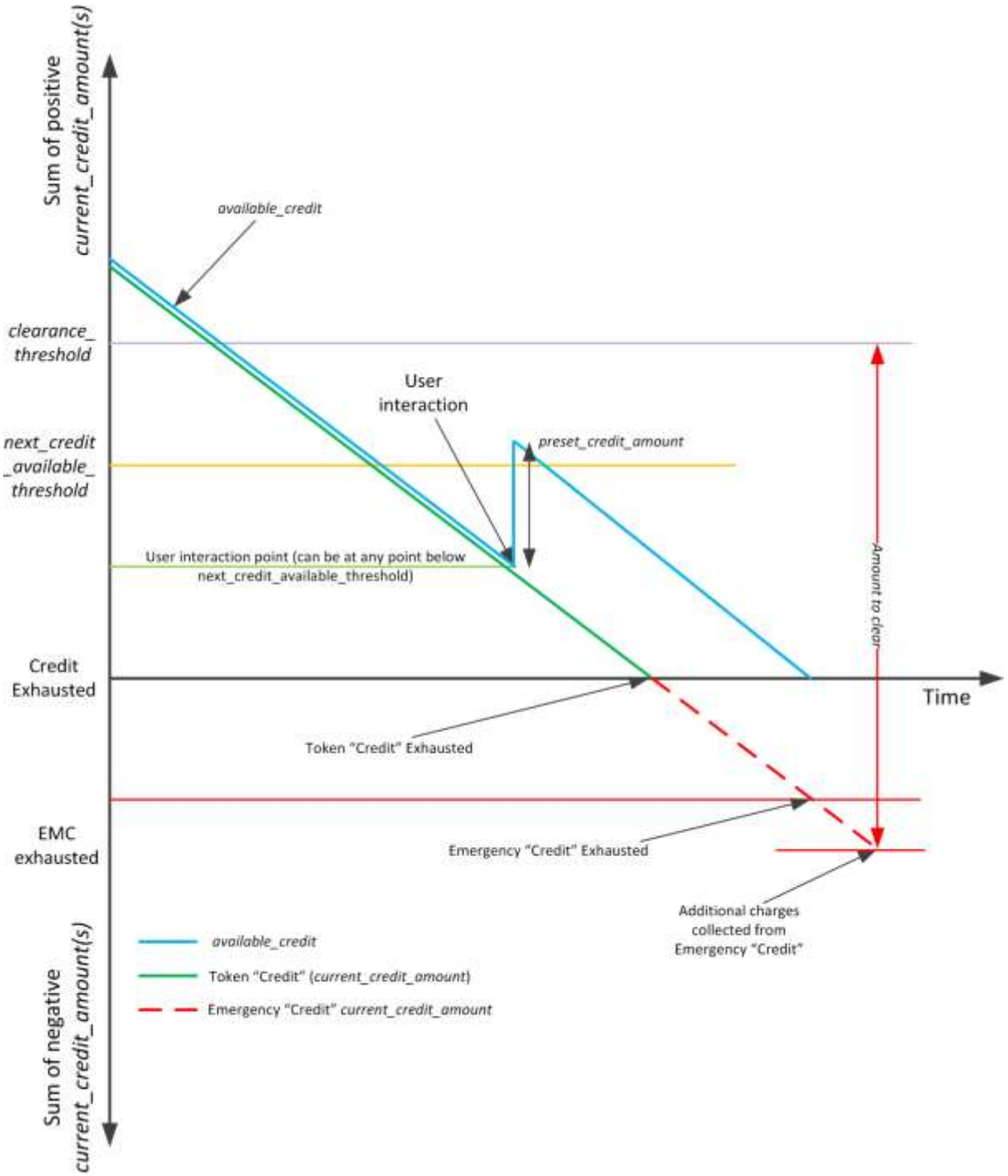


Figure 27 – Interaction of *current_credit_amount* and *available_credit* with Token "Credit" and Emergency "Credit"

As a result of invoking the Emergency "Credit" the *available_credit* will become bigger than *next_credit_available_threshold* but the Emergency "Credit" will not change *credit_status* from (1) *Selected* to (0) *Enabled.* However if the top up or method invocation to *set_amount_to_value*, forced the *available_credit* (by means of increasing the *current_credit_amount* of a "Credit" object) to be more than *next_credit_available_threshold* then the status shall be forced to (0) *Enabled.*

© Copyright 1997-2020 DLMS User Association

4.6.4 Charge (class_id = 113, version = 0)

Instances of the “Charge” IC allow the management of a single Charge. Depending on the attributes configured such as amount per price and the period, the Charge is taken at appropriate times from the “Credit” object *In use*.

NOTE 1 The details of the collection (charge-taking) cycle may be project dependent, and thus they are subject to project specific companion specifications since they affect third-party estimates of current values.

Each “Charge” object characterises itself by the values of its attributes.

All “Charges” associated with one supply are referenced in the *charge_reference_list* attribute of the related “Account” object.

Charge		**0...n**	**class_id = 113, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. total_amount_paid	(dyn.)	double-long				x + 0x08
3. charge_type	(static)	enum				x + 0x10
4. priority	(static)	unsigned				x + 0x18
5. unit_charge_active	(static)	structure				x + 0x20
6. unit_charge_passive	(static)	structure				x + 0x28
7. unit_charge_activation_time	(static)	octet-string				x + 0x30
The following attribute relates to time based and consumption based collection only.						
8. period	(static)	double-long-unsigned				x + 0x38
The following attributes relate to all charge types.						
9. charge_configuration	(static)	bit-string				x + 0x40
10. last_collection_time	(dyn.)	date-time				x + 0x48
11. last_collection_amount	(dyn.)	double-long				x + 0x50
12. total_amount_remaining	(dyn.)	double-long				x + 0x58
The following attributes relate to payment event based collection only.						
13. proportion	(static)	long-unsigned				x + 0x60
Specific methods		***m/o***				
1. update_unit_charge (data)		o				x + 0x68
2. activate_passive_unit_charge (data)		o				x + 0x70
The following methods relate to time-based and payment event based collection only .						
3. collect (data)		o				x + 0x78
The following methods relate to payment event based collection only.						
4. update_total_amount_remaining (data)		o				x + 0x80
5. set_total_amount_remaining (data)		o				x + 0x88

Attribute description	
logical_name	Identifies the “Charge” object instance. See 6.2.17.

© Copyright 1997-2020 DLMS User Association

total_ amount_paid	Holds the total amount collected for this ”Charge” object. It is not normally reset while the “Account” remains active. NOTE 2 This value is incremented by the application at the same time and at the same rate as the charge that is collected.
charge_type	Designates the type of collection associated with this instance of the “Charge” class. enum: (0) consumption_based_collection, (1) time_based_collection, (2) payment_event_based_collection The *charge_type* indicates which attributes are expected to be processed for this “Charge” object. The processing is also influenced by the *charge_configuration* attribute. NOTE 3 In the case of *payment_event_based_collection* charges this attribute dictates the mechanism of charge collection. The application collects *payment_event_based_collection* charges from appropriate “Credits” as soon as credit is distributed from a new token.
priority	Describes the priority of this particular “Charge” when making collection from a “Credit” object. Every “Charge” object shall have a different priority, except in the case of priority 0. A value of 1 represents highest priority and a value of 255 the lowest. A value of 0 indicates that the “Charge” is not activated, though it can still be configured and its *total_amount_remaining* can be adjusted by invoking the appropriate methods of the “Charge” object. “Charge” objects with priority value 0 shall not appear in the *charge_reference_list* of the “Account” object, but “Charge” objects with *priority* <> 0 do necessarily appear in the *charge_reference_list* of an “Account”.

unit_charge_ active	Defines the active price, that is the amount charged per unit consumed, per unit time, or per payment received, in terms of the currency attribute of the relevant “Account” instance and where relevant, the *scaler_unit* attribute of the object identified by the *commodity_reference* structure.
	
	unit_charge_active ::= structure
	{
	charge_per_unit_scaling: charge_per_unit_scaling_type,
	commodity_reference: commodity_reference_type,
	charge_table: charge_table_type
	}
	Where:
	charge_per_unit_scaling_type ::= structure
	{
	commodity_scale : integer,
	price_scale: integer
	}

© Copyright 1997-2020 DLMS User Association

unit_charge_ active (continued)

```
commodity_reference_type ::=    structure
{
        class_id:               long-unsigned,
        logical_name:           octet-string,
        attribute_index:        integer
}

charge_table_type ::= array  charge_table_element

charge_table_element ::=    structure
{
        index:                  octet-string,
        charge_per_unit:        long
}
```

Explanations related to charge_per_unit_scaling

Where the *charge_type* = (0) *consumption_based_collection* the field in *charge_per_unit_scaling* is expressed as the exponent (to the base of 10) of the multiplication factor of the base unit in which the relevant attribute values are expressed internally:

– commodity_scale is applied to the units associated with *commodity_reference*,

– price_scale is applied to the *currency_scale* of the *currency* attribute of the relevant “Account” object.

Where the *charge_type* IS NOT (0) *consumption_based_collection* , the *commodity_scale* shall be set to 0 and the *price_scale* is expressed as the exponent (to the base of 10) of the *currency_scale* to be applied to the *currency* attribute of the “Account” object.

NOTE 4 For example, suppose the primary *currency_name* is Euros (as determined in the “Account” object) with values expressed in units of one cent (one hundredth of a Euro) and the commodity being supplied is active import electrical energy with values expressed in units of 1 000 Wh (one kWh) from the *scaler_unit* attribute of the *commodity_reference*). Then for a price in tenths of a cent per kilowatt-hour the price scale will have value –3 (minus-three) since it is in thousandths of a base unit and the commodity scale will have value 0 (zero) since it is in base units (i.e. kWh).

For the case where the primary currency is kWh or similarly related to the *commodity_scale* then a TOU tariff is not possible and as such the *charge_table* element of the *unit_charge_active* and *unit_charge_passive* shall not be relevant but the *commodity_scale* and *price_scale* shall be the same value.

NOTE 5 The *commodity_scale* and *price_scale* do not imply any particular rate of collection.

Explanations related to *charge_type*

When the *charge_type* = (0) *consumption_based_collection* the *commodity_reference* identifies a *scaler_unit* attribute of a total register, measuring whatever is being supplied, for instance, electrical import energy.

When the *charge_type* = (1) *time_based_collection* or (2) *payment_event_based_collection*, then the *commodity_reference* shall be a structure of zero elements.

© Copyright 1997-2020 DLMS User Association

unit_charge_ active (continued)	Explanations to charge_table elements Each element in the array *charge_table* is a structure identifying what is being charged for and collected. Where *charge_type* = (0) *consumption_based_collection*: – index is an identifier of a derivative of the main commodity reference, for example tariff rate registers, – charge_per_unit is the charge amount per unit that is scaled by the *charge_per_unit_scaling* factors. NOTE 6 For example, if the end consumer generates energy then the “Charge” should be negative, meaning that the utility pays the end consumer for the energy he produces. NOTE 7 It is also possible to model exported energy using a separate “Account” object and other related objects. Where the *charge_type* = (1) *time_based_collection* or (2) *payment_event_based_collection*, then a single entry into the array *charge_table* is made, containing: – index is an octet-string of length 0; charge_per_unit is a value equivalent to the amount charged per *period* or per payment event that is scaled by the *charge_per_unit_scaling* factors.
unit_charge_ passive	Holds the details of prices to be applied at the occurrence of the activation date. Its data structure is the same as the *unit_charge_active*. On activation, the whole structure of *unit_charge_passive* is copied into *unit_charge_active*.
unit_charge_ activation_ time	Defines the time when the object itself invokes the specific method *activate_unit_charge_passive*. A definition with "not specified" notation in all fields of the attribute will deactivate this automatism. Partial "not specified" notation in just some fields of date and time is not allowed. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
period	Where *charge_type* = (0) *consumption_based_collection* or (1) *time_based_collection* it defines the period over which the Charge will be collected. If period is set to zero there is no automatic collection of this “Charge”: collection is done by invoking the *collect* method. NOTE 8 Implementations may vary: it is possible to collect the whole charge in a single collection, or to divide the collection into parts. Where *charge_type* = (2) *payment_event_based_collection* this attribute is not used. double-long-unsigned, expressed in seconds.
charge_ configuration	This attribute is a bit-string defined as follows: charge_configuration ::= bit-string Bit 0 = Percentage based collection, Bit 1 = Continuous collection If bit 0 is set and *charge_type* = (2) *payment_event_based_collection* then this “Charge” object will collect a proportion of each top-up in accordance with the proportion attribute. This applies only to token top ups and not to method invocations.

© Copyright 1997-2020 DLMS User Association

charge_ configuration (continued)	NOTE 9 The proportion attribute of the “Charge” object and the provision attribute of the “Account” object provide more information on this collection. If bit 1 is set then collection will not terminate when the *total_amount_remaining* is equal to zero. If bit 1 is cleared then charge collection will terminate when the *total_amount_remaining* attribute is equal to zero.
last_ collection_ time	Holds the *date-time* when the last collection took place. This includes incremental collection made against consumption. octet-string, formatted as specified in 4.1.6.1 for *date-time.*
last_ collection_ amount	Holds the last collection amount relating to the *last_collection_time* attribute. double-long, scaled according to the *price_scale* element of *unit_charge_active*.
total_amount_ remaining	Where *charge_configuration* bit 1 (Continuous collection) is cleared, this attribute holds the total amount remaining for this “Charge” object. NOTE 10 The value of this attribute is initially configured by invoking the *set_total_amount_remaining* method. NOTE 11 This attribute is used to limit the collections for repayment of a debt. It is not a direct measure of the consumer’s liability. The value is not allowed to go negative (it is set to zero instead) and is not incremented by collection of a “Charge” with a negative price. Where *charge_configuration* bit 1 (Continuous collection) is set, this attribute is zero. NOTE 12 See also the Additional Notes at the end of the “Charge” IC specification. double-long, scaled according to the *price_scale* element of *unit_charge_active*.
proportion	Where the *charge_configuration* bit 0 (Percentage based collection) is set and *charge_type* = (2) *payment_event_based_collection*, then this attribute is the proportion of each top-up amount processed within the “Token gateway” object linked via the “Account” object to this “Charge” object. The range 0x0000 to 0x2710 (0 to 10,000) represents 0 to 100%. NOTE 13 The amount of collection that occurs may be limited by the *max_provision* and *max_provision_period* attributes of the “Account” object. If a fixed amount is required to be collected at every top-up (*charge_type* = 2), then the amount to be taken should be set in the first element of the *charge_table* array in the *unit_charge_active* / *unit_charge_passive* attribute. Where either *charge_type* <> (2) *payment_event_based_collection* or *charge_configuration* bit 0 (Percentage based collection) is cleared then this attribute has no effect. In the case when *charge_type* = (2) *payment_event_based_collection* and the collection of a fixed amount is required see *unit_charge_active* and *unit_charge_passive.*

© Copyright 1997-2020 DLMS User Association

Method description	

update_ unit_charge (data)	Allows updating a subset of unit charge values inside the *unit_charge_passive* attribute. The *charge_per_unit_scaling* and commodity_reference values are not affected by this method.
	It remains necessary to activate the *unit_charge_passive* so that the values can take effect.
	
	data ::= array charge_table_element
	charge_table_element ::= structure
	{
	index: octet-string,
	charge_per_unit: long
	}
	See the *charge_table_element* of the *unit_charge_active* attribute for a description of the elements.
	NOTE 14 In the case where the index exists then the value is overwritten, and if the index does not exist then the new value is appended to the array.

activate_ passive_unit_ charge (data)	Copies the whole structure of the *unit_charge_passive* attribute into the *unit_charge_active* attribute. NOTE 15 This method has no effect on the collection activity or the priority of the “Charge” object. data ::= integer (0)
collect (data)	Where *charge_type* <> (0) *consumption_based_collection*, this method makes collection of the amount defined in *unit_charge_active* when *charge_configuration* bit 0 (percentage based collection) is cleared. Where *charge_type* = (0) *consumption_based_collection*, this method has no effect. data ::= integer (0)
update_total_ amount_ remaining (data)	Allows the update of the *total_amount_remaining* attribute. The value is to be scaled according to the *price_scale* of *unit_charge_active*. The value is added to *total_amount_remaining.* The amount previously in *total_amount_remaining* attribute shall be given as the response parameter. NOTE 16 This does not affect *total_amount_paid*. data ::= double-long, scaled according to the *price_scale* of *unit_charge_active*. and returns double-long, scaled according to the *price_scale* of *unit_charge_active*.
set_total_ amount_ remaining (data)	Sets the *total_amount_remaining* attribute. The value shall be not less than 0. The amount previously in *total_amount_remaining* attribute shall be given as the response parameter. NOTE 17 *total_amount_remaining* is set without affecting *total_amount_paid*,. data ::= double-long, scaled according to the *price_scale* of *unit_charge_active*, and returns double-long, scaled according to the *price_scale* of *unit_charge_active*.

© Copyright 1997-2020 DLMS User Association

4.6.5 Token gateway (class_id = 115, version = 0)

An instance of the “Token gateway” IC implements the Token Carrier Interface.

NOTE 1 A single instance of the “Token gateway” object is instantiated for each “Account” object and hence each supply contract.

Token gateway		**0...n**	**class_id = 115, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. token	(dyn.)	octet-string				x + 0x08
3. token_time	(dyn.)	octet-string				x + 0x10
4. token_description	(dyn.)	array				x + 0x18
5. token_delivery_method	(dyn.)	enum				x + 0x20
6. token_status	(dyn.)	structure				x + 0x28
Specific methods		***m/o***				
1. enter (data)		**m**				x + 0x30

Attribute description	
logical_name	Identifies the “Token gateway” object instance. See 6.2.17.
token	Contains the unprocessed octet string of the most recently received or token for the purpose of capture in a history profile.
token_time	Contains, for the most recently received and processed token, the time and date at which a received token arrived at the DLMS/COSEM server. octet-string, formatted as specified in 4.1.6.1 for *date-time*.
token_description	Contains a description of the token for the most recently received and processed token which is supplied by the meter’s application program. NOTE 2 For example this could contain the type of token (credit or engineering), and/or the token origin. The use of this attribute is to be described in the token specification or project specific companion specifications. token_description :: = array token_description_element token_description_element ::= octet-string
token_delivery_ method	Reflects the route by which the last token was received. enum: (0) via remote communications, (1) via local communications, (2) via manual entry

© Copyright 1997-2020 DLMS User Association

token_status	*token_status* is a structure containing a generic enumeration that shows the status of the last token operation and an optional bit-string that can be associated with the token status giving extra information about the token *status_code*.
	NOTE 3 It is expected that the optional bit string content will be documented in a project specific companion specification.
	NOTE 4 The exact meaning and criteria for evaluation of the *status_code* values will be slightly different depending on the token protocol and format. For example a token conforming to IEC 62055-41 has very precise criteria and meanings for each of the terms Validation, Authentication and Token result, while other specifications may have differing terms.
	
	token_status ::= structure
	{
	status_code: enum,
	data_value: bit-string
	}
	
	status_code: enum:
	(0) Token format result OK,
	(1) Authentication result OK,
	(2) Validation result OK,
	(3) Token execution result OK,
	(4) Token format failure,
	(5) Authentication failure,
	(6) Validation result failure,
	(7) Token execution result failure,
	(8) Token received and not yet processed
	On receipt of a token the initial state shall be set to 8 while the token is being processed and until another state can be deduced.
	The use of the data_value bit-string field is to be defined in project specific companion specifications.

Method description	
enter (data)	This method is invoked to transfer a token to the DLMS/COSEM server in the form of octet-string. If successful it may return the *token_status* attribute. data ::= octet-string, and returns *token_status*

Additional remarks

There are a number of configurable limits that may be held by “Data” objects and relate to the behaviour and processing of tokens and some aspects of the payment metering system. Some of these limits have been defined in this specification, have standard OBIS codes and are described below:

“Max credit limit”: On receipt of a token the meter’s application process shall check that the addition of the credit token’s value to the associated credit object(s) will not force the *available_credit* in the “Account” object above the limit programmed in the “Max credit limit” object. If the received token is found to force the *available_credit* to be more than this limit then the meter shall reject the token and return the appropriate status.

© Copyright 1997-2020 DLMS User Association

"Max vend limit": On receipt of a token the meter's application process shall check that the value of the credit token is not more than the limit programed in the "Max vend limit" object. If the received token is found to be more than this limit then the meter must reject the token and return the appropriate status.

See also 6.2.17.

4.7 Interface classes for setting up data exchange via local ports and modems

4.7.1 IEC local port setup (class_id = 19, version = 1)

This IC allows modelling the configuration of communication ports using the protocols specified in IEC 62056-21:2002. Several ports can be configured.

IEC local port setup		**0...n**	**class_id = 19, version = 1**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. default_mode	(static)	enum				x + 0x08
3. default_baud	(static)	enum				x + 0x10
4. prop_baud	(static)	enum				x + 0x18
5. response_time	(static)	enum				x + 0x20
6. device_addr	(static)	octet-string				x + 0x28
7. pass_p1	(static)	octet-string				x + 0x30
8. pass_p2	(static)	octet-string				x + 0x38
9. pass_w5	(static)	octet-string				x + 0x40
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the "IEC local port setup" object instance. 6.2.18.
default_mode	Defines the protocol used by the meter on the port. enum: (0) protocol according to IEC 62056-21:2002 (modes A...E), (1) protocol according to DLMS UA 1000-2 Ed. 10 Clause 8. Using this enumeration value all other attributes of this IC are not applicable, (2) protocol not specified. Using this enumeration value, attribute 4), *prop_baud* is used for setting the communication speed on the port. All other attributes are not applicable.
default_baud	Defines the baud rate for the opening sequence. enum: (0) 300 baud, (1) 600 baud, (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud, (8) 57 600 baud, (9) 115 200 baud

© Copyright 1997-2020 DLMS User Association

prop_baud	Defines the baud rate to be proposed by the meter. enum: (0) 300 baud, (1) 600 baud, (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud, (8) 57 600 baud, (9) 115 200 baud
response_time	Defines the minimum time between the reception of a request (end of request telegram) and the transmission of the response (begin of response telegram). enum: (0) 20 ms, (1) 200 ms
device_addr	Device address according to IEC 62056-21.
pass_p1	Password 1 according to IEC 62056-21.
pass_p2	Password 2 according to IEC 62056-21.
pass_w5	Password W5 reserved for national applications.

4.7.2 IEC HDLC setup (class_id = 23, version = 1)

This IC allows modelling and configuring communication channels according to DLMS UA 1000-2 Ed. 10 Clause 8. Several communication channels can be configured.

IEC HDLC setup		**0...n**	**class_id = 23, version = 1**			
Attributes		***Data type***	**Min.**	**Max.**	**Def.**	**Short name**
1. logical_name	(static)	octet-string				x
2. comm_speed	(static)	enum	0	9	5	x + 0x08
3. window_size_transmit	(static)	unsigned	1	7	1	x + 0x10
4. window_size_receive	(static)	unsigned	1	7	1	x + 0x18
5. max_info_field_length_transmit	(static)	long-unsigned	32	2030	128	x + 0x20
6. max_info_field_length_receive	(static)	long-unsigned	32	2030	128	x + 0x28
7. inter_octet_time_out	(static)	long-unsigned	20	6000	25	x + 0x30
8. inactivity_time_out	(static)	long-unsigned	0		120	x + 0x38
9. device_address	(static)	long-unsigned	0x0010	0x3FFD		x + 0x40
Specific methods		***m/o***				

NOTE 1 The maximum value of the attributes *max_info_field_length_transmit* and *max_info_field_length_receive* has been increased from 128 to 2 030 for efficiency reasons.

NOTE 2 A *max_info_field_length_receive* of 128 bytes is needed to ensure a minimal performance.

NOTE 3 The maximum value of the *inter-octet-time-out* attribute has been increased from 1 000 ms to 6 000 ms in order to allow using communication media, where long delays may occur. The default value has been changed to 25 ms to align with 8.4.4.3.4 of DLMS UA 1000-2 Ed. 10, 10.7.

Attribute description	
logical_name	Identifies the “IEC HDLC setup” object instance. See 6.2.20.

© Copyright 1997-2020 DLMS User Association

comm_speed	The communication speed supported by the corresponding port. enum: (0) 300 baud, (1) 600 baud, (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud, (8) 57 600 baud, (9) 115 200 baud This communication speed can be overridden if the HDLC mode of a device is entered through a special mode of another protocol.
window_size_ transmit	The maximum number of frames that a device or system can transmit before it needs to receive an acknowledgement from a corresponding station. During logon, other values can be negotiated.
window_size_ receive	The maximum number of frames that a device or system can receive before it needs to transmit an acknowledgement to the corresponding station. During logon, other values can be negotiated.
max_info_length_ transmit	The maximum information field length that a device can transmit. During logon, a smaller value can be negotiated.
max_info_length_ receive	The maximum information field length that a device can receive. During logon, a smaller value can be negotiated.
inter_octet_ time_out	Defines the time, expressed in milliseconds, over which, when no character is received from the primary station, the device will treat the already received data as a complete frame.
inactivity_ time_out	Defines the time, expressed in seconds over which, when no frame is received from the primary station, the device will process a disconnection. When this value is set to 0, this means that the inactivity_time_out is not operational.
device_address	Contains the physical device address of a device. In the case of one byte addressing: 0x00 NO_STATION Address, 0x01…0x0F Reserved for future use, 0x10…0x7D Usable address space, 0x7E ‘CALLING’ device address, 0x7F Broadcast address In the case of two byte addressing: 0x0000 NO_STATION address, 0x0001…0x000F Reserved for future use, 0x0010…0x3FFD Usable address space, 0x3FFE ‘CALLING’ physical device address, 0x3FFF Broadcast address

© Copyright 1997-2020 DLMS User Association

4.7.3 IEC twisted pair (1) setup (class_id = 24, version = 1)

4.7.3.1 General

The communication medium *twisted pair with carrier signalling* is widely used in metering. The main advantages of using this medium are the ease of installation and the reliability of communications due to carrier signalling. This medium can be used:

- between Local Network Access Points (LNAPs) and metering end devices (M interface);
- between Local Network Access Points (LNAPs) and Neighbourhood Network Access Points (NNAPs); and
- for direct connection between a HHU and the metering end device.

IEC 62056-3-1:2013 specifies three communication profiles using the medium twisted pair with carrier signalling:

- without DLMS;
- with DLMS; and
- with DLMS/COSEM.

IEC 62056-31:1999 supports only the first two profiles.

The new, DLMS/COSEM profile introduces a Support Manager Layer entity performing the initialisation of the bus, discovery management, alarm management and communication speed negotiation. It also allows higher baud rates up to 9 600 Bd. The Transport Layer supports segmentation and reassembly.

The IC “IEC Twisted pair (1) set up” (class_id = 24, version = 0) supports the first two communication profiles specified in IEC 62056-31:1999.

The new version 1 supports the DLMS/COSEM profile. With its introduction, the use of version 0 is deprecated.

The use of the communication profiles specified in IEC 62056-3-1:2013 requires using the registration services provided by the Euridis Association: www.euridis.org.

The following COSEM interface objects are necessary to set up data exchange over the medium *Twisted pair with carrier signalling:*

- “IEC Twisted pair (1) setup”: class_id = 24, version = 1;
- “MAC address”: class_id = 43, version = 0;
- “Data”: class_id = 1, version = 0.

For OBIS codes, see 6.2.21.

4.7.3.2 IEC twisted pair (1) setup (class_id = 24, version = 1)

Instances of this IC allow setting up data exchange over the medium *twisted pair with carrier signalling* as specified in IEC 62056-3-1:2013. Several communication channels can be configured.

© Copyright 1997-2020 DLMS User Association

IEC twisted pair (1) setup		0...n	class_id = 24, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mode	(static)	enum	0		1	x + 0x08
3. comm_speed	(static)	enum	(2)	(7)	(2)	x + 0x10
4. primary_address_list	(static)	primary_address_list_type				x + 0x18
5. tabi_list	(static)	tabi_list_type				x + 0x20
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “IEC twisted pair setup” object instance. See 6.2.21.
mode	This attribute specifies the working mode of this interface enum: (0) inactive. The interface ignores all frames received, (1) always active, (2) to (127) reserved, (128) to (250) manufacturer specific.
comm_speed	Holds the communication speed supported by the port. enum: (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud NOTE IEC 62056-3-1:2013 supports baud rates from 1 200 to 9 600.
primary_address_list	Holds the list of Primary Station Addresses (ADP) for which each logical device of the real equipment (the secondary station) has been programmed. See IEC 62056-3-1:2013, 5.2.4. primary_address_list_type ::= array unsigned
tabi_list	Represents the list of the TAB(i) for which the real equipment (the secondary station) has been programmed in the case of forgotten station call (see IEC 62056-3-1:2013). When using IEC 62056-3-1 profile with DLMS/COSEM, the *tabi_list* attribute is made of only one element of value 0, the value used for the discovery process. tabi_list_type ::= array tabi_element tabi_element: integer

4.7.3.3 Fatal error register

Each device implementing the DLMS/COSEM communication profile specified in IEC 62056-3-1:2013 shall provide an error register holding the result of the last communication with the primary station. The structure of the fatal error register shall be as specified inTable 30.

© Copyright 1997-2020 DLMS User Association

Table 30 – Fatal error register

Ref	Name	Description
Bit 0	EP-3F	Transmission error. The time out TOE is elapsed without the byte being sent, leading to a non-ability to send the remaining part of the frame.
Bit 1	EP-4F	Reception error. The number of bytes received is higher than the maximum expected.
Bit 2	EP-5F	Expiry of TARSO wake-up while receiving an RSO frame. Not relevant for secondary station (server); concerns the primary station (client) only.
Bit 3	EL-1F	Alarm indication received during an association. No relevant. Concern the primary station (client) only.
Bit 4	EL-2F	Incorrect response from the Secondary Station after MaxRetry repeated transmissions of a request.
Bit 5	EA-1F	Incorrect TAB from the server. Not relevant for secondary station (server); concerns the primary station (client) only.
Bit 6	EA-2F	Authentication error on the data received from the server. Not relevant for secondary station (server); concerns the primary station (client) only.
Bit 7	EA-3F	Authentication error detected by the secondary station.

© Copyright 1997-2020 DLMS User Association

4.7.4 Modem configuration (class_id = 27, version = 1)

This IC allows modelling the configuration and initialisation of modems used for data transfer from/to a device. Several modems can be configured.

Modem configuration		**0...n**	**class_id = 27, version = 1**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. comm_speed	(static)	enum	0	9	5	x + 0x08
3. initialization_string	(static)	array				x + 0x10
4. modem_profile	(static)	array				x + 0x18
Specific methods		***m/o***				

Attribute description

logical_name

Identifies the “Modem configuration” object instance. See 6.2.6.

comm_speed

The communication speed between the device and the modem, not necessarily the communication speed on the WAN.

enum:
(0) 300 baud,
(1) 600 baud,
(2) 1 200 baud,
(3) 2 400 baud,
(4) 4 800 baud,
(5) 9 600 baud,
(6) 19 200 baud,
(7) 38 400 baud,
(8) 57 600 baud,
(9) 115 200 baud

initialization_string

Contains all the necessary initialization commands to be sent to the modem in order to configure it properly. This may include the configuration of special modem features.

array initialization_string_element

initialization_string_element ::= structure
{
request: octet-string,
response: octet-string,
delay_after_response: long-unsigned
}

If the array contains more than one initialization_string_element, the requests are sent in a sequence. The next request is sent after the expected response matching the previous request and waiting a delay_after_response time [ms], to allow the modem to execute the request.

NOTE It is assumed that the modem is pre-configured so that it accepts the initialization_string. If no initialization is needed, the initialization string is empty.

modem_profile

Defines the mapping from Hayes standard commands/responses to modem specific strings.

array modem_profile_element

modem_profile_element: octet-string

© Copyright 1997-2020 DLMS User Association

modem_profile (continued)

The modem_profile array shall contain the corresponding strings for the modem used in following order:

Element 0:	OK,
Element 1:	CONNECT,
Element 2:	RING,
Element 3.	NO CARRIER,
Element 4:	ERROR,
Element 5:	CONNECT 1 200,
Element 6	NO DIAL TONE,
Element 7:	BUSY,
Element 8:	NO ANSWER,
Element 9:	CONNECT 600,
Element 10:	CONNECT 2 400,
Element 11:	CONNECT 4 800,
Element 12	CONNECT 9 600,
Element 13:	CONNECT 14 400,
Element 14:	CONNECT 28 800,
Element 15:	CONNECT 33 600,
Element 16:	CONNECT 56 000

4.7.5 Auto answer (class_id = 28, version = 2)

Version 0 of the Auto answer class models how the device handles incoming calls to request the connection of the modem.

NOTE 1 Version 1 of the Auto answer class was an interim version.

In version 2, new capabilities are added to manage wake-up requests that may be in the form of a wake-up call or a wake-up message e.g. an (empty) SMS message. After a successful wake-up request, the device connects to the network. See also Annex A.

For both functions, additional security is provided by adding the possibility of checking the calling number: calls or messages are accepted only from a pre-defined list of callers. This feature requires the presence of a calling line identification (CLI) service in the communication network used.

NOTE 2 The wake-up process is fully decoupled from AL services, i.e. a wake-up message cannot contain any xDLMS service requests. This is to avoid creating a backdoor. xDLMS messages may be exchanged in SMS messages once the wake-up process is completed.

Auto answer		**0...n**	**class_id = 28, version = 2**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mode	(static)	enum				x + 0x08
3. listening_window	(static)	array				x + 0x10
4. status	(dyn.)	enum				x + 0x18
5. number_of_calls	(static)	unsigned				x + 0x20
6. number_of_rings	(static)	nr_rings_type				x + 0x28
7. list_of_allowed_callers	(static)	array				x + 0x30
Specific methods		***m/o***				

Attribute description

logical_name Identifies the “Auto answer” object instance. See 6.2.6.

© Copyright 1997-2020 DLMS User Association

mode	Defines the working mode of the line when the device is auto answering. enum: (0) line dedicated to the device, (1) shared line management with a limited number of calls allowed. Once the number of calls is reached, the window status becomes inactive until the next start date, whatever the result of the call, (2) shared line management with a limited number of successful calls allowed. Once the number of successful communications is reached, the window status becomes inactive until the next start date, (3) currently no modem connected, (200...255) manufacturer specific modes.

listening_ window	Defines the time points when the communication window(s) become active (start_time) and inactive (end_time). The start_time implicitly defines the period.
	EXAMPLE When the day of month is not specified (equal to 0xFF) this means that we have a daily listening window management. Daily, monthly ...window management can be defined.
	array window_element
	
	window_element ::= structure
	{
	start_time: octet-string,
	end_time: octet-string
	}
	start_time and end_time are formatted as specified in 4.1.6.1 for *date-time.*

status	Here the status of the window is defined. enum: (0) Inactive: the device will manage no new incoming call. This status is automatically reset to Active when the next listening window starts, (1) Active: the device can answer to the next incoming call, (2) Locked: This value can be set automatically by the device or by a specific client when this client has completed its reading session and wants to give the line back to the customer before the end of the window duration. This status is automatically reset to Active when the next listening window starts.
number_of_ calls	This number is the reference used in modes 1 and 2. When set to 0, this means there is no limit.

number_of_ rings	Defines the number of rings before the meter connects the modem. Two cases are distinguished: The number of rings within the window defined by the attribute *listening_window* and the number of rings outside the *listening_window*.
	nr_rings_type ::= structure
	{
	nr_rings_in_window: unsigned (0 = no connection in the window),
	nr_rings_out_of_window: unsigned (0 = no connection outside the window)
	}
	If the number of rings inside and outside the window is the same, the modem always connects regardless of the settings of the *listening_window*.

© Copyright 1997-2020 DLMS User Association

list_of_ allowed_ callers

Contains an – optional – list of calling numbers which further limits the connectivity of the modem based on the calling number. It also controls the acceptance of wake-up calls or wake-up messages (e.g. SMS) from a calling number.

This requires the presence of a calling line identification (CLI) service in the communication network used.

list_of_allowed_callers ::= array list_of_allowed_callers_element

list_of_allowed_callers_element ::= structure
{
    caller_id: octet-string,
    call_type: enum
}

- the caller_id element holds a calling number from which calls or messages (e.g. SMS) are accepted. The wild-card characters '?' and '*' are supported. With '?' any single character matches, with '*' any character string matches. '*' can only be used at the beginning or at the end of a number, but neither in between nor alone.

  Example 1: "+994193500" = only calls from "+994193500" are accepted.

  Example 2: "+9941935????" = calls from all numbers in the range of "+99419350000" to "+99419359999" are accepted.

  Example 3: "7777*" = calls from all numbers starting with "7777" are accepted.

  Example 4: "*9000" = calls from all numbers ending with "9000" are accepted.

- the call_type element defines the purpose of the call, i.e. if it's a standard CSD call or a wake-up call / wake-up message.

  enum:

  (0) = normal CSD call; the modem only connects if the calling number matches an entry in the list. This is tested in addition to all other attributes, e.g. *number_of_rings*, *listening_windows*, etc.

  (1) = wake-up request; calls or messages from this calling number are handled as wake-up requests. The wake-up request is processed immediately regardless of all other attributes like *number_of_rings* and *listening_window.*

  The received message shall be completely empty; otherwise it is not treated as a wake-up message.

  If the message contains a valid xDLMS APDU from a client in a pre-established AA, the corresponding xDLMS service is executed instead of processing the wake-up request.

  If the message is not empty but does not contain any valid xDLMS APDU from a client in a pre-established AA then the server does not react.

  For a call of type (1) the modem *does not* connect - independently of the outcome of the wake-up process.

© Copyright 1997-2020 DLMS User Association

list_of_ allowed_ callers (continued)	If the same calling number is defined as initiator of a normal CSD call (type (0)) and as initiator of a wake-up request (type (1)) the following rule applies for the incoming calls: the wake-up request is only processed if the caller disconnects the line before the number_of_rings criterion (depending on listening _window) is reached. If the number_of_rings criterion is met then a modem connection is established. The number_of_rings parameter shall be set large enough to allow the initiator of the call to control the behaviour of the receiver of the call without any knowledge of the time instances of the rings at the receiver's side. NOTE 3 If the *list_of_allowed_callers* is empty (= array [0]) the auto answer function operates in type (0) = normal CSD call and the modem connects independently of the calling number.

4.7.6 Auto connect (class_id = 29, version = 2)

Version 1 of the "Auto connect" class models how the device performs auto dialling or sends messages using various services.

In version 2 new capabilities are added to model the connection of the device to a communication network. Network connection may be permanent, within a time window or on invocation of the connect method.

Auto connect		**0...n**	**class_id = 29, version = 2**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mode	(static)	enum				x + 0x08
3. repetitions	(static)	unsigned				x + 0x10
4. repetition_delay	(static)	long-unsigned				x + 0x18
5. calling_window	(static)	array				x + 0x20
6. destination_list	(static)	array				x + 0x28
Specific methods		***m/o***				
1. connect (data)		o				x + 0x30

Attribute description	
logical_name	Identifies the "Auto connect" object instance. See 6.2.6.
mode	Controls the auto connect functionality in terms of the timing, the message type and the infrastructure to be used. Modes (1) to (3) are dedicated to CSD services. Modes (4) to (6) are dedicated to sending specific messages using a specific infrastructure. Modes (101) to (104) apply to packet switched network connections only (e.g. GPRS).

© Copyright 1997-2020 DLMS User Association

mode (continued)	enum: (0) no auto connect; the device never connects, (1) auto dialling allowed anytime, the values defined in the *calling_window* are ignored, (2) auto dialling allowed within the validity time of the *calling_window*, (3) “regular” auto dialling allowed within the validity time of the *calling_window*; “alarm” initiated auto dialling allowed anytime, (4) SMS sending via Public Land Mobile Network (PLMN), (5) SMS sending via PSTN, (6) email sending, (7...99) reserved, (101) the device is permanently connected to the communication network, (102) the device is permanently connected to the communication network within the validity time of the calling window. The device is disconnected outside the calling window. No connection possible outside the calling window, (103) the device is permanently connected to the communication network within the validity time of the calling window. The device is disconnected outside the calling window but it connects to the communication network as soon as the connect method is invoked, (104) the device is usually disconnected. It connects to the communication network as soon as the connect method is invoked, 105...199) reserved, (200...255) manufacturer specific modes.
repetitions	The maximum number of retries in case of unsuccessful connection attempts.
repetition_ delay	The time delay, expressed in seconds until an unsuccessful connection attempt can be repeated. repetition_delay = 0 means delay is not specified.

calling_ window	Contains the time points when the window becomes active (start_time), and inactive (end_time). The start_time implicitly defines the period.
	EXAMPLE When the day of month is not specified (equal to 0xFF) this means that the calling window is managed on a daily basis. Daily, monthly ...window management can be defined.
	array window_element
	
	window_element ::= structure
	{
	start_time: octet-string,
	end_time: octet-string
	}
	start_time and end_time are formatted as specified in 4.1.6.1 for *date-time*.

© Copyright 1997-2020 DLMS User Association

destination_ list	Contains the list of destinations (for example phone numbers, email addresses or their combinations) where the message(s) have to be sent under certain conditions. The conditions and their link to the elements of the array are not defined here. array destination destination ::= octet-string
Method description	
connect (data)	Initiates the connection process to the communication network according to the rules defined via the mode attribute. data ::= integer (0)

4.7.7 GPRS modem setup (class_id = 45, version = 0)

This IC allows setting up GPRS modems, by handling all data necessary data for modem management.

GPRS modem setup		**0...n**	**class_id = 45, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. APN	(static)	octet-string				x + 0x08
3. PIN_code	(static)	long-unsigned				x + 0x10
4. quality_of_service	(static)	structure				x + 0x18
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “GPRS modem setup” object instance. See 6.2.23.
APN	Defines the access point name of the network.
PIN_code	Holds the personal identification number.

quality_of_ service	Specifies the quality of service parameters. It is a structure of 2 elements:
	- the first element defines the default or minimum characteristics of the network concerned. These parameters have to be set to best effort value;
	- the second element defines the requested parameters.
	
	quality_of_service ::= structure
	{
	default: qos_element,
	requested: qos_element
	}

© Copyright 1997-2020 DLMS User Association

quality_of_ service (continued)	qos_element ::= structure
	{
	precedence: unsigned,
	delay: unsigned,
	reliability: unsigned,
	peak throughput: unsigned,
	mean throughput: unsigned
	}

4.7.8 GSM diagnostic (class_id = 47, version = 2)

The cellular network is undergoing constant changes in terms of registration status, signal quality etc. It is necessary to monitor and log the relevant parameters in order to obtain diagnostic information that allows identifying communication problems in the network.

An instance of the “GSM diagnostic” IC stores parameters of the GSM/GPRS, UMTS, CDMA, LTE, LTE-M or NB-IoT network.

A GSM diagnostic “Profile generic” object is also available to capture the attributes of the GSM diagnostic object, see 7.4.5.

GSM diagnostic		**0...n**	**class_id = 47, version = 2**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. operator	(dyn.)	visible-string				x + 0x08
3. status	(dyn.)	enum	0	255	0	x + 0x10
4. cs_attachment	(dyn.)	enum	0	255	0	x + 0x18
5. ps_status	(dyn)	enum	0	255	0	x + 0x20
6. cell_info	(dyn.)	cell_info_type				x + 0x30
7. adjacent_cells	(dyn.)	array				x + 0x38
8. capture_time	(dyn.)	date-time				x + 0x40
Specific methods		***m/o***				

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “GSM Diagnostic” object instance. For logical names, see 6.2.23.
operator	Holds the name of the network operator e.g. “YourNetOp”
status	Indicates the registration status of the modem. enum: (0) not registered, (1) registered, home network, (2) not registered, but MT is currently searching a new operator to register to, (3) registration denied; (4) unknown, (5) registered, roaming, (6)..(255) reserved.
cs_attachment	Indicates the current circuit switched status. enum: (0) inactive, (1) incoming call, (2) active, (3)..(255) reserved.
ps_status	The *ps_status* value field indicates the packet switched status of the modem. enum: (0) inactive, (1) GPRS, (2) EDGE, (3) UMTS, (4) HSPDA, (5) LTE, (6) CDMA, (7) LTE Cat M1, (8) LTE Cat NB1, (9) LTE Cat NB2, (10) ..(255) reserved.

cell_info	Represents the cell information:
	cell_info_type ::= structure
	{
	cell_ID: double-long-unsigned,
	location_ID: long-unsigned,
	signal_strength: unsigned,
	ber: unsigned,
	mcc: long-unsigned,
	mnc: long-unsigned,
	channel_number: double-long-unsigned
	}
	cell_ID: Four-byte cell ID in hexadecimal format
	location_ID: Two-byte location area code (LAC) in the case of GSM networks or Tracking Area Code (TAC) in the case of UMTS, CDMA or LTE networks in hexadecimal format (e.g. "0x00C3" equals 195 in decimal)

© Copyright 1997-2020 DLMS User Association

	signal_strength	The Received Signal Strength Indicator value (0) –113 dBm or less, (1) –111 dBm, (2)..(30) –109…-53 dBm, (31) –51 dBm or greater, (99) not known or not detectable, other reserved,
	ber	Bit Error Rate (BER) measurement in percent: (0)…(7) as RXQUAL_n values specified in ETSI GSM 05.08, 8.2.4 and RSRQ n values specified in 3GPP TS 27.007 V16.1.0 (2019-06), 8.2.4 for LTE (99) not known or not detectable.
	mcc	Mobile Country Code of the serving network, as defined in ITU-T E.212 (05.2008).
	mnc	Mobile Network Code of the serving network, as defined in ITU-T E.212 (05.2008).
	channel_number	The absolute radio-frequency channel number (ARFCN or EARFCN for LTE network)

adjacent_cells	array adjacent_cell_info
	
	adjacent_cell_info ::= structure
	{
	cell_ID: double-long-unsigned,
	signal_strength: unsigned
	}	

	cell_ID	Four-byte cell ID in hexadecimal format
	signal_strength	The Received Signal Strength Indicator value (0) –113 dBm or less, (1) –111 dBm, (2)..(30) –109…-53 dBm, (31) –51 dBm or greater, (99) not known or not detectable, other reserved
capture_time	Holds the date and time when the data have been last captured. *date-time* is formatted as specified in 4.1.6.1.	

© Copyright 1997-2020 DLMS User Association

4.7.9 LTE monitoring (class_id = 151, version = 1)

Instances of the 'LTE monitoring" IC allow monitoring LTE modems by handling all data necessary data for this purpose.

Version 1 of the LTE monitoring IC also covers LTE Cat M1 and NB-IoT networks.

LTE monitoring		0...n	class_id = 151, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
5. logical_name	(static)	octet-string				x
6. LTE_network_parameters	(dyn.)	LTE_network_ parameters_type				x + 0x08
7. LTE_quality_of_service	(dyn)	LTE_QoS_type				x + 0x10
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the "LTE monitoring" object instance. See 6.2.23

LTE_network_ parameters	Represents the network parameters for the LTE network
	LTE_network_parameters_type ::= structure
	{
	T3402: long-unsigned,
	T3412: long-unsigned,
	T3412ext2: double-long-unsigned,
	T3324 long-unsigned,
	TeDRX double-long unsigned,
	TPTW long-unsigned,
	qRxlevMin: integer,
	qRxlevMinCE-r13: integer,
	qRxlevMinCE1-r13 integer
	}

T3402	timer in seconds, used on PLMN selection procedure and sent by the network to the modem. Refer to 3GPP TS 24.301 V13.11.0 for details.
T3412	timer in seconds, used to manage the periodic tracking area updating procedure and sent by the network to the modem. Refer to 3GPP TS 24.301 V13.11.0 for details.
T3412ext2	timer in seconds (extended periodic tracking area update timer). Refer to 3GPP TS 24.301 V13.11.0 and 3GPP TS 24.008 V13.7.0 (2016-10) for details.
T3324	timer in seconds (Power saving mode active timer). Refer to 3GPP TS 24.301 V13.11.0 and 3GPP TS 24.008 V13.7.0 (2016-10) for details.
TeDRX	timer (Extended Idle mode DRX cycle timer) Refer to Refer to 3GPP TS 24.301 V13.11.0 and 3GPP TS 24.008 V13.7.0 (2016-10) for details. The double-long unsigned value shall be multiplied by 0,01 to get the real value in seconds. E.g. 512 represents 5,12 seconds

© Copyright 1997-2020 DLMS User Association

	TPTW	timer (Extended Idle mode DRX paging time window) Refer to 3GPP TS 24.301 V13.11.0 and 3GPP TS 24.008 V13.7.0 (2016-10) for details. The long unsigned value shall be multiplied by 0,01 to get the real value in seconds. E.g. 512 represents 5,12 seconds
	qRxlevMin	the minimum required Rx level in the cell in dBm as defined in 3GPP TS 36.304 V13.8.0 (2018-01).
	qRxlevMinCE-r13	the minimum required Rx level in enhanced coverage CE Mode A, LTE Cat M1. For this mode a value from -70...-22 in steps of 1 is required. NOTE 1 This field is not used in case of LTE Cat NB1 or NB2.
	qRxlevMinCE1-r13	the minimum required Rx level in enhanced coverage CE Mode B, LTE Cat M1. For this mode a value from -78...-22 in steps of 1. NOTE 2: This field is not used in case of LTE Cat NB1 orNB2.

LTE_quality_of_service	Represents the quality of service of the LTE network
	LTE_QoS_type ::= structure
	{
	(N)RSRQ: integer,
	(N)RSRP: integer,
	SNR: integer,
	Coverage Enhancement: enum
	}	

	(N)RSRQ	represents the signal quality as defined in 3GPP TS 36.133. The use of this parameter can be determined from the GSM diagnostic IC, ps_status attribute: – For LTE Cat M1, a value range from -30 up to 46 is necessary to represent RSRQ. Refer to 3GPP TS 36.133 V13.11.0 (2018-03) for details. – For LTE Cat NB1 and LTE Cat NB2 a value range from -30 up to 46 is necessary to represent NRSRQ. Refer to 3GPP TS 36.133 V14.4.0 (2017-06) for details. (-30) -34 dB, (-29) -33,5 dB, (-28)..(-1) -33 dB .. -19,5 dB, (0) -19,5 dB or less, (1) -19 dB, (2)..(31) -18,5 dB .. -4 dB, (32) -3.5 dB, (33) -3 dB, (34) -3 dB or less, (35) -2.5 dB, (36) -2 dB, (37)..(45) -1.5 dB .+2.5 dB, (46) 2.5 dB or better, (99) not known or not detectable, other values reserved

© Copyright 1997-2020 DLMS User Association

(N)RSRP	represents the signal level as defined in 3GPP TS 36.133: The use of this parameter is determined from the GSM diagnostic IC, ps_status attribute: – For ps_status = 7 (LTE Cat M1) a value range from -17 up to 97 is necessary to represent RSRP. Refer to 3GPP TS 36.133 V13.11.0 (2018-03) for details (-17) -156 dBm, (-16) -155 dBm, (-15)..(-1) -154 dBm .. -140 dBm, (0) -140 dBm or less, (1) -139 dB, (2) -138 dBm, (3)..(96) -137 dBm .. -44 dBm, (97) -44 dBm or better, (127) not known or not detectable, other values reserved NOTE 3 depending on the use of Coverage Enhancement modes, a reported value of (0) will be used or not (for devices that do not use CE modes, values (-17..-1) will not be used. A Reported value of (0) means in that case a RSRP < -140 dBm). – For ps_status = 8 or 9 (LTE Cat NB1 or LTE Cat NB2) a value range from 0 to 113 is necessary to represent NRSRP value. Refer to 3GPP TS 36.133 V14.4.0 (2017-06) for details. (0) -156 dBm, (1) -155 dBm, (2) -154 dBm (3)..(112) -153 dBm .. -44 dBm, (113) -44 dBm or better, (127) not known or not detectable, other values reserved NOTE 4 For the NRSRP value (NB-IoT) the mapping is different from the mapping for the RSRP value (LTE Cat M1).
SNR	the Signal to Noise Ratio in a range from -20 dB to 50 dB. Refer to 3GPP TS 36.101 V15.4.0 for details.

© Copyright 1997-2020 DLMS User Association

Coverage Enhancement	Coverage Enhancement Mode A or B in case of LTE Cat M1, CE Level 0,1 or 2 in case of LTE Cat NB1/NB2. Refer to 3GPP TS 36.331 V15.5.1), 3GPP TS 36.321 V15.5.0 and 3GPP TS 36.213 V15.5.0 (2019-05)) for details.

Enum	LTE Cat M1	NB-IoT
(0)	CE Mode A	CE Level 0
(1)	CE Mode B	CE Level 1
(2)	(not used)	CE Level 2
other values	reserved	

© Copyright 1997-2020 DLMS User Association

4.8 Interface classes for setting up data exchange via M-Bus

4.8.1 Overview

The M-Bus related interface classes specified in this subclause 4.8.1 are used in two different scenarios:

a) a DLMS/COSEM server hosted by a M-Bus master and exchanging dedicated M-Bus APDUs with M-Bus slaves;

b) a DLMS/COSEM client hosted by a M-Bus master and exchanging DLMS/COSEM APDUs with DLMS/COSEM servers hosted by M-Bus slaves;

In case a) instances of the following M-Bus interface classes are used to set up and manage the M-Bus media in the DLMS/COSEM server:

- M-Bus client (class_id = 72), see 4.8.3;
- M-Bus master port setup (class_id = 74), see 4.8.5;
- M-Bus diagnostic (class_id = 77, version = 0), see 4.8.7.

In case b) instances of the following M-Bus interface classes are used in the DLMS/COSEM server:

- DLMS/COSEM server M-Bus port setup (class_id = 76), see 4.8.6;
- M-Bus slave port setup (class_id = 25), see 4.8.2; and/or
- Wireless Mode Q channel (class_id = 73), see 4.8.4;
- M-Bus diagnostic (class_id = 77, version = 0), see 4.8.7.

4.8.2 M-Bus slave port setup (class_id = 25, version = 0)

NOTE 1 The name of this IC has been changed from “M-BUS port setup” to “M-Bus slave port setup”, to indicate that it serves to set up data exchange when a COSEM server communicates with a COSEM client using wired M-Bus.

This IC allows modelling and configuring communication channels according to EN 13757-2. Several communication channels can be configured.

M-Bus slave port setup		**0...n**	**class_id = 25, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. default_baud	(static)	enum	0	5	0	x + 0x08
3. avail_baud	(static)	enum	0	7		x + 0x10
4. addr_state	(static)	enum				x + 0x18
5. bus_address	(static)	unsigned				x + 0x20
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “M-Bus slave port setup” object instance. See 6.2.22.
default_baud	Defines the baud rate for the opening sequence. enum: (0) 300 baud, (3) 2 400 baud, (5) 9 600 baud

© Copyright 1997-2020 DLMS User Association

avail_baud	Defines the baud rates that can be negotiated after startup. enum: (0) 300 baud, (1) 600 baud, (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud
addr_state	Defines whether or not the device has been assigned an address since last power up of the device. enum: (0) Not assigned an address yet, (1) Assigned an address either by manual setting, or by automated method.
bus_address	The currently assigned address on the bus for the device. NOTE 2 If no bus address is assigned, the value is 0.

4.8.3 M-Bus client (class_id = 72, version = 1)

Instances of the “M-Bus client” allow setting up M-Bus slave devices using wired M-Bus and to exchange data with them. Each “M-Bus client” object controls one M-Bus slave device. For details on the M-Bus dedicated application layer, see EN 13757-3.

NOTE 1 Version 1 of the “M-Bus client” IC is in line with EN 13757-3:2013.

The M-Bus client device may have one or more physical M-Bus interfaces, which can be configured using instances of the “M-Bus master port setup” IC, see 4.8.5.

An M-Bus slave device is identified with its Primary Address, Identification Number, Manufacturer ID etc. as defined in EN 13757-3:2013, Clause 5, Variable Data Send and Variable Data respond. These parameters are carried by the respective attributes of the M-Bus client IC.

Values to be captured from an M-Bus slave device are identified by the *capture_definition* attribute, containing a list of data identifiers (DIB, VIB) for the M-Bus slave device. The data are captured periodically or on an appropriate trigger. Each data element is stored in an M-Bus value object, of “Extended Register” IC. M-Bus value objects may be captured in M-Bus “Profile generic” objects, eventually along with other, non M-Bus specific objects. If the data type used by M-Bus is not a data type specified in COSEM, then a data type conversion has to take place. The conversion process is specified in EN13757-3:2018 Annex H.

Using the methods of “M-Bus client” objects, M-Bus slave devices can be installed and de-installed.

It is also possible to send data to M-Bus slave devices and to perform operations like resetting alarms, synchronizing the clock, transferring an encryption key etc.

Configuration field as defined in EN 13757-3:2013, 5.12 provides information about the encryption mode and number of encrypted bytes.

Encryption key status provides information if encryption key has been set, transferred to M-Bus slave device and is in use with M-Bus slave device.

© Copyright 1997-2020 DLMS User Association

M-Bus client		0...n	class_id = 72, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	***Short name***
1. logical_name	(static)	octet-string				x
2. mbus_port_reference	(static)	octet-string				x + 0x08
3. capture_definition	(static)	array			empty	x + 0x10
4. capture_period	(static)	double-long-unsigned			0	x + 0x18
5. primary_address		unsigned			0	x + 0x20
6. identification_number	(dyn.)	double-long-unsigned			0	x + 0x28
7. manufacturer_id	(dyn.)	long-unsigned			0	x + 0x30
8. version	(dyn.)	unsigned			0	x + 0x38
9. device_type	(dyn.)	unsigned			0	x + 0x40
10. access_number	(dyn.)	unsigned			0	x + 0x48
11. status	(dyn.)	unsigned			0	x + 0x50
12. alarm	(dyn.)	unsigned			0	x + 0x58
13. configuration	(dyn.)	long-unsigned			0	x + 0x60
14. encryption_key_status	(dyn.)	enum			0	x + 0x68
Specific methods		***m/o***				
1. slave_install (data)		o				x + 0x70
2. slave_deinstall (data)		o				x + 0x78
3. capture (data)		o				x + 0x80
4. reset_alarm (data)		o				x + 0x88
5. synchronize_clock (data)		o				x + 0x90
6. data_send (data)		o				x + 0x98
7. set_encryption_key (data)		o				x + 0xA0
8. transfer_key (data)		o				x + 0xA8

Attribute description

logical_name

Identifies the “M-Bus client” object instance. See 6.2.22.

mbus_port_reference

Provides reference to an “M-Bus master port setup” object, used to configure an M-Bus port, each interface allowing to exchange data with one or more M-Bus slave devices.

capture_definition

Provides the *capture_definition* for M-Bus slave devices.

NOTE 2 This attribute can be pre-configured or written as part of the installation procedure.

array capture_definition_element

```
capture_definition_element ::= structure
{
    data_information_block:     octet-string,
    value_information_block:    octet-string
}
```

NOTE 3 The elements data_information_block and value_information_block correspond to Data Information Block (DIB) and Value Information Block (VIB) described in EN 13757-3:2018, 6.2 and Clause 6.3 respectively.

© Copyright 1997-2020 DLMS User Association

capture_period	>= 1: Automatic capturing assumed. Specifies the capture period in seconds. 0: No automatic capturing: capturing is triggered externally or capture events occur asynchronously.
primary_ address	Carries the primary address of the M-Bus slave device. The range is 0...250. Each M-bus device is bound to a channel of the M-Bus master. However, there is no direct link between the primary address and the channel number. NOTE 4 The specification of the B field of the OBIS codes limits the range to 1…64 within one logical device. See 7.3.2. If the slave device is already configured and thus, its primary address is different from 0, then this value shall be written to the *primary_address* attribute. From this moment, the data exchange with the M-Bus slave device is possible. Otherwise, the *slave_install* method shall be used; see below. NOTE 5 The *primary_address* attribute cannot be used to store a desired primary address for an unconfigured slave device. If the *primary_address* attribute is set, this means that the M-Bus client can immediately operate with this primary address, which is not the case with an unconfigured slave device.
identification_ number	Carries the Identification Number element of the data header as specified in EN 13757-7:2018, 7.5.1. This attribute, together with attributes 7, 8 and 9 are filled with the values found in the first message received after installation. If in subsequent messages these values are not the same, the message is discarded.
manufacturer_ id	Carries the Manufacturer Identification element of the data header as specified in EN 13757-7:2018, 7.5.2.
version	Carries the Version element of the data header as specified in EN 13757-7:2018, 7.5.3.
device_type	Carries the Device type identification element of the data header as specified in EN 13757-7:2018, 7.5.4, Table 13.
access_ number	Carries the Access Number element of the data header as specified in EN 13757-7:2018, 7.5.5.
status	Carries the Status byte element of the data header as specified in EN 13757-7:2018, 7.5.6, Table 14 and 15. It is updated with every readout of the M-Bus slave device.
alarm	Carries the Alarm state specified in EN 13757-3:2018 Annex D. It is updated with every readout of the M-Bus slave device.
configuration	Carries the Configuration field (previously: Signature field) as specified in EN 13757-7:2018, 7.5.8. It contains information about the encryption mode and the number of encrypted bytes. It is updated with every readout of the M-Bus slave device.
encryption_ key_status	Provides information on the status of the encryption key. See also Annex B. enum: (0) no encryption key, (1) encryption_key set, (2) encryption_key transferred, (3) encryption_key set and transferred, (4) encryption_key in use.

© Copyright 1997-2020 DLMS User Association

Method description	
slave_install (data)	Installs a slave device, which is yet unconfigured (its primary address is 0). data ::= unsigned This method can be successfully invoked only if the current value of the *primary_address* attribute is 0. The following actions are performed: - the M-Bus address 0 is checked for presence of a new device; - if no uninstalled M-Bus slave is found, the method invocation fails; - if the *slave_install* method is invoked with “0” as a parameter, then the primary address is assigned automatically. This is done by checking the *primary_address* attribute of all M-Bus client objects in the DLMS/COSEM device and then selecting the first unused number. The *primary_address* attribute is set to this address and it is then transferred to the M-Bus slave device; - if the *slave_install* method is invoked with a primary address (other than 0) as a parameter, then the *primary_address* attribute is set to this value and it is then transferred to the M-Bus slave device. NOTE 6 Unconfigured slave devices are configured with primary address as specified in EN 13757-7:2018, 8.4.
slave_deinstall (data)	De-installs the slave device. The main purpose of this service is to de-install the M-Bus slave device and to prepare the master for the installation of a new device. The following actions are performed: - the M-Bus address is set to 0 in the M-Bus slave device; - the encryption key transferred previously to the M-Bus slave device is destroyed; the default key is not affected; - the *encryption_key_status* is set to (0): no encryption_key; - the attribute *primary_address* is also set to 0. NOTE 7 A new M-Bus slave can be installed only once the value of the *primary_address* attribute is 0. data ::= unsigned (0)
capture (data)	Captures values – as specified by the *capture_definition* attribute – from the M-Bus slave device. data ::= integer (0)
reset_alarm (data)	Resets alarm state of the M-Bus slave device. data ::= integer (0)
synchronize_ clock (data)	Synchronize the clock of the M-Bus slave device with that of the M-Bus client device. data ::= integer (0)
data_send (data)	Sends data to the M-Bus slave device. data ::= array data_definition_element

© Copyright 1997-2020 DLMS User Association

data_send (data) (continued)

```
data_definition_element ::= structure
{
    data_information_block:     octet-string,
    value_information_block:    octet-string,
    data:                       CHOICE
    {
        -- simple data types
        null-data                [0],
        bit-string               [4],
        double-long              [5],
        double-long-unsigned     [6],
        octet-string             [9],
        visible-string           [10],
        utf8-string              [12],
        bcd                      [13]
        integer                  [15],
        long                     [16],
        unsigned                 [17],
        long-unsigned            [18],
        long64                   [20],
        long64-unsigned          [21],
        float32                  [23],
        float64                  [24]
    }
}
```

set_ encryption_ key (data)

Sets the encryption key in the M-Bus client and enables encrypted communication with the M-Bus slave device.

data ::= octet-string (encryption_key)

After the installation of the M-Bus slave, the M-Bus client holds an empty encryption key. With this, encryption of M-Bus frames is disabled.

Encryption can be disabled by invoking the *set_encryption_key* method with an octet-string of zero length.

Changing the encryption key requires two steps:

- first, the key is sent to the M-Bus slave, encrypted with the default key, using the *transfer_key* method;
- second, the key is set in the M-Bus master using the *set_encryption_key* method.

transfer_key (data)

Transfers an encryption key to the M-Bus slave device.

data ::= octet-string (encrypted_key)

Before encrypted M-Bus frames can be used, an operational encryption key has to be sent to the M-Bus slave, by invoking the *transfer_key method*. The method invocation parameter is the operational encryption key encrypted with the default key of the M-Bus slave device. The M-Bus frame sent is not encrypted. After the execution, the encryption is enabled and all further telegrams are encrypted.

Each M-Bus slave device shall be delivered with a default encryption key.

A new encryption key can be set in the M-Bus client by invoking the *set_encryption_key* method with the new encryption key as a parameter.

With further invocations of the *transfer_key method*, new encryption keys can be sent to the M-Bus slave. The method invocation parameter is the new encryption key encrypted with the default key. The M-Bus frame is encrypted.

When an M-Bus slave is de-installed, the encryption key is destroyed but the default key is not affected. Encryption remains disabled until a new encryption key is transferred.

© Copyright 1997-2020 DLMS User Association

4.8.4 Wireless Mode Q channel (class_id = 73, version = 1)

Instances of this IC define the operational parameters for communication using the mode Q interfaces. See also EN 13757-5:2015.

Wireless Mode Q channel		0...n	class_id = 73, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. addr_state	(static)	enum				x + 0x08
3. device_address	(static)	octet-string				x + 0x10
4. address_mask	(static)	octet-string				x + 0x18
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “Wireless Mode Q channel” object instance. See 6.2.22.
addr_state	Defines whether or not the device has been assigned an address since last power up of the device. enum: (0) not assigned an address yet, (1) assigned an address either by manual setting or by automated method.
device_address	The currently assigned address of the device on the network.
address_mask	The group address the device will respond to when short form addressing is used.

4.8.5 M-Bus master port setup (class_id = 74, version = 0)

Instances of this IC define the operational parameters for communication using the EN 13757-2 interfaces if the device acts as an M-bus master.

M-Bus master port setup		0...n	class_id = 74, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. comm_speed	(static)	enum	0	7	3	x + 0x08
Specific methods		***m/o***				

Attribute description		
logical_name	Identifies the “M-Bus master port setup” object instance. See 6.2.22.	
comm_speed	The communication speed supported by the port	
	enum: (0)	300 baud,
	(1)	600 baud,
	(2)	1 200 baud,
	(3)	2 400 baud,
	(4)	4 800 baud,
	(5)	9 600 baud,
	(6)	19 200 baud,
	(7)	38 400 baud

© Copyright 1997-2020 DLMS User Association

4.8.6 DLMS/COSEM server M-Bus port setup (class_id = 76, version = 0)

Instances of the “DLMS/COSEM server M-Bus port setup” are used in DLMS/COSEM servers hosted by M-Bus slave devices, using the DLMS/COSEM wired or wireless M-Bus (wM-Bus) communication profile.

DLMS/COSEM server M-Bus port setup		**0...n**	**class_id = 76, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. M-Bus_profile_selection	(static)	octet-string				x + 0x08
3. M-Bus_port_communication_state	(dyn.)	enum				x + 0x10
4. M-Bus_Data_Header_Type	(dyn.)	enum			2	x + 0x18
5. primary_address	(static)	unsigned			0	x + 0x20
6. identification_number	(static)	double-long-unsigned			0	x + 0x28
7. manufacturer_id	(static)	long-unsigned			0	x + 0x30
8. version	(static)	unsigned			0	x + 0x38
9. device_type	(static)	unsigned			0	x + 0x40
10. max_pdu_size	(static)	long-unsigned				x + 0x48
11. listening_window	(static)	array				x + 0x50
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “DLMS/COSEM server M-Bus port setup” object instance. See 6.2.22.
M-Bus_profile_ selection	References an M-Bus communication port setup object describing the physical capabilities for wired or wireless communication. The referenced object is either an “M-Bus slave port setup” object (class_id = 25) or a “Wireless Mode Q channel” object (class_id = 73).
M-Bus_port_ communication_state	Carries the communication status of the M-Bus node. See EN 13757-4:2013, 11.6.3 Table 27. enum: (0) No access: Meter provides no access windows (unidirectional meter), (1) Temporary no access: Meter supports bidirectional access in general, but there is no access window after this transmission (e.g. temporary no access in order to keep duty cycle limits or to limit energy consumption), (2) Limited access: Meter provides a short access windows only immediately after this transmission (e.g. battery operated meter), (3) Unlimited access: Meter provides unlimited access at least until next transmission (e.g. mains powered devices) (4) This attribute is relevant only in wM-Bus.

© Copyright 1997-2020 DLMS User Association

M-Bus_Data_ Header_Type	Carries the type of the M-Bus Data Header, derived from the CI_{TL} value from the current communication. In the case of a push operation the default value (2) shall be applied. enum: (0) M-Bus_Data_Header_Type == None_M-Bus_Data_Header, the value of the CI_{TL} field shall be 0x10 when segmentation is not used, and shall be 0x00 .. 0x1F when segmentation is used (with the FIN bit set to 0 in all segments except in the last segment); (1) M-Bus_Data_Header_Type == Short_M-Bus_Data_Header, the value of the CI_{TL} field shall be 0x61 in an M-Bus frame sent by a master and 0x7D in a an M-Bus frame sent by a slave; (2) M-Bus_Data_Header_Type == Long_M-Bus_Data_Header, the value of the CI_{TL} field shall be 0x60 in an M-Bus frame sent by a master and 0x7C in a an M-Bus frame sent by a slave.
primary_ address	Carries the primary address of the M-Bus slave device. See DLMS UA 1000-2 Ed. 10, Table 102. If the slave device is already configured and thus, its primary address is different from 0, then the data exchange with the M-Bus slave device is possible.
identification_ number	Carries the Identification Number element of the Data Header as specified in EN 13757-7:2018, 7.5.1. In the case of unidirectional communication this value – together with attributes 6, 7, 8 and 9 – shall be parametrized. In the case of bi-directional communication this attribute – together with attributes 6, 7, 8 and 9 – are filled with the values found in the first message received after installation by different M-Bus network management interfaces. NOTE If in subsequent messages these values are not the same, the message is discarded.
manufacturer_id	Carries the Manufacturer Identification element of the Data Header as specified in EN 13757-7:2018, 7.5.2.
version	Carries the Version element of the Data Header as specified in EN 13757-7:2018, 7.5.3.
device_type	Carries the Device type identification element of the Data Header as specified in EN 13757-7:2018, 7.5.4, Table 13.
max_pdu_size	Contains length capability available from M-Bus lower layers (expressed in bytes). Specifies the maximum length of the DLMS payload (an APDU or a part of it) that can be carried by one M-Bus frame. In the case of long messages, either block transfer provided by the DLMS/COSEM application layer or segmentation provided by the transport layer or both mechanisms can be used.

© Copyright 1997-2020 DLMS User Association

listening_window	Defines the time points when the point-to-point communication window(s) become active (start_time) and inactive (end_time).The start_time implicitly defines the period.
	EXAMPLE When the day of month is not specified (equal to 0xFF) this means that we have a daily listening window management. Daily, monthly ...window management can be defined.
	array window_element
	window_element ::= structure
	{
	start_time: octet-string,
	end_time: octet-string
	}
	start_time and end_time are formatted as specified in 4.1.6.1 for *date-time.*
	This attribute is relevant only in wM-Bus.

4.8.7 M-Bus diagnostic (class_id = 77, version = 0)

Instances of the IC “M-Bus diagnostic” hold information related to the operation of the M-Bus network, like current signal strength, channel identifier, link status to the M-Bus network and counters related to the frame exchange, transmission and frame reception quality.

M-Bus diagnostic		**0...n**	**class_id = 77, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				X
2. received-signal-strength	(dyn.)	unsigned				x + 0x08
3. channel_Id	(dyn.)	unsigned			0	x + 0x10
4. link_status	(dyn.)	enum				x + 0x18
5. broadcast_frames_counter	(dyn.)	array			0	x + 0x20
6. transmissions_counter	(dyn.)	double-long-unsigned			0	x + 0x28
7. FCS_OK_frames_counter	(dyn.)	double-long-unsigned			0	x + 0x30
8. FCS_NOK_frames_counter	(dyn.)	double-long-unsigned			0	x + 0x38
9. capture_time	(dyn.)	structure				x + 0x40
Specific methods		***m/o***				
1. reset (data)		o				x + 0x48

Attribute description	
logical_name	Identifies the “M-Bus diagnostic” object instance. See 6.2.22.
received_signal_ strength	When the wM-Bus profile is used, this attribute holds the value of signal strength of the last wM-Bus frame received, expressed in dBm. This attribute is relevant only for wireless bidirectional M-Bus communication.

© Copyright 1997-2020 DLMS User Association

channel_Id	When the wM-Bus profile is used, this attribute holds the identification of the channel currently used. The default value is 0. This attribute is relevant only for wM-Bus communication.
link_status	When the wM-Bus profile is used, this attribute holds the current status of link to the M-Bus network. The possible stati are as specified in EN 13757-5:2015, 9.7.4.1.7 Link State. enum: (0) Default (data never received), (1) Link in normal operation, (2) Link temporarily interrupted, (3) Link permanently interrupted NOTE 1 If synchronisation with network is lost, trials start automatically to re-establish the link by performing radio scans. As long as re-synchronisation attempts are in progress, the link_status is temporarily interrupted. Link_status changes to normal operation when the link is re-established. Link_status changes to permanently interrupted , when the manufacturer specified number of attempts is exceeded. This attribute is relevant only for wM-Bus communication.

broadcast_frames_ counter	Holds the broadcast-frames-counter values with time stamp of last frame received and differentiated by client identifiers.
	array broadcast_frame_counter_definition
	broadcast_frame_counter_definition ::= structure
	{
	client_id: unsigned,
	counter: double-long-unsigned,
	time_stamp: date-time
	}
	The default value is 0.

transmissions_ counter	Counts the number of frames transmitted by the related M-Bus port. The transmission counter is incremented at the beginning of each transmission phase. A client system can write this variable to update the counter. When the transmissions counter reaches the maximum value, it automatically returns to 0 on the next increment. The default value is 0.
FCS_OK_frames_ counter	Counts the number of frames received with a correct checksum. When the FCS_OK_frames_counter field reaches the maximum value, it automatically returns to 0 on the next increment. The default value is 0.
FCS_NOK_frames_ counter	Counts the number of frames received with an incorrect checksum. When the FCS_NOK frames counter field reaches the maximum value, it automatically returns to 0 on the next increment. The default value is 0.

© Copyright 1997-2020 DLMS User Association

capture_time	Holds the time stamp of the most recent change of the value of the attributes 2, 4, 6, 7 or 8.
	capture_time ::= structure
	{
	attribute_id: unsigned,
	time_stamp: date-time
	}

Method description	
reset (data)	Clears all counters, received_signal_strength, link_status and capture_time. data ::= integer(0) NOTE 2 Channel_id management is outside the scope of the specification of this IC. See EN 13757-4:2013.

© Copyright 1997-2020 DLMS User Association

4.9 Interface classes for setting up data exchange over the Internet

4.9.1 TCP-UDP setup (class_id = 41, version = 0)

This IC allows modelling the setup of the TCP or UDP sub-layer of the COSEM TCP or UDP based transport layer of a TCP-UDP/IP based communication profile.

In TCP-UDP/IP based communication profiles, all AAs between a physical device hosting one or more COSEM client application processes and a physical device hosting one or more COSEM server APs rely on a single TCP or UDP connection. The TCP or UDP entity is wrapped in the COSEM TCP-UDP based transport layer. Within a physical device, each AP – client AP or server logical device – is bound to a Wrapper Port (WPort). The binding is done with the help of the SAP Assignment object. See 4.4.5.

On the other hand, a COSEM TCP or UDP based transport layer may be capable to support more than one TCP or UDP connections, between a physical device and several peer physical devices hosting COSEM APs.

When a COSEM physical device supports various data link layers – for example Ethernet and PPP – an instance of the TCP-UDP setup object is necessary for each of them.

TCP-UDP setup		**0...n**	**class_id = 41, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. TCP-UDP_port	(static)	long-unsigned				x + 0x08
3. IP_reference	(static)	octet-string				x + 0x10
4. MSS	(static)	long-unsigned	40	65...535	576	x + 0x18
5. nb_of_sim_conn	(static)	unsigned	1			x + 0x20
6. inactivity_time_out	(static)	long-unsigned			180	x + 0x28
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “TCP-UDP setup” object instance. See 6.2.23.
TCP-UDP_port	Holds the TCP-UDP port number on which the physical device is listening for the DLMS/COSEM application. For DLMS/COSEM, the following port numbers have been registered by the IANA. See http://www.iana.org/assignments/port-numbers dlms/cosem 4059/TCP DLMS/COSEM dlms/cosem 4059/UDP DLMS/COSEM
IP_reference	References an IP setup object by its logical name. The referenced object contains information about the IP Address settings of the IP layer supporting the TCP-UDP layer.
MSS	With the help of the Maximum Segment Size (MSS) option, a TCP can indicate the maximum receive segment size to its partner. Note, that: - this option shall only be sent in the initial connection request (i.e. in segments with the SYN control bit sent); - if this option is not present, conventionally MSS is considered as its default value, 576; - MSS is not negotiable; its value is indicated by this attribute.

© Copyright 1997-2020 DLMS User Association

nb_of_sim_ conn	The maximum number of simultaneous connections the COSEM TCP-UDP based transport layer is able to support.
inactivity_ time_out	Defines the time, expressed in seconds over which, if no frame is received from the COSEM client, the inactive TCP connection shall be aborted. When this value is set to 0, this means that the *inactivity_time_out* is not operational. In other words, a TCP connection, once established, in normal conditions – no power failure, etc. – will never be aborted by the COSEM server. Note, that all actions related to the management of the inactivity time-out function – measuring the inactivity time, aborting the TCP connection if the time-out is over, etc. – are managed inside the TCP-UDP layer implementation.

4.9.2 IPv4 setup (class_id = 42, version = 0)

NOTE 1 Compared to earlier editions of the Blue Book, this specification provides improvements in presenting the attributes. As this does not constitute technical changes, the version of the IC remains 0.

This IC allows modelling the setup of the IPv4 layer, handling all information related to the IP Address settings associated to a given device and to a lower layer connection on which these settings are used.

There shall be an instance of this IC in a device for each different network interface implemented. For example, if a device has two interfaces (using the TCP-UDP/IPv4 profile on both of them), there shall be two instances of the IPv4 setup IC in that device: one for each of these interfaces.

IPv4 setup	**0...n**	**class_id = 42, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. DL_reference (static)	octet-string				x + 0x08
3. IP_address	double-long-unsigned				x + 0x10
4. multicast_IP_address	array				x + 0x18
5. IP_options	array				x + 0x20
6. subnet_mask	double-long-unsigned				x + 0x28
7. gateway_IP_address	double-long-unsigned				x + 0x30
8. use_DHCP_flag (static)	boolean				x + 0x38
9. primary_DNS_address	double-long-unsigned				x + 0x40
10. secondary_DNS_address	double-long-unsigned				x + 0x48
Specific methods	***m/o***				
1. add_mc_IP_address (data)	o				x + 0x60
2. delete_mc_IP_address (data)	o				x + 0x68
3. get_nbof_mc_IP_addresses (data)	o				x + 0x70

Attribute description	
logical_name	Identifies the “IPv4 setup” object instance. See 6.2.23.
DL_reference	References a Data link layer (e.g. Ethernet or PPP) setup object by its logical name. The referenced object contains information about the specific settings of the data link layer supporting the IP layer.

© Copyright 1997-2020 DLMS User Association

IP_address	Carries the value of the IP address (IPv4) of this physical device on the network to which the device is connected via the referenced lower layer interface. It can be either (static) or (dynamic). In the latter case, dynamic IP address assignment (for example DHCP) is used. If no IP address is assigned, the value is 0. EXAMPLE The IPv4 address 192.168.0.1 (in dotted decimal notation) corresponds to C0A80001 (hexa) which gives 3232235521 (double-long-unsigned).
multicast_IP_address	Contains an array of IP addresses. IP addresses in this array shall fall into the multicast group address range (“Class D” addresses, including IP addresses in the range of 224.0.0.0 to 239.255.255.255). When a device receives an IP datagram with one of these IP addresses in the destination IP address field, it shall consider that this datagram is addressed to it. multicast_IP_address ::= array double-long-unsigned

IP_options	Contains the necessary parameters to support the selected IP options – for example Datagram time-stamping or security services (IPSec).
	IP_options ::= array IP_options_element
	IP_options_element ::= structure
	{
	IP_Option_Type: unsigned,
	IP_Option_Length: unsigned,
	IP_Option_Data: octet-string
	}
	NOTE 2 In all cases, as specified in RFC 791, the IP_Option_Length field includes the total length of all three fields: IP_Option_Type, IP_Option_Length and IP_Option_Data.
	Allowed IP_Option_Types:
	- Security: IP_Option_Type = 0x82, IP_Option_Length = 11
	If this option is present, the device shall be allowed to send security, compartmentation, handling restrictions and TCC (closed user group) parameters within its IP Datagrams. The IP_Option_Data shall contain the value of the Security, Compartments, Handling Restrictions and Transmission Control Code values, as specified in RFC 791.
	- Loose Source and Record Route: IP_Option_Type = 0x83
	If this option is present, the device shall supply routing information to be used by the gateways in forwarding the datagram to the destination, and to record the route information. The IP_Option_Length and IP_Option_Data values are specified in RFC 791.
	- Strict Source and Record Route: IP_Option_Type = 0x89
	If this option is present, the device shall supply routing information to be used by the gateways in forwarding the datagram to the destination, and to record the route information. The IP_Option_Length and IP_Option_Data values are specified in RFC 791.
	- Record Route: IP_Option_Type = 0x07
	If this option is present, the device shall as well:
	- send originated IP Datagrams with that option, providing means to record the route of these Datagrams;
	- as a router, send routed IP Datagrams with the route option adjusted according to this option.

© Copyright 1997-2020 DLMS User Association

IP_options (continued)	The IP_Option_Length and IP_Option_Data values are specified in RFC 791. - Internet Timestamp: IP_Option_Type = 0x44 If this option is present, the device shall as well: - send originated IP Datagrams with that option, providing means to time-stamp the datagram in the route to its destination; - as a router, send routed IP Datagrams with the time-stamp option adjusted according to this option. The IP_Option_Length and IP_Option_Data values are specified in RFC 791.
subnet_mask	Contains the subnet mask. When sub-networking is used in a network segment, each device concerned shall behave conforming to the sub-networking rules. In order to do that, the device, besides of its IP address, needs also to know, how the IP address is structured within this sub-networked segment. The subnet_mask attribute carries this information. With IPv4, the *subnet_mask* is a 32 bits word, expressed exactly in the same format as an IP Address (for example 255.255.255.0), but has another meaning: the '0' bits of the *subnet_mask* indicate the portion of the IP Address which is still used as Device_ID on a sub-networked IP Network [2].
gateway_IP_ address	Contains the IP Address of the gateway device. In most IP implementations, there is code in the module that handles outgoing datagrams to decide if a datagram can be sent directly to the destination on the local network or if it shall be sent to a gateway. In order to be able to send non-local datagrams to the gateway, the device shall know the IP address of the gateway device assigned to the given network segment. If no IP address is assigned, the value is 0.
use_DHCP_ flag	When this flag is set to TRUE, the device uses DHCP (Dynamic Host Configuration Protocol) to dynamically determine the IP_address, subnet_mask and *gateway_IP_address* parameters. On the other hand, when this flag is set to FALSE, the *IP_address*, *subnet_mask* and *gateway_IP_address* parameters shall be locally set.
primary_DNS_ address	The IP Address of the primary Domain Name Server (DNS). If no IP address is assigned, the value is 0.
secondary_ DNS_ address	The IP Address of the secondary Domain Name Server (DNS). If no IP address is assigned, the value is 0.
Method description	
add_mc_IP_ address (IP_Address)	Adds one multicast IP address to the *multicast_IP_address* array. IP_Address ::= double-long-unsigned
delete_mc_IP _address (IP_Address)	Deletes one IP Address from the *multicast_IP_address* array. The IP Address to be deleted is identified by its value. IP_Address ::= double-long-unsigned
get_nbof_mc_ IP_addresses (data)	Returns the number of IP Addresses contained in the *multicast_IP_address* array. data ::= unsigned

[2] See more about sub-networking in RFC 940 and RFC 950.

© Copyright 1997-2020 DLMS User Association

4.9.3 IPv6 setup (class_id = 48, version = 0)

NOTE 1 See also Annex C.

The IPv6 setup IC allows modelling the setup of the IPv6 layer, handling all information related to the IPv6 address settings associated to a given device and to a lower layer connection on which these settings are used.

There shall be an instance of this IC in a device for each different network interface implemented. For example, if a device has two interfaces (using the UDP/IP and/or TCP/IP profile on both of them), there shall be two instances of the IPv6 setup IC in that device: one for each of these interfaces.

IPv6 setup		**0...n**	**class_id = 48, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. DL_reference	(static)	octet-string				x + 0x08
3. address_config_mode	(static)	enum			0	x + 0x10
4. unicast_IPv6_addresses		array				x + 0x18
5. multicast_IPv6_addresses	(static)	array				x + 0x20
6. gateway_IPv6_addresses	(static)	array			0	x + 0x28
7. primary_DNS_address	(static)	octet-string			0	x + 0x30
8. secondary_DNS_address	(static)	octet-string			0	x + 0x38
9. traffic_class	(static)	unsigned	0	63	0	x + 0x40
10. neighbor_discovery_setup	(static)	array				x + 0x48
Specific methods		***m/o***				
1. add_IPv6_address (data)		*o*				x + 0x60
2. remove_IPv6_address (data)		*o*				x + 0x68

Attribute description	
logical_name	Identifies the “IPv6 setup” object instance. See 6.2.23.
DL_reference	References a Data link layer setup object by its logical name. The referenced object contains information about the specific settings of the data link layer supporting the IPv6 layer.
address_ config_mode	Defines the IPv6 address configuration mode enum: (0) Auto-configuration (default), (1) DHCPv6, (2) Manual, (3) ND (Neighbour Discovery) NOTE 2 The *address_config_mode* is common for all IPv6 addresses managed by an instance of the IPv6 setup class.
unicast_IPv6_ addresses	Carries unicast IPv6 address(es) assigned to the related interface of the physical device on the network (unique local unicast, link local unicast and / or global unicast addresses). An IPv6 address can be either (static) or (dynamic) or both. unicast_IPv6_addresses ::= array octet-string The format of each unicast IPv6 address shall be as specified in RFC 3513.

© Copyright 1997-2020 DLMS User Association

unicast_IPv6_ addresses (continued)	If no unicast IPv6 address is assigned to the interface, an array of zero elements is present (default). To reset all unicast IPv6 address(es) configured, an array of zero elements shall be written.
multicast_IPv6_ addresses	Contains an array of IPv6 addresses used for multicast. multicast_IPv6_addresses ::= array octet-string The format of each multicast IPv6 address shall be as specified in RFC 3513 . If no multicast IPv6 address is assigned to the interface, an array of zero elements is present (default). To reset all multicast IPv6 address(es) configured, an array of zero elements shall be written.
gateway_IPv6_ addresses	Contains the IPv6 addresses of the IPv6 gateway device. gateway_IPv6_addresses ::= array octet-string The format of each gateway IPv6 address shall be as specified in RFC 3513. If no gateway IPv6 address is assigned to the interface, an array of zero elements is present (default). To reset all gateway IPv6 address(es) configured, an array of zero elements shall be written.
primary_DNS_ address	Contains the IPv6 address of the primary Domain Name Server (DNS). If no IPv6 address is assigned, the length of the octet-string shall be 0.
secondary_DNS_ address	Contains the IPv6 address of the secondary Domain Name Server (DNS). If no IPv6 address is assigned, the length of the octet-string shall be 0.
traffic_class	Contains the traffic class element of the IPv6 header. The 6 most-significant bits are used for DSCP (Differentiated Services Codepoint), which is used to classify packets as specified in RFC 2474, Clause 3.

neighbor_ discovery_setup	Contains the configuration to be used for both routers and hosts to support the Neighbor Discovery protocol for IPv6 (RFC 4861).
	array neighbor_discovery_setup
	
	neighbor_discovery_setup ::= structure
	{
	RS_max_retry: unsigned,
	RS_retry_wait_time: long-unsigned,
	RA_send_period: double-long-unsigned
	}
	Where:

RS_max_retry	Gives the maximum number of router solicitation retries to be performed by a node if the expected router advertisement has not been received. Range: 1-255 Default value: 3
RS_retry_wait_time	Gives the waiting time in milliseconds between two successive router solicitation retries. Range: 0-65 535 Default value: 10 000
RA_send_period	Gives the router advertisement transmission period in seconds. Range: 0-4 294 967 295

© Copyright 1997-2020 DLMS User Association

Method description	

add_IPv6_ address (data)	Adds one IPv6 address for the physical interface to the IPv6 address array.
	data ::= structure
	{
	IPv6_address_type: enum,
	IPv6_address: octet-string
	}
	Where IPv6_address_type defines the type of IPv6 address to add:
	enum: (0) unicast,
	(1) multicast,
	(2) gateway
	IPv6_address specifies the IPv6 address to add. The new address will be automatically added at the end of the list. If an address has to be added to a specific position, this can be done by writing the whole array.

remove_IPv6_ address (data)	Removes one IPv6 address for the physical interface to the IPv6 address array.
	data ::= structure
	{
	IPv6_address_type: enum,
	IPv6_address: octet-string
	}
	Where IPv6_address_type defines the type of IPv6 address to remove:
	enum: (0) unicast,
	(1) multicast,
	(2) gateway
	IPv6_address specifies the IPv6 address to remove.

4.9.4 MAC address setup (class_id = 43, version = 0)

NOTE 1 The name and the use of this interface class has been changed in Edition 10 of the Blue Book from “Ethernet setup” to “MAC address setup” to allow a more general use, without changing the version.

Instances of this IC hold the MAC address of the physical device (or, more generally, a device or software.) There shall be an instance of this IC for each network interface of a physical device.

NOTE 2 In the case of the three-layer HDLC based communication profile, the MAC address (lower HDLC address) is carried by an IEC HDLC setup object.

NOTE 3 In the case of the S-FSK PLC communication profile, the MAC address is carried by a S-FSK Phy&MAC setup object.

MAC address setup	**0...n**	**class_id = 43, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. MAC_address	octet-string				x + 0x08
Specific methods	***m/o***				

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “MAC address setup” object instance. See 6.2.21, 6.2.23 and 6.2.26.
mac_address	Holds the MAC address.

4.9.5 PPP setup (class_id = 44, version = 0)

NOTE 1 Compared to earlier editions of the Blue Book, this specification provides improvements in presenting the attributes. As this does not constitute technical changes, the version of the IC remains 0.

This IC allows modelling the setup of interfaces using the PPP protocol, by handling all information related to PPP settings associated to a given physical device and to a lower layer connection on which these settings are used. There shall be an instance of this IC for each network interface of a physical device.

PPP setup		**0...n**	**class_id = 44, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. PHY_reference	(static)	octet-string				x + 0x08
3. LCP_options	(static)	LCP_options_type				x + 0x10
4. IPCP_options	(static)	IPCP_options_type				x + 0x18
5. PPP_authentication	(static)	PPP_auth_type				x + 0x20
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “PPP setup” object instance. See 6.2.23.
PHY_reference	References another object by its logical_name. The object referenced contains information about the specific physical layer interface, supporting the PPP layer.

LCP_options	Contains the necessary parameters to support the selected LCP configuration options.
	LCP_options_type ::= array LCP_options_type_element
	
	LCP_options_type_element ::= structure
	{
	LCP_Option_Type: unsigned,
	LCP_Option_Length: unsigned,
	LCP_Option_Data: CHOICE
	{
	structure [2] -- for Callback-data
	boolean [3] -- for ProtF-Compr and AdCtr-Compr,
	double-long-unsigned [6] -- for ACCM and Mag-Num,
	unsigned [17] -- for FCS-Alternatives,
	long-unsigned [18] -- for MRU and Auth-Prot
	}
	}
	NOTE 2 In all cases, as specified in IETF STD 51 / RFC 1661, the LCP_Option_Length field includes the total length of all three fields: LCP_Option_Type, LCP_Option_Length and LCP_Option_Data.

© Copyright 1997-2020 DLMS User Association

NOTE 3 For assigned values see *Point-to-Point (PPP) Protocol Field Assignments,* available at: http://www.iana.org/assignments/ppp-numbers/ppp-numbers.xml

The supported LCP_Option_Types are the following:

- Maximum-Receive-Unit (MRU), LCP_Option_Type = 1. See IETF STD 51 / RFC 1661.

  This configuration option may be sent to inform the peer that the implementation can receive larger packets, or to request that peer send smaller packets. The default value is 1 500 octets;

- Async-Control-Character-Map (ACCM), LCP_Option_Type = 2. See IETF STD 51 / RFC 1662.

  This configuration option provides a method to negotiate the use of control character transparency on asynchronous links;

- Authentication-Protocol, LCP_Option_Type = 3. See IETF STD 51 / RFC 1661.

  This configuration option provides a method to negotiate the use of a specific protocol for authentication. By default, authentication is not required. The value indicates the authentication protocol used on the given PPP link. Possible values are:

  0x0000 – No authentication protocol is used,
  0xc023 – The PAP protocol is used,
  0xc223 – The CHAP protocol is used,
  0xc227 – The EAP protocol is used.

- Magic-Number, LCP_Option_Type = 5. See IETF STD 51 / RFC 1661.

  This configuration option provides a method to detect looped-back links and other data link layer anomalies;

- Protocol-Field-Compression (PFC), LCP_Option_Type = 7. See IETF STD 51 / RFC 1661.

  This configuration option provides a method to negotiate the compression of the PPP protocol fields;

- Address-and-Control-Field-Compression (ACFC), LCP_Option_Type = 8. See IETF STD 51 / RFC 1661.

  This configuration option provides a method to negotiate the compression of the data link layer address and control fields;

- FCS-Alternatives, LCP_Option_Type = 9. See RFC 1570.

  This configuration option provides a method for an implementation to specify another FCS format to be sent by the peer, or to negotiate away the FCS altogether. The value of the FCS-Alter (FCS Alternatives) options field identifies the FCS used. This field is one octet, and is comprised of the "logical or" of the following values:

  bit 1 Null FCS,
  bit 2 CCITT 16-bit FCS,
  bit 4 CCITT 32-bit FCS.

- Callback, LCP_Option_Type = 13. See RFC 1570.

  This configuration option provides a method for an implementation to request a dial-up peer to call back. This provides enhanced security by ensuring that the remote site can connect only from a single location as defined by the callback number.

© Copyright 1997-2020 DLMS User Association

LCP_options (continued)

```
callback_data ::= structure
{
   callback_active:          boolean,    // default: false,
   callback_data_length:     unsigned,
   callback_operation:       unsigned,
   callback-message:         octet-string
}
```

Where:

- the callback-active field indicates whether the callback option is active on this PPP link;
- the callback_operation field indicates the contents of the Message field;

  callback_operation: unsigned

  (0) Location determined by user authentication,
  (1) Dialling string,
  (2) Location identifier,
  (3) E.164 number,
  (4) X.500 distinguished name,
  (5) Unassigned,
  (6) Location is determined during CBCP negotiation.

- the callback-message field is zero or more octets, and its general contents are determined by the callback_operation field. The actual format of the information is site or application specific.

IPCP_options

Contains the necessary parameters for the IP Control Protocol – the Network Control Protocol module of the PPP – that allow the negotiation of desirable Internet Protocol parameters. For details on IPCP, please refer to RFC 1332.

IPCP_options_type ::= array IPCP_options_type_element

```
IPCP_options_type_element ::=    structure
{
      IPCP_Option_Type:          unsigned,
      IPCP_Option_Length:        unsigned,
      IPCP_Option_Data:          CHOICE
      {
            array                          [1] -- for Pref-Peer-IP,
            -- each IP address is of type double-long-unsigned
            boolean                        [3] -- for GAO and USIP,
            double-long-unsigned           [6] -- for Pref-Local-IP,
            long-unsigned                  [18] -- for IP-Comp-Prot
      }
}
```

NOTE 4 In all cases, as specified in RFC 1332, the IPCP_Option_Length field includes the total length of all three fields: IPCP_Option_Type, IPCP_Option_Length and IPCP_Option_Data.

© Copyright 1997-2020 DLMS User Association

IPCP_options (continued)

The supported IPCP_Option_Types are the following:

- IP-Compression-Protocol (IP-Comp-Prot) IPCP_Option_Type = 2. See RFC 1332.

  This configuration option provides a way to negotiate the use of a specific compression protocol. By default, compression is not enabled. Possible values are:

  0x0000 – No IP Compression is used (default),

  0x002d – Van Jacobson Compressed TCP/IP (RFC 1332),

  0x0003 – Robust Header Compression (ROHC) (RFC 3241),

  0x0061 – IP Header Compression (RFC 2507, RFC 3544)

- Preferred-Local-IP-Address (Pref-Local-I), IPCP_Option_Type = 3. See RFC 1332.

  This configuration option provides a way to negotiate the IP address to be used on the local end of the link. It allows the sender of the Configure-Request to state, which IP-address is desired, or to request that the peer provide the information. The peer can provide this information by NAK-ing the option, and returning a valid IP-Address;

  NOTE 5 In RFC 1332 the name of option Type 3 is “IP-Address”.

  NOTE 6 Option types 20, 21 and 22 have been specified for the purposes of DLMS/COSEM; they are not specified in RFC 1332.

- Preferred-Peer-IP-Addresses (Pref-Peer-IP), IPCP_Option_Type = 20.

  This configuration option provides a way to negotiate the IP Address to be used on the remote end of the link. When the Grant-Access-Only-to-Pref-Peer-on-List (GAO) parameter is set to be TRUE, the device shall accept PPP connection only with a remote device having one of the IP Addresses on this list. When the Use-Static-IP-Pool (USIP) parameter is set to TRUE, the COSEM Server device shall try to assign one of these IP Addresses to the remote device;

- Grant-Access-Only-to-Pref-Peer-on-List (GAO), IPCP_Option_Type = 21.

  This configuration option indicates whether the device can accept PPP connection only with peer devices with IP Address on the above list or not. Its default value is FALSE;

- Use-Static-IP-Pool (USIP), IPCP_Option_Type = 22.

  This configuration option indicates whether the device should try to assign one of the IP Addresses of the Preferred-Peer-IP-Addresses to the remote end device during the IP Address negotiation phase or not.

PPP_ authentication

Contains the parameters required by the PPP authentication procedure used.

```
PPP_auth_type ::=      CHOICE
{
      null-data    [0] -- used when no authentication is required,
      structure    [2] -- PAP_login or CHAP_algorithm or EAP_params
}

PAP_login ::=      structure
{
      user-name:        octet-string,
      PAP-password:     octet-string
}
```


© Copyright 1997-2020 DLMS User Association

PPP_ authentication (continued)

CHAP_algorithm ::= structure
{
user-name: octet-string,
algorithm_id: unsigned
}

Possible values for CHAP-algorithm-id parameter today are as follows:

0x05: CHAP with MD5 (default), see RFC 1994.
0x06:SHA-1,
0x80: MS-CHAP, see RFC 2433
0x81: MS-CHAP-2, see RFC 2759.

NOTE 7 New values can be used as become assigned.

When CHAP is used, a “secret” is also required to verify the “challenge” sent by the client. This “secret” is not accessible in the PPP setup object.

NOTE 8 The PPP Challenge Handshake Authentication Protocol (CHAP) is specified in RFC 1994.

EAP_params ::= structure
{
md5_challenge (Type 4): boolean,
one_time_password (OTP, Type 5): boolean,
generic_token_card (GTC, Type 6): boolean
}

NOTE 9 The Extensible Authentication Protocol (EAP) is specified in RFC 3748.

4.9.6 SMTP setup (class_id = 46, version = 0)

This IC allows setting up data exchange using the SMTP protocol.

SMTP modem setup		**0...n**	**class_id = 46, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. server_port	(static)	long-unsigned			25	x + 0x08
3. user_name	(static)	octet-string				x + 0x10
4. login_password	(static)	octet-string				x + 0x18
5. server_address	(static)	octet-string				x + 0x20
6. sender_address	(static)	octet-string				x + 0x28
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “SMTP setup” object instance. See 6.2.23.
server_port	Defines the value of the TCP-UDP port related to this protocol. By default, this value is the SMTP port numberID assigned by IANA: - smtp 25/tcp, smtp 25/udp Simple Mail Transfer
user_name	Defines the user name to be used for the login to the SMTP server.
login_ password	Password to be used for login. When the string is void, this means that there is no authentication.
server_address	Defines the server address as an octet string. This server address can be a name, which shall be resolvable by the primary DNS or the secondary DNS.

© Copyright 1997-2020 DLMS User Association

	In the case when it is directly the IP address of the server, which is specified here, it shall be a string in dotted format. EXAMPLE: 163.187.45.87.
sender_ address	Defines the sender address as an octet string. This sender address can be a name. In the case when it is directly the IP address of the sender, which is specified here, it will be a string in dotted format.

4.9.7 NTP setup (class_id = 100, version = 0)

Instances of the “NTP setup” IC allow setting up time synchronisation using the NTP protocol as specified in RFC 5905. One or several instances may be configured to support multiple time servers.

NTP Setup		**0...n**	**class_id = 100, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. activated	(static)	boolean			FALSE	x + 0x08
3. server_address	(static)	octet-string				x + 0x10
4. server_port	(static)	long-unsigned			123	x + 0x18
5. authentication_method	(static)	enum				x + 0x20
6. authentication_keys	(static)	array				x + 0x28
7. client_key	(static)	octet-string				x + 0x30
Specific methods		***m/o***				
1. synchronize (data)		m				x + 0x38
2. add_authentication_key (data)		o				x + 0x40
3. delete_authentication_key (data)		o				x + 0x48

Attribute description	
logical_name	Identifies the “NTP setup” object instance. See 6.2.23.
activated	Defines if the NTP time synchronisation is active or not. Synchronisation active = TRUE
server_address	Defines the NTP server address as an octet string. This server address can be a name, which must be resolvable by the primary DNS or the secondary DNS. In the case when it is directly the IP address of the server the dot-decimal notation shall be used for IPv4 addresses and the text format for IPv6 addresses. Examples: 163.187.45.87 (IPv4) or 2001:db8::1:0:0:1 (IPv6)
server_port	Defines the value of the UDP port related to this protocol. By default, this value is the NTP port number ID assigned by IANA: ntp 123/udp Network Time Protocol
authentication_ method	Defines the authentication mode used for NTP protocol enum: (0) no_security, (1) shared_secrets, (2) auto_key_IFF

© Copyright 1997-2020 DLMS User Association

authentication_keys	Contains the necessary symmetric keys if shared secrets mode of authentication is used.
	authentication_keys ::= array authentication_key
	authentication_key ::= structure
	{
	key_id double-long-unsigned,
	key octet-string
	}

client_key	Specifies the client key (NTP server public key) for NTP auto key authentication mechanism using IFF authentication scheme (auto_key_IFF).
Method description	
synchronize (data)	Synchronizes the time of the DLMS server with the NTP server. data ::= integer(0)

add_authentication_ key (data)	Adds a new symmetric authentication key to authentication key array.
	data ::= structure
	{
	key_id double-long-unsigned,
	key octet-string
	}

delete_authentication_ key (data)	Deletes a symmetric authentication key from the key array. The key to be deleted is identified by its key_id. data ::= double-long-unsigned

© Copyright 1997-2020 DLMS User Association

4.10 Interface classes for setting up data exchange using S-FSK PLC

4.10.1 General

This subclause specifies COSEM interface classes to set up and manage the protocol layers of DLMS/COSEM S-FSK PLC communication profile:

- the S-FSK Physical layer and the MAC sub-layer as defined in IEC 61334-5-1:2001 and IEC 61334-4-512:2001;
- the LLC sub-layer as specified in IEC 61334-4-32:1996.

The MIB variables / logical link parameters specified in IEC 61334-4-512:2001, IEC 61334-5-1:2001, IEC 61334-4-32:1996 and ISO/IEC 8802-2:1998 respectively have been mapped to attributes and/or methods of COSEM ICs. The specification of these elements has been taken from the above standards and the text has been adapted to the DLMS/COSEM environment.

NOTE IEC 61334-4-512:2001 also specifies some management variables to be used on the Client side. However, the Client side object model is not covered in this document.

For definitions related to S-FSK PLC profile see 3.2.

4.10.2 Overview

COSEM objects for setting up the S-FSK PLC channel and the LLC layer, if implemented, shall be located in the Management Logical Device of COSEM servers.

Figure 27 shows an example with a COSEM physical device comprising three logical devices. Each logical device shall contain a Logical Device Name (LDN) object. Each logical device contains one or more Association objects, one for each client supported.

NOTE As in this example there is more than one logical device, the mandatory Management Logical Device contains a SAP Assignment object instead of a Logical Device Name object.

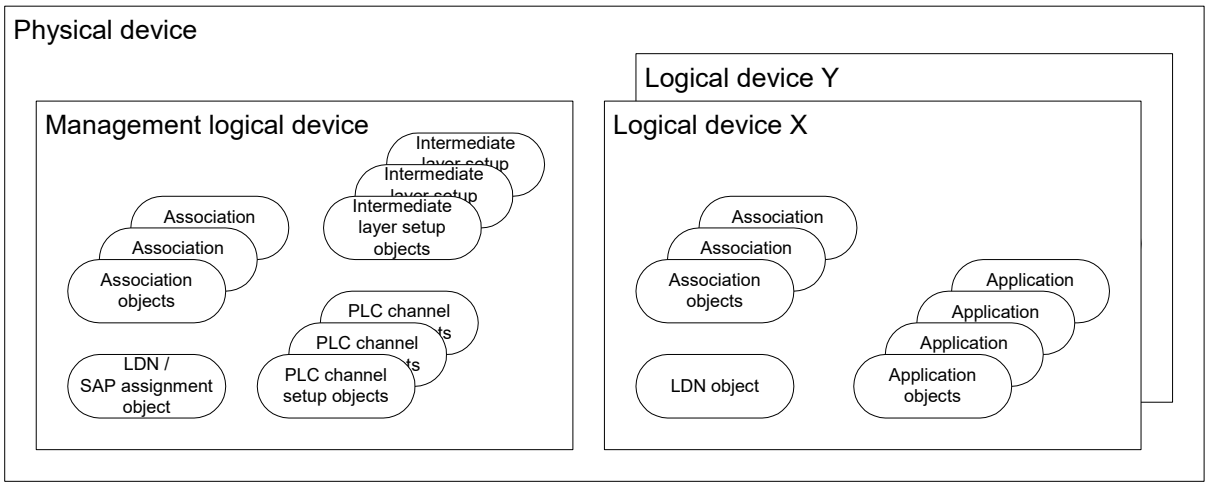


Figure 28 – Object model of DLMS/COSEM servers

The management logical device contains the setup objects of the physical and MAC layers of the PLC channel, as well as setup objects for the intermediate layer(s). It may contain further application objects.

The other logical devices, in addition to the Association and Logical Device Name objects mentioned above, contain further application objects, holding parameters and measurement values.

IEC 61334-4-512:2001 uses DLMS named variables to model the MIB objects and specifies their DLMS name in the range 8…184. For compatibility with existing implementations, the short names 8…400 [sic] are reserved for devices using the IEC 61334-5-1:2001 S-FSK PLC profiles

© Copyright 1997-2020 DLMS User Association

without COSEM. Therefore, when mapping the attributes and methods of the COSEM objects specified in this document to DLMS named variables (SN mapping) this range shall not be used.

Table 31 shows the mapping of MIB variables to attributes and/or methods of COSEM ICs.

Note that on the one hand, not all MIB variables specified in IEC 61334-4-512:2001 have been mapped to attributes and methods of COSEM ICs. On the other hand, some new management variables are specified in this document.

Table 31 – Mapping IEC 61334-4-512:2001 MIB variables to COSEM IC attributes / methods

Name	Reference (unless otherwise indicated)	Interface class	class_id / attribute / method
S-FSK Physical layer management			
delta-electrical-phase	variable 1	S-FSK Phy&MAC set-up (class_id = 50, version = 1)	50 / Attr. 3
max-receiving-gain	variable 2		50 / Attr. 4
max-transmitting-gain	–		50 / Attr. 5
search-initiator-threshold	–		50 / Attr. 6
frequencies	–		50 / Attr. 7
transmission-speed	–		50 / Attr. 15
MAC layer management			
mac-address	variable 3	S-FSK Phy&MAC set-up (class_id = 50, version = 1)	50 / Attr. 8
mac-group-addresses	variable 4		50 / Attr. 9
repeater	variable 5		50 / Attr. 10
repeater-status	–		50 / Attr. 11
search-initiator time-out	–	S-FSK MAC synchronization timeouts (class_id = 52, version = 0)	52 / Attr. 2
synchronization-confirmation-time-out	variable 6		52 / Attr. 3
time-out-not-addressed	variable 7		52 / Attr. 4
time-out-frame-not-OK	variable 8		52 / Attr. 5
min-delta-credit	variable 9	S-FSK Phy&MAC set-up (class_id = 50, version = 1)	50 / Attr. 12
initiator-mac-address	IEC 61334-5-1:2001, 4.3.7.6		50 / Attr. 13
synchronization-locked	variable 10		50 / Attr. 14
IEC 61334-4-32 LLC layer management			
max-frame-length	IEC 61334-4-32:1996,5.1.4	IEC 61334-4-32 LLC setup (class_id = 55, version = 1)	55 / Attr. 2
reply-status-list	variable 11		55 / Attr. 3
broadcast-list	variable 12	–	–
L-SAP-list	variable 13	NOTE In DLMS/COSEM, L-SAPs of logical devices are held by a SAP Assignment object	

© Copyright 1997-2020 DLMS User Association

ACSE management			
application-context-list	variable 14	NOTE In DLMS/COSEM the Association objects play a similar role.	
Application management			
active-initiator	variable 15	S-FSK Active initiator (class_id = 51, version = 0)	51 / Attr. 2
MIB system objects			
reporting-system-list	variable 16	S-FSK Reporting system list (class_id = 56, version = 0)	56 / Attr. 2
Other MIB objects			
reset-NEW-not-synchronized	variable 17	S-FSK Active initiator (class_id = 51, version = 0)	51 / Method 1
new-synchronization	IEC 61334-5-1:2001, 4.3.7.6	–	
initiator-electrical-phase	variable 18		50 / Attr. 2
broadcast-frames-counter	variable 19	S-FSK MAC counters (class_id = 53, version = 0)	53 / Attr. 4
repetitions-counter	variable 20		53 / Attr. 5
transmissions-counter	variable 21		53 / Attr. 6
CRC-OK-frames-counter	variable 22		53 / Attr. 7
CRC-NOK-frames-counter	–		53 / Attr. 8
synchronization-register	variable 23		53 / Attr. 2
desynchronization-listing	variable 24		53 / Attr. 3

© Copyright 1997-2020 DLMS User Association

4.10.3 S-FSK Phy&MAC set-up (class_id = 50, version = 1)

NOTE 1 The use of version 0 of this interface class is deprecated.

An instance of the “S-FSK Phy&MAC set-up” class stores the data necessary to set up and manage the physical and the MAC layer of the PLC S-FSK lower layer profile.

S-FSK Phy&MAC setup		**0...n**	**class_id = 50, version = 1**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. initiator_electrical_phase	(static)	enum	0	3		x + 0x08
3. delta_electrical_phase	(dyn.)	enum	0	6		x + 0x10
4. max_receiving_gain	(static)	unsigned				x + 0x18
5. max_transmitting_gain	(static)	unsigned				x + 0x20
6. search_initiator_threshold	(static)	unsigned			98	x + 0x28
7. frequencies	(static)	frequencies_type				x + 0x30
8. mac_address	(dyn.)	long-unsigned			FFE	x + 0x38
9. mac_group_addresses	(static)	array				x + 0x40
10. repeater	(static)	enum				x + 0x48
11. repeater_status	(dyn.)	boolean				x + 0x50
12. min_delta_credit	(dyn.)	unsigned				x + 0x58
13. initiator_mac_address	(dyn.)	long-unsigned				x + 0x60
14. synchronization_locked	(dyn.)	boolean				x + 0x68
15. transmission_speed	(static)	enum	0	6	3	x + 0x70
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “S-FSK Phy&MAC setup” object instance. See 6.2.25.
initiator_ electrical_phase	Holds the MIB variable *initiator-electrical-phase* (variable 18) specified in IEC 61334-4-512:2001, 5.8. It is written by the client system to indicate the phase to which it is connected. enum: (0) Not defined (default), (1) Phase 1, (2) Phase 2, (3) Phase 3 NOTE 2 This enumeration is different from that of IEC 61334-4-512.
delta_ electrical_phase	Holds the MIB variable *delta-electrical-phase* (variable 1) specified in IEC 61334-4-512:2001, 5.2 and IEC 61334-5-1:2001, 3.5.5.3. It indicates the phase difference between the client's connecting phase and the server's connecting phase. The following values are predefined: enum: (0) Not defined: the server is temporarily not able to determine the phase difference, (1) The server system is connected to the same phase as the client system. The phase difference between the server's connecting phase and the client's connecting phase is equal to:

© Copyright 1997-2020 DLMS User Association

	(2) 60 degrees, (3) 120 degrees, (4) 180 degrees, (5) -120 degrees, (6) -60 degrees
max_ receiving_gain	Holds the MIB variable *max-receiving-gain* (variable 2) specified in IEC 61334-4-512:2001, 5.2 and IEC 61334-5-1:2001, 3.5.5.3. Corresponds to the maximum allowed gain bound to be used by the server system in the receiving mode. The default unit is dB. NOTE 3 In IEC 61334-4-512:2001, no units are specified. The possible values of the gain may depend on the hardware. Therefore, after writing a value to this attribute, the value should be read back to know the actual value.
max_ transmitting_gain	Holds the value of the *max-transmitting-gain*. Corresponds to the maximum attenuation bound to be used by the server system in the transmitting mode. The default unit is dB. The possible values of the gain may depend on the hardware. Therefore, after writing a value to this attribute, the value should be read back to know the actual value.
search_initiator_ threshold	This attribute is used in the intelligent search initiator process. If the value of the initiator signal is above the value of this attribute, a fast synchronization process is possible. The default value is 98 dBµV.

frequencies	Contains frequencies required for S-FSK modulation.
	frequencies_type ::= structure
	{
	mark_frequency: double-long-unsigned,
	space_frequency: double-long-unsigned
	}
	The default unit is Hz.

mac_address	Holds the MIB variable *mac-address* (variable 3) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. NOTE 4 MAC addresses are expressed on 12 bits. Contains the value of the address of the physical attachment (MAC address) associated to the local system. In the unconfigured state, the MAC address is “NEW-address”. This attribute is locally written by the CIASE when the system is registered (with a Register service). The value is used in each outgoing or incoming frame. The default value is "NEW-address". This attribute is set to NEW: - by the MAC sub-layer, once the time-out-not-addressed delay is exceeded; - when a client system “resets” the server system. See the 4.10.4. When this attribute is set to NEW: - the system loses its synchronization (function of the MAC-sublayer); - the *mac_group_address* attribute is reset (array of 0 elements); - the system automatically releases all AAs which can be released. NOTE 5 The second item is not present in IEC 61334-4-512:2001. The predefined MAC addresses are shown in Table 32.

© Copyright 1997-2020 DLMS User Association

mac_group_ addresses	Holds the MIB variable *mac-group-address* (variable 4) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. Contains a set of MAC group addresses used for broadcast purposes. array mac-address mac-address ::= long-unsigned The ALL-configured-address, ALL-physical-address and NO-BODY addresses are not included in this list. These ones are internal predefined values. This attribute shall be written by the initiator using DLMS services to declare specific MAC group addresses on a server system. This attribute is locally read by the MAC sublayer when checking the destination address field of a MAC frame not recognized as an individual address or as one of the three predefined values (ALL-configured-address, ALL-physical-address and NO-BODY).
repeater	Holds the MIB variable *repeater* (variable 5) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. It specifies whether the server system effectively repeats all frames or not. enum: (0) never repeater, (1) always repeater, (2) dynamic repeater If the *repeater* variable is equal to 0, the server system should never repeat the frames. If it is set to 1, the server system is a repeater: it has to repeat all frames received without error and with a current credit greater than zero. If it is set to 2, then the repeater status can be dynamically changed by the server itself. NOTE 6 The value 2 value is not specified in IEC 61334-4-512. This attribute is internally read by the MAC sub-layer each time a frame is received. The default value shall be specified in project specific companion specifications.
repeater_status	Holds the current *repeater status* of the device. boolean: (0) FALSE = no repeater, (1) TRUE = repeater
min_delta_credit	Holds the MIB variable *min-delta-credit* (variable 9) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. NOTE 7 Only the three least significant bits are used. The Delta Credit (DC) is the subtraction of the Initial Credit (IC) and Current Credit (CC) fields of a correct received MAC frame. The delta-credit minimum value of a correct received MAC frame, directed to a server system, is held by this variable. The default value is set to the maximal initial credit (see IEC 61334-5-1:2001, 4.2.3.1 for further explanations on the credit and the value of MAX_INITIAL_CREDIT). A client system can reinitialise this variable by setting its value to the maximal initial credit.

© Copyright 1997-2020 DLMS User Association

initiator_mac_ address	Holds the MIB variable *initiator-mac-address* specified in IEC 61334-5-1:2001, 4.3.7.6. Its value is either the MAC address of the active-initiator or the NO-BODY address, depending on the value of the *synchronization_locked* attribute (see below). See also IEC 61334-5-1:2001, 3.5.3, 4.1.6.3 and 4.1.7.2.
synchronization_ locked	Holds the MIB variable *synchronization-locked* (variable 10) specified in IEC 61334-4-512:2001, 5.3. Controls the synchronization locked / unlocked state. See IEC 61334-5-1:2001 for more details. If the value of this attribute is equal to TRUE, the system is in the synchronization-locked state. In this state, the *initiator-mac-address* is always equal to the MAC address field of the active-initiator MIB object. See attribute 2 of the S-FSK Active initiator IC in 4.10.4. If the value of this attribute is equal to FALSE, the system is in the synchronization-unlocked state. In this state, the *initiator_mac_address* attribute is always set to the NO-BODY value: a value change in the MAC address field of the active-initiator MIB object does not affect the content of the *initiator_mac_address* attribute which remains at the NO-BODY value. The default value of this variable shall be specified in the implementation specifications. NOTE 8 In the synchronization-unlocked state, the server synchronizes on any valid frame. In the synchronization locked state, the server only synchronizes on frames issued or directed to the client system the MAC address of which is equal to the value of the *initiator_mac_address* attribute.
transmission_ speed	The transmission speed supported by the physical device. See also IEC 61334-5-1:2001, 3.2.2. enum: 50Hz 60 Hz (0) 300 baud 360 baud (1) 600 baud 720 baud (2) 1 200 baud 1 440 baud (3) -- default 2 400 baud 2 880 baud (4) 4 800 baud 5 760 baud (5) 7 200 baud 8 640 baud (6) 9 600 baud 11 520 baud

Table 32 – MAC addresses in the S-FSK profile

Address	Value
NO-BODY	000
Local MAC	001…FIMA-1
Initiator	FIMA…LIMA
MAC group address	LIMA + 1…FFB
All configured	FFC
NEW	FFE
All Physical	FFF
NOTE MAC addresses are expressed on 12 bits. These addresses are specified in IEC 61334-5-1:2001, 4.2.3.2, 4.3.7.5.1, 4.3.7.5.2 and 4.3.7.5.3. FIMA = First Initiator MAC address; C00 LIMA = Last Initiator MAC address; DFF	

© Copyright 1997-2020 DLMS User Association

4.10.4 S-FSK Active initiator (class_id = 51, version = 0)

An instance of the “S-FSK Active initiator” IC stores the data of the active initiator. The active initiator is the client system, which has last registered the server system with a CIASE Register request. See IEC 61334-4-511:2000, 7.2.

S-FSK Active initiator	**0...n**	**class_id = 51, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. active_initiator (dyn.)	initiator_descriptor				x + 0x08
Specific methods	***m/o***				
1. reset_NEW_not_synchronized (data)					x + 0x10

Attribute description	
logical_name	Identifies the “S-FSK Active initiator” object instance. See 6.2.25.

active_initiator	Holds the MIB variable *active-initiator* (variable 15) specified in IEC 61334-4-512:2001, 5.6.
	Contains the identifiers of the active initiator, which has last registered the system with a Register request. See IEC 61334-4-511:2000, 7.2.
	The Initiator system is identified with its System Title, MAC address and L-SAP selector:
	initiator_descriptor ::= structure
	{
	system_title: octet-string,
	MAC_address: long-unsigned,
	L_SAP_selector: unsigned
	}
	The size and the structure of the system title may be specified in system specifications. When the system title is used as part of the initialisation vector of cryptographic algorithms, then the size shall meet the requirements applicable for the initialisation vector.
	The MAC_address element is used to update the *initiator-mac-address* MAC management variable when the system is configured in the synchronization-locked state. See the specification of the *initiator_mac_address* and the *synchronization_locked* attributes of the S-FSK Phy&MAC setup IC in 4.10.3.
	As long as the server is not registered by an active initiator, the L_SAP_selector field is set to 0 and the system_title field is equal to an octet string of 0s.
	The default value of the initiator-descriptor is: system_title = octet-string of 0s, MAC_address = NO-BODY and L_SAP_selector = 0.
	The value of this attribute can be updated by the invocation of the *reset_NEW_not_synchronized* method or by the CIASE Register service.

© Copyright 1997-2020 DLMS User Association

Method description	
reset_NEW_not_ synchronized (data)	Holds the MIB variable *reset-NEW-not-synchronized* (variable 17) specified in IEC 61334-4-512:2001, 5.8. Allows a client system to "reset" the server system. The submitted value corresponds to a client MAC address. The writing is refused if: - the value does not correspond to a valid client MAC address or the predefined NO-BODY address; - the submitted value is different from the NO-BODY address and the *synchronization_locked* attribute is not equal to TRUE. For the description of the Intelligent Search Initiator process, see DLMS UA 1000-2 Ed. 10, 10.4.5.7. When this method is invoked, the following actions are performed: - the system returns to the unconfigured state (UNC: MAC-address equals NEW-address). This transition automatically causes the synchronization lost (function of the MAC sub layer); - the system changes the value of the *active_initiator* attribute: the MAC_address is set to the submitted value, the L-SAP_selector is set to the value 0 and the system_title is set to an octet-string of 0s; - all AAs that can be released are released.

4.10.5 S-FSK MAC synchronization timeouts (class_id = 52, version = 0)

An instance of the "S-FSK MAC synchronization timeouts" IC stores the timeouts related to the synchronization process.

S-FSK MAC synchronization timeouts		**0...n**	**class_id = 52, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. search_initiator_timeout	(static)	long-unsigned				x + 0x08
3. synchronization_confirmation_ timeout	(static)	long-unsigned				x + 0x10
4. time_out_not_addressed	(static)	long-unsigned				x + 0x18
5. time_out_frame_not_OK	(static)	long-unsigned				x + 0x20
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the "S-FSK synchronization timeouts" object instance. See 6.2.25.
search_initiator_ timeout	This timeout supports the intelligent search initiator function. It defines the value of the time, expressed in seconds, during which the server system is searching for the initiator with the strongest signal. During this timeout, all initiators, which may be heard by the servers, are expected to talk. After the expiry of this timeout, the server will accept a Register request from the initiator having provided the strongest signal and it will be locked to that initiator. If the value of the timeout is equal to 0, this means that the feature is not used.

© Copyright 1997-2020 DLMS User Association

search_initiator_ timeout (continued)	The timeout is started at the beginning of the Search Initiator Phase, when the server receives the first frame with a valid initiator MAC address. The timeout is restarted when the Search Initiator Phase is over and the server locks on the initiator. During the Check Initiator Phase, it is restarted on the reception of each valid frame. A Fast synchronization may be performed if the level of signal is good enough (Level of initiator signal >= Search-Initiator-Threshold) and one of the MAC addresses (Source or Destination) is an Initiator MAC address. This means that the module (the meter) is next to a DC or next to a module that is already locked on that DC. The module locks in this case on that initiator.
synchronization_ confirmation_ timeout	Holds the MIB variable *synchronization-confirmation-timeout* (variable 6*)* specified in IEC 61334-4-512:2001, 5.3 and IEC 61334-5-1:2001, 4.3.7.6. Defines the value of the time, expressed in seconds, after which a server system which just gets frame synchronized (detection of a data path equal to AAAA54C7 hex) will automatically lose its frame synchronization if the MAC sublayer does not identify a valid MAC frame. The timeout starts after the reception of the first four bytes of a physical frame. The value of this variable can be modified by a client system. This time-out ensures a fast desynchronization of a system, which has synchronized on a wrong physical frame. See IEC 61334-5-1:2001, 3.5.3 for more details. The default value of this variable should be specified in the implementation specifications. A value equal to 0 is equivalent to cancel the use of the related *synchronization_confirmation_timeout* counter.
time_out_not_ addressed	Holds the MIB variable *time-out-not-addressed* (variable 7) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. Defines the time, in minutes, after which a server system that has not been individually addressed: - returns to the non configured state (UNC: MAC-address equals NEW-address): this transition automatically involves the loss of the synchronization (function of the MAC sub layer) and releasing all AAs that can be released; - loses its active initiator: the MAC address of the active-initiator is set to NO-BODY, the LSAP selector is set to the value 00 and the System Title is set to an octet-string of 0s. Because broadcast addresses are not individual system addresses, the timer associated with the *time-out-not-addressed* delay ensures that a forgotten system will sooner or later return to the unconfigured state. It will be then discovered again. A forgotten system is a system, which has not been individually addressed for more than the "*time-out-not-addressed*" amount of time. The default value of this variable should be specified in the implementation specifications. A value equal to 0 is equivalent to cancel the use of the related time-out-not-addressed counter.

© Copyright 1997-2020 DLMS User Association

time_out_frame_ not_OK	Holds the MIB variable *time-out-frame-not-OK* (variable 8), specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. Defines the time, in seconds, after which a server system that has not received a properly formed MAC frame (incorrect NS field, inconsistent number of received sub frames, false Cyclic Redundancy Code checking) loses its frame synchronization. The default value of this variable shall be specified in the implementation specifications. A value equal to 0 is equivalent to cancel the use of the related *time-out-frame-not-OK* counter.

4.10.6 S-FSK MAC counters (class_id = 53, version = 0)

An instance of the “S-FSK MAC counters” IC stores counters related to the frame exchange, transmission and repetition phases.

S-FSK MAC counters		**0…n**	**class_id = 53, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. synchronization_register	(dyn.)	array				x + 0x08
3. desynchronization_listing	(dyn.)	structure				x + 0x10
4. broadcast_frames_counter	(dyn.)	array				x + 0x18
5. repetitions_counter	(dyn.)	double-long-unsigned			0	x + 0x20
6. transmissions_counter	(dyn.)	double-long-unsigned			0	x + 0x28
7. CRC_OK_frames_counter	(dyn.)	double-long-unsigned			0	x + 0x30
8. CRC_NOK_frames_counter	(dyn.)	double-long-unsigned				x + 0x38
Specific methods		***m/o***				
1. reset (data)						x + 0x50

Attribute description	
logical_name	Identifies the “S-FSK MAC counters” object instance. See 6.2.25.

synchronization_ register	Holds the MIB variable *synchronization-register* (variable 23), specified in IEC 61334-4-512:2001, 5.8.
	array synchronization_couples
	
	synchronization_couples ::= structure
	{
	mac_address: long-unsigned,
	synchronizations_counter: double-long-unsigned
	}
	This variable counts the number of synchronization processes performed by the system. Processes that lead to a synchronization loss due to the detection of a wrong initiator are registered. The other processes that lead to a synchronization loss (time-out, management writing) **are not registered.**

© Copyright 1997-2020 DLMS User Association

synchronization_ register (continued)

This variable provides a balance sheet of the different systems on which the server system is "potentially" able to synchronize.

A synchronization process is initialized when the Management Application Entity (connection manager) receives a MA_Sync.indication (Synchronization State = SYNCHRO_FOUND) primitive from the MAC Sublayer Entity. This process is registered in the synchronization-register variable only if the MA_Sync.indication (Synchronization State = SYNCHRO_FOUND) primitive is followed by one of the three primitives:

1) MA_Data.indication (DA, SA, MSDU) primitive;
2) MA_Sync.indication (Synchronization State = SYNCHRO_CONF, SA, DA);
3) MA_Sync.indication (Synchronization State = SYNCHRO_LOSS, Synchro Loss Cause = wrong_initiator, SA, DA.

NOTE The third primitive is only generated if the server system is configured in a synchronization-locked state. See 4.10.3.

Processes which lead to the generation of MA_Sync.indication (Synchronization State = SYNCHRO_LOSS) primitives indicating synchronization loss due to:

- the physical layer;
- the time-out-not-addressed counter;
- setting the mac_address attribute of the S-FSK Phy&MAC setup object to NEW; see 4.10.3; or
- invoking the reset_NEW_not_synchronized method of the S-FSK Active initiator object; see 4.10.4 (this is known as Management Writing)

are not taken into account in this variable.

For details on the MA_Sync.indication service primitive, see IEC 61334-5-1:2001, 4.1.7.1.

If the synchronization process ends with one of the three primitives listed above, the synchronization-register variable is updated by taking into account the SA and DA fields of the primitive.

The updating of the *synchronization-register* variable is carried out as follows:

First, the Management Entity checks the SA and DA fields.

- If one of these fields corresponds to a client MAC address (CMA) the Entity:
  - checks if the client MAC address (CMA) appears in one of the couples contained in the synchronization-register variable;
  - if it appears, the related synchronizations-counter subfield is incremented;
  - if it does not appear, a new (mac-address, synchronizations-counter) couple is added. This couple is initialized to the (CMA, 1) value.
- If none of the SA and DA fields correspond to a client MAC address, it is supposed that the system found its synchronization reference on a DiscoverReport type frame. In that case, the mac-address which should be registered in the synchronization-register variable is the predefined NEW value (FFE). The updating of the synchronization-register variable is carried out in the same way as it is done for a normal client MAC address (CMA).

© Copyright 1997-2020 DLMS User Association

synchronization_ register (continued)	When a *synchronizations-counter* field reaches the maximum value, it automatically returns to 0 on the next increment. The maximum number of synchronization couples {mac-address, synchronizations-counter} contained in this variable should be specified in the implementation specifications. When this maximum is reached, the updating of the variable follows a First-In-First-Out (FIFO) mechanism: only the newest source MAC addresses are memorized. The default value of this variable is an empty array.

desynchronization_ listing	Holds the MIB variable *desynchronization-listing* (variable 24), specified in IEC 61334-4-512:2001, 5.8.
	structure
	{
	nb_physical_layer_desynchronization: double-long-unsigned,
	nb_time_out_not_addressed_desynchronization: double-long-unsigned,
	nb_timeout_frame_not_OK desynchronization: double-long-unsigned,
	nb_write_request_desynchronization: double-long-unsigned,
	nb_wrong_initiator_desynchronization: double-long-unsigned
	}
	This variable counts the number of desynchronizations that occurred depending on their cause. On reception of synchronization loss notification, the Management Entity updates this attribute by incrementing the counter related to the cause of the desynchronization.
	When one of the counters reaches the maximum value, it automatically returns to 0 on the next increment.
	The default value of this variable is a structure with all elements equal to 0.

broadcast_frames_ counter	Holds the MIB variable *broadcast-frames-counter* (variable 19) specified in IEC 61334-4-512:2001, 5.8.
	array broadcast-couples
	
	broadcast-couples ::= structure
	{
	source-mac-address: long-unsigned,
	frames-counter: double-long-unsigned
	}
	It counts the broadcast frames received by the server system and issued from a client system (source-mac-address = any valid client-mac-address, destination-mac address = ALL-physical). The number of frames is classified according to the origin of the transmitter. The counter is incremented even if the LLC-destination-address is not valid on the server system. When the frames-counter field reaches its maximum value, it automatically returns to 0 on the next increment.
	The maximum number of broadcast-couples {source-mac-address, frames-counter} contained in this variable should be specified in the implementation specifications. When this maximum is reached, the updating of the variable follows a First-In-First-Out (FIFO) mechanism: only the newest source MAC addresses are memorized.

© Copyright 1997-2020 DLMS User Association

repetitions_ counter	Holds the MIB variable *repetitions-counter* (variable 20) specified in IEC 61334-4-512:2001, 5.8. Counts the number of repetition phases. The repetition phases following a transmission are not counted. If the MAC sub-layer is configured in the no-repeater mode, this variable is not updated. The repetitions counter measures the activity of the system as a repeater. A received frame repeated five times (from CC=4 to CC=0) is counted only once in the repetitions counter since it corresponds to one repetition phase. The counter is incremented at the beginning of each repetition phase. When the repetitions counter reaches the maximum value, it automatically returns to 0 on the next increment. The default value is 0.
transmissions_ counter	Holds the MIB variable *transmissions-counter* (variable 21) specified in IEC 61334-4-512:2001, 5.8. Counts the number of transmission phases. A transmission phase is characterized by the transmission and the repetition of a frame. A repetition phase, which follows the reception of a frame, is not counted. The transmission counter is incremented at the beginning of each transmission phase. A client system can write this variable to update the counter. When the transmissions counter reaches the maximum value, it automatically returns to 0 on the next increment. The default value is 0.
CRC_OK_frames_ counter	Holds the MIB variable *CRC-OK-frames-counter* (variable 22) specified in IEC 61334-4-512:2001, 5.8 Counts the number of frames received with a correct Frame Check Sequence Field. When the CRC OK frames counter field reaches the maximum value, it automatically returns to 0 on the next increment. The default value is 0.
CRC_NOK_frames_ counter	Counts the number of frames received with an incorrect Frame Check Sequence Field. When the CRC NOK frames counter field reaches the maximum value, it automatically returns to 0 on the next increment. The default value is 0.
Method description	
reset (data)	Clears all counters. data ::= integer (0)

4.10.7 IEC 61334-4-32 LLC setup (class_id = 55, version = 1)

An instance of the “IEC 61334-4-32 LLC setup” IC holds parameters necessary to set up and manage the LLC layer as specified in IEC 61334-4-32.

IEC 61334-4-32 LLC setup		**0...n**	**class_id = 55, version = 1**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. max_frame_length	(static)	long-unsigned				x + 0x08
3. reply_status_list	(dyn.)	array				x + 0x10
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “IEC 61334-4-32 LLC setup” object instance. See 6.2.25.

© Copyright 1997-2020 DLMS User Association

max_frame_ length	Holds the length of the LLC frame in bytes. See IEC 61334-4-32:1996, 5.1.4. For the S-FSK PLC profile, the minimum / default / maximum values are 26 / 134 / 242 bytes respectively. See IEC 61334-5-1:2001, 4.2.2. NOTE For other lower layer profiles, see the corresponding values in the relevant specification.

reply_status_list	Holds the MIB variable *reply-status-list* (variable 11) specified in IEC 61334-4-512:2001, 5.4.
	Lists the L-SAPs that have a not empty RDR (Reply Data on Request) buffer, which has not already been read. The length of a waiting L-SDU is specified in number of sub frames (different from zero). The variable is locally generated by the LLC sub layer.
	array reply_status
	
	reply_status ::= structure
	{
	L_SAP_selector: unsigned,
	length_of_waiting_L_SDU: unsigned
	}
	
	length_of_waiting_L_SDU in the case of the S-FSK profile is in number of sub-frames; valid values are 1 to 7.

4.10.8 S-FSK Reporting system list (class_id = 56, version = 0)

An instance of the “S-FSK Reporting system list” IC holds the list of reporting systems.

S-FSK Reporting system list		**0...n**	**class_id = 56, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. reporting_system_list	(dyn.)	array				x + 0x08
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “S-FSK Reporting system list” object instance. See 6.2.25.
reporting_ system_list	Holds the MIB variable *reporting-system-list* (variable 16) specified in IEC 61334-4-512:2001, 5.7. array system-title system-title ::= octet-string Contains the system-titles of the server systems which have made a DiscoverReport request and which have not already been registered. The list has a finite size and it is sorted upon the arrival. The first element is the newest one. Once full, the oldest ones are replaced by the new ones.

© Copyright 1997-2020 DLMS User Association

reporting_ system_list (continued)	The reporting system list is updated: - when a DiscoverReport CI_PDU is received by the server system (whatever its state: non configured or configured): the CIASE adds the reporting system-title at the beginning of the list, and verifies that it does not exist anywhere else in the list, if so it destroys the old one. A system-title can only be present once in the list; - when a Register CI_PDU is received by the server system (whatever its state: non configured or configured): the CIASE checks the reporting-system list. If a system-title is present in the reporting-system-list and in the Register CI-PDU, the CIASE deletes the system-title in the reporting-system-list: this system is no more considered as a reporting system.

4.11 Interface classes for setting up the LLC layer for ISO/IEC 8802-2

4.11.1 General

This subclause specifies the ICs available for setting up the ISO/IEC 8802-2 LLC layer, used in some DLMS/COSEM communication profiles, in the various types of operation.

For definitions related to the ISO/IEEE 8802-2 LLC layer see ISO/IEC 8802-2:1998, 1.4.2.

4.11.2 ISO/IEC 8802-2 LLC Type 1 setup (class_id = 57, version = 0)

An instance of the “ISO/IEC 8802-2 LLC Type 1 setup” IC holds the parameters necessary to set up the ISO/IEC 8802-2 LLC layer in Type 1 operation.

ISO/IEC 8802-2 LLC Type 1 setup		**0...n**	**class_id = 57, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. max_octets_ui_pdu	(static)	long unsigned			128	x + 0x08
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “ISO/IEC 8802-2 LLC Type 1 setup” object instance. See 6.2.26.
max_octets_ ui_pdu	Refer to the appropriate MAC protocol specification for any limitation on the maximum number of octets in a UI PDU. No restrictions are imposed by the LLC sublayer. However, in the interest of having a value that all users of Type 1 LLC may depend upon, all MACs shall at least be capable of accommodating UI PDUs with information fields up to and including 128 octets in length. See ISO/IEC 8802-2:1998, 6.8.1 *Maximum number of octets in a UI PDU.*

4.11.3 ISO/IEC 8802-2 LLC Type 2 setup (class_id = 58, version = 0)

An instance of the “ISO/IEC 8802-2 LLC Type 2 setup” IC holds the parameters necessary to set up the ISO/IEC 8802-2 LLC layer in Type 2 operation.

© Copyright 1997-2020 DLMS User Association

ISO/IEC 8802-2 LLC Type 2 setup		0...n	class_id = 58, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. transmit_window_size_k	(static)	unsigned	1	127	1	x + 0x08
3. receive_window_size_rw	(static)	unsigned	1	127	1	x + 0x10
4. max_octets_i_pdu_n1	(static)	long unsigned			128	x + 0x18
5. max_number_transmissions_n2	(static)	unsigned				x + 0x20
6. acknowledgement_timer	(static)	long-unsigned				x + 0x28
7. p_bit_timer	(static)	long-unsigned				x + 0x30
8. reject_timer	(static)	long-unsigned				x + 0x38
9. busy_state_timer	(static)	long-unsigned				x + 0x40
Specific methods		***m/o***				

Attribute description

logical_name	Identifies the “ISO/IEC 8802-2 LLC Type 2 setup” object instance. See 6.2.26.
transmit_ window_ size_k	The transmit window size (k) shall be a data link connection parameter that can never exceed 127. It shall denote the maximum number of sequentially numbered I PDUs that the sending LLC may have outstanding (i.e., unacknowledged). The value of k is the maximum number by which the sending LLC send state variable V(S) can exceed the N(R) of the last received I PDU. See ISO/IEC 8802-2:1998, 7.8.4 *Transmit window size, k.*
receive_ window_ size_rw	The receive window size (RW) shall be a data link connection parameter that can never exceed 127. It shall denote the maximum number of unacknowledged sequentially numbered I PDUs that the local LLC allows the remote LLC to have outstanding. It is transmitted in the information field of XID (see ISO/IEC 8802-2:1998, 5.4.1.1.2) and applies to the XID sender. The XID receiver shall set it’s transmit window (k) to a value less than or equal to the receive window of the XID sender to avoid overrunning the XID sender. See ISO/IEC 8802-2:1998, 7.8.6 *Receive window size, RW.*
max_octets_i _pdu_n1	N1 is a data link connection parameter that denotes the maximum number of octets in an I PDU. Refer to the various MAC descriptions to determine the precise value of N1 for a given medium access method. LLC itself places no restrictions on the value of N1. However, in the interest of having a value of N1 that all users of Type 2 LLC may depend upon, all MACs shall at least be capable of accommodating I PDUs with information fields up to an including 128 octets in length. See ISO/IEC 8802-2:1998, 7.8.3 *Maximum number of octets in an I PDU, N1.*
max_number_ transmissions_n2	N2 is a data link connection parameter that indicates the maximum number of times that a PDU is sent following the running out of the acknowledgment timer, the P-bit timer, the reject timer, or the busy-state timer. See ISO/IEC 8802-2:1998, 7.8.2 *Maximum number of transmissions, N2.*
acknowledgement_ timer	The acknowledgment timer is a data link connection parameter that shall define the time interval during which the LLC shall expect to receive an acknowledgment to one or more outstanding I PDUs or an expected response PDU to a sent unnumbered command PDU. The unit is seconds.

© Copyright 1997-2020 DLMS User Association

	See ISO/IEC 8802-2:1998, 7.8.1.1 *Acknowledgement timer.*
p_bit_timer	The P-bit timer is a data link connection parameter that shall define the time interval during which the LLC shall expect to receive a PDU with the F bit set to “1” in response to a sent Type 2 command with the P bit set to “1”. The unit is seconds. See ISO/IEC 8802-2:1998, 7.8.1.2 *P-bit timer.*
reject_timer	The reject timer is a data link connection parameter that shall define the time interval during which the LLC shall expect to receive a reply to a sent REJ PDU. The unit is seconds. See ISO/IEC 8802-2:1998, 7.8.1.3 *Reject timer.*
busy_state_timer	The busy-state timer is a data link connection parameter that shall define the timer interval during which the LLC shall wait for an indication of the clearance of a busy condition at the other LLC. The unit is seconds. See ISO/IEC 8802-2:1998, 7.8.1.4 *Busy-state timer.*

4.11.4 ISO/IEC 8802-2 LLC Type 3 setup (class_id = 59, version = 0)

An instance of the “ISO/IEC 8802-2 LLC Type 3 setup” IC holds the parameters necessary to set up the ISO/IEC 8802-2 LLC layer in Type 3 operation.

ISO/IEC 8802-2 LLC Type 3 setup		**0...n**	**class_id = 59, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				X
2. max_octets_acn_pdu_n3	(static)	long unsigned				x + 0x08
3. max_number_transmissions_n4	(static)	unsigned				x + 0x10
4. acknowledgement_time_t1	(static)	long unsigned				x + 0x18
5. receive_lifetime_var_t2	(static)	long unsigned				x + 0x20
6. transmit_lifetime_var_t3	(static)	long unsigned				x + 0x28
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “ISO/IEC 8802-2 LLC Type 3 setup” object instance. See 6.2.26.
max_octets_ acn_pdu_n3	N3 is a logical link parameter that denotes the maximum number of octets in an ACn command PDU. Refer to the various MAC descriptions to determine the precise value of N3 for a given medium access method. LLC places no restrictions on the value of N3. See ISO/IEC 8802-2:1998, 8.6.2 *Maximum number of octets in an ACn command PDU, N3.*
max_number_ transmissions_n4	N4 is a logical link parameter that indicates the maximum number of times that an ACn command PDU is sent by LLC trying to accomplish a successful information exchange. Normally, N4 is set large enough to overcome the loss of a PDU due to link error conditions. If the medium access control sublayer has its own retransmission capability, the value of N4 may be set to one so

© Copyright 1997-2020 DLMS User Association

	that LLC does not itself requeue a PDU to the medium access control sublayer. See ISO/IEC 8802-2:1998, 8.6.1 *Maximum number of transmissions, N4.*
acknowledgement_time_t1	The acknowledgment time is a logical link parameter that determines the period of the acknowledgment timers, and as such shall define the time interval during which the LLC shall expect to receive an ACn response PDU from a specific LLC from which the LLC is awaiting a response PDU. The acknowledgment time shall take into account any delay introduced by the MAC sublayer and whether the timer is started at the beginning or at the end of the sending of the ACn command PDU by the LLC. The proper operation of the procedure shall require that the acknowledgment time be greater than the normal time between the sending of an ACn command PDU and the reception of the corresponding ACn response PDU. If the medium access control sublayer performs its own retransmissions and if the logical link parameter N4 is set to one to prevent LLC from re-queuing a PDU, then the acknowledgment time T1 may be set to infinity, making the acknowledgment timers unnecessary. The unit is seconds. Infinity is indicated by all bits set to 1. See ISO/IEC 8802-2:1998, 8.6.4, *Acknowledgement time, T1.*

© Copyright 1997-2020 DLMS User Association

receive_lifetime_var_t2	This time value is a logical link parameter that determines the period of all of the receive variable lifetime timers. T2 shall be longer by a margin of safety than the longest possible period during which the first transmission and all retries of a single PDU may occur. The margin of safety shall take into account anything affecting LLCs perception of the arrival time of PDUs, such as LLC response time, timer resolution, and variations in the time required for the medium access control sublayer to pass received PDUs to LLC. If the destruction of the received state variables is not desired, the value of time T2 may be set to infinity. In this case the receive variable lifetime timer need not be implemented. The unit is seconds. Infinity is indicated by all bits set to 1. See ISO/IEC 8802-2:1998, 8.6.5 *Receive lifetime variable, T2.*
transmit_lifetime_var_t3	This time value is a logical link parameter that determines the minimum lifetime of the transmit sequence state variables. T3 shall be longer by a margin of safety than 1) the logical link variable T2 at stations to which ACn commands are sent; and 2) the longest possible lifetime of an ACn command-response pair. The lifetime of an ACn command-response pair shall take into account the sum of processing time, queuing delays, and transmission time for the command and response PDUs at the local and remote stations. If the destruction of the transmit state variables is not desired, the value of time T3 may be set to infinity. Note, if the receive variable lifetime parameter, T2 is set to infinity at remote stations to which ACn commands are sent, then the T3 parameter shall be set to infinity at the local station. The unit is seconds. Infinity is indicated by all bits set to 1. See ISO/IEC 8802-2:1998, 8.6.6 *Transmit lifetime variable, T3.*

© Copyright 1997-2020 DLMS User Association

4.12 Interface classes for setting up and managing DLMS/COSEM narrowband OFDM PLC profile for PRIME networks

4.12.1 Overview

See also Annex D.

COSEM objects for data exchange using narrowband OFDM PLC profile for PRIME networks, if implemented, shall be located in the Management Logical Device of COSEM servers.

Figure 29 shows an example with a COSEM physical device comprising three logical devices.

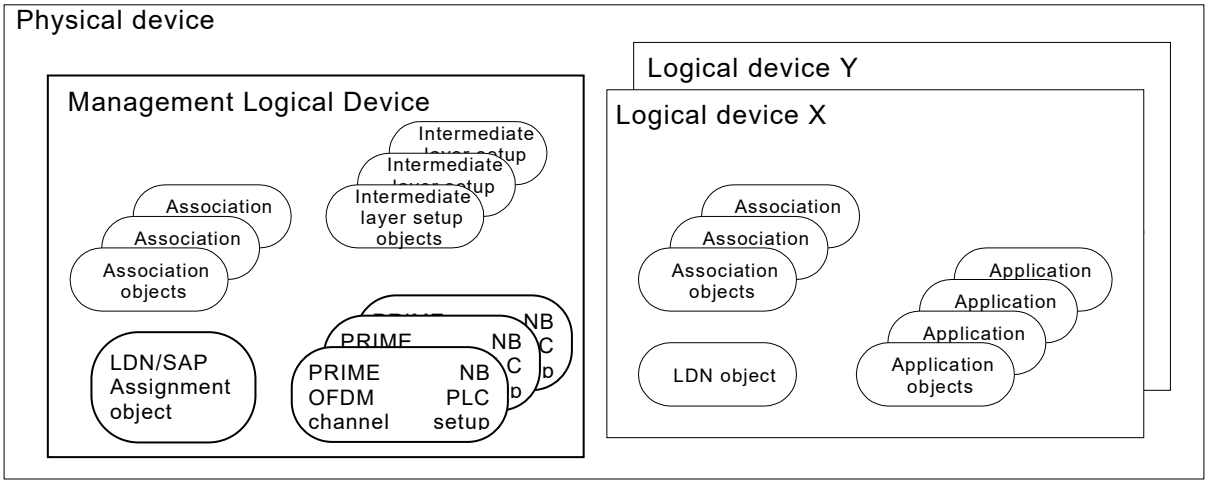


Figure 29 – Object model of DLMS/COSEM servers

Each logical device shall contain a Logical Device Name (LDN) object.

NOTE As in this example there is more than one logical device, the mandatory Management logical device contains a “SAP Assignment” object instead of a Logical Device object.

Each logical device contains one or more “Association” objects, one for each client supported.

The management logical device contains the setup objects of the physical and MAC layers of narrowband OFDM PLC profile for PRIME networks as well as setup objects for the intermediate layer(s). It may contain further application objects.

The other logical devices, in addition to the “Association” and Logical Device Name objects mentioned above, contain further application objects, holding parameters and measurement values.

To set up and manage the 61334-4-32 LLC SSCS, one IC is specified:

- “61334-4-32 LLC SSCS setup”, see 4.12.3.

To manage the PRIME NB OFDM PLC physical layer (PhL), one IC is specified:

- “PRIME NB OFDM PLC Physical layer counters”, see 4.12.5;

To set up and manage the PLC PRIME OFDM MAC layer, four ICs are specified:

- “PRIME NB OFDM PLC MAC setup”: see 4.12.6;
- “PRIME NB OFDM PLC MAC functional parameters”: see 4.12.7;
- “PRIME NB OFDM PLC MAC counters”: see 4.12.8;
- “PRIME NB OFDM PLC MAC network administration data”: see 4.12.9.

For application identification, one IC is specified:

© Copyright 1997-2020 DLMS User Association

- “PRIME NB OFDM PLC Application identification”, see 4.12.11.

4.12.2 Mapping of PRIME NB OFDM PLC PIB attributes to COSEM IC attributes

ITU-T G.9904:2012 defines variables in Table 10-1 and Table 10-2 for PHY PIB attributes, Table 10-3 to Table 10-8 for MAC PIB attributes and Table 10-9 for Applications PIB attributes.

Table 33 shows the mapping of PRIME NB OFDM PLC PIB attributes to attributes of COSEM ICs. Only variables related to the switch and Terminal nodes are mapped. Variables relevant for the base node are not mapped, because the base node acts as a client regarding the distribution network.

Table 33 – Mapping of PRIME NB OFDM PLC PIB attributes to COSEM IC attributes

Name	Identifier	Interface class	class_id / attribute
PHY PIB attributes – PHY read-only variable that provide statistical information [1]			
phyStatsCRCIncorrectCount	0x00A0	PRIME NB OFDM PLC Physical layer counters (class_id = 81, version = 0)	81 / Attr. 2
PhyStatsCRCFailCount	0x00A1		81 / Attr. 3
phyStatsTxDropCount	0x00A2		81 / Attr. 4
phyStatsRxDropCount	0x00A3		81 / Attr. 5
phyStatsRxTotalCount	0x00A4		Not modelled
phyStatsBlkAvgEvm	0x00A5		Not modelled
phyEmaSmoothing	0x00A8		Not modelled
PHY read-only parameters, providing information on specific implementation [2]			
phyTxQueueLen	0x00B0		Not modelled
phyRxQueueLen	0x00B1		
phyTxProcessingDelay	0x00B2		
phyRxProcessingDelay	0x00B3		
PhyAgcMinGain	0x00B4		
PhyAgcStepValue	0x00B5		
PhyAgcStepNumber	0x00B6		
MAC read-write variables, read-only variables [3]			
macMinSwitchSearchTime	0x0010		82 / Attr. 2
macMaxPromotionPdu	0x0011	PRIME NB OFDM PLC MAC setup (class_id = 82, version = 0)	82 / Attr. 3
macMaxPromotionPduTxPeriod	0x0012		82 / Attr. 4
macBeaconsPerFrame	0x0013		82 / Attr. 5
macSCPMaxTxAttempts	0x0014		82 / Attr. 6
macCtlReTxTimer	0x0015		82 / Attr. 7
macMaxCtlReTx	0x0018		82 / Attr. 8
macEMASmoothing	0x0019		Not modelled
macSCPRBO	0x0016		Not modelled
macSCPChSenseCount	0x0017		Not modelled

© Copyright 1997-2020 DLMS User Association

MAC read-only variables that provide functional information [4]			
macLNID	0x0020	PRIME NB OFDM PLC MAC functional parameters (class_id = 83 version = 0)	83 / Attr. 2
macLSID	0x0021		83 / Attr. 3
macSID	0x0022		83 / Attr. 4
macSNA	0x0023		83 / Attr. 5
macState	0x0024		83 / Attr. 6
macSCPLength	0x0025		83 / Attr. 7
macNodeHierarchyLevel	0x0026		83 / Attr. 8
macBeaconSlotCount	0x0027		83 / Attr. 9
macBeaconRxSlot	0x0028		83 / Attr. 10
macBeaconTxSlot	0x0029		83 / Attr. 11
macBeaconRxFrequency	0x002A		83 / Attr. 12
macBeaconTxFrequency	0x002B		83 / Attr. 13
macCapabilities	0x002C		83/ Attr. 14
MAC read-only variable that provide statistical information [5]			
macTxDataPktCount	0x0040	PRIME NB OFDM PLC MAC counters (class_id = 84, version = 0)	84 / Attr. 2
macRxDataPktCount	0x0041		84 / Attr. 3
macTxCtrlPktCount	0x0042		84 / Attr. 4
macRxCtrlPktCount	0x0043		84 / Attr. 5
macCSMAFailCount	0x0044		84 / Attr. 6
macCSMAChBusyCount	0x0045		84 / Attr. 7
Read-only lists, made available by MAC layer through management interface [6]			
macListMcastEntries	0x0052	PRIME NB OFDM PLC MAC network administration data (class_id = 85, version = 0)	85 / Attr. 2
macListSwitchTable	0x0053		85 / Attr. 3
macListDirectTable	0x0055		85 / Attr. 4
macListAvailableSwitches	0x0056		85 / Attr. 5
macListPhyComm	0x0057		85 / Attr. 6
Application PIB attributes [7]			
AppFwVersion	0x0075	PRIME NB OFDM PLC Application identification (class_id = 86, version = 0)	86 / Attr. 2
AppVendorId	0x0076		86 / Attr. 3
AppProductId	0x0077		86 / Attr. 4

[1] See ITU-T G.9904:2012, Table 10-1.
[2] See ITU-T G.9904:2012, Table 10-2.
[3] See ITU-T G.9904:2012, Table 10-3, 10-4.
[4] See ITU-T G.9904:2012, Table 10-5.
[5] See ITU-T G.9904:2012, Table 10-6.
[6] See ITU-T G.9904:2012, Table 10-7.
[7] See ITU-T G.9904:2012, Table 10-9.

NOTE Whereas in COSEM interface class specifications the underscore notation is used, in Recommendation ITU-T G.9904:2012 and in Table 1 – the camel notation is used.

© Copyright 1997-2020 DLMS User Association

4.12.3 61334-4-32 LLC SSCS setup (class_id = 80, version = 0)

An instance of the “61334-4-32 LLC SSCS” (Service Specific Convergence Sublayer, 432 CL) setup IC holds addresses that are provided by the base node during the opening of the convergence layer, as a response to the establish request of the service node. They allow the service node to be part of the network managed by the base node.

61334-4-32 LLC SSCS setup	**0...n**	**class_id = 80, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. service_node_address (dyn.)	long-unsigned				x + 0x08
3. base_node_address (dyn.)	long-unsigned				x + 0x10
Specific methods	***m/o***				
1. reset (data)	o				x + 0x20

Attribute description	
logical_name	Identifies the “61334-4-32 LLC SSCS setup object instance”. See 6.2.26.
service_node_ address	Holds the value of the address assigned to the service node during its registration by the base node. After deregistration, the value of this address is NEW, meaning 0xFFE.
base_node_ address	Holds the value of the base node address to which the service node is registered. After deregistration this address is 0.
Method description	
reset (data)	This method is used for deallocating the service node address. The value of the *service_node_address* becomes NEW and the value of the *base_node_address* becomes 0.

4.12.4 PRIME NB OFDM PLC Physical layer parameters

The physical layer parameters are not modelled.

4.12.5 PRIME NB OFDM PLC Physical layer counters (class_id = 81, version = 0)

An instance of the “PRIME NB OFDM PLC Physical layer counters” IC stores counters related to the physical layers exchanges. The objective of these counters is to provide statistical information for management purposes.

The attributes of instances of this IC shall be read only. They can be reset using the reset method.

© Copyright 1997-2020 DLMS User Association

PRIME NB OFDM PLC Physical layer counters		0...n	class_id = 81, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. phy_stats_crc_incorrect_count	(dyn.)	long-unsigned				x + 0x08
3. phy_stats_crc_fail_count	(dyn.)	long-unsigned				x + 0x10
4. phy_stats_tx_drop_count	(dyn.)	long-unsigned				x + 0x18
5. phy_stats_rx_drop_count	(dyn.)	long-unsigned				x + 0x20
Specific methods		***m/o***				
1. reset (data)		o				

Attribute description	
logical_name	Identifies the “PRIME NB OFDM PLC Physical layer counters” object instance. See 6.2.26.
phy_stats_crc_ incorrect_count	PIB attribute 0x00A0: Number of bursts received on the physical layer for which the CRC was incorrect.
phy_stats_crc_ failed_count	PIB attribute 0x00A1: Number of bursts received on the physical layer for which the CRC was correct, but the *Protocol* field of PHY header had invalid value. This count would reflect number of times corrupt data was received and the CRC calculation failed to detect it.
phy_stats_tx_ drop_count	PIB attribute 0x00A2: Number of times when PHY layer received new data to transmit (PHY_DATA.request) and had to either overwrite on existing data in its transmit queue or drop the data in new request due to full queue.
phy_stats_rx_ drop_count	PIB attribute 0x00A3: Number of times when the PHY layer received new data on the channel and had to either overwrite on existing data in its receive queue or drop the newly received data due to full queue.
	NOTE When a counter reaches the maximum value (0xFFFF), it is automatically rolled-over.
Method description	
reset (data)	This method is used for resetting all the counters held by an instance of this interface.

4.12.6 PRIME NB OFDM PLC MAC setup (class_id = 82, version = 0)

An instance of the “PRIME NB OFDM PLC MAC setup” IC holds the necessary parameters to set up and manage the PRIME NB OFDM PLC MAC layer.

These attributes influence the functional behaviour of an implementation. These attributes may be defined external to the MAC, typically by the management entity and implementations may allow changes to their values during normal running, i.e. even after the device start-up sequence has been executed.

© Copyright 1997-2020 DLMS User Association

PRIME NB OFDM PLC MAC setup		0...n	class_id = 82, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mac_min_switch_search_time	(static)	unsigned	16	32	24	x + 0x08
3. mac_max_promotion_pdu	(static)	unsigned	1	4	2	x + 0x10
4. mac_promotion_pdu_tx_period	(static)	unsigned	2	8	5	x + 0x18
5. mac_beacons_per_frame	(static)	unsigned	1	5	5	x + 0x20
6. mac_scp_max_tx_attempts	(static)	unsigned	2	5	5	x +0x28
7. mac_ctl_re_tx_timer	(static)	unsigned	2	20	15	x + 0x30
8. mac_max_ctl_re_tx	(static)	unsigned	3	5	3	x + 0x38
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “PRIME NB OFDM PLC MAC setup” object instance. See 6.2.26.
mac_min_switch_search_time	PIB attribute 0x0010: Minimum time for which a service node in *Disconnected* status should scan the channel for beacons before it can broadcast PNPDU. This attribute is not maintained in base nodes. The unit of this attribute is seconds.
mac_max_promotion_pdu	PIB attribute 0x0011: Maximum number of PNPDUs that may be transmitted by a service node in a period of *mac_promotion_pdu_tx_period* seconds. This attribute is not maintained in base nodes.
mac_promotion_pdu_tx_period	PIB attribute 0x0012: Time quantum for limiting the number of PNDPUs transmitted from a service node. No more than *mac_max_promotion_pdu* may be transmitted in a period of *mac_promotion_pdu_tx_period*. The unit of this attribute is seconds.
mac_beacons_per_frame	PIB attribute 0x0013: Maximum number of beacon slots that may be provisioned in a frame. This attribute is maintained in base nodes.
mac_scp_max_tx_attempts	PIB attribute 0x0014: Number of times the CSMA algorithm would attempt to transmit requested data when a previous attempt was withheld due to PHY indicating channel busy.
mac_ctl_re_tx_timer	PIB attribute 0x0015: Number of seconds for which a MAC entity waits for acknowledgement of receipt of MAC control packet from its peer entity. On expiry of this time, the MAC entity may retransmit the MAC control packet. The unit of this attribute is seconds.
mac_max_ctl_re_tx	PIB attribute 0x0018: Maximum number of times a MAC entity will try to retransmit an unacknowledged MAC control packet. If the retransmit count reaches this maximum, the MAC entity shall abort further attempts to transmit the MAC control packet.
	NOTE When a counter reaches the maximum value (0xFFFF), it is automatically rolled-over.

© Copyright 1997-2020 DLMS User Association

4.12.7 PRIME NB OFDM PLC MAC functional parameters (class_id = 83 version = 0)

The attributes of an instance of the “PRIME NB OFDM PLC MAC functional parameters” IC belong to the functional behaviour of MAC. They provide information on specific aspects.

The attributes of instances of this IC shall be read only.

PRIME NB OFDM PLC MAC functional parameters		**0…n**	**class_id = 83, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mac_LNID	(static)	long	0	16 383		x + 0x08
3. mac_LSID	(static)	unsigned	0	255		x + 0x10
4. mac_SID	(static)	unsigned	0	255		x + 0x18
5. mac_SNA	(static)	octet-string				x + 0x20
6. mac_state	(static)	enum	0	3		x +0x28
7. mac_scp_length	(static)	long				x + 0x30
8. mac_node_hierarchy_level	(static)	unsigned	0	63		x + 0x38
9. mac_beacon_slot_count	(static)	unsigned	0	7		x + 0x40
10. mac_beacon_rx_slot	(static)	unsigned	0	7		x + 0x48
11. mac_beacon_tx_slot	(static)	unsigned	0	7		x + 0x50
12. mac_beacon_rx_frequency	(static)	unsigned	0	31		x + 0x58
13. mac_beacon_tx_frequency	(static)	unsigned	0	31		x + 0x60
14. mac_capabilities	(static)	long-unsigned				x + 0x68
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “PRIME NB OFDM PLC MAC functional parameters” object instance. See 6.2.26.
mac_LNID	PIB attribute 0x0020: LNID allocated to this node at time of its registration.
mac_LSID	PIB attribute 0x0021: LSID allocated to this node at the time of its promotion. This attribute is not maintained if the node is in a *Terminal* functional state.
mac_SID	PIB attribute 0x0022: SID of the switch node through which this node is connected to the subnetwork. This attribute is not maintained in a base node.
mac_SNA	PIB attribute 0x0023: Subnetwork address to which this node is registered. The base node returns the SNA it is using.

© Copyright 1997-2020 DLMS User Association

mac_state	PIB attribute 0x0024: Present functional state of the node. enum: (0) Disconnected, (1) Terminal, (2) Switch, (3) Base
mac_scp_length	PIB attribute 0x0025: The SCP length, in symbols, in present frame.
mac_node_ hierarchy_level	PIB attribute 0x0026: Level of this node in subnetwork hierarchy.
mac_beacon_ slot_count	PIB attribute 0x0027: Number of beacon slots provisioned in present frame structure.
mac_beacon_ rx_slot	PIB attribute 0x0028: Beacon slot in which this device's switch node transmits its beacon. This attribute is not maintained in a base node.
mac_beacon_ tx_slot	PIB attribute 0x0029: Beacon slot in which this device transmits its beacon. This attribute is not maintained in service nodes that are in a *Terminal* functional state.
mac_beacon_ rx_frequency	PIB attribute 0x002A: Number of frames between receptions of two successive beacons. A value of 0x0 indicates beacons are received in every frame. This attribute is not maintained in a base node.
mac_beacon_ tx_frequency	PIB attribute 0x002B: Number of frames between transmissions of two successive beacons. A value of 0x0 indicates beacons are transmitted in every frame. This attribute is not maintained in service nodes that are in a *Terminal* functional state.
mac_capabilities	PIB attribute 0x002C: This attribute defines the capabilities of the node. It is a bitmap each bit defining a capability.
	Bit 0: Switch Capable
	Bit 1: Packet Aggregation
	Bit 2: Contention Free Period
	Bit 3: Direct connection
	Bit 4: Multicast
	Bit 5: PHY Robustness Management
	Bit 6: ARQ
	Bit 7; Reserved for future use
	Bit 8: Direct Connection Switching
	Bit 9: Multicast Switching Capability
	Bit 10: PHY Robustness Management Switching Capability
	Bit 11: ARQ Buffering Switching Capability
	Bit 12 to 15: Reserved for future use

4.12.8 PRIME NB OFDM PLC MAC counters (class_id = 84, version = 0)

An instance of the "PRIME NB OFDM PLC MAC counters" IC stores statistical information on the operation of the MAC layer for management purposes. The attributes of instances of this IC shall be read only. They can be reset using the reset method.

© Copyright 1997-2020 DLMS User Association

PRIME NB OFDM PLC MAC counters	0...n	class_id = 84, version = 0			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. mac_tx_data_pkt_count (dyn.)	double-long-unsigned		4 294 967 295		x + 0x08
3. mac_rx_data_pkt_count (dyn.)	double-long-unsigned		4 294 967 295		x + 0x10
4. mac_tx_ctrl_pkt_count (dyn.)	double-long-unsigned		4 294 967 295		x + 0x18
5. mac_rx_ctrl_pkt_count (dyn.)	double-long-unsigned		4 294 967 295		x + 0x20
6. mac_csma_fail_count (dyn.)	double-long-unsigned		4 294 967 295		x + 0x28
7. mac_csma_ch_busy_count (dyn.)	double-long-unsigned		4 294 967 295		x + 0x30
Specific methods	***m/o***				
1. reset (data)	o				x + 0x40

Attribute description	
logical_name	Identifies the “PRIME NB OFDM PLC MAC counters” object instance. See 6.2.26.
mac_tx_data_ pkt_count	PIB attribute 0x0040: Count of successfully transmitted MSDUs.
mac_rx_data_ pkt_count	PIB attribute 0x0041: Count of successfully received MSDUs whose destination address was this node.
mac_tx_ctrl_ pkt_count	PIB attribute 0x0042: Count of successfully transmitted MAC control packets.
mac_rx_ctrl_ pkt_count	PIB attribute 0x0043: Count of successfully received MAC control packets whose destination was this node.
mac_csma_fail_ count	PIB attribute 0x0044: Count of failed CSMA transmit attempts.
mac_csma_ch_ busy_count	PIB attribute 0x0045: Count of number of times this node has to back off SCP transmission due to channel busy state.
	NOTE When a counter reaches the maximum value (0xFFFFFFFF), it is automatically rolled-over.
Method description	
reset (data)	This method is used for resetting all the counters held by an instance of this interface class. data ::= integer (0)

4.12.9 PRIME NB OFDM PLC MAC network administration data (class_id = 85, version = 0)

This IC holds the parameters related to the management of the devices connected to the network.

© Copyright 1997-2020 DLMS User Association

PRIME NB OFDM PLC MAC network administration data		0...n	class_id = 85, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mac_list_multicast_entries	(dyn.)	array				x + 0x08
3. mac_list_switch_table	(dyn.)	array				x + 0x10
4. mac_list_direct_table	(dyn.)	array				x + 0x18
5. mac_list_available_switches	(dyn.)	array				x + 0x20
6. mac_list_phy_comm	(dyn)	array				x + 0x28
Specific methods		***m/o***				
1. reset (data)		*o*				x + 0x30

Attribute description	
logical_name	Identifies the “PRIME NB OFDM PLC MAC network administration data” object instance. See 6.2.26.

mac_list_ multicast_ entries	PIB attribute 0x0052: List of entries in multicast switching table. This list is not maintained in service nodes in a *Terminal* functional state.
	mac_list_multicast_entries_type ::= array mac_list_multicast_entries_element
	
	mac_list_multicast_entries_element ::= structure
	{
	mcast_entry_LCID: integer, -- LCID of multicast group
	mcast_entry_members: long -- number of child nodes
	}
	The number of child nodes is the number of the members of this group, including the Node itself.

mac_list_ switch_table	PIB attribute 0x0053: Switch table. This table is not maintained by service nodes in a *Terminal* state. mac switch_table ::= array stbl_entry_LSID stbl_entry_LSID ::= long -- SID of attached Switch node

mac_list_ direct_table	PIB attribute 0x0055: Direct table.
	mac_direct_table ::= array mac_direct_table_element
	
	mac_direct_table_element ::= structure
	{
	dconn_entry_src_SID: long,
	dconn_entry_src_LNID: long,
	dconn_entry_src_LCID: long,
	dconn_entry_dst_SID: long,
	dconn_entry_dst_LNID: long,
	dconn_entry_dst_LCID: long,
	dconn_entry_DID: octet-string (size 6 bytes)
	}

© Copyright 1997-2020 DLMS User Association

mac_list_ direct_table (continued)

Where:

- dconn_entry_src_SID is the SID of switch through which the source service node is connected;
- dconn_entry_src_LNID is the NID allocated to the source service node;
- dconn_entry_src_LCID is the LCID allocated to this connection at the source;
- dconn_entry_dst_SID is the SID of the switch through which the destination service node is connected;
- dconn_entry_dst_LNID is the NID allocated to the destination service node;
- dconn_entry_dst_LCID is the LCID allocated to this connection at the destination;
- dconn_entry_DID is the EUI-48 of the direct switch.

mac_list_ available_ switches

PIB attribute 0x0056: List of switch nodes whose beacons are received.

mac_list_available_switches ::= array mac_list_available_switches_element

```
mac_list_available_switches_element ::= structure
{
    slist_entry_SNA:      octet-string (size 6 bytes),
    slist_entry_LSID:     long,
    slist_entry_level:    integer,
    slist_entry_rx_level: integer,
    slist_entry_rx_snr:   integer
}
```

Where:

- slist_entry_SNA is EUI-48 of the subnetwork;
- slist_entry_LSID is SID of this switch;
- slist_entry_level is level of this switch in subnetwork hierarchy;
- slist_entry_rx_level is the received signal level for this switch;
- slist_entry_rx_snr is the signal to noise ratio for this switch.

mac_list_ phy_comm

PIB attribute 0x0057: List of PHY communication parameters. It is maintained in every node. For terminal nodes it contains only one entry for the switch the node is connected through. For other nodes is contains also entries for every directly connected child node.

```
mac_list_phy_comm ::= array          phy_comm_element
phy_comm_element ::= structure
{
    phy_Comm_EUI:          octet-string,
    phy_Comm_Tx_Pwr:       integer,
    phy_Comm_Tx_Cod:       integer,
    phy_Comm_Rx_Cod:       integer,
    phy_Comm_Rx_Lvl:       integer,
    phy_Comm_SNR:          integer,
    phy_Comm_Tx_Pwr_Mod:   integer,
    phy_Comm_Tx_Cod_Mod:   integer,
    phy_Comm_Rx_Cod_Mod:   integer
}
```


© Copyright 1997-2020 DLMS User Association

mac_list_ phy_comm (continued)	Where: - phy_Comm_EUI is the EUI-48 of the other device; - phy_Comm_Tx_Pwr is the Tx power of GPDU packets sent to the device; - phy_Comm_Tx_Cod is the Tx coding of GPDU packets sent to the device; - phy_Comm_Rx_Cod is the Rx coding of GPDU packets received from the device; - phy_Comm_Rx_Lvl is the Rx power level of GPDU packets received from the device; - phy_Comm_SNR is the SNR of GPDU packets received from the device; - phy_Comm_Tx_Pwr_Mod is the number of times the Tx power was modified; - phy_Comm_Tx_Cod_Mod is the number of times the Tx coding was modified; phy_Comm_Rx_Cod_Mod is the number of times the Rx coding was modified.
Method description	
reset (data)	This method is used for resetting all the entries (to an array of 0 elements) of the attributes 2 to 6 of the instance of this interface class. data ::= integer (0)

4.12.10 PRIME NB OFDM PLC MAC address setup (class_id = 43, version = 0)

An instance of the MAC address setup IC holds the EUI-48 MAC address of the device. The size of this octet string is 6 due to the fact that this address is a EUI-48 and is unique. See also 4.9.4 and 6.2.26.

4.12.11 PRIME NB OFDM PLC Application identification (class_id = 86, version = 0)

An instance of the “PRIME NB OFDM PLC Application identification IC” holds identification information related to administration and maintenance of PRIME NB OFDM PLC devices. They are not communication parameters but allow the device management.

PRIME NB OFDM PLC Application identification		**0...n**	**class_id = 86, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. firmware_version	(static)	octet-string		128		x + 0x08
3. vendor_Id	(static)	long-unsigned				x + 0x10
4. product_Id	(static)	long-unsigned				x + 0x18
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the device setup object instance. See 6.2.26.
firmware_ version	PIB attribute 0x0075: Textual description of the firmware version running on the device.
vendor_Id	PIB attribute 0x0076: Unique vendor identifier assigned by PRIME Alliance.
product_Id	PIB attribute 0x0077: Vendor assigned unique identifier for specific product.

© Copyright 1997-2020 DLMS User Association

4.13 Interface classes for setting up and managing the DLMS/COSEM narrowband OFDM PLC profile for G3-PLC networks

4.13.1 Overview

This subclause 4.13 specifies interface classes for setting up and managing the MAC and 6LoWPAN Adaptation layers of the DLMS/COSEM G3-PLC profile, based ITU-T G.9903:2017.

NOTE 1 ITU-T G.9903:2013 was supported using the version 0 of these interface classes which are retained in subclauses 5.13.1 to 5.13.4.

NOTE 2 ITU-T G.9903:2014 was supported using version 1 of the G3-PLC MAC layer counters in section 4.13.3, version 1 of the G3-PLC MAC setup which is retained in subclause 5.13.6, and G3-PLC 6LoWPAN adaptation layer setup version 1 which is retained in subclause 5.13.7.

For this purpose, the elements of the PAN Information Base (PIB) have been mapped to attributes of COSEM ICs.

COSEM objects for data exchange using G3-PLC, if implemented, shall be located in the Management Logical Device of COSEM servers.

To set up and manage the DLMS/COSEM G3-PLC profile layers (including PHY, IEEE 802.15.4 MAC and 6LoWPAN), three ICs are specified:

- “G3-PLC MAC layer counters”, see 4.13.3;
- “G3-PLC MAC setup”, see 4.13.4;
- “G3-PLC 6LoWPAN adaptation layer setup”, see 4.13.5.

An instance of the existing COSEM interface class “MAC address” (class_id = 43, version = 0) is needed to indicate the EUI-48 MAC address of the G3-PLC modem (corresponding to aExtendedAddress constant in IEEE 802.15.4).

IPv6 configuration is provided by an instance of “IPv6 setup” class.

NOTE 3 The PHY layer of ITU-T G.9903:2014 is out of scope of the G3-PLC setup ICs.

4.13.2 Mapping of G3-PLC IB attributes to COSEM IC attributes

In terms of IEEE 802.15.4, a meter is a Reduced Function Device (RFD) while a concentrator / Neighbourhood Network Access Point (NNAP) is a Full Function Device (FFD) / PAN coordinator. In terms of DLMS/COSEM the meter is the server and the concentrator / NNAP is the client (or an agent for a client).

As COSEM models only the server and not the client, the G3-PLC setup classes concern only the RFD (Reduced Function Device) and not the PAN coordinator.

Table 34 shows the mapping of G3-PLC PIB attributes to attributes of COSEM interface classes.

Table 34 – Mapping of G3-PLC IB attributes specified in ITU-T G.9903:2017 to COSEM IC attributes

Name	Identifier	Interface class	class_id / attribute
MAC counters – Read only PIB attributes that provide statistic information [1]			
mac_Tx_data_packet_count	0x0101	G3-PLC MAC layer counters (class_id = 90, version = 1)	90 / Att. 2
mac_Rx_data_packet_count	0x0102		90 / Att. 3
mac_Tx_cmd_packet_count	0x0103		90 / Att. 4
mac_Rx_cmd_packet_count	0x0104		90 / Att. 5
mac_CSMA_fail_count	0x0105		90 / Att. 6
mac_CSMA_no_ACK_count	0x0106		90 / Att. 7

© Copyright 1997-2020 DLMS User Association

Name	Identifier	Interface class	class_id / attribute
mac_bad_CRC_count	0x0109		90 / Att. 8
mac_Tx_data_broadcast_count	0x0108		90 / Att. 9
mac_Rx_data_broadcast_count	0x0107		90 / Att. 10
MAC setup PIB attributes – Read only & read-write & write only variables [1 2]			
mac_short_address	0x0053	G3-PLC MAC setup (class_id = 91, version = 2)	91 / Att. 2
mac_RC_coord	0x010F		91 / Att. 3
mac_PAN_id	0x0050		91 / Att. 4
mac_key_table	0x0071		91 / Att. 5
mac_frame_counter	0x0077		91 / Att. 6
mac_tone_mask	0x0110		91 / Att. 7
mac_TMR_TTL	0x010D		91 / Att. 8
mac_max_frame_retries	0x0059		91 / Att. 9
mac_POS_table_entry_TTL	0x010E		91 / Att. 10
mac_neighbour_table	0x010A		91 / Att. 11
mac_high_priority_window_size	0x0100		91 / Att. 12
mac_CSMA_fairness_limit	0x010C		91 / Att. 13
mac_beacon_randomization_window_length	0x0111		91 / Att. 14
mac_A	0x0112		91 / Att. 15
mac_K	0x0113		91 / Att. 16
mac_min_CW_attempts	0x0114		91 / Att. 17
mac_cenelec_legacy_mode	0x0115		91 / Att. 18
mac_FCC_legacy_mode	0x0116		91 / Att. 19
mac_max_BE	0x0047		91 / Att. 20
mac_max_CSMA_backoffs	0x004E		91 / Att. 21
mac_min_BE	0x004F		91 / Att. 22
mac_broadcast_max_CW_enabled	0x011E		91 / Att. 23
mac_transmit_atten	0x011F		91 / Att. 24
mac_POS_table	0x0120		91 / Att. 25
6LoWPAN adaptation layer IB attributes – Read only & read-write variables [3 4]			
adp_max_hops	0x0F	G3-PLC 6LoWPAN adaptation layer setup (class_id = 92, version = 2)	92 / Att. 2
adp_weak_LQI_value	0x1A		92 / Att. 3
adp_security_level	0x00		92 / Att. 4
adp_prefix_table	0x01		92 / Att. 5
adp_routing_configuration	0x09, 0x0A, 0x0D, 0x11-0x19, 0x1B, 0x1F		92 / Att. 6
adp_broadcast_log_table_entry_TTL	0x02		92 / Att. 7
adp_routing_table	0x0C		92 / Att. 8
adp_context_information_table	0x07		92 / Att. 9
adp_blacklist_table	0x1E		92 / Att. 10
adp_broadcast_log_table	0x0B		92 / Att. 11

© Copyright 1997-2020 DLMS User Association

Name	Identifier	Interface class	class_id / attribute
adp_group_table	0x0E		92 / Att. 12
adp_max_join_wait_time	0x20		92 / Att. 13
adp_path_discovery_time	0x21		92 / Att. 14
adp_active_key_index	0x22		92 / Att. 15
adp_metric_type	0x03		92 / Att. 16
adp_coord_short_address	0x08		92 / Att. 17
adp_disable_default_routing	0xF0		92 / Att. 18
adp_device_type	0x10		92 / Att. 19
adp_default_coord_route_enabled	0x24		92 / Att. 20
adp_destination_address_set	0x23		92 / Att. 21
[1] See ITU-T G.9903:2017, 9.3.6.2.2 and 9.3.6.2.3. [2] The following attributes of the G3-PLC MAC sublayer IB attributes have been excluded as there is no need to expose them: *macBSN*, *macDSN*, *macAckWaitDuration*, *macFreqNotching*, *macTimeStampSupported*, *macPromiscuousMode*, *macSecurityEnabled.* [3] See ITU-T G.9903:2017, 9.4.1.1. [4] The following attributes of the G3-PLC Adaptation sublayer IB attributes have been excluded as there is no need to expose them ; *adpSoftVersion, adpSnifferMode*.			
NOTE Whereas in ITU-T G.9903 the camel-case notation is used, in COSEM interface class specifications – and in this table – the underscore notation is used.			

4.13.3 G3-PLC MAC layer counters (class_id = 90, version = 1)

An instance of the “G3-PLC MAC layer counters” IC stores counters related to the MAC layer exchanges. The objective of these counters is to provide statistical information for management purposes.

The attributes of instances of this IC shall be read only. They can be reset using the reset method.

G3-PLC MAC layer counters	**0…n**	**class_id = 90, version = 1**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. mac_Tx_data_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x08
3. mac_Rx_data_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x10
4. mac_Tx_cmd_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x18
5. mac_Rx_cmd_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x20
6. mac_CSMA_fail_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x28
7. mac_CSMA_no_ACK_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x30
8. mac_bad_CRC_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x38
9. mac_Tx_data_broadcast_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x40
10. mac_Rx_data_broadcast_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x48
Specific methods	***m/o***				
1. reset (data)	*O*				x + 0x50

© Copyright 1997-2020 DLMS User Association

Attribute description NOTE When a counter reaches the maximum value (0xFFFFFFFF), it's automatically rolled-over.	
logical_name	Identifies the "G3-PLC MAC layer counters" object instance. See 6.2.28.
mac_Tx_data_packet_count	PIB attribute 0x0101: Statistic counter of successfully transmitted data packets (MSDUs).
mac_Rx_data_packet_count	PIB attribute 0x0102: Statistic counter of successfully received data packets (MSDUs).
mac_Tx_cmd_packet_count	PIB attribute 0x0103: Statistic counter of successfully transmitted command packets.
mac_Rx_cmd_packet_count	PIB attribute 0x0104: Statistic counter of successfully received command packets.
mac_CSMA_fail_count	PIB attribute 0x0105: Counts the number of times when CSMA backoffs reach macMaxCSMABackoffs.
mac_CSMA_no_ACK_count	PIB attribute 0x0106: Counts the number of times when an ACK is not received while transmitting a unicast data frame (The loss of ACK is attributed to collisions).
mac_bad_CRC_count	PIB attribute 0x0109: Statistic counter of the number of frames received with bad CRC.
mac_Tx_data_broadcast_count	PIB attribute 0x0108: Statistic counter of the number of broadcast frames sent.
mac_Rx_data_broadcast_count	PIB attribute 0x0107: Statistic counter of successfully received broadcast packets.
Method description	
reset (data)	This method forces a reset of the object. By invoking this method, the value of all counters is set to 0. data ::= integer (0)

4.13.4 G3-PLC MAC setup (class_id = 91, version = 2)

An instance of the "G3-PLC MAC setup" IC holds the necessary parameters to set up and manage the G3-PLC IEEE 802.15.4 MAC sub-layer.

These attributes influence the functional behaviour of an implementation. Implementations may allow changes to the attributes during normal running, i.e. even after the device start-up sequence has been executed.

© Copyright 1997-2020 DLMS User Association

G3-PLC MAC setup		0...n	class_id = 91, version = 2			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				X
2. mac_short_address	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x08
3. mac_RC_coord	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x10
4. mac_PAN_id	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x18
5. mac_key_table	(dyn.)	array			Empty	x + 0x20
6. mac_frame_counter	(dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x28
7. mac_tone_mask	(static)	bit-string			'FFFFFFFFF0 00000000'H for CENELEC-A bandplan; 'FFFFFFFFFF FFFFFFFF'H for FCC bandplan	x + 0x30
8. mac_TMR_TTL	(static)	unsigned	0	255	10	x + 0x38
9. mac_max_frame_retries	(static)	unsigned	0	10	5	x + 0x40
10. mac_POS_table_entry_TTL	(static)	unsigned	0	255	255	x + 0x48
11. mac_neighbour_table	(dyn.)	array				x + 0x50
12. mac_high_priority_window_size	(static)	unsigned	1	7	7	x + 0x58
13. mac_CSMA_fairness_limit	(static)	unsigned	See below	255	25	x + 0x60
14. mac_beacon_randomization_window_length	(static)	unsigned	1	254	12	x + 0x68
15. mac_A	(static)	unsigned	3	20	8	x + 0x70
16. mac_K	(static)	unsigned	1	See below	5	x + 0x78
17. mac_min_CW_attempts	(static)	unsigned	0	255	10	x + 0x80
18. mac_cenelec_legacy_mode	(static)	unsigned	0	255	1	x + 0x88
19. mac_FCC_legacy_mode	(static)	unsigned	0	255	1	x + 0x90
20. mac_max_BE	(static)	unsigned	0	20	8	x + 0x98
21. mac_max_CSMA_backoff	(static)	unsigned	0	255	50	x + 0xA0
22. mac_min_BE	(static)	unsigned	0	20	3	x + 0xA8
23. mac_broadcast_max_CW_enabled	(static)	boolean			FALSE	x + 0xB0
24. mac_transmit_atten	(static)	unsigned	0	25	0	x + 0xB8
25. mac_POS_table	(dyn)	array				x + 0xC0
Specific methods		***m/o***				
1. mac_get_neighbour_table_entry(data)		O				x + 0xE0

Attribute description

logical_name	Identifies the "G3-PLC MAC setup" object instance. See 6.2.28.

© Copyright 1997-2020 DLMS User Association

mac_short_address	PIB attribute 0x0053: The 16-bit address the device is using to communicate through the PAN. Its value shall be equal to 0xFFFF when the device does not have a short address. An associated device necessarily has a short address, so that a device cannot be in the state where it is associated but does not have a short address.
mac_RC_coord	PIB attribute 0x010F: Route cost to coordinator, to be used in the beacon payload as RC_COORD.
mac_PAN_id	PIB attribute 0x0050: The 16-bit identifier of the PAN through which the device is operating. A value equal to 0xFFFF indicates that the device is not associated.

mac_key_table	PIB attribute 0x0071: This attribute holds GMK keys required for MAC layer ciphering. The attribute can hold up to two 16-bytes keys. The Key Identifier value must be different for each key.
	For security reason, the key entries cannot be read, only written.
	array mac_GMK
	mac_GMK ::= structure
	{
	key_id: unsigned,
	key: octet-string
	}

key_id	The Key Identifier used to refer to this key, can take the value 0 or 1.
key	The AES-128 key used for ciphering the frames exchanged at MAC layer.
mac_frame_counter	PIB attribute 0x0077: The outgoing frame counter for this device, used when ciphering frames at MAC layer.
mac_tone_mask	PIB attribute 0x0110: Defines the tone mask to use during symbol formation. Each bit controls if a given carrier is used (value = 1) or notched (value = 0). The bit-string is always 72 bits long, the number of meaningful bits depending on the device band-plan. The first bit in the bit-string (bit 0) bit represents the lowest frequency of the CENELEC -A or FCC band-plans. The mapping in the CENELEC-A bandplan is given in the following table:

Bit number	Carrier number	Carrier Frequency (kHz)
bit 0 (first bit)	1	35,937 5
bit 1	2	37,5
bit 2	3	39,062 5
bit 3	4	40,625
bit 4	5	42,187 5
bit 5	6	43,75
bit 6	7	45,312 5
bit 7	8	46,875
bit 8	9	48,437 5
bit 9	10	50

© Copyright 1997-2020 DLMS User Association

mac_tone_mask (continued) (CENELEC-A bandplan continued)

Bit number	Carrier number	Carrier Frequency (kHz)
bit 10	11	51,562 5
bit 11	12	53,125
bit 12	13	54,687 5
bit 13	14	56,25
bit 14	15	57,812 5
bit 15	16	59,375
bit 16	17	60,937 5
bit 17	18	62,5
bit 18	19	64,062 5
bit 19	20	65,625
bit 20	21	67,187 5
bit 21	22	68,75
bit 22	23	70,312 5
bit 23	24	71,875
bit 24	25	73,437 5
bit 25	26	75
bit 26	27	76,562 5
bit 27	28	78,125
bit 28	29	79,687 5
bit 29	30	81,25
bit 30	31	82,812 5
bit 31	32	84,375
bit 32	33	85,937 5
bit 33	34	87,5
bit 34	35	89,062 5
bit 35	36	90,625
bits 36 to 71 (last bit)	not used	not used

The mapping in the FCC bandplan is given in the following table:

Bit number	Carrier number	Carrier Frequency (kHz)
bit 0 (first bit)	1	154,687 5
bit 1	2	159,375
bit 2	3	164,062 5
bit 3	4	168,75
bit 4	5	173,437 5
bit 5	6	178,125
bit 6	7	182,812 5
bit 7	8	187,5
bit 8	9	192,187 5
bit 9	10	196,875
bit 10	11	201,562 5
bit 11	12	206,25
bit 12	13	210,937 5
bit 13	14	215,625
bit 14	15	220,312 5

© Copyright 1997-2020 DLMS User Association

mac_tone_mask
(continued)

(FCC bandplan continued)

Bit number	Carrier number	Carrier Frequency (kHz)
bit 15	16	225
bit 16	17	229,687 5
bit 17	18	234,375
bit 18	19	239,062 5
bit 19	20	243,75
bit 20	21	248,437 5
bit 21	22	253,125
bit 22	23	257,812 5
bit 23	24	262,5
bit 24	25	267,187 5
bit 25	26	271,875
bit 26	27	276,562 5
bit 27	28	281,25
bit 28	29	285,937 5
bit 29	30	290,625
bit 30	31	295,312 5
bit 31	32	300
bit 32	33	304,687 5
bit 33	34	309,375
bit 34	35	314,062 5
bit 35	36	318,75
bit 36	37	323,437 5
bit 37	38	328,125
bit 38	39	332,812 5
bit 39	40	337,5
bit 40	41	342,187 5
bit 41	42	346,875
bit 42	43	351,562 5
bit 43	44	356,25
bit 44	45	360,937 5
bit 45	46	365,625
bit 46	47	370,312 5
bit 47	48	375
bit 48	49	379,687 5
bit 49	50	384,375
bit 50	51	389,062 5
bit 51	52	393,75
bit 52	53	398,437 5
bit 53	54	403,125
bit 54	55	407,812 5
bit 55	56	412,5
bit 56	57	417,187 5

© Copyright 1997-2020 DLMS User Association

mac_tone_mask
(continued)

(FCC bandplan continued)

bit 57	58	421,875
bit 58	59	426,562 5
bit 59	60	431,25
bit 60	61	435,937 5
bit 61	62	440,625
bit 62	63	445,312 5
bit 63	64	450
bit 64	65	454,687 5
bit 65	66	459,375
bit 66	67	464,062 5
bit 67	68	468,75
bit 68	69	473,437 5
bit 69	70	478,125
bit 70	71	482,812 5
bit 71 (last bit)	72	487,5

mac_TMR_TTL

PIB attribute 0x010D: Maximum time to live for an entry in the neighbour table in minutes.

NOTE 1: This attribute was used with different definitions in earlier versions of the Interface Class. See 5.13.3 and 5.13.6.

mac_max_frame_retries

PIB attribute 0x0059: Maximum number of retransmissions.

mac_POS_table_entry_TTL

PIB attribute 0x010E: Maximum time to live for an entry in the POS table in minutes

NOTE 2: This attribute was used with different name and different definitions in earlier versions of the Interface Class. See 5.13.3 and 5.13.6.

mac_neighbour_table

PIB attribute 0x010A: See ITU-T G.9903:2017, 9.3.7.2.2 for CENELEC and FCC bands.

Every device shall maintain a ''neighbour table'' which contains information about all the devices with which a unicast message has been exchanged. Each time a tone map response command is received the table is updated. In case the table is full, the entry corresponding to the shortest valid time is removed. This table shall be accessible by the adaptation, MAC sublayers and physical layer.

array neighbour_table

© Copyright 1997-2020 DLMS User Association

mac_neighbour_table
(continued)

```
neighbour_table::= structure
{
    short_address:                long-unsigned,
    payload_modulation_scheme:    boolean,
    tone_map:                     bit-string,
    modulation_type:              enum,
    tx_gain:                      integer,
    tx_res:                       boolean,
    tx_coef:                      bit-string,
    reverse_lqi:                  unsigned,
    phase_differential            integer,
    TMR_valid_time:               unsigned,
    no_data:                      unsigned
}
```

short_ address	The MAC Short Address of the node which this entry refers to.
payload_ modulation_ scheme	Payload Modulation scheme to be used when transmitting to this neighbour. 0: Differential, 1: Coherent
tone_map	The tone map to be used when transmitting to this neighbour. NOTE 3 The tone map parameter defines which frequency sub-band can be used for communication with the device. A bit set to 1 means that the frequency sub-band can be used, and a bit set to 0 means that frequency sub-band is not to be used. The first bit in the bit-string (bit 0) corresponds to Tone Map bit TM[0]. In the CENELEC-A bandplan, each bit of the tone map field is associated with one sub-band consisting of a group of 6 tones.

Bit number	TM field	Sub-band (kHz)
0	TM[0]	35,937 5 to 43,75
1	TM[1]	45,312 5 to 53,125
2	TM[2]	54,687 5 to 62,5
3	TM[3]	64,062 5 to 71,875
4	TM[4]	73,437 5 to 81,25
5	TM[5]	82,812 5 to 90,625

In the FCC bandplan, each bit of the tone map field is associated with one sub-band consisting of a group of 3 tones.

Bit number	TM field	Sub-band (kHz)
0	TM[0]	154,687 5 to 164,062 5
1	TM[1]	168,75 to 178,125
2	TM[2]	182,812 5 to 192,187 5
3	TM[3]	196,875 to 206,25
4	TM[4]	210,937 5 to 220,312 5
5	TM[5]	225 to 234,375

© Copyright 1997-2020 DLMS User Association

mac_neighbour_table (continued)

tone_map	(tone map bit table continued – FCC)

Bit number	TM field	Sub-band (kHz)
6	TM[6]	239,062 5 to 248,437 5
7	TM[7]	253,125 to 262,5
8	TM[8]	267,187 5 to 276,562 5
9	TM[9]	281,25 to 290,625
10	TM[10]	295,312 5 to 304,687 5
11	TM[11]	309,375 to 318,75
12	TM[12]	323,437 5 to 332,812 5
13	TM[13]	337,5 to 346,875
14	TM[14]	351,562 5 to 360,937 5
15	TM[15]	365,625 to 375
16	TM[16]	379,687 5 to 389,062 5
17	TM[17]	393,75 to 403,125
18	TM[18]	407,812 5 to 417,187 5
19	TM[19]	421,875 to 431,25
20	TM[20]	435,937 5 to 445,312 5
21	TM[21]	450 to 459,375
22	TM[22]	464,062 5 to 473,437 5
23	TM[23]	478,125 to 487,5

modulation_type	The modulation type to use for communicating with the device. enum (0) Robust Mode, (1) DBPSK, (2) DQPSK, (3) D8PSK, (4) 16-QAM NOTE 4 The 16-QAM modulation is optional and only applicable for FCC band.
tx_gain	Tx Gain to be used when transmitting to this neighbour.
tx_res	Defines the Tx Gain resolution corresponding to one gain step. 0 : 6 dB 1 : 3 dB
tx_coef	A parameter that specifies transmitter gain for each group of tones represented by one valid bit of the tone map. NOTE 5 One group of tones gathers 6 consecutive tones (or carriers) for CENELEC bands, and 3 consecutive tones for FCC band. The receiver measures the frequency-dependent attenuation of the channel and may request the transmitter to compensate for this attenuation by increasing the transmit power on sections of the spectrum that are experiencing attenuation in order to equalize the received signal.

© Copyright 1997-2020 DLMS User Association

mac_neighbour_table (continued)

tx_coef (continued)

Each group of tones is mapped to a 4-bit value for CENELEC-A or a 2-bit value for FCC where a "0" in the last bit indicates a positive gain value, hence an increase in the transmitter gain scaled by TXRES is requested for that section and a "1" indicates a negative gain value, hence a decrease in the transmitter gain scaled by TXRES is requested for that section. Implementing this feature is optional and it is intended for frequency selective channels. If this feature is not implemented, the value zero shall be used.

In the CENELEC bandplan, the first bit in the bit-string (bit 0) corresponds to TXCOEF[0] and the last bit in the bit-string (bit 23) corresponds to TXCOEF[23], noting that the bits are ordered in groups as defined in ITU-T G.9903:2017, table 9-21.

In the FCC bandplan, the first bit in the bit-string (bit 0) corresponds to TXCOEF[0] and the last bit in the bit-string (bit 47) corresponds to TXCOEF[47], noting that the bits are ordered in groups as defined in ITU-T G.9903:2017, table 9-22.

NOTE 6: A description of tx_coeff is included in G.9903:2017 tables 9-21 and 9-22

In the CENELEC-A bandplan, each group of 4 bits of the TXCOEF field is associated with one sub-band consisting of a group of 6 tones.

Bit number	TXCOEF field	Sub-band (kHz)
0	TXCOEF[0]	35,937 5 to 43,75
1	TXCOEF[1]	
2	TXCOEF [2]	
3	TXCOEF [3]	
4	TXCOEF [4]	45,312 5 to 53,125
5	TXCOEF [5]	
6	TXCOEF[6]	
7	TXCOEF[7]	
8	TXCOEF[8]	54,6875 to 62,5
9	TXCOEF[9]	
10	TXCOEF[10]	
11	TXCOEF[11]	
12	TXCOEF[12]	64,062 5 to 71,875
13	TXCOEF[13]	
14	TXCOEF[14]	
15	TXCOEF[15]	
16	TXCOEF[16]	73,437 5 to 81,25
17	TXCOEF[17]	
18	TXCOEF[18]	
19	TXCOEF[19]	
20	TXCOEF[20]	82,812 5 to 90,625
21	TXCOEF[21]	
22	TXCOEF[22]	
23	TXCOEF[23]	

© Copyright 1997-2020 DLMS User Association

mac_neighbour_table (continued)

tx_coef (continued)

In the FCC bandplan, each group of 2 bits of the TXCOEF field is associated with one sub-band consisting of a group of 3 tones

Bit number	TXCOEF field	Sub-band (kHz)
0	TXCOEF[0]	154,687 5 to 164,062 5
1	TXCOEF[1]	
2	TXCOEF [2]	168,75 to 178,125
3	TXCOEF [3]	
4	TXCOEF [4]	182,812 5 to 192,187 5
5	TXCOEF [5]	
6	TXCOEF[6]	196,875 to 206,25
7	TXCOEF[7]	
8	TXCOEF[8]	210,937 5 to 220,312 5
9	TXCOEF[9]	
10	TXCOEF[10]	225 to 234,375
11	TXCOEF[11]	
12	TXCOEF[12]	239,062 5 to 248,437 5
13	TXCOEF[13]	
14	TXCOEF[14]	253,125 to 262,5
15	TXCOEF[15]	
16	TXCOEF[16]	267,187 5 to 276,562 5
17	TXCOEF[17]	
18	TXCOEF[18]	281,25 to 290,625
19	TXCOEF[19]	
20	TXCOEF[20]	295,312 5 to 304,687 5
21	TXCOEF[21]	
22	TXCOEF[22]	309,375 to 318,75
23	TXCOEF[23]	
24	TXCOEF[24]	323,437 5 to 332,812 5
25	TXCOEF[25]	
26	TXCOEF[26]	337,5 to 346,875
27	TXCOEF[27]	
28	TXCOEF[28]	351,562 5 to 360,937 5
29	TXCOEF[29]	
30	TXCOEF[30]	365,625 to 375
31	TXCOEF[31]	
32	TXCOEF[32]	379,687 5 to 389,062 5
33	TXCOEF[33]	
34	TXCOEF[34]	393,75 to 403,125
35	TXCOEF[35]	
36	TXCOEF[36]	407,812 5 to 417,187 5
37	TXCOEF[37]	
38	TXCOEF[38]	421,875 to 431,25
39	TXCOEF[39]	
40	TXCOEF[40]	435,937 5 to 445,312 5
41	TXCOEF[41]	
42	TXCOEF[42]	450 to 459,375
43	TXCOEF[43]	
44	TXCOEF[44]	464,062 5 to 473,437 5
45	TXCOEF[45]	
46	TXCOEF[46]	478,125 to 487,5
47	TXCOEF[47]	

© Copyright 1997-2020 DLMS User Association

mac_neighbour_table (continued)	reverse_lqi	Link Quality Indicator of the link to the neighbour (reverse LQI). NOTE 7 LQI value measured during reception of the PPDU from the neighbour. The LQI measurement is a characterization of the strength and/or quality of a received packet.
	phase_ differential	Phase difference in multiples of 60 degrees between the mains phase of the local node and the neighbour node. PhaseDifferential can assume six integer values between 0 and 5.
	TMR_valid_ time	Remaining time in minutes until when the parameters are considered valid. When an entry is created, this value shall be set to the default value. - When it reaches 0, a tone map request may be issued if data is sent to this device. Upon successful reception of a tone map response, this value is set to mac_TMR_TTL. - When a MAC fail occurs (when **mac_max_frame_retries** attempts have failed), this value shall be set to 0.
	no_data	Value that may be used as needed for some implementations as determined by a companion specification.
mac_high_priority_ window_size	PIB attribute 0x0100: The high priority contention window size in number of slots.	
mac_CSMA_fairness_limit	PIB attribute 0x010C: Channel access fairness limit. Specifies how many failed back-off attempts, back-off exponent is set to minBE. This attribute can take a value between 2 × (macMaxBE – macMinBE) and 255.	
mac_beacon_ randomization_ window_length	PIB attribute 0x0111: Duration time in seconds for the beacon randomization.	
mac_A	PIB attribute 0x0112: This parameter controls the adaptive CW linear decrease.	
mac_K	PIB attribute 0x0113: Rate Adaptation factor for channel access fairness limit. This attribute can take a value between 1 and macCSMAFairnessLimit.	
mac_min_CW_attempts	PIB attribute 0x0114: Number of consecutive attempts while using minimum CW.	
mac_cenelec_legacy_ mode	PIB attribute 0x0115: This read only attribute indicates the capability of the node. 0: The following configuration is used ÷ - Elementary interleaving, - Interleaver parameters n_i and n_j are not swapped when I(i,j) = 0. 1: The following configuration is used÷ - Full Block interleaving, - Interleaver parameters n_i and n_j are swapped when I(i,j) = 0.	

© Copyright 1997-2020 DLMS User Association

mac_FCC_legacy_mode	PIB attribute 0x0116: This read only attribute indicates the capability of the node. 0: The following configuration is used: - Differential FCH modulation, - Elementary interleaving, - Interleaver parameters n_i and n_j are not swapped when I(i,j) = 0. - Single RS block 1: The following configuration is used÷ - Coherent FCH modulation, - Full Block interleaving, - Interleaver parameters n_i and n_j are swapped when I(i,j) = 0, - Two RS blocks.
mac_max_BE	PIB attribute 0x0047: Maximum value of backoff exponent. It should always be greater than mac_min_BE.
mac_max_CSMA_backoffs	PIB attribute 0x004E: Maximum number of backoff attempts.
mac_min_BE	PIB attribute 0x004F: Minimum value of backoff exponent.
mac_broadcast_max_CW_enabled	PIB attribute 0x011E: If enabled, MAC uses maximum contention window.
mac_transmit_atten	PIB attribute 0x011F: Attenuation of the output level in dB.

mac_POS_table	PIB attribute 0x0120: The neighbour table contains some information about all the devices within the POS of the device.
	array POS_table
	POS_table ::= structure
	{
	short_address: long-unsigned,
	LQI: unsigned,
	POS_valid_time: unsigned
	}
	NOTE 8 Each time a message is received (filtered according to IEEE 802.15.4, 7.5.6.2), an entry in the POS table is created or updated (if the entry is already present). In case the table is full, the entry corresponding to the shortest valid time is removed.

short_ address	The MAC Short Address of the neighbour which this entry refers to.
LQI	Link quality indicator of the last received packet from this neighbour. NOTE 9: LQI is referred to as forward_LQI in some implementations.
POS_valid_ time	Remaining time in minutes until when this entry is considered valid. Every time an entry is created, it is set to mac_POS_table_entry_TTL. When it reaches zero, this entry is no longer valid in the table and may be removed.

© Copyright 1997-2020 DLMS User Association

Method description	
mac_get_neighbour_ table_entry (data)	This method is used to retrieve the mac neighbour table for one MAC short address. It may be used to perform topology monitoring by the client. The method invocation parameter contains a mac_short_address. data: long-unsigned The response parameter includes the neighbour table for this mac_short_address. data ::= array neighbour_table where mac_neighbour_table is as defined in the *mac_neighbour_table* attribute of the present IC.

© Copyright 1997-2020 DLMS User Association

4.13.5 G3-PLC 6LoWPAN adaptation layer setup (class_id = 92, version = 2)

An instance of the “G3-PLC 6LoWPAN adaptation layer setup” IC holds the necessary parameters to set up and manage the G3-PLC 6LoWPAN Adaptation layer.

These attributes influence the functional behaviour of an implementation. Implementations may allow changes to their values during normal running, i.e. even after the device start-up sequence has been executed.

G3-PLC 6LoWPAN adaptation layer setup		**0…n**	**class_id = 92, version = 2**			
Attribute (s)		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. adp_max_hops	(static)	unsigned	1	14	8	x + 0x08
3. adp_weak_LQI_value	(static)	unsigned	0	255	52	x + 0x10
4. adp_security_level	(static)	unsigned	0	5	5	x + 0x18
5. adp_prefix_table	(dyn)	array				x + 0x20
6. adp_routing_configuration	(static)	array				x + 0x28
7. adp_broadcast_log_table _entry_TTL	(static)	long- unsigned	0	65535	2	x + 0x30
8. adp_routing_table	(dyn)	array				x + 0x38
9. adp_context_information _table	(dyn)	array				x + 0x40
10. adp_blacklist_table	(dyn)	array				x + 0x48
11. adp_broadcast_log_table	(dyn)	array				x + 0x50
12. adp_group_table	(dyn)	array				x + 0x58
13. adp_max_join_wait_time	(static)	long- unsigned	0	1023	20	x + 0x60
14. adp_path_discovery_time	(static)	unsigned	0	255	40	x + 0x68
15. adp_active_key_index	(static)	unsigned	0	1	0	x + 0x70
16. adp_metric_type	(static)	unsigned	0x00	0x0F	0x0F	x + 0x78
17. adp_coord_short_address	(static)	long- unsigned	0x0000	0x7FFF	0x0000	x + 0x80
18. adp_disable_default_routing	(static)	boolean			FALSE	x + 0x88
19. adp_device_type	(static)	enum	0	2	2	x + 0x90
20. adp_default_coord_route _enabled	(static)	boolean			FALSE	x + 0x98
21. adp_destination_address _set	(dyn)	array				x + 0xA0
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “G3-PLC OFDM 6LoWPAN adaptation layer setup” object instance. See 6.2.28.
adp_max_hops	PIB attribute 0x0F: Defines the maximum number of hops to be used by the routing algorithm.

© Copyright 1997-2020 DLMS User Association

adp_weak_LQI_value	PIB attribute 0x1A: The weak link value defines the LQI value below which a link to a neighbour is considered as a weak link. A value of 52 represents an SNR of 3 dB.
adp_security_level	PIB attribute 0x00: The minimum security level to be used for incoming and outgoing adaptation frames. Only values 0 (no ciphering) and 5 (ciphering with 32 bits integrity code) are supported.
adp_prefix_table	PIB attribute 0x01: Contains the list of prefixes defined on this PAN. NOTE 1 it is assumed that the link local IPv6 address exists independently and is not affected by the prefixes defined in the prefix table.

adp_routing_ configuration	Specifies all parameters linked to the routing mechanism described in ITU-T G.9903:2017. The elements are specified in 9.4.1.1 of that Recommendation.
	NOTE 2 The Link cost calculation is provided in ITU-T G.9903:2017 Annex B.
	array routing_configuration
	routing_configuration ::= structure
	{
	adp_net_traversal_time: unsigned,
	adp_routing_table_entry_TTL: long-unsigned,
	adp_Kr: unsigned,
	adp_Km: unsigned,
	adp_Kc: unsigned,
	adp_Kq: unsigned,
	adp_Kh: unsigned,
	adp_Krt: unsigned,
	adp_RREQ_retries: unsigned,
	adp_RREQ_wait: unsigned,
	adp_blacklist_table_entry_TTL: long-unsigned,
	adp_unicast_RREQ_gen_enabled: boolean,
	adp_RLC_Enabled: boolean,
	adp_add_rev_link_cost: unsigned
	}

adp_net_ traversal_ time	PIB attribute 0x11: Maximum time that a packet is expected to take to reach any node from any node in seconds. Range : 0-255 Default value : 20
adp_routing_ table_entry_ TTL	PIB attribute 0x12: Maximum time-to-live of a routing table entry (in minutes). Range : 0-65 535 Default value : 360
adp_Kr	PIB attribute 0x13: A weight factor for the Robust Mode to calculate link cost. Range : 0-31 Default value : 0
adp_Km	PIB attribute 0x14: A weight factor for modulation to calculate link cost. Range : 0-31 Default value : 0

© Copyright 1997-2020 DLMS User Association

adp_Kc	PIB attribute 0x15: A weight factor for number of active tones to calculate link cost. Range : 0-31 Default value : 0
adp_Kq	PIB attribute 0x16: A weight factor for LQI to calculate route cost. Range : 0-50 Default value : 10
adp_Kh	PIB attribute 0x17: A weight factor for hop to calculate link cost. Range : 0-31 Default value : 4
adp_Krt	PIB attribute 0x1B: A weight factor for the number of active routes in the routing table to calculate link cost. Range : 0-31 Default value : 0
adp_RREQ_ retries	PIB attribute 0x18: The number of RREQ re-transmission in case of RREP reception time out. Range : 0-255 Default value : 0
adp_RREQ_wait	PIB attribute 0x19: Time in seconds to wait between two consecutive RREQ or RLCREQ generations. Range : 0-255 Default value : 30 NOTE 3 : The title and definition of this item has been modified from previous version
adp_blacklist_ table_entry_ TTL	PIB attribute 0x1F: Maximum time-to-live of a blacklisted neighbour entry (in minutes). Range : 0-65 535 Default value : 10
adp_unicast_ RREQ_gen_ enabled	PIB attribute 0x0D: If TRUE, the RREQ shall be generated with its ''unicast RREQ'' flag set to '1'. If FALSE, the RREQ shall be generated with its ''unicast RREQ'' flag set to '0'. Default value : TRUE
adp_RLC_ enabled	PIB attribute 0x09: Enable the sending of RLCREQ frame by the device. Default value : FALSE
adp_add_rev_ link_cost	PIB attribute 0x0A: It represents an additional cost to take into account a possible asymmetry in the link. Range : 0-255 Default value : 0

© Copyright 1997-2020 DLMS User Association

adp_broadcast_ log_table_entry_TTL	PIB attribute 0x02: Maximum time to live of an adpBroadcastLogTable entry (in minutes).

adp_routing_table	PIB attribute 0x0C: Contains the routing table.
	array routing_table
	routing_table ::= structure
	{
	destination_address: long-unsigned,
	next_hop_address: long-unsigned,
	route_cost: long-unsigned,
	hop_count: unsigned,
	weak_link_count: unsigned,
	valid_time: long-unsigned
	}
	NOTE 4 This table is actualized each time a route is built or updated (triggered by data traffic) and each time the TTL timer expires.

destination_ address	Address of the destination.
next_hop_ address	Address of the next hop on the route towards the destination.
route_cost	Cumulative link cost along the route towards the destination.
hop_count	Number of hops of the selected route to the destination. Range: 0-14 NOTE 5 Practically the maximum allowed value is limited by adp_max_hops.
weak_link_ count	Number of weak links to destination. Range : 0-14 NOTE 6 Practically the maximum allowed value is limited by adp_max_hops.
valid_time	Remaining time in minutes until when this entry in the routing table is considered valid.

adp_context_ information_table	PIB attribute 0x07: Contains the context information associated to each CID extension field. See ITU-T G.9903:2017. The elements are specified in Table 9-30 of that Recommendation.
	
	array context_information_table
	context_information_table ::= structure
	{
	CID: bit-string,
	context_length: unsigned,
	context: octet-string,
	C: boolean,
	valid_lifetime: long-unsigned
	}

CID	Corresponds to the 4-bit context information used for source and destination addresses (SCI, DCI). The first bit in the bit-string (bit 0) corresponds to the least significant bit of CID in G.9903:2017, Table 9-30.

© Copyright 1997-2020 DLMS User Association

	context_ length	Indicates the length of the carried context (up to 128-bit contexts may be carried). Range: 0-128
	context	Corresponds to the carried context used for compression/decompression purposes.
	C	Indicates if the context is valid for use in compression. FALSE : Only decompression is allowed, TRUE : Compression and decompression are allowed. A context may be used for decompression purposes only. Moreover, recommendations made in RFC 6775 should be followed to take into account the propagation of the context to all nodes of the PAN
	valid_lifetime	Remaining time in minutes during which the context information table is considered valid. It is updated upon reception of the advertised context. Range: 0-65 535

adp_blacklist_table	PIB attribute 0x1E: Contains the list of the blacklisted neighbours.
	array blacklisted_neighbour_set
	blacklisted_neighbour_set ::= structure
	{
	blacklisted_neighbour_address: long-unsigned,
	valid_time: long-unsigned
	}	

	blacklisted_ neighbour_ address	The 16-bit address of the blacklisted neighbour.
	valid_time	Remaining time in minutes until which this entry in the blacklisted neighbour table is considered valid.

adp_broadcast_ log_table	PIB attribute 0x0B: Contains the broadcast log table.
	NOTE 7 This table provides a list of the broadcast packets recently received by this device.
	array broadcast_log_table
	broadcast_log_table ::= structure
	{
	source_address: long-unsigned,
	sequence_number: unsigned,
	valid_time: long-unsigned
	}	

	source_ address	The 16-bit source address of a broadcast packet. This is the address of the broadcast initiator.
	sequence_ number	The sequence number contained in the BC0 header.
	valid_time	Remaining time in minutes until when this entry in the broadcast log table is considered valid.

© Copyright 1997-2020 DLMS User Association

adp_group_table	PIB attribute 0x0E: Contains the group addresses to which the device belongs. array group_address group_address ::= long-unsigned
	group_address: Group address to which this node has been subscribed.
adp_max_join_ wait_time	PIB attribute 0x20: Network join timeout in seconds for LBD.
adp_path_ discovery_time	PIB attribute 0x21: Timeout for path discovery in seconds.
adp_active_key_index	PIB attribute 0x22: Index of the active GMK to be used for data transmission.
adp_metric_type	PIB attribute 0x03: Metric Type to be used for routing purposes.
adp_coord_short_ address	PIB attribute 0x08: Defines the short address of the coordinator.
adp_disable_default_ routing	PIB attribute 0xF0: If TRUE, the default routing (LOADng) is disabled. If FALSE, the default routing (LOADng) is enabled.
adp_device_type	PIB attribute 0x10: Defines the type of the device connected to the modem: enum: (0) PAN device, (1) PAN coordinator, (2) Not Defined
adp_default_coord_ route_enabled	PIB attribute 0x24: If TRUE, the adaptation layer adds a default route to the coordinator after successful completion of the bootstrapping procedure. If FALSE no default route will be created.
adp_destination_ address_set	PIB attribute 0x23: Contains the list of the addresses of the devices for which this LOADng router is providing connectivity. array D_address D_address ::= long-unsigned
	D_address: The 16-bit address of a destination node attached to this LOADng router and for which this LOADng router provides connectivity.

© Copyright 1997-2020 DLMS User Association

4.14 Interface classes for setting up and managing DLMS/COSEM HS-PLC ISO/IEC 12139-1 neighbourhood networks

4.14.1 Overview

COSEM objects for data exchange using DLMS/COSEM HS-PLC ISO/IEC 12139-1 neighbourhood networks, if implemented, shall be located in the Management Logical Device of COSEM servers.

For setting up and managing DLMS/COSEM HS-PLC ISO/IEC 12139-1 neighbourhood networks the following ICs are specified:

- HS-PLC ISO/IEC 12139-1 MAC setup, see 4.14.2;
- HS-PLC ISO/IEC 12139-1 CPAS setup, see 4.14.3;
- HS-PLC ISO/IEC 12139-1 IP SSAS setup, see 4.14.4;
- HS-PLC ISO/IEC 12139-1 HDLC SSAS setup, see 4.14.5.

4.14.2 HS-PLC ISO/IEC 12139-1 MAC setup (class_id = 140, version = 0)

Instances of the "HS-PLC ISO/IEC 12139-1 MAC setup" IC hold parameters necessary to set up and manage the MAC layer of the HS-PLC ISO/IEC 12139-1 profile.

HS-PLC ISO/IEC 12139-1 MAC setup		**0...n**	**class_id = 140 version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name		octet-string				x
2. group_id	(static)	long64-unsigned	0	2^{46}		x + 0x08
3. secondary_group_id	(static)	long64-unsigned	0	2^{46}		x + 0x10
4. station_id	(static)	long64-unsigned	0	2^{48}		x + 0x18
5. parent_station_id	(static)	long64-unsigned	0	2^{48}		x + 0x20
6. repeater_status	(static)	boolean				x + 0x28
7. encryption_mode	(static)	enum	1	2	1	x + 0x30
8. initial_encryption_key	(static)	octet-string				x + 0x38
9. rts/cts	(static)	boolean	0	1	0	x + 0x40
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “HS-PLC ISO/IEC 12139-1 MAC setup” object instance. See 6.2.30.
group_id	Holds the group identifier of HS-PLC ISO/IEC 12139-1. Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information.
secondary_group_id	Holds the secondary group identifier of HS-PLC ISO/IEC 12139-1. Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information.
station_id	Holds the station identifier of HS-PLC ISO/IEC 12139-1. Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information.

© Copyright 1997-2020 DLMS User Association

parent_station_id	Holds the parent station identifier* of HS-PLC ISO/IEC 12139-1. Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information. * The parent station means the directly neighbouring station from a station itself in a multi-stage HS-PLC ISO/IEC 12139-1 link from the station to NNAP (a source station – Repeater(s) –NNAP).
repeater_status	Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information. Holds the current repeater status of the device. boolean: FALSE = no repeater, TRUE = repeater
encryption_mode	Holds the encryption mode the PLC station uses. Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information. enum: (0) AES128, (1) Reserved.
initial_encryption_key	Encryption key which is used initially during PLC registration (cell join) period. Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information.
rts/cts	Holds the status of the RTS/CTS parameter. The RTS/CTS (Request To Send / Clear To Send) parameter is used to prevent collision caused by hidden HS-PLC ISO/IEC 12139-1 stations). Refer to ISO/IEC 12139-1:2009, Clause 7 for detailed information. RTS/CTS enable/disable boolean: FALSE = Disable, TRUE = Enable

4.14.3 HS-PLC ISO/IEC 12139-1 CPAS setup (class_id = 141, version = 0)

Instances of the "HS-PLC ISO/IEC 12139-1 CPAS setup" IC hold parameters necessary to set up and manage the CPAS layer of the HS-PLC ISO/IEC 12139-1 profile.

HS-PLC ISO/IEC 12139-1 CPAS setup		**0...n**	**class_id = 141 version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name		octet-string				x
2. cpas_address	(static)	long64-unsigned	0	2^{48}		x + 0x08
3. cpas_ether_type	(static)	long-unsigned				x + 0x10
4. master_station_cpas_address	(static)	long64-unsigned	0	2^{48}		x + 0x18
Specific methods		**m/o**				

Attribute description	
logical_name	Identifies the "HS-PLC ISO/IEC 12139-1 CPAS setup" object instance. See 6.2.30.
cpas_address	Holds the CPAS address of the HS-PLC ISO/IEC 12139-1 profile. See IEC 62056-8-6:2017, 5.4.2.

© Copyright 1997-2020 DLMS User Association

cpas_ether_type	Holds the EtherType value of the CPAS sublayer. See IEC 62056-8-6:2017, 5.4.2.
master_station_ cpas_address	Holds the master station's* CPAS address of the HS-PLC ISO/IEC 12139-1 profile. * The master station means the destination station of the CPAS frame (i.e. typically NNAP) in a multi-stage HS-PLC ISO/IEC 12139-1 link (a source station - Repeater(s) - A destination station).

4.14.4 HS-PLC ISO/IEC 12139-1 IP SSAS setup (class_id = 142, version = 0)

Instances of the "HS-PLC ISO/IEC 12139-1 IP SSAS setup" IC hold parameters necessary to set up and manage the IP SSAS of the HS-PLC ISO/IEC 12139-1 profile.

HS-PLC ISO/IEC 12139-1 IP SSAS setup	**0...n**	**class_id = 142, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	octet-string				x
2. ip_header_comp_type (static)	enum				x + 0x08
3. ip_alive_time (static)	long-unsigned		0xFFFF	0	x + 0x10
Specific methods	**m/o**				

Attribute description	
logical_name	Identifies the "HS-PLC ISO/IEC 12139-1 IP SSAS setup" object instance. See 6.2.30.
ip_header_comp_type	Holds the IP_Header_Comp_Type value as specified in IEC 62056-8-6:2017, Table 3. enum: (0) General IPv4 packet (No compression), (1) General IPv6 packet (No compression), (2) Van Jacobson header compression (RFC 1144), (3) IP header compression (RFC 2508), (4) ROHC (RFC 3095).
ip_alive_time	Holds the IP SSAS alive time value in seconds. 0 : No expiry

4.14.5 HS-PLC ISO/IEC 12139-1 HDLC SSAS setup (class_id = 143, version = 0)

Instances of the "HS-PLC ISO/IEC 12139-1 HDLC SSAS setup" IC hold parameters necessary to set up and manage the HDLC SSAS of the HS-PLC ISO/IEC 12139-1 profile.

© Copyright 1997-2020 DLMS User Association

HS-PLC ISO/IEC 12139-1 HDLC SSAS setup	0...n	class_id = 143, version = 0			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	octet-string				x
2. master_station_id (static)	long64-unsigned	0	2^{48}		x + 0x08
Specific methods	***m/o***				

Attribute description	
logical_name	Identifies the “HS-PLC ISO/IEC 12139-1 HDLC SSAS setup” object instance. See 6.2.30.
master_station_id	Holds the master station identifier of the HS-PLC ISO/IEC 12139-1 network.

© Copyright 1997-2020 DLMS User Association

4.15 ZigBee® setup classes

4.15.1 Overview

This subclause specifies COSEM interface classes required for the external configuration and management of a ZigBee® network to allow interfacing with a multi-part installation that internally uses ZigBee® communications. ZigBee® is a low-power radio communications technology and open standard that is operated by the ZigBee® Alliance, see www.zigbee.org.

ZigBee® is a registered trademark of the ZigBee® Alliance.

NOTE 1 A multi-part installation is one where the meter provides information and/or services to the householder on behalf of the utility. For example, the meter interacts with an in home display, and/or an external load control switch, and/or a smart appliance, to inform the customer of their usage in real time, to control heating devices, and possibly to disconnect peak loads when supply is constrained. While it is possible that the consumer will control the ZigBee® network, in normal operations the utility will control the radio system. This is to ensure that security is maintained for PAN, so that ZigBee® devices such as load switches controlled by the utility operate in a secure manner.

ZigBee® defines a local network of devices linked by radio, with routing and forwarding of messages and with encryption for privacy at network-level.

NOTE 2 Such a local network is known as a PAN – a Personal Area Network. This name is used in the ZigBee® community as ZigBee® is underpinned by the IEEE 802.15.4 standard, which uses the term PAN. This is broadly equivalent to a HAN (Home Area Network – name used in the context of smart metering in the UK) or PAN.

Each PAN has one device designated as ZigBee® coordinator, which has responsibility for creating and managing the network, and which normally acts also as a ZigBee® Trust Center for the management of ZigBee® network, PCLK's and APS Link keys.

There is a process of PAN creation (and corresponding destruction) which is performed by the coordinator; this declares the existence of the network without any devices apart from the coordinator forming part of it. Other ZigBee® devices can join a network created with cooperation of the coordinator, and equally can choose to leave, or can be invited to leave by the coordinator (this is not currently enforceable). Normally devices are members of the network indefinitely; they do not repeatedly join and leave. To create a PAN the coordinator has to receive an external trigger and needs to have setup information including:

- extended PAN ID;
- link keys or install code (for initial communication with new devices);
- radio channel information.

During the process of creating the PAN, the coordinator scans for nearby radio devices, "exchanges" keys, chooses the short addresses, and confirms use of radio channels. Details of the information available from the ZigBee® servers on each device are also exchanged.

NOTE 3 Full details of the joining process are documented in ZigBee® 053474, the ZigBee® specification. More information on ZigBee® technology can be sought at http://www.zigbee.org/.

Figure 30 shows an example architecture with a Comms hub on the left, that comprises a DLMS/COSEM server as well as the ZigBee® coordinator. The DLMS/COSEM server has interface objects needed to set up and control the ZigBee® network and may have other COSEM objects to support a metering application. Further, there are two native ZigBee® devices and another DLMS/COSEM server – an electricity meter or another meter type – on the right which is also a "normal" ZigBee® network device.

Any ZigBee® device can be "joined" to the network by remote control.

It is assumed that the Comms hub will also have a further network connection to the WAN, and it is assumed that this is managed by existing DLMS structures – e.g. PSTN, GSM/3G, PLC etc. – but this is out of scope of this document.

© Copyright 1997-2020 DLMS User Association

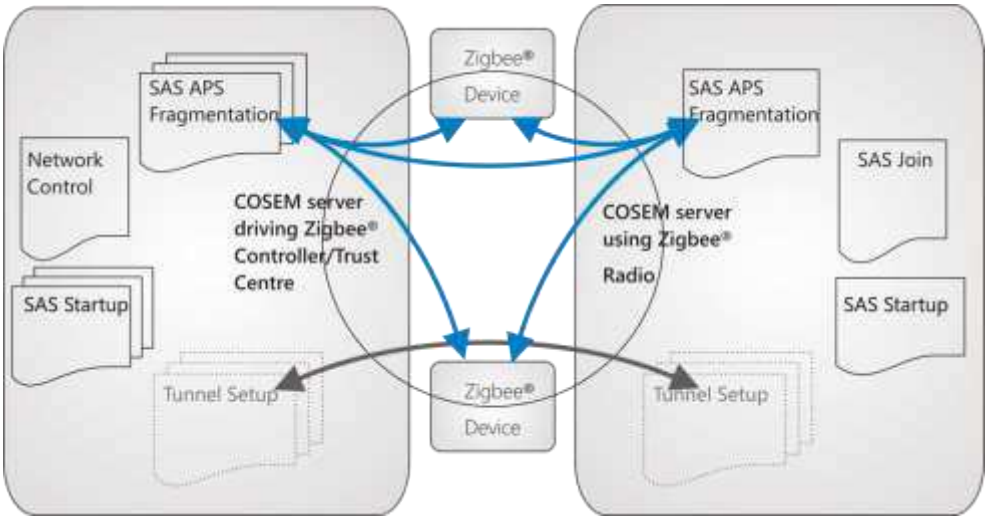


Figure 30 – Example of a ZigBee® network

The role of COSEM interface objects in the process of creating / destructing the PAN is to set up ZigBee® parameters and allow a DLMS/COSEM client to trigger actions, typically when commissioning the installation, in a system where WAN communications between a central system and a smart meter installation is by means of DLMS.

Operation of the ZigBee® network is not the responsibility of DLMS/COSEM. The DLMS/COSEM server is merely the vehicle for controlling the ZigBee® network by an external manager (DLMS/COSEM Client).

The use of ZigBee® setup classes in the ZigBee® coordinator and other DLMS/COSEM ZigBee® devices is shown in Table 35.

Table 35 – Use of ZigBee® setup COSEM interface classes

ZigBee® coordinator	Other DLMS/COSEM ZigBee® devices	Reference
ZigBee® SAS startup	ZigBee® SAS startup	4.15.2
–	ZigBee® SAS join	4.15.3
ZigBee® SAS APS fragmentation	ZigBee® SAS APS fragmentation	4.15.4
ZigBee® network control	–	4.15.5
Optionally: ZigBee® tunnel setup	Optionally: ZigBee® tunnel setup	4.15.6

This set of COSEM ICs supports the ZigBee® 2007 and ZigBee® PRO protocol stacks. The ZigBee® IP protocol stack is not supported at this time.

NOTE 4 In the specification of the ZigBee® COSEM ICs, the length of the octet-strings is indicated for information.

4.15.2 ZigBee® SAS startup (class_id = 101, version = 0)

Instances of this IC are used to configure a ZigBee® PRO device with information necessary to create or join the network. The functionality that is driven by this object and the effect on the network depends on whether the object is located in a ZigBee® coordinator or in another ZigBee® device.

© Copyright 1997-2020 DLMS User Association

ZigBee® SAS startup		0...n	class_id = 101, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. short_address	(dyn.)	long-unsigned			0xFFFF	x + 0x08
3. extended_pan_id	(dyn.)	octet-string			0	x + 0x10
4. pan_id	(dyn.)	long-unsigned			0xFFFF	x + 0x18
5. channel_mask	(dyn.)	double-long-unsigned			0	x + 0x20
6. protocol_version	(static)	unsigned	0x02		0x02	x + 0x28
7. stack_profile	(static)	enum			0x02	x + 0x30
8. start_up_control	(dyn.)	unsigned			2	x + 0x38
9. trust_center_address	(dyn.)	octet-string			0	x + 0x40
10. link_key	(dyn.)	octet-string			0	x + 0x48
11. network_key	(dyn.)	octet-string			0	x + 0x50
12. use_insecure_join	(static)	boolean			FALSE	x + 0x58
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “ZigBee® SAS startup” object instance. See 6.2.29.
short_address	Defines the 16-bit address by which this device is known locally on the ZigBee® network. Value 0x0000 is given to a coordinator / Trust Center device, and only to these devices. NOTE 1 This short address will be issued to the device by the coordinator when the device joins the network.
extended_pan_ID	Defines the unique address by which the network is known externally. NOTE 2 The length of the octet-string is 8 octets.
pan_ID	Defines the 16-bit address by which network is known locally.
channel_mask	Defines (as a bit mask) the set of radio channels which this PAN is permitted to use. Actual usage of channels within this set is determined by the coordinator on the basis of local conditions. The mask is as defined in the ZigBee® specification.
protocol_version	Defines the version of the ZigBee® protocol to be used on the network.
stack_profile	Identifies the capabilities of the ZigBee ® stack. enum: (1) ZigBee®, (2) ZigBee® PRO
start_up_control	Indicates the commissioning state of the ZigBee® device: 2: un-commissioned. Indicates that the device will seek to join the network if/when it is established; 0: commissioned. Indicates that the device should consider itself a part of the network indicated by the *extended_pan_id attribute*. In this case it will not perform any explicit join or rejoin operation. See NOTE 3 below.

© Copyright 1997-2020 DLMS User Association

trust_center_ address	Defines the unique extended address of the Trust Center used for this network. This is often, but not necessarily, the address of the ZigBee® coordinator. NOTE 3 The length of the octet-string is 8 octets.
link_key	Defines the key value used to secure point-to-point communications between particular devices within the Smart Energy Profile. The way in which the raw key value is protected is not defined in this specification. This is the APS link key or the PCLK. It's down to the implementation if this attribute is Hashed or in the clear or even if this is used. If this is the Trust Center then this is not usually implemented. This is usually read only, but may need to be writable for testing. NOTE 4 The length of the octet-string is 16 octets.
network_key	Defines the key value used to secure general communications between devices. The way in which the raw key value is protected is not defined in this specification. The network key value may be set internally by the coordinator. It's down to the implementation if this attribute is Hashed or in the clear or even if this is used. If this is the TC then this is not usually implemented. This is read only. NOTE 5 The length of the octet-string is 16 octets.
use_insecure_join	Indicates whether the coordinator is permitted to allow insecure joining as defined by ZigBee® specification.

NOTE 6 The "ZigBee® SAS startup" object reflects the state of the ZigBee® HAN to the WAN (or a diagnostic tool connected to a ZigBee® connected DLMS/COSEM Server). For example, if the object is in a Comms hub, then the attribute can be read out to show that the ZigBee® HAN has not been commissioned. The main function of this object is to provide information about a ZigBee® network to a client on the WAN (or via the optical port). The reason that there are multiple instances of the "ZigBee® SAS startup" object in Figure 29 is to express the idea that there may be multiple ZigBee® networks operated from a single Comms Hub.

4.15.3 ZigBee® SAS join (class_id = 102, version = 0)

Instances of this IC configure the behaviour of a ZigBee® PRO device on joining or loss of connection to the network. "ZigBee® SAS join" objects are present in all devices where a DLMS/COSEM server controls the ZigBee® Radio behaviour, but it is not used when a device is acting as coordinator (as the coordinator device creates rather than joins a network). "ZigBee® SAS join" objects can be factory configured, or configured using another communications technique – e.g. an optical port.

ZigBee® SAS join		**0...n**	**class_id = 102, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. scan_attempts	(static)	unsigned			3	x + 0x08
3. time_between_scans	(static)	long-unsigned			1	x + 0x10
4. rejoin_interval		long-unsigned			60	x + 0x18
5. rejoin_retry_interval	(static)	long-unsigned			900	x + 0x20
Specific methods		***m/o***				

Attribute description

© Copyright 1997-2020 DLMS User Association

logical_name	Identifies the “ZigBee® SAS join” object instance. See 6.2.29.
scan_attempts	Defines the number of consecutive scans that a ZigBee® device will perform on each attempt to rejoin a network, in the event of losing contact.
time_ between_ scans	Defines the period in seconds between consecutive scans in a single attempt to rejoin a network.
rejoin_interval	Defines the period in seconds that the device should wait after apparently becoming disconnected from a network, before attempting to rejoin.
rejoin_retry_ interval	Defines the period in seconds for which the device should wait after a failed attempt to rejoin a network before trying again.

Remarks on usage

At boot time or when instructed to join a network, the device should complete up to three (3) (Note: as per the default) scan attempts to find a ZigBee® coordinator or router with which to associate.

If a device has not been commissioned, this means that when the user presses a button or uses another methodology to instruct the device to join a network, it will scan all of the channels up to three times (as per the default) to find a network that allows joining. If it has already been commissioned, it should scan up to three times (Note: as per the default) to find its original PAN to join. (ZigBee® Pro devices should try to find a network using their original extended PAN ID and ZigBee® devices can only try to find a network using their original PAN ID).

Remark on the rejoin_retry_interval

Imposes an upper bound on the *rejoin_interval* parameter – this is restarted if device is touched by human user, i.e. by a button press. This parameter is intended to restrict how often a device will scan to find its network in case the network is no longer present and therefore a scan attempt by the device would always fail (i.e., if a device finds it has lost network connectivity, it will try to re-join the network, scanning all channels if necessary). If the scan fails to successfully re-join, the device will wait for 15 min before attempting to re-join again. To be network friendly, it would be recommended to adaptively extend this time period if successive re-joins fail. It would also be recommended the device should try a re-join when triggered (via a control, button, etc.) and fall back to the *rejoin_retry_interval* if attempts to re-join fail again.

4.15.4 ZigBee® SAS APS fragmentation (class_id = 103, version = 0)

Instances of this IC configure the fragmentation feature of ZigBee® PRO transport layer. This fragmentation is not of concern to COSEM; the object merely allows configuration of the fragmentation function by an external manager (DLMS/COSEM client).

Instances of this IC are present in all devices where a DLMS/COSEM server controls the ZigBee® Radio behaviour.

ZigBee® SAS APS fragmentation		**0...n**	**class_id = 103, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. aps_interframe_delay	(static)	long-unsigned			50	x + 0x08
3. aps_max_window_size	(static)	long-unsigned			1	x + 0x10
Specific methods		***m/o***				

Attribute description

© Copyright 1997-2020 DLMS User Association

logical_name	Identifies the “ZigBee® SAS APS fragmentation” object instance. See 6.2.29.
aps_interframe_ delay	Defines the delay in milliseconds between sending two blocks of a fragmented transmission.
aps_max_ window_size	Defines the maximum number of unacknowledged frames that can be transmitted consecutively. NOTE These initial values will allow for installation, and setting these values will depend on the specification of the ZigBee® Interface.

4.15.5 ZigBee® network control (class_id = 104, version = 0)

There will be a single instance of the “ZigBee® network control” IC in any device that can act as a ZigBee® coordinator controlled by the DLMS/COSEM client. This class allows interaction between a DLMS/COSEM client (head-end system) and a ZigBee® coordinator at times such as when the installation is commissioned.

ZigBee® network control	**0..1**	**class_id = 104, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. enable_disable_joining	boolean			FALSE	x + 0x08
3. join_timeout (static)	long-unsigned			60	x + 0x10
4. active_devices (dyn.)	array				x + 0x18
Specific methods	***m/o***				
1. register_device (data)	m				x + 0x20
2. unregister_device (data)	m				x + 0x28
3. unregister_all_devices (data)	o				x + 0x30
4. backup_PAN (data)	o				x + 0x38
5. restore_PAN (data)	o				x + 0x40
6. identify_device (data)	o				x + 0x48
7. remove_mirror (data)	o				x + 0x50
8. update_network_key (data)	o				x + 0x58
9. update_link_key (data)	o				x + 0x60
10. create_PAN (data)	m				x + 0x68
11. remove_PAN (data)	m				x + 0x70

Attribute description	
logical_name	Identifies the “ZigBee® network control” object instance. See 6.2.29.
enable_disable_ joining	A flag controlling whether devices are allowed to join the ZigBee® network. The flag is normally FALSE (joining disabled). At certain times – when a new device is expected to join – the flag is set externally to TRUE and then remains set for the duration of the *join_timeout*, or until externally reset if that happens sooner.
join_timeout	Defines the time period in seconds during which the coordinator device will permit joining of new devices, following setting of the *enable_disable_joining* flag.

© Copyright 1997-2020 DLMS User Association

active_devices

This attribute shows all of the currently authorised devices within the ZigBee® PAN.

array active_device

active_device ::= structure

```
{
    mac_address:                    octet-string,
    status:                         bit-string,
    maxRSSI:                        integer,
    averageRSSI:                    integer,
    minRSSI :                       integer,
    maxLQI:                         unsigned,
    averageLQI:                     unsigned,
    minLQI :                        unsigned,
    last_communication_date-time:   octet-string,
    number_of_hops:                 unsigned,
    transmission_failures:          unsigned,
    transmission_successes:         unsigned,
    application_version:            unsigned,
    stack_version:                  unsigned
}
```

NOTE 1 The length of the mac_address is 8 octets.

NOTE 2 The length of the status is 8 bits.

The Max/Average/Min values are taken over the last 24 h period for each device. Period is from 00:00:01 to 00:00:00. They are always historical i.e. they refer to the last day.

status ::= bit-string[8]

Bit 0 =	Authorised on PAN
Bit 1 =	Actively reporting on PAN
Bit 2 =	Unauthorised on PAN but has reported
Bit 3 =	Authorised after swap-out
Bit 4 =	SEP Transmitting
Bit 4 =	Reserved
Bit 6 =	Reserved
Bit 7 =	Reserved

Other elements are detailed in the ZigBee® specification.

© Copyright 1997-2020 DLMS User Association

Method description	

register_device (data)	This method is called externally to instruct the coordinator that a device should be added to the list of authorised devices. This method does not actually join the devices to the network, they are just authorised to join at some time in the future.
	data ::= structure
	{
	ieee_address: octet-string,
	key_type: enum,
	key: octet-string,
	device_type: enum,
	}
	
	Where:
	- ieee_address holds the IEEE address of the device;
	NOTE 3 The length of the octet-string is 8 octets;
	- key_type determines the type of content of key;
	enum:
	(0) Pre-configured Link Key,
	(1) Install code,
	(2)...(255) Reserved
	NOTE 4 Install Codes will be subject to agreement across the members who are implementing a ZigBee® network. Therefore the length of install codes, the use of a CRC, and how they are padded when transported in this method are subject to agreement as part of a project specific companion specification.
	- key is a pre-configured link key or install code. This will depend on the device being authorised to join the network. Its maximum length is 16 octets,
	- device_type is linked to the enumeration shown below so that the coordinator has a method of understanding the possible services that the joining device may require.
	NOTE 5 For example, it is likely that an implementation would require Mirror function for Gas, Water and Heat Meters connected by ZigBee®. However, determination of which ZigBee® support is needed for which device will be dependent on the organisation designing the smart metering solution, perhaps a government or a large utility.

© Copyright 1997-2020 DLMS User Association

register_device (data) (continued)

device-type ::= enum

Value	Description
(0)	Electricity Meter
(1)	Gas Meter
(2)	Water Meter
(3)	Thermal Meter
(4)	Pressure Meter
(5)	Heat Meter
(6)	Cooling Meter
(7)	Electric Vehicle charging Meter
(8)	PV Generation Meter
(9)	Wind Turbine Generation Meter
(10)	Water Turbine Generation Meter
(11)	Micro Generation Meter
(12)	Solar Hot Water Generation Meter
(13)	ZigBee® Controlled Load Switch
(14)	ZigBee® Based Boost Button
(128)	IHD
(129)	Range extender
(130)	CAD (Consumer Access Device)
(131)	Thermostat
(132)	Prepayment Terminal
(133)	ZigBee® Controlled Load Switch
(134)	ZigBee® Based Boost Button

unregister_ device (data)

This method is called externally to instruct the coordinator that a device should leave the network.

data ::= octet-string

It holds the ieee_address.

NOTE 6 The length of the octet-string is 8 octets.

unregister_all_ devices (data)

This method is called externally to instruct the coordinator that all devices should leave the network.

data ::= integer (0)

NOTE 7 The likely use of this function is to ensure a network is empty of devices prior to destroying it.

© Copyright 1997-2020 DLMS User Association

backup_PAN (data)

This method instructs the coordinator to create a back-up of information that would be necessary to re-create the PAN.

NOTE 8 The storage location of the back-up is not currently defined and is an internal function of the DLMS/COSEM server.

data ::= integer (0)

Method invocation return parameters are detailed below:

```
data ::= structure
{
     date_time:              octet-string,
     extended_PAN_ID:        octet-string,
     devices_to_backup:      array device_to_backup
}
```

Where:

- date-time refers to the time at which the backup process is due to begin. It is formatted as specified in 4.1.6.1;
- extended_PAN_ID identifies the PAN.

  NOTE 9 The length of the octet-string is 8.

```
device_to_backup ::= structure
{
     MAC_address:            octet-string,
     hashed_TC_link_key:     octet-string
}
```

Where:

- MAC_address hold the MAC address.

  NOTE 10 The length of the octet-string is 8;

  NOTE 11 The length of octet-string holding the hashed_TC_link_key is 16 octets.

NOTE 12 The method of hashing the link key is not part of this specification; it is defined in the ZigBee® Smart Energy Specification. MMO is currently used.

restore_PAN (data)

This method instructs the coordinator to restore a PAN using backup information. The storage location of the back-up is not currently defined and is an internal function of the DLMS/COSEM Server.

data ::= structure

```
{
     extended_PAN_ID:        octet-string,
     devices_to_restore:     array     device_to_restore
}
```

Where:

- extended_PAN_ID identifies the PAN.

  NOTE 13 The length of the octet-string is 8;

© Copyright 1997-2020 DLMS User Association

restore_PAN (data) (continued)	device_to_restore ::= structure
	{
	MAC_address: octet-string,
	hashed_TC_link_key: octet-string
	}
	
	Where:
	- MAC_address holds the MAC address.
	NOTE 14 The length of the octet-string is 8;
	NOTE 15 The length of octet-string holding the hashed_TC_link_key is 16 octets.
	
	NOTE 16 The method of hashing the link key is not part of this specification; it is defined in the ZigBee® Smart Energy Specification. MMO is currently used.

identify_device (data)	This method is called externally to instruct a device to identify itself to an engineer present on site, for example by sounding a buzzer. data ::= ieee_address ieee_address: octet-string ieee_address holds the IEEE address of the device. NOTE 17 The length of the octet-string is 8 octets.

remove_mirror (data)	This method causes the removal of a ZigBee® mirror that reflects the real device identified by the mac_address parameter.
	data ::= structure
	{
	mac_address: octet-string,
	mirror_control: bit-string
	}
	Where:
	- MAC_address holds the MAC address.
	NOTE 18 The length of the octet-string is 8.
	- the mirror_control parameter is provided to support the execution of implementation-specific actions, which should be defined in a project specific companion specification.

© Copyright 1997-2020 DLMS User Association

remove_mirror (continued)

EXAMPLE The following example is one of the possible options for the bit-string functionality.

0 =	Force Gas Meter Removal. NOTE 19 Where a device removal is forced, the keys values for this device are removed; an APS ack from the device is not required.
1 =	Clear all Mirror Data
2 =	Clear Consumption registers / indexes
3 =	Clear Demand & Max Demand registers
4 =	Clear ZigBee® attributes
5 =	Clear MPAN
6 =	Clear Billing Information
7 =	Clear Logs
8 =	Clear OTA Firmware waiting
9…14 =	Reserved
15 =	Action all

Should the bit be set then the action is carried out.

The full meaning and actions that the DLMS/COSEM server should undertake are implementation-specific actions, which should be defined in a project specific companion specification.

update_ network_key (data)

This method requests that the ZigBee® coordinator updates the network key and propagate it to all devices on the PAN. For details of ZigBee® key management, see the ZigBee® specification.

data ::= integer (0)

update_link_key (data)

This method requests that the ZigBee® coordinator updates the link key and propagate it to the identified device on the PAN. For details of ZigBee® key management, see the ZigBee® specification.

data ::= ieee_address

ieee_address: octet-string

ieee_address holds the IEEE address of the device.

NOTE 20 The length of the octet-string is 8 octets.

create_PAN (data)

This method is called externally to instruct a coordinator to create a network using the configuration held in the ZigBee® SAS startup object.

data ::= integer (0)

remove_PAN (data)

This method is called externally to instruct a coordinator to destroy a network by turning off the ZigBee® radio and removing all of the settings associated with the current PAN.

data ::= integer (0)

4.15.6 ZigBee® tunnel setup (class_id = 105, version = 0)

A ZigBee® tunnel is established between two ZigBee® PRO devices to allow DLMS APDUs to be transferred between them. The tunnel in effect extends WAN connectivity to ZigBee® devices not connected to the WAN through a ZigBee® device connected to the same ZigBee® network and connected to the WAN.

The ZigBee® tunnel setup objects would be present on the coordinator and on all other DLMS/COSEM devices that are not connected to the WAN.

© Copyright 1997-2020 DLMS User Association

Creation of the tunnel is managed on demand and invisibly from the point of view of the DLMS/COSEM client. The target device is implicitly identified by the COSEM addressing information.

NOTE 1 See also the Gateway specification in DLMS UA 1000-2 Ed. 10, 10.7.

ZigBee® tunnel setup		**0...n**	**class_id = 105, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. maximum_incoming_transfer_size	(static)	long-unsigned			0x05DC	x + 0x08
3. maximum_outgoing_transfer_size	(static)	long-unsigned			0x05DC	x + 0x10
4. protocol_address	(static)	octet-string length			0	x + 0x18
5. close_tunnel_timeout	(static)	long-unsigned			0xFFFF	x + 0x20
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “ZigBee® tunnel setup” object instance. See 6.2.29.
maximum_ incoming_ transfer_size	Defines the maximum size, in octets, of the data packet that can be transferred to the tunnel client in the payload of a single **TransferData** (TransferData is a ZigBee® parameter) command. Behaviour on receipt of a larger packet is not defined in this specification.
maximum_ outcoming_transfer_ size	Defines the maximum size, in octets, of the data packet that can be sent from the tunnel client in the payload of a single **TransferData** command. Behaviour on transmission of a larger packet is not defined in this specification. See NOTE 2 below.
protocol_address	The *protocol_address* is implementation-specific, which should be defined in a project specific companion specification. The length of the octet-string is 6 octets.
close_tunnel_ timeout	Defines the time, in seconds that the ZigBee® server waits before closing an inactive tunnel on its own (without waiting for the *CloseTunnel* Command from the client within the ZigBee® specification) and freeing its resources The timer is re-started with each reception of a command.

NOTE 2 The word *outcoming* is unusual, but is adopted for commonality with ZigBee® specifications.

© Copyright 1997-2020 DLMS User Association

4.16 Interface classes for setting up and managing the DLMS/COSEM profile for LPWAN networks

4.16.1 General

This clause specifies ICs for setting up devices using the DLMS/COSEM LPWAN communication profile and to diagnose the LPWAN network:

- the ICs specified in 4.16.2 are used to define setup and diagnostic objects for the SCHC-LPWAN part;
- the ICs specified in 4.16.3 are used to define setup and diagnostic objects for the lower layers.

For each specific LPWAN technology, a setup and a diagnostic IC should be specified. ICs for the LoRaWAN technology are specified in 4.16.3.2.

4.16.2 Generic interface classes

4.16.2.1 SCHC-LPWAN setup (class_id = 126, version = 0)

Instances of this IC are available for configuring the parameters needed to set up a LPWAN device.

SCHC-LPWAN setup		**0..n**	**class_id = 126 , version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. lpwan_reference	(static)	octet-string				x + 0x08
3. schc_cd_rules	(static)	array				x + 0x10
4. schc_fr_param	(static)	structure				x + 0x18
Specific methods (if required)		***m/o***				

Attribute description	
logical_name	Identifies the “LPWAN setup” object instance. See 6.2.23.
lpwan_reference	References an LPWAN technology specific setup object.

schc_cd_rules	Contains the necessary parameters to support LPWAN Static Context Header Compression (SCHC) compression and decompression rules
	array schc_cd_rules_element
	schc_cd_rules_element ::= structure
	{
	rule_id: unsigned,
	field_descriptors: array field_descriptor
	}
	
	field_descriptor ::= structure
	{
	field_id: field_id_type,
	field_length: unsigned,
	field_position: unsigned,
	direction_indicator: direction_indicator_type,
	target_value: CHOICE
	{
	-- simple data types
	null-data [0],
	boolean [3],

© Copyright 1997-2020 DLMS User Association

```
                bit-string                  [4],
                double-long                 [5],
                double-long-unsigned        [6],
                octet-string                [9],
                visible-string              [10],
                utf8-string                 [12],
                bcd                         [13],
                integer                     [15],
                long                        [16],
                unsigned                    [17],
                long-unsigned               [18],
                long64                      [20],
                long64-unsigned             [21],
                enum                        [22],
                float32                     [23],
                float64                     [24],
                date-time                   [25],
                date                        [26],
                time                        [27],

                -- complex data types
                array                       [1],
                structure                   [2],
                compact-array               [19]
        },

    matching_operator:                      matching_operator_type,
    compression_decompression_action:
    compression_decompression_action_type
}

field_id_type ::= enum
{
        (0)     IPv6-version,
        (1)     IPv6-DiffServ,
        (2)     IPv6-FlowLabel,
        (3)     IPv6-Length,
        (4)     IPv6-NextHeader,
        (5)     IPv6-HopLimit,
        (6)     IPv6-Dev-Prefix,
        (7)     IPv6-DevIID,
        (8)     IPv6-AppPrefix,
        (9)     IPv6-AppIID,
        (10)    UDP-DevPort,
        (11)    UDP-AppPort,
        (12)    UDP-Length,
        (13)    UDP-Checksum,
        (14)    DLMS-Wrapper-version,
        (15)    DLMS-Wrapper-SSAP,
        (16)    DLMS-Wrapper-CSAP,
        (17)    DLMS-Wrapper-Length
}

direction_indicator_type ::= enum
{
        (0)     Uplink,
        (1)     Downlink,
```


© Copyright 1997-2020 DLMS User Association

```
        (2)     bidirectional
}

matching_operator_type ::= enum
{
        (0)     equal,
        (1)     ignore,
        (2)     MSBx,
        (3)     match-mapping
}

compression_decompression_action_type::= enum
{
        (0)     not-sent,
        (1)     value-sent,
        (2)     mapping-sent,
        (3)     LSB,
        (4)     compute-length,
        (5)     compute-checksum,
        (6)     DevIID,
        (7)     AppIID,
}
```

schc_fr_param

Contains the necessary parameters to support LPWAN Static Context Header Compression (SCHC) fragmentation and reassembly parameters.

```
schc_fr_param ::= structure
{
        rule_id_scheme:         enum
                                {
                                        (0) fixed-size,
                                        (1) variable-size
                                },
        max_packet_size:        long-unsigned,
        padding_l2_word_size:   unsigned,
        padding_bits_value:     unsigned,
        delay_after_transmission: long-unsigned,
        interleaved_packet_tran: boolean,
        window_size:            unsigned,
        rule_params:            array rule_param_element

}

rule_param_element ::= structure
{
        rule_id:                unsigned,
        rule_id_len:            unsigned,
        dtag_len:               unsigned,
        w_len:                  unsigned,
        fcn_len:                unsigned,
        rcs_algorithm:          enum
                                {
                                        (0) None,
                                        (1) CRC32 using 0xEDB88320
                                },
```


© Copyright 1997-2020 DLMS User Association

```
        reliability_mode:  enum
                                {
                                    (0) No-ACK,
                                    (1) ACK-on-Error,
                                    (2) ACK-Always
                                },
        retransmission_timer:   long-unsigned,
        inactivity_timer:       long-unsigned,
        max_ack_request:        unsigned
}
```

Where:

rule_id_scheme	The RuleID numbering scheme, fixed-size or variable-size Rule IDs, the way the Rule ID is transmitted;
max_packet_size	The maximum packet size that should ever be reconstructed by SCHC Decompression [octets];
padding_l2_word_size	The size of the L2 Word [bits];
padding_bits_value	The value of the padding bits (0 or 1);
delay_after_transmission	The delay to be added after transmission [milliseconds];
interleaved_packet_tran	The support for interleaved packet transmission;
window_size	The WINDOW_SIZE, for modes that use windows;
rule_id_len	The length in bits for the RuleID header field of SCHC F/R message [bits];
dtag_len	The length in bits for the Datagram Tag (DTag) header field of SCHC F/R message [bits];
w_len	The length in bits for the W header field of SCHC F/R message [bits];
fcn_len	The length in bits for the Fragment Compressed Number (FCN) header field of SCHC F/R message [bits];
rcs_algorithm	The algorithm for the calculation of Reassembly Check Sequence (RCS) field of SCHC F/R message;
reliability_mode	The reliability mode used;
retransmission_timer	The Retransmission Timer duration [milliseconds];
inactivity_timer	The Inactivity Timer duration [milliseconds];
max_ack_request	The MAX_ACK_REQUEST value.

© Copyright 1997-2020 DLMS User Association

C/D rules defined for the DLMS over SCHC profile are listed in Table 36.

Table 36 – C/D Rule 1

field_id	field_ length	field_ position	direction_ indicator	target_ value	matching_ operator	compression_ decomprassion_ action
IPv6-version	4	1	bidirectional	6	equal	not-sent
IPv6-DiffServ	8	1	bidirectional	0	equal	not-sent
IPv6-FlowLabel	20	1	bidirectional	0	equal	not-sent
IPv6-Length	16	1	bidirectional		ignore	comp-length
IPv6-NextHeader	8	1	bidirectional	17	equal	not-sent
IPv6-HopLimit	8	1	bidirectional	255	equal	not-sent
IPv6-Dev-Prefix	64	1	bidirectional	FE80::/64	equal	not-sent
IPv6-DevIID	64	1	bidirectional		ignore	DevIID
IPv6-AppPrefix	64	1	bidirectional	FE80::/64	equal	not-sent
IPv6-AppIID	64	1	bidirectional	1	ignore	not-sent
UDP-DevPort	16	1	bidirectional	4059	equal	not-sent
UDP-AppPort	16	1	bidirectional	4059	equal	not-sent
UDP-Length	16	1	bidirectional		ignore	comp-length
UDP-Checksum	16	1	bidirectional		ignore	comp-chk
DLMS-Wrapper-version	16	1	bidirectional	1	equal	not-sent
DLMS-Wrapper-SSAP	16	1	bidirectional	1	equal	not-sent
DLMS-Wrapper-CSAP	16	1	bidirectional	1	equal	not-sent
DLMS-Wrapper-Length	16	1	bidirectional		ignore	comp-length
NOTE If the device has to manage a second client, a second rule can be defined where the only field-id that is different will be DLMS-Wrapper-CSAP.						

4.16.2.2 SCHC-LPWAN diagnostic (class_id = 127, version = 0)

Instances of this IC are intended to provide diagnostic information regarding the transport of SCHC packets over the LPWAN network that can be useful at the application layer to investigate potential communications problems and service performance.

SCHC-LPWAN diagnostic		**0..n**	**class_id = 127 , version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name.	(static)	octet-string				x
2. schc_packets_tx_counter	(dynamic)	double-long-unsigned			0	x + 0x08
3. schc_packets_rx_counter	(dynamic)	double-long-unsigned			0	x + 0x10
4. schc_fragments_tx_counter	(dynamic)	double-long-unsigned			0	x + 0x18
5. schc_fragments_rx_counter	(dynamic)	double-long-unsigned			0	x + 0x20
6. schc_ack_tx_counter	(dynamic)	double-long-unsigned			0	x + 0x28
7. schc_ack_rx_counter	(dynamic)	double-long-unsigned			0	x + 0x30
Specific methods (if required)		***m/o***				
1. reset (data)		...				x + 0x40

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “LPWAN Diagnostic” object instance. See 6.2.23.
schc_packets_tx_ counter	It counts the SCHC non fragmented packet transmitted by the device.
schc_packets_rx_ counter	It counts the SCHC non fragmented packet received by the device.
schc_fragments_tx_ counter	It counts the SCHC fragments transmitted by the device.
schc_fragments_rx_ counter	It counts the SCHC fragments received by the device.
schc_ack_tx_counter	It counts the ACK frame transmitted by the device.
schc_ack_rx_counter	It counts the ACK frame received by the device.
Method description	
reset (data)	This method resets all counters to 0. data::= integer (0)

© Copyright 1997-2020 DLMS User Association

4.16.3 LPWAN specific interface classes

4.16.3.1 General

Interface classes specified in this clause are related to a given LPWAN technology. Currently, interface classes related to LoRaWAN network are specified. ICs for other LPWAN technologies will be added later.

4.16.3.2 LoRaWAN® interface classes

4.16.3.2.1 General

These ICs define the LoRaWAN parameters used during registration and deregistration process, needed at the device side.

NOTE 1 DEVEUI is defined using the MAC address setup interface class (class_id = 43).

NOTE 2 ABP (Activation by Personalization is not permitted).

4.16.3.2.2 LoRaWAN setup (class_id = 128, version = 0)

Instances of this interface class hold the parameters needed to set up a LoRaWAN device.

This interface class mirrors the parametrization published in LoRa Alliance document : *LoRaWAN-TR006 DLMS End Device Monitoring Guidelines.doc.*

LoRaWAN® setup		**0..n**	**class_id = 128 , version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. class	(dynamic)	enum				x + 0x08
3. state	(dynamic)	enum				x + 0x10
4. max_transmit_EIRP_setting	(static)	integer				x + 0x18
5. ADR_mode	(dynamic)	boolean				x + 0x20
6. regional_parameters	(dynamic)	enum				x + 0x28
7. device_operation	(dynamic)	structure				x + 0x38
8. modem_versions	(static)	structure				x + 0x40
9. devAddr	(dynamic)	double-long-unsigned				x + 0x48
10. join_strategy	(dynamic)	enum				x + 0x50
11. multicasts_parameters	(dynamic)	array				x + 0x58
Specific methods (if required)		***m/o***				
1. disconnect_from_network()		...				x + 060
2. change_class(data)		...				x + 068
3. change_region(data)		...				x + 0x70

Attribute description	
logical_name	Identifies the “LoRaWAN setup” object instance. See 6.2.23.
class	Define the class of the device. Read only attribute, use change_class method to change the value. enum: (0) Class A, (1) Class B, (2) Class C

© Copyright 1997-2020 DLMS User Association

state	Shows the state of the network connection process of the device. enum: (0) not connected, (1) connecting state (joining), (2) not connected, connected at least one time, (3) connected, (4) connection error (join error)
max_transmit_ EIRP_setting	Read only parameter. Maximum transmission capability of the device in dBm.
ADR_mode	**A**daptative **D**ata **R**ate is a LoRaWAN parameter allowing the network server to control the device datarate and TX power. Recommended for static devices (devices expected to not roam between networks); it optimises energy consumption and network capacity. ADR is disabled when ADR_Mode is set to FALSE. ADR is enabled when ADR_Mode is set to TRUE.

regional_ parameters	Identifies the list of region names that a network and a device need to know, to open a LoRaWAN service. Those regions are used to support local radio regulation specifics, like duty cycle (or LBT), max time over air per message, max power, …
	They also govern the selection of default radio channels, default radio settings, the channel mode (fixed or dynamic, …)
	
	regional_parameters := enum
	{
	(0) EU868,
	(1) US915,
	(2) CN779 (Deprecates 2021-01-01),
	(3) EU433,
	(4) AU915,
	(5) CN470,
	(6) AS923-1,
	(7) AS923-2,
	(8) AS923-3,
	(9) KR920,
	(10) IN865,
	(11) RU864
	}

device_operation	Counters that can help to identify issues.
	device_operation ::= structure
	{
	TotalJoinRequestCounter: long unsigned,
	TimeSinceLastJoinRequest: long unsigned,
	TimeSinceLastJoinAccept: long unsigned
	}
	TotalJoinRequestCounter: Number of join requests since last reset.

© Copyright 1997-2020 DLMS User Association

	TimeSinceLastJoinRequest: Number of seconds elapsed since the device initiated the connection process. TimeSinceLastJoinAccept Number of seconds elapsed since the device connected to the network.

modem_versions	Identifies the modem hardware, software and protocol versions and the version of the regional parameters.
	
	versions ::= structure
	{
	HardwareVersion: CHOICE
	{
	octet-string,
	visible-string
	},
	SoftwareVersion: CHOICE
	{
	octet-string,
	visible-string
	},
	RegionalParametersVersion: CHOICE
	{
	octet-string,
	visible-string
	},
	ProtocolVersion: CHOICE
	{
	octet-string,
	visible-string
	}
	}

devAddr	32 bits device address identifying the device inside the current network. Assigned by the Network Server of the device. Should be 0 when not joined.
join_strategy	Defines the algorithm, timing/DR to apply during the joining. To be documented by vendor. Vendor can assign a code to a Join strategy (where applying).
multicasts_ parameters	Both DLMS, IP and LoRaWAN are multicast technologies, LoRaWAN supporting up to four multicast groups. One or more group(s) can be created for a set of DLMS devices then SCHC gateway will forward all IP multicast data corresponding to that group to the end devices. LoRa Alliance has published a recommended application layer[3] messaging definition allowing to remotely manage devices groups, and program class B or C distribution window for a given group. multicasts_parameters ::= array multicast_parameters

[3] https://lora-alliance.org/resource-hub/lorawanr-remote-multicast-setup-specification-v100

© Copyright 1997-2020 DLMS User Association

multicast_parameters ::= structure
{

mc_addr:	double-long-unsigned,
mc_key:	octet-string,
mc_min_fcount:	double-long-unsigned,
mc_max_fcount:	double-long-unsigned,
mc_start_time:	date-time,
mc_duration:	long-unsigned,
mc_class:	enum,
mc_downlink_datarate:	unsigned-integer
mc_downlink_frequency:	double-long-unsigned

}
Where:

mc_addr:	multicast device address; Note: Should be 0 when multicast group is not used,32 bits
mc_key:	key used to encode the payload of the multicast group; Note: 128 bits
mc_min_fcount:	minimum multicast Fcounter, device should drop payload with lower FCount;
mc_max_fcount:	maximum multicast Fcounter, device should drop payload with higher FCount
mc_start_time:	Time to start the multicast session, if applicable
mc_duration:	Maximum duration of the multicast session
mc_class:	Device shall switch to the class during the length of the multicast window enum: Class B, Class C All others reserved
mc_downlink_datarate:	Datarate of the multicast session
mc_downlink_frequency:	Frequency of the multicast session, in Hz

An end-device supports a maximum of 4 simultaneous groups, and a minimum of 0. Up to 4 elements may be used in the array.

Method description	
disconnect_from_network()	This method correspond to a LoRaWAN new join procedure. Data::= integer (0)
change_class	This method is used to change the class attribute. Data is an enum.

© Copyright 1997-2020 DLMS User Association

(data)	
change_region (data)	This method is used to change locally the Regional Parameter. Can be used only in disconnected state.

4.16.3.3 LoRaWAN diagnostic (class_id = 129, version = 0)

Instances of this IC are intended to provide diagnostic information regarding the LoRaWAN lower layers to investigate potential communications problems and service performance.

LoRaWAN diagnostic		**0..n**	**class_id = 129 , version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. internal_error_code	(dynamic)	enum				x + 0x08
3. out_frames_u_counter	(dynamic)	double-long-unsigned			0	x + 0x10
4. out_frames_c_counter	(dynamic)	double-long-unsigned			0	x + 0x18
5. in_frames_u_counter	(dynamic)	double-long-unsigned			0	x + 0x20
6. in_frames_c_counter	(dynamic)	double-long-unsigned			0	x + 0x28
7. in_mac_command_counter	(dynamic)	double-long-unsigned			0	x + 0x30
8. in_mac_ans_error_counter	(dynamic)	double-long-unsigned			0	x + 0x38
9. in_mac_ignored_counter	(dynamic)	double-long-unsigned			0	x + 0x40
10. in_per	(dynamic)	integer				x + 0x48
11. in_mean_rssi_rx1	(dynamic)	integer				x + 0x50
12. in_mean_snr_rx1	(dynamic)	integer				x + 0x58
13. in_mean_rssi_rx2	(dynamic)	integer				x + 0x60
14. in_mean_snr_rx2	(dynamic)	integer				x + 0x68
Specific methods (if required)		***m/o***				
1. reset (data)		...				x + 0x70

Attribute description	
logical_name	Identifies the “LPWAN Diagnostic” object instance. See 6.2.23.
internal_error_code	Fault condition identifier. To be documented by vendor. When an end-device is faulty, vendor can assign a code to the fault condition.
out_frames_u_counter	Number of frames sent unconfirmed.
out_frames_c_counter	Number of frames sent confirmed.
in_frames_u_counter	Number of frames received unconfirmed.
in_frames_c_counter	Number of frames received confirmed.
In_mac_command_counter	Number of MAC command received from the network. Note : all commands are answered.
in_mac_ans_error_counter	Number of MAC answers replied with error.

© Copyright 1997-2020 DLMS User Association

in_mac_ans_ignored_ counter	Number of MAC commands ignored.
in_per	Per : packet error rate. Computed over the 5 received downlink Unit is percentage (56).
in_mean_rssi_rx1	Computed over the last 5 received downlink on rx1. Unit is dBm
in_mean_snr_rx1	Computed over the last 5 received downlink on rx1. Unit is dB
in_mean_rssi_rx2	Computed over the last 5 received downlink on rx2. Unit is dBm
in_mean_snr_rx2	Computed over the last 5 received downlink on rx2. Unit is dB
Method description	
reset (data)	data:: = Integer (0) This method resets all counters to 0.

© Copyright 1997-2020 DLMS User Association

4.17 Interface classes for setting up and managing the DLMS/COSEM profile for Wi-SUN networks

4.17.1 Wi-SUN setup (class_id = 95, version 0)

This IC has the purpose of modelling the Wi-SUN network setup parameters.

All attributes in [FANSPEC] and [PHYSPEC] are written in camelCase and for the benefit of easy tracability these parameters are transposed into snake_case (underscore notation) within the DLMS/COSEM specification.

Wi-SUN setup		**0..n**	**class_id = 95, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. network_name	(static)	octet-string				x + 0x08
3. routing_method	(dynamic)	enum	0	1	1	x + 0x10
4. pan_id	(dynamic)	long-unsigned	0	65534		x + 0x18
5. disc_imin	(static)	unsigned	1	255		x + 0x20
6. disc_imax	(static)	unsigned	1	8		x + 0x28
7. data_message_imin	(static)	unsigned	1	255	10	x + 0x30
8. data_message_imax	(static)	unsigned	1	8	3	x + 0x38
9. default_dio_interval_min	(static)	unsigned	1	255		x + 0x40
10. default_dio_interval_doublings	(static)	unsigned	1	8		x + 0x48
11. channel_plan	(static)	structure				x + 0x50
12. channel_function	(static)	enum	0	3		x + 0x58
13. excluded_channels	(static)	CHOICE				x + 0x60
14. join_state	(dynamic)	enum	1	5		x + 0x68
15. reg_channel_exclusions	(static)	structure				x + 0x70
Specific methods (if required)		***m/o***				
1. reset_join_state (data)		*o*				x + 0x78

Attribute description	
logical_name	Identifies the “Wi-SUN setup” object instance. See clause 6.2.32.
network_name	Provides the name of the network that the device is allowed to join, or is a member of. This is represented by ASCII characters. NOTE 1 The network name has a maximum of 32 octets.
routing_method	Provides the routing method employed by the device, two options are available: enum: (0) Layer 2 routing, (1) Layer 3 routing NOTE 2 Default value according to [FANSPEC] is 1.

© Copyright 1997-2020 DLMS User Association

pan_id	Provides the PAN ID (configured at the Border Router and is known by every node on the network. This attribute is not controlled in the application layer. NOTE 3 See [FANSPEC], 6.3.1.1. NOTE 4 IEEE 802.15.4 reserves the value 65 535 for the broadcast PAN ID.
disc_imin	Provides the time in seconds of the time-out for network discovery. NOTE 5 Typical values of 60s for large PANs (larger than 100 devices) or 15s for small PANs (less than 100 devices) may be employed.
disc_imax	Provides the number of doublings of the value of “imin” (1-8 doublings of disc_imin). NOTE 6 Typical value of 4 for large PANs (approximately 16 minutes), value of 2 for small PANs (approximately 60 seconds).
data_message_ imin	Provides the network broadcast timeout minimum value in seconds. NOTE 7 Typical default value of 10s.
data_message_ imax	Provides the maximum broadcast timeout as a function of data_message_imin (1-8 doublings of data_message_imin). NOTE 8 Typical default value of 3 (80 seconds).
default_dio_ interval_min	Provides the ROLL RPL DIO minimum Interval (imin) The DIO trickle timer imin value is defined as 2 to the power of this value in ms (millseconds). NOTE 9 Typical default value for a large network: 19 (an imin of approximately 8 minutes) and a small network: 15 (an imin of approximately 32 seconds).
default_dio_interval_ doublings	Provides the ROLL RPL DIO Interval IMAX and is a number of doublings of the value of default_dio_interval_min (1-8 doublings). A value of 1 for large PANs (approximately 16 minutes i.e. 8 x 2 minutes), value of 2 for small PANs (approximately 128 seconds i.e 32 x 2 x 2).

channel_plan	Regional spectrum settings as specified in [PHYSPEC],Table 3. The values of regulatory_domain and operating_class uniquely identify the combination of operating parameters that the implementation is programmed to sustain. These parameters are subject to geographic regulation and may change from time to time depending on when regional spectrum authorities update their national frequency tables.
	
	channel_plan ::= structure
	{
	regulatory_domain_identifier: unsigned,
	operating_class_designator: unsigned
	}
	Where:
	– regulatory_domain_dentifier identifies the regulatory domain that the product is operating in, thereby identifying various PHY layer parameters including frequency bands, PHY modes, and channel spacing according to [PHYSPEC], Table 3,
	– operating_class_designator identidfies the operating class of the device according to [PHYSPEC], Table 3. Operating class is unique only within the regulatory domain.
	NOTE 10 The above sets the channel spacing, channel 0 frequency and number of channels plus the symbol rate.

© Copyright 1997-2020 DLMS User Association

channel_function

Provides the channel function. There are currently 4 channel functions: fixed channel or a channel hopping mode where certain other parameters are specified as enumerated below.

enum:

Value	Description	Note
0	Fixed Channel	The transmitting node uses a single, fixed channel from the available channels.
1	TR51CF	The transmitting node uses the channel function described in ANSI/TIA-4957.200 (TR51CF), 7.1.
2	DH1CF	The transmitting node uses the Direct Hash channel function (DH1CF described in [FANSPEC], 6.3.4.5).
3	Vendor defined	The transmitting node uses a vendor defined channel function explicitly provided by the Hop Count and Hop List (defined in [FANSPEC] 6.3.2.3.2.1.4).
Other	reserved	

excluded_channels

Provides a means via which a node can have either a range (excluded_channel_ranges) or individual channels (excluded_channel_mask) notched out (not visited) of the node's channel hop sequence. Excluded_channel_ranges and excluded_channel_mask are configured for device specific purposes and shall be a subset of the channel_plan that fulfils regulatory channel restriction requirements. If no excluded channel ranges are set then the attribute shall be set to null-data.

Either excluded_channel_range OR excluded_channel_mask may be configured in a mutually exclusive way.

```
excluded_channels: CHOICE
{
        null-data [0],
        excluded_channel_ranges,
        excluded_channel_mask
}
```

null-data is selected when there are no excluded channels.

```
excluded_channel_ranges::= structure
{
        number_of_ranges:       integer,
        ranges_list:            array channel_range
}
        channel_range::= structure
        {
                start_channel_number:       long unsigned,
                end_channel_number:         long unsigned
        }
```

Where:

© Copyright 1997-2020 DLMS User Association

– number_of_ranges is the number of ranges in the ranges_list;

– ranges_list is an array of start and end channel numbers representing one or multiple excluded channel ranges;

– channel_range is a structure of start and end channel numbers that together represent an excluded channel range. The start and end channel shall also be considered to be included in the excluded range.

NOTE 11 Channels which are illegal or invalid may still not be used even if they are not included in a channel_range.

excluded_channel_mask ::= bit-string

Excluded_channel_mask is a variable length bit_string (varying in multples of 8) representing the series of channels that are included or not included in the device's channel hop sequence. The Excluded Channel Mask field provides a means via which a node explicitly reports, for all channels within its channel hop sequence, which channels are and are not visited.

Bit	Channel
0 (first bit)	0
1	1
2	2
3	3
....	....
n	n

NOTE 12 [FANSPEC] states "If a bit in the mask is set to 1, the corresponding channel MUST NOT be used. Bits that are not valid channels (unused following rounding to a multiple of 8) MUST be set to 1 on transmit and ignored on receipt."

This attribute is to be used for efficiently notching out individual channels rather than a range of channels within a band plan.

join_state

Provides the join state of the device.

enum: (0) Initialisation state,
(1) Select PAN,
(2) Authenticate,
(3) Acquire PAN Config,
(4) Configure Routing,
(5) Operational,
(6)...(255) reserved

NOTE 13 Applications on a device should only send or receive traffic when the device is in join state 5.

reg_channel_exclusions

Regional channel exclusions as designated in [FANSPEC]. The values of regulatory_domain and operating_class from channel_plan uniquely identify the combination of operating parameters that the implementation is programmed to sustain. This field identifies any specific channels that must not be used when operating in a specific country.

reg_channel_exclusions ::= structure

© Copyright 1997-2020 DLMS User Association

```
{
    num_channels:          long_unsigned,
    excluded_channels:     bit-string
}
```

Where:

num_channels is defined as the total number of channels, according to [PHYSPEC], Table 3 when operating with the regulatory_domain and operating_class defined in channel_plan,

excluded_channels is a bit-string, sized as num_channels, where the first bit, index 0 is Channel 0 and the last bit is “num_channels – 1”. For each index into excluded_channels, a value of “0” indicates the channel can be used and a value of “1” indicates the channel must not be used.

NOTE 14 Unlike the Channel Exclusion in the [FANSPEC], the regulatory channel exclusion is permanent and cannot be modified after configuration. These channels represent regulatory channel exclusions that apply to the particular country or region in that the configured device is deployed.

Method description	
reset_join_state (data)	Resets join state attribute to 0.

4.17.2 Wi-SUN diagnostic (class_id = 96, version 0)

Instances of this IC are intended to provide diagnostic information regarding the Wi-SUN network that can be useful to investigate potential communication problems and service performance.

To facilitate management from the application, the following diagnostics are provided.

Wi-SUN diagnostics		**0..n**	**class_id = 96, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. sysdescr	(dynamic)	octet-string				x + 0x08
3. ifnumber	(dynamic)	integer			1	x + 0x10
4. iftable	(dynamic)	array				x + 0x18
5. neighbour_table_information	(dynamic)	array				x + 0x20
6. transmission_information	(dynamic)	array				x + 0x28
Specific methods (if required)		***m/o***				
1. reset		*o*				x + 0x30

Attribute description	
logical_name	Identifies the “Wi-SUN diagnostic” object instance. See clause 6.2.32.
sysdescr	A textual description of the entity. This value should include the full name and version identification of the system's hardware type, software operating-system, and networking software. It is mandatory that this only contains printable ASCII characters.

© Copyright 1997-2020 DLMS User Association

ifnumber	The number of interfaces (ifentry) present within the physical device as per RFC 1213. Each interface shall have a value of ifentry defined in iftable.

iftable	A list of interface entries as per RFC 1213. The number of entries in the ifentry array is given by the value of attribute ifnumber; normally this shall be 1.
	array ifentry
	ifentry::=structure
	{
	ifdescr octet-string,
	ifphysaddress octet-string,
	ifinoctets double-long-unsigned,
	ifinucastpackets double-long-unsigned,
	ifinnucastpackets double-long-unsigned,
	ifinerrors double-long-unsigned,
	ifinunknownprotos double-long-unsigned,
	ifoutoctets double-long-unsigned,
	ifoutucastpackets double-long-unsigned,
	ifoutnucastpackets double-long-unsigned,
	ifouterrors double-long-unsigned,
	ifoutQlen double-long-unsigned,
	}
	
	Where:

Item	description
ifdescr	An ASCII description of the interface as per RFC 1213. A textual string containing information about the interface. This string should include the name of the manufacturer, the product name and the version of the hardware interface.
ifphysadress	The interface's address at the protocol layer immediately `below' the network layer in the protocol stack. For interfaces which do not have such an address (e.g., a serial line), this object should contain an empty octet string. In the case of Wi-SUN this entry shall be the EUI64.
ifinoctets	The total number of octets received on the interface, including framing characters.
ifinucastpackets	The number of subnetwork-unicast packets delivered to a higher-layer protocol.In the case of Wi-SUN diagnostic this shall be the number of packets passed to the DLMS/COSEM application layer as per RFC 1213.
ifinnucastpackets	The number of non-unicast (i.e., subnetwork-broadcast or subnetwork-multicast) packets delivered to a higher-layer protocol as per RFC 1213 .
ifinerrors	The number of inbound packets that contained errors preventing them from being deliverable to a higher layer protocol as per RFC 1213.
ifunknownprotos	The number of packets received via the interface at the network layer which were

© Copyright 1997-2020 DLMS User Association

	discarded because of an unknown or unsupported protocol as per RFC 1213.
ifoutoctets	The total number of octets transmitted out of the interface at the network layer, including framing characters as per RFC 1213.
ifoutucastpackets	The total number of packets that higher-level protocols requested be transmitted to a subnetwork-unicast address, including those that were discarded or not sent. In the case of Wi-SUN diagnostic higher layer protocols shall be defined as the DLMS/COSEM application layer.
ifoutnucastpackets	The total number of packets that higher-level protocols requested be transmitted to a non-unicast (i.e., a subnetwork-broadcast or subnetwork-multicast) address, including those that were discarded or not sent. In the case of Wi-SUN diagnostic higher layer protocol shall be defined as the DLMS/COSEM application layer as per RFC 1213.
ifouterrors	The number of outbound packets that could not be transmitted because of errors.
ifoutqlen	The length of the output packet queue (in packets) at the network layer,

neighbour_table_information

The neighbour table keeps track of devices no more than 1 hop away.

array neighbour

neighbour ::= structure

```
{
    neighbour_id:     octet-string,
    neighbour_type:   enum,
    device_rank:      unsigned,
    rssi:             long-unsigned,
    etx_to_parent:    double-long-unsigned
}
```

Where:

- neighbour_id is the EUI 64 of the neighbour device. The EUI-64 shall be of the modified EUI-64 format described in RFC 4291;
- neighbour_type is an enumeration defining whether the neighbour device is a parent, child or sibling.

  enum:
  (0) Undetermined,
  (1) Parent,
  (2) Child,
  (3) Sibling
- device_rank is a function of distance in hops of the device to the root device in the DODAG. The smaller the number the closer the device is to the DODAG root device;
- rssi is a measure of the signal strength of the neighbour;

© Copyright 1997-2020 DLMS User Association

	- etx_to_parent is the Expected Number of transmissions required from the device to the destination via the parent in question required to transmit a packet. 1 is the perfect situation and FFFFFFFF is assumed to be equal to infinity and therefore a non-functioning link. The first entry in the array of neighbours shall relate to the preferred parent and all subsequent entries shall be secondary parents. NOTE Parent represents the node in the routing nearer to the Border Router (and may include the Border Router), child represents a node in the routing that has some dependency on this node, and sibling represents a node where communications is possible but is neither the parent or child of this node.

transmission_ information	Provides transmission information and quality of service per interface.
	
	array transmission_element
	
	transmission_element ::= structure
	{
	neighbour_id octet-string,
	number_of_receptions double-long-unsigned,
	number_of_sucessful_trans double-long-unsigned,
	number_of_failed_trans double-long-unsigned
	}
	
	Where:
	- neighbour_id is the EUI-64 of the FAN interface. The EUI-64 shall be of the modified EUI-64 format described in RFC 4291;
	- number_of_receptions is a simple counter of the total number of receptions from the device identified by interface_id;
	- number_of_sucessful_trans is a simple counter of the total number of successful transmissions acknowledged by the device identified by interface_id;
	- number_of_failed_trans is a simple counter of the total number of failed transmissions acknowledged by the device identified by interface_id.

Method description	
reset (data)	Clears attributes 2…4 of the object instance, that is: - sysdescr, - ifnumber, - iftable data ::= integer (0)

4.17.3 RPL diagnostic (class_id = 97, version 0)

As per [FANSPEC] 6.2.3.1.8 RPL is supported for the dissemination of IPv6 multicast packets. Instances of this IC are intended to provide diagnostic information regarding the Wi-SUN network from RPL diagnostic parameters as per RFC 6550 that can be useful to investigate potential communication problems and service performance issues.

© Copyright 1997-2020 DLMS User Association

RPL diagnostic		0..n	class_id = 97, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. rpl_ instance_id	(dynamic)	unsigned	0		191	x + 0x08
3. dodag_version_number	(dynamic)	unsigned				x + 0x10
4. dodag_rank	(dynamic)	long				x + 0x18
5. grounded	(dynamic)	boolean				x + 0x20
6. mode_of_operation	(dynamic)	enum				x + 0x28
7. dodag_prf	(dynamic)	unsigned	0	7		x + 0x30
8. dodag_dtsn	(dynamic)	unsigned				x + 0x38
9. dodag_id	(dynamic)	octet-string				x + 0x40
Specific methods (if required)		***m/o***				
1. reset (data)		*o*				x + 0x48

Attribute description	
logical_name	Identifies the “RPL diagnostic” object instance. See clause 6.2.32.
rpl_ instance_id	Holds the RPL instance ID as defined in RFC 6550. The values below are defined in RFC 6550 to identify whether or not the RPL Instance ID relates to a Global Instance ID, as in the case of coordinated networks or Local Instance ID as issued by a DODAG in the case of unilateral networks. Bits 0 and 1 of the unsigned value act as flags where bit 0 when clear defines that the rpl_instance_id is global and has 127 posible values. Bit 0 set means that the rpl_instance_id is local, and in such case bit 1 shall always be clear assuming that the server is the recipient of messages from the rpl_instance_id allocating DODAG.
dodag_version_ number	8-bit unsigned integer set by the DODAG root to the DODAGVersionNumber. RFC 6550, 8.2 describes the rules for DODAGVersionNumbers and how they affect DIO processing.
dodag_rank	Indicates the DODAG Rank of the node sending the DIO message. RFC 6550, 8.2 describes how Rank is set and how it affects DIO processing.
grounded	The Grounded ’G’ flag indicates whether the DODAG advertised can satisfy the application-defined goal. If the flag is set, the DODAG is grounded. If the flag is cleared, the DODAG is floating.
mode_of_ operation	The Mode of Operation (MOP) field identifies the mode of operation of the RPL instance as administratively provisioned at and distributed by the DODAG root. All nodes that join the DODAG shall honor the MOP in order to fully participate as a router, or else they must only join as a leaf. MOP is encoded as detailed below: enum: (0) No Downward routes maintained by RPL, (1) Non-Storing Mode of Operation, (2) Storing Mode of Operation with no multicast support,

Instance ID Type	Start value	End value
Global Instance ID	0	127
Local Instance ID	128	191
Not Used	192	255

© Copyright 1997-2020 DLMS User Association

	(3) Storing Mode of Operation with multicast support.
dodag_prf	Defines how preferable the root of this DODAG is compared to other DODAG roots within the instance. DAG Preference ranges from 0x00 (least preferred) to 0x07 (most preferred). The default is 0 (least preferred). RFC 6550, 8.2 describes how DAG Preference affects DIO processing.
dodag_dtsn	Destination Advertisement Trigger Sequence Number (DTSN) 8-bit unsigned integer set by the node issuing the DIO message. The Destination Advertisement Trigger Sequence Number (DTSN) flag is used as part of the procedure to maintain Downward routes. The details of this process are described in RFC 6550, 9.
dodag_id	128-bit IPv6 address represented as octet-string (length 16) set by a DODAG root that uniquely identifies a DODAG. The DODAGID MUST be a routable IPv6 address belonging to the DODAG root.
Method description	
reset (data)	Clears or (sets to default value) all dynamic fields of the object instance. data ::= integer (0)

4.17.4 MPL diagnostic (class_id = 98, version 0)

Instances of this IC are intended to provide diagnostic information regarding the Wi-SUN network from MPL diagnostic parameters as per RFC 7731 that can be useful at the application layer to investigate potential communication problems and service performance issues.

MPL diagnostic		**0..n**	**class_id =98 , version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical name	(static)	octet-string				x
2. proactive_forwarding	(dynamic)	boolean				x + 0x08
3. z	(dynamic)	unsigned	0	0	0	x + 0x18
4. tunit	(dynamic)	unsigned	0x01	0xFE		x + 0x20
5. se_lifetime	(dynamic)	long-unsigned	0x0001	0xFFFE		x + 0x28
6. dm_k	(dynamic)	unsigned				x + 0x30
7. dm_imin	(dynamic)	long-unsigned	0x0001	0xFFFE		x + 0x38
8. dm_imax	(dynamic)	unsigned	0x01	0xFE		x + 0x40
9. dm_t_exp	(dynamic)	long-unsigned	0x0001	0xFFFE		x + 0x48
10. c_k	(dynamic)	unsigned				x + 0x50
11. c_imin	(dynamic)	long-unsigned	0x0001	0xFFFE		x + 0x58
12. c_imax	(dynamic)	unsigned	0x01	0xFE		x + 0x60
13. c_t_exp	(dynamic)	long-unsigned	0x0001	0xFFFE		x + 0x68
Specific methods (if required)		***m/o***				
1. reset (data)		*o*				x + 0x70

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “MPL diagnostic” object instance. See clause 6.2.32.
proactive_forwarding	A flag to indicate PROACTIVE_FORWARDING. NOTE 1 This flag is set if PROACTIVE_FORWARDING = TRUE, as per RFC 7774, 2.1.
z	Reserved for future use. Defined as per RFC 7774, 2.1. NOTE 2 This is set to zero in all DHCP Servers, DHCP Clients shall ignore any none zero values.
tunit	Unit time of timer parameters (SE_LIFETIME and *_IMIN) in this option. NOTE 3 0 and 0xFF are reserved and are not be used, as per RFC 7774, 2.1.
se_lifetime	SEED_SET_ENTRY_LIFETIME/TUNIT, in milliseconds (ms). NOTE 4 0 and 0xFFFF are reserved and are not to be used, as per RFC 7774, 2.1.RFC 7774, clause 2.1.
dm_k	DATA_MESSAGE_K, as per RFC 7774, 2.1.
dm_imin	DATA_MESSAGE_IMIN/TUNIT, in milliseconds (ms). NOTE 5 0 and 0xFFFF are reserved and are not to be used, as per RFC 7774, 2.1.
dm_imax	DATA_MESSAGE_IMAX. The actual maximum timeout is described as a number of doublings of DATA_MESSAGE_IMIN, as described in RFC 6206, 4.1. NOTE 6 0 and 0xFF are reserved and MUST NOT be used, as per RFC 7774, 2.1.
dm_t_exp	DATA_MESSAGE_TIMER_EXPIRATIONS, as per RFC 7774, 2.1. NOTE 7 Values 0 and 0xFFFF are reserved and are not to be used, as per RFC 7774, 2.1.
c_k	CONTROL_MESSAGE_K, as per RFC 7774, clause 2.1.
c_imin	CONTROL_MESSAGE_IMIN/TUNIT, in milliseconds (ms). NOTE 8 0 and 0xFFFF are reserved and are not to be used, as per RFC 7774.
c_imax	CONTROL_MESSAGE_IMAX. The actual maximum timeout is described as a number of doublings of CONTROL_MESSAGE_IMIN. NOTE 9 0 and 0xFF are reserved and are not to be used, as per RFC 7774.
c_t_exp	CONTROL_MESSAGE_TIMER_EXPIRATIONS. NOTE 10 0 and 0xFFFF are reserved and are not to be used, as per RFC 7774.
Method description	
reset (data)	Clears all dynamic fields of the object instance. data ::= integer (0)

© Copyright 1997-2020 DLMS User Association

4.18 Interface classes for setting up and managing the DLMS/COSEM profile for ISO/IEC 14908 PLC networks

4.18.1 General

The interface classes specified here are used in devices implementing the IEC 62056-8-8 DLMS/COSEM communication profile for ISO/IEC 14908 series networks.

4.18.2 ISO/IEC 14908 identification (class_id = 130, version = 0)

Instances of the ISO/IEC 14908 identification IC allow the identification of the device and the network to which the device is connected.

ISO/IEC14908 identification		0...n	class_id = 130, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. node_ID	(static.)	unsigned	1	127		x + 0x08
3. subnet_ID	(static)	unsigned	1	255		x + 0x10
4. domain_ID	(static)	octet-string				x + 0x18
5. program_ID	(static)	octet-string				x + 0x20
6. unique_node_ID	(static)	octet-string				x + 0x28
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “ISO/IEC 14908 identification” object instance. See 6.2.30.
node_ID	Identification of the node.
subnet_ID	Identification of the subnet to which the device is connected.
domain_ID	Identification of the network. It shall be 3 bytes.
program_ID	Uniquely identifies the functionalities and version of the firmware running on the device. It shall be 8 bytes.
unique_node_ID	Identification of the physical connection module connected to the ISO/IEC 14908-1:2012 network. Each compliant module is assigned a unique 48-bit identifier called Unique_Node_ID. This ID is unique worldwide and is set at the time of manufacture. The value of this identifier does not change from the time of manufacture.

© Copyright 1997-2020 DLMS User Association

4.18.3 ISO/IEC 14908 protocol setup (class_id = 131, version = 0)

Instances of the ISO/IEC 14908 protocol setup IC allow the configuration of the ISO/IEC 14908 device.

ISO/IEC 14908 protocol setup		0...n	class_id = 131, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. inactivity_timeout	(static)	long-unsigned				x + 0x08
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “ISO/IEC 14908 protocol setup” object instance. See 6.2.30
inactivity_timeout	Minutes of absence of adaptation layer messages from the NNAP before the LNAP is considered to be out of communication; see IEC 62056-8-8:2020, F4.3.

4.18.4 ISO/IEC 14908 protocol status (class_id = 132, version = 0)

Instances of the ISO/IEC 14908 protocol status IC allow the status of the protocol in the ISO/IEC 14908 device to be determined.

ISO/IEC 14908 protocol status		0...n	class_id = 132, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. transmission_errors	(dyn)	long-unsigned				x + 0x08
3. transmit_tx_failure	(dyn)	long-unsigned				x + 0x10
4. transmit_tx_retries	(dyn)	long-unsigned				x + 0x18
5. receive_tx_full	(dyn)	long-unsigned				x + 0x20
6. lost_messages	(dyn)	long-unsigned				x + 0x28
7. missed_messages	(dyn)	long-unsigned				x + 0x30
8. layer2_received	(dyn)	long-unsigned				x + 0x38
9. layer3_received	(dyn)	long-unsigned				x + 0x40
10. messages_received	(dyn)	double-long-unsigned				x + 0x48
11. messages_validated	(dyn)	double-long-unsigned				x + 0x50
Specific methods		***m/o***				
1. reset (data)		o				X + 0x70

Attribute description	
logical_name	Identifies the “ISO/IEC 14908 protocol status” object instance. See 6.2.30.

© Copyright 1997-2020 DLMS User Association

transmission_errors	This is a count of the number of transmission errors that have occurred on the network. A transmission error is detected via a CRC error during packet reception or as a packet that is less than 8 bytes long. This could result from a collision, a noisy medium, signal attenuation, etc. See ISO/IEC 14908-1:2012, 13.8.2.
transmit_tx_failure	The number of times that the node failed to receive expected acknowledgments or responses after retrying the configured number of times. See ISO/IEC 14908-1:2012, B.8.
transmit_tx_retries	The number of retries sent by this node. This does not include retries used for messages sent with repeated service. See ISO/IEC 14908-1:2012, B.8.
receive_tx_full	The number of times that an incoming packet was discarded because there was no room in the transaction database. See ISO/IEC 14908-1:2012, B.8.
lost_messages	This is the number of messages that were addressed to the node that were thrown away because there was no application buffer available for the message; See ISO/IEC 14908-1:2012, 13.8.2.
missed_messages	This is the number of messages that were on the network but could not be received because there was no network buffer (packet buffer) available for the message or the network buffer was too small to receive the message; See ISO/IEC 14908-1:2012, 13.8.2.
layer2_received	The number of Layer-2 messages received by this node. Layer-2 messages are those that have correct CRC and can be addressed to any node. See ISO/IEC 14908-1:2012, B.8.
layer3_received	The number of messages transmitted from layer 3 of the protocol processor. These can include network variable updates, explicit messages, acknowledgments, retries, reminders, service pin messages, and any other type of message. See ISO/IEC 14908-1:2012, B.8.
messages_received	Number of messages received, which are delivered to the Adaptation layer.
messages_validated	Number of correct messages received at Adaptation layer level.

Method description	
reset (data)	Forces a reset of the object. By invoking this method, the values are set to the default value. The default value is an instance specific constant. data ::= integer (0) NOTE: IEC 62056-8-8:2020 specifies version = 0 which does not support the reset method.

© Copyright 1997-2020 DLMS User Association

4.18.5 ISO/IEC 14908 diagnostic (class_id = 133, version = 0)

Instances of the ISO/IEC 14908 diagnostic IC provides information about the device status inside the PLC network.

ISO/IEC 14908 diagnostic		0...n	class_id = 133, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. plc_signal_quality_status	(dyn)	enum				x + 0x08
3. com_module_state	(dyn)	enum				x + 0x10
4. received_message_status	(dyn)	unsigned				x + 0x18
5. no_receive_buffer	(dyn)	long-unsigned				x + 0x20
6. transmit_no_data	(dyn)	long-unsigned				x + 0x28
7. backlog_overflows	(dyn)	long-unsigned				x + 0x30
8. late_ack	(dyn)	long-unsigned				x + 0x38
9. frequency_invalid	(dyn)	long-unsigned				x + 0x40
Specific methods		***m/o***				
1. reset (data)		o				X + 0x60

Attribute description	
logical_name	Identifies the “ISO/IEC 14908 diagnostic” object instance. See 6.2.30.
plc_signal_quality_ status	Provide the PLC signal quality status: enum: (0) No PLC traffic detected, (1) PLC traffic detected, but no traffic addressed to this device, (2) A packet addressed to this device has been received. The signal margin is less than or equal to 9 dB, (3) A packet addressed to this device has been received. The signal margin is between 12 dB or 15 dB, (4) A packet addressed to this device has been received. The signal margin is greater than or equal to 18 dB, (5) Meter is commissioned but its inactivity_timeout as defined in 4.18.1 has expired. The last packet addressed to this meter had a signal margin is less than or equal to 9 dB, (6) Meter is commissioned but its inactivity_timeout as defined in 4.18.1 has expired. The last packet addressed to this meter had a signal margin at 12 dB or 15 dB, (7) Meter is commissioned but its inactivity_timeout as defined in 4.18.1 has expired. The last packet addressed to this meter had a signal margin greater than or equal to 18 dB. Note. This parameter is updated every time a packet is received by the device.

© Copyright 1997-2020 DLMS User Association

com_module_state

Last received state of the device's power line IEC 62056-8-8:2020 communication module.

enum:

(0),(1)	Invalid states,
(2)	Unconfigured,
(3)	Applicationless,
(4)	Configured,
(6)	Hard-offline,
(12)	Configured, soft offline,
(140)	Configured, in bypass mode,
(255)	No reply when status requested,
other	Invalid states.

This field is updated on each device power-up, each re-synch request from the NNAP and each communication module reset. Any value which is not listed above is invalid state. See IEC 62056-8-8:2020,13.4.

NOTE Configured soft offline state (12) and Configured, in bypass mode state (140) are sub states of Configured state (4). See IEC 62056-8-8:2020, 13.4 and 13.8

received_message_status

Unsigned integer, constructed as:

Bit 7 (MSB)	b6	b5	b4	b3	b2	b1	Bit 0 (LSB)
Transceiver phase			reserved				See below

Bit 0 represents the PLC Link Explicit Message Received.

Transceiver phase bits are used as below:

	Bit 7	b6	Bit 5
No phase inversion present	Not set	Not set	Not set
In phase normal	Not set	Not set	Set
Plus 120 degrees inverted	Not set	Set	Not set
Minus 120 degrees normal	Not set	Set	Set
Phase 180 degrees inverted	Set	Not set	Not set
Plus 120 degrees normal	Set	Not set	Set
Minus 120 degrees inverted	Set	Set	Not set
Reserved	Set	Set	Set

no_receive_buffer

Number of times the IEC 62056-8-8:2020 lower layers detected a network inbound packet addressed to it, but has no buffer to store it. Hence the message could not be made available to the adaptation layer.

transmit_no_data

Number of times the adaptation layer attempted to transmit data to the IEC 62056-8-8:2020 stack but no buffer is available. Updated on occurrence. IEC 62056-8-8:2020,13.4.

© Copyright 1997-2020 DLMS User Association

backlog_overflows	Number of times the MAC layer predictive backlog counter overflowed. See IEC 62056-8-8:2020, B.8.
late_ack	Number of times an ACK or response was received for a transaction that has been already completed. Typically, this occurs due to too short transaction timer values or network congestion.
frequency_invalid	Number of times the IEC 62056-8-8:2020 stack detects an invalid frequency. Updated on occurrence.

Method description	
reset (data)	Forces a reset of the object. By invoking this method, the values of attributes 5..9 are set to the default value. The default value is an instance specific constant. data ::= integer (0) NOTE IEC 62056-8-8:2020 specifies version = 0 which does not support the reset method.

© Copyright 1997-2020 DLMS User Association

5 Maintenance of the interface classes

5.1 General

5.1.1 New versions of interface classes

Any modification of an existing IC affecting the transmission of service requests or responses results in a new version (version ::= version+1) and shall be documented accordingly.

NOTE Previous versions of this Technical Report had additional text that applied a rule on the re-use and modification of attribute and method enumerators. This rule is no longer required as it placed an unnecessary constraint on interface classes with large numbers of attributes and/or methods.

Any modification of ICs will be recorded by moving the old version of an IC into the relevant clause below.

5.1.2 New interface classes

The DLMS UA reserves the right to be the exclusive administrator of interface classes.

5.1.3 Removal of interface classes

Besides one association object and the logical device name object no instantiation of an IC is mandatory within a meter. Therefore, even unused ICs will not be removed from the standard. They will be kept to ensure compatibility with possibly existing implementations.

5.2 Previous versions of interface classes – general

The subsequent subclauses list those IC specifications which were included in previous editions of this document. The previous IC versions differ from the current versions by at least one attribute and/or method and by the version number.

For new implementations in metering devices, only the current versions should be used.

Communication drivers at the client side should also support previous versions.

5.3 Previous versions of interface classes for parameters and measurement data

5.3.1 Profile generic (class_id = 7, version = 0)

The version listed here was valid in edition 1 and it is replaced as of edition 2 and 3.

© Copyright 1997-2020 DLMS User Association

Profile generic		**0...n**	**class_id = 7, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. buffer	(dyn.)	array			x	x + 0x08
3. capture_objects	(static)	array				x + 0x10
4. capture_period	(static)	double-long-unsigned			x	x + 0x18
5. sort_method	(static)	enum			x	x + 0x20
6. sort_object	(static)	ObjectDefinition			x	x + 0x28
7. entries_in_use	(dyn.)	double-long-unsigned	0		0	x + 0x30
8. profile_entries	(static)	double-long-unsigned	1		1	x + 0x38
Specific methods		***m/o***				
1. reset (data)						x + 0x58
2. capture (data)						x + 0x60
3. get_buffer_by_range (data)						x + 0x68
4. get_buffer_by_index (data)						x + 0x70

Attribute description

buffer

The buffer attribute contains a sequence of entries. Each entry contains values of the captured objects (as returned by calling *read (current_value)*). The sequence is ordered according to the specified sort method. The buffer gets filled by subsequent calls to *capture ()*.

array entry

entry ::= structure
Instance Specific

Def. The buffer is empty at installation.

Remark: Reading the entire buffer delivers only those entries which are “in use”

capture_ objects

Specifies the list of capture objects (registers, clocks and profiles) that are assigned to this profile object. Upon a call of the *capture ()* service, the specified attributes of these objects are copied into the buffer of the profile.

array ObjectDefinition

ObjectDefinition ::= structure
{
logical_name: octet-string,
class_id: long-unsigned,
attribute_index: unsigned
}

where attribute_index is a pointer to the attribute within the object. attribute_index = 1 refers to the first attribute (i.e. *logical_name*), attribute_index = 2 to the 2nd, etc.)

capture_period

>= 1: Automatic capturing assumed. Specifies the capturing period in seconds

0: no automatic capturing, capturing is trigger externally or capture events occur asynchronously

© Copyright 1997-2020 DLMS User Association

sort_method	If the profile is unsorted, it works as a „first in first out“ buffer (it is hence sorted by capturing, and not necessarily by the time maintained in the clock object). If the buffer is full, the next call to *capture ()* will push out the first (oldest) entry of the buffer to make space for the new entry. If the profile is sorted, a call to *capture ()* will store the new entry at the appropriate position in the buffer, moving all following entries and probably losing the least interesting entry. If the new entry would enter the buffer after the last entry and if the buffer is already full, the new entry will not be retained at all. enum: 1: fifo, 2: lifo (last in first out), 3: largest, 4: smallest, 5: nearest_to_zero, 6: farest_from_zero ***Def.*** fifo
sort_object	If the profile is sorted, this attribute specifies the register or clock that the ordering is based upon. ObjectDefinition see above ***Def.*** no object to sort by (only possible with *sort_method* = fifo or lifo)
entries_in_use	Counts the number of entries stored in the buffer. After a call of *reset ()* the buffer does not contain any entries, and this value is zero. Upon each subsequent call of *capture ()*, this value will be incremented up to the maximum number of entries that will get stored (see *profile_entries*). double-long-unsigned 0…profile_entries ***Def.*** 0
profile_entries	Specifies, how many entries should be retained in the buffer. double-long-unsigned 1… (limited by physical size) ***Def.*** 1
Method description	
reset (data)	Clears the buffer. The buffer has no valid entries afterwards; *entries_in_use* is zero after this call. This call does not trigger any additional operations of the capture objects, specifically, it does not reset any captured buffers or registers. data ::= integer (0)
capture (data)	Copies the values of the objects to capture into the buffer by calling *read (<object_attribute>)* of each capture object. Depending on the *sort_method* and the actual state of the buffer this produces a new entry or a replacement for the less significant entry. As long as not all entries are already used, the *entries_in_use* attribute will be incremented. This call does not trigger any additional operations within the capture objects such as *capture ()* or *reset ()*. Note that only some attributes of the captured objects might be stored, not the complete object. data ::= integer (0)

© Copyright 1997-2020 DLMS User Association

write (...)	Any write access to one of the attributes describing the static structure of the buffer will automatically call a *reset ()* and this call will propagate to all other profiles capturing this profile. If writing to *profile_entries* is attempted with a value too large for the buffer, it will be set to the maximum possible value (restricted by physical size, typically laid down in the firmware).		
get_buffer_by_ range (data)	Reads all entries between the given limits.		
	restricting_ object	in	ObjectDefinition Defines the register or clock restricting the range of entries to be retrieved.
	from_value	in	instance_specific Oldest or smallest entry to retrieve.
	to_value	in	instance_specific Newest or largest entry to retrieve.
	selected_values	in	List of columns to retrieve: array ObjectDefinition If the array is empty (has no entries), all captured data is returned. Otherwise, only the columns specified in the array are returned. The type *ObjectDefinition* is specified above (*Capture_Objects*)
	entries	out	See *entries_in_use* above.
	array (of *instance_ specific_value)*	out	Sorted sequence of entries containing the requested values of the capture objects.
	data ::= structure {restricting_object; from_value; to_value; selected_values} In the response “data” is an array of instance_specific_value.		
get_buffer_by_ index (data)	Read all entries between the given limits.		
	from_index	in	double-long-unsigned First entry to retrieve.
	to_index	in	double-long-unsigned Last entry to retrieve.
	from_selected_value	in	long-unsigned index of first value to retrieve
	to_selected_value	in	long-unsigned index of last value to retrieve.
	entries	out	See *entries_in_use* above.
	array of *instance_ specific_value*	out	Sorted sequence of entries containing the requested values of the capture objects.
	data ::= structure {from_index; to_index; selected_values}. In the response “data” is an array of instance_specific_value.		

© Copyright 1997-2020 DLMS User Association

5.3.2 Compact data (class_id = 62, version = 0)

Instances of the “Compact data” IC allow capturing the values of COSEM object attributes as determined by the *capture_objects* attribute. Capturing can take place:

- on an external trigger (explicit capturing); or
- upon reading the *compact_buffer* attribute (implicit capturing)

as determined by the *capture_method* attribute.

The values are stored in the *compact_buffer* attribute as an octet-string.

The set of data types is identified by the *template_id* attribute. The data type of each attribute captured is held by the *template_description* attribute.

The client can reconstruct the data in the uncompacted form – i.e. including the COSEM attribute descriptor, the data type and the data values – using the *capture_objects*, *template_id* and *template_description* attributes.

Compact data		**0...n**	**class_id = 62, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. compact_buffer	(dyn.)	octet-string				x + 0x08
3. capture_objects	(static)	array				x + 0x10
4. template_id	(static)	unsigned				x + 0x18
5. template_description	(dyn.)	octet-string				x + 0x20
6. capture_method	(static)	enum				x + 0x28
Specific methods		***m/o***				
1. reset (data)		o				x + 0x38
2. capture (data)		o				x + 0x40

Attribute description	
logical_name	Identifies the “Compact data” object instance. See 6.2.41.
compact_buffer	Contains the values of the attributes captured as an *octet-string*. When the data captured is of type *octet-string*, *bit-string*, *visible-string*, *utf8-string* or *array* the length is also included here.

capture_objects	Specifies the list of COSEM object attributes that are assigned to the “Compact data” object instance.
	The *template_id* attribute shall be the first element in the *capture_objects* array.
	Upon an explicit or implicit invocation of the *capture (data)* method the values of the selected attributes are captured into the *compact_buffer*.
	array capture_object_definition
	capture_object_definition ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer,
	data_index: long-unsigned
	}

© Copyright 1997-2020 DLMS User Association

Where:

– attribute_index is a pointer to the attribute within the object, identified by class_id and logical_name: attribute_index 1 refers to the 1st attribute (i.e. the *logical_name*), attribute_index 2 to the 2nd attribute etc.; attribute_index 0 refers to all public attributes;

– data_index is a pointer selecting one or several specific elements of an attribute with a complex data type (structure or array).

  – if the data type of the attribute is simple, then data_index has no meaning;

  – if the data type of the attribute is a structure or an array, then data_index points to one or several specific elements in the structure or array;

  – when the attribute is the *buffer* of a “Profile generic” object, the data_index carries selective access parameters.

data_index:	MS-Byte		LS-Byte
	Upper nibble	Lower nibble	

  – 0x0000 = identifies the whole attribute;

  – 0x0001 to 0x0FFF = identifies one element in the complex attribute. The first element in the complex attribute is identified by data_index 1;

  – 0x1000 to 0xFFFF = selective access to the array holding the *buffer* of a “Profile generic” object. The data-index selects entries within a number of last (recent) time periods, or a number of last (recent) entries, as well as the columns in the array.

The encoding is specified in Table 6.

template_id

Contains the identifier of the template.

It shall uniquely identify the instance of the “Compact data” IC and the *template_description*.

template_ description

Provides the data type of each attribute captured. It is an *octet-string* generated automatically by the server upon the programming of the *capture_objects* and it has the following structure:

– the first octet is 0x02 (the tag of a structure);

– this is followed by the number of elements in the structure – the same as the number of elements in the *capture_objects* array – encoded as a variable length integer;

– this is followed by the data type of each attribute, in the same order as in the *capture_object* array:

  – in the case of attributes with simple data type, the data type is represented by a single octet, carrying the tag of the data type. In the case of *bit-string* [4], *octet-string* [9], *visible-string* [10], *utf8-string* [12] the length of the string is part of the data held in the *compact_buffer*;

  – in the case of an *array* [1], the data type is represented by a single octet 0x01. This is followed by the type description of the elements in the array. The number of elements in the array is part of the data held in the *compact_buffer*;

© Copyright 1997-2020 DLMS User Association

	– in the case of a *structure* [2], the data type is represented by a single octet 0x02, followed by the number of elements inside the structure followed by the tag of each element of the structure.
capture_method	Defines the way the *compact_buffer* is updated. enum: (0) Capture upon invoking the *capture* (data) method. This may occur remotely or locally (explicit capturing), (1) Capture upon reading the *compact_buffer* attribute (implicit capturing).
Method description	
reset (data)	Clears the *compact_buffer*. After invoking this method the *compact_buffer* holds an octet-string of 0 length, until a new capture takes place. This call does not trigger any additional operations of the capture objects. Specifically, it does not reset any captured attributes. data ::= integer(0)
capture (data)	Copies the values of the attributes into the *compact_buffer* by reading each capture object. This call does not trigger any additional operations within the capture objects such as capture () or reset (). data ::= integer(0)

Behaviour of the object after modification of certain attributes:

Any modification of the *capture_objects* resets the *compact_buffer* and automatically updates the *template_description*.

Restrictions

When defining the *capture_object* attribute, circular references shall be avoided.

© Copyright 1997-2020 DLMS User Association

5.4 Previous versions of interface classes for access control and management

5.4.1 Association SN (class_id = 12, version = 0)

The version listed here was valid in Edition 1 and it is replaced as of Edition 2 and 3.

Device Association View	**0..1** [a]	**class_id = 12, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	objlist_type				x + 0x08
Specific methods	***m/o***				
1. getlist_by_classid (data)	o				x + 0x20
2. getobj_by_logicalname (data)	o				x + 0x28
3. read_by_logicalname (data)	o				x + 0x30
4. get_attributes&services (data)	o				x + 0x38
5. change_LLS_secret (data)	o				x + 0x40
6. change_HLS_secret (data)	o				x + 0x48
7. get_HLS_challenge (data)	o				x + 0x50
8. reply_to_HLS_challenge (data)	o				x + 0x58

Attribute description	
logical_name	Identifies the type of the client-server association. See 6.2.33.

object_list	Contains the list of all objects with their short_name (DLMS ObjectName of the first attribute), class_id, version [b] and *logical_name*.
	Objlist_type ::= array objlist_element
	objlist_element ::= structure
	{
	short_name: long,
	class_id: long-unsigned,
	version = unsigned,
	logical_name: octet-string
	}

Method description	
getlist_by_ classid (data)	Delivers the subset of the object_list for a specific class_id. data ::= class_id: long-unsigned For the response: data ::= objlist_type

getobj_by_ logicalname (data)	Delivers the entry of the object_list for a specific *logical_name* and class_id.
	data ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
	For the response: data ::= objlist_element

read_by_ logicalname (data)	Reads attributes for specific objects. The objects are specified by their class_id and their logical_name. data ::= array AttributeIdentification

© Copyright 1997-2020 DLMS User Association

read_by_ logicalname (data) (continued)	AttributeIdentification ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: unsigned
	}
	where attribute_index is a pointer (i.e. offset) to the attribute within the object.
	attribute_index = 0 delivers all attributes [c], attribute_index = 1 delivers the first attribute (i.e. *logical_name*), etc.
	For the response: data is according to the type of the attribute.

get_attributes& services (data)	Delivers information about the access rights to the attributes and services within the actual association. The objects are specified by their class_id and their logical_name.
	array ObjectIdentification
	ObjectIdentification ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
	For the response
	data ::= array AccessDescription
	
	AccessDescription ::= structure
	{
	read_attributes: bit-string,
	write_attributes: bit-string,
	services: bit-string
	}
	The position in the bit-string identifies the attribute/service (first position ↔ first attribute, first position ↔ first service) and the value of the bit specifies whether the attribute/service is available (bit set) or not available (bit clear).

change_LLS_ secret (data)	Changes the LLS secret (for example password). data ::= octet-string new LLS secret
change_HLS_ secret (data)	Changes the HLS secret (for example encryption key). data ::= octet-string [d] new HLS secret
get_HLS_ challenge (data)	Asks the server for the client “challenge” (for example random number). data ::= octet-string client challenge
reply_to_HLS_ challenge (data)	Delivers the “secretly” processed “challenge” back to the server. data ::= octet-string client’s response to the challenge If the authentication is accepted, then the response is successful [0]. If the authentication is not accepted, then the response is set to data-access-error [1].

a Per client server association

b Within an client-server association, there are never two objects with the same class_id and logical_name differing only in the versions.

c If at least one attribute has no read access right under the current association, then a *read_by_logicalname* () to attribute index = 0 reveals the error message “scope-of-access-violated” (comp. IEC 61334-4-41:1996, A.12 (p. 221).

d The structure of the “new secret” depends on the security mechanism implemented. The “new secret” may contain additional checkbits and it may be encrypted.

© Copyright 1997-2020 DLMS User Association

5.4.2 Association SN (class_id = 12, version = 1)

This IC allows modelling of AAs between a server and a client, using the application context *short name referencing*. A COSEM logical device may have one instance of this IC for each association the device is able to support.

The **short_name** of the current "Association SN" object itself is fixed within the COSEM context. It is given in Clause 4.1.3 as 0xFA00.

Association SN	**0...n**	**class_id = 12, version = 1**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	objlist_type				x + 0x08
Specific methods	***m/o***				
1. reserved from previous versions	o				
2. reserved from previous versions	o				
3. read_by_logicalname (data)	**o**				x + 0x30
4. get_attributes&methods (data)	o				x + 0x38
5. change_LLS_secret (data)	o				x + 0x40
6. change_HLS_secret (data)	o				x + 0x48
7. reserved from previous versions					
8. reply_to_HLS_authentication (data)	o				x + 0x58

Attribute description	
logical_name	Identifies the "Association SN" object instance. See 6.2.33.

object_list	Contains the list of all objects with their base_names (short_name), class_id, version and *logical_name*. The base_name is the DLMS objectName of the first attribute (*logical_name*).
	objlist_type ::= array objlist_element
	
	objlist_element ::= structure
	{
	base_name: long,
	class_id: long-unsigned,
	version = unsigned,
	logical_name: octet-string
	}
	selective access (see 4.1.4) to the attribute *object_list* may be available (optional). The selective access parameters are as defined below.

© Copyright 1997-2020 DLMS User Association

Parameters for selective access to the *object_list* attribute

Access selector value	Parameter	Comment
1	class_id: long-unsigned	Delivers the subset of the object_list for a specific class_id. For the response: data ::= objlist_type

2	structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
		Delivers the entry of the object_list for a specific class_id and logical_name.
		For the response:
		data ::= objlist_element

Method description	

read_by_ logicalname	Reads attributes for selected objects. The objects are specified by their class_id and their *logical_name*. With this method, the parameterized access feature can also be used.
	data ::= array attribute_identification
	
	attribute_identification ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}
	where attribute_index is a pointer (i.e. offset) to the attribute within the object.
	attribute_index 0 delivers all attributes [a], attribute_index 1 delivers the first attribute (i.e. *logical_name*), etc.).
	For the response: data is according to the type of the attribute.

get_attributes& methods (data)	Delivers information about the access rights to the attributes and methods within the actual association. The objects are specified by their class_id and their *logical_name*. With this method, the parameterized access feature can also be used.
	data ::= array object_identification
	
	object_identification ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
	For the response
	data ::= array access_description
	
	access_description ::= structure
	
	{
	read_attributes: bit-string,
	write_attributes: bit-string,
	methods: bit-string
	}

© Copyright 1997-2020 DLMS User Association

get_attributes& methods (data) (continued)	The position in the bit-string identifies the attribute/method (first position ↔ first attribute, first position ↔ first method) and the value of the bit specifies whether the attribute/method is available (bit set) or not available (bit clear). Depending on the implementation, some attributes or methods of some objects may not be needed. In this case, such attributes or methods may not be accessible (neither read access, nor write access to attributes, no access to methods).
change_LLS_ secret (data)	Changes the LLS secret (for example password). data ::= octet-string new LLS secret
change_HLS_ secret (data)	Changes the HLS secret (for example encryption key). data ::= octet-string [b] new HLS secret
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers the client's “secretly” processed “challenge StoC” (f(StoC)) back to the server as the *data* service parameter of the invoked Write.request service. data ::= octet-string client’s response to the challenge data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.

[a] If at least one attribute has no read access right under the current association, then a *read_by_logicalname* () to attribute index 0 reveals the error message “scope of access violation” (see IEC 61334-4-41:1996, A.12 (p. 221).

[b] The structure of the “new secret” depends on the security mechanism implemented. The “new secret” may contain additional check bits and it may be encrypted.

5.4.3 Association SN (class_id = 12, version = 2)

COSEM logical devices able to establish AAs within a COSEM context using SN referencing, model the AAs using instances of the “Association SN” IC. A COSEM logical device may have one instance of this IC for each AA the device is able to support.

The **short_name** of the “Association SN” object itself is fixed within the COSEM context. See 4.1.3.

Association SN	**0...n**	**class_id = 12, version = 2**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	objlist_type				x + 0x08
3. access_rights_list (static)	access_rights_type				x + 0x10
4. security_setup_reference (static)	octet-string				x + 0x18
Specific methods	***m/o***				
1. *reserved from previous versions*	o				
2. *reserved from previous versions*	o				
3. read_by_logicalname (data)	o				x + 0x30
4. *reserved from previous versions*	o				
5. change_secret (data)	o				x + 0x40
6. *reserved from previous versions*	o				
7. *reserved from previous versions*					
8. reply_to_HLS_authentication (data)	o				x + 0x58

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “Association SN” object instance. See 6.2.33.

object_list	Contains the list of all objects with their base_name (short_name), class_id, version and *logical_name*. The base_name is the DLMS objectName of the first attribute (*logical_name)*.
	objlist_type ::= array objlist_element
	
	objlist_element ::= structure
	{
	base_name: long,
	class_id: long-unsigned,
	version: unsigned,
	logical_name: octet-string
	}
	selective access (see 4.1.4) to the attribute *object_list* may be available. The access selector values and their parameters are as defined below.

access_rights_ list	Contains the access rights to attributes and methods.
	The link between the *object_list* and the *access_rights_list* is the base_name, present in both the objlist_element structure and the access_right_element structure. Therefore, the base_names on the two lists shall be the same. The number – and preferably, the order – of the elements in the array of objlist_element and the array of access_right_element shall also be the same.
	access_rights_type ::= array access_rights_element
	access_rights_element ::= structure
	{
	base_name: long,
	attribute_access: attribute_access_descriptor,
	method_access: method_access_descriptor
	}
	attribute_access_descriptor ::= array attribute_access_item
	
	attribute_access_item ::= structure
	{
	attribute_id: integer,
	access_mode: enum:
	(0) no_access,
	(1) read_only,
	(2) write_only,
	(3) read_and_write,
	(4) authenticated_read_only,
	(5) authenticated_write_only,
	(6) authenticated_read_and_write
	
	access_selectors: CHOICE
	{
	null-data [0],
	array integer [1]
	}
	}
	
	method_access_descriptor ::= array method_access_item
	method_access_item ::= structure

© Copyright 1997-2020 DLMS User Association

access_rights_ list (continued)	{
	method_id: integer,
	access_mode: enum:
	(0) no_access,
	(1) access,
	(2) authenticated_access
	}
	selective access (see 4.1.4) to the attribute *access_rights_list* may be available (optional). The access selector values and their parameters are as defined below.

security_setup_ reference	References the “Security setup” object by its *logical_name*. The referenced object manages security for a given “Association SN” object instance.

Parameters for selective access to the *object_list* and *access_rights_list* attribute

Access selector value	Parameter	Available with attribute	Comment
1	class_id: long-unsigned	2	Delivers the subset of the object_list for a specific class_id. For the response: data ::= objlist_type

2	structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
		2	Delivers the entry of the object_list for a specific class_id and *logical_name*.
			For the response: data ::= objlist_element

3	base_name: long	2, 3	In the case of attribute 2, delivers the entry of the object_list for a specific base_name. For the response: data ::= objlist_element In the case of attribute 3, delivers the entry of the access_rights_list for a specific base_name. For the response: data ::=access_rights_element

Method description	

read_by_ logicalname (data)	Reads attributes for selected objects. The objects are specified by their class_id and their *logical_name*. With this method, the parameterized access feature can also be used.
	data ::= array attribute_identification
	attribute_identification ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}
	where attribute_index is a pointer (i.e. offset) to the attribute within the object.
	
	attribute_index 0 delivers all attributes; attribute_index 1 delivers the first attribute (i.e. *logical_name*), etc.).
	For the response: data is according to the type of the attribute.
	NOTE 1 If at least one attribute has no read access right under the current association, then a *read_by_logicalname* () to attribute index 0 reveals the error message “scope-of-access-violated”, see DLMS UA 1000-2 Ed. 10, 9.5.

© Copyright 1997-2020 DLMS User Association

change_secret (data)	Changes the LLS or HLS secret (for example password). data ::= octet-string new secret NOTE 2 The structure of the "new secret" depends on the security mechanism implemented. The "new secret" may contain additional check bits and it may be encrypted.
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers to the server the result of the secret processing by the client of the server's challenge to the client, f(StoC), as the data service parameter of the Read.request primitive invoked with parameterised access. data ::= octet-string client's response to the challenge If the authentication is accepted, then the response (Read.confirm primitive) contains Result == OK and the result of the secret processing by the server of the client's challenge to the server, f(CtoS) in the data service parameter of the Read.response service. data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.

5.4.4 Association SN (class_id = 12, version = 3)

NOTE 1 This new version 3 of the "Association SN" IC supports the client user identification process, see 4.4.2.

COSEM logical devices able to establish AAs within a COSEM context using SN referencing, model the AAs using instances of the "Association SN" IC. A COSEM logical device may have one instance of this IC for each AA the device is able to support.

The **short_name** of the "Association SN" object itself is fixed within the COSEM context. See 4.1.3.

Association SN	**0...n**	**class_id = 12, version = 3**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	objlist_type				x + 0x08
3. access_rights_list (static)	access_rights_type				x + 0x10
4. security_setup_reference (static)	octet-string				x + 0x18
5. user_list (static)	array				x + 0x20
6. current_user	structure				x + 0x28
Specific methods	***m/o***				
1. *reserved from previous versions*	o				
2. *reserved from previous versions*	o				
3. read_by_logicalname (data)	o				x + 0x30
4. *reserved from previous versions*	o				
5. change_secret (data)	o				x + 0x40
6. *reserved from previous versions*	o				
7. *reserved from previous versions*					
8. reply_to_HLS_authentication (data)	o				x + 0x58
9. add_user (data)	o				x + 0x60
10. remove_user (data)	o				x + 0x68

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “Association SN” object instance. See 6.2.33.

object_list	Contains the list of all objects with their base_name (short_name), class_id, version and *logical_name*. The base_name is the DLMS objectName of the first attribute (*logical_name).*
	objlist_type ::= array objlist_element
	objlist_element ::= structure
	{
	base_name: long,
	class_id: long-unsigned,
	version: unsigned,
	logical_name: octet-string
	}
	selective access (see 4.1.4) to the attribute *object_list* may be available. The access selector values and their parameters are as defined below.

access_rights_ list	Contains the access rights to attributes and methods.
	The link between the *object_list* and the *access_rights_list* is the base_name, present in both the objlist_element structure and the access_right_element structure. Therefore, the base_names on the two lists shall be the same. The number – and preferably, the order – of the elements in the array of objlist_element and the array of access_right_element shall also be the same.
	access_rights_type ::= array access_rights_element
	access_rights_element ::= structure
	{
	base_name: long,
	attribute_access: attribute_access_descriptor,
	method_access: method_access_descriptor
	}
	
	attribute_access_descriptor ::= array attribute_access_item
	
	attribute_access_item ::= structure
	{
	attribute_id: integer,
	access_mode: enum:
	(0) no_access,
	(1) read_only,
	(2) write_only,
	(3) read_and_write,
	(4) authenticated_read_only,
	(5) authenticated_write_only,
	(6) authenticated_read_and_write
	access_selectors: CHOICE
	{
	null-data [0],
	array integer [1]
	}
	}

access_rights_ list (continued)	method_access_descriptor ::= array method_access_item method_access_item ::= structure

© Copyright 1997-2020 DLMS User Association

	{
	method_id: integer,
	access_mode: enum:
	(0) no_access,
	(1) access,
	(2) authenticated_access
	}
	selective access (see 4.1.4) to the attribute *access_rights_list* may be available (optional). The access selector values and their parameters are as defined below

security_ setup_ reference	References the “Security setup” object by its *logical_name*. The referenced object manages security for a given “Association SN” object instance.

user_list	Contains the list of users allowed to use the AA managed by the given instance of the “Association SN” IC.
	array user_list_entry
	
	user_list_entry ::= structure
	{
	user_id: unsigned,
	user_name: visible-string
	}
	Where:
	- user_id is the identifier of the user (this value is carried by the calling-AE-invocation-id field of the AARQ);
	- user_name is the name of the user.
	If the user_list attribute is empty – i.e. it is an array of 0 elements – any user can use the AA, i.e. the calling-AE-invocation-id field of the AARQ is ignored.
	If the user_list attribute is not empty then only the users in the list can establish the AA, i.e. the calling-AE-invocation-id field of the AARQ shall be present and its value shall match one of user_ids in the user_list or else, the AA is not established.

current_user	Holds the identifier of the current user. current_user ::= user_list_entry (see above) If the user_list is empty, then current_user shall be a structure {user_id: unsigned 0, user_name: visible string of 0 elements}

Parameters for selective access to the *object_list* and *access_rights_list* attribute

Access selector value	Parameter	Available with attribute	Comment
1	class_id: long-unsigned	2	Delivers the subset of the object_list for a specific class_id. For the response: data ::= objlist_type

2	structure
	{
	class_id: long-unsigned,
	logical_name: octet-string
	}
		2	Delivers the entry of the object_list for a specific class_id and *logical_name*.
			For the response: data ::= objlist_element

3	base_name: long	2, 3	In the case of attribute 2, delivers the entry of the object_list for a specific base_name. For the response: data ::= objlist_element

© Copyright 1997-2020 DLMS User Association

Access selector value	Parameter	Available with attribute	Comment
			In the case of attribute 3, delivers the entry of the access_rights_list for a specific base_name. For the response: data ::=access_rights_element

Method description	

read_by_ logicalname (data)	Reads attributes for selected objects. The objects are specified by their class_id and their *logical_name*. With this method, the parameterized access feature can also be used.
	data ::= array attribute_identification
	
	attribute_identification ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}
	where attribute_index is a pointer (i.e. offset) to the attribute within the object.
	attribute_index 0 delivers all attributes; attribute_index 1 delivers the first attribute (i.e. *logical_name*), etc.).
	For the response: data is according to the type of the attribute.
	NOTE 2 If at least one attribute has no read access right under the current association, then a *read_by_logicalname* () to attribute index 0 reveals the error message "scope-of-access-violated", see DLMS UA 1000-2 Ed. 10, 9.5.

change_secret (data)	Changes the LLS or HLS secret (for example password). data ::= octet-string new secret NOTE 3 The structure of the "new secret" depends on the security mechanism implemented. The "new secret" may contain additional check bits and it may be encrypted.
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers to the server the result of the secret processing by the client of the server's challenge to the client, f(StoC), as the data service parameter of the Read.request primitive invoked with parameterised access. data ::= octet-string client's response to the challenge If the authentication is accepted, then the response (Read.confirm primitive) contains Result == OK and the result of the secret processing by the server of the client's challenge to the server, f(CtoS) in the data service parameter of the Read.response service. data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.
add_user (data)	Adds a user to the user_list. data ::= user_list_entry (see above)
remove_user (data)	Removes a user from the user_list. data ::= user_list_entry (see above)

© Copyright 1997-2020 DLMS User Association

5.4.5 Association LN (class_id = 15, version = 0)

This IC allows modelling of AAs between a server and a client, using the application context *logical name referencing.* Each AA is modelled by one Association LN object.

Association LN	**0...MaxNbofAss.**	**class_id = 15, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	object_list_type				x + 0x08
3. associated_partners_id	associated_partners_type				x + 0x10
4. application_context_name	context_name_type				x + 0x18
5. xDLMS_context_info	xDLMS-context-type				x + 0x20
6. authentication_mechanism_name	mechanism_name_type				x + 0x28
7. LLS_secret	octet-string				x + 0x30
8. association_status	enum				x + 0x38
Specific methods	***m/o***				
1. reply_to_HLS_authentication (data)	o				x + 0x60
2. change_HLS_secret (data)	o				x + 0x68
3. add_object (data)	o				x + 0x70
4. remove_object (data)	o				x + 0x78

Attribute description	
logical_name	Identifies the “Association LN” object instance. See 6.2.33.

object_list	Contains the list of visible COSEM objects with their class_id, version, logical name and the access rights to their attributes and methods within the given application association.
	object_list_type ::= array object_list_element
	object_list_element ::= structure
	{
	class_id: long-unsigned,
	version: unsigned,
	logical_name: octet-string,
	access_rights: access_right
	}

© Copyright 1997-2020 DLMS User Association

object_list (continued)

```
access_right ::= structure
{
        attribute_access:       attribute_access_descriptor,
        method_access:          method_access_descriptor
}

attribute_access_descriptor ::= array       attribute_access_item

attribute_access_item ::= structure
{
        attribute_id:                   integer,
        access_mode:            enum:
                                    (0) no_access,
                                    (1) read_only,
                                    (2) write_only,
                                    (3) read_and_write

        access_selectors:       CHOICE
        {
                null-data       [0],
                array integer   [1]
        }
}

method_access_descriptor ::= array      method_access_item
method_access_item ::=              structure
{
        method_id:      integer,
        access_mode:    boolean
}
```

Where:

- the attribute_access_descriptor and the method_access_ descriptor always contain all implemented attributes or methods;
- access_selectors contain a list of the supported selector values;

selective access (see 4.1.4) to the attribute *object_list* may be available (optional). The selective access parameters are as defined below.

associated_ partners_id

Contains the identifiers of the COSEM client and server (logical device) APs within the physical devices hosting these processes, which belong to the AA modelled by the “Association LN” object.

```
associated_partners_type ::= structure
{
        client_SAP:     integer,
        server_SAP:             long-unsigned
}
```

The range for the client_SAP is 0…0x7F.

The range for the server_SAP is 0x000…0x3FFF.

© Copyright 1997-2020 DLMS User Association

application_ context_name

In the COSEM environment, it is intended that an application context pre-exists and is referenced by its name during the establishment of an AA. This attribute contains the name of the application context for that AA.

context_name_type ::= CHOICE

{
    context_name_structure [2],
    octet-string [9]
}

The application context name is specified as OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.2.

When the context_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.

context_name_structure ::= structure

{
    joint_iso_ctt_element: unsigned,
    country_element: unsigned,
    country_name_element: long-unsigned,
    identified_organization_element: unsigned,
    DLMS_UA_element: unsigned,
    application_context_element: unsigned,
    context_id_element: unsigned
}

Example 1: In the case of context_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 01 11 01 (all values are hexadecimal).

When the context_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.

Example 2: In the case of context_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 01 01 (all values are hexadecimal).

xDLMS_ context_info

Contains all the necessary information on the xDLMS context for the given AA.

xDLMS-context-type ::= structure

{
    conformance: bit-string,
    max_receive_pdu_size: long-unsigned,
    max_send_pdu_size: long-unsigned,
    dlms_version_number: unsigned,
    quality_of_service: integer,
    cyphering_info: octet-string
}

Where:

- the conformance element contains the xDLMS conformance block supported by the server;
- the max_receive_pdu_size element contains the maximum length for an xDLMS APDU, expressed in bytes that the client may send. This is the same as the server-max-receive-pdu-size parameter of the xDLMS initiateResponse APDU (see DLMS UA 1000-2 Ed. 10, 9.5);
- the max_send_pdu_size, in an active association contains the maximum length for an xDLMS APDU, expressed in bytes that the server may send. This is the same as the client-max-receive-pdu-size parameter of the xDLMS initiateRequest APDU (see DLMS UA 1000-2 Ed. 10, 9.5);

© Copyright 1997-2020 DLMS User Association

xDLMS_ context_info (continued)	- the dlms_version_number element contains the DLMS version number supported by the server; - the quality_of_service element is not used; - the cyphering_info, in an active association, contains the dedicated key parameter of the xDLMS initiateRequest APDU (see DLMS UA 1000-2 Ed. 10, 9.5).

authentication_ mechanism_ name	Contains the name of the authentication mechanism for the AA.
	mechanism_name_type ::= CHOICE
	{
	mechanism_name_structure [2],
	octet-string [9]
	}
	The authentication mechanism name is specified as an OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.3.
	
	When the mechanism_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.
	
	mechanism_name_structure ::= structure
	{
	joint_iso_ctt_element: unsigned,
	country_element: unsigned,
	country_name_element: long-unsigned,
	identified_organization_element: unsigned,
	DLMS_UA_element: unsigned,
	authentication_mechanism_name_element: unsigned,
	mechanism_id_element: unsigned
	}
	Example 3: In the case of mechanism_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 02 11 01 (all values are hexadecimal):
	When the mechanism_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.
	EXAMPLE 4: In the case of mechanism_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 02 01 (all values are hexadecimal).
	No mechanism_name is required when no authentication is used.

LLS_secret	Contains the authentication value for the LLS authentication process.
association_ status	Indicates the current status of the association, which is modelled by the object. enum: (0) non-associated, (1) association-pending, (2) associated

Parameters for selective access to the *object_list* attribute

- if no selective access is requested, (no Access_Selection_Parameters parameter is present in the GET.request (.indication) service primitive for the object_list attribute) the corresponding .response (.confirmation) service shall contain all object_list_element of the *object_list* attribute;
- when selective access is requested to the *object_list* attribute (the Access_Selection_Parameters parameter is present), the response shall contain a 'filtered' list of object_list_element, as follows:

© Copyright 1997-2020 DLMS User Association

Access selector	Access parameter	Comment
1	NULL	All information excluding the access_rights shall be included in the response.
2	class_list	Access by class. In this case, only those object_list_element of the *object_list* shall be included in the response, which have a class_id equal to one of the class_id-s of the class-list. No access_right information is included. class_list ::= array class_id class_id ::= long-unsigned

3	object_id_list	Access by object. The full information record of object instances on the object_id_list shall be returned.
		object_id_list ::= array object_id
		object_id ::= structure
		{
		class_id: long-unsigned,
		logical_name: octet-string
		}

4	object_id	The full information record of the required COSEM object instance shall be returned. object_id: See above.

Method description	
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers the client's “secretly” processed “challenge StoC” (f(StoC)) back to the server as the *data* service parameter of the ACTION.request primitive invoked. data ::= octet-string client’s response to the challenge If the authentication is accepted, then the response (ACTION.confirm primitive) contains Result == OK and the server’s "secretly" processed "challenge CtoS" (f(CtoS)) back to the client in the *data* service parameter of the response service. data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.
change_HLS_ secret (data)	Changes the HLS secret (for example encryption key). data ::= octet-string[a] new HLS secret
add_object (data)	Adds the referenced object to the object_list. data ::= object_list_element (see above)
remove_object (data)	Removes the referenced object from the object_list. data ::= object_list_element (see above)

[a] The structure of the “new secret” depends on the security mechanism implemented. The “new secret” may contain additional check bits and it may be encrypted.

5.4.6 Association LN (class_id = 15, version = 1)

COSEM logical devices able to establish AAs within a COSEM context using LN referencing, model the AAs through instances of the “Association LN” IC. A COSEM logical device has one instance of this IC for each AA the device is able to support.

© Copyright 1997-2020 DLMS User Association

Association LN		0...MaxNbofAss.	class_id = 15, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. object_list	(static)	object_list_type				x + 0x08
3. associated_partners_id		associated_partners_type				x + 0x10
4. application_context_name		context_name_type				x + 0x18
5. xDLMS_context_info		xDLMS_context_type				x + 0x20
6. authentication_mechanism_name		mechanism_name_type				x + 0x28
7. secret		octet-string				x + 0x30
8. association_status		enum				x + 0x38
9. security_setup_reference	(static)	octet-string				x + 0x40
Specific methods		***m/o***				
1. reply_to_HLS_authentication (data)		o				x + 0x60
2. change_HLS_secret (data)		o				x + 0x68
3. add_object (data)		o				x + 0x70
4. remove_object (data)		o				x + 0x78

Attribute description	
logical_name	Identifies the “Association LN” object instance. See 6.2.33.

object_list	Contains the list of visible COSEM objects with their class_id, version, logical name and the access rights to their attributes and methods within the given application association.
	object_list_type ::= array object_list_element
	
	object_list_element ::= structure
	{
	class_id: long-unsigned,
	version: unsigned,
	logical_name: octet-string,
	access_rights: access_right
	}
	
	access_right ::= structure
	{
	attribute_access: attribute_access_descriptor,
	method_access: method_access_descriptor
	}
	attribute_access_descriptor ::= array attribute_access_item

© Copyright 1997-2020 DLMS User Association

object_list (continued)

```
attribute_access_item ::=      structure
{
        attribute_id:          integer,
        access_mode:    enum:

                                    (0)   no_access,
                                    (1)   read_only,
                                    (2)   write_only,
                                    (3)   read_and_write,
                                    (4)   authenticated_read_only,
                                    (5)   authenticated_write_only,
                                    (6)   authenticated_read_and_write


        access_selectors:      CHOICE
        {
              null-data        [0],
              array integer    [1]
        }
}
method_access_descriptor ::= array      method_access_item

method_access_item ::= structure
{
        method_id:       integer,
        access_mode:     enum:
                                    (0)   no_access,
                                    (1)   access,
                                    (2)   authenticated_access
}
```

Where:

- the attribute_access_descriptor and the method_access_descriptor elements always contain all implemented attributes or methods;
- access_selectors contain a list of the supported selector values.

selective access (see 4.1.4) to the attribute *object_list* may be available (optional). The selective access parameters are as defined below.

associated_ partners_ id

Contains the identifiers of the COSEM client and server (logical device) APs within the physical devices hosting these APs, which belong to the AA modelled by the “Association LN” object.

```
associated_partners_type ::= structure
{
        client_SAP:      integer,
        server_SAP:      long-unsigned
}
```

The range for the client_SAP is 0…0x7F.

The range for the server_SAP is 0x0000…0x3FFF.

The SAPs shall be in the range allowed by the data type and the media.

© Copyright 1997-2020 DLMS User Association

application_ context_ name

In the COSEM environment, it is intended that an application context pre-exists and is referenced by its name during the establishment of an AA. This attribute contains the name of the application context for that AA.

context_name_type ::= CHOICE

```
{
    context_name_structure      [2],
    octet-string                [9]
}
```

The application context name is specified as OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.2.

When the context_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.

context_name_structure ::= structure

```
{
    joint_iso_ctt_element:              unsigned,
    country_element:                    unsigned,
    country_name_element:               long-unsigned,
    identified_organization_element:    unsigned,
    DLMS_UA_element:                    unsigned,
    application_context_element:        unsigned,
    context_id_element:                 unsigned
}
```

Example 1: In the case of context_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 01 11 01 (all values are hexadecimal).

When the context_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.

Example 2: In the case of context_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 01 01 (all values are hexadecimal).

xDLMS_ context_info

Contains all the necessary information on the xDLMS context for the given AA.

xDLMS_context_type ::= structure

```
{
    conformance:            bit-string,
    max_receive_pdu_size:   long-unsigned,
    max_send_pdu_size:      long-unsigned,
    dlms_version_number:    unsigned,
    quality_of_service:     integer,
    cyphering_info:         octet-string
}
```

Where:

- the conformance element contains the xDLMS conformance block supported by the server;
- the max_receive_pdu_size element contains the maximum length for an xDLMS APDU, expressed in bytes that the client may send. This is the same as the server-max-receive-pdu-size parameter of the xDLMS initiateResponse APDU;
- the max_send_pdu_size element, in an active AA contains the maximum length for an xDLMS APDU, expressed in bytes that the server may send. This is the same as the client-max-receive-pdu-size parameter of the xDLMS initiateRequest APDU;
- the dlms_version_number element contains the DLMS version number supported by the server;

© Copyright 1997-2020 DLMS User Association

xDLMS_ context_info (continued)	- the quality_of_service element is not used; - the cyphering_info element, in an active association, contains the dedicated key parameter of the xDLMS initiateRequest APDU. See DLMS UA 1000-2 Ed. 10, 9.5.

authentication_ mechanism_ name	Contains the name of the authentication mechanism for the AA.
	mechanism_name_type ::= CHOICE
	{
	mechanism_name_structure [2],
	octet-string [9]
	}
	The authentication mechanism name is specified as an OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.3.
	
	When the mechanism_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.
	
	mechanism_name_structure ::= structure
	{
	joint_iso_ctt_element: unsigned,
	country_element: unsigned,
	country_name_element: long-unsigned,
	identified_organization_element: unsigned,
	DLMS_UA_element: unsigned,
	authentication_mechanism_name_element: unsigned,
	mechanism_id_element: unsigned
	}
	Example 3: In the case of mechanism_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 02 11 01 (all values are hexadecimal):
	When the mechanism_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.
	Example 4: In the case of mechanism_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 02 01 (all values are hexadecimal).
	No mechanism_name is required when no authentication is used.

secret	Contains the secret for the LLS or HLS authentication process.
association_ status	Indicates the current status of the association, which is modelled by the object. enum: (0) non-associated, (1) association-pending, (2) associated
security_setup _ reference	References the Security setup object by its logical name. The referenced object manages security for a given Association LN object instance.

A SET operation on an attribute of an association LN object becomes effective when this association object is used to establish a new association.

© Copyright 1997-2020 DLMS User Association

Parameters for selective access to the object_list attribute

- If no selective access is requested, (no Access_Selection_Parameters parameter is present in the GET.request (.indication) service primitive for the object_list attribute) the corresponding .response (.confirmation) service shall contain all object_list_elements of the object_list attribute.
- When selective access is requested to the object_list attribute (the Access_Selection_Parameter parameter is present), the response shall contain a 'filtered' list of object_list_elements, as follows:

Access selector	Access parameter	Comment
1	NULL	All information excluding the access_rights shall be included in the response.
2	class_list	Access by class_id. In this case, only those object_list_elements of the *object_list* shall be included in the response, which have a class_id equal to one of the class_id-s of the class_list. No access_right information is included. class_list ::= array class_id class_id: long-unsigned

3	object_id_list	Access by object. The full information record of object instances on the object_id_list shall be returned.
		object_id_list ::= array object_id
		object_id ::= structure
		{
		class_id: long-unsigned,
		logical_name: octet-string
		}

4	object_id	The full information record of the required COSEM object instance shall be returned. object_id ::= structure See above.

Method description	
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers to the server the result of the secret processing by the client of the server's challenge to the client, f(StoC), as the *data* service parameter of the ACTION.request primitive invoked. data ::= octet-string client's response to the challenge If the authentication is accepted, then the response (ACTION.confirm primitive) contains Result == OK and the result of the secret processing by the server of the client's challenge to the server, f(CtoS) in the *data* service parameter of the response service. data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.
change_HLS_secret (data)	Changes the HLS secret (for example encryption key). data ::= octet-string new HLS secret The structure of the "new secret" depends on the security mechanism implemented. The "new secret" may contain additional check bits and it may be encrypted.
add_object (data)	Adds the referenced object to the *object_list*. data ::= object_list_element (see above)

© Copyright 1997-2020 DLMS User Association

remove_object (data)	Removes the referenced object from the *object_list*. data ::= object_list_element (see above)

5.4.7 Association LN (class_id = 15, version = 2)

NOTE 1 This new version 2 of the “Association LN” IC supports the client user identification process, see 4.4.2.

COSEM logical devices able to establish AAs within a COSEM context using LN referencing, model the AAs through instances of the “Association LN” IC. A COSEM logical device has one instance of this IC for each AA the device is able to support.

Association LN	**0...MaxNbofAss.**	**class_id = 15, version = 2**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. object_list (static)	object_list_type				x + 0x08
3. associated_partners_id	associated_partners				x + 0x10
4. application_context_name	context_name_type				x + 0x18
5. xDLMS_context_info	xDLMS_context_type				x + 0x20
6. authentication_mechanism_name	mechanism_name_type				x + 0x28
7. secret	octet-string				x + 0x30
8. association_status	enum				x + 0x38
9. security_setup_reference (static)	octet-string				x + 0x40
10. user_list (static)	array				x + 0x48
11. current_user	structure				x + 0x50
Specific methods	***m/o***				
1. reply_to_HLS_authentication (data)	o				x + 0x60
2. change_HLS_secret (data)	o				x + 0x68
3. add_object (data)	o				x + 0x70
4. remove_object (data)	o				x + 0x78
5. add_user (data)	o				x + 0x80
6. remove_user (data)	o				x + 0x88

Attribute description	
logical_name	Identifies the “Association LN” object instance. See 6.2.33.
object_list	Contains the list of visible COSEM objects with their class_id, version, *logical_name* and the access rights to their attributes and methods within the given AA. object_list_type ::= array object_list_element

© Copyright 1997-2020 DLMS User Association

object_list
(continued)

```
object_list_element ::= structure
{
        class_id:               long-unsigned,
        version:                unsigned,
        logical_name:           octet-string,
        access_rights:          access_right
}
access_right ::= structure
{
        attribute_access:       attribute_access_descriptor,
        method_access:          method_access_descriptor
}

attribute_access_descriptor ::= array      attribute_access_item

attribute_access_item ::=      structure
{
        attribute_id:           integer,
        access_mode:    enum:
                                (0)    no_access,
                                (1)    read_only,
                                (2)    write_only,
                                (3)    read_and_write,
                                (4)    authenticated_read_only,
                                (5)    authenticated_write_only,
                                (6)    authenticated_read_and_write

        access_selectors:       CHOICE
        {
                null-data       [0],
                array integer   [1]
        }
}

method_access_descriptor ::= array      method_access_item

method_access_item ::= structure
{
        method_id:      integer,
        access_mode:    enum:
                                (0)    no_access,
                                (1)    access,
                                (2)    authenticated_access
}
```

Where:

- the attribute_access_descriptor and the method_access_descriptor elements always contain all implemented attributes or methods;
- access_selectors contain a list of the supported selector values.

selective access (see 4.1.4) to the attribute *object_list* may be available (optional). The selective access parameters are as defined below.

© Copyright 1997-2020 DLMS User Association

associated_ partners_ id

Contains the identifiers of the COSEM client and server (logical device) APs within the physical devices hosting these APs, which belong to the AA modelled by the “Association LN” object.

associated_partners_type ::= structure

```
{
    client_SAP:     integer,
    server_SAP:     long-unsigned
}
```

The range for the client_SAP is 0…0x7F.

The range for the server_SAP is 0x0000…0x3FFF.

The SAPs shall be in the range allowed by the data type and the media.

application_ context_name

In the COSEM environment, it is intended that an application context pre-exists and is referenced by its name during the establishment of an AA. This attribute contains the name of the application context for that AA.

context_name_type ::= CHOICE

```
{
    context_name_structure      [2],
    octet-string                [9]
}
```

The application context name is specified as OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.2.

When the context_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.

context_name_structure ::= structure

```
{
    joint_iso_ctt_element:              unsigned,
    country_element:                    unsigned,
    country_name_element:               long-unsigned,
    identified_organization_element:    unsigned,
    DLMS_UA_element:                    unsigned,
    application_context_element:        unsigned,
    context_id_element:                 unsigned
}
```

Example 1: In the case of context_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 01 11 01 (all values are hexadecimal).

When the context_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.

Example 2: In the case of context_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 01 01 (all values are hexadecimal).

xDLMS_ context_info

Contains all the necessary information on the xDLMS context for the given AA.

xDLMS_context_type ::= structure

```
{
    conformance:            bit-string,
    max_receive_pdu_size:   long-unsigned,
    max_send_pdu_size:      long-unsigned,
    dlms_version_number:    unsigned,
    quality_of_service:     integer,
    cyphering_info:         octet-string
}
```


© Copyright 1997-2020 DLMS User Association

xDLMS_ context_info (continued)

Where:

- the conformance element contains the xDLMS conformance block supported by the server. The length of the bit-string is 24 bits.
- the max_receive_pdu_size element contains the maximum length for an xDLMS APDU, expressed in bytes that the client may send. This is the same as the server-max-receive-pdu-size parameter of the xDLMS initiateResponse APDU;
- the max_send_pdu_size element, in an active AA contains the maximum length for an xDLMS APDU, expressed in bytes that the server may send. This is the same as the client-max-receive-pdu-size parameter of the xDLMS initiateRequest APDU;
- the dlms_version_number element contains the DLMS version number supported by the server;
- the quality_of_service element is not used;
- the cyphering_info element – in an active AA – contains the dedicated key parameter of the xDLMS initiateRequest APDU. See DLMS UA 1000-2 Ed. 10, 9.5.

authentication_ mechanism_ name

Contains the name of the authentication mechanism for the AA.

```
mechanism_name_type ::=    CHOICE
{
      mechanism_name_structure      [2],
      octet-string                  [9]
}
```

The authentication mechanism name is specified as an OBJECT IDENTIFIER in DLMS UA 1000-2 Ed. 10, 9.4.2.2.3.

When the mechanism_name_type is encoded as a structure, it includes the arc labels of the OBJECT IDENTIFIER.

```
mechanism_name_structure ::= structure
{
      joint_iso_ctt_element:                  unsigned,
      country_element:                        unsigned,
      country_name_element:                   long-unsigned,
      identified_organization_element:        unsigned,
      DLMS_UA_element:                        unsigned,
      authentication_mechanism_name_element:  unsigned,
      mechanism_id_element:                   unsigned
}
```

Example 3: In the case of mechanism_id(1) the A-XDR encoding is: 02 07 11 02 11 10 12 02 F4 11 05 11 08 11 02 11 01 (all values are hexadecimal):

When the mechanism_name_type is encoded as an octet-string, it holds the value of the OBJECT IDENTIFIER. See DLMS UA 1000-2 Ed. 10, 11.4.

Example 4: In the case of mechanism_id(1) the A-XDR encoding is: 09 07 60 85 74 05 08 02 01 (all values are hexadecimal).

No mechanism_name is required when no authentication is used.

secret

Contains the secret for the LLS or HLS authentication process.

© Copyright 1997-2020 DLMS User Association

association_ status	Indicates the current status of the association, which is modelled by the object. enum: (0) non-associated, (1) association-pending, (2) associated
security_setup_ reference	References a “Security setup” object by its logical name. The referenced object manages security for a given “Association LN” object instance.

user_list	Contains the list of users allowed to use the AA managed by the given instance of the “Association LN” IC.
	array user_list_entry
	user_list_entry ::= structure
	{
	user_id: unsigned,
	user_name: visible-string
	}
	Where:
	- user_id is the identifier of the user (this value is carried by the calling-AE-invocation-id field of the AARQ);
	- user_name is the name of the user.
	If the *user_list* attribute is empty – i.e. it is an array of 0 elements – any user can use the AA, i.e. the calling-AE-invocation-id field of the AARQ is ignored.
	If the *user_list* attribute is not empty then only the users in the list can establish the AA, i.e. the calling-AE-invocation-id field of the AARQ shall be present and its value shall match one of user_ids in the *user_list* or else the AA is not established.

current_user	Holds the identifier of the current user. current_user ::= user_list_entry (see above) If the *user_list* is empty, then current_user shall be a structure {user_id: unsigned 0, user_name: visible string of 0 elements}

Parameters for selective access to the *object_list* attribute

- If no selective access is requested, (no Access_Selection_Parameters parameter is present in the GET.request (.indication) service primitive for the object_list attribute) the corresponding .response (.confirmation) service shall contain all object_list_elements of the object_list attribute.
- When selective access is requested to the object_list attribute (the Access_Selection_Parameter parameter is present), the response shall contain a ‘filtered’ list of object_list_elements, as follows:

Access selector	Access parameter	Comment
1	NULL	All information excluding the access_rights shall be included in the response.
2	class_list	Access by class_id. In this case, only those object_list_elements of the *object_list* shall be included in the response, which have a class_id equal to one of the class_id-s of the class_list. No access_right information is included. class_list ::= array class_id class_id: long-unsigned
3	object_id_list	Access by object. The full information record of object instances on the object_id_list shall be returned.

© Copyright 1997-2020 DLMS User Association

Access selector	Access parameter	Comment

		object_id_list ::= array object_id
		object_id ::= structure
		{
		class_id: long-unsigned,
		logical_name: octet-string
		}

4	object_id	The full information record of the required COSEM object instance shall be returned. object_id ::= structure See above.

Method description	
reply_to_HLS_ authentication (data)	The remote invocation of this method delivers to the server the result of the secret processing by the client of the server's challenge to the client, f(StoC), as the *data* service parameter of the ACTION.request primitive invoked. data ::= octet-string client's response to the challenge If the authentication is accepted, then the response (ACTION.confirm primitive) contains Result == OK and the result of the secret processing by the server of the client's challenge to the server, f(CtoS) in the *data* service parameter of the response service. data ::= octet-string server's response to the challenge If the authentication is not accepted, then the result parameter in the response shall contain a non-OK value, and no data shall be sent back.
change_HLS_ secret (data)	Changes the HLS secret (for example encryption key). data ::= octet-string new HLS secret The structure of the "new secret" depends on the security mechanism implemented. The "new secret" may contain additional check bits and it may be encrypted.
add_object (data)	Adds the referenced object to the *object_list*. data ::= object_list_element (see above)
remove_object (data)	Removes the referenced object from the *object_list*. data ::= object_list_element (see above)
add_user (data)	Adds a user to the *user_list*. data ::= user_list_entry (see above)
remove_user (data)	Removes a user from the *user_list*. data ::= user_list_entry (see above)

5.4.8 Security setup (class_id = 64, version = 0)

Instances of this IC contain the necessary information on the security policy applicable and the security suite in use within a particular AA, between two systems identified by their client system title and server system title respectively. They also contain methods to increase the level of security and to transfer the global keys. See DLMS UA 1000-2 Ed. 10 Clause 9.2.

© Copyright 1997-2020 DLMS User Association

Security setup		0...n	class_id = 64, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. security_policy	(static)	enum	0	3	0	x + 0x08
3. security_suite	(static)	enum	0	0	0	x + 0x10
4. client_system_title	(dyn.)	octet-string				x + 0x18
5. server_system_title	(static)	octet-string				x + 0x20
Specific methods		***m/o***				
1. security_activate (data)		o				x + 0x28
2. global_key_transfer (data)		o				x + 0x30

Attribute description	
logical_name	Identifies the “Security setup” object instance. See 6.2.36.
security_policy	Enforces authentication and/or encryption algorithm provided with security_suite. enum: (0) nothing, (1) all messages to be authenticated, (2) all messages to be encrypted, (3) all messages to be authenticated and encrypted (4)...(15) reserved
security_suite	Specifies authentication, encryption and key transport algorithm. enum: (0) AES-GCM-128 for authenticated encryption and AES-128 for key wrapping (1) ...(15) reserved
client_system_title	Carries the (current) client system title: - in the S-FSK PLC environment, the active initiator sends its system title using the CIASE protocol; NOTE 1 It is also held by the *active_initiator* attribute of the S-FSK Active initiator object; see 4.10.4; - during confirmed or unconfirmed AA establishment, it is carried by the calling-AP-title field of the AARQ APDU; - If a client system title has already been sent during a registration process, like in the case of the S-FSK PLC profile, the client system title carried by the AARQ APDU should be the same. If not, the AA shall be rejected and appropriate diagnostic information shall be sent. - in a pre-established AA, it can be written by the client using an unsecured SET / Write service.
server_system_title	Carries the server system title. - in the S-FSK PLC environment, the server sends its system title during the discover process, using the CIASE protocol; - during confirmed AA establishment, it is carried by the responding-AP-title field of the AARE APDU. This attribute shall be read only.

© Copyright 1997-2020 DLMS User Association

Method description	
security_ activate (data)	Activates and strengthens the security policy: enum: (0) nothing, (1) all messages to be authenticated, (2) all messages to be encrypted, (3) all messages to be authenticated and encrypted The new security policy applies as soon as the method invocation has been confirmed with success. NOTE 2 The security policy can only be strengthened.

global_key_ transfer (data)	Updates one or more global keys. The *data* parameter includes wrapped key data. Key data include the key identifiers and the keys themselves.
	array key_data
	
	key_data ::= structure
	{
	key_id: enum: (0) global unicast encryption key,
	(1) global broadcast encryption key,
	(2) authentication key
	key_wrapped: octet-string
	}
	The key wrapping algorithm is as specified by the security suite. The KEK is the master key.
	The new key(s) are valid as soon as the method invocation has been confirmed with success.

5.4.9 Push Setup (class_id = 40, version = 0)

The "Push setup" IC contains a list of references to COSEM object attributes to be pushed. It also contains the push destination and method as well as the communication time windows and the handling of retries.

The push takes place upon invoking the *push* method, triggered by a Push "Single action schedule" object, by an alarm "Register monitor" object, by a dedicated internal event or externally. After the push operation has been triggered, it is executed according to the settings made in the given "Push setup" object. Depending on the communication window settings, the push is executed immediately or as soon as a communication window becomes active, after a random delay. If the push was not successful, retries are made. Push windows, delays and retries are shown in Figure 31.

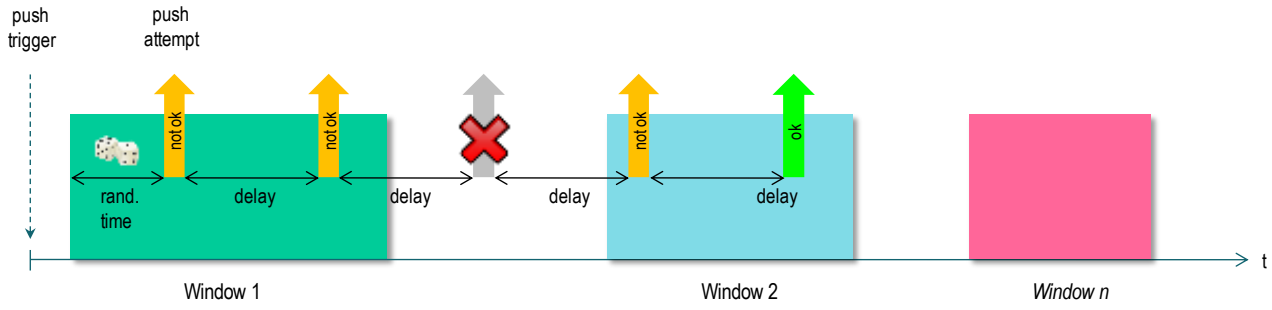


Figure 31 – Push windows and delays

© Copyright 1997-2020 DLMS User Association

Push setup		0...n	class_id = 40, version = 0			
Attribute (s)		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. push_object_list	(static)	array				x + 0x08
3. send_destination_and_method	(static)	structure				x + 0x10
4. communication_window	(static)	array				x + 0x18
5. randomisation_start_interval	(static)	long-unsigned				x + 0x20
6. number_of_retries	(static)	unsigned				x + 0x28
7. repetition_delay	(static)	long-unsigned				x + 0x30
Specific methods		***m/o***				
1. push (data)		m				x + 0x38

Attribute description

logical_name Identifies the “Push setup” object instance. See 6.2.24.

push_ object_list

Defines the list of attributes to be pushed.

Upon invocation of the *push* (data) method the items are sent to the destination defined in the *send_destination_and_method* attribute.

array object_definition

```
object_definition ::=     structure
{
        class_id:               long-unsigned,
        logical_name:           octet-string,
        attribute_index:        integer,
        data_index:             long-unsigned
}
```

Where:

- attribute_index is a pointer to the attribute within the object, identified by class_id and *l*ogical_name: attribute_index 1 refers to the 1st attribute (i.e. the *logical_name*), attribute_index 2 to the 2nd attribute etc.; attribute_index 0 refers to all public attributes;
- data_index is a pointer selecting one or several specific elements of an attribute with a complex data type (structure or array).

data_index	MS-Byte		LS-Byte
	Upper nibble	Lower nibble	

If the data_type of the attribute is simple, then data_index has no meaning.

If the attribute is a structure or an array, then data_index points to an element in the structure or array. The first element in the complex attribute is identified by data_index 1.

© Copyright 1997-2020 DLMS User Association

push_ object_list (continued)

When the attribute is the *buffer* of a “Profile generic” object, the data_index carries selective access parameters.

- 0x0000 = identifies the whole attribute;
- 0x0001 to 0x0FFF = identifies one element in the complex attribute;
- 0x1000 to 0xFFFF = selective access to the array holding the *buffer* of a “Profile generic” object. The data-index selects entries within a number of last (recent) time periods, or a number of last (recent) entries, as well as the columns in the array.

The encoding is specified in Table 37.

NOTE 1 If the push_object_list array is empty, the push operation is disabled.

NOTE 2 The push_object_list attribute itself can be pushed as well to clearly identify the data pushed.

Similarly to the case of the “Profile generic” IC, all attributes included in the push_object_list attribute are pushed regardless of the access rights to them. Therefore, writing of the push_object_list attribute should be restricted to clients with appropriate access rights.

send_ destination_ and_method

Contains the destination address (e.g. phone number, email address, IP address) where the data specified by the *push_object_list* has to be sent, as well as the sending method.

send_destination_and_method ::= structure

```
{
    transport_service:    transport_service_type,
    destination:          octet-string,
    message:              message_type
}
```

Where:

– the transport_service element defines the type of service used to push the data:

transport_service_type ::= enum:

```
(0)        TCP,
(1)        UDP,
(2)        reserved for FTP,
(3)        reserved for SMTP,
(4)        SMS,
(5)        HDLC,
(6)        reserved for M-Bus,
(7)        reserved for ZigBee®,
(200...255) manufacturer specific
```

– the destination element contains the target address where the data has to be sent. The elements of the target address depend on the transport service used.

Each “Push setup” object instance specifies a single destination. If it is required to push data to several destinations, several “Push setup” objects have to be instantiated,

© Copyright 1997-2020 DLMS User Association

send_ destination_ and_method (continued)	– the message_type element identifies the encoding of the xDLMS APDU used. message_type ::= enum: (2) A-XDR encoded xDLMS APDU, (3) XML encoded xDLMS APDU, (128...255) manufacturer specific – all other transport_service_type and message_type values are reserved for future use.

communication_ window	Defines the time points when the communication window(s) for the push become(s) active (start_time) and inactive (end_time). See Figure 31.
	array window_element
	window_element ::= structure
	{
	start_time: octet-string,
	end_time: octet-string
	}
	start_time and end_time are formatted as specified in Clause 4.1.6.1 for *date-time* including wildcards.
	If the end of a communication window is reached an already started push operation is completed.
	If no communication windows are defined (array [0]) the push operation is always possible.

randomisation_ start_ interval	To avoid simultaneous push operations by a lot of devices at exactly the same point in time, a randomisation interval – in seconds – can be defined. This means that the push operation is not started immediately after the invocation of the *push* method but randomly delayed within the interval. The *randomisation_start_interval* attribute defines the maximum value which can be obtained from a randomizing algorithm. The resulting value is used to delay the first push operation. It is not used anymore in case of retries. If no communication window is active when invoking the *push* method, the delay starts at the beginning of the next communication window. If the *push* method is invoked during an active communication window, the push operation is still delayed. The *randomisation_start_interval* is only active for the first push attempt. If no *communication_window* is defined, the *randomisation_start_interval* is active for every push attempt. If the *randomisation_start_interval* is set to 0 no delay is active. If the randomisation time is longer than the first communication window, the first push attempt will be made during any later window.
number_of_ retries	Defines the maximum number of retries in the case of unsuccessful or skipped push attempts. After a successful push operation no further push attempts are made until the push operation is triggered again. A push is treated as successful if a lower layer transmission confirmation has been received. The conditions of detecting an unsuccessful push attempt may depend on the communication profile used and on the implementation and it is therefore out of the Scope of this document.

© Copyright 1997-2020 DLMS User Association

repetition_ delay	The time delay, expressed in seconds until the next push attempt is started after an unsuccessful push. NOTE 3 The repetition delay itself is not influenced by the communication window. But a retry only can be made if a communication window is active at that time. Otherwise it is handled like an unsuccessful push attempt. NOTE 4 The push data is not stored in an intermediate buffer. In the case of retries, the current values of the attributes may change with every push attempt.
Method description	
push (data)	Activates the push process leading to the elaboration and the sending of the push data taking into account the values of the attributes defined in the given instance of this IC. data ::= integer (0)

Table 37 – Encoding of selective access parameters with data_index

	MS_Byte upper nibble: Selects the time periods or entries.
0xF	**Last complete number of months:** Selective access to the *buffer* resulting in all entries of the last complete number of months and the first entry at midnight of the current month.
0xE	**Last complete number of days:** Selective access to the *buffer* resulting in all entries of the last complete number of days and the first entry at midnight of today.
0xD	**Last complete number of hours:** Selective access to the *buffer* resulting in all entries of the last complete number of hours and the first entry of the current hour.
0xC	**Last complete number of minutes:** Selective access to the *buffer* resulting in all entries of the last complete number of minutes and the first entry of the current minute.
0xB	**Last number of seconds:** Selective access to the *buffer* resulting in all entries of the last number of seconds.
0xA	**Last complete number of months including the current month:** Same as 0xF above but all entries up to now are retrieved.
0x9	**Last complete number of days including the current day:** Same as 0xE above but all entries up to now are retrieved.
0x8	**Last complete number of hours including the current hour:** Same as 0xD above but all entries up to now are retrieved.
0x7	**Last complete number of minutes including the current minute:** This is the same as 0xC above but all entries up to now are retrieved.
0x6…0x2	Reserved
0x1	**Last number of entries**
0x0	**Whole attribute** or a single element in an attribute with complex data type is selected (MS-Byte lower nibble 0x0…0xF, LS-Byte 0x00…0xFF).
	MS-Byte lower nibble: Defines the number of columns of a “Profile generic” *buffer* selected.
0x0	All columns
0x1 to 0xF	Number of columns, starting from column 1.
0x00…0xFF	**LS-Byte:** - in the case of selective access by time period, (i.e. number of month, days, hours …) defines the number of recent complete time periods (1 to 255), - in the case of selective access by entry defines the number of recent entries.

© Copyright 1997-2020 DLMS User Association

Example 1)	0xE401: The entries for the last complete day are selected. The first 4 columns are included.
Example 2)	0xA300: The entries for zero last complete months and the current month are selected . The first 3 columns are included.
Example 3)	0x800C: The entries for the last complete 12 hours are selected. All columns are included.
Example 4)	0x1080: The last 128 entries are selected. All columns are included.

5.4.10 Push Setup (class_id = 40, version = 1)

The "Push setup" interface class contains a list of references to COSEM object attributes to be pushed. It also contains the push destination and method as well as the communication time windows and the handling of retries.

In version 1 the possibility of data protection has been added offering the same options as defined in the "Data protection" IC.

This version of the interface class is intended to facilitate the use of relative and absolute data selection. It would be common practice to use an instance of the Push setup IC with relative data selection for routine data push, and an instance of the Push setup IC with absolute data for special cases.

The push takes place upon invoking the push method, triggered by a Push "Single action schedule" object, by an Alarm "Register monitor" object, by a dedicated internal event or externally. After the push operation has been triggered, it is executed according to the settings made in the given "Push setup" object. Depending on the communication window settings, the push is executed immediately or as soon as a communication window becomes active, after a random delay. If the push was not successful, retries are made. Push windows, delays and retries are shown in Figure 32.

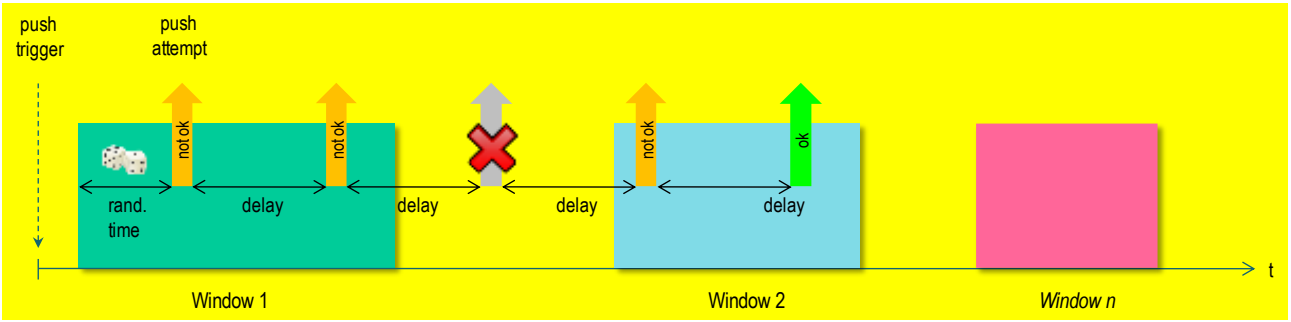


Figure 32 – Push windows and delays

© Copyright 1997-2020 DLMS User Association

Push setup		0...n	class_id = 40, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. push_object_list	(static)	array				x + 0x08
3. send_destination_and_method	(static)	structure				x + 0x10
4. communication_window	(static)	array				x + 0x18
5. randomisation_start_interval	(static)	long-unsigned				x + 0x20
6. number_of_retries	(static)	unsigned				x + 0x28
7. repetition_delay	(static)	long-unsigned				x + 0x30
8. port_reference	(static)	octet-string				x + 0x38
9. push_client_SAP	(static)	integer				x + 0x40
10. push_protection_parameters	(static)	array				x + 0x48
Specific methods		***m/o***				
1. push (data)		m				x + 0x58

Attribute description

logical_name	Identifies the “Push setup” object instance. See 6.2.24.
push_object_list	Defines the list of attributes to be pushed. Upon invocation of the *push* (data) method the items are sent to the destination defined in the *send_destination_and_method* attribute. Two, mutually exclusive selective access mechanisms are available : – relative selective access, i.e. entries defined relative to current date or entry are returned: this mechanism is controlled by the *data_index* element; or – absolute selective access, i.e. entries in an explicitly defined date range or entry range are returned: entries selected by this mechanism are controlled by the *restriction_element* and the columns are controlled by the lower nibble of the MS byte in the *data_index*.

© Copyright 1997-2020 DLMS User Association

push_object_list (continued)

array push_object_definition

push_object_definition ::= structure

{

class_id:	long-unsigned,
logical_name:	octet-string,
attribute_index:	integer,
data_index:	long-unsigned,
restriction:	restriction_element

}

Where:

- attribute_index is a pointer to the attribute within the object, identified by class_id and logical_name: attribute_index 1 refers to the first attribute (i.e. the logical_name), attribute_index 2 to the second attribute etc.; attribute_index 0 refers to all public attributes, noting DLMS UA 1000-2 Ed. 10, 9.1.4.3.7;
- data_index is a pointer selecting a specific element of an attribute with a complex data type (structure or array).
  - if the data type of the attribute is simple, then data_index has no meaning;
  - if the data type of the attribute is a structure or an array – other than the *buffer* of a Profile generic object – then data_index points to one or several specific elements in the structure or array;
  - when the attribute is the *buffer* of a “Profile generic” object, the data_index carries selective access parameters relative to current date or entry.

data_index	**MS-Byte**		**LS-Byte**
	Upper nibble	Lower nibble	

- 0x0000 = the whole attribute is pushed.
- 0x0001 to 0x0FFF = identifies one element in the complex attribute;
- 0x1000 to 0xFFFF = selective access to the array holding the buffer of a “Profile generic” object. The data-index selects entries within a number of last (recent) time periods, or a number of last (recent) entries, as well as the columns in the array.

The encoding is specified in Table .

When the attribute is the buffer of a “Profile generic” object, then restriction_element specifies absolute selective access parameters in an explicitly defined date range or entry range.

© Copyright 1997-2020 DLMS User Association

push_object_list (continued)

```
restriction_element::= structure
{
        restriction_type:       enum:
                                (0) none,
                                (1) range by date,
                                (2) range by entry
        restriction_value: CHOICE
        {
                null-data,                  // no restrictions apply
                restriction_by_date,
                restriction_by_entry
        }
}
restriction_by_date ::= structure
{
        from_date: octet-string,
        to_date: octet-string
}
restriction_by_entry ::= structure
{
        from_entry: double-long-unsigned,
        to_entry: double-long-unsigned
}
```

- restriction_element defines absolute selective access to a “Profile generic” buffer by date range (from_date to to_date) or by entries (from_entry to to_entry). To use this absolute selective access mechanism, data_index shall be :
- MS Byte upper nibble set to 0x0;
- MS Byte lower nibble set to 0x0 to 0xF in accordance with Table ;
- lower byte set to 0x00.
- restriction_element is composed of restriction_type and restriction_value:
  - for restriction by date range the restriction_type element holds (1) restriction by date and the restriction_value element holds restriction_by_date structure;
  - for restriction by entries the restriction_type element holds (2) restriction by entry and the restriction_value element holds restriction_by_entry structure;
  - otherwise the restriction_type element holds (0) none and the restriction_value element holds null-data. This choice shall be taken also if relative selective access is to be used.

NOTE 1 If the push_object_list array is empty, the push operation is disabled.

NOTE 2 The push_object_list attribute itself can be also pushed to identify the data pushed.

Similarly to the case of the “Profile generic” IC, all attributes included in the push_object_list attribute are pushed regardless of the access rights to them. Therefore, writing of the push_object_list attribute should be restricted to clients with appropriate access rights.

© Copyright 1997-2020 DLMS User Association

send_ destination_ and_method

Contains the destination address (e.g. phone number, email address, IP address) where the data specified by the *push_object_list* has to be sent, as well as the sending method.

```
send_destination_and_method ::= structure
{
    transport_service:   transport_service_type,
    destination:         octet-string,
    message:             message_type,
}
```

Where:

- the transport_service element defines the type of service used to push the data:

```
transport_service_type ::=      enum:
                                (0)       TCP,
                                (1)       UDP,
                                (2)       reserved for FTP,
                                (3)       reserved for SMTP,
                                (4)       SMS,
                                (5)       HDLC,
                                (6)       M-Bus,
                                (7)       reserved for ZigBee ®,
                                (8)       DLMS gateway,
                                (200...255)   manufacturer specific
```

- the destination element contains the target address where the data has to be sent. The elements of the target address depend on the transport service used.

  Each “Push setup” object instance specifies a single destination. If it is required to push data to several destinations, several “Push setup” objects have to be instantiated.

- the message_type element identifies the encoding of the xDLMS APDU used.

```
message_type::= enum:
                        (4)             A-XDR encoded xDLMS APDU,
                        (5)             XML encoded xDLMS APDU,
                        (128...255)     manufacturer specific
```

- all other transport_service_type and message_type values are reserved for future use.

communication_ window

Defines the time points when the communication window(s) for the push become(s) active (start_time) and inactive (end_time). See Figure 32.

```
array       window_element
            window_element ::= structure
            {
            start_time:  octet-string,
            end_time:    octet-string
            }
```

start_time and end_time are formatted as specified in 4.1.6.1 for *date-time* including wildcards.

© Copyright 1997-2020 DLMS User Association

communication_ window (continued)	If the end of a communication window is reached an already started push operation is completed. If no communication windows are defined (array [0]) the push operation is always possible.
randomisation_ start_ interval	To avoid simultaneous push operations by a lot of devices at exactly the same point in time, a randomisation interval – in seconds – can be defined. This means that the push operation is not started immediately after the invocation of the push method but randomly delayed within the interval. The *randomisation_start_interval* attribute defines the maximum value which can be obtained from a randomizing algorithm. The resulting value is used to delay the first push operation. It is not used anymore in case of retries. If no communication window is active when invoking the *push* method, the delay starts at the beginning of the next communication window. If the push method is invoked during an active communication window, the push operation is still delayed. The *randomisation_start_interval* is only active for the first push attempt. If no *communication_window* is defined, the randomisation_start_interval is active for every push attempt. If the randomisation_start_interval is set to 0 no delay is active. If the randomisation time is longer than the first communication window, the first push attempt will be made during any later window.
number_of_ retries	Defines the maximum number of retries in the case of unsuccessful or skipped push attempts. After a successful push operation no further push attempts are made until the push operation is triggered again. A push is treated as successful if a lower layer transmission confirmation has been received. The conditions of detecting an unsuccessful push attempt may depend on the communication profile used and on the implementation and it is therefore out of the Scope of this specification.
repetition_ delay	The time delay, expressed in seconds until the next push attempt is started after an unsuccessful push. NOTE 3 The repetition delay itself is not influenced by the communication window. But a retry can only be made if a communication window is active at that time. Otherwise it is handled like an unsuccessful push attempt. NOTE 4 The push data is not stored in an intermediate buffer. In the case of retries, the current values of the attributes may change with every push attempt.
port_reference	Contains the logical name of a communication port setup object allowing the selection of a specific communication channel for the push based on the transport_service_type. This mainly applies in cases where several channels of the same type are supported. If this information is not available or not needed, the attribute may be left empty (octet-string [0]).
push_ client_SAP	Defines the client SAP where the push is directed to in the supporting layer of the DataNotification service. The push process takes place within the application context of the AA which is linked to the *push_client_SAP* attribute. The security context is determined by the “Security setup” object referenced in the related “Association” object.

© Copyright 1997-2020 DLMS User Association

push_protection_parameters

Specifies all protection parameters to be applied to the DataNotification APDU when the push data is sent. It offers the same options as the “Data Protection” IC but it is bound to the sending of the data defined in the *push_object_list* attribute.

array protection_parameters_element

protection_parameters_element ::= structure
{
  protection_type: enum:
    (0) authentication,
    (1) encryption,
    (2) authentication and encryption,
    (3) digital signature

  protection_options: structure
  {
    transaction_id: octet-string,
    originator_system_title: octet-string,
    recipient_system_title: octet-string,
    other_information: octet-string,
    key_info: key_info_element
  }
}

Where:

– transaction_id holds the identifier of the transaction;

– originator_system_title holds the system title of the originator that applies the protection;

– recipient_system_title holds the system title of the recipient which will check and remove the given protection;

– other_information carries other information. Its contents may be specified in project specific companion specifications. An octet-string of length 0 indicates that this field is not used;

– key_info holds the information necessary for the recipient to obtain the right key for checking and removing authentication and encryption. In the case of digital signature, key_info is not necessary and it shall be a structure of 0 elements.

The fields transaction-id ….other-information are A-XDR encoded OCTET STRINGs. The length and the value of each field are included in the AAD when applicable.

© Copyright 1997-2020 DLMS User Association

push_protection_parameters (continued)	key_info_element ::= structure
	{
	key_info_type: enum:
	(0) identified_key,
	-- used with identified_key_info_options
	(1) wrapped_key,
	-- used with wrapped_key_info_options
	(2) agreed_key
	-- used with agreed_key_info_options
	key_info_options: CHOICE
	{
	identified_key_info_options,
	wrapped_key_info_options,
	agreed_key_info_options
	}
	}
	identified_key_info_options ::= enum:
	(0) global_unicast_encryption_key,
	(1) global_broadcast_encryption_key
	wrapped_key_info_options ::= structure
	{
	kek_id: enum:
	(0) master_key,
	key_ciphered_data: octet-string
	}
	agreed_key_info_options ::= structure
	{
	key_parameters: octet-string,
	key_ciphered_data: octet-string
	}
	This attribute is first written by the client. The server may need to fill in some elements.
	For the use of the various elements see Table 17 and Table 18 of the ”Data protection” IC (class_id = 30, version = 0).

Method description	
push(data)	Activates the push process leading to the elaboration and the sending of the push data taking into account the values of the attributes defined in the given instance of this class. data ::= integer(0)

© Copyright 1997-2020 DLMS User Association

5.5 Previous versions of interface classes for time- and event- bound control

5.5.1 Parameter monitor (class_id = 65, version = 0)

Instances of the “Parameter monitor” IC monitor a list of COSEM object attributes holding parameters.

The parameters can be changed as usual. If the value of an attribute changes and this attribute is present in the *parameter_list* attribute, the identifier and the value of that attribute is automatically captured to the *changed_parameter* attribute. The time when the change of the parameter occurred is captured in the *capture_time* attribute. These attributes may be captured then by a “Profile generic” object. In this way, a log of all parameter changes can be built. For the OBIS code of the Parameter monitor log objects, see 7.4.5.

NOTE 1 In the case of simultaneous or quasi simultaneous parameter changes the order of capturing and logging the changed parameters has to be managed by the application.

Several “Parameter monitor” objects and corresponding “Profile generic” objects can be instantiated to manage a number of parameter groups. The link between the “Parameter monitor” object and the corresponding “Profile generic” object is via the *capture_object* attribute of the “Profile generic” object.

NOTE 2 As the various parameters may be of different type and length, the entries in the profile column holding the parameters will be also of different type and length. This can be managed for example by capturing different kind of parameters into different Parameter list “Profile generic” objects and parameter logs.

NOTE 3 The “Profile generic” object holding the parameter change log may capture other suitable object attributes, like the *time* attribute of the “Clock” object, and any other relevant values.

Parameter monitor	**0...n**	**class_id = 65, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. changed_parameter	structure				x + 0x08
3. capture_time	date-time				x + 0x10
4. parameter_list	array				x + 0x18
Specific methods	***m/o***				
1. add_parameter (data)	o				x + 0x20
2. delete_parameter (data)	o				x + 0x28

Attribute description	
logical_name	Identifies the “Parameter monitor“ object instance. See 6.2.14.

changed_parameter	Holds the identifier and the value of the most recently changed parameter.
	structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer,
	attribute_value: CHOICE
	-- CHOICE as specified in the “Data” interface class
	}

© Copyright 1997-2020 DLMS User Association

capture_time	Provides data and time information showing when the value of the *changed_parameter* attribute has been captured. *date-time* is formatted as specified in 4.1.6.1.

parameter_list	Holds the list of parameters managed by a given instance of the "Parameter monitor" IC.
	parameter_list ::= array parameter_list_element
	
	parameter_list_element ::= structure
	{
	class_id: long-unsigned,
	logical_name: octet-string,
	attribute_index: integer
	}
	NOTE 4 The list of parameters monitored may be changed by using the *add_parameter* or *delete_parameter* methods or writing this attribute.

Method description	
add_parameter (data)	Adds one parameter to the *parameter_list*. data ::= parameter_list_element NOTE 5 A parameter can be logged as soon as it is added to the list. Adding an element to the list does not affect the *buffer* of the "Profile generic" object capturing the *changed_parameter* attribute.
delete_parameter (data)	Deletes one parameter from the *parameter_list*. data ::= parameter_list_element NOTE 6 When a parameter is deleted from the parameter list, its changes will not be logged any more. Removing an element from the list does not affect the *buffer* of the "Profile generic" object capturing the *changed_parameter* attribute.

5.6 Previous versions of payment metering related interface classes

There are no previous versions to report.

5.7 Previous versions of interface classes for setting up data exchange via local ports and modems

5.7.1 IEC local port setup (class_id = 19, version = 0)

Instances of this IC define the operational parameters for communication using IEC 62056-21:2002. Several ports can be configured.

© Copyright 1997-2020 DLMS User Association

IEC local port setup		0...n	class_id = 19, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. default_mode	(static)	enum				x + 0x08
3. default_baud	(static)	enum				x + 0x10
4. prop_baud	(static)	enum				x + 0x18
5. response_time	(static)	enum				x + 0x20
6. device_addr	(static)	octet-string				x + 0x28
7. pass_p1	(static)	octet-string				x + 0x30
8. pass_p2	(static)	octet-string				x + 0x38
9. pass_w5	(static)	octet-string				x + 0x40
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “IEC local port setup” object instance. See 6.2.18.
default_mode	Defines the protocol used by the meter on the port. enum: (0) protocol according to IEC 62056-21:2002 (modes A...E) (1) protocol according to DLMS UA 1000-2 Ed. 10 Clause 8. Using this enumeration value all other attributes of this IC are not applicable.
default_baud	Defines the baud rate for the opening sequence enum: (0) 300 baud, (1) 600 baud, (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud, (8) 57 600 baud, (9) 115 200 baud
prop_baud	Defines the baud rate to be proposed by the meter enum: (0) 300 baud, (1) 600 baud, (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud, (8) 57 600 baud, (9) 115 200 baud
response_time	Defines the minimum time between the reception of a request (end of request telegram) and the transmission of the response (begin of response telegram).

© Copyright 1997-2020 DLMS User Association

	enum: (0) 20 ms, (1) 200 ms
device_addr	Device address according to IEC 62056-21:2002.
pass_p1	Password 1 according to IEC 62056-21:2002.
pass_p2	Password 2 according to IEC 62056-21:2002.
pass_w5	Password W5 reserved for national applications.

5.7.2 IEC HDLC setup, (class_id = 23, version = 0)

An instance of the “IEC HDLC setup” contains all data necessary to set up a communication channel according to Clause 8 of DLMS UA 1000-2 Ed. 10. Several communication channels can be configured.

IEC HDLC setup		**0...n**	**class_id = 23, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. comm_speed	(static)	enum	0	9	5	x + 0x08
3. window_size_transmit	(static)	unsigned	1	7	1	x + 0x10
4. window_size_receive	(static)	unsigned	1	7	1	x + 0x18
5. max_info_field_length_ transmit	(static)	unsigned	32	128	128	x + 0x20
6. max_info_field_length_ receive	(static)	unsigned	32	128	128	x + 0x28
7. inter_octet_time_out	(static)	long-unsigned	20	1000	25	x + 0x30
8. inactivity_time_out	(static)	long-unsigned	0		120	x + 0x38
9. device_address	(static)	long-unsigned	0x0010	0x3FFD		x + 0x40
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “IEC HDLC setup” object instance. See 6.2.20.
comm_speed	The communication speed supported by the corresponding port: enum: (0) 300 baud, (1) 600 baud, (2) 1 200 baud, (3) 2 400 baud, (4) 4 800 baud, (5) 9 600 baud, (6) 19 200 baud, (7) 38 400 baud, (8) 57 600 baud, (9) 115 200 baud This communication speed can be overridden if the HDLC mode of a device is entered through a special mode of another protocol.

© Copyright 1997-2020 DLMS User Association

window_size_ transmit	The maximum number of frames that a device or system can transmit before it needs to receive an acknowledgement from a corresponding station. During logon, other values can be negotiated.
window_size_ receive	The maximum number of frames that a device or system can receive before it needs to transmit an acknowledgement to the corresponding station. During logon, other values can be negotiated.
max_info_length_ transmit	The maximum information field length that a device can transmit. During logon, a smaller value can be negotiated.
max_info_length_ receive	The maximum information field length that a device can receive. During logon, a smaller value can be negotiated.
inter_octet_time_out	Defines the time, expressed in milliseconds, over which, when no character is received from the primary station, the device will treat the already received data as a complete frame.
inactivity_time_out	Defines the time, expressed in seconds over which, when no frame is received from the primary station, the device will process a disconnection. When this value is set to 0, this means that the *inactivity_time_out* is not operational.
device_address	Contains the physical device address of a device: In the case of single byte addressing: 0x00 NO_STATION Address, 0x01…0x0F Reserved for future use, 0x10...0x7D Usable address space, 0x7E ‘CALLING’ device address, 0x7F Broadcast address In the case of double byte addressing: 0x0000 NO_STATION address, 0x0001...0x000F Reserved for future use, 0x0010…0x3FFD Usable address space, 0x3FFE ‘CALLING’ physical device address, 0x3FFF Broadcast address

5.7.3 IEC twisted pair (1) setup (class_id = 24, version = 0)

NOTE The use of version 0 of the IEC twisted pair (1) setup IC is deprecated.

This IC allows modelling and configuring communication channels according to IEC 62056-31:1999. Several communication channels can be configured.

IEC twisted pair (1) setup		**0...n**	**class_id = 24, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. secondary_address	(static)	octet-string				x + 0x08
3. primary_address_list	(static)	primary_address_list_type				x + 0x10
4. tabi_list	(static)	tabi_list_type				x + 0x18
5. fatal_error	(dyn.)	enum				x + 0x20
Specific methods		***m/o***				

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “IEC twisted pair setup” object instance. See 6.2.21.
secondary_address	*Secondary_address* memorizes the ADS of the secondary station that corresponds to the real equipment. octet-string (SIZE(6))
primary_address_ list	*Primary_address_list* memorizes the list of ADP or primary station physical addresses for which each logical device of the real equipment (the secondary station) has been programmed. primary_address_list_type ::= array primary_address_element primary_address_element ::= octet-string The length of the octet-string is one octet.
tabi_list	*tabi_list* represents the list of the TAB(i) for which the real equipment (the secondary station) has been programmed in case of forgotten station call. tabi_list_type ::= array tabi_element tabi_element ::= integer
fatal_error	*fatal_error* represents the last occurrence of one of the fatal errors of the protocols described in IEC 62056-31:1999. The initial default value of this variable is 0x00. Then, each fatal error is spotted. enum: (0) No-error, (1) t-EP-1F, (2) t-EP-2F, (3) t-EL-4F, (4) t-EL-5F, (5) eT-1F, (6) eT-2F, (7) e-EP-3F, (8) e-EP-4F, (9) e-EP-5F, (10) e-EL-2F

MAC address

Secondary stations – typically metering end devices – hold a secondary station address (ADS). The length of the ADS shall be 6 octets and consists of the elements shown in Table 38. This address shall be worldwide unique and it shall assigned by the manufacturer to the secondary station. The ADS assigned is valid through the lifetime of the secondary station.

© Copyright 1997-2020 DLMS User Association

Table 38 – ADS address elements

Elements of ADS	Description	Length (byte)	Type	Range
manufacturer_id	Assigned upon request by the Euridis Association to the manufacturer. It shall be used in all devices using the *Twisted pair with carrier signalling* medium and made by this manufacturer.	1	BCD	0-99
year_of_manufacture	Holds the year of manufacturing the device (the lower two numbers only).	1		0-99
equipment_id	Related to the type of equipment concerned and provides information on the functionality available. Like the manufacturer_id, it is assigned by the Euridis Association.	1		0-99
device_serial_number	The manufacturing number of the device assigned by the manufacturer. It should have the same value as the value held by the object Device ID 1, see Table 60.	3		000001-999999 000000 is reserved
EXAMPLE 031267123456: 03: ITRON International; 12: Year 2012; 67: Smart meter LINKY Single phase 123456: Serial number beginning each year from 000001.				

5.7.4 PSTN modem configuration (class_id = 27, version = 0)

NOTE The name of this IC was changed to “Modem configuration” in Edition 6.0.

An instance of the “PSTN modem configuration” IC stores data related to the initialization of modems, which are used for data transfer from/to a device. Several modems can be configured.

PSTN modem configuration		**0...n**	**class_id = 27, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. comm_speed	(static)	enum	0	9	5	x + 0x08
3. initialization_string	(static)	array				x + 0x10
4. modem_profile	(static)	array				x + 0x18
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “PSTN modem configuration” object instance. See 6.2.6.
comm_speed	The communication speed between the device and the modem, not necessarily the communication speed on the WAN.

© Copyright 1997-2020 DLMS User Association

enum:
(0) 300 baud,
(1) 600 baud,
(2) 1 200 baud,
(3) 2 400 baud,
(4) 4 800 baud,
(5) 9 600 baud,
(6) 19 200 baud,
(7) 38 400 baud,
(8) 57 600 baud,
(9) 115 200 baud

initialization_string

This data contains all the necessary initialization commands to be sent to the modem in order to configure it properly. This may include the configuration of special modem features.

array initialization_string_element

initialization_string_element ::= structure
{
request: octet-string,
response: octet-string
}

If the array contains more than one initialization string element, they are subsequently sent to the modem after receiving an answer matching the defined response.

REMARK It is assumed that the modem is pre-configured so that it accepts the initialization_string. If no initialization is needed, the initialization string is empty.

modem_profile

This data defines the mapping from Hayes standard commands/responses to modem specific strings.

array modem_profile_element

modem_profile_element: octet-string

The *modem_profile* array shall contain the corresponding strings for the modem used in following order :

Element 0:	OK,
Element 1:	CONNECT,
Element 2:	RING,
Element 3	NO CARRIER,
Element 4:	ERROR,
Element 5:	CONNECT 1 200,
Element 6	NO DIAL TONE,
Element 7:	BUSY,
Element 8:	NO ANSWER,
Element 9:	CONNECT 600,
Element 10:	CONNECT 2 400,
Element 11:	CONNECT 4 800,
Element 12	CONNECT 9 600,
Element 13:	CONNECT 14 400,
Element 14:	CONNECT 28 800,
Element 15:	CONNECT 36 600,
Element 16:	CONNECT 56 000

© Copyright 1997-2020 DLMS User Association

5.7.5 Auto answer (class_id = 28, version = 0)

This IC allows modelling how the device manages the “Auto answer” function of the modem, i.e. answering of incoming calls. Several modems can be configured.

Auto answer		**0...n**	**class_id = 28, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mode	(static)	enum				x + 0x08
3. listening_window	(static)	array				x + 0x10
4. status	(dyn.)	enum				x + 0x18
5. number_of_calls	(static)	unsigned				x + 0x20
6. number_of_rings	(static)	nr_rings_type				x + 0x28
Specific methods		***m/o***				

Attribute description

logical_name Identifies the “Auto answer” object instance. See 6.2.6.

mode Defines the working mode of the line when the device is auto answer.

enum:

(0) line dedicated to the device,

(1) shared line management with a limited number of calls allowed. Once the number of calls is reached, the window status becomes inactive until the next start date, whatever the result of the call,

(2) shared line management with a limited number of successful calls allowed. Once the number of successful communications is reached, the window status becomes inactive until the next start date,

(3) currently no modem connected,

(200...255) manufacturer specific modes.

listening_ window Contains the start and end instant when the window becomes active (for the start instant), and inactive (for the end instant). The start_date defines implicitly the period.

EXAMPLE When the day of month is not specified (equal to 0xFF) this means that we have a daily share line management. Daily, monthly ...window management can be defined.

array window_element

window_element ::= structure

{

start _time: octet-string,

end_time: octet-string

}

start_time and end_time are formatted as specified in 4.1.6.1 for *date-time.*

© Copyright 1997-2020 DLMS User Association

status	Here is defined the status of the window. enum: (0) Inactive: the device will manage no new incoming call. This status is automatically reset to Active when the next listening window starts, (1) Active: the device can answer to the next incoming call, (2) Locked: This value can be set automatically by the device or by a specific client when this client has completed its reading session and wants to give the line back to the customer before the end of the window duration. This status is automatically reset to Active when the next listening window starts.
number_ of_calls	This number is the reference used in modes 1 and 2. When set to 0, this means there is no limit.

number_ of_rings	Defines the number of rings before the meter connects the modem. Two cases are distinguished: number of rings within the window defined by attribute *listening_window* and number of rings outside the *listening_window*.
	nr_rings_type ::= structure
	{
	nr_rings_in_window: unsigned, (0: no connect in window)
	nr_rings_out_of_window: unsigned (0: no connect out of window)
	}

5.7.6 PSTN auto dial (class_id = 29, version = 0)

NOTE The name of this IC was changed to “Auto connect” in Edition 6.0.

An instance of the “PSTN auto dial” IC stores data related to the management data transfer between the device and the modem to perform auto dialling. Several modems can be configured.

PSTN auto dial		**0...n**	**class_id = 29, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mode	(static)	enum				x + 0x08
3. repetitions	(static)	unsigned				x + 0x10
4. repetition_delay	(static)	long-unsigned				x + 0x18
5. calling_window	(static)	array				x + 0x20
6. phone_list	(static)	array				x + 0x28
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “PSTN auto dial” object instance. See 6.2.6.

© Copyright 1997-2020 DLMS User Association

mode	Defines if the device can perform auto-dialling. enum: (0) no auto dialling, (1) auto dialling allowed anytime, (2) auto dialling allowed within the validity time of the calling window, (3) “regular” auto dialling allowed within the validity time of the calling window; “alarm” initiated auto dialling allowed anytime, (200…255) manufacturer specific modes.
repetitions	The maximum number of trials In the case of unsuccessful dialling attempts.
repetition_delay	The time delay, expressed in seconds until an unsuccessful dial attempt can be repeated. repetition_delay 0 means delay is not specified.

calling_window	Contains the start and end date/time stamp when the window becomes active (for the start instant), or inactive (for the end instant). The start_date defines implicitly the period.
	EXAMPLE When day of month is not specified (equal to 0xFF) this means that we have a daily share line management. Daily, monthly …window management can be defined.
	array window_element
	
	window_element ::= structure
	{
	start_time: octet-string,
	end_time: octet-string
	}
	start_time and end_time are formatted as specified in 4.1.6.1 for *date-time*.

phone_list	Contains phone numbers, the device modem has to call under certain conditions. The link between entries in the array and the conditions are not contained in here. array phone_number phone_number ::= octet-string

5.7.7 Auto connect (class_id = 29, version = 1)

This IC allows modelling the management of data transfer from the device to one or several destinations.

The messages to be sent, the conditions on which they shall be sent and the relation between the various modes, the calling windows and destinations are not defined here.

Depending on the mode, one or more instances of this IC may be necessary to perform the function of sending out messages.

© Copyright 1997-2020 DLMS User Association

Auto connect		0...n	class_id = 29, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mode	(static)	enum				x + 0x08
3. repetitions	(static)	unsigned				x + 0x10
4. repetition_delay	(static)	long-unsigned				x + 0x18
5. calling_window	(static)	array				x + 0x20
6. destination_list	(static)	array				x + 0x28
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “Auto connect” object instance. See 6.2.6.
mode	Defines the mode controlling the auto dial functionality concerning the timing, the message type to be sent and the infrastructure to be used. enum: (0) no auto dialling, (1) auto dialling allowed anytime, (2) auto dialling allowed within the validity time of the calling window, (3) “regular” auto dialling allowed within the validity time of the calling window; “alarm” initiated auto dialling allowed anytime, (4) SMS sending via Public Land Mobile Network (PLMN), (5) SMS sending via PSTN, (6) email sending, (200..255) manufacturer specific modes
repetitions	The maximum number of trials in the case of unsuccessful dialling attempts.
repetition_delay	The time delay, expressed in seconds until an unsuccessful dial attempt can be repeated. repetition_delay 0 means delay is not specified

calling_window	Contains the start and end date/time stamp when the window becomes active (for the start instant), or inactive (for the end instant). The start_date defines implicitly the period.
	EXAMPLE When day of month is not specified (equal to 0xFF) this means that we have a daily share line management. Daily, monthly ...window management can be defined.
	array window_element
	
	window_element ::= structure
	{
	start_time: octet-string,
	end_time: octet-string
	}
	start_time and end_time are formatted as specified in 4.1.6.1 for *date-time*.

© Copyright 1997-2020 DLMS User Association

destination_list	Contains the list of destinations (for example phone numbers, email addresses or their combinations) where the message(s) have to be sent under certain conditions. The conditions and their link to the elements of the array are not defined here. array destination destination ::= octet-string

5.7.8 GSM diagnostic (class_id = 47, version = 0)

The GSM/GPRS network is undergoing constant changes in terms of registration status, signal quality etc. It is necessary to monitor and log the relevant parameters in order to obtain diagnostic information that allows identifying communication problems in the network.

An instance of the “GSM diagnostic” class stores parameters of the GSM/GPRS network necessary for analysing the operation of the network.

A GSM diagnostic “Profile generic” object is also available to capture the attributes of the GSM diagnostic object, see 7.4.5.

GSM diagnostic		**0…n**	**class_id = 47, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. operator	(dyn.)	visible-string				x + 0x08
3. status	(dyn.)	enum	0	255	0	x + 0x10
4. cs_attachment	(dyn.)	enum	0	255	0	x + 0x18
5. ps_status	(dyn)	enum	0	255	0	x + 0x20
6. cell_info	(dyn.)	cell_info_type				x + 0x30
7. adjacent_cells	(dyn.)	array				x + 0x38
8. capture_time	(dyn.)	date-time				x + 0x40
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “GSM Diagnostic” object instance. For logical names, see 6.2.23.
operator	Holds the name of the network operator e.g. “YourNetOp”
status	Indicates the registration status of the modem. enum: (0) not registered, (1) registered, home network, (2) not registered, but MT is currently searching a new operator to register to, (3) registration denied, (4) unknown, (5) registered, roaming (6) … (255) reserved

© Copyright 1997-2020 DLMS User Association

cs_ attachment	Indicates the current circuit switched status. enum: (0) inactive, (1) incoming call, (2) active, (3) ... (255) reserved
ps_status	The *ps_status* value field indicates the packet switched status of the modem. enum: (0) inactive, (1) GPRS, (2) EDGE, (3) UMTS, (4) HSDPA, (5) ... (255) reserved

cell_info	Represents the cell information:
	cell_info_type ::= structure
	{
	cell_ID: long-unsigned,
	location_ID: long-unsigned,
	signal_quality: unsigned,
	ber: unsigned
	}
	Where:
	- cell_ID: Two-byte cell ID in hexadecimal format;
	- location_ID: Two-byte location area code (LAC) in hexadecimal format (e.g. "00C3" equals 195 in decimal);
	- signal_quality: Represents the signal quality:
	(0) –113 dBm or less,
	(1) –111 dBm,
	(2...30) –109...-53 dBm,
	(31) –51 or greater,
	(99) not known or not detectable;
	- ber: Bit Error Rate (BER) measurement in percent:
	(0...7) as RXQUAL_n values specified in ETSI GSM 05.08, 8.2.4.
	(99) not known or not detectable.

adjacent_ cells	array adjacent_cell_info
	adjacent_cell_info ::= structure
	{
	cell_ID: long-unsigned,
	signal_quality: unsigned
	}
	Where:
	- cell_ID: Two-byte cell ID in hexadecimal format;
	- signal_quality: Represents the signal quality:
	(0) –113 dBm or less,
	(1) –111 dBm,
	(2...30) –109...-53 dBm,
	(31) –51 or greater,
	(99) not known or not detectable.

© Copyright 1997-2020 DLMS User Association

capture_time	Holds the date and time when the data have been last captured. *date-time* is formatted as specified in 4.1.6.1.

5.7.9 GSM diagnostic (class_id = 47, version = 1)

The cellular network is undergoing constant changes in terms of registration status, signal quality etc. It is necessary to monitor and log the relevant parameters in order to obtain diagnostic information that allows identifying communication problems in the network.

An instance of the “GSM diagnostic” class stores parameters of the GSM/GPRS, UMTS, CDMA or LTE network necessary for analysing the operation of the network.

A GSM diagnostic “Profile generic” object is also available to capture the attributes of the GSM diagnostic object, see 7.4.5.

GSM diagnostic		**0...n**	**class_id = 47, version = 1**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. operator	(dyn.)	visible-string				x + 0x08
3. status	(dyn.)	enum	0	255	0	x + 0x10
4. cs_attachment	(dyn.)	enum	0	255	0	x + 0x18
5. ps_status	(dyn)	enum	0	255	0	x + 0x20
6. cell_info	(dyn.)	cell_info_type				x + 0x30
7. adjacent_cells	(dyn.)	array				x + 0x38
8. capture_time	(dyn.)	date-time				x + 0x40
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “GSM Diagnostic” object instance. For logical names, see 6.2.23.
operator	Holds the name of the network operator e.g. “YourNetOp”
status	Indicates the registration status of the modem. enum: (0) not registered, (1) registered, home network, (2) not registered, but MT is currently searching a new operator to register to, (3) registration denied, (4) unknown, (5) registered, roaming, (6)... (255) reserved
cs_attachment	Indicates the current circuit switched status. enum: (0) inactive, (1) incoming call, (2) active, (3) ... (255) reserved

© Copyright 1997-2020 DLMS User Association

ps_status

The *ps_status* value field indicates the packet switched status of the modem.

enum:

(0) inactive,
(1) GPRS,
(2) EDGE,
(3) UMTS,
(4) HSDPA,
(5) LTE,
(6) CDMA,
(7) ... (255) reserved

cell_info

Represents the cell information:

cell_info_type ::= structure

```
{
    cell_ID:              double-long-unsigned,
    location_ID:          long-unsigned,
    signal_quality:       unsigned,
    ber:                  unsigned,
    mcc:                  long-unsigned,
    mnc:                  long-unsigned,
    channel_number:       double-long-unsigned
}
```

Where:

– cell_ID: Four-byte cell ID in hexadecimal format;

– location_ID: Two-byte location area code (LAC) in the case of GSM networks or Tracking Area Code (TAC) in the case of UMTS, CDMA or LTE networks in hexadecimal format (e.g. "00C3" equals 195 in decimal);

– signal_quality: Represents the signal quality:

(0) –113 dBm or less,
(1) –111 dBm,
(2...30) –109...-53 dBm,
(31) –51 or greater,
(99) not known or not detectable;

– ber: Bit Error Rate (BER) measurement in percent:

(0...7) as RXQUAL_n values specified in ETSI GSM 05.08, 8.2.4
(99) not known or not detectable.

– mcc: Mobile Country Code of the serving network, as defined in ITU-T E.212 (05.2008);

– mnc: Mobile Network Code of the serving network, as defined in ITU-T E.212 (05.2008);

– channel_number: Represents the absolute radio-frequency channel number (ARFCN or eaRFCN for LTE network).

© Copyright 1997-2020 DLMS User Association

adjacent_cells	array adjacent_cell_info
	
	adjacent_cell_info ::= structure
	{
	cell_ID: double-long-unsigned,
	signal_quality: unsigned
	}
	Where:
	- cell_ID: Four-byte cell ID in hexadecimal format;
	- signal_quality: Represents the signal quality:
	(0) –113 dBm or less,
	(1) –111 dBm,
	(2…30) –109…-53 dBm,
	(31) –51 or greater,
	(99) not known or not detectable.

capture_time	Holds the date and time when the data have been last captured. *date-time* is formatted as specified in 4.1.6.1.

© Copyright 1997-2020 DLMS User Association

5.7.10 LTE Monitoring (class_id = 155, version = 0)

Instances of the ‘LTE monitoring” IC allow monitoring LTE modems by handling all data necessary data for this purpose.

LTE monitoring		0...n	class_id = 151, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. lte_quality_of_service	(dyn.)	LTE_qos_type				x + 0x08
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “LTE monitoring” object instance. See 6.2.23.

lte_quality_of_ service	Represents the quality of service for the LTE network
	LTE_qos_type ::= structure
	{
	T3402: long-unsigned,
	T3412: long-unsigned,
	RSRQ: unsigned,
	RSRP: unsigned,
	qRxlevMin: integer
	}
	Where:
	- T3402: timer in seconds, used on PLMN selection procedure and sent by the network to the modem. Refer to 3GPP TS 24.301 V13.4.0 (2016-01) for details;
	- T3412: timer in seconds used to manage the periodic tracking area updating procedure and sent by the network to the modem. Refer to 3GPP TS 24.301 V13.4.0 (2016-01) for details;
	- RSRQ: represents the signal quality as defined in 3GPP TS 24.301 V13.4.0 (2016-01):
	(0) –19,5dB,
	(1) –19 dB,
	(2...31) –18,5…-3,5 dB,
	(32) –3dB,
	(99) Not known or not detectable;
	- RSRP: represents the signal level as defined in 3GPP TS 24.301 V13.4.0 (2016-01):
	(0) –140dBm,
	(1) –139 dBm,
	(2... 94) –138…-45 dBm,
	(95) –44dBm,
	(99) Not known or not detectable;
	- qRxlevMin: specifies the minimum required Rx level in the cell in dBm as defined in 3GPP TS 24.301 V13.4.0 (2016-01).

© Copyright 1997-2020 DLMS User Association

5.8 Previous versions of interface classes for setting up data exchange via M-Bus

5.8.1 M-Bus client (class_id = 72, version = 0)

Instances of this IC allow setting up M-Bus slave devices and to exchange data with them. Each M-Bus client object controls one M-Bus slave device. For details on the M-Bus dedicated application layer, see EN 13757-3:2004.

The M-Bus client device may have one or more physical M-Bus interfaces, which can be configured using instances of the M-Bus master port setup IC, see 4.8.5.

An M-Bus slave device is identified with its Primary Address, Identification Number, Manufacturer ID etc. as defined in EN 13757-3:2004 Clause 5, *Variable Data respond*. These parameters are carried by the respective attributes of the M-Bus client IC, see 4.8.3.

Values to be captured from an M-Bus slave device are identified by the *capture_definition* attribute, containing a list of data identifiers (DIB, VIB) for the M-Bus slave device. The data are captured periodically or on an appropriate trigger. Each data element is stored in an M-Bus value object, of “Extended Register” IC. M-Bus value objects may be captured in M-Bus Profile generic objects, eventually along with other, not M-Bus specific objects.

Using the methods of M-Bus client objects, M-Bus slave devices can be installed and de-installed. It is also possible to send data to M-Bus slave devices and to perform operations like resetting alarms, setting the clock, transferring an encryption key etc.

NOTE 1 In Edition 10, the mapping of attributes to short name has been corrected, without a change in the version.

© Copyright 1997-2020 DLMS User Association

M-Bus client		0...n	class_id = 72, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	***Short name***
1. logical_name	(static)	octet-string				x
2. mbus_port_reference	(static)	octet-string				x + 0x08
3. capture_definition	(static)	array				x + 0x10
4. capture_period	(static)	double-long-unsigned				x + 0x18
5. primary_address	(dyn.)	unsigned				x + 0x20
6. identification_number	(dyn.)	double-long-unsigned				x + 0x28
7. manufacturer_id	(dyn.)	long-unsigned				x + 0x30
8. version	(dyn.)	unsigned				x + 0x38
9. device_type	(dyn.)	unsigned				x + 0x40
10. access_number	(dyn.)	unsigned				x + 0x48
11. status	(dyn.)	unsigned				x + 0x50
12. alarm	(dyn.)	unsigned				x + 0x58
Specific methods		***m/o***				
1. slave_install (data)		o				x + 0x60
2. slave_deinstall (data)		o				x + 0x68
3. capture (data)		o				x + 0x70
4. reset_alarm (data)		o				x + 0x78
5. synchronize_clock (data)		o				x + 0x80
6. data_send (data)		o				x + 0x88
7. set_encryption_key (data)		o				x + 0x90
8. transfer_key (data)		o				x + 0x98

Attribute description	
logical_name	Identifies the “M-Bus client” object instance. See 6.2.22.
mbus_port_ reference	Provides reference to an “M-Bus master port setup” object, used to configure an M-Bus port, each interface allowing to exchange data with one or more M-Bus slave devices.

capture_ definition	Provides the capture_definition for M-Bus slave devices.
	array capture_definition_element
	capture_definition_element ::= structure
	{
	data_information_block: octet-string,
	value_information_block: octet-string
	}
	NOTE 2 The elements data_information_block and value_information_block correspond to Data Information Block (DIB) and Value Information Block (VIB) described in EN 13757-3:2013, 6.2 and Clause 7 respectively.

capture_ period	>= 1: Automatic capturing assumed. Specifies the capture period in seconds. 0: No automatic capturing: capturing is triggered externally or capture events occur asynchronously.

© Copyright 1997-2020 DLMS User Association

primary_ address	Carries the primary address of the M-Bus slave device, in the range 0...250. If the slave device is already configured and thus, its primary address is different from 0, then this value shall be written to the *primary_address* attribute. From this moment, the data exchange with the M-Bus slave device is possible. Otherwise, the *slave_install* method shall be used; see below. NOTE 3 The *primary_address* attribute cannot be used to store a desired primary address for an unconfigured slave device. If the primary address attribute is set, this means that the M-Bus client can immediately operate with this primary address, which is not the case with an unconfigured slave device.
identification_ number	Carries the Identification Number element of the data header as specified in EN 13757-3:2004, 5.4. It is either a fixed fabrication number or a number changeable by the customer, coded with 8 BCD packed digits (4 Byte), and which thus runs from 00 000 000 to 99 999 999. It can be preset at fabrication time with a unique number, but could be changeable afterwards, especially if in addition a unique and not changeable fabrication number (DIF = 0x0C, VIF = 0x78, see 7.2 is provided.
manufacturer_ id	Carries the Manufacturer Identification element of the data header as specified in EN 13757-3:2004, 5.5. It is coded unsigned binary with 2 bytes. This *manufacturer_id* is calculated from the ASCII code of the IEC 62056-21 manufacturer ID (three uppercase letters), using the formula specified in EN 13757-3:2004, 5.5.
version	Carries the Version element of the data header as specified in EN 13757-3:2004, 5.6. It specifies the generation or version of the meter and depends on the manufacturer. It can be used to make sure, that within each version number the identification # is unique.
device_type	Carries the Device type identification element of the data header as specified in EN 13757-3:2004, 5.7, Table 3.
access_ number	Carries the Access Number element of the data header as specified in EN 13757-3:2004, 5.8. It has unsigned binary coding, and it is incremented (modulo 256) by one before or after each RSP-UD from the slave. Since it can also be used to enable private end users to detect an unwanted over-frequently readout of their consumption meters, it should not be resettable by any bus communication.
status	Carries the Status byte element of the data header as specified in EN 13757-3:2004, 5.9, Table 4 and 5.
alarm	Carries the Alarm state specified in EN 13757-3:2004 Annex D. It is coded with data type D (Boolean, in this case 8 bit). Set bits signal alarm bits or alarm codes. The meaning of these bits is manufacturer specific.

© Copyright 1997-2020 DLMS User Association

Method description	
slave_ install (data)	Installs a slave device, which is yet unconfigured (its primary address is 0). This method can be successfully invoked only if the value of the primary_address attribute is 0. The following actions are performed: - the M-Bus address 0 is checked for presence of a new device; - if no uninstalled M-Bus slave is found, the method invocation fails; - if the *slave_install* method is invoked with no parameter, then the primary address is assigned automatically. This is done by checking the *primary_address* attribute of all M-Bus client objects in the DLMS/COSEM device and then selecting the first unused number. The primary_address attribute is set to this address and it is then transferred to the M-Bus slave device. if the *slave_install* method is invoked with a primary address as a parameter, then the *primary_address* attribute is set to this value and it is then transferred to the M-Bus slave device. data ::= unsigned (no data, or a valid primary address) NOTE 4 Unconfigured slave devices are configured with primary address as specified in EN 13757-3:2004, Annex E.5.
slave_ deinstall (data)	De-installs the slave device. The main purpose of this service is to uninstall the M-Bus slave device and to prepare the master for the installation of a new device. The following actions are performed: - the M-Bus address is set to 0 in the M-Bus slave device; - the encryption key transferred previously to the M-Bus slave device is destroyed; the default key is not affected; - the attribute *primary_address* is also set to 0. NOTE 5 A new M-Bus slave can be installed only, once the value of the *primary_address* attribute is 0. data ::= integer (0)
capture	Capture values – as specified by the *capture_definition* attribute – from the M-Bus slave device. data ::= integer (0)
reset_alarm	Reset alarm state of the M-Bus slave device. data ::= integer (0)
synchronize_ clock	Synchronize the clock of the M-Bus slave device with that of the M-Bus client device. data ::= integer (0)

© Copyright 1997-2020 DLMS User Association

data_send

Send data to the M-Bus slave device.

```
data ::=      array        data_definition_element
data_definition_element ::= structure
{
      data_information_block:          octet-string,
      value_information_block:         octet-string
      data:                            CHOICE
      {
            -- simple data types
            null-data                  [0],
            bit-string                 [4],
            double-long                [5],
            double-long-unsigned       [6],
            octet-string               [9],
            visible-string             [10],
            utf8-string                [12],
            integer                    [15],
            long                       [16],
            unsigned                   [17],
            long-unsigned              [18],
            long64                     [20],
            long64-unsigned            [21],
            float32                    [23],
            float64                    [24]
      }
}
```

set_ encryption_key

Sets encryption key to be used with the M-Bus slave device.

Changing the encryption key requires two steps: First, the key is sent to the M-Bus slave, encrypted with the master key, using the *transfer_key* method. Second, the key is set in the M-Bus master using the *set_encryption_key* method.

data ::= octet-string (encryption_key)

After the installation of the M-Bus slave, the M-Bus client holds an empty encryption key. With this, encryption of M-Bus telegrams is disabled. Encryption can be disabled by invoking the *set_encryption_key* method with null- data as a parameter.

transfer_key

Transfers an encryption key to the M-Bus slave.

data ::= octet-string (encrypted_key)

Each M-Bus slave device shall be delivered with a default encryption key.

Before encrypted M-Bus telegrams can be used, an operational encryption key has to be sent to the M-Bus slave, by invoking the *transfer_key* method. The method invocation parameter is the operational encryption key encrypted with the default key of the M-Bus slave device. The M-Bus telegram sent is not encrypted. After the execution, the encryption is enabled and all further telegrams are encrypted.

A new encryption key can be set in the M-Bus client by invoking the *set_encryption_key* method with the new encryption key as a parameter.

With further invocations of the *transfer_key* method, new encryption keys can be sent to the M-Bus slave. The method invocation parameter is the new encryption key encrypted with the default key. The M-Bus telegram is encrypted.

When an M-Bus slave is de-installed, the encryption key is destroyed, but the default key is not affected. Encryption remains disabled until a new encryption is transferred.

© Copyright 1997-2020 DLMS User Association

5.9 Previous versions of interface classes for setting up data exchange over the internet

There are no previous versions to report.

5.10 Previous versions of interface classes for data exchange using S-FSK PLC

5.10.1 S-FSK Phy&MAC setup (class_id = 50, version = 0)

NOTE 1 The use of this version is deprecated.

An instance of the “S-FSK Phy&MAC setup” IC stores the data necessary to set up and manage the physical and the MAC layer of the PLC S-FSK lower layer profile.

S-FSK Phy&MAC setup		**0...n**	**class_id = 50, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. initiator_electrical_phase	(static)	enum	0	3		x + 0x08
3. delta_electrical_phase	(dyn.)	enum	0	6		x + 0x10
4. max_receiving_gain	(static)	unsigned				x + 0x18
5. max_transmitting_gain	(static)	unsigned				x + 0x20
6. search_initiator_gain	(static)	unsigned				x + 0x28
7. frequencies	(static)	frequencies_type				x + 0x30
8. mac_address	(dyn.)	long-unsigned			FFE	x + 0x38
9. mac_group_addresses	(static)	array				x + 0x40
10. repeater	(static)	enum			1	x + 0x48
11. repeater_status	(dyn.)	boolean				x + 0x50
12. min_delta_credit	(dyn.)	unsigned				x + 0x58
13. initiator_mac_address	(dyn.)	long-unsigned				x + 0x60
14. synchronization_locked	(dyn.)	boolean				x + 0x68
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “S-FSK Phy&MAC setup” object instance. See 6.2.25.
initiator_ electrical_phase	Holds the MIB variable initiator-electrical-phase (variable 18) specified in IEC 61334-4-512:2001, 5.8. It is written by the client system to indicate the phase to which it is connected. enum: (0) Not defined (default), (1) Phase 1, (2) Phase 2, (3) Phase 3 NOTE 2 This enumeration is different from that of IEC 61334-4-512:2001.
delta_ electrical_phase	Holds the MIB variable delta-electrical-phase (variable 1) specified in IEC 61334-4-512:2001, 5.2 and IEC 61334-5-1:2001, 3.5.5.3. It indicates the phase difference between the client's connecting phase and the server's connecting phase. The following values are predefined:

© Copyright 1997-2020 DLMS User Association

delta_ electrical_phase (continued)	enum: (0) Not defined: the server is temporarily not able to determine the phase difference, (1) The server system is connected to the same phase as the client system. The phase difference between the server's connecting phase and the client's connecting phase is equal to: (2) 60 degrees, (3) 120 degrees, (4) 180 degrees, (5) -120 degrees, (6) -60 degrees
max_ receiving_gain	Holds the MIB variable max-receiving-gain (variable 2) specified in IEC 61334-4-512:2001, 5.2 and in IEC 61334-5-1:2001, 3.5.5.3. Corresponds to the maximum allowed gain bound to be used by the server system in the receiving mode. The default unit is dB. NOTE 3 In IEC 61334-4-512:2001, no unit is specified. The possible values of the gain may depend on the hardware. Therefore, after writing a value to this attribute, the value should be read back to know the actual value.
max_ transmitting_ gain	Holds the value of the max-transmitting-gain. Corresponds to the maximum attenuation bound to be used by the server system in the transmitting mode. The default unit is dB. The possible values of the gain may depend on the hardware. Therefore, after writing a value to this attribute, the value should be read back to know the actual value.
search_ initiator_gain	This attribute is used in the intelligent search initiator process. If the value of the *max_receiving_gain* is below the value of this attribute, a fast synchronization process is possible.

frequencies	Contains frequencies required for S-FSK modulation.
	frequencies_type ::= structure
	{
	mark_frequency: double-long-unsigned,
	space_frequency: double-long-unsigned
	}
	The default unit is Hz.

mac_address	Holds the MIB variable mac-address (variable 3) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. NOTE 4 MAC addresses are expressed on 12 bits. Contains the value of the address of the physical attachment (MAC address) associated to the local system. In the unconfigured state, the MAC address is “NEW-address”. This attribute is locally written by the CIASE when the system is registered (with a Register service). The value is used in each outgoing or incoming frame. The default value is "NEW-address". This attribute is set to NEW: - by the MAC sub-layer, once the time-out-not-addressed delay is exceeded; - when a client system “resets” the server system. See the “S-FSK Active initiator” IC in Clause 4.10.4.

© Copyright 1997-2020 DLMS User Association

mac_address (continued)	When this attribute is set to NEW: - the system loses its synchronization (function of the MAC-sublayer); - the *mac_group_address* attribute is reset (array of 0 elements); - the system automatically releases all AAs which can be released. NOTE 5 The second item is not present in IEC 61334-4-512:2001. The predefined MAC addresses are shown in Table 33.
mac_group_ addresses	Holds the MIB variable mac-group-address (variable 4) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. Contains a set of MAC group addresses used for broadcast purposes. array mac-address mac-address ::= long-unsigned The ALL-configured-address, ALL-physical-address and NO-BODY addresses are not included in this list. These ones are internal predefined values. This attribute shall be written by the initiator using DLMS services to declare specific MAC group addresses on a server system. This attribute is locally read by the MAC sublayer when checking the destination address field of a MAC frame not recognized as an individual address or as one of the three predefined values (ALL-configured-address, ALL-physical-address and NO-BODY).
repeater	Holds the MIB variable repeater (variable 5) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. It specifies whether the server system effectively repeats all frames or not. enum: (0) never repeater, (1) always repeater, (2) dynamic repeater If the repeater variable is equal to 0, the server system should never repeat the frames. If it is set to 1, the server system is a repeater: it has to repeat all frames received without error and with a current credit greater than zero. If it is set to 2, then the repeater status can be dynamically changed by the server itself. NOTE 6 The value 2 value is not specified in IEC 61334-4-512:2001. This attribute is internally read by the MAC sub-layer each time a frame is received. The default value is 2, and the server system is a repeater (*repeater_status* = TRUE).
repeater_status	Holds the current repeater status of the device. boolean: FALSE = no repeater, TRUE = repeater
min_delta_ credit	Holds the MIB variable min-delta-credit (variable 9) specified in IEC 61334-4-512:2001, 5.3 and in IEC 61334-5-1:2001, 4.3.7.6. NOTE 7 Only the three least significant bits are used. The Delta Credit (DC) is the subtraction of the Initial Credit (IC) and Current Credit (CC) fields of a correct received MAC frame. The delta-credit minimum value of a correct received MAC frame, directed to a server system, is held by this variable.

© Copyright 1997-2020 DLMS User Association

min_delta_ credit (continued)	The default value is set to the maximal initial credit (see IEC 61334-5-1:2001, 4.2.3.1 for further explanations on the credit and the value of MAX_INITIAL_CREDIT). A client system can reinitialise this variable by setting its value to the maximal initial credit.
initiator_mac_ address	Holds the MIB variable initiator-mac-address specified in IEC 61334-5-1:2001, 4.3.7.6. Its value is either the MAC address of the active-initiator or the NO-BODY address, depending on the value of the synchronization_locked attribute (see below). See also IEC 61334-5-1:2001, 3.5.3, 4.1.6.3 and 4.1.7.2. If the value NO-BODY is written then the server mac_address (see the *mac_address* attribute) has to be set to NEW.
synchronization_ locked	Holds the MIB variable synchronization-locked (variable 10) specified in IEC 61334-4-512:2001, 5.3. Controls the synchronization locked / unlocked state. See IEC 61334-5-1:2001 for more details. If the value of this attribute is equal to TRUE, the system is in the synchronization-locked state. In this state, the initiator-mac-address is always equal to the MAC address field of the active-initiator MIB object. See attribute 2 of the S-FSK Active initiator IC, in 4.10.4. If the value of this attribute is equal to FALSE, the system is in the synchronization-unlocked state. In this state, the *initiator_mac_address* attribute is always set to the NO-BODY value: a value change in the MAC address field of the active-initiator MIB object does not affect the content of the *initiator_mac_address* attribute which remains at the NO-BODY value. The default value of this variable shall be specified in the implementation specifications. NOTE 8 In the synchronization-unlocked state, the server synchronizes on any valid frame. In the synchronization locked state, the server only synchronizes on frames issued or directed to the client system the MAC address of which is equal to the value of the *initiator_mac_address* attribute.

5.10.2 S-FSK IEC 61334-4-32 LLC setup (class_id = 55, version = 0)

An instance of the “S-FSK IEC 61334-4-32 LLC setup” IC holds parameters necessary to set up and manage the LLC layer as specified in IEC 61334-4-32:1996.

S-FSK IEC 61334-4-32 LLC setup		**0…n**	**class_id = 55, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. max_frame_length	(static)	unsigned	26	242	134	x + 0x08
3. reply_status_list	(dyn.)	array				x + 0x10
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “S-FSK IEC 61334-4-32 LLC setup” object instance. See 6.2.26

© Copyright 1997-2020 DLMS User Association

max_frame_ length	Holds the length of the LLC frame in bytes. See IEC 61334-4-32:1996, 5.1.4. In the case of the S-FSK profile, as specified in IEC 61334-5-1:2001, 4.2.2, the maximum value is 242, but lower values may be chosen due to performance considerations.

reply_status_list	Holds the MIB variable *reply-status-list* (variable 11) specified in IEC 61334-4-512:2001, 5.4.
	Lists the L-SAPs that have a not empty RDR (Reply Data on Request) buffer, which has not already been read. The length of a waiting L-SDU is specified in number of sub frames (different from zero). The variable is locally generated by the LLC sub layer.
	array reply_status
	reply_status ::= structure
	{
	L-SAP-selector: unsigned,
	length-of-waiting-L-SDU: unsigned
	}
	length-of-waiting-LSDU in the case of the S-FSK profile is in number of sub-frames; valid values are 1 to 7.

5.11 Previous versions of interface classes for setting up the LLC layer for ISO/IEC 8802-2

There are no previous versions to report.

5.12 Previous versions of interface classes for setting up and managing DLMS/COSEM narrowband OFDM PLC profile for PRIME networks

There are no previous versions to report.

5.13 Previous versions of interface classes for setting up and managing DLMS/COSEM narrowband OFDM PLC profile for G3-PLC networks

5.13.1 Mapping of G3-PLC IB attributes to COSEM IC attributes (Original version)

In terms of IEEE 802.15.4, a meter is Reduced Function Device (RFD) while a concentrator / Neighbourhood Network Access Point (NNAP) is a Full Function Device (FFD) / PAN coordinator. In terms of DLMS/COSEM the meter is the server and the concentrator / NNAP is the client (or an agent for a client).

As COSEM models only the server and not the client, the G3 NB OFDM PLC setup classes concern only the RFD (Reduced Function Device) and not the PAN coordinator.

Table 39 shows the mapping of G3 NB OFDM PLC PIB attributes to attributes of COSEM interface classes.

Table 39 – Mapping of G3-PLC IB attributes specified in ITU-T G.9903:2013 Amd. 1 to COSEM IC attributes

Name	Identifier	Interface class	class_id / attribute
MAC counters – Read only PIB attributes that provide statistic information [1]			
mac_Tx_data_packet_count	0x02000101		90 / Att. 2

© Copyright 1997-2020 DLMS User Association

Name	Identifier	Interface class	class_id / attribute
mac_Rx_data_packet_count	0x02000102	PRIME NB OFDM PLC Physical layer parameters (class_id = 90, version = 0)	90 / Att. 3
mac_Tx_cmd_packet_count	0x02000201		90 / Att. 4
mac_Rx_cmd_packet_count	0x02000202		90 / Att. 5
mac_CSMA_fail_count	0x02000103		90 / Att. 6
mac_CSMA_no_ACK_count	0x02000104		90 / Att. 7
mac_bad_CRC_count	0x02000108		90 / Att. 8
mac_broadcast_count	0x02000106		90 / Att. 9
mac_multicast_count	0x02000107		90 / Att. 10
MAC setup PIB attributes – Read only & read-write variables [1]			
mac_short_address	0x01000112	G3 NB OFDM PLC MAC setup (class_id = 91, version = 0)	91 / Att. 2
mac_coord_short_address	0x01000107		91 / Att. 3
mac_PAN_id	0x0100010F		91 / Att. 4
mac_max_orphan_timer	0x02000109		91 / Att. 5
mac_security_enabled	0x01000111		91 / Att. 6
mac_freq_notching	0x0000006D		91 / Att. 7
mac_TMR_TTL	0x02000113		91 / Att. 8
mac_max_frame_retries	0x0100010D		91 / Att. 9
mac_neighbour_table_entry_TTL	0x02000114		91 / Att. 10
mac_neighbour_table	0x0000006B		91 / Att. 11
6LoWPAN adaptation layer IB attributes – Read only & read-write variables [2]			
adp_max_hops	0x10	G3 NB OFDM PLC 6LoWPAN adaptation layer setup (Class_id = 92, version = 0)	92 / Att. 2
adp_weak_LQI_value	0x1B		92 / Att. 3
adp_tone_mask	0x0F		92 / Att. 4
adp_discovery_attempts_wait_time	0x06		92 / Att. 5
adp_routing_configuration	0x12 – 0x19, 0x1A, 0x1C, 0x26		92 / Att. 6
adp_broadcast_log_table_entry_TTL	0x02		92 / Att. 7
adp_routing_table	0x0C		92 / Att. 8
adp_context_information_table	0x07		92 / Att. 9
adp_blacklist_table	0x25		92 / Att. 10
adp_broadcast_log_table	0x0B		92 / Att. 11
adp_group_table	0x0E		92 / Att. 12
adp_max_join_wait_time	0x21		92 / Att. 13
adp_path_discovery_time	0x22		92 / Att. 14
adp_use_new_GMK_time	0x23		92 / Att. 15
adp_exp_prec_GMK_time	0x24		92 / Att. 16

[1)] See ITU-T G.9903 Amd. 1:2013 clauses 9.3.6.2.2 and 9.3.6.2.3

[2)] See ITU-T G.9903 Amd. 1:2013 clause 9.4.1.1

NOTE Whereas in ITU-T G.9903 Amd. 1:2013 the camel notation is used, in COSEM interface class specifications – and in this table – the underscore notation is used.

© Copyright 1997-2020 DLMS User Association

5.13.2 G3 NB OFDM PLC MAC layer counters (class_id = 90, version = 0)

An instance of the “G3 NB OFDM PLC MAC layer counters” IC stores counters related to the MAC layer exchanges. The objective of these counters is to provide statistical information for management purposes.

The attributes of instances of this IC shall be read only. They can be reset using the reset method.

G3 NB OFDM PLC MAC layer counters	**0…n**	**class_id = 90, version = 0**			
Attributes	***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name (static)	octet-string				x
2. mac_Tx_data_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x08
3. mac_Rx_data_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x10
4. mac_Tx_cmd_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x18
5. mac_Rx_cmd_packet_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x20
6. mac_CSMA_fail_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x28
7. mac_CSMA_no_ACK_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x30
8. mac_bad_CRC_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x38
9. mac_broadcast_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x40
10. mac_multicast_count (dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x48
Specific methods	***m/o***				
1. reset (data)	*o*				x + 0x50

Attribute description	
logical_name	Identifies the “G3 NB OFDM PLC MAC layer counters” object instance. See 6.2.28.
mac_Tx_data_packet_count	PIB attribute 0x02000101: Statistic counter of successfully transmitted data packets (MSDUs).
mac_Rx_data_packet_count	PIB attribute 0x02000102: Statistic counter of successfully received data packets (MSDUs).
mac_Tx_cmd_packet_count	PIB attribute 0x02000201: Statistic counter of successfully transmitted command packets.
mac_Rx_cmd_packet_count	PIB attribute 0x02000202: Statistic counter of successfully received command packets.
mac_CSMA_fail_count	PIB attribute 0x02000103: Counts the number of times when CSMA backoffs exceed macMaxCSMABackoffs.
mac_CSMA_no_ACK_count	PIB attribute 0x02000104: Counts the number of times when an ACK is not received while transmitting an unicast data frame (The loss of ACK is attributed to collisions).
mac_bad_CRC_count	PIB attribute 0x02000108: Statistic counter of the number of frames received with bad CRC.
mac_broadcast_count	PIB attribute 0x02000106: Statistic counter of the number of broadcast frames sent.

© Copyright 1997-2020 DLMS User Association

mac_multicast_count	PIB attribute 0x02000107: Statistic counter of the number of multicast frames sent.

NOTE 1: When a counter reaches the maximum value (0xFFFFFFFF), it's automatically rolled-over.

Method description	
reset (data)	This method forces a reset of the object. By invoking this method, the value of all counters is set to 0. data ::= integer (0) NOTE 2 The Short Name reference in this document is x + 0x50, to match the version in BB 12.2, whereas the BB 11 recorded this Short Name as x + 0x60.

5.13.3 G3 NB OFDM PLC MAC setup (class_id = 91, version = 0)

An instance of the "G3 NB OFDM PLC MAC setup" IC holds the necessary parameters to set up and manage the G3 NB OFDM PLC IEEE 802.15.4 MAC sub-layer.

These attributes influence the functional behaviour of an implementation. Implementations may allow changes to the attributes during normal running, i.e. even after the device start-up sequence has been executed.

G3 NB OFDM PLC MAC setup		**0…n**	**class_id = 91, version = 0**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				X
2. mac_short_address	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x08
3. mac_coord_short_address	(dyn.)	long-unsigned	0x0000	0xFFFF	0x0000	x + 0x10
4. mac_PAN_id	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x18
5. mac_max_orphan_timer	(static)	double-long-unsigned	0	4 294 967 295	0	x + 0x20
6. mac_security_enabled	(static)	boolean			TRUE	x + 0x28
7. mac_freq_notching	(static)	boolean			FALSE	x + 0x30
8. mac_TMR_TTL	(static)	double-long-unsigned	0	262 143	120	x + 0x38
9. mac_max_frame_retries	(static)	unsigned	0	10	5	x + 0x40
10. mac_neighbour_table_ entry_TTL	(static)	double-long-unsigned	0	262 143	15 300	x + 0x48
11. mac_neighbour_table	(dyn.)	array				
Specific methods		***m/o***				
1. mac_get_neighbour_ table_entry (data)		o				x + 0x60

Attribute description	
logical_name	Identifies the "G3 NB OFDM PLC MAC setup" object instance. See 6.2.28.
mac_ short_address	PIB attribute 0x01000112: Device short address. The 16-bit address the device is using to communicate on the PAN. Its value shall be equal to 0xFFFF when the device does not have a short address. An associated device necessarily has a short address, so that a device cannot be in the state where it is associated but does not have a short address.

© Copyright 1997-2020 DLMS User Association

mac_coord_ short_address	PIB attribute 0x01000107: Coordinator short address. NOTE 1 The short address assigned to the coordinator through which the device is associated.
mac_PAN_id	PIB attribute 0x0100010F: PAN ID. NOTE 2 The 16-bit identifier of the PAN on which the device is operating. A value equal to 0xFFFF indicates that the device is not associated.
mac_max_ orphan_timer	PIB attribute 0x02000109: The maximum number of seconds without communication with a particular device after which it is declared as an orphan.
mac_security_ enabled	PIB attribute 0x01000111: Security enabled. boolean: TRUE : security enabled, FALSE: security disabled
mac_freq_ notching	PIB attribute 0x0000006D: S-FSK 63 and 74 kHz frequency notching. boolean: TRUE : notching enabled, FALSE: notching disabled Default value is FALSE (disabled).
mac_TMR_TTL	PIB attribute 0x02000113: Maximum valid time of tone map parameters in the neighbour table in seconds.
mac_max_ frame_retries	PIB attribute 0x0100010D: Maximum number of retransmissions.
mac_neighbour_ table_entry_TTL	PIB attribute 0x02000114: Maximum valid time for an entry in the neighbour table in seconds.

mac_ neighbour_ table	PIB attribute 0x0000006B: See ITU-T G.9903 Amd. 1:2013, 9.3.7.2 for CENELEC and FCC bands.
	The neighbour table contains information about all the devices within the POS of the device. One element of the table represents one PLC direct neighbour of the device.
	array mac_neighbour_table_element
	mac_neighbour_table_element ::= structure
	{
	mac_short_address: long-unsigned,
	payload_modulation_scheme: boolean,
	tone_map: bit-string,
	modulation: enum,
	tx_gain: integer,
	tx_res: boolean,
	tx_coeff: array,
	lqi: unsigned,
	TMR_valid_time: double-long-unsigned,
	neighbour_valid_time: double-long-unsigned
	}
	NOTE 3 This table is actualized each time any frame is received from a neighbour device, and each time a Tone Map Response is received.

short_ address	The MAC Short Address of the node which this entry refers to.
payload_ modulation_ scheme	0: Differential, 1: Coherent

© Copyright 1997-2020 DLMS User Association

tone_map	The Tone Map parameter defines which frequency sub-band can be used for communication with the device. A bit set to 1 means that the frequency sub-band can be used, and a bit set to 0 means that frequency sub-band shall not be used.
modulation	The modulation type to use for communicating with the device. enum: (0) Robust Mode, (1) DBPSK, (2) DQPSK, (3) D8PSK, (4) 16-QAM NOTE 4 The 16-QAM modulation is optional and only applicable for FCC band.
tx_gain	Defines the Tx Gain to use to transmit frames to that device.
tx_res	Defines the Tx Gain resolution corresponding to one gain step. 0 : 6 dB 1 : 3 dB
tx_coeff	A parameter that specifies transmitter gain for each group of tones represented by one valid bit of the tone map. The receiver measures the frequency-dependent attenuation of the channel and may request the transmitter to compensate for this attenuation by increasing the transmit power on sections of the spectrum that are experiencing attenuation in order to equalize the received signal. Each group of tones is mapped to a 4-bit value for CENELEC-A or a 2-bit value for FCC where a "0" in the most significant bit indicates a positive gain value, hence an increase in the transmitter gain scaled by TXRES is requested for that section and a "1" indicates a negative gain value, hence a decrease in the transmitter gain scaled by TXRES is requested for that section. Implementing this feature is optional and it is intended for frequency selective channels. If this feature is not implemented, the value zero shall be used. NOTE 5 One group of tones gathers 6 consecutive tones (or carriers) for CENELEC bands, and 3 consecutive tones for FCC band.
lqi	Link quality indicator. NOTE 6 LQI value measured during reception of the PPDU from the neighbour. The LQI measurement is a characterization of the strength and/or quality of a received packet.
TMR_valid_time	Remaining time in seconds until which the tone map response parameters in the neighbour table are considered valid. - When the entry is created, this value shall be set to the default value 0. - When it reaches 0, a tone map request may be issued if data is sent to this device. Upon successful reception of a tone map response, this value is set to macTMRTTL.

© Copyright 1997-2020 DLMS User Association

	neighbour_ valid_time	Remaining time in seconds until which this entry in the neighbour table is considered valid. Every time an entry is created or a frame (data or ACK) is received from this neighbour, it is set to macNeighbourTableEntryTTL. When it reaches zero, this entry is no longer valid in the table and may be removed.

Method description	
mac_get_ neighbour_ table_entry (data)	This method is used to retrieve the mac neighbour table for one MAC short address. It may be used to perform topology monitoring by the client. The method invocation parameter contains a mac_short_address. data ::= long-unsigned The response parameter includes the neighbour table for this mac_short_address. data ::= array mac_neighbour_table_element where mac_neighbour_table-element is as defined in the *mac_neighbour_table* attribute of the present IC.

5.13.4 G3 NB OFDM PLC 6LoWPAN adaptation layer setup (class_id = 92, version = 0)

An instance of the “G3 NB OFDM PLC 6LoWPAN adaptation layer setup” IC holds the necessary parameters to set up and manage the G3 NB OFDM PLC 6LoWPAN Adaptation layer.

These attributes influence the functional behaviour of an implementation. Implementations may allow changes to their values during normal running, i.e. even after the device start-up sequence has been executed.

© Copyright 1997-2020 DLMS User Association

G3 NB OFDM PLC 6LoWPAN adaptation layer setup		0...n	class_id = 92, version = 0			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. adp_max_hops	(static)	unsigned	1	14	8	x + 0x10
3. adp_weak_LQI_value	(static)	unsigned	0	255	3 for CENELEC A 5 for FCC	x + 0x18
4. adp_tone_mask	(static)	bit-string			All bits set to 1	x + 0x20
5. adp_discovery_attempts_wait_time	(static)	long-unsigned	1	3 600	60	x + 0x28
6. adp_routing_configuration	(static)	array				x + 0x30
7. adp_broadcast_log_table_entry_TTL	(static)	long-unsigned	0	3 600	90	x + 0x38
8. adp_routing_table	(dyn.)	array				x + 0x40
9. adp_context_information_table	(dyn)	array				x + 0x48
10. adp_blacklist_table	(dyn)	array				x + 0x50
11. adp_broadcast_log_table	(dyn)	array				x + 0x58
12. adp_group_table	(dyn)	array				x + 0x60
13. adp_max_join_wait_time	(static)	long-unsigned	0	1 023	20	x + 0x68
14. adp_path_discovery_time	(static)	long-unsigned	0	65535	5 000	x + 0x70
15. adp_use_new_GMK_time	(static)	long-unsigned	0	65535	3 600	x + 0x78
16. adp_exp_prec_GMK_time	(static)	long-unsigned	0	3 600	3 600	X + 0x80
Specific methods		***m/o***				

Attribute description	
logical_name	Identifies the “G3 NB OFDM 6LoWPAN adaptation layer setup” object instance. See 6.2.28.
adp_max_hops	PIB attribute 0x10: Defines the maximum number of hops to be used by the routing algorithm.
adp_weak_ LQI_value	PIB attribute 0x1B: The weak link value defines the threshold below which a direct neighbour is not taken into account during the commissioning procedure (compared to the LQI measured).
adp_tone_mask	PIB attribute 0x0F: Defines the Tone Mask to use during symbol formation. The length of the bit-string is 70 bits.
adp_discovery_ attempts_ wait_time	PIB attribute 0x06: Allows programming the maximum wait time between invocations of two consecutive network discovery primitives (in seconds).

© Copyright 1997-2020 DLMS User Association

adp_routing_ configuration

The routing configuration element specifies all parameters linked to the routing mechanism described in ITU-T G.9903 Amd. 1:2013. The elements are specified in 9.4.1.2 of that Recommendation.

NOTE 1 The Link cost calculation is provided in ITU-T G.9903 Amd. 1:2013, Annex B.

array routing_configuration

routing_configuration ::= structure
{

adp_net_traversal_time:	long-unsigned,
adp_routing_table_entry_TTL:	double-long-unsigned,
adp_Kr:	unsigned,
adp_Km:	unsigned,
adp_Kc:	unsigned,
adp_Kq:	unsigned,
adp_Kh:	unsigned,
adp_Krt:	unsigned,
adp_RREQ_retries:	unsigned,
adp_RREQ_RERR_wait:	long-unsigned,
adp_blacklist_table_entry_TTL:	long-unsigned

}

adp_net_ traversal_ time	PIB attribute 0x12: The Max duration between RREQ and the correspondent RREP (in seconds). Range: 0-65 535 Default value : 20
adp_routing_ table_entry_ TTL	PIB attribute 0x13: The time to live of a route request table entry (in seconds). Range: 0-262 143 Default value : 90
adp_Kr	PIB attribute 0x14: A weight factor for ROBO to calculate link cost. Range: 0-31 Default value: 0
adp_Km	PIB attribute 0x15: A weight factor for modulation to calculate link cost. Range: 0-31 Default value: 0
adp_Kc	PIB attribute 0x16: A weight factor for number of active tones to calculate link cost. Range: 0-31 Default value : 0
adp_Kq	PIB attribute 0x17: A weight factor for LQI to calculate route cost. Range: 0-31 Default value: 10 for CENELEC A band / 40 for FCC band.
adp_Kh	PIB attribute 0x18: A weight factor for hop to calculate link cost. Range: 0-31 Default value: 4 for CENELEC A band / 2 for FCC band.

© Copyright 1997-2020 DLMS User Association

adp_routing_ configuration (continued)	adp_Krt	PIB attribute 0x1C: A weight factor for the number of active routes in the routing table to calculate link cost. Range: 0-31 Default value: 0
	adp_RREQ_ retries	PIB attribute 0x19: The number of RREQ re-transmission in case of RREP reception time out. Range: 0-255 Default value: 0
	adp_RREQ_ RERR_wait	PIB attribute 0x1A: The number of seconds to wait between two consecutive RREQ – RERR generations. Range: 0-65 535 Default value: 30
	adp_ blacklist_ table_entry_ TTL	PIB attribute 0x26: Time to live of a blacklisted neighbour set entry in minutes. Range: 0-65 535 Default value: 10
adp_broadcast_ log_table_ entry_TTL	PIB attribute 0x02: Defines the time while an entry in the adpBroadcastLogTable remains active in the table (in seconds).	

adp_ routing_table	PIB attribute 0x0C: The routing table contains information about the different routes in which the device is implicated.
	array routing_table
	
	routing_table ::= structure
	{
	destination_address: long-unsigned,
	next_hop_address: long-unsigned,
	route_cost: long-unsigned,
	hop_count: unsigned,
	weak_link_count: unsigned,
	status: enum,
	valid_time: unsigned
	}
	NOTE 2 This table is actualized each time a route is built or updated (triggered by data traffic) and each time the TTL timer expires.	

	destination_ address	Address of the destination.
	next_hop_ address	Address of the next hop on the route towards the destination.
	route_cost	Cumulative link cost along the route towards the destination.
	hop_count	Number of hops of the selected route to the destination. Range: 0-15 NOTE 3 Practically the maximum allowed value is limited by adp_max_hops.
	weak_link_ count	Number of weak links to destination. Range: 0-15
	NOTE 4 Practically the maximum allowed value is limited by adp_max_hops.	

© Copyright 1997-2020 DLMS User Association

adp_ routing_table (continued)	status	Status of the routing table entry. enum: (0) Invalid route, (1) Valid route, (2) Route under construction
	valid_time	Remaining time in seconds until which this entry in the routing table is considered valid.

adp_context_ information_ table	PIB attribute 0x07: Contains the context information associated to each CID extension field.
	array context_information_table
	context_information_table ::= structure
	{
	CID: bit-string,
	context_length: unsigned,
	context: octet-string,
	C: boolean,
	valid_lifetime: long-unsigned
	}	

	CID	Corresponds to the 4-bit context information used for source and destination addresses (SCI, DCI). Range: 0x00-0x0F
	context_ length	Indicates the length of the carried context (up to 128-bit contexts may be carried). Range: 0-128
	context	Corresponds to the carried context used for compression/decompression purposes. Range: 0x0000:0000:0000:0000:0000:0000:0000:0000 – 0xFFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF
	C	Indicates if the context is valid for use in compression 0 : Only decompression is allowed, 1 : Compression and decompression are allowed
	A context may be used for decompression purposes only. Moreover, recommendations made in RFC 6775 should be followed to take into account the propagation of the context to all nodes of the PAN.	
	valid_ lifetime	Remaining time in minutes during which the context information table is considered valid. It is updated upon reception of the advertised context. Range: 0-65 535

adp_blacklist_ table	PIB attribute 0x25: Contains the list of the blacklisted neighbours.
	array blacklisted_neighbour_set
	blacklisted_neighbour_set ::= structure
	{
	blacklisted_neighbour_address: long-unsigned,
	valid_time: long-unsigned
	}	

	blacklisted_ neighbour_ address	The 16-bit address of the blacklisted neighbour.
	valid_time	Remaining time in minutes until which this entry in the blacklisted neighbour table is considered valid.

© Copyright 1997-2020 DLMS User Association

adp_broadcast_log_table	PIB attribute 0x0B: Contains the broadcast log table.
	NOTE 5 This table provides a list of the broadcast packets recently received by this device.
	array broadcast_log_table
	broadcast_log_table ::= structure
	{
	source_address: long-unsigned,
	sequence_number: unsigned,
	time_to_live: long-unsigned
	}

source_address	The 16-bit source address of a broadcast packet. This is the address of the broadcast initiator.
sequence_number	The sequence number contained in the BC0 header.
time_to_live	The remaining time to live of this entry in the broadcast log table, in seconds.

adp_group_table	PIB attribute 0x0E: Contains the group addresses to which the device belongs.
	array group_table
	group_table ::= structure
	{
	group_address: long-unsigned
	}
	group_address Group address to which this node has been subscribed.

adp_max_join_wait_time	PIB attribute 0x21: Network join timeout in seconds for LBD.
adp_path_discovery_time	PIB attribute 0x22: Timeout for path discovery in msec.
adp_use_new_GMK_time	PIB attribute 0x23: The wait time in seconds for a device to use new GMK after rekeying.
adp_exp_prec_GMK_time	PIB attribute 0x24: The time in seconds to keep PrecGMK after switching to a new GMK.

5.13.5 Mapping of G3-PLC PIB attributes to COSEM IC attributes (previous version)

In terms of IEEE 802.15.4, a meter is a Reduced Function Device (RFD) while a concentrator / Neighbourhood Network Access Point (NNAP) is a Full Function Device (FFD) / PAN coordinator. In terms of DLMS/COSEM the meter is the server and the concentrator / NNAP is the client (or an agent for a client).

As COSEM models only the server and not the client, the G3-PLC setup classes concern only the RFD (Reduced Function Device) and not the PAN coordinator.

Table 40 shows the mapping of G3-PLC PIB attributes to attributes of COSEM interface classes.

Table 40 – Mapping of G3-PLC IB attributes to COSEM IC attributes

Name	Identifier	Interface class		class_id / attribute
MAC counters – Read only PIB attributes that provide statistic information [1]				
mac_Tx_data_packet_count	0x0101			90 / Att. 2

© Copyright 1997-2020 DLMS User Association

Name	Identifier	Interface class		class_id / attribute
mac_Rx_data_packet_count	0x0102	G3-PLC MAC layer counters (class_id = 90, version = 1)		90 / Att. 3
mac_Tx_cmd_packet_count	0x0103			90 / Att. 4
mac_Rx_cmd_packet_count	0x0104			90 / Att. 5
mac_CSMA_fail_count	0x0105			90 / Att. 6
mac_CSMA_no_ACK_count	0x0106			90 / Att. 7
mac_bad_CRC_count	0x0109			90 / Att. 8
mac_Tx_data_broadcast_count	0x0108			90 / Att. 9
mac_Rx_data_broadcast_count	0x0107			90 / Att. 10
MAC setup PIB attributes – Read only & read-write & write only variables [1] [2]				
mac_short_address	0x0053	G3-PLC MAC setup (class_id = 91, version = 1)		91 / Att. 2
mac_RC_coord	0x010F			91 / Att. 3
mac_PAN_id	0x0050			91 / Att. 4
mac_key_table	0x0071			91 / Att. 5
mac_frame_counter	0x0077			91 / Att. 6
mac_tone_mask	0x0110			91 / Att. 7
mac_TMR_TTL	0x010D			91 / Att. 8
mac_max_frame_retries	0x0059			91 / Att. 9
mac_neighbour_table_entry_TTL	0x010E			91 / Att. 10
mac_neighbour_table	0x010A			91 / Att. 11
mac_high_priority_window_size	0x0100			91 / Att. 12
mac_CSMA_fairness_limit	0x010C			91 / Att. 13
mac_beacon_randomization_window_length	0x0111			91 / Att. 14
mac_A	0x0112			91 / Att. 15
mac_K	0x0113			91 / Att. 16
mac_min_CW_attempts	0x0114			91 / Att. 17
mac_cenelec_legacy_mode	0x0115			91 / Att. 18
mac_FCC_legacy_mode	0x0116			91 / Att. 19
mac_max_BE	0x0047			91 / Att. 20
mac_max_CSMA_backoffs	0x004E			91 / Att. 21
mac_min_BE	0x004F			91 / Att. 22
6LoWPAN adaptation layer IB attributes – Read only & read-write variables [3] [4]				
adp_max_hops	0x0F	G3-PLC MAC setup (class_id = 91, version = 1)		92 / Att. 2
adp_weak_LQI_value	0x1A			92 / Att. 3
adp_security_level	0x00			92 / Att. 4
adp_prefix_table	0x01			92 / Att. 5
adp_routing_configuration	0x09, 0x0A, 0x0D, 0x11-0x19, 0x1B, 0x1F			92 / Att. 6
adp_broadcast_log_table_entry_TTL	0x02			92 / Att. 7
adp_routing_table	0x0C			92 / Att. 8
adp_context_information_table	0x07			92 / Att. 9

© Copyright 1997-2020 DLMS User Association

Name	Identifier	Interface class		class_id / attribute
adp_blacklist_table	0x1E			92 / Att. 10
adp_broadcast_log_table	0x0B			92 / Att. 11
adp_group_table	0x0E			92 / Att. 12
adp_max_join_wait_time	0x20			92 / Att. 13
adp_path_discovery_time	0x21			92 / Att. 14
adp_active_key_index	0x22			92 / Att. 15
adp_metric_type	0x03			92 / Att. 16
adp_coord_short_address	0x08			92 / Att. 17
adp_disable_default_routing	0xF0			92 / Att. 18
adp_device_type	0x10			92 / Att. 19
[1] See ITU-T G.9903:2014, 9.3.6.2.2 and 9.3.6.2.3. [2] The following attributes of the G3-PLC MAC sublayer IB attributes have been excluded as there is no need to expose them: *macBSN*, *macDSN*, *macAckWaitDuration*, *macFreqNotching*, *macTimeStampSupported*, *macPromiscuousMode*, *macSecurityEnabled.* [3] See ITU-T G.9903:2014, 9.4.1.1. [4] The following attributes of the G3-PLC Adaptation sublayer IB attributes have been excluded as there is no need to expose them ; *adpSoftVersion, adpSnifferMode*.				
NOTE Whereas in ITU-T G.9903 Amd. 1:2013the camel-case notation is used, in COSEM interface class specifications – and in this table – the underscore notation is used.				

5.13.6 G3-PLC MAC setup (class_id = 91, version = 1)

An instance of the “G3-PLC MAC setup” IC holds the necessary parameters to set up and manage the G3-PLC IEEE 802.15.4 MAC sub-layer.

These attributes influence the functional behaviour of an implementation. Implementations may allow changes to the attributes during normal running, i.e. even after the device start-up sequence has been executed.

© Copyright 1997-2020 DLMS User Association

G3-PLC MAC setup		0...n	class_id = 91, version = 1			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				x
2. mac_short_address	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x08
3. mac_RC_coord	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x10
4. mac_PAN_id	(dyn.)	long-unsigned	0x0000	0xFFFF	0xFFFF	x + 0x18
5. mac_key_table	(dyn.)	array				x + 0x20
6. mac_frame_counter	(dyn.)	double-long-unsigned	0	4 294 967 295	0	x + 0x28
7. mac_tone_mask	(static)	bit-string			0x00000000FFFFFFF	x + 0x30
8. mac_TMR_TTL	(static)	unsigned	0	255	2	x + 0x38
9. mac_max_frame_retries	(static)	unsigned	0	10	5	x + 0x40
10. mac_neighbour_table_entry_TTL	(static)	unsigned	0	255	255	x + 0x48
11. mac_neighbour_table	(dyn.)	array				x + 0x50
12. mac_high_priority_window_size	(static)	unsigned	1	7	7	x + 0x58
13. mac_CSMA_fairness_limit	(static)	unsigned	See below	255	25	x + 0x60
14. mac_beacon_randomization_window_length	(static)	unsigned	1	254	12	x + 0x68
15. mac_A	(static)	unsigned	3	20	8	x + 0x70
16. mac_K	(static)	unsigned	1	See below	5	x + 0x78
17. mac_min_CW_attempts	(static)	unsigned	0	255	10	x + 0x80
18. mac_cenelec_legacy_mode	(static)	unsigned	0	255	1	x + 0x88
19. mac_FCC_legacy_mode	(static)	unsigned	0	255	1	x + 0x90
20. mac_max_BE	(static)	unsigned	0	20	8	x + 0x98
21. mac_max_CSMA_backoffs	(static)	unsigned	0	255	50	x + 0xA0
22. mac_min_BE	(static)	unsigned	0	20	3	x + 0xA8
Specific methods		***m/o***				
1. mac_get_neighbour_table_entry (data)		o				x + 0xB0

Attribute description	
logical_name	Identifies the “G3-PLC MAC setup” object instance. See 6.2.28.
mac_short_address	PIB attribute 0x0053: The 16-bit address the device is using to communicate through the PAN. Its value shall be equal to 0xFFFF when the device does not have a short address. An associated device necessarily has a short address, so that a device cannot be in the state where it is associated but does not have a short address.
mac_RC_coord	PIB attribute 0x010F: Route cost to coordinator, to be used in the beacon payload as RC_COORD.
mac_PAN_id	PIB attribute 0x0050: The 16-bit identifier of the PAN through which the device is operating. A value equal to 0xFFFF indicates that the device is not associated.

© Copyright 1997-2020 DLMS User Association

mac_key_table	PIB attribute 0x0071: This attribute holds GMK keys required for MAC layer ciphering. The attribute can hold up to two 16-bytes keys. The Key Identifier value must be different for each key.
	For security reason, the key entries cannot be read, only written.
	array mac_GMK
	mac_GMK ::= structure
	{
	key_id: unsigned,
	key: octet-string
	}

key_id	The Key Identifier used to refer to this key, can take the value 0 or 1.
key	The AES-128 key used for ciphering the frames exchanged at MAC layer.
mac_frame_counter	PIB attribute 0x0077: The outgoing frame counter for this device, used when ciphering frames at MAC layer.
mac_tone_mask	PIB attribute 0x0110: Defines the tone mask to use during symbol formation.
mac_TMR_TTL	PIB attribute 0x010D: Maximum time to live of tone map parameters entry in the neighbour table in minutes.
mac_max_frame_retries	PIB attribute 0x0059: Maximum number of retransmissions.
mac_neighbour_table_ entry_TTL	PIB attribute 0x010E: Maximum time to live for an entry in the neighbour table in minutes.

mac_neighbour_table	PIB attribute 0x010A: See ITU-T G.9903:2014, 9.3.7.2 for CENELEC and FCC bands.
	The neighbour table contains information about all the devices within the POS of the device. One element of the table represents one PLC direct neighbour of the device.
	array neighbour_table
	neighbour_table ::= structure
	{
	short_address: long-unsigned,
	payload_modulation_scheme: boolean,
	tone_map: bit-string,
	modulation: enum,
	tx_gain: integer,
	tx_res: enum,
	tx_coeff: bit-string,
	lqi: unsigned,
	phase_differential integer,
	TMR_valid_time: unsigned,
	neighbour_valid_time: unsigned
	}
	NOTE 1 This table is actualized each time any frame is received from a neighbour device, and each time a Tone Map Response is received.

© Copyright 1997-2020 DLMS User Association

mac_neighbour_table continued	short_ address	The MAC Short Address of the node which this entry refers to.
	payload_ modulation_ scheme	Payload Modulation scheme to be used when transmitting to this neighbour. FALSE: Differential, TRUE: Coherent
	tone_map	The Tone Map parameter defines which frequency sub-band can be used for communication with the device. A bit set to 1 means that the frequency sub-band can be used, and a bit set to 0 means that frequency sub-band shall not be used.
	modulation	The modulation type to use for communicating with the device. enum : (0) Robust Mode, (1) DBPSK, (2) DQPSK, (3) D8PSK, (4) 16-QAM NOTE 2 The 16-QAM modulation is optional and only applicable for FCC band.
	tx_gain	Defines the Tx Gain to use to transmit frames to that device.
	tx_res	Defines the Tx Gain resolution corresponding to one gain step. 0 : 6 dB, 1 : 3 dB

© Copyright 1997-2020 DLMS User Association

mac_neighbour_table (continued)	tx_coeff	A parameter that specifies transmitter gain for each group of tones represented by one valid bit of the tone map. The receiver measures the frequency-dependent attenuation of the channel and may request the transmitter to compensate for this attenuation by increasing the transmit power on sections of the spectrum that are experiencing attenuation in order to equalize the received signal. Each group of tones is mapped to a 4-bit value for CENELEC-A or a 2-bit value for FCC where a "0" in the most significant bit indicates a positive gain value, hence an increase in the transmitter gain scaled by TXRES is requested for that section and a "1" indicates a negative gain value, hence a decrease in the transmitter gain scaled by TXRES is requested for that section. Implementing this feature is optional and it is intended for frequency selective channels. If this feature is not implemented, the value zero shall be used. *Example: in CENELEC bands, with gain values equal to [1, -5, 4, -2, 0, 1], tx_coeff encoding is equal to: '0001 1101 0100 1010 0000 0001'* NOTE 3 One group of tones gathers 6 consecutive tones (or carriers) for CENELEC bands, and 3 consecutive tones for FCC band.
	lqi	Link Quality Indicator of the link to the neighbour (reverse LQI). NOTE 4 LQI value measured during reception of the PPDU from the neighbour. The LQI measurement is a characterization of the strength and/or quality of a received packet.
	phase_ differential	Phase difference in multiples of 60 degrees between the mains phase of the local node and the neighbour node. PhaseDifferential can assume six integer values between 0 and 5.
	TMR_valid_ time	Remaining time in minutes until which the tone map response parameters in the neighbour table are considered valid. - When the entry is created, this value shall be set to the default value 0. - When it reaches 0, a tone map request may be issued if data is sent to this device. Upon successful reception of a tone map response, this value is set to mac_TMR_TTL.
	neighbour_ valid_time	Remaining time in minutes until which this entry in the neighbour table is considered valid. Every time an entry is created or a frame (data or ACK) is received from this neighbour, it is set to mac_neighbour_table_entry_TTL. When it reaches zero, this entry is no longer valid in the table and may be removed.

© Copyright 1997-2020 DLMS User Association

mac_high_priority_ window_size	PIB attribute 0x0100: The high priority contention window size in number of slots.
mac_CSMA_fairness_limit	PIB attribute 0x010C: Channel access fairness limit. Specifies how many failed back-off attempts, back-off exponent is set to minBE. This attribute can take a value between 2 × (macMaxBE – macMinBE) and 255.
mac_beacon_ randomization_window_ length	PIB attribute 0x0111: Duration time in seconds for the beacon randomization.
mac_A	PIB attribute 0x0112: This parameter controls the adaptive CW linear decrease.
mac_K	PIB attribute 0x0113: Rate adaptation factor for channel access fairness limit. This attribute can take a value between 1 and macCSMAFairnessLimit.
mac_min_CW_attempts	PIB attribute 0x0114: Number of consecutive attempts while using minimum CW.
mac_cenelec_legacy_ mode	PIB attribute 0x0115: This read only attribute indicates the capability of the node. 0: The following configuration is used (legacy mode): - Elementary interleaving; - Interleaver parameters n_i and n_j are not swapped when I(i,j) = 0. 1: The following configuration is used (non legacy mode): - Full Block interleaving; - Interleaver parameters n_i and n_j are swapped when I(i,j) = 0.
mac_FCC_legacy_mode	PIB attribute 0x0116: This read only attribute indicates the capability of the node. 0: The following configuration is used (legacy mode): - Differential FCH modulation; - Elementary interleaving; - Interleaver parameters n_i and n_j are not swapped when I(i,j) = 0; - Single RS block. 1: The following configuration is used (non legacy mode): - Coherent FCH modulation; - Full Block interleaving; - Interleaver parameters n_i and n_j are swapped when I(i,j) = 0; - Two RS blocks.
mac_max_BE	PIB attribute 0x0047: Maximum value of backoff exponent. It should always be greater than macMinBE.
mac_max_CSMA_backoffs	PIB attribute 0x004E: Maximum number of backoff attempts.
mac_min_BE	PIB attribute 0x004F: Minimum value of backoff exponent.

© Copyright 1997-2020 DLMS User Association

Method description	
mac_get_neighbour_ table_entry (data)	This method is used to retrieve the mac neighbour table for one MAC short address. It may be used to perform topology monitoring by the client. The method invocation parameter contains a mac_short_address. data ::= long-unsigned The response parameter includes the neighbour table for this mac_short_address. data ::= array neighbour_table where mac_neighbour_table is as defined in the *mac_neighbour_table* attribute of the present IC.

5.13.7 G3-PLC 6LoWPAN adaptation layer setup (class_id = 92, version = 1)

An instance of the “G3-PLC 6LoWPAN adaptation layer setup” IC holds the necessary parameters to set up and manage the G3-PLC 6LoWPAN Adaptation layer.

These attributes influence the functional behaviour of an implementation. Implementations may allow changes to their values during normal running, i.e. even after the device start-up sequence has been executed.

G3-PLC 6LoWPAN adaptation layer setup		**0...n**	**class_id = 92, version = 1**			
Attributes		***Data type***	***Min.***	***Max.***	***Def.***	**Short name**
1. logical_name	(static)	octet-string				X
2. adp_max_hops	(static)	unsigned	1	14	8	x + 0x08
3. adp_weak_LQI_value	(static)	unsigned	0	255	52	x + 0x10
4. adp_security_level	(static)	unsigned	0	7	5	x + 0x18
5. adp_prefix_table	(dyn.)	array				x + 0x20
6. adp_routing_configuration	(static)	array				x + 0x28
7. adp_broadcast_log_table_ entry_TTL	(static)	long-unsigned	0	65535	2	x + 0x30
8. adp_routing_table	(dyn.)	array				x + 0x38
9. adp_context_information_ table	(dyn)	array				x + 0x40
10. adp_blacklist_table	(dyn)	array				x + 0x48
11. adp_broadcast_log_table	(dyn)	array				x + 0x50
12. adp_group_table	(dyn)	array				x + 0x58
13. adp_max_join_wait_time	(static)	long-unsigned	0	1 023	20	x + 0x60
14. adp_path_discovery_time	(static)	unsigned	0	255	40	x + 0x68
15. adp_active_key_index	(static)	unsigned	0	1	0	x + 0x70
16. adp_metric_type	(static)	unsigned	0x00	0x0F	0x0F	x + 0x78
17. adp_coord_short_address	(static)	long-unsigned	0x0000	0x7FFF	0x0000	x + 0x80
18. adp_disable_default_routing	(static)	boolean			FALSE	x + 0x88
19. adp_device_type	(static)	enum	0	2	2	x + 0x90
Specific methods		***m/o***				

© Copyright 1997-2020 DLMS User Association

Attribute description	
logical_name	Identifies the “G3-PLC OFDM 6LoWPAN adaptation layer setup” object instance. See 6.2.28.
adp_max_hops	PIB attribute 0x0F: Defines the maximum number of hops to be used by the routing algorithm.
adp_weak_LQI_value	PIB attribute 0x1A: The weak link value defines the LQI value below which a link to a neighbour is considered as a weak link. A value of 52 represents an SNR of 3 dB.
adp_security_level	PIB attribute 0x00: The minimum security level to be used for incoming and outgoing adaptation frames. Only values 0 (no ciphering) and 5 (ciphering with 32 bits integrity code) are supported.
adp_prefix_table	PIB attribute 0x01: Contains the list of prefixes defined on this PAN.

adp_routing_ configuration	The routing configuration element specifies all parameters linked to the routing mechanism described in ITU-T G.9903:2014. The elements are specified in 9.4.1.2 of that Recommendation.
	NOTE 1 The Link cost calculation is provided in ITU-T G.9903:2014 Annex B.
	
	array routing_configuration
	routing_configuration ::= structure
	{
	adp_net_traversal_time: unsigned,
	adp_routing_table_entry_TTL: long-unsigned,
	adp_Kr: unsigned,
	adp_Km: unsigned,
	adp_Kc: unsigned,
	adp_Kq: unsigned,
	adp_Kh: unsigned,
	adp_Krt: unsigned,
	adp_RREQ_retries: unsigned,
	adp_RREQ_RERR_wait: unsigned,
	adp_blacklist_table_entry_TTL: long-unsigned,
	adp_unicast_RREQ_gen_enable: boolean,
	adp_RLC_Enabled: boolean,
	adp_add_rev_link_cost: unsigned
	}

adp_net_ traversal_ time	PIB attribute 0x11: Maximum time that a packet is expected to take to reach any node from any node in seconds. Range : 0-255 Default value : 20
adp_routing_ table_entry_ TTL	PIB attribute 0x12: Maximum time-to-live of a routing table entry (in minutes). Range : 0-65 535 Default value : 60

© Copyright 1997-2020 DLMS User Association

adp_routing_ configuration (continued)	adp_Kr	PIB attribute 0x13: A weight factor for the Robust Mode to calculate link cost. Range : 0-31 Default value : 0
	adp_Km	PIB attribute 0x14: A weight factor for modulation to calculate link cost. Range : 0-31 Default value : 0
	adp_Kc	PIB attribute 0x15: A weight factor for number of active tones to calculate link cost. Range : 0-31 Default value : 0
	adp_Kq	PIB attribute 0x16: A weight factor for LQI to calculate route cost. Range : 0-50 Default value : 10 for CENELEC A band / 40 for FCC band.
	adp_Kh	PIB attribute 0x17: A weight factor for hop to calculate link cost. Range : 0-31 Default value : 4 for CENELEC A band / 2 for FCC band.
	adp_Krt	PIB attribute 0x1B: A weight factor for the number of active routes in the routing table to calculate link cost. Range : 0-31 Default value : 0
	adp_RREQ_ retries	PIB attribute 0x18: The number of RREQ re-transmission in case of RREP reception time out. Range : 0-255 Default value : 0
	adp_RREQ_ RERR_wait	PIB attribute 0x19: The number of seconds to wait between two consecutive RREQ – RERR generations. Range : 0-255 Default value : 30
	adp_blacklist_ table_entry_ TTL	PIB attribute 0x1F: Maximum time-to-live of a blacklisted neighbour entry (in minutes). Range : 0-65 535 Default value : 10

© Copyright 1997-2020 DLMS User Association

adp_routing_ configuration (continued)	adp_unicast_ RREQ_gen_ enable	PIB attribute 0x0D: If TRUE, the RREQ shall be generated with its "unicast RREQ" flag set to '1'. If FALSE, the RREQ shall be generated with its "unicast RREQ" flag set to '0'. Default value : TRUE
	adp_RLC_ enabled	PIB attribute 0x09: Enable the sending of RLCREQ frame by the device. Default value : FALSE
	adp_add_rev_ link_cost	PIB attribute 0x0A: It represents an additional cost to take into account a possible asymmetry in the link. Range : 0-255 Default value : 0
adp_broadcast_ log_table_entry_TTL	PIB attribute 0x02: Maximum time to live of an adpBroadcastLogTable entry (in minutes).	

adp_routing_table	PIB attribute 0x0C: Contains the routing table.
	array routing_table
	routing_table ::= structure
	{
	destination_address: long-unsigned,
	next_hop_address: long-unsigned,
	route_cost: long-unsigned,
	hop_count: unsigned,
	weak_link_count: unsigned,
	valid_time: long-unsigned
	}
	NOTE 2 This table is actualized each time a route is built or updated (triggered by data traffic) and each time the TTL timer expires.	

	destination_ address	Address of the destination.
	next_hop_ address	Address of the next hop on the route towards the destination.
	route_cost	Cumulative link cost along the route towards the destination.
	hop_count	Number of hops of the selected route to the destination. Range: 0-14 NOTE 3 Practically the maximum allowed value is limited by adp_max_hops.
	weak_link_ count	Number of weak links to destination. Range : 0-14 NOTE 4 Practically the maximum allowed value is limited by adp_max_hops.
	valid_time	Remaining time in minutes until when this entry in the routing table is considered valid.

© Copyright 1997-2020 DLMS User Association

adp_context_ information_table	PIB attribute 0x07: Contains the context information associated to each CID extension field.
	
	array context_information_table
	context_information_table ::= structure
	{
	CID: bit-string,
	context_length: unsigned,
	context: octet-string,
	C: boolean,
	valid_lifetime: long-unsigned
	}	

	CID	Corresponds to the 4-bit context information used for source and destination addresses (SCI, DCI). Range: 0x00-0x0F
	context_ length	Indicates the length of the carried context (up to 128-bit contexts may be carried). Range: 0-128
	context	Corresponds to the carried context used for compression/decompression purposes. Range: 0x0000:0000:0000:0000:0000:0000:0000:0000 – 0xFFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF
	C	Indicates if the context is valid for use in compression. FALSE : Only decompression is allowed, TRUE : Compression and decompression are allowed A context may be used for decompression purposes only. Moreover, recommendations made in RFC 6775 should be followed to take into account the propagation of the context to all nodes of the PAN.
	valid_lifetime	Remaining time in minutes during which the context information table is considered valid. It is updated upon reception of the advertised context. Range: 0-65 535

adp_blacklist_table	PIB attribute 0x1E: Contains the list of the blacklisted neighbours.
	array blacklisted_neighbour_set
	blacklisted_neighbour_set ::= structure
	{
	blacklisted_neighbour_address: long-unsigned,
	valid_time: long-unsigned
	}	

© Copyright 1997-2020 DLMS User Association

adp_blacklist_table (continued)	blacklisted_ neighbour_ address	The 16-bit address of the blacklisted neighbour.
	valid_time	Remaining time in minutes until which this entry in the blacklisted neighbour table is considered valid.

adp_broadcast_log_ table	PIB attribute 0x0B: Contains the broadcast log table.
	NOTE 5 This table provides a list of the broadcast packets recently received by this device.
	array broadcast_log_table
	broadcast_log_table ::= structure
	{
	source_address: long-unsigned,
	sequence_number: unsigned,
	valid_time: long-unsigned
	}	

	source_ address	The 16-bit source address of a broadcast packet. This is the address of the broadcast initiator.
	sequence_ number	The sequence number contained in the BC0 header.
	valid_time	Remaining time in minutes until when this entry in the broadcast log table is considered valid.
adp_group_table	PIB attribute 0x0E: Contains the group addresses to which the device belongs. array group_address group_address ::= long-unsigned	
	group_address	Group address to which this node has been subscribed.
adp_max_join_wait_time	PIB attribute 0x20: Network join timeout in seconds for LBD.	
adp_path_discovery_time	PIB attribute 0x21: Timeout for path discovery in seconds.	
adp_active_key_index	PIB attribute 0x22: Index of the active GMK to be used for data transmission	
adp_metric_type	PIB attribute 0x03: Metric Type to be used for routing purposes	
adp_coord_short_ address	PIB attribute 0x08: Defines the short address of the coordinator.	
adp_disable_default_ routing	PIB attribute 0xF0: If TRUE, the default routing (LOADng) is disabled. If FALSE, the default routing (LOADng) is enabled.	
adp_device_type	PIB attribute 0x10: Defines the type of the device connected to the modem: enum: (0) PAN device, (1) PAN coordinator, (2) Not Defined	

© Copyright 1997-2020 DLMS User Association

5.14 Previous versions of interface classes for setting up and managing DLMS/COSEM HS-PLC ISO/IEC 12139-1 neighbourhood networks

There are no previous versions to report.

5.15 Previous versions of ZigBee® setup classes

There are no previous versions to report.

© Copyright 1997-2020 DLMS User Association

6 Relation to OBIS

6.1 General

This Clause 6 specifies the use of COSEM interface objects to model various data items.

It also specifies the logical names of the objects. The naming system is based on OBIS, the Object Identification System: each logical name is an OBIS code.

The following rules for object instantiations are applicable with interface classes:

- if the use of IC “Data” is specified but it is not available in a given implementation, “Register” or “Extended register” (with scaler = 0, unit = 255) may be used;
- when, instead of a “Data” object, a “Register” or “Extended register” object is used, then the data types allowed for the value attribute of the “Data” IC are allowed.

OBIS codes are specified in the following subclauses:

- 6.2 specifies the use and the logical names of abstract COSEM objects, i.e. objects not related to an energy type;
- 6.3 specifies the use and logical names for electricity related COSEM objects;
- 6.4 specifies the use and logical names for heat cost allocator (HCA) related COSEM objects;
- 6.5 specifies the use and logical names for thermal energy related COSEM objects;
- 6.6 specifies the use and logical names for gas related COSEM objects;
- 6.7 specifies the use and logical names for water related COSEM objects;

  NOTE The use and the logical names of COSEM objects related to other media / energy types are under consideration.

- the detailed OBIS code allocations – for all media / energy types – are specified in Clause 7.

Unless otherwise specified the use of value group B shall be:

- if just one object is instantiated, the value in value group B shall be 0;
- if more than one object is instantiated in the same physical device, the value group B shall number the measurement or communication channels as appropriate, from 1…64. This is indicated by the letter “*b*”.

Unless otherwise specified the use of value group E shall be:

- if just one object is instantiated, value in value group E shall be 0;
- if more than one object is instantiated in the same physical device, the value group E shall number the instantiations from zero to the maximum value needed. This is indicated by the letter “*e*”. For the values allocated, see Clause 7.

All codes, which are not explicitly listed, but which are outside the manufacturer, utility or consortia specific ranges are reserved for future use.

6.2 Abstract COSEM objects

6.2.1 Use of value group C

Table 41 shows the use of value group C for abstract objects in the COSEM context. See also Table 57.

© Copyright 1997-2020 DLMS User Association

Table 41 – Use of value group C for abstract objects in the COSEM context

Value group C Abstract objects (A = 0)	
0	General purpose COSEM objects
1	Instances of IC "Clock"
2	Instances of IC “Modem configuration” and related IC-s
10	Instances of IC "Script table"
11	Instances of IC "Special days table"
12	Instances of IC "Schedule"
13	Instances of IC "Activity calendar"
14	Instances of IC “Register activation”
15	Instances of IC "Single action schedule"
16	Instances of IC “Register monitor”, “Parameter monitor”
17	Instances of IC “Limiter”
18	Instances of IC “Array manager”
19	COSEM objects related to payment metering: “Account”, “Credit”, “Charge”, “Token gateway”
20	Instances of IC "IEC local port setup"
21	Standard readout definitions
22	Instances of IC “IEC HDLC setup”
23	Instances of IC "IEC twisted pair (1) setup"
24	COSEM objects related to M-Bus
25	Instances of IC “TCP-UDP setup”, “IPv4 setup”, “IPv6 setup”, “MAC address setup”, “PPP setup”, “GPRS modem setup”, “SMTP setup”, “GSM diagnostic”, “FTP setup”, “NTP setup”, “LTE monitoring”, “SCHC-LPWAN setup”, “SCHC-LPWAN diagnostic”, “SCHC-LoRaWAN setup”, “SCHC-LoRaWAN diagnostic” Instances of “Push setup”,
26	COSEM objects for data exchange using S-FSK PLC networks
27	COSEM objects for ISO/IEC 8802-2 LLC layer setup
28	COSEM objects for data exchange using narrow-band OFDM PLC for PRIME networks
29	COSEM objects for data exchange using narrow-band OFDM PLC for G3-PLC networks
30	COSEM objects for data exchange using ZigBee®
31	Instances of IC “Wireless Mode Q” (M-Bus)
32	Instances of IC “ISO/IEC 14908 identification”, “ISO/IEC 14908 protocol setup”, “ ISO/IEC 14908 protocol setup”, “ ISO/IEC protocol status”, “ISO/IEC 14908 diagnostic”
33	COSEM objects for data exchange using HS-PLC ISO/IEC 12139-1 networks
34	Instances of IC “Wi-SUN setup”, “Wi-SUN diagnostic”, “RPL diagnostic”, “MPL diagnostic”
40	Instances of IC “Association SN/LN”
41	Instances of IC “SAP assignment”
42	COSEM logical device name
43	COSEM objects related to security: Instances of IC “Security setup” and “Data protection”
44	Instances of IC “Image transfer”, “Function control” and “Communication port protection” objects

© Copyright 1997-2020 DLMS User Association

Value group C Abstract objects (A = 0)	
65	Instances of IC “Utility tables”
66	Instances of “Compact data”
128…199	Manufacturer specific COSEM related abstract objects
All other	Reserved

6.2.2 Data of historical billing periods

COSEM provides three mechanisms to represent values of historical billing periods. These may be applicable to abstract and media specific objects.

The mechanisms are shown in Figure 33 and are described below:

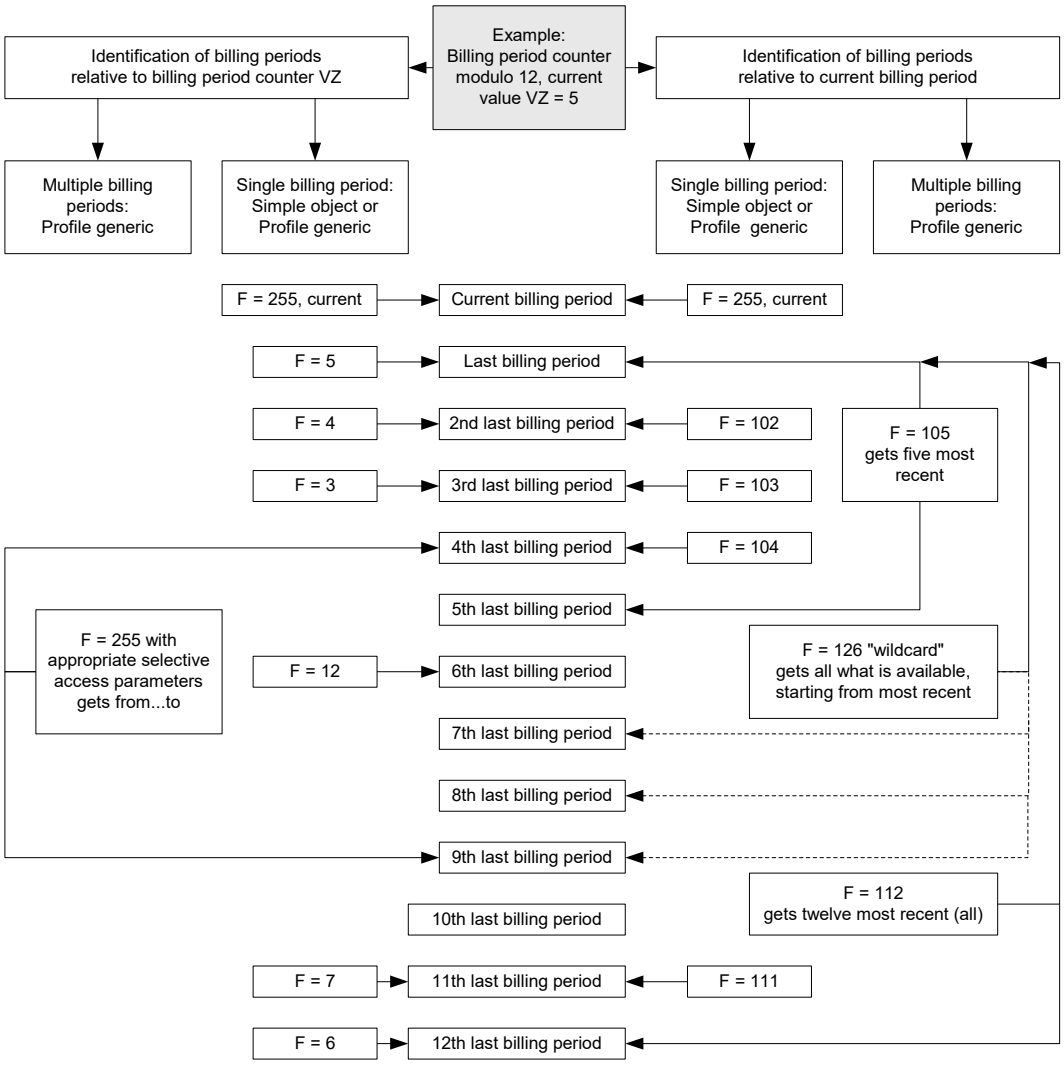


Figure 33 – Data of historical billing periods – example with modulo 12, VZ = 5

- a value of a single historical billing period may be represented using the same IC as used for representing the value of the current billing period. With F = 0…99, the billing period is identified by the value of the billing period counter VZ. F = VZ identifies the youngest value, F = VZ-1 identifies the second youngest value etc. F = 101…125 identifies the last, second last …25th last billing period. (F = 255 identifies the current billing period). Simple objects

© Copyright 1997-2020 DLMS User Association

can only be used to represent values of historical billing periods, if “Profile generic” objects are not implemented;

- a value of a single historical billing period may also be represented by “Profile generic” objects, which are one entry deep, and contain the historical value itself and the time stamp of the storage. With F = 0…99, the billing period is identified by the value of the billing period counter VZ. F = VZ identifies the youngest value, F = VZ-1 identifies the second youngest value etc. F=101 identifies the most recent billing period;
- values of multiple historical billing periods are represented with “Profile generic” objects, with suitable controlling attributes. With F = 102…125 the two last, …25 last values can be reached. F = 126 identifies an unspecified number of historical values;
- when values of historical billing periods are represented by “Profile generic” objects, more than one billing periods schemes may be used. The billing period scheme is identified by the billing period counter object captured in the profile.

6.2.3 Billing period values / reset counter entries

These values are represented by instances of the IC "Data”.

For billing period / reset counters and for number of available billing periods the data type shall be *unsigned*, *long-unsigned* or *double-long-unsigned*. For time stamps of billing periods, the data type shall be *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1.

These objects may be related to energy type – see also 6.3.3 and 6.4.3 – and channels.

When the values of historical periods are represented by “Profile generic” objects, the time stamp of the billing period objects shall be part of the captured objects.

Billing period values / reset counter entries	IC	OBIS code					
		A	B	C	D	E	F
For item names and OBIS codes see Table 60.	1, Data	0	*b*	0	1	*e*	255

6.2.4 Other abstract general purpose OBIS codes

Program entries shall be represented by instances of the IC "Data" with data type *unsigned, long-unsigned* or *octet-string*.

For identifying the firmware the following objects are available:

- Active firmware identifier objects hold the identifier of the currently active firmware;
- Active firmware version objects hold the version of the currently active firmware;

NOTE *Firmware version* can be used to distinguish between different releases of a firmware identified by the same *Firmware identifier*.

- Active firmware signature objects hold the digital signature of the currently active firmware. The digital signature algorithm is not specified here.

These three elements are inextricably linked to the currently active firmware.

Firmware identifiers may be also energy type and channel related.

Time entry values shall be represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the value attribute octet-string, formatted as *date-time* in 4.1.6.1.

For detailed OBIS codes, see Table 60.

© Copyright 1997-2020 DLMS User Association

Abstract general purpose OBIS codes	IC	OBIS code					
		A	B	C	D	E	F
Program entries	1, Data	0	*b*	0	2	*e*	255
Time entries	1, Data	0	*b*	0	9	*e*	255

6.2.5 Clock objects (class_id = 1, 8)

Instances of the IC "Clock" – see 4.5.1 – control the system clock of the physical device.

"UNIX clock" objects are instances of the Interface class "Data", with data type *double-long-unsigned*. They hold the number of seconds since 1970-01-01 00:00:00.

NOTE: DLMS uses *double-long-unsigned* rather than the UNIX standard *double-long-signed* data type to extend the roll over date to 2106.

"Microseconds clock" objects are instances of the IC "Data", with data type *long64-unsigned*. They hold the number of microseconds since 1970-01-01 00:00:00.

NOTE Previously, the name of these objects was "High resoluiton clock".

"Minutes clock" objects are instances of the IC "Data", with data type *double-long-unsigned*. They hold the whole number of minutes since 1970-01-01 00:00:00.

"Hours clock" objects are instances of the IC "Data", with data type *double-long-unsigned*. They hold the whole number of the hours since 1970-01-01 00:00:00.

"Days clock" objects are instances of the IC "Data", with data type *long-unsigned*. They hold the whole number of the days since 1970-01-01 00:00:00.

"Weeks clock" objects are instances of the IC "Data", with data type *long-unsigned*. They hold the whole number of weeks since 1970-01-01 00:00:00.

These objects are all inter-related in that it is anticipated that there is a single notion of time within the device.

Clock objects	IC	OBIS code					
		A	B	C	D	E	F
Clock	8, Clock	0	*b*	1	0	*e*	255
UNIX clock	1, Data [a]	0	*b*	1	1	*e*	255
Microseconds clock	1, Data [a]	0	*b*	1	2	*e*	255
Minutes clock	1, Data [a]	0	*b*	1	3	*e*	255
Hours clock	1, Data [a]	0	*b*	1	4	*e*	255
Days clock	1, Data [a]	0	*b*	1	5	*e*	255
Weeks clock	1, Data [a]	0	*b*	1	6	*e*	255

6.2.6 Modem configuration and related objects

In this group, the following objects are available:

- Instances of the IC "Modem configuration" – see 4.7.4 – define and control the behaviour of the device regarding the communication through a modem;
- Instances of the IC "Auto connect" – see 4.7.6 – define the necessary parameters for the management of sending information from the metering device to one or more destinations and for connection to the network;

© Copyright 1997-2020 DLMS User Association

- Instances of the IC “Auto answer" – see 4.7.5 – define and control the behaviour of the device regarding the auto answering function using a modem and handling wake-up calls and messages.

Modem configuration and related objects	IC	OBIS code					
		A	B	C	D	E	F
Modem configuration	27, Modem configuration	0	*b*	2	0	0	255
Auto connect	29, Auto connect	0	*b*	2	1	0	255
Auto answer	28, Auto answer	0	*b*	2	2	0	255

6.2.7 Script table objects (class_id = 9)

Instances of the IC “Script table” – see 4.5.2 – control the behaviour of the device.

Several instances are predefined and normally available as hidden scripts only with access to the *execute* () method. The following table contains only the identifiers for the “standard” instances of the listed scripts. Implementation specific instances of these scripts should use values different from zero in value group D.

- *MDI reset / End of billing period* “Script table” objects define the actions to be performed at the end of the billing period, for example the reset of maximum demand indicator registers and archiving data. If there are several billing period schemes available, then there shall be one script present in the array of scripts for each billing period scheme;
- *Tariffication* “Script table” objects define the entry point into tariffication by standardizing utility-wide how to invoke the activation of certain tariff conditions;
- *Disconnect control* “Script table” objects hold the scripts to invoke the methods of “Disconnect control” objects;
- *Image activation* “Script table” objects are is used to locally activate an Image transferred to the server, at the date and time held by an Image activation “Single action schedule” object;
- *Push* “Script table” objects hold scripts to activate the push operation. Normally every entry in the array of scripts calls the push method of one “Push setup” object instance;
- *Load profile control* “Script table” allow to change attributes of “Profile generic” objects e.g. to change the capture period and thus allow extended time control;
- *M-Bus profile control* “Script table” allow to change attributes of M-Bus related “Profile generic” objects e.g. to change the capture period and thus allow extended time control;
- *Function control* “Script table” objects allow making changes to “Function control” objects;
- *Broadcast* “Script table” objects allow standardising utility wide the entry point into regularly needed functionality.

Script table objects	IC	OBIS code					
		A	B	C	D	E	F
Global meter reset [a] Script table	9, Script table	0	*b*	10	0	0	255
MDI reset / End of billing period [a] Script table		0	*b*	10	0	1	255
Tariffication Script table		0	*b*	10	0	100	255
Activate test mode [a] Script table		0	*b*	10	0	101	255
Activate normal mode [a] Script table		0	*b*	10	0	102	255
Set output signals Script table		0	*b*	10	0	103	255

© Copyright 1997-2020 DLMS User Association

Script table objects	IC	OBIS code					
		A	B	C	D	E	F
Switch optical test output [b, c] Script table		0	*b*	10	0	104	255
Power quality measurement management Script table		0	*b*	10	0	105	255
Disconnect control Script table		0	*b*	10	0	106	255
Image activation Script table		0	*b*	10	0	107	255
Push Script table		0	*b*	10	0	108	255
Load profile control Script table		0	*b*	10	0	109	255
M-Bus profile control Script table		0	*b*	10	0	110	255
Function control Script table		0	*b*	10	0	111	255
Broadcast Script table		0	*b*	10	0	125	255

[a] The activation of these scripts is performed by calling the execute() method to the script identifier 1 of the corresponding script object.

[b] The optical test output is switched to measuring quantity Y and the test mode is activated by calling the execute method of the script table object 0.x.10.0.104.255 using Y as parameter; where Y is given by Clause 7.5.1, Table 65. The default value of A is 1 (Electricity).

EXAMPLE In the case of electricity meters, A = 1, default, execute (21) switches the test output to display the active power + of phase 1.

[c] The optical test output is also switched back to its default value when this script is activated.

6.2.8 Special days table objects (class_id = 11)

Instances of the IC “Special days table” – see 4.5.4 – define and control the behaviour of the device regarding calendar functions on special days for clock control.

Special days table objects	IC	OBIS code					
		A	B	C	D	E	F
Special days table	11, Special days table	0	*b*	11	0	*e*	255

6.2.9 Schedule objects (class_id = 10)

Instances of the IC “Schedule” – see 4.5.3 – define and control the behaviour of the device in a sequenced way.

Schedule objects	IC	OBIS code					
		A	B	C	D	E	F
Schedule	10, Schedule	0	*b*	12	0	*e*	255

6.2.10 Activity calendar objects (class_id = 20)

Instances of the IC “Activity calendar” – see 4.5.5 – define and control the behaviour of the device in a calendar-based way.

Activity calendar objects	IC	OBIS code					
		A	B	C	D	E	F
Activity calendar	20, Activity calendar	0	*b*	13	0	*e*	255

© Copyright 1997-2020 DLMS User Association

6.2.11 Register activation objects (class_id = 6)

Instances of the IC “Register activation”– see 4.3.5 – are used to handle different tariffication structures.

Register activation objects	IC	OBIS code					
		A	B	C	D	E	F
Register activation	6, Register activation	0	*b*	14	0	*e*	255

6.2.12 Single action schedule objects (class_id = 22)

Instances of the IC “Single action schedule” – see 4.5.7 – control the behaviour of the device. Implementation specific instances should use values different from zero in value group D.

Instances of Push “Single action schedule” objects activate scripts in Push “Script table” objects, which invoke the push method of the appropriate “Push setup” objects.

Load profile control “Single action schedule” objects activate scripts in Load profile control “Script table” objects and thus allow extended time control.

M-Bus profile control “Single action schedule” objects activate scripts in M-Bus profile control “Script table” objects and thus allow extended time control.

Function control “Single action schedule” objects activate scripts in Function control “Script table” objects.

Single action schedule objects	IC	OBIS code					
		A	B	C	D	E	F
End of billing period Single action schedule	22, Single action schedule	0	*b*	15	0	0	255
Disconnect control Single action schedule		0	*b*	15	0	1	255
Image activation Single action schedule		0	*b*	15	0	2	255
Output control Single action schedule		0	*b*	15	0	3	255
Push Single action schedule		0	*b*	15	0	4	255
Load profile control Single action schedule		0	*b*	15	0	5	255
M-Bus profile control Single action schedule		0	*b*	15	0	6	255
Function control Single action schedule		0	*b*	15	0	7	255

6.2.13 Register monitor and alarm monitor objects (class_id = 21)

Instances of the IC “Register monitor” – see 4.5.6 – control the register monitoring and alarm monitoring function of the device. They define the value to be monitored, the set of thresholds to which the value is compared, and the actions to be performed when a threshold is crossed.

In general, the logical name(s) shown in the table below shall be used. See also 6.3.9 and 6.3.10.

Alarm monitor objects monitor *Alarm register* or *Alarm descriptor* objects.

Register monitor objects	IC	OBIS code					
		A	B	C	D	E	F
Register monitor	21, Register monitor	0	*b*	16	0	*e*	255
Alarm monitor		0	*b*	16	1	0…9	255

© Copyright 1997-2020 DLMS User Association

6.2.14 Parameter monitor objects (class_id = 65)

Instances of the IC “Parameter monitor” – see 4.5.10 – control the Parameter monitoring function of the device. They define the list of parameters to be monitored and hold the identifier and the value of the last parameter changed, as well as the *capture_time*.

Parameter monitor objects	IC	OBIS code					
		A	B	C	D	E	F
Parameter monitor	65, Parameter monitor	0	*b*	16	2	*e*	255

6.2.15 Limiter objects (class_id = 71)

Instances of the IC “Limiter” handle the monitoring of values in normal and emergency conditions. See also 4.5.9.

Limiter objects	IC	OBIS code					
		A	B	C	D	E	F
Limiter	71, Limiter	0	*b*	17	0	*e*	255

6.2.16 Array manager objects (class_id = 123)

Instances of the IC “Array manager” – see 4.4.11 – allow managing COSEM interface object attributes of type *array*.

Array manager objects	IC	OBIS code					
		A	B	C	D	E	F
Array manager	123	0	*b*	18	0	*e*	255

6.2.17 Payment metering related objects

Payment accounting can be applied to any commodity.

An instance of the “Account” – see 4.6.2 – IC holds the summary information for a given contract and lists the “Credit” and “Charge” objects used by that “Account”. If more than one “Account” is for any reason required in a given context then field D should be other than 0.

One or several instances of the “Credit” IC – see 4.6.3.4 – represent the different credit sources.

One or several instances of the “Charge” IC – see 4.6.4 – represent the different charges applicable.

One or more instances of the “Token gateway” IC – see 4.6.5 – are available to enter tokens. If only a single gateway is defined in a single “Account” then field E of the OBIS code shall be zero. If more than one “Token gateway” object is for any reason required in a single “Account” then field E should be other than 0.

The “Account” is linked to it’s associated “Credit”, “Charge” and “Token gateway” objects by use of the value group D and B field such that an “Account” with D=0 should be linked to a “Token gateway” with D=40 and have a “Credit” objects with D=10 and “Charge” objects with D=20. Whereas an “Account” with D=1 should have “token gateway” with D=41, “Credit” objects with D=11 and “Charge” objects with D=21 etc. Multiple “Credit” and “Charge” objects are identified using different values in the value group E field. See also Additional Notes there describing the “Max credit_limit” and „Max vend limit” objects there.

© Copyright 1997-2020 DLMS User Association

Instances of “Profile Generic“ IC hold the history of the token credit and of the charge collections with a “Parameter Monitor” interface class monitoring a value used to trigger capture.

Payment metering related objects	IC	OBIS code					
		A	B	C	D	E	F
Account	111, Account	0	*b*	19	0..9	0	255
Credit	112, Credit	0	*b*	19	10..19	e	255
Charge	113, Charge	0	*b*	19	20..29	e	255
Token gateway	115, Token gateway	0	*b*	19	40…49	*e*	255
Configurable limit objects							
Max credit limit	01, Data	0	*b*	19	50…59	1	255
Max vend limit	01, Data	0	*b*	19	50…59	2	255

6.2.18 IEC local port setup objects (class_id = 19)

These objects define and control the behaviour of local ports using the protocol specified in IEC 62056-21:2002. See also 4.7.1.

IEC local port setup objects	IC	OBIS code					
		A	B	C	D	E	F
IEC optical port setup	19, IEC local port setup	0	*b*	20	0	0	255
IEC electrical port setup		0	*b*	20	0	1	255

6.2.19 Standard readout profile objects (class_id = 7)

A set of objects is defined to carry the standard readout as it would appear with IEC 62056-21:2002 (modes A to D). See also 4.3.6.

Standard readout objects	IC	OBIS code					
		A	B	C	D	E	F
General local port readout	7, Profile generic	0	*b*	21	0	0	255
General display readout		0	*b*	21	0	1	255
Alternate display readout		0	*b*	21	0	2	255
Service display readout		0	*b*	21	0	3	255
List of configurable meter data		0	*b*	21	0	4	255
Additional readout profile 1		0	*b*	21	0	5	255
……							
Additional readout profile *n*		0	*b*	21	0	N	255

For the parametrization of the standard readout “Data” objects can be used.

Standard readout parametrization objects	IC	OBIS code					
		A	B	C	D	E	F
Standard readout parametrization	1, Data	0	*b*	21	0	*e*	255

© Copyright 1997-2020 DLMS User Association

6.2.20 IEC HDLC setup objects (class_id = 23)

Instances of the IC "IEC HDLC setup" – see 4.7.2 – hold the parameters of the HDLC based data link layer.

IEC HDLC setup objects	IC	OBIS code					
		A	B	C	D	E	F
IEC HDLC setup	23, IEC HDLC setup	0	*b*	22	0	0	255

6.2.21 IEC twisted pair (1) setup objects (class_id = 24 etc.)

An instance of the IC "IEC twisted pair (1) set up" IC – see 4.7.3 – stores the parameters necessary to manage a communication profile specified in IEC 62056-3-1:2013.

An instance of the IC "MAC address set up" IC stores the Secondary Station Address ADS.

An instance of the "Data" stores the Fatal Error register.

Instances of the IC "Profile generic" IC instances allow the configuration of IEC 62056-3-1 readout lists.

IEC twisted pair (1) setup and related objects	IC	OBIS code					
		A	B	C	D	E	F
IEC twisted pair (1) setup	24, IEC twisted pair (1) setup	0	*b*	23	0	0	255
IEC twisted pair (1) MAC address setup	43, MAC address setup	0	*b*	23	1	0	255
IEC twisted pair (1) Fatal Error register	1, Data	0	b	23	2	0	255
IEC 62056-3-1 Short readout	7, Profile generic	0	*b*	23	3	0	255
IEC 62056-3-1 Long readout		0	*b*	23	3	1	255
IEC 62056-3-1 Alternate readout profile 0		0	b	23	3	2	255
IEC 62056-3-1 Additional readout profile 1		0	b	23	3	3	255
IEC 62056-3-1 Additional readout profile 2		0	b	23	3	4	255
IEC 62056-3-1 Additional readout profile 7		0	b	23	3	9	255

For the parametrization of the IEC 62056-3-1 readout "Data" objects can be used.

Standard readout parametrization objects	IC	OBIS code					
		A	B	C	D	E	F
IEC 62056-3-1 readout parametrization	1, Data	0	*b*	23	3	*e*	255

6.2.22 Objects related to data exchange over M-Bus

The following objects are available to model and control data exchange using the M-Bus protocol specified in the EN 13757 series:

- instances of the IC "M-Bus slave port setup" define and control the behaviour of M-Bus slave ports of a DLMS/COSEM device. See 4.8.2;

© Copyright 1997-2020 DLMS User Association

- instances of the IC “M-Bus client” are used to configure DLMS/COSEM devices as M-Bus clients. There is one “M-Bus client” object for each M-Bus slave. Value group B identifies the M-Bus channels. See 4.8.3;
- M-Bus value objects, instances of the IC “Extended register”, hold the values captured from M-Bus slave devices on the relevant channel. The link between the M-Bus client setup objects and the M-Bus value objects is provided by the channel number.
- M-Bus “Profile generic” objects capture M-Bus value objects possibly along with other, not M-Bus specific objects;
- M-Bus “Disconnect control” objects control disconnect devices of M-Bus devices (e.g. gas valves);
- instances of the IC “Wireless mode Q” define and control the behaviour of the device regarding the communication parameters according to mode Q of EN 13757-5:2015. A node having more than one network address, i.e. a multi-homed node, will have multiple objects of these types. See 4.8.4;
- M-Bus control log objects are instances of the IC “Profile generic”. They log the changes of the state of the disconnect devices;
- instances of the IC “M-Bus master port setup” define and control the behaviour of M-Bus master ports of DLMS/COSEM devices, allowing to exchange data with M-Bus slaves. See 4.8.5;
- instances of the IC ”DLMS/COSEM server M-Bus port setup” are used in DLMS/COSEM servers hosted by M-Bus slave devices, using the DLMS/COSEM wired or wireless M-Bus communication profile. See 4.8.6;
- instances of the IC “M-Bus diagnostic” hold information related to the operation of the M-Bus network. See 4.8.7.

Objects related to data exchange over M-Bus	**IC**	**OBIS code**					
		A	**B**	**C**	**D**	**E**	**F**
M-Bus slave port setup	25, M-Bus slave port setup	0	*b*	24	0	0	255
M-Bus client	72, M-Bus client	0	*b*	24	1	0	255
M-Bus value	4, Extended register	0	*b*	24	2	e [a]	255
M-Bus profile generic	7, Profile generic	0	*b*	24	3	e	255
M-Bus disconnect control	70, Disconnect control	0	*b*	24	4	0	255
M-Bus control log	7, Profile generic	0	*b*	24	5	0	255
M-Bus master port setup	74, M-Bus master port setup	0	*b*	24	6	0	255
Wireless Mode Q channel	73, Wireless Mode Q channel	0	*b*	31	0	0	255
DLMS/COSEM server M-Bus port setup	76, DLMS/COSEM server M-Bus port setup	0	*b*	24	8	e [b]	255
M-Bus diagnostic	77, M-Bus diagnostic	0	*b*	24	9	e [b]	255

[a] “e” is equal to the index of the captured value in accordance to index of capture_definition_element in the *capture_definition* attribute of the M-Bus client object.

[b] If there is more than one M-Bus network interface present then there may be one object instantiated for each interface. For example, if a device has two interfaces (one wired M-Bus and one wireless M-Bus) and uses the DLMS/COSEM M-Bus communication profiles on both, there shall be one instance for each interface.

6.2.23 Objects to set up data exchange over the Internet

In this group, the following objects are available:

- Instances of the IC “TCP-UDP setup” – see 4.9.1 – handle all information related to the setup of the TCP and UDP layer of the Internet based communication profile(s), and point

© Copyright 1997-2020 DLMS User Association

to the IP setup object(s) handling the setup of the IP layer on which the TCP-UDP connection(s) is (are) used;

- Instances of the IC “IPv4 setup” – see 4.9.2 – handle all information related to the setup of the IPv4 layer of the Internet based communication profile(s) and point to the data link layer setup object(s) handling the setup of the data link layer on which the IP connections is (are) used;
- Instances of the IC “IPv6 setup” – see 4.9.3 – handle all information related to the setup of the IPv6 layer of the Internet based communication profile(s) and point to the data link layer setup object(s) handling the setup of the data link layer on which the IP connections is (are) used;
- Instances of the IC “MAC address setup” – see 4.9.4 – handle all information related to the setup of the Ethernet data link layer of the Internet based communication profile(s);
- Instances of the IC “PPP setup” – see 4.9.5 – handle all information related to the setup of the PPP data link layer of the Internet based communication profiles;
- Instances of the IC “GPRS modem setup” – see 4.7.7 – handle all information related to the setup of the GPRS modem;
- Instances of the IC “SMTP setup” – see 4.9.6 – handle all information related to the setup of the SMTP service.

NOTE The following objects have internet related OBIS codes, although they are not strictly related to use over the internet only.

- Instances of the IC “GSM diagnostic” – see 4.7.8 – handle all diagnostic information related to the GSM/GPRS network.
- Instances of the IC "NTP setup" – see 4.9.7 – handle all information related to the setup of the NTP time synchronisation service;
- Instances of the IC “LTE monitoring ” – see 4.7.9 – allow monitoring LTE modems.
- Instances of the IC “SCHC-LPWAN setup” – see 4.16.2.1 – handle all information related to the setup of the SCHC-LPWAN layer;

© Copyright 1997-2020 DLMS User Association

- Instances of the IC “SCHC-LPWAN diagnostic” – see 4.16.2.2– handle all diagnostic information related to the SCHC-LPWAN network.

Objects to set up data exchange over the Internet	IC	OBIS code					
		A	B	C	D	E	F
TCP-UDP setup	41, TCP-UDP setup	0	*b*	25	0	0	255
IPv4 setup	42, IPv4 setup	0	*b*	25	1	0	255
MAC address setup	43, MAC address setup	0	*b*	25	2	0	255
PPP setup	44, PPP setup	0	*b*	25	3	0	255
GPRS modem setup	45, GPRS modem setup	0	*b*	25	4	0	255
SMTP setup	46, SMTP setup	0	*b*	25	5	0	255
GSM diagnostic	47, GSM diagnostic	0	*b*	25	6	0	255
IPv6 setup	48, IPv6 setup	0	*b*	25	7	0	255
Reserved for FTP setup							
NTP setup	100, NTP setup	0	*b*	25	10	0	255
LTE monitoring	151, LTE monitoring	0	*b*	25	11	0	255
SCHC-LPWAN setup	126, SCHC-LPWAN setup	0	*b*	25	12	0	255
SCHC-LPWAN diagnostic	127, SCHC-LPWAN diagnostic	0	*b*	25	13	0	255
SCHC-LoRaWAN setup	128, SCHC – LoRaWAN setup	0	*b*	25	14	0	255
SCHC-LoRaWAN diagnostic	129, SCHC – LoRaWAN diagnostic	0	*b*	25	15	0	255

6.2.24 Objects to set up Push Setup (class_id = 40)

Instances of the IC “Push setup” – see 4.4.8 – handle all information about the data to be pushed, the push destination and the method by which the data should be pushed.

Push Setup	IC	OBIS code					
		A	B	C	D	E	F
Push setup	40, Push setup	0	*b*	25	9	0	255

6.2.25 Objects for setting up data exchange using S-FSK PLC

In this group, the following objects are available:

- Instances of the IC “S-FSK Phy&MAC setup” – see 4.10.3 – handle all information related to setting up the PLC S-FSK lower layer profile specified in IEC 61334-5-1:2001;
- Instances of the IC “S-FSK Active initiator” – see 4.10.4 – handle all information related to the active initiator in the PLC S-FSK lower layer profile specified in IEC 61334-5-1:2001;
- Instances of the IC “S-FSK MAC synchronization timeouts” – see 4.10.5 – manage all timeouts related to the synchronization process of devices using the PLC S-FSK lower layer profile specified in IEC 61334-5-1:2001;
- Instances of the IC “S-FSK MAC counters” – see 4.10.6 – store counters related to the frame exchange, transmission and repetition phases in the PLC S-FSK lower layer profile specified in IEC 61334-5-1:2001;
- Instances of the IC “IEC 61334-4-32 LLC setup” – see 4.10.7 – handle all information related to the LLC layer specified in IEC 61334-4-32:1996;

© Copyright 1997-2020 DLMS User Association

- Instances of the IC “S-FSK Reporting system list” – see 4.10.8 – hold information on reporting systems in the PLC S-FSK lower layer profile specified in IEC 61334-5-1:2001.

Objects to set up data exchange using S-FSK PLC	IC	OBIS code					
		A	B	C	D	E	F
S-FSK Phy&MAC setup	50, S-FSK Phy&MAC setup	0	*b*	26	0	0	255
S-FSK Active initiator	51, S-FSK Active initiator	0	*b*	26	1	0	255
S-FSK MAC synchronization timeouts	52, S-FSK MAC synchronization	0	*b*	26	2	0	255
S-FSK MAC counters	53, S-FSK MAC counters	0	*b*	26	3	0	255
NOTE This is a placeholder for a Monitoring IC to be specified.							
IEC 61334-4-32 LLC setup	55, IEC 61334-4-32 LLC setup	0	*b*	26	5	0	255
S-FSK Reporting system list	56, S-FSK Reporting system list	0	*b*	26	6	0	255

6.2.26 Objects for setting up the ISO/IEC 8802-2 LLC layer

In this group, the following objects are available:

- Instances of the IC “ISO/IEC 8802-2 LLC Type 1 setup” – see 4.11.2 – handle all information related to the LLC layer specified in ISO/IEC 8802-2:1998 in Type 1 operation;
- Instances of the IC “ISO/IEC 8802-2 LLC Type 2 setup” – see 4.11.3 – handle all information related to the LLC layer specified in ISO/IEC 8802-2:1998 in Type 2 operation;
- Instances of the IC “ISO/IEC 8802-2 LLC Type 3 setup” – see 4.11.4 – handle all information related to the LLC layer specified in ISO/IEC 8802-2:1998 in Type 3 operation.

Objects to set up the ISO/IEC 8802-2 LLC layer	IC	OBIS code					
		A	B	C	D	E	F
ISO/IEC 8802-2 LLC Type 1 setup	57, ISO/IEC 8802-2 LLC Type 1 setup	0	*b*	27	0	0	255
ISO/IEC 8802-2 LLC Type 2 setup	58, ISO/IEC 8802-2 LLC Type 2 setup	0	*b*	27	1	0	255
ISO/IEC 8802-2 LLC Type 3 setup	59, ISO/IEC 8802-2 LLC Type 3 setup	0	*b*	27	2	0	255

6.2.27 Objects for data exchange using narrowband OFDM PLC for PRIME networks

For setting up and managing data exchange using narrowband OFDM PLC for PRIME networks one instance of each following classes shall be implemented for each interface:

- an instance of the 61334-4-32 LLC SSCS setup – see 4.12.3 – holds the addresses related to the CL_432 layer;
- an instance of the IC “PRIME NB OFDM PLC Physical layer counters” – see 4.12.5 – stores counters related to the physical layers exchanges;
- an instance of the IC “PRIME NB OFDM PLC MAC setup” – see 4.12.6 – holds the necessary parameters to set up the PRIME NB OFDM PLC MAC layer;
- an instance of the IC “PRIME NB OFDM PLC MAC functional parameters” – see 4.12.7 – provides information on specific aspects concerning the functional behaviour of the MAC layer;
- an instance of the IC “PRIME NB OFDM PLC MAC counters” – see 4.12.8 – stores statistical information on the operation of the MAC layer for management purposes;

© Copyright 1997-2020 DLMS User Association

- an instance of the IC “PRIME NB OFDM PLC MAC network administration data” – see 4.12.9 – holds the parameters related to the management of the devices connected to the network;
- an instance of the IC “MAC address setup” – holds the MAC address of the device. See 4.12.10;
- an instance of the IC “PRIME NB OFDM PLC Application identification” – see 4.12.11 – holds identification information related to administration and maintenance of PRIME NB OFDM PLC devices.

Objects for data exchange using PRIME NB OFDM PLC	IC	OBIS code					
		A	B	C	D	E	F
61334-4-32 LLC SSCS setup	80, 61334-4-32 LLC SSCS setup	0	*b*	28	0	0	255
PRIME NB OFDM PLC Physical layer counters	81, PRIME NB OFDM PLC Physical layer counters	0	*b*	28	1	0	255
PRIME NB OFDM PLC MAC setup	82, PRIME NB OFDM PLC MAC setup	0	*b*	28	2	0	255
PRIME NB OFDM PLC MAC functional parameters	83, PRIME NB OFDM PLC MAC functional parameters	0	*b*	28	3	0	255
PRIME NB OFDM PLC MAC counters	84, PRIME NB OFDM PLC MAC counters	0	*b*	28	4	0	255
PRIME NB OFDM PLC MAC network administration data	85, PRIME NB OFDM PLC MAC network administration data	0	*b*	28	5	0	255
PRIME NB OFDM PLC MAC address setup	43, MAC address setup	0	*b*	28	6	0	255
PRIME NB OFDM PLC Application identification	86, PRIME NB OFDM PLC Application identification	0	*b*	28	7	0	255

6.2.28 Objects for data exchange using narrow-band OFDM PLC for G3-PLC networks

For setting up and managing data exchange using G3-PLC profile, one instance of each following classes shall be implemented for each interface:

- an instance of the IC “G3-PLC MAC layer counters” – see 4.13.3 – to store counters related to the MAC layer exchanges;
- an instance of the IC “G3-PLC MAC setup” – see 4.13.4 – to hold the necessary parameters to set up the G3-PLC MAC IEEE 802.15.4 layer;
- an instance of the IC “G3-PLC 6LoWPAN adaptation layer setup” – see 4.13.5 – to hold the necessary parameters to set up the Adaptation layer.

Objects for data exchange using G3-PLC	IC	OBIS code					
		A	B	C	D	E	F
G3-PLC MAC layer counters	90, G3-PLC MAC layers counters	0	*b*	29	0	0	255
G3-PLC MAC setup	91, G3-PLC MAC setup	0	*b*	29	1	0	255
G3-PLC 6LoWPAN adaptation layer setup	92, G3-PLC 6LoWPAN adaptation layer setup	0	*b*	29	2	0	255

© Copyright 1997-2020 DLMS User Association

6.2.29 ZigBee® setup objects

The following objects are available for setting up and managing a ZigBee® network; see also 4.15.

Objects set up and manage ZigBee® networks	IC	OBIS code					
		A	B	C	D	E	F
ZigBee® SAS startup	101, ZigBee® SAS Startup	0	*b*	30	0	e	255
ZigBee® SAS join	102, ZigBee® SAS join	0	*b*	30	1	e	255
ZigBee® SAS APS fragmentation	103, ZigBee® SAS APS fragmentation	0	*b*	30	2	e	255
ZigBee® network control	104, ZigBee® network control	0	*b*	30	3	e	255
ZigBee® tunnel setup	105 ,ZigBee® tunnel setup	0	*b*	30	4	e	255

6.2.30 Objects for setting up and managing data exchange using ISO/IEC 14908 PLC networks

For setting up and managing data exchange using ISO/IEC 14908 PLC networks at least one instance of each of the following interface classes shall be implemented, see also 4.16

Objects to set up and manage ISO/IEC 14908 PLC netowrks	IC	OBIS identification					
		A	B	C	D	E	F
ISO/IEC 14908 identification	130, ISO/IEC 14908 identification	0	*b*	32	0	0	255
ISO/IEC 14908 protocol setup	131, ISO/IEC 14908 protocol setup	0	*b*	32	1	0	255
ISO/IEC 14908 protocol status	132, ISO/IEC 14908 protocol status	0	*b*	32	2	0	255
ISO/IEC 14908 diagnostic	133, ISO/IEC 14908 diagnostic	0	*b*	32	3	0	255

6.2.31 Objects for data exchange using HS-PLC ISO/IEC 12139-1 ISO/EC 12139-1 networks

For setting up and managing data exchange using HS-PLC ISO/IEC 12139-1 networks one instance of each following classes shall be implemented for each interface:

- an instance of the IC “HS-PLC ISO/IEC 12139-1 MAC setup” – see 4.14.2 – holds the necessary parameters for setting up the MAC layer;
- an instance of the IC “HS-PLC ISO/IEC 12139-1 CPAS setup” – see 4.14.3 – holds the necessary parameters for setting up the CPAS;
- an instance of the IC “HS-PLC ISO/IEC 12139-1 IP SSAS setup” – see 4.14.4 – holds the necessary parameters for setting up the IP SSAS;
- an instance of the IC “HS-PLC ISO/IEC 12139-1 HDLC SSAS setup” – see 4.14.5 – holds the necessary parameters for setting up the HDLC SSAS.

Objects for data exchange using HS-PLC ISO/IEC 12139-1	IC	OBIS code					
		A	B	C	D	E	F
HS-PLC ISO/IEC 12139-1 MAC setup	140, HS-PLC ISO/IEC 12139-1 MAC setup	0	*b*	33	0	0	255
HS-PLC ISO/IEC 12139-1 CPAS setup	141, HS-PLC ISO/IEC 12139-1 CPAS setup	0	*b*	33	1	0	255
HS-PLC ISO/IEC 12139-1 IP SSAS setup	142, HS-PLC ISO/IEC 12139-1 IP SSAS setup	0	*b*	33	2	0	255

© Copyright 1997-2020 DLMS User Association

HS-PLC ISO/IEC 12139-1 HDLC SSAS setup	143, HS-PLC ISO/IEC 12139-1 HDLC SSAS setup	0	*b*	33	3	0	255

6.2.32 Objects for data exchange using Wi-SUN networks

For setting up and managing data exchange using Wi-SUN networks one instance of each following classes shall be implemented for each interface:

- An instance of the IC “IPv6 setup” – see 4.9.3 – handle all information related to the setup of the IPv6 layer of the Internet based communication profile(s) and point to the data link layer setup object(s) handling the setup of the data link layer on which the IP connections is (are) used;
- An instance of the IC “Wi-SUN setup” – see 4.17.1 – handle all information related to the setup of the SMTP service.
- An instance of the IC “Interface diagnostic” – see 4.17.2 – handle all diagnostic information related to the network.
- An instance of the IC “RPL diagnostic” – see 4.17.3 – handle all diagnostic information related to RPL.
- An instance of the IC “MPL diagnostic” – see 4.17.4 – handle all diagnostic information related to MPL.

Objects to set up data exchange over the Wi-SUN FAN	**IC**	**OBIS code**					
		A	**B**	**C**	**D**	**E**	**F**
Wi-SUN setup	95, Wi-SUN setup	0	*b*	34	0	0	255
Wi-SUN diagnostic	96, Wi-SUN diagnostic	0	*b*	34	1	0	255
RPL diagnostic	97, RPL diagnostic	0	*b*	34	2	0	255
MPL diagnostic	98, MPL diagnostic	0	*b*	34	3	0	255

6.2.33 Association objects (class_id = 12, 15)

A series of Association SN / LN objects – see 4.4.3, 5.4– are available to model application associations between a DLMS/COSEM client and server.

Association objects	**IC**	**OBIS code**					
		A	**B**	**C**	**D**	**E**	**F**
Current association	12, Association SN 15, Association LN	0	0	40	0	0	255
Association, instance 1		0	0	40	0	1	255
……							
Association, instance *n*		0	0	40	0	n	255

6.2.34 SAP assignment object (class_id = 17)

An instance of the IC “SAP assignment” – see 4.4.5 – holds information about the addresses (Service Access Points, SAPs) of logical devices within a physical device.

© Copyright 1997-2020 DLMS User Association

SAP Assignment object	IC	OBIS code					
		A	B	C	D	E	F
SAP assignment of current physical device	17, SAP assignment	0	0	41	0	0	255

6.2.35 COSEM logical device name object

Each COSEM logical device shall be identified by its Logical Device Name, unique worldwide. See 4.1.8.2. It is held by the *value* attribute of a "Data" object, with data type *octet-string*, *visible-string* or utf8-string. For short name referencing, the base_name of the object is fixed. See 4.1.3.

COSEM logical device name object	IC	OBIS code					
		A	B	C	D	E	F
COSEM logical device name	1, Data [a]	0	0	42	0	0	255
[a] If the IC "Data" is not available, "Register" (with scaler = 0, unit = 255) may be used.							

6.2.36 Information security related objects (class_id = 64 etc.)

Instances of the IC "Security setup" – see 4.4.7 – are used to set up the message security features. For each Association object, there is one Security setup object managing security within that AA. See 5.4.4 and 5.4.5. Value group E numbers the instances.

Security setup objects	IC	OBIS code					
		A	B	C	D	E	F
Security setup	64, Security setup	0	0	43	0	*e*	255

Invocation counter objects hold the invocation counter element of the initialization vector. They are instances of the IC "Data". The value in value group B identifies the communication channel.

NOTE The same client may use different communication channels e.g. a remote port and a local port. The invocation counter on the different channels may be different.

The value in value group E shall be the same as in the logical name of the corresponding "Security setup" object.

Invocation counter objects	IC	OBIS code					
		A	B	C	D	E	F
Invocation counter	1, Data [a]	0	*b*	43	1	*e*	255
NOTE In earlier version of the Blue Book, these objects were called Frame counter objects.							
[a] If the IC "Data" is not available, "Register" (with scaler = 0, unit = 255) may be used.							

Instances of the IC "Data protection" – see 4.4.9 – are used to apply / remove protection on COSEM data, i.e. sets of attributes values, method invocation and return parameters. Value group E numbers the instances.

Data protection objects	IC	OBIS code					
		A	B	C	D	E	F
Data protection	30, Data protection	0	0	43	2	*e*	255

© Copyright 1997-2020 DLMS User Association

6.2.37 Image transfer objects (class_id = 18)

Instances of the IC “Image transfer“ – see 4.4.6 – control the Image transfer process.

Image transfer related objects	IC	OBIS code					
		A	B	C	D	E	F
Image transfer	18, Image transfer	0	0	44	0	*e*	255

6.2.38 Function control objects (class_id = 122)

Instances of the IC “Function control“ – see 4.4.10 – allow enabling and disabling functions in the server.

Function control related objects	IC	OBIS code					
		A	B	C	D	E	F
Function control	122, Function control	0	0	44	1	*e*	255

6.2.39 Communication port protection objects (class_id = 124)

Instances of the “Communication port protection” IC – see 4.4.12 – control the communication port protection mechanism.

Communication port protection objects	IC	OBIS code					
		A	B	C	D	E	F
Communication port protection objects	124	0	*b*	44	2	*e*	255

6.2.40 Utility table objects (class_id = 26)

Instances of the IC “Utility tables” – see 4.3.7 – allow representing ANSI utility tables. The Utility table IDs are mapped to OBIS codes as follows:

- value group A: use value of 0 to specify abstract object;
- value group B: instance of table set;
- value group C: use value 65 – signifies utility tables specific definitions;
- value group D: table group selector;
- value group E: table number within group;
- value group F: use value 0xFF for data of current billing period.

Utility table objects	IC	OBIS code					
		A	B	C	D	E	F
Standard tables 0-127	26, Utility tables	0	*b*	65	0	*e*	255
Standard tables 128-255		0	*b*	65	1	*e*	255
…							
Standard tables 1920-2047		0	*b*	65	15	*e*	255
Manufacturer tables 0-127		0	*b*	65	16	*e*	255
Manufacturer tables 128-255		0	*b*	65	17	*e*	255
…							

© Copyright 1997-2020 DLMS User Association

Utility table objects	IC	OBIS code					
		A	B	C	D	E	F
Manufacturer tables 1920-2047		0	*b*	65	31	*e*	255
Std pending tables 0-127		0	*b*	65	32	*e*	255
Std pending tables 128-255		0	*b*	65	33	*e*	255
…							
Std pending tables 1920-2047		0	*b*	65	47	*e*	255
Mfg pending tables 0-127		0	*b*	65	48	*e*	255
Mfg pending tables 128-255		0	*b*	65	49	*e*	255
…							
Mfg pending tables 1920-2047		0	*b*	65	63	*e*	255

6.2.41 Compact data objects (class_id = 62)

"Compact data" objects – see 4.3.10 – store data and metadata separated, thus they allow reducing overhead.

Compact data objects	IC	OBIS code					
		A	B	C	D	E	F
Compact data	62, Compact data	0	*b*	66	0	e	255

6.2.42 Device ID objects

A series of objects are used to hold ID numbers of the device. These ID numbers can be defined by the manufacturer (e.g. manufacturing number) or by the user.

They are held by the *value* attribute of "Data" objects, with data type *double-long-unsigned*, *octet-string*, *visible-string, utf8-string*, *unsigned, long-unsigned.* If more than one of those is used, it is allowed to combine them into a "Profile generic" object. In this case, the captured objects are *value* attributes of the device ID "Data" objects, the capture period is 1 to have just actual values, the sort method is FIFO and the profile entries are limited to 1. Alternatively, a "Register table" object – see 4.3.8 – can be used. See also Table 60.

Device ID objects	IC	OBIS code					
		A	B	C	D	E	F
Device ID 1…10 object (manufacturing number)	1, Data	0	*b*	96	1	0…9	255
Device ID-s object	7, Profile generic	0	*b*	96	1	255	255
Device ID-s object	61, Register table	0	*b*	96	1	255	255

6.2.43 Metering point ID objects

One object is available to store a media type independent metering point ID. It is held by the *value* attribute of a "Data" object, with data type *double-long-unsigned*, *octet-string*, *visible-string, utf8-string*, *unsigned, long-unsigned.*

© Copyright 1997-2020 DLMS User Association

Metering point ID objects	IC	OBIS code					
		A	B	C	D	E	F
Metering point ID	1, Data	0	*b*	96	1	10	255

6.2.44 Parameter changes and calibration objects

A set of simple COSEM objects describes the history of the configuration of the device. All values are modelled by instances of the IC "Data".

Parameter changes objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes seeTable 60.	1, Data	0	*b*	96	2	*e*	255

6.2.45 I/O control signal objects

A series of objects are available to define and control the status of I/O lines of the physical metering equipment.

The status is held by the *value* attribute of a “Data” object, with data type *octet-string, boolean* or *bit-string.* Alternatively, the status is held by a “Status mapping” object, see 4.3.9, which holds both the status word and the mapping of its bits to the reference table. If there are several I/O control status objects used, it is allowed to combine them into an instance of the IC “Profile generic” or “Register table”, using the OBIS code of the global state of I/O control signals object. See also Table 60.

I/O control signal objects	IC	OBIS code					
		A	B	C	D	E	F
I/O control signal objects, contents manufacturer specific	1, Data	0	*b*	96	3	0…4	255
I/O control signal objects, contents mapped to a reference table	63, Status mapping	0	*b*	96	3	0…4	255
I/O control signal objects, global	7, Profile generic or 61, Register table	0	*b*	96	3	0	255

6.2.46 Disconnect control objects (class_id = 70)

Instances of the IC “Disconnect control” – see 4.5.8 – manage internal or external disconnect units (e.g. electricity breaker, gas valve) in order to connect or disconnect – partly or entirely – the premises of the consumer to / from the supply. See also 6.2.22.

Disconnect control objects	IC	OBIS code					
		A	B	C	D	E	F
Disconnect control	70, Disconnect control	0	b	96	3	10	255

6.2.47 Arbitrator objects (class_id = 68)

Instances of the IC “Arbitrator” – see 4.5.12 – are used

Arbitrator Objects	IC	OBIS code					
		A	B	C	D	E	F
General-purpose Arbitrator	68, Arbitrator	0	*b*	96	3	*20*…	255

© Copyright 1997-2020 DLMS User Association

						29	

6.2.48 Status of internal control signals objects

A series of objects are available to hold the status of internal control signals.

The status carries binary information from a bitmap, and it shall be held by the *value* attribute of a “Data” object, with data type *boolean, bit-string, unsigned, long-unsigned, double-long-unsigned, long64-unsigned or octet-string*. Alternatively, the status is held by a “Status mapping” object, see 4.3.9, which holds both the status word and the mapping of its bits to the reference table. If there are several status of internal control signals objects used, it is allowed to combine them into an instance of the IC “Profile generic” or “Register table”, using the OBIS code of the global “Internal control signals” object. See also Table 60.

Internal control signals objects	IC	OBIS code					
		A	B	C	D	E	F
Internal control signals, contents manufacturer specific	1, Data	0	*b*	96	4	0...4	255
Internal control signals, contents mapped to a reference table	63, Status mapping	0	*b*	96	4	0...4	255
Internal control signals, global	7, Profile generic or 61, Register table	0	*b*	96	4	0	255

6.2.49 Internal operating status objects

A series of objects are available to hold internal operating statuses.

The status carries binary information from a bitmap, and it shall be held by the value attribute of a “Data” object, with data type *boolean*, *bit-string, unsigned, long-unsigned, double-long-unsigned, long64-unsigned or octet-string*. Alternatively, the status is held by a “Status mapping” object, see 4.3.9, which holds both the status word and the mapping of its bits to the reference table. If there are several status of internal control signals objects used, it is allowed to combine them into an instance of the IC “Profile generic” or “Register table”, using the OBIS code of the global ”Internal operating status” object. See also Table 60.

Internal operating status objects	IC	OBIS code					
		A	B	C	D	E	F
Internal operating status objects, contents manufacturer specific	1, Data	0	*b*	96	5	0...4	255
Internal operating status objects, contents mapped to a reference table	63, Status mapping	0	*b*	96	5	0...4	255
Internal operating status objects, global	7, Profile generic or 61, Register table	0	*b*	96	5	0	255

Internal operating status objects can also be related to an energy type. See 6.3.7.

6.2.50 Battery entries objects

A series of objects are available for holding information relative to the battery of the device. These objects are instances of IC “Data”, “Register” or “Extended register” as appropriate.

© Copyright 1997-2020 DLMS User Association

Battery entries objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60	1, Data, 3, Register or 4, Extended register	0	*b*	96	6	0…6	255

6.2.51 Power failure monitoring objects

A series of objects are available for power failure monitoring:

- For simple power failure monitoring, it is possible to count the number of power failure events affecting all three phases, one of the three phases, any of the phases, and the auxiliary supply;
- For advanced power failure monitoring, it is possible to define a time threshold to make a distinction between short and long power failure events. It is possible to count the number of such long power failure events separately from the short ones, as well as to store their time of occurrence and duration (time from power down to power up) in all three phases, in one of the three phases and in any of the phases;
- The number of power failure events objects are represented by instances of the IC “Data”, “Register” or “Extended register” with data types *unsigned*, *long-unsigned*, *double-long-unsigned* or *long64-unsigned*;
- The power failure duration, time and time threshold data are represented by instances of the IC “Data”, “Register” or “Extended register” with appropriate data types;
- If power failure duration objects are represented by instances of the IC “Data”, then the default scaler shall be 0, and the default unit shall be the second.

Power failure monitoring objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register or 4, Extended register	0	*b*	96	7	0… 21	255

These objects may be collected in a “Power failure event log” object. See Table 75.

6.2.52 Operating time objects

A series of objects are available for holding the cumulated operating time and the various tariff registers of the device. These objects are instances of the IC “Data”, ”Register” or “Extended register”. The data type shall be *unsigned*, *long-unsigned* or *double-long-unsigned* with appropriate scaler and unit. If the IC “Data” is used, the unit shall be the second by default.

Operating time objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register or 4, Extended register	0	*b*	96	8	0… 63	255

6.2.53 Environment related parameters objects

A series of objects are available to store environmental related parameters. They are held by the *value* attribute of instances of the “Register” IC or “Extended Register” IC, with appropriate data types.

© Copyright 1997-2020 DLMS User Association

Environment related parameters objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	3, Register or 4, Extended register	0	*b*	96	9	0…2	255

6.2.54 Status register objects

A series of objects are available to hold statuses that can be captured in load profiles. See also Table 60.

Status register objects	IC	OBIS code					
		A	B	C	D	E	F
Status register, contents manufacturer specific	1, Data	0	*b*	96	10	1… 10	255
Status register, contents mapped to reference table	63, Status mapping	0	*b*	96	10	1… 10	255

The status register is held by the value attribute of a “Data” object, with data type *bit-string, unsigned, long-unsigned, double-long-unsigned, long64-unsigned or octet-string.* It carries binary information from a bitmap. It’s contents is not specified.

Alternatively, the status register may be held by the *status_word* attribute of a “Status mapping” object, see 4.3.9. The *mapping_table* attribute holds mapping information between the bits of the status word and entries of a reference table.

6.2.55 Event code objects

In the meter or in its environment, various events may be generated. A series of objects are available to hold an identifier of a most recent event (event code). Different instances of event code objects may be captured in different instances of event logs; see 6.2.66.

NOTE The definition of event identifiers is out of the Scope of this document.

Event code objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	11	0… 99	255

Events may also set flags in error registers and alarm registers. See also 6.2.64.

6.2.56 Communication port log parameter objects

A series of objects are available to hold various communication log parameters. They are represented by instances of IC “Data”, “Register” or “Extended register”.

Communication port log parameter objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	12	0…6	255

6.2.57 Consumer message objects

A series of objects are available to store information sent to the energy end-user. The information may appear on the display of the meter and / or on a consumer information port.

© Copyright 1997-2020 DLMS User Association

Consumer message objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	13	0, 1	255

6.2.58 Currently active tariff objects

A series of objects are available to hold the identifier of the currently active tariff. They carry the same information as the *active_mask* attribute of the corresponding “Register activation” object.

Currently active tariff objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	14	0… 15	255

6.2.59 Event counter objects

A series of objects are available to count events. The number of the events is held by the value attribute.

Event counter objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes seeTable 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	15	0… 99	255

6.2.60 Profile entry digital signature objects

Instances of “Data”, “Register” or “Extended register” objects hold digital signatures of “Profile generic” object buffer entries. If the *capture_object* attribute of a “Profile generic” object contains a reference to a “Profile entry digital signature” object, then the digital signature is calculated and captured together with the other attribute values.

The security context is determined by the “Security setup” object which is which is visible in the same AA (object_list).

NOTE The digital signature may be generated when the entry is captured or “on the fly”, when an entry is accessed.

Profile entry digital signature objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	16	0…9	255

6.2.61 Profile entry counter objects

A series of objects are available to keep track of the total number of entries captured in a “Profile generic” object. These objects are instances of the IC “Data”, Register” or “Extended register”. The data type shall be *unsigned*, *long-unsigned, double-long-unsigned or long64-unsigned* with appropriate scaler and unit. If the *capture_objects* attribute of a “Profile generic” object contains a reference to a “Profile entry counter” object, then the counter is monotonically increased by one and captured together with the other attribute values when the *capture* method is invoked or when auto capture takes place.

© Copyright 1997-2020 DLMS User Association

If a “Profile entry counter” object reaches its maximum value, it will wrap-around and the count will re-start at zero.

Profile entry counter objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	17	0… 127	255

6.2.62 Meter tamper event related objects

A series of objects are available to register characteristics of various meter tamper events. These objects are instances of the IC “Data”, Register” or “Extended register”. The data type shall be *unsigned*, *long-unsigned or double-long-unsigned* with appropriate scaler and unit. For time stamps, the data type shall be *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1.

- Meter open events are related to cases when the meter case is open;
- Terminal cover open events are related to cases when a terminal cover is removed (open);
- Tilt events are related to cases when the meter is not in its normal operation position;
- Strong DC magnetic field events are related to cases when the presence of a strong DC magnetic field is detected;
- Metrology tamper events are related to cases when an anomaly in the operation of the metrology is detected due to a perceived tamper;
- Communication tamper events are related to cases when an anomaly in the operation of the communication interfaces is detected due to a perceived tamper.

The method of detecting the various tampers is out of the Scope of this document.

Meter tamper event related objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 60.	1, Data, 3, Register, or 4, Extended register	0	*b*	96	20	*e*	255

6.2.63 Error register objects

A series of objects are used to communicate error indications of the device. The different error registers are held by the *value* attribute of “Data” objects, with data type or *bit-string*, *octet-string, unsigned, long-unsigned, double-long-unsigned* or *long64-unsigned.*

The individual bits of the error register may be set and cleared by a pre-defined selection of events – see 6.2.51. Depending on the type of the error, some errors may clear themselves when the reason setting the error flag disappears.

If more than one of those objects is used, it is allowed to combine them into one instance of the IC "Profile generic". In this case, the captured objects are the *value* attributes “Data” objects, the capture period is 1 to have just actual values, the sort method is FIFO, the profile entries are limited to 1. Alternatively, an instance of the IC “Register table” can be used.

Error register objects can also be related to an energy type and to a channel. See 7.4.2, 7.5.5.2, 7.6.4.2, 7.7.4.2,7.8.6.2, 7.9.4.2.

© Copyright 1997-2020 DLMS User Association

Error register objects	IC	OBIS code					
		A	B	C	D	E	F
Error register 1…10 object	1, Data	0	b	97	97	0…9	255
Error profile object	7, Profile generic	0	b	97	97	255	255
Error table object	61, Register table	0	b	97	97	255	255

6.2.64 Alarm register, Alarm filter and Alarm descriptor objects

A number of objects are available to hold alarm registers. The different alarm registers are held by the value attribute of “Data” objects, with data type *bit-string*, *octet-string*, *unsigned*, *long-unsigned*, *double-long-unsigned* or *long64-unsigned*. When selected events occur, they set the corresponding flag and the device may raise an alarm. Depending on the type of alarm, some alarms may clear themselves when the reason setting the alarm flag disappears.

If more than one of those objects is used, it is also allowed to combine them into one instance of the IC "Profile generic". In this case, the captured objects are the *value* attributes of “Data” objects, the capture period is 1 to have just actual values, the sort method is FIFO, and the profile entries are limited to 1. Alternatively, an instance of the IC “Register table” can be used.

Alarm filter objects are available to define if an event is to be handled as an alarm when it appears. The different alarm filters are held by the value attribute of “Data” objects, with data type *bit-string, octet-string, unsigned, long-unsigned, double-long-unsigned* or *long64-unsigned*. The bit mask has the same structure as the corresponding alarm register object. If a bit in the alarm filter is set, then the corresponding alarm is enabled, otherwise it is disabled. *Alarm filter* objects act on *Alarm register* and *Alarm descriptor* objects the same way.

Alarm descriptor objects are available to persistently hold the occurrence of alarms. The different alarm descriptors are of the same type as the corresponding *Alarm register*. When a selected event occurs, the corresponding flag is set in the *Alarm register* as well as in the *Alarm descriptor* objects. An alarm descriptor flag remains set even if the corresponding alarm condition has disappeared. Alarm descriptor flags do not reset themselves; they can be reset by writing the value attribute only.

NOTE The alarm conditions, the structure of the *Alarm register / Alarm filter / Alarm descriptor* objects are subject to a project specific companion specification.

Alarm register, Alarm filter and Alarm descriptor objects	IC	OBIS code					
		A	B	C	D	E	F
Alarm register objects 1…10	1, Data	0	b	97	98	0…9	255
Alarm register profile object	7, Profile generic	0	b	97	98	255	255
Alarm register table object	61, Register table	0	b	97	98	255	255
Alarm filter objects 1…10	1, Data	0	b	97	98	10…19	255
Alarm descriptor objects 1…10	1, Data	0	b	97	98	20…29	255

6.2.65 General list objects

Instances of the IC “Profile generic” are used to model lists of any kind of data, for example measurement values, constants, statuses, events. They are modelled by “Profile generic” objects. One standard object per billing period scheme is defined.

List objects may be also related to an energy type and to a channel.

© Copyright 1997-2020 DLMS User Association

General list objects	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see 7.4.3.	7, Profile generic	0	*b*	98	*d*	*e*	255[a]
[a] F = 255 means a wildcard here. See 7.11.3.							

6.2.66 Event log objects (class_id = 7)

Instances of the IC “Profile generic” are used to store Event logs. Event logs may be also media related. In this case, the value of value group A shall be the relevant media identifier. See also 7.4.5, 7.5.5.4, 7.8.6.4.

Event log objects	IC	OBIS code					
		A	B	C	D	E	F
Event log	7, Profile generic	*a*	*b*	99	98	*e*	255 [a]
[a] F = 255 means a wildcard here. See 7.11.3.							
NOTE 1 Event logs may capture for example the time of occurrence of the event, the event code and other relevant data.							
NOTE 2 Project specific companion specifications may specify a more precise meaning of the instances of the different event logs, i.e. the data captured and the number of events captured.							

6.2.67 Inactive objects

Inactive objects are objects, which are present in the meter, but which do not have an assigned functionality. Inactive instances of any IC may be present. See also 7.3.3.2.

Inactive objects	IC	OBIS code					
		A	B	C	D	E	F
Inactive objects	Any	0	*b*	127	0	*e*	255

© Copyright 1997-2020 DLMS User Association

6.3 Electricity related COSEM objects

6.3.1 Value group D definitions

The different ways of processing measurement values as defined by value group D – see 7.5.2.1 – are modelled as shown in Table 42.

Table 42 – Representation of various values by appropriate ICs

Type of value	Represented by
cumulative values	Instances of IC "Register" or “Extended register”.
maximum and minimum values	Instances of IC "Profile generic" with sorting method *maximum* or *minimum*, depth according to implementation and captured objects according to implementation. A single maximum value or minimum value can alternatively be represented by an instance of the IC “Register” or "Extended register”.
current and last average values	Instances of IC “Demand register”. The logical name is the OBIS code of the current average value (D = 4, 14 or 24). For display purposes: Instances of IC “Register” or “Demand register”. The logical name is the OBIS code of current average (D = 4, 14 or 24) or last average (D= 5, 15 or 25) as appropriate.
instantaneous values	Instances of IC "Register”.
time integral values	Instances of IC "Register” or “Extended register”.
occurrence counters	Instances of IC “Data” or "Register".
contracted values	Instances of IC "Register" or “Extended register”.
Under/Over limit thresholds	Instances of IC "Register" or “Extended register”.
Over/Under limit occurrence counters	Instances of IC "Register" or “Extended register”.
Under/Over limit durations	Instances of IC "Register" or “Extended register”.
Over/Under limit magnitudes	Instances of IC "Register" or “Extended register”.

6.3.2 ID numbers – Electricity

The different electricity ID numbers are held by instances of the IC "Data”, with data type *unsigned, long-unsigned, double-long-unsigned, octet-string* or *visible-string.*

If more than one of those is used, it is allowed to combine them into a "Profile generic" object. In this case, the captured objects are *value* attributes of electricity ID “Data” objects, the capture period is 1 to have just actual values, the sort method is FIFO, the profile entries are limited to 1. Alternatively, a “Register table” object can be used. See also Table 72.

ID objects – Electricity	IC	OBIS code					
		A	B	C	D	E	F
Electricity ID 1…10 object	1, Data	1	*b*	0	0	0…9	255
Electricity ID-s object	7, Profile generic	1	*b*	0	0	255	255
Electricity ID-s object	61, Register table	1	*b*	0	0	255	255

6.3.3 Billing period values / reset counter entries

These values are represented by instances of the IC "Data”.

For billing period / reset counters and for number of available billing periods the data type shall be *unsigned*, *long-unsigned* or *double-long-unsigned*. For time stamps of billing periods, the

© Copyright 1997-2020 DLMS User Association

data type shall be *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1.

These objects may be related to channels.

When the values of historical periods are represented by “Profile generic” objects, the time stamp of the billing period objects shall be part of the captured objects.

Billing period values / reset counter entries	IC	OBIS code					
		A	B	C	D	E	F
For item names and OBIS codes see Table 72.	1, Data	1	*b*	0	1	*e*	255

6.3.4 Other electricity related general purpose objects

Program entries shall be represented by instances of the IC "Data" with data type *unsigned, long-unsigned, octet-string* or *visible-string.* For “Meter connection diagram ID” objects data type *enumerated* can be used as well. Program entries can also be related to a channel.

Output pulse constant, reading factor, CT/VT ratio, nominal value, input pulse constant, transformer and line loss coefficient values shall be represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed.

Measurement period, recording interval and billing period duration values shall be represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *unsigned, long-unsigned* or *double-long-unsigned.* The default unit is the second.

Time entry values shall be represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *octet-string*, formatted as *date-time* in 4.1.6.1. The data types *unsigned, integer, long-unsigned double-long-unsigned* or *long64-unsigned* can also be used where appropriate.

The *Clock synchronization method* shall be represented by an instance of an IC “Data” with data type *enum*.

Synchronization method enum:
(0) no synchronization,
(1) adjust to quarter,
(2) adjust to measuring period,
(3) adjust to minute,
(4) reserved,
(5) adjust to preset time,
(6) shift time

For the detailed OBIS codes, see Table 72.

© Copyright 1997-2020 DLMS User Association

Electricity related general purpose objects	IC	OBIS code					
		A	B	C	D	E	F
Program entries	1, Data	1	*b*	0	2	*e*	255
Output pulse values or constants	1, Data 3, Register 4, Extended Register	1	*b*	0	3	*e*	255
Reading factor and CT/VT ratio		1	*b*	0	4	*e*	255
Nominal values	3, Register 4, Extended Register	1	*b*	0	6	*e*	255
Input pulse values or constants	1, Data 3, Register 4, Extended Register	1	*b*	0	7	*e*	255
Measurement period- / recording interval- / billing period duration	1, Data 3, Register 4, Extended Register	1	*b*	0	8	*e*	255
Time entries		1	*b*	0	9	*e*	255
Transformer and line loss coefficients	3, Register 4, Extended Register	1	*b*	0	10	*e*	255

6.3.5 Measurement algorithm

These values are represented by instances of the IC “Data”, with data type *enum*.

Measurement algorithm objects	IC	OBIS code					
		A	B	C	D	E	F
Measuring algorithm for active power	1, Data	1	*b*	0	11	1	255
Measurement algorithm for active energy		1	*b*	0	11	2	255
Measurement algorithm for reactive power		1	*b*	0	11	3	255
Measurement algorithm for reactive energy		1	*b*	0	11	4	255
Measurement algorithm for apparent power		1	*b*	0	11	5	255
Measurement algorithm for apparent energy		1	*b*	0	11	6	255
Measurement algorithm for power factor calculation		1	*b*	0	11	7	255

The enumerated values are specified in Table 43:

Table 43 – Measuring algorithms – enumerated values

Measuring algorithm for active power and energy	
(0)	not specified
(1)	only the fundamentals of voltage and current are used
(2)	all harmonics of voltage and current are used
(3)	only the DC part of voltage and current is used
(4)	all harmonics and the DC part of voltage and current are used
Measuring algorithm for reactive power and energy	
(0)	not specified
(1)	(sum of) reactive power of each phase, calculated from the fundamental of the per phase voltage and the per phase current

© Copyright 1997-2020 DLMS User Association

(2)	polyphase reactive power calculated from polyphase apparent power and polyphase active power
(3)	(sum of) reactive power calculated from per phase apparent power and per phase active power
Measurement algorithm for apparent power and energy	
(0)	not specified
(1)	$S = U \times I$, with voltage: only fundamental, and current: only fundamental
(2)	$S = U \times I$, with voltage: only fundamental, and current: all harmonics
(3)	$S = U \times I$, with voltage: only fundamental, and current: all harmonics and DC part
(4)	$S = U \times I$, with voltage: all harmonics, and current: only fundamental
(5)	$S = U \times I$, with voltage: all harmonics, and current: all harmonics
(6)	$S = U \times I$, with voltage: all harmonics, and current: all harmonics and DC part
(7)	$S = U \times I$, with voltage: all harmonics and DC part, and current: only fundamental
(8)	$S = U \times I$, with voltage: all harmonics and DC part, and current: all harmonics
(9)	$S = U \times I$, with voltage: all harmonics and DC part, and current: all harmonics and DC part
(10)	$S = \sqrt{P^2 + Q^2}$, with P: only fundamental in U and I, and Q: only fundamental in U and I, where P and Q are polyphase quantities
(11)	$S = \sqrt{P^2 + Q^2}$, with P: all harmonics in U and I, and Q: only fundamental in U and I where P and Q are polyphase quantities
(12)	$S = \sqrt{P^2 + Q^2}$, with P: all harmonics and DC part in U and I, and Q: only fundamental in U and I where P and Q are polyphase quantities
(13)	$S = \Sigma\sqrt{P^2 + Q^2}$, with P: only fundamental in U and I, and Q: only fundamental in U and I where P and Q are single phase quantities
(14)	$S = \Sigma\sqrt{P^2 + Q^2}$, with P: all harmonics in U and I, and Q: only fundamental in U and I where P and Q are single phase quantities
(15)	$S = \Sigma\sqrt{P^2 + Q^2}$, with P: all harmonics and DC part in U and I, and Q: only fundamental in U and I where P and Q are single-phase quantities
Measurement algorithm for power factor calculation	
(0)	not specified
(1)	displacement power factor: the displacement between fundamental voltage and current vectors, which can be calculated directly from fundamental active power and apparent power, or another appropriate algorithm,
(2)	true power factor, the power factor produced by the voltage and current, including their harmonics . It may be calculated from apparent power and active power, including the harmonics.

6.3.6 Metering point ID (electricity related)

A series of objects are available to hold electricity related metering point IDs. They are held by the *value* attribute of “Data” objects, with data type *unsigned, long-unsigned, double-long unsigned*, *octet-string* or *visible-string*. If more than one of those is used, it is allowed to combine them into one instance of the IC "Profile generic". In this case, the captured objects are the *value* attributes of the electricity related metering point ID “Data” objects, the capture period is

© Copyright 1997-2020 DLMS User Association

1 to have just actual values, the sort method is FIFO, the profile entries are limited to 1. Alternatively, an instance of the IC “Register table” can be used. For detailed OBIS codes, see Table 72.

Metering point ID objects	IC	OBIS code					
		A	B	C	D	E	F
Metering point ID 1…10 (electricity related)	1, Data [a]	1	*b*	96	1	0…9	255
Metering point ID-s object	7, Profile generic	1	*b*	96	1	255	255
Metering point ID-s object	64, Register table	1	*b*	96	1	255	255
[a] If the IC “Data” is not available, “Register” (with scaler = 0, unit = 255) may be used.							

6.3.7 Electricity related status objects

A number of electricity related objects are available to hold information about the internal operating status, the starting of the meter and the status of voltage and current circuits.

The status is held by the value attribute of a “Data” object, with data type *boolean, bit-string, unsigned, long-unsigned, double-long-unsigned, long64-unsigned or octet-string.*

Alternatively, the status is held by a “Status mapping” object, which holds both the status word and the mapping of its bits to the reference table.

If there are several electricity related internal operating status objects used, it is allowed to combine them into an instance of the IC “Profile generic” or “Register table”, using the OBIS code of the global internal operating status. For detailed OBIS codes, see Table 72.

Electricity related status objects	IC	OBIS code					
		A	B	C	D	E	F
Internal operating status signals, electricity related, contents manufacturer specific	1, Data	1	*b*	96	5	0…5	255
Internal operating status signals, electricity related, contents mapped to reference table	63, Status mapping	1	*b*	96	5	0…5	255
Electricity related status data, contents manufacturer specific	1, Data	1	*b*	96	10	0…3	255
Electricity related status data, contents mapped to reference table	63, Status mapping	1	*b*	96	10	0…3	255

6.3.8 List objects – Electricity (class_id = 7)

These COSEM objects are used to model lists of any kind of data, for example measurement values, constants, statuses, events. They are modelled by “Profile generic” objects.

One standard object per billing period scheme is defined. See also 7.5.5.3.

List objects – Electricity	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 74.	7, Profile generic	1	*b*	98	*d*	*e*	255[a]
[a] F = 255 means a wildcard here. See 7.11.3.							

© Copyright 1997-2020 DLMS User Association

6.3.9 Threshold values

A number of objects are available for representing thresholds for instantaneous quantities. The thresholds may be “under limit”, “over limit”, “missing” and “time thresholds”. Time thresholds are used to detect “under limit”, “over limit” and “missing” conditions.

Objects are also available to represent the number of occurrences when these thresholds are exceeded, the duration of such events and the magnitude of the quantity during such events.

These values are represented by instances of IC “Data”, “Register” or “Extended register”.

All these quantities may be related to tariffs.

As defined in 7.5.4.2, value group F may be used to identify multiple thresholds.

For OBIS codes, see Table 44 below and Table 66.

Table 44 – Threshold objects, electricity

Threshold objects	IC	OBIS code					
		A	B	C	D	E	F
Threshold objects for instantaneous values	1, Data, 3, Register, 4, Extended register	1	*b*	1...10, 13, 14, 16...20, 21...30, 33, 34, 36...40, 41...50, 53, 54, 56...60, 61...70, 73, 74, 76...80, 82, 84...89	31...34, 35...38, 39...42, 43...45	0...63	0...99, 255
Threshold objects for harmonics of voltage, current and active power		1	*b*	11, 12, 15, 31, 32, 35, 51, 52, 55, 71, 72, 75, 90...92		0...120, 124... 127	

For monitoring the supply voltage, a more sophisticated functionality is also available, that allows counting the number of occurrences classified by the duration of the event and the depth of the voltage dip. For OBIS codes, see 7.5.3.6, Table 71.

6.3.10 Register monitor objects (class_id = 21)

Further to 6.2.13, the following definitions apply:

- for monitoring thresholds of instantaneous values, the logical name of the “Register monitor” object may be the OBIS identifier of the threshold;
- for monitoring current average and last average values, the logical name of the “Register monitor” object may be the OBIS identifier of the demand value monitored.

© Copyright 1997-2020 DLMS User Association

See Table 45.

Table 45 – Register monitor objects, electricity

Register monitor objects	IC	OBIS code					
		A	B	C	D	E	F
Instantaneous values, under limit / over limit / missing	21, Register monitor	1	*b*	*c1*	31, 35, 39	0-63	0-99, 255
		1	*b*	*c2*		0-120, 124-127	
Current average and last average values		1	*b*	*c1*	4, 5, 14, 15, 24, 25	0-63	
		1	*b*	*c2*		0-120, 124-127	
c1 = 1-10, 13,14, 16-20, 21-30, 33,34, 36-40, 41-50, 53,54, 56-60, 61-70, 73,74, 76-80, 82, 84-89. c2 = 11, 12, 15, 31, 32, 35, 51, 52, 55, 71, 72, 75, 90-92, 100-107.							

For the use of value group D, see Table 66.

For the use of value group E, see Table 67 and Table 68.

For the use of value group F, see 7.5.4.2.

© Copyright 1997-2020 DLMS User Association

6.4 HCA related COSEM objects

6.4.1 General

The use of interface classes to represent various data is described in the following tables.

Grouping the data by major categories supports the link between the data and the OBIS value groups associated with them.

6.4.2 ID numbers – HCA

The different HCA ID numbers are held by instances of the IC "Data", with data type *unsigned, long-unsigned, double-long-unsigned, long64-unsigned, octet-string* or *visible-string*.

If more than one of those is used, it is allowed to combine them into a "Profile generic" object. In this case, the captured objects are *value* attributes of the HCA ID data objects, the capture period is 1 to have just actual values, the sort method is FIFO, the profile entries are limited to 1. Alternatively, an instance of the IC "Register table" can be used. See also Table 80.

HCA ID	3IC	OBIS code					
		A	B	C	D	E	F
HCA ID 1…10 object	1, Data	4	*b*	0	0	0…9	255
HCA ID-s object	7, Profile generic	4	*b*	0	0	255	255
HCA ID-s object	61, Register table	4	*b*	0	0	255	255

6.4.3 Billing period values / reset counter entries - HCA

These values are represented by instances of the IC "Data".

For billing period / reset counters and for number of available billing periods the data type shall be *unsigned*, *long-unsigned* or *double-long-unsigned*. For time stamps of billing periods, the data type shall be *double-long-unsigned* (in the case of UNIX time), *octet-string, date* or *date-time* formatted as specified in 4.1.6.1. These objects may be related to channels.

When the values of historical periods are represented by "Profile generic" objects, the time stamp of the billing period objects shall be part of the captured objects.

Billing period values / reset counter entries objects "Storage information"	IC	OBIS code					
		A	B	C	D	E	F
For item names and OBIS codes see See Table 80.	1, Data	4	*b*	0	1	1,2, 10, 11	255

© Copyright 1997-2020 DLMS User Association

6.4.4 General purpose objects – HCA

The use of ICs shall be as specified below:

- *Configuration entries* are represented by instances of the IC "Data" with data type *unsigned, long-unsigned* or *octet-string* or *enumerated*. Configuration entries can also be related to a channel;
- *Device measuring principle* values are represented by instances of the IC “Data” with data type *unsigned* or *enumerated* for the *value* attribute;

Enum	Value
(0)	Single sensor
(1)	Single sensor + start sensor
(2)	Dual sensor
(3)	Triple sensor
Other	reserved

- *Conversion factor* values are represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- *Threshold values* are represented by instances of the IC "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- *Period values* are represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *unsigned*, *long-unsigned* or *double-long-unsigned*;
- *Time entry* values are represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *double-long-unsigned* (in the case of UNIX time), *octet-string, date* or *date-time* formatted as specified in 4.1.6.1.

General purpose objects – HCA related [a]	IC	OBIS code					
		A	B	C	D	E	F
Configuration objects	1, Data	4	*b*	0	2	0-2	255
Device measuring principle	1, Data	4	*b*	0	2	3	255
Conversion factors	1, Data or 3, Register or 4, Extended register	4	*b*	0	4	0-6	255
Threshold values	3, Register or 4, Extended register	4	*b*	0	5	10-11	255
Period information	1, Data or 3, Register or 4, Extended register	4	*b*	0	8	0,4,6	255
Time entries	1, Data or 3, Register or 4, Extended register	4	*b*	0	9	1-3	255
[a] For item names and OBIS codes see Table 80.							

6.4.5 Measured Values – HCA

6.4.5.1 Consumption – HCA

The use of ICs shall be as specified below:

© Copyright 1997-2020 DLMS User Association

Consumption values are represented by instances of the IC "Register", “Extended register” or “Profile generic” with data types *double-long,double-long-unsigned, octet-string, visible-string, integer, long,unsigned, long-unsigned, long64, long64-unsigned, float32* or *float64*.

Consumption	IC	OBIS code					
		A	B	C [a]	D [b]	E [c]	F
Integral value, current	3, Register or 4, Extended register	4	*b*	1, 2	0	0	255
Integral value over measurement periods (periodical value)					1		0…99 101… 125, 255
Integral value relative to billing periods (Set date value, Billing date value)					2,3		
Minimum, Maximum of integral value over billing periods	4, Extended register or 7, Profile generic				4,5		
Integral test value	3, Register, 4, Extended register, or 7, Profile generic				6		255

[a] See Table 77– Value group C codes - HCA

[b] See Table 78– Value group D codes – HCA

[c] No further classification in value group E is made. Therefore, E shall be 0.

6.4.5.2 Temperature Values – HCA

The use of ICs shall be as specified below:

The objects are used to represent temperature information as measurand values.

Temperature values are represented by instances of the IC "Register", “Extended register” or “Profile generic”.

Temperature	IC	OBIS code					
		A	B	C [a]	D [b]	E [c]	F
Temperature value, current	3, Register or 4, Extended register	4	*b*	3..7	0	255	255
Minimum, Maximum of temperature	4, Extended register, or 7, Profile generic				4,5		
Temperature test value	3, Register, 4, Extended register, or 7, Profile generic				6		

[a] See Table 77– Value group C codes – HCA

[b] See Table 78– Value group D codes – HCA

[c] Value group E not used. Therefore, E shall be 255.

© Copyright 1997-2020 DLMS User Association

6.4.6 Error register objects – HCA

A series of objects are used to communicate error indications of the device. See also Table 84.

The different error registers are held by the *value* attribute of “Data”, “Register” or “Extended register“ objects, with data type *bit-string*, *octet-string, unsigned, long-unsigned, double-long-unsigned* or *long64-unsigned.*

Error register objects - HCA	IC	OBIS code					
		A	B	C	D	E	F
Error register objects	1, Data or 3, Register or 4, Extended register	4	*b*	97	97	e	255

6.4.7 List objects – HCA

The use of ICs shall be as specified below:

These COSEM objects are used to model lists of any kind of data, for example measurement values, constants, statuses, events. They are modelled by “Profile generic” objects. Examples are present in 7.6.5.3.

List objects – HCA	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see XX	7, Profile generic	4	*b*	98	*d*	*e*	255[a]
[a] F = 255 means a wildcard here. See 7.6.							

6.4.8 Data profile objects – HCA

The use of ICs shall be as specified below:

HCA related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data. See also 7.6.4.4.

Data profile objects - HCA	IC	OBIS code					
		A	B	C	D	E	F
Data profile objects	7, Profile generic	4	*b*	99	1	e	255

© Copyright 1997-2020 DLMS User Association

6.5 Thermal energy meter related COSEM objects

6.5.1 General

The use of interface classes to represent various data is described in the following tables.

Grouping the data by major categories supports the link between the data and the OBIS value groups associated with them.

6.5.2 ID numbers – Thermal energy meter

The different thermal energy meter ID numbers are held by instances of the IC "Data", with data type *unsigned, long-unsigned, double-long-unsigned, long64-unsigned, octet-string* or *visible-string*.

If more than one of those is used, it is allowed to combine them into a "Profile generic" object. In this case, the captured objects are *value* attributes of the thermal meter ID data objects, the capture period is 1 to have just actual values, the sort method is FIFO, the profile entries are limited to 1. Alternatively, an instance of the IC "Register table" can be used. See also Table 88

Thermal energy meter ID	IC	OBIS code					
		A	B	C	D	E	F
Thermal energy meter ID 1…10 object	1, Data	5/6	*b*	0	0	0…9	255
Thermal energy meter ID-s object	7, Profile generic	5/6	*b*	0	0	255	255
Thermal energy meter ID-s object	61, Register table	5/6	*b*	0	0	255	255

6.5.3 Billing period values / reset counter entries - Thermal energy meter

These values are represented by instances of the IC "Data".

For billing period / reset counters and for number of available billing periods the data type shall be *unsigned*, *long-unsigned* or *double-long-unsigned*. For time stamps of billing periods, the data type shall be *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1.

These objects may be related to channels.

When the values of historical periods are represented by "Profile generic" objects, the time stamp of the billing period objects shall be part of the captured objects.

Billing period values / reset counter entries objects "Storage information"	IC	OBIS code					
		A	B	C	D	E	F
For item names and OBIS codes see Table 88.	1, Data	5/6	*b*	0	1	1,2, 10, 11	1,2, 255

© Copyright 1997-2020 DLMS User Association

6.5.4 General purpose objects – Thermal energy meter

The use of ICs shall be as specified below:

- *Configuration entries* are represented by instances of the IC "Data" with data type *unsigned, long-unsigned* or *octet-string*. Configuration entries can also be related to a channel;
- *Conversion factor* values are represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- *Threshold values* are represented by instances of the IC "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- *Timing information for averaging, measurement, billing periods and recording interval is* presented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *unsigned*, *long-unsigned* or *double-long-unsigned*;
- *Time entry* values are represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1.

General purpose objects – Thermal energy meter related [a]	IC	OBIS code					
		A	B	C	D	E	F
Configuration objects	1, Data	5/6	*b*	0	2	0-4, 10 -13	255
Conversion factors	1, Data	5/6	*b*	0	4	1-3	255
Threshold values	3, Register or 4, Extended register	5/6	*b*	0	5	1-9, 21-24	255
Timing information	1, Data	5/6	*b*	0	8	0-7, 11-14, 21-25, 31-34	255
Time entries	1, Data	5/6	*b*	0	9	1-3	255
[a] For item names and OBIS codes see Table 88.							

6.5.5 Measured values - Thermal energy meterConsumption – Thermal energy meter

The use of ICs shall be as specified below:

Consumption values are represented by instances of the IC "Register", “Extended register” or “Profile generic” with data types *double-long,double-long-unsigned, octet-string, visible-string, integer, long,unsigned, long-unsigned, long64, long64-unsigned, float32* or *float64*.

© Copyright 1997-2020 DLMS User Association

Consumption	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Energy, volume, mass values	3, Register or 4, Extended register, or 7, Profile generic	5/6	*b*	1...7	0...3, 7	0, 1...9	255
Integral value relative to billing periods	3, Register or 4, Extended register, or 7, Profile generic				3, 8, 9		0…99, 101...125, 255
Periodical value	3, Register or 4, Extended register, or 7, Profile generic				1,12,13		0…99, 101...125
Set date value	3, Register or 4, Extended register, or 7, Profile generic				2		
Minimum, Maximum of integral value over billing periods	3, Register or 4, Extended register, or 7, Profile generic				4, 5, 14, 15		
Integral test value	3, Register, 4, Extended register				6	255	255

[a] See Table 85– Value group C codes – thermal energy meters

[b] See Table 86– Value group D codes - thermal energy meters

[c] All other values reserved for further use (tariff rate E=0 means Total)

6.5.5.2 Monitoring values – Thermal energy meter

The use of ICs shall be as specified below:

The objects are to be used to represent information as monitored values.

Monitoring values are held by the *value* attribute of “Register” or “Extended register“ objects with data types *double-long,double-long-unsigned, octet-string, visible-string, integer, long,unsigned, long-unsigned, long64, long64-unsigned, float32* or *float64*.

© Copyright 1997-2020 DLMS User Association

Monitoring values	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Energy, volume, mass Maximum of integral value	4, Extended register, or 7, Profile generic	5/6	*b*	1...7	5,15	0, 1..9	0...99, 101...125, 255
Power or flow rate values	3, Register, or 4, Extended register, or 7, Profile generic			8,9	1, 4, 5, 12...15		
Temperature or pressure value, current	3, Register or 4, Extended register, or 7, Profile generic			10... 13	0		255
Minimum, Maximum of temperature or pressure	4, Extended register, or 7, Profile generic				4, 5, 14, 15		0...99, 101...125
Temperature or pressure test, instantaneous value	3, Register, or 4, Extended register				6,7		255
Average values of temperature or pressure	5, Demand register, or 7, Profile generic				10,11		
Occurrence counters	3, Register, or 4, Extended register, or 7, Profile generic			1...13	20, 22, 24		
Occurrence durations	3, Register, or 4, Extended register, or 7, Profile generic				21, 23, 25		

[a] See Table 85 – Value group C codes – Thermal energy meter
[b] See Table 86 – Value group D codes - Thermal energy meter
[c] All other values reserved for further use (tariff rate E=0 means Total)

6.5.6 Error register objects – Thermal energy meter

A series of objects are used to communicate error indications of the device. See also Table 89.

The different error registers are held by the value attribute of “Data”, “Register” or “Extended register“ objects, with data type *bit-string, octet-string, unsigned, long-unsigned, double-long-unsigned* or *long64-unsigned.*

– Error register objects – Thermal energy meters	IC	OBIS code					
		A	B	C	D	E	F
Overall error status-[a]	1, Data or 3, Register or 4, Extended register	5/6	*b*	97	97	0	255
Subsystem where error has occurred [b]		5/6	*b*	97	97	1	255
Duration of error condition [c]		5/6	*b*	97	97	2	255

[a]	This object is a 'mirror' of the object 0.x.97.97.0.
[b]	A further subdivision of error information.
[c]	This is the time during which the meter has not been able to calculate energy.

© Copyright 1997-2020 DLMS User Association

6.5.7 List objects – Thermal energy meter

These COSEM objects are used to model lists of any kind of data, for example measurement values, constants, statuses, events. They are modelled by “Profile generic” objects.

List objects – Thermal energy meter	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see XX	7, Profile generic	5/6	*b*	98	*d*	*e*	255[a]
[a] F = 255 means a wildcard here. Also note 7.7.4.3.							

6.5.8 Data profile objects – Thermal energy meter

Thermal energy meter related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data.

Data profile objects – Thermal energy meters	IC	OBIS code					
		A	B	C	D	E	F
Data profile objects. For names and OBIS codes see Table 92.	7, Profile generic	5/6	*b*	99	1	1-3	255
					2	1-3	
					3	1	
					99	e	

© Copyright 1997-2020 DLMS User Association

6.6 Gas related COSEM objects

NOTE The reader is advised to refer to 7.8.1, General introduction to gas measurement before reading this section.

6.6.1 General

The use of interface classes to represent various data is described in the following tables.

Grouping the data by major categories supports the link between the data and the OBIS value groups associated with them.

6.6.2 ID numbers – Gas

The different gas ID numbers are represented by instances of the IC "Data", with data type *unsigned, long-unsigned, double-long-unsigned, octet-string or visible-string.*

If more than one of those is used, it is allowed to combine them into a "Profile generic" object. In this case, the captured objects are *value* attributes of the gas ID data objects, the capture period is 1 to have just actual values, the sort method is FIFO, the profile entries are limited to 1. Alternatively, an instance of the IC "Register table" can be used. See also Table 104.

Gas ID	IC	OBIS code					
		A	B	C	D	E	F
Gas ID 1…10 object	1, Data	7	*b*	0	0	0…9	255
Gas ID-s object	7, Profile generic	7	*b*	0	0	255	255
Gas ID-s object	61, Register table	7	*b*	0	0	255	255

6.6.3 Billing period values / reset counter entries – Gas

Billing period values are represented by instances of the IC "Data".

For billing period / reset counters and for number of available billing periods the data type shall be *unsigned*, *long-unsigned* or *double-long-unsigned*. For time stamps of billing periods, the data type shall be *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1.

These objects may be related to channels.

When the values of historical periods are represented by "Profile generic" objects, the time stamp of the billing period objects shall be part of the captured objects.

Billing period values / reset counter entries objects	IC	OBIS code					
		A	B	C	D	E	F
For item names and OBIS codes see Table 104	1, Data	7	*b*	0	1	*e*	255

6.6.4 Other general purpose objects – Gas

Configuration entries are represented by instances of the IC "Data" with data type *unsigned, long-unsigned* or *octet-string* or *enumerated*. Configuration entries can also be related to a channel.

For the digital and analogue output configuration objects the enumerated values are specified in Table 46 below.

© Copyright 1997-2020 DLMS User Association

Table 46 – Digital / Analogue output configurations – enumerated values

Digital output configuration	
(0)	Output switched off (transistor blocking, "switch open")
(1)	Volume pulse output, logic active
(2)	Status output, logic active (signalling active => output switched on)
(3)	Time-synchronised output, logic active
(4)	Output switched on (transistor conducting, "switch closed")
(5)	Volume pulse output, logic inactive
(6)	Status output, logic inactive (signalling active => output switched off)
(7)	Time-synchronised output, logic inactive
(8)	High frequency pulse output
(9)	Event output, logic active (message active => output switched on)
(10)	Event output, logic inactive (message active => output switched off)
(99)	Continuous pulse (for test purposes)
Analogue output configuration	
(0)	inactive
(1)	1= 4–20 mA
(2)	2= 0–20 mA
(3)	3= Voltage output

The use of ICs shall be as specified below:

- Output pulse constant values are represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- Conversion factor values are represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- Threshold values are represented by instances of the IC "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- Nominal values of volume sensors are represented by instances of the IC "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- Input pulse constant values are represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- Interval and period values are represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *unsigned*, *long-unsigned* or *double-long-unsigned*;
- Time entry values are represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *octet-string*, formatted as *date-time* in 4.1.6.1. The data types *unsigned*, *integer*, *long-unsigned* or *double-long-unsigned* can also be used where appropriate;
- Station management information values are represented by instances of the IC "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- Gas parameters for volume conversion currently used in compressibility calculation values are represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed;
- Gas measuring method specific parameters, including gas parameters for Venturi measurement and for density measurement are represented by instances of the IC “Data”, "Register" or “Extended register”. For the *value* attribute, only simple data types are allowed.

© Copyright 1997-2020 DLMS User Association

- "Sensor manager" objects – see 4.5.11 – manage complex information related to sensors. See 4.5.11. This interface class will be used by the following (metrological) sensors e.g. in gas applications:
  - absolute temperature;
  - absolute pressure;
  - velocity of sound;
  - density of gas;
  - relative density;
  - gauge pressure;
  - differential pressure;
  - density of air.

  Value group E of the OBIS code can be mapped to the value group C codes, identifying the various physical quantities, see Table 94. The possible values are 41, 42, 44, 45...49.

  EXAMPLE Absolute pressure sensor manager object OBIS code 7.0.0.15.42.255.

  If there is more than one sensor for the same physical quantity, then value group B shall be used to identify the sensors.

For detailed item names and OBIS codes see Table 104.

Gas related general purpose objects	IC	OBIS code					
		A	B	C	D	E	F
Configuration objects	1, Data	7	*b*	0	2	*e*	255
Output pulse constants converted / unconverted	1, Data	7	*b*	0	3	*e*	255
Conversion factors	1, Data	7	*b*	0	4	*e*	255
Threshold values	3, Register or 4, Extended register	7	*b*	0	5	*e*	255
Nominal values volume sensor	3, Register or 4, Extended register	7	*b*	0	6	*e*	255
Input pulse constants	1, Data	7	*b*	0	7	*e*	255
Intervals and periods	1, Data	7	*b*	0	8	*e*	255
Time entries	1, Data	7	*b*	0	9	*e*	255
Station management information	3, Register or 4, Extended register	7	*b*	0	10, 11	*e*	255
Gas parameters for volume conversion, currently used in compressibility calculation	1, Data, 3, Register or 4, Extended register	7	*b*	0	12	*e*	255
Gas measuring method specific parameters	1, Data, 3, Register or 4, Extended register	7	*b*	0	13, 14	*e*	255
Sensor manager objects	67, Sensor manager	7	*b*	0	15	*e*	255

6.6.5 Internal operating status objects – Gas

A number of gas related objects are available to hold information about the *internal operating status.* The status is held by the *value* attribute of a "Data" object, with data type *unsigned, long-unsigned, double-long-unsigned, long64-unsigned, octet-string, boolean or bit-string.* Alternatively, the status is held by a "Status mapping" object, which holds both the status word and the mapping of its bits to the reference table. If there are several gas related internal operating status objects used, it is allowed to combine them into an instance of the IC "Profile generic" or "Register table", using the OBIS code of the global internal operating status.

© Copyright 1997-2020 DLMS User Association

Gas related internal operating status objects	IC	OBIS code					
		A	B	C	D	E	F
Internal operating status signals, gas related, content manufacturer specific	1, Data	7	*b*	96	5	0…9	255
Internal operating status signals, gas related, content mapped to reference table	63, Status mapping	7	*b*	96	5	0…9	255

6.6.6 Measured values – Gas

6.6.6.1 Indexes and index differences – Gas

Table 47 shows the objects to be used to represent indexes and index differences of volume, mass and energy.

Table 47 – Indexes and index differences

Indexes and index differences	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Index	Register or Extended register	7	*b*	1…8, 11…16, 21…26, 31…36, 61…66	0…3	0, 1…63	255
Index relative to billing periods					24…26 42…44 63…65 81…83		0…99 101…126
Index difference over measurement periods					6….23		255
Index difference over billing periods					27…32, 45…50, 66…71, 84…89		255
Maximum of index differences over billing periods	Extended register, or Profile generic				33…41, 51…62, 72…80, 90…98		0…99 101…126, 255

[a] See Table 94 – Value group C codes – Gas

[b] See Table 95 – Value group D codes – Gas – Indexes and index differences

[c] See Table 100 – Value group E codes – Gas – Indexes and index differences – Tariff rates

© Copyright 1997-2020 DLMS User Association

6.6.6.2 Flow rate – Gas

Table 48 shows the objects to be used to represent flow rate values.

Table 48 – Flow rate

Flow rate	IC	OBIS code					
		A	B	C[a]	D[b]	E	F
Flow rate, instantaneous values	Register or Extended register	7	*b*	43	0, 1, 2, 13,	0	255
Flow rate current average and last average values over averaging periods	Register, Extended register or Demand register				15…18, 19…22, 35…38, 39…42, 55…58, 59…62, 63…66, 67…70	0	255
Maximum of last averages of flow rates relative to measuring period	Extended register or Profile generic				23…26, 27…30, 43…46, 47…50		255
Maximum of last averages of flow rates relative to billing period 1	Extended register or Profile generic				31…34, 51…54	0	0…99, 101…126, 255
[a] See Table 94 – Value group C codes – Gas [b] See Table 96 – Value group D codes – Gas – Flow rate							

6.6.6.3 Process values – Gas

Table 49 shows the objects to be used to represent process values.

Table 49 – Process values

Process values	IC	OBIS code					
		A	B	C[a]	D[b]	E	F
Measurands of the gas process in different conditions, average values, minima and maxima relative to different process intervals	Register or Extended register	7	*b*	41, 42, 44…49	0, 2, 3, 10, 11, 13, 15…92	0	255
[a] See Table 94 – Value group C codes – Gas [b] See Table 97 – Value group D codes – Gas – Process values							

6.6.7 Conversion related factors and coefficients – Gas

Table 50 shows the objects available to represent correction, conversion, Compressibility, Superior calorific value and gas law deviation coefficient values. Various OBIS code allocations are made taking into consideration the specifics of the measuring process.

© Copyright 1997-2020 DLMS User Association

Table 50 – Conversion related factors and coefficients

Conversion related factors and coefficients	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Process independent current value or weighted value	Register or Extended register	7	*b*	51…55	0, 2, 3, 10, 11	0,1	255
Current average and last average values						11…28	

[a] See Table 94 – Value group C codes – Gas

[b] See Table 98 – Value group D codes – Gas – Conversion related factors and coefficients

[c] See Table 101 – Value group E codes – Gas – Conversion related factors and coefficients

6.6.8 Calculation methods – Gas

Table 51 shows the OBIS codes of the objects used to identify calculation methods for correction, compression and compressibility calculation as well as for superior calorific value and gas law deviation coefficient.

Only one calculation method per calculation process can be in use, but gas measurement devices may have the capability to support various calculation methods. The choice of the methods may depend on regulation, national or company requirements etc. The use of these objects should be part of project specific companion specifications. The contents of the calculation methods listed are not defined in this document.

The calculation method is held by the *value* attribute with data types *octet-string*, *visible-string*, *unsigned*, *long-unsigned* or *enumerated*.

NOTE Calculation methods for compressibility factor Z can be found for example in ISO 12213-3, EN 12405-1, EN 12405-2 and AGA 8. EN 12405-1 also offers methods for volume conversion and EN 12405-2 for energy conversion.

Table 51 – Calculation methods

Calculation methods	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Calculation methods	Data, Register or Extended register	7	*b*	51…55	12	0…20	255

[a] See Table 94 – Value group C codes – Gas

[b] See Table 98 – Value group D codes – Gas – Conversion related factors and coefficients

[c] See Table 102 – Value group E codes – Gas – Calculation methods – Value group E codes – Gas – Calculation methods

6.6.9 Natural gas analysis

Table 52 shows the objects to be used to represent values of natural gas analysis.

© Copyright 1997-2020 DLMS User Association

Table 52 – Natural gas analysis

Natural gas analysis	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Reference values of gas analysis process	Register or Extended register	7	*b*	70	8,9	0	255
Gas characteristics Process independent current value, and weighted value					10…20, 60…84	0,1	
Gas characteristics Average values for current and last intervals						11…28	
[a] See Table 94 – Value group C codes – Gas [b] See Table 99 – Value group D codes – Gas – Natural gas analysis values [c] See Table 103 – Value group E codes – Gas – Natural gas analysis values – Averages							

6.6.10 List objects – Gas

These COSEM objects are used to model lists of any kind of data, for example measurement values, constants, statuses, events. They are modelled by “Profile generic” objects. See also 7.8.6.3.

One standard object per billing period scheme is defined. See also 7.8.6.3.

List objects - Gas	IC	OBIS code					
		A	B	C	D	E	F
For names and OBIS codes see Table 106 – OBIS codes for list objects – Gas.	7, Profile generic	7	*b*	98	*d*	*e*	255[a]
[a] F = 255 means a wildcard here. See 7.11.3.							

© Copyright 1997-2020 DLMS User Association

6.7 Water meter related COSEM objects

6.7.1 General

The use of interface classes to represent various data is described in the following subclauses.

Grouping the data by major categories supports the link between the data and the OBIS value groups associated with them.

6.7.2 ID numbers – water meter

The different water meter ID numbers are instances of the IC "Data", with data type *unsigned, long-unsigned, double-long-unsigned, long64-unsigned, octet-string* or *visible-string.*

If more than one of those is used, it is allowed to combine them into a "Profile generic" object. In this case, the captured objects are *value* attributes of the water meter ID data objects, the capture period is 1 to have just actual values, the sort method is FIFO, the profile entries are limited to 1. Alternatively, an instance of the IC "Register table" can be used. See also Table 111.

Water meter ID	IC	OBIS code					
		A	B	C	D	E	F
Water meter ID 1…10 object	1, Data	8/9	*b*	0	0	0…9	255
Water meter ID-s object	7, Profile generic	8/9	*b*	0	0	255	255
Water meter ID-s object	61, Register table	8/9	*b*	0	0	255	255

6.7.3 Billing period values / reset counter entries – water meter

These values are represented by instances of the IC "Data".

For billing period / reset counters and for number of available billing periods the data type shall be *unsigned*, *long-unsigned* or *double-long-unsigned*. For time stamps of billing periods, the data type shall be *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1. These objects may be related to channels.

When the values of historical periods are represented by "Profile generic" objects, the time stamp of the billing period objects shall be part of the captured objects.

Billing period values / reset counter entries objects "Storage information"	IC	OBIS code					
		A	B	C	D	E	F
For item names and OBIS codes see Table 111.	1, Data	8/9	*b*	0	1	1,2, 10.. 12	255

© Copyright 1997-2020 DLMS User Association

6.7.4 General purpose objects – water meter

The use of ICs shall be as specified below:

- *Program entries* are represented by instances of the IC "Data" with data type *unsigned, long-unsigned* or *octet-string.* Program entries can also be related to a channel;
- *Threshold values* are represented by instances of the IC "Register" or “Extended register”.
- *Input pulse constants* are represented by instances of the IC "Data", “Register” or “Extended register” with data type *unsigned, long-unsigned* or *octet-string.*. *Input pulse constants* entries can also be related to a channel;
- *Timing information for measurement and registration periods is* presented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *unsigned*, *long-unsigned* or *double-long-unsigned*;
- *Time entry* values are represented by instances of IC “Data”, “Register” or “Extended register” with the data type of the *value* attribute *double-long-unsigned* (in the case of UNIX time), *octet-string* or *date-time* formatted as specified in 4.1.6.1.

General purpose objects – Water meter related [a]	IC	OBIS code					
		A	B	C	D	E	F
Program entries	1, Data	8/9	*b*	0	2	0,3	255
Threshold values	3, Register or 4, Extended register	8/9	*b*	0	5	1	255
Input pulse constants	3, Register or 4, Extended register	8/9	*b*	0	7	1	255
Measurement-/registration period duration	1, Data 3, Register or 4, Extended register	8/9	*b*	0	8	1,6	255
Time entries	1, Data 3, Register or 4, Extended register	8/9	*b*	0	9	1-3	255

[a] For item names and OBIS codes see Table 111.

6.7.5 Measured values – water meter

6.7.5.1 Consumption – water meter

The use of ICs shall be as specified below:

Consumption values are held by the value attribute of “Register” or “Extended register“ objects with data types double-long,double-long-unsigned, octet-string, visible-string, integer, long, unsigned, long-unsigned, long64, long64-unsigned, float32 or float64.

© Copyright 1997-2020 DLMS User Association

Consumption	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Accumulated volume	3, Register or 4, Extended register	8/9	*b*	1	0-3	0-12	255
					1-3		0-99, 101-125
Minimum, Maximum of integral value over billing periods	4, Extended register, or 7, Profile generic				4,5		
Integral test value	3, Register, 4, Extended register, or 7, Profile generic				6		255

[a] See. Table 108– Value group C codes - water meters

[b] See. Table 109 – Value group D codes - water meters

[c] All other values reserved for further use (tariff rate E=0 means Total)

6.7.5.2 Monitoring values – water meter

The use of ICs shall be as specified below:

Monitoring values are held by the *value* attribute of “Register” or “Extended register“ objects with data types *double-long,double-long-unsigned, octet-string, visible-string, integer, long,unsigned, long-unsigned, long64, long64-unsigned, float32* and *float64*.

Monitoring values	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Flow rate values	3, Register or 4, Extended register	8/9	*b*	2	0-3	0-12	255
					1-3		0-99, 101-125
Minimum, Maximum of Flow rate	4, Extended register, or 7, Profile generic				4,5		
Flow rate test value	3, Register, 4, Extended register, or 7, Profile generic				6		255

© Copyright 1997-2020 DLMS User Association

Monitoring values	IC	OBIS code					
		A	B	C[a]	D[b]	E[c]	F
Temperature values	3, Register or 4, Extended register	8/9	*b*	3	0-3	0-12	255
					1-3		0..99, 101-125
Minimum, Maximum of temperature	4, Extended register, or 7, Profile generic				4,5		
Temperature test value	3, Register, 4, Extended register, or 7, Profile generic				6		255

[a] See Table 108 – Value group C codes - water meters

[b] See Table 109 – Value group D codes - water meters

[c] All other values reserved for further use (tariff rate E=0 means Total)

6.7.6 Error register objects – water meter

A series of objects are used to communicate error indications of the device. See also Table 112.

The different error registers are held by the *value* attribute of “Data”, “Register” or “Extended register“ objects, with data type *bit-string*, *octet-string, unsigned, long-unsigned, double-long-unsigned* or *long64-unsigned.*

Error register objects – Water meters –	IC	OBIS code					
		A	B	C	D	E	F
Error register objects	1, Data or 3, Register or 4, Extended register	8/9	*b*	97	97	e	255

6.7.7 List objects – water meter

These COSEM objects are used to model lists of any kind of data, for example measurement values, constants, statuses, events. They are modelled by “Profile generic” objects

List objects – Water meters	IC	OBIS code					
		A	B	C	D	E	F
List objects – water	7, Profile generic	8/9	*b*	98	*d*	*e*	255[a]
[a] F = 255 means a wildcard here.							

6.7.8 Data profile objects – water meter

Water meter related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data.

Data profile objects – Water meters	IC	OBIS code					
		A	B	C	D	E	F
Data profile objects	7, Profile generic	8/9	*b*	99	1	*e*	255

© Copyright 1997-2020 DLMS User Association

6.7.9 Coding of OBIS identifications

To identify different instances of the same IC, their logical_name shall be different. In COSEM, the logical_name is taken from the OBIS definition (see 6.2, 6.3, 6.4, 6.5, 6.6, 6.7 and Clause 7).

OBIS codes are used within the COSEM environment as an *octet-string [6].* Each octet contains the unsigned value of the corresponding OBIS value group, coded without tags.

If a data item is identified by less than six value groups, all unused value groups shall be filled with 255.

Octet 1 contains the binary coded value of A (A = 0, 1, 2 …9) in the four rightmost bits. The four leftmost bits contain the information on the identification system. The four leftmost bits set to zero indicate that the OBIS identification system (version 1) is used as *logical_name*.

Identification system used	**Four leftmost bits of octet 1 (MSB left)**
OBIS, see Clause 7.	0 0 0 0
Reserved for future use	0 0 0 1 … 1 1 1 1

Within all value groups, the usage of a certain selection is fully defined; others are reserved for future use. If in the value groups B to F a value belonging to the manufacturer specific range (see 7.2.2) is used, then the whole OBIS code shall be considered as manufacturer specific, and the value of the other groups does not necessarily carry a meaning defined neither by Clause 4 nor Clause 7 of this Technical Report.

© Copyright 1997-2020 DLMS User Association

7 COSEM Object Identification System (OBIS)

7.1 Scope

The OBject Identification System (OBIS) defines the identification codes for commonly used data items in metering equipment. This Clause 7 specifies the overall structure of the identification system and the mapping of all data items to their identification codes.

OBIS provides a unique identifier for all data within the metering equipment, including not only measurement values, but also abstract values used for configuration or obtaining information about the behaviour of the metering equipment. The ID codes defined in this document are used for the identification of:

- logical names of the instances of the ICs, the objects, as defined in Clauses 4 and 6;
- data transmitted through communication lines, see 7.11.1;
- data displayed on the metering equipment, see 7.11.2.

This document applies to all types of metering equipment, such as fully integrated meters, modular meters, tariff attachments, data concentrators etc.

To cover metering equipment measuring energy types other than electricity, combined metering equipment measuring more than one type of energy or metering equipment with several physical measurement channels, the concepts of medium and channels are introduced. This allows meter data originating from different sources to be identified.

7.2 OBIS code structure

7.2.1 Value groups and their use

OBIS codes identify data items used in energy metering equipment, in a hierarchical structure using six value groups A to F, see Table 53.

Table 53 – OBIS code structure and use of value groups

Value group	Use of the value group
A	Identifies the media (energy type) to which the metering is related. Non-media related information is handled as abstract data.
B	Generally, identifies the measurement channel number, i.e. the number of the input of a metering equipment having several inputs for the measurement of energy of the same or different types (for example in data concentrators, registration units). Data from different sources can thus be identified. It may also identify the communication channel, and in some cases it may identify other elements. The definitions for this value group are independent from the value group A.
C	Identifies abstract or physical data items related to the information source concerned, for example current, voltage, power, volume, temperature. The definitions depend on the value in the value group A. Further processing, classification and storage methods are defined by value groups D, E and F. For abstract data, value groups D to F provide further classification of data identified by value groups A to C.
D	Identifies types, or the result of the processing of physical quantities identified by values in value groups A and C, according to various specific algorithms. The algorithms can deliver energy and demand quantities as well as other physical quantities.
E	Identifies further processing or classification of quantities identified by values in value groups A to D.
F	Identifies historical values of data, identified by values in value groups A to E, according to different billing periods. Where this is not relevant, this value group can be used for further classification.

7.2.2 Manufacturer specific codes

In value groups B to F, the following ranges are available for manufacturer-specific purposes:

© Copyright 1997-2020 DLMS User Association

- group B: 128…199;
- group C: 128…199, 240;
- group D: 128…254;
- group E: 128…254;
- group F: 128…254.

If any of these value groups contain a value in the manufacturer specific range, then the whole OBIS code shall be considered as manufacturer specific, and the value of the other groups does not necessarily carry a meaning defined in Clause 6 or 7.

In addition, manufacturer specific ranges are defined in Table 60 with A = 0, C = 96, in Table 72 with A = 1, C = 96, in Table 80 with A = 4, C = 96, in Table 88 with A = 5/6, C = 96, in Table 104 with A = 7, C = 96 and in Table 111 with A = 8/9, C = 96.

7.2.3 Reserved ranges

By default, all codes not allocated are reserved. [4]

7.2.4 Summary of rules for manufacturer, utility, consortia and country specific codes

Table 54 summarizes the rules for manufacturer specific codes specified in 7.2.2, utility specific codes specified in 7.3.2, consortia specific codes specified in 7.3.4.2 and country specific codes specified in 7.3.4.3.

Table 54 – Rules for manufacturer, utility, consortia and country specific codes

Code type	Value group					
	A	B	C	D	E	F
Manufacturer specific [1]	0, 1, 4…9, F	128…199	*c*	*d*	*e*	*f*
		b	128… 199, 240	*d*	*e*	*f*
		b	*c*	128…254	*e*	*f*
		b	*c*	*d*	128…254	*f*
		b	*c*	*d*	*e*	128…254
Manufacturer specific abstract [2]	0	0…64	96	50…99	0…255	0…255
Manufacturer specific, media related general purpose [2]	1, 4…9, F	0…64	96	50…99	0…255	0…255
Utility specific [3]	0, 1, 4…9, F	65…127	0…255	0…255	0…255	0…255
Consortia specific [4]	0, 1, 4…9, F	0…64	93	See Table 58.		
Country specific [5]		0…64	94	See Table 59.		

4 Administered by the DLMS User Association (see Foreword).

© Copyright 1997-2020 DLMS User Association

NOTE 1	“b”, “c”, “d”, “e”, “f” means any value in the relevant value group.
NOTE 2	The range D = 50…99 is available for identifying objects, which are not represented by another defined code, but need representation on the display as well. If this is not required, the range D = 128…254 should be used.
NOTE 3	If the value in value group B is 65…127, the whole OBIS code should be considered as utility specific and the value of other groups does not necessarily carry a meaning defined neither in Clause 6 nor Clause 7.
NOTE 4	The usage of value group E and F are defined in consortia specific documents.
NOTE 5	The usage of value group E and F are defined in country specific documents.

Objects for which this Technical Report defines standard identifiers shall not be re-identified by manufacturer, utility, consortia or country specific identifiers.

On the other hand, an object previously identified by a manufacturer-, utility-, consortia- or country- specific identifier may receive a standard identifier in the future, if its use is of common interest for the users of this document.

7.2.5 Standard object codes

Standard object codes are meaningful combinations of defined values of the six value groups.

Notation: In the following tables, in the various value groups, *“b”, “c”, ”d”, “e”, “f”* signifies any value in the respective value group. If only one object is instantiated, the value shall be 0. If a value group is shaded, then this value group is not used.

NOTE The DLMS UA maintains a list of standard COSEM object definitions at www.dlms.com. The validity of the combination of OBIS codes and class_id-s as well as the data types of the attributes are tested during conformance testing.

7.3 Value group definitions – overview

7.3.1 Value group A

The range for value group A is 0 to 15; see Table 55.

Table 55 – Value group A codes

Value group A	
0	Abstract objects
1	Electricity related objects
…	
4	Heat cost allocator related objects
5, 6	Thermal energy related objects
7	Gas related objects
8	Cold water related objects
9	Hot water related objects
…	
15	Other media
All other	Reserved

The following subclauses contain value group definitions B to F common for all values of value group A.

7.3.2 Value group B

The range for value group B is 0 to 255; see Table 56.

© Copyright 1997-2020 DLMS User Association

Table 56 – Value group B codes

Value group B	
0	No channel specified
1…64	Channel 1..64
65…127	Utility specific codes
128…199	Manufacturer specific codes
200…255	Reserved

If channel information is not essential, the value 0 shall be assigned.

The range 65…127 is available for utility specific use. If the value of value group B is in this range, the whole OBIS code shall be considered as utility specific and the value of other groups does not necessarily carry a meaning defined neither in Clause 4 nor in Clause 7.

7.3.3 Value group C

7.3.3.1 General

The range for value group C is 0 to 255. The definitions depend on the value in value group A. The codes for abstract objects are specified in 7.3.3.2. See also:

- electricity related codes specified in 7.5.1;
- heat cost allocator related codes specified in 7.6.2;
- thermal energy related codes specified in 7.7.2;
- gas related codes specified in 7.8.2;
- water related codes specified in 7.9.2;
- other media related codes specified in 7.10.2.

7.3.3.2 Abstract objects

Abstract objects are data items, which are not related to a certain type of physical quantity. See Table 57.

Table 57 – Value group C codes – Abstract objects

Value group C Abstract objects (A = 0)	
0…89	Context specific identifiers [a]
93	Consortia specific identifiers (See 7.3.4.2).
94	Country specific identifiers (See 7.3.4.3)
96	General and service entry objects – Abstract (See 7.4.1)
97	Error register objects – Abstract (See 7.4.2)
98	List objects – Abstract (See 7.4.3, 7.4.4)
99	Data profile objects – Abstract (See 7.4.5)
…	
127	Inactive objects [b]
128…199, 240	Manufacturer specific codes

© Copyright 1997-2020 DLMS User Association

All other	Reserved
[a]	Context specific identifiers identify objects specific to a certain protocol and/or application. For the COSEM context, the identifiers are defined in 6.2.
[b]	An inactive object is an object, which is defined and present in a meter, but which has no assigned functionality.

7.3.4 Value group D

7.3.4.1 General

The range for value group D is 0 to 255.

7.3.4.2 Consortia specific identifiers

Table 58 specifies the use of value group D for consortia specific applications. In this table, there are no reserved ranges for manufacturer specific codes. The usage of value group E and F are defined in consortia specific documents.

Objects that are already identified in this document shall not be re-identified by consortia specific identifiers.

Table 58 – Value group D codes – Consortia specific identifiers

Value group D Consortia specific identifiers (A = any, C = 93)	
All values	Reserved
NOTE At the time of the publication of this document, no consortia specific identifiers are allocated.	

7.3.4.3 Country specific identifiers

Table 59 specifies the use of value group D for country specific applications. Wherever possible, the country calling codes are used. In this table, there are no reserved ranges for manufacturer specific codes. The usage of value group E and F are defined in country specific documents.

Objects that are already identified in this document shall not be re-identified by country specific identifiers.

Table 59 – Value group D codes – Country specific identifiers

Value group D Country specific identifiers [a] (A = any, C = 94)			
00	Finland (Country calling code = 358)	**50**	
01	USA (= Country calling code)	**51**	Peru (= Country calling code)
02	Canada (Country calling code = 1)	**52**	South Korea (Country calling code = 82)
03	Serbia (Country calling code = 381)	**53**	Cuba (= Country calling code)
04		**54**	Argentina (= Country calling code)
05		**55**	Brazil (= Country calling code)
06		**56**	Chile (= Country calling code)
07	Russia (Country calling code = 7)	**57**	Colombia (= Country calling code)
08		**58**	Venezuela (= Country calling code)
09		**59**	
10	Czech Republic (Country calling code = 420)	**60**	Malaysia (= Country calling code)
11	Bulgaria (Country calling code = 359)	**61**	Australia (= Country calling code)
12	Croatia (Country calling code = 385)	**62**	Indonesia (= Country calling code)

© Copyright 1997-2020 DLMS User Association

Value group D Country specific identifiers [a] (A = any, C = 94)			
13	Ireland (Country calling code = 353)	**63**	Philippines (= Country calling code)
14	Israel (Country calling code = 972)	**64**	New Zealand (= Country calling code)
15	Ukraine (Country calling code = 380)	**65**	Singapore (= Country calling code)
16	Yugoslavia [a]	**66**	Thailand (= Country calling code)
17		**67**	
18		**68**	
19		**69**	
20	Egypt (= Country calling code)	**70**	
21		**71**	Latvia (Country calling code = 371)
22		**72**	
23		**73**	Moldova (Country calling code = 373)
24		**74**	
25		**75**	Belarus (Country calling code = 375)
26		**76**	
27	South Africa (= Country calling code)	**77**	
28		**78**	
29		**79**	
30	Greece (= Country calling code)	**80**	
31	Netherlands (= Country calling code)	**81**	Japan (= Country calling code)
32	Belgium (= Country calling code)	**82**	Mexico
33	France (= Country calling code)	**83**	
34	Spain (= Country calling code)	**84**	
35	Portugal (Country calling code = 351)	**85**	Hong Kong (Country calling code = 852)
36	Hungary (= Country calling code)	**86**	China (= Country calling code)
37	Lithuania (Country calling code = 370)	**87**	Bosnia and Herzegovina (Country calling code = 387)
38	Slovenia (Country calling code = 386)	**88**	
39	Italy (= Country calling code)	**89**	
40	Romania (= Country calling code)	**90**	Turkey (= Country calling code)
41	Switzerland (= Country calling code)	**91**	India (= Country calling code)
42	Slovakia (Country calling code = 421)	**92**	Pakistan (= Country calling code)
43	Austria (= Country calling code)	**93**	
44	United Kingdom (= Country calling code)	**94**	
45	Denmark (= Country calling code)	**95**	
46	Sweden (= Country calling code)	**96**	Saudi Arabia (Country calling code = 966)
47	Norway (= Country calling code)	**97**	United Arab Emirates (Country calling code = 971)
48	Poland (= Country calling code)	**98**	Iran (= Country calling code)
49	Germany (= Country calling code)	**99**	
	All other codes are reserved		
[a]	With the dissolution of the former Yugoslavia into separate nations, country code 38 was decommissioned.		

© Copyright 1997-2020 DLMS User Association

7.3.4.4 Identification of general and service entry objects

For the use of value group D to identify:

- abstract general and service entry objects, see 7.4, Table 60;
- electricity related general and service entry objects, see Table 72;
- HCA related general and service entry objects, see Table 80;
- thermal energy related general and service entry objects, see Table 88;
- gas related general and service entry objects, see Table 104;
- water related general and service entry objects, see Table 111.

7.3.5 Value group E

The range for value group E is 0 to 255. It can be used for identifying further classification or processing of values defined by values in value groups A to D, as specified in the relevant energy type specific clauses. The various classifications and processing methods are exclusive.

For the use of value group E to identify:

- abstract general and service entry objects, see 7.4, Table 60;
- electricity related general and service entry objects, see Table 72;
- HCA related general and service entry objects, see Table 80;
- thermal energy related general and service entry objects, see Table 88;
- gas related general and service entry objects, see Table 104;
- water related general and service entry objects, see Table 111.

7.3.6 Value group F

7.3.6.1 General

The range for value group F is 0 to 255. In all cases, if value group F is not used, it is set to 255.

7.3.6.2 Identification of billing periods

Value group F specifies the allocation to different billing periods (sets of historical values) for the objects defined by value groups A to E, where storage of historical values is relevant. A billing period scheme is identified with its billing period counter, number of available billing periods, time stamp of the billing period and billing period length. Several billing period schemes may be possible. For more, see 6.2.2 and 7.11.3.

© Copyright 1997-2020 DLMS User Association

7.4 Abstract objects (Value group A = 0)

7.4.1 General and service entry objects – Abstract

Table 60 specifies OBIS codes for abstract objects. See also Table 41.

Table 60 – OBIS codes for general and service entry objects

General and service entry objects	OBIS code					
	A	B	C	D	E	F
Billing period values/reset counter entries (First billing period scheme if there are two)						
Billing period counter (1)	0	*b*	0	1	0	VZ or 255
Billing period counter (1) in a recent billing period	0	*b*	0	1	0	101-125
Billing period counters (1) in unspecified number of recent billing periods	0	*b*	0	1	0	126
Number of available billing periods (1)	0	*b*	0	1	1	255
Time stamp of the most recent billing period (1)	0	*b*	0	1	2	255
Time stamp of the billing period (1) VZ (last reset)	0	*b*	0	1	2	VZ
Time stamp of the billing period (1) VZ_{-1}	0	*b*	0	1	2	VZ_{-1}
…	…	…	…	…	…	…
Time stamp of the billing period (1) VZ_{-n}	0	*b*	0	1	2	VZ_{-n}
Time stamp of the billing period (1) in a recent billing period	0	*b*	0	1	2	101-125
Time stamp of the billing period (1) in unspecified number of recent billing periods	0	*b*	0	1	2	126
Billing period values/reset counter entries (Second billing period scheme)						
Billing period counter (2) in billing period VZ	0	*b*	0	1	3	VZ or 255
Billing period counter (2) in a recent billing period	0	*b*	0	1	3	101-125
Billing period counters (2) in unspecified number of recent billing periods	0	*b*	0	1	3	126
Number of available billing periods (2)	0	*b*	0	1	4	255
Time stamp of the most recent billing period (2)	0	*b*	0	1	5	255
Time stamp of the billing period (2) VZ (last reset)	0	*b*	0	1	5	VZ
Time stamp of the billing period (2) VZ_{-1}	0	*b*	0	1	5	VZ_{-1}
…	…	…	…	…	…	…
Time stamp of the billing period (2) VZ_{-n}	0	*b*	0	1	5	VZ_{-n}
Time stamp of the billing period (2) in a recent billing period	0	*b*	0	1	5	101-125
Time stamp of the billing period (2) in unspecified number of recent billing periods	0	*b*	0	1	5	126
Program entries						
Active firmware identifier	0	*b*	0	2	0	
Active firmware version	0	*b*	0	2	1	
Active firmware signature	0	*b*	0	2	8	
Time entries						
Local time	0	*b*	0	9	1	
Local date	0	*b*	0	9	2	
Device IDs						

© Copyright 1997-2020 DLMS User Association

General and service entry objects	OBIS code					
	A	B	C	D	E	F
Complete device ID	0	*b*	96	1		
Device ID # 1 (manufacturing number)	0	*b*	96	1	0	
…			…	…	…	
Device ID # 10	0	*b*	96	1	9	
Metering point ID (abstract)	0	0	96	1	10	
Parameter changes, calibration and access						
Number of configuration program changes	0	*b*	96	2	0	
Date [a] of last configuration program change	0	*b*	96	2	1	
Date [a] of last time switch program change	0	*b*	96	2	2	
Date [a] of last ripple control receiver program change	0	*b*	96	2	3	
Status of security switches	0	*b*	96	2	4	
Date [a] of last calibration	0	*b*	96	2	5	
Date [a] of next configuration program change	0	*b*	96	2	6	
Date [a] of activation of the passive calendar	0	*b*	96	2	7	
Number of protected configuration program changes [b]	0	*b*	96	2	10	
Date [a] of last protected configuration program change [b]	0	*b*	96	2	11	
Date [a] (corrected) of last clock synchronization/setting	0	*b*	96	2	12	
Date of last firmware activation	0	*b*	96	2	13	
Input/output control signals						
State of input/output control signals, global [c]	0	*b*	96	3	0	
State of input control signals (status word 1)	0	*b*	96	3	1	
State of output control signals (status word 2)	0	*b*	96	3	2	
State of input/output control signals (status word 3)	0	*b*	96	3	3	
State of input/output control signals (status word 4)	0	*b*	96	3	4	
Disconnect control	0	*b*	96	3	10	
Arbitrator	0	*b*	96	3	20.. 29	
Internal control signals						
Internal control signals, global [c]	0	*b*	96	4	0	
Internal control signals (status word 1)	0	*b*	96	4	1	
Internal control signals (status word 2)	0	*b*	96	4	2	
Internal control signals (status word 3)	0	*b*	96	4	3	
Internal control signals (status word 4)	0	*b*	96	4	4	
Internal operating status						
Internal operating status, global [c]	0	*b*	96	5	0	
Internal operating status (status word 1)	0	*b*	96	5	1	
Internal operating status (status word 2)	0	*b*	96	5	2	
Internal operating status (status word 3)	0	*b*	96	5	3	
Internal operating status (status word 4)	0	*b*	96	5	4	
Battery entries						
Battery use time counter	0	*b*	96	6	0	
Battery charge display	0	*b*	96	6	1	
Date of next battery change	0	*b*	96	6	2	
Battery voltage	0	*b*	96	6	3	
Battery initial capacity	0	*b*	96	6	4	
Battery installation date and time	0	*b*	96	6	5	
Battery estimated remaining use time	0	*b*	96	6	6	
Aux. supply use time counter	0	*b*	96	6	10	

© Copyright 1997-2020 DLMS User Association

General and service entry objects	OBIS code					
	A	B	C	D	E	F
Aux. voltage (measured)	0	*b*	96	6	11	
Power failure monitoring						
Number of power failures						
In all three phases	0	0	96	7	0	
In phase L1	0	0	96	7	1	
In phase L2	0	0	96	7	2	
In phase L3	0	0	96	7	3	
In any phase [sic]	0	0	96	7	21	
Auxiliary supply	0	0	96	7	4	
Number of long power failures						
In all three phases	0	0	96	7	5	
In phase L1	0	0	96	7	6	
In phase L2	0	0	96	7	7	
In phase L3	0	0	96	7	8	
In any phase	0	0	96	7	9	
Time of power failure [d]						
In all three phases	0	0	96	7	10	
In phase L1	0	0	96	7	11	
In phase L2	0	0	96	7	12	
In phase L3	0	0	96	7	13	
In any phase	0	0	96	7	14	
Duration of long power failure [e]						
In all three phases	0	0	96	7	15	
In phase L1	0	0	96	7	16	
In phase L2	0	0	96	7	17	
In phase L3	0	0	96	7	18	
In any phase	0	0	96	7	19	
Time threshold for long power failure						
Time threshold for long power failure	0	0	96	7	20	
NOTE 1 See *Number of power failures in any phase* above	*0*	*b*	*96*	*7*	*21*	
Operating time						
Time of operation	0	b	96	8	0	
Time of operation rate 1…rate 63	0	b	96	8	1… 63	
Environment related parameters						
Ambient temperature	0	*b*	96	9	0	
Ambient pressure	0	*b*	96	9	1	
Relative humidity	0	*b*	96	9	2	
Status register						
Status register (Status register 1 if several status registers are used)	0	*b*	96	10	1	
Status register 2	0	*b*	96	10	2	
…	0	*b*	96	10	…	
Status register 10	0	*b*	96	10	10	
Event code						
Event code objects # 1…#100	0	*b*	96	11	0… 99	
Communication port log parameters						
Reserved	0	*b*	96	12	0	

© Copyright 1997-2020 DLMS User Association

General and service entry objects	OBIS code					
	A	B	C	D	E	F
Number of connections	0	*b*	96	12	1	
Reserved	0	*b*	96	12	2	
Reserved	0	*b*	96	12	3	
Communication port parameter 1	0	*b*	96	12	4	
GSM field strength	0	*b*	96	12	5	
Telephone number / Communication address of the physical device	0	*b*	96	12	6	
Consumer messages						
Consumer message via local consumer information port	0	*b*	96	13	0	
Consumer message via the meter display and / or via consumer information port	0	*b*	96	13	1	
Currently active tariff						
Currently active tariff objects # 1…#16 NOTE 2 Object #16 (E = 15) carries the name of register with the lowest tariff (default tariff register)	0	*b*	96	14	0…15	
Event counter objects						
Event counter objects #1…#100	0	*b*	96	15	0…99	
Profile entry digital signature objects						
Profile entry digital signature objects #1…#10	0	*b*	96	16	0…9	
Profile entry counter objects						
Profile entry digital counter objects #1..#128	0	*b*	96	17	0…127	
Meter tamper event related objects						
Meter open event counter	0	*b*	96	20	0	
Meter open event, time stamp of current event occurrence	0	*b*	96	20	1	
Meter open event, duration of current event	0	*b*	96	20	2	
Meter open event, cumulative duration	0	*b*	96	20	3	
Reserved	0	*b*	96	20	4	
Terminal cover open event counter	0	*b*	96	20	5	
Terminal cover open event, time stamp of current event occurrence	0	*b*	96	20	6	
Terminal cover open event, duration of current event	0	*b*	96	20	7	
Terminal cover open event, cumulative duration	0	*b*	96	20	8	
Reserved	0	*b*	96	20	9	
Tilt event counter	0	*b*	96	20	10	
Tilt event, time stamp of current event occurrence	0	*b*	96	20	11	
Tilt event, duration of current event	0	*b*	96	20	12	
Tilt event, cumulative duration	0	*b*	96	20	13	
Reserved	0	*b*	96	20	14	
Strong DC magnetic field event counter	0	*b*	96	20	15	
Strong DC magnetic field event, time stamp of current event occurrence	0	*b*	96	20	16	
Strong DC magnetic field event, duration of current event	0	*b*	96	20	17	
Strong DC magnetic field event, cumulative duration	0	*b*	96	20	18	
Reserved	0	*b*	96	20	19	
Supply control switch / valve tamper event counter	0	*b*	96	20	20	
Supply control switch / valve tamper event, time stamp of current event occurrence	0	*b*	96	20	21	
Supply control switch / valve tamper event, duration of current event	0	*b*	96	20	22	

© Copyright 1997-2020 DLMS User Association

General and service entry objects	OBIS code					
	A	B	C	D	E	F
Supply control switch / valve tamper event, cumulative duration	0	*b*	96	20	23	
Reserved	0	*b*	96	20	24	
Metrology tamper event counter	0	*b*	96	20	25	
Metrology tamper event, time stamp of current event occurrence	0	*b*	96	20	26	
Metrology tamper event, duration of current event	0	*b*	96	20	27	
Metrology tamper event, cumulative duration	0	*b*	96	20	28	
Reserved	0	*b*	96	20	29	
Communication tamper event counter	0	*b*	96	20	30	
Communication tamper event, time stamp of current event occurrence	0	*b*	96	20	31	
Communication tamper event, duration of current event	0	*b*	96	20	32	
Communication tamper event, cumulative duration	0	*b*	96	20	33	
Reserved	0	*b*	96	20	34	
Manufacturer specific [f] … Manufacturer specific	0 0	*b* *b*	96 96	50 99	*e* *e*	*f* *f*
All other codes are reserved						

[a]	Date of the event may contain the date only, the time only or both, encoded as specified in 4.1.6.1.
[b]	Protected configuration is characterized by the need to open the main meter cover to modify it, or to break a metrological seal.
[c]	Global status words with E = 0 contain the individual status words E = 1…4. The contents of the status words are not defined in this document.
[d]	Time of power failure is recorded when either a short or long power failure occurs.
[e]	Duration of long power failure holds the duration of the last long power failure.
[f]	The range D = 50…99 is available for identifying objects, which are not represented by another defined code, but need representation on the display as well. If this is not required, the range D = 128…254 should be used.

7.4.2 Error registers, alarm registers / filters / descriptor objects – Abstract

The OBIS codes for abstract error registers, alarm registers and alarm filters are shown in Table 61.

Table 61 – OBIS codes for error registers, alarm registers and alarm filters – Abstract

Error register, alarm register and alarm filter objects – Abstract	OBIS code					
	A	B	C	D	E	F
Error register objects 1…10	0	*b*	97	97	0…9	
Alarm register objects 1…10	0	*b*	97	98	0…9	
Alarm filter objects 1…10	0	*b*	97	98	10…19	
Alarm descriptor objects 1…10	0	*b*	97	98	20…29	
NOTE The information to be included in the error objects is not defined in this document.						

© Copyright 1997-2020 DLMS User Association

7.4.3 List objects – Abstract

Lists – identified with a single OBIS code – are defined as a series of any kind of data (for example measurement value, constants, status, events). See Table 62.

Table 62 – OBIS codes for list objects – Abstract

List objects – Abstract	OBIS code					
	A	B	C	D	E	F
Data of billing period (with billing period scheme 1 if there are more than one schemes available)	0	*b*	98	1	*e*	255 [a]
Data of billing period (with billing period scheme 2)	0	*b*	98	2	*e*	255 [a]
[a] F = 255 means a wildcard here. See 7.11.3.						

7.4.4 Register table objects – Abstract

Register tables are defined to hold a number of values of the same type. See Table 63.

Table 63 – OBIS codes for Register table objects – Abstract

Register table objects – Abstract	OBIS code					
	A	B	C	D	E	F
General use, abstract	0	*b*	98	10	*e*	

7.4.5 Data profile objects – Abstract

Abstract data profiles – instances of the “Profile generic IC” and identified with one single OBIS code as specified inTable 64 – are used to hold a series of measurement values of one or more similar quantities and/or to group various data.

Table 64 – OBIS codes for data profile objects – Abstract

Data profile objects – Abstract	OBIS code					
	A	B	C	D	E	F
Load profile with recording period 1 [a]	0	*b*	99	1	*e*	
Load profile with recording period 2 [a]	0	*b*	99	2	*e*	
Load profile during test [a]	0	*b*	99	3	0	
Connection profile	0	*b*	99	12	*e*	
GSM diagnostic profile	0	*b*	99	13	*e*	
Charge collection history (Payment metering)	0	*b*	99	14	e	
Token credit history (Payment metering)	0	*b*	99	15	e	
Parameter monitor log	0	*b*	99	16	*e*	
Token transfer log (Payment metering)	0	*b*	99	17	*e*	
LTE monitoring profile	0	*b*	99	18	*e*	
Event log [a]	0	*b*	99	98	*e*	
[a] These objects should be used if they (also) hold data not specific to the energy type.						

© Copyright 1997-2020 DLMS User Association

7.5 Electricity (Value group A = 1)

7.5.1 Value group C codes – Electricity

Table 65 specifies the use of value group C for electricity related objects.

The quadrant definitions for active and reactive power ar shown in Figure 34.

Table 65 – Value group C codes – Electricity

Value group C codes – Electricity (A = 1)				
0	General purpose objects (See 7.5.5.1)			
ΣL_i	**L_1**	**L_2**	**L_3**	(See also Note 2)
1	**21**	**41**	**61**	Active power+ (QI+QIV)
2	**22**	**42**	**62**	Active power– (QII+QIII)
3	**23**	**43**	**63**	Reactive power+ (QI+QII)
4	**24**	**44**	**64**	Reactive power– (QIII+QIV)
5	**25**	**45**	**65**	Reactive power QI
6	**26**	**46**	**66**	Reactive power QII
7	**27**	**47**	**67**	Reactive power QIII
8	**28**	**48**	**68**	Reactive power QIV
9	**29**	**49**	**69**	Apparent power+ (QI+QIV) (See also Note 3)
10	**30**	**50**	**70**	Apparent power– (QII+QIII)
11	**31**	**51**	**71**	Current: any phase (C = 11) / L_i phase [a] (C= 31, 51, 71)
12	**32**	**52**	**72**	Voltage: any phase (C = 12) / L_i phase [a] (C= 32, 52, 72)
13	**33**	**53**	**73**	Power factor (See also Note 4)
14	**34**	**54**	**74**	Supply frequency
15	**35**	**55**	**75**	Active power (abs(QI+QIV)+(abs(QII+QIII)) [a]
16	**36**	**56**	**76**	Active power (abs(QI+QIV)-abs(QII+QIII))
17	**37**	**57** [d]	**77**	Active power QI
18	**38**	**58**	**78**	Active power QII
19	**39**	**59**	**79**	Active power QIII
20	**40**	**60**	**80**	Active power QIV
….				
81	Angles [b]			
82	Unitless quantity (pulses or pieces)			
83	Transformer and line loss quantities [c]			
84	ΣL_i Power factor– (See also Note 4)			
85	L_1 Power factor–			
86	L_2 Power factor–			
87	L_3 Power factor–			
88	ΣL_i Ampere-squared hours (QI+QII+QIII+QIV)			
89	ΣL_i Volt-squared hours (QI+QII+QIII+QIV)			
90	ΣL_i current (algebraic sum of the – unsigned – value of the currents in all phases)			
91	L_0 current (neutral) [a]			
92	L_0 voltage (neutral) [a]			
93	Consortia specific identifiers (See 7.3.4.2)			
94	Country specific identifiers (See 7.3.4.3)			

© Copyright 1997-2020 DLMS User Association

Value group C codes – Electricity (A = 1)				
96	General and service entry objects – Electricity (See 7.5.5.1)			
97	Error register objects – Electricity (See 7.5.5.2)			
98	List objects – Electricity (See 7.5.5.3)			
99	Data profile objects – Electricity (See 7.5.5.4)			
ΣL_i	**L_1**	**L_2**	**L_3**	(See also Note 2)
100	**101**	**102**	**103**	Reactive power inductive (QI+QIII)
104	**105**	**106**	**107**	Reactive power capacitive (QII+QIV)
108..123	Reserved			
124	$L_1 - L_2$ line voltage			
125	$L_2 - L_3$ line voltage			
126	$L_3 - L_1$ line voltage			
127	Reserved			
128...199, 240	Manufacturer specific codes			
All other	Reserved			
NOTE 1	*L_i Quantity* is the value (to be measured) of a measurement system connected between the phase *i* and a reference point. In 3-phase 4-wire systems, the reference point is the neutral. In 3-phase 3-wire systems, the reference point is the phase L_2.			
NOTE 2	*ΣL_i Quantity* is the total measurement value across all systems.			
NOTE 3	If just one apparent energy/demand value is calculated over the four quadrants, C = 9 shall be used.			
NOTE 4	Power factor quantities with C = 13, 33, 53, 73 are calculated either as PF = Active power+ (C = 1, 21, 41, 61) / Apparent power+ (C = 9, 29, 49, 69) or PF = Active power– (C = 2, 22, 42, 62) / Apparent power- (C = 10, 30, 50, 70). In the first case, the sign is positive (no sign), it means power factor in the import direction (PF+). In the second case, the sign is negative, it means power factor in the export direction (PF–). Power factor quantities C = 84, 85, 86 and 87 are always calculated as PF– = Active power– / Apparent power–. This quantity is the power factor in the export direction; it has no sign.			

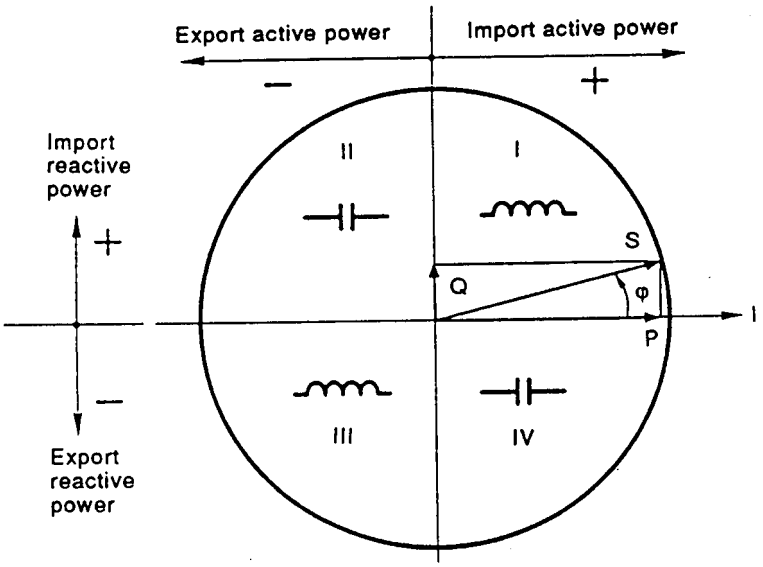
[a] For details of extended codes, see 7.5.3.3.

[b] For details of extended codes, see 7.5.3.4.

[c] For details of extended codes, see 7.5.3.5.

[d] This was recorded erroneously as 58 in Blue Book 12.2 and earlier versions.

© Copyright 1997-2020 DLMS User Association



NOTE The quadrant definitions are according to IEC 62053-23:2003, Figure C1.

Figure 34 – Quadrant definitions for active and reactive power

7.5.2 Value group D codes – Electricity

7.5.2.1 Processing of measurement values

Table 66 specifies the use of value group D for electricity related objects.

Table 66 – Value group D codes – Electricity

Value group D codes – Electricity (A = 1, C <> 0, 93, 94, 96, 97, 98, 99)	
0	Billing period average (since last reset)
1	Cumulative minimum 1
2	Cumulative maximum 1
3	Minimum 1
4	Current average 1
5	Last average 1
6	Maximum 1
7	Instantaneous value
8	Time integral 1
9	Time integral 2
10	Time integral 3
11	Cumulative minimum 2
12	Cumulative maximum 2
13	Minimum 2
14	Current average 2
15	Last average 2
16	Maximum 2
17	Time integral 7
18	Time integral 8
19	Time integral 9

© Copyright 1997-2020 DLMS User Association

Value group D codes – Electricity (A = 1, C <> 0, 93, 94, 96, 97, 98, 99)	
20	Time integral 10
21	Cumulative minimum 3
22	Cumulative maximum 3
23	Minimum 3
24	Current average 3
25	Last average 3
26	Maximum 3
27	Current average 5
28	Current average 6
29	Time integral 5
30	Time integral 6
31	Under limit threshold
32	Under limit occurrence counter
33	Under limit duration
34	Under limit magnitude
35	Over limit threshold
36	Over limit occurrence counter
37	Over limit duration
38	Over limit magnitude
39	Missing threshold
40	Missing occurrence counter
41	Missing duration
42	Missing magnitude
43	Time threshold for under limit
44	Time threshold for over limit
45	Time threshold for missing magnitude
46	Contracted value
49	Average value for recording interval 1
50	Average value for recording interval 2
51	Minimum for recording interval 1
52	Minimum for recording interval 2
53	Maximum for recording interval 1
54	Maximum for recording interval 2
55	Test average
56	Current average 4 for harmonics measurement
58	Time integral 4

© Copyright 1997-2020 DLMS User Association

Value group D codes – Electricity (A = 1, C <> 0, 93, 94, 96, 97, 98, 99)	
128...254	Manufacturer specific codes
All other	Reserved
NOTES	
Averaging scheme 1	Controlled by measurement period 1 (see Table 72), a set of registers is calculated by a metering device (codes 1...6). The typical usage is for billing purposes.
Averaging scheme 2	Controlled by measurement period 2, a set of registers is calculated by a metering device (codes 11...16). The typical usage is for billing purposes.
Averaging scheme 3	Controlled by measurement period 3, a set of registers is calculated by a metering device (codes 21...26). The typical usage is for instantaneous values.
Averaging scheme 4	Controlled by measurement period 4, a test average value (code 55) is calculated by the metering device.
Current average 1, 2, 3	See the definition of the “Demand register” IC in 4.3.4. The value is calculated using measurement period 1, 2 and/or 3 respectively.
Last average 1,2,3	See the definition of the “Demand register” IC in 4.3.4. The value is calculated using measurement period 1, 2 or 3 respectively.
Minimum	The smallest of last average values during a billing period, see Table 72.
Maximum	The largest of last average values during a billing period.
Cumulative min.	The cumulative sum of minimum values over all the past billing periods.
Cumulative max.	The cumulative sum of maximum values over all the past billing periods.
Current average 4	For harmonics measurement
Current average 5	See the definition of the “Demand register” IC in 4.3.4. The value is calculated using recording interval 1; see Table 72.
Current average 6	See the definition of the “Demand register” IC in 4.3.4. The value is calculated using recording interval 2.
Time integral 1	For a current billing period (F= 255): Time integral of the quantity calculated from the origin (first start of measurement) to the instantaneous time point. For a historical billing period (F= 0...99): Time integral of the quantity calculated from the origin to the end of the billing period given by the billing period code.
Time integral 2	For a current billing period (F = 255): Time integral of the quantity calculated from the beginning of the current billing period to the instantaneous time point. For a historical billing period (F = 0...99): Time integral of the quantity calculated over the billing period given by the billing period code.
Time integral 3	Time integral of the positive difference between the quantity and a prescribed threshold value.
Time integral 4 ("Test time integral")	Time integral of the quantity calculated over a time specific to the device or determined by test equipment.
Time integral 5	Used as a base for load profile recording: Time integral of the quantity calculated from the beginning of the current recording interval to the instantaneous time point for recording period 1, see Table 72.
Time integral 6	Used as a base for load profile recording: Time integral of the quantity calculated from the beginning of the current recording interval to the instantaneous time point for recording period 2, see Table 72.
Time integral 7	Time integral of the quantity calculated from the origin (first start of measurement) up to the end of the last recording period with recording period 1, see Table 72.
Time integral 8	Time integral of the quantity calculated from the origin (first start of measurement) up to the end of the last recording period with recording period 2, see Table 72.
Time integral 9	Time integral of the quantity calculated from the beginning of the current billing period up to the end of the last recording period with recording period 1, see Table 72.
Time integral 10	Time integral of the quantity calculated from the beginning of the current billing period up to the end of the last recording period with recording period 2, see Table 72.
Under limit values	Values under a certain threshold (for example dips).
Over limit values	Values above a certain threshold (for example swells).
Missing values	Values considered as missing (for example interruptions).

© Copyright 1997-2020 DLMS User Association

7.5.2.2 Use of value group D for identification of other objects

For identifiers of electricity related general purpose objects see 7.5.5.1.

7.5.3 Value group E codes – Electricity

7.5.3.1 General

The following clauses define the use of value group E for identifying further classification or processing the measurement quantities defined by values in value groups A to D. The various classifications and processing methods are exclusive.

7.5.3.2 Tariff rates

Table 67 shows the use of value group E for identification of tariff rates typically used for energy (consumption) and demand quantities.

Table 67 – Value group E codes – Electricity – Tariff rates

Value group E codes – Electricity – Tariff rates (A = 1)	
0	Total
1	Rate 1
2	Rate 2
3	Rate 3
…	…
63	Rate 63
128…254	Manufacturer specific codes
All other	Reserved

7.5.3.3 Harmonics

Table 68 shows the use of value group E for the identification of harmonics of instantaneous values of voltage, current or active power.

Table 68 – Value group E codes – Electricity – Harmonics

Value group E codes – Electricity – Measurement of harmonics of voltage, current or active power (A = 1, C = 12, 32, 52, 72, 92, 11, 31, 51, 71, 90, 91, 15, 35, 55, 75, D = 7, 24, 56)	
0	Total (fundamental + all harmonics)
1	1st harmonic (fundamental)
2	2nd harmonic
…	*n*th harmonic
120	120th harmonic
124	Total Harmonic Distortion (THD) [a]
125	Total Demand Distortion (TDD) [b]
126	All harmonics [c]
127	All harmonics to nominal value ratio [d]
128…254	Manufacturer specific codes

© Copyright 1997-2020 DLMS User Association

Value group E codes – Electricity – Measurement of harmonics of voltage, current or active power (A = 1, C = 12, 32, 52, 72, 92, 11, 31, 51, 71, 90, 91, 15, 35, 55, 75, D = 7, 24, 56)	
All other	Reserved
a	THD is calculated as the ratio of the square root of the sum of the squares of each harmonic to the value of the fundamental quantity, expressed as a percent of the value of the fundamental.
b	TDD is calculated as the ratio of the square root of the sum of the squares of each harmonic to the maximum value of the fundamental quantity, expressed as percent of the maximum value of the fundamental.
c	Calculated as the square root of the sum of the squares of each harmonic.
d	This is calculated as ratio of the square root of the sum of the squares of each harmonic, to the nominal value of the fundamental quantity, expressed as percent of the nominal value of the fundamental.

7.5.3.4 Phase angles

Table 69 shows the use of value group E for identification of phase angles.

Table 69 – Value group E codes – Electricity – Extended phase angle measurement

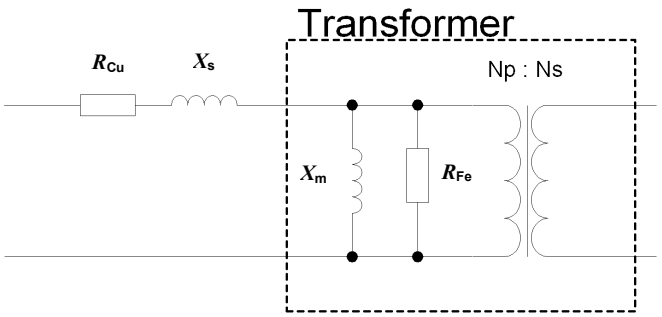
Value group E codes – Electricity – Extended phase angle measurement (A = 1, C = 81; D = 7)								
Angle	**U(L1)**	**U(L2)**	**U(L3)**	**I(L1)**	**I(L2)**	**I(L3)**	**I(L0)**	**<= From**
U(L1)	(00)	01	02	04	05	06	07	
U(L2)	10	(11)	12	14	15	16	17	
U(L3)	20	21	(22)	24	25	26	27	
I(L1)	40	41	42	(44)	45	46	47	
I(L2)	50	51	52	54	(55)	56	57	
I(L3)	60	61	62	64	65	(66)	67	
I(L0)	70	71	72	74	75	76	(77)	
^ To (reference)								

7.5.3.5 Transformer and line loss quantities

Table 70 shows the meaning of value group E for the identification of transformer and line loss quantities. The use of value group D shall be according to Table 66, the use of value group F shall be according to Table 118. For these quantities, no tariffication is available.

© Copyright 1997-2020 DLMS User Association

The model of the line and the transformer used for loss calculation is shown on Figure 35.



Legend:

- R_{Cu} Line resistance losses, OBIS code 1.x.0.10.2.VZ
- X_s Line reactance losses, OBIS code 1.x.0.10.3.VZ
- X_m Transformer magnetic losses, OBIS code 1.x.0.10.0.VZ
- R_{Fe} Transformer iron losses, OBIS code 1.x.0.10.1.VZ
- N_p Number of turns on the primary side of the transformer
- N_s Number of turns on the secondary side of the transformer

NOTE Serial elements of the transformer are normally low compared to that of the line, therefore they are not considered here.

Figure 35 – Model of the line and the transformer for calculation of loss quantities

Table 70 – Value group E codes – Electricity – Transformer and line losses

Value group E codes – Electricity – Transformer and line losses (A = 1, C = 83)			
E=	**Quantity**	**Formula**	**Quadrant / comment**
1	ΣL_i Active line losses+	On Load Active, positive OLA+ = (CuA_1+) + (CuA_2+) + (CuA_3+)	QI+QIV
2	ΣL_i Active line losses–	On Load Active, negative OLA– = (CuA_1–) + (CuA_2–) + (CuA_3–)	QII+QIII
3	ΣL_i Active line losses	On Load Active OLA = (CuA_1) + (CuA_2) + (CuA_3)	QI+QII+QIII+QIV
4	ΣL_i Active transformer losses+	No Load Active, positive NLA+ = (FeA_1+) + (FeA_2+) + (FeA_3+)	QI+QIV
5	ΣL_i Active transformer losses–	No Load active, negative NLA– = (FeA_1–) + (FeA_2–) + (FeA_3–)	QII+QIII
6	ΣL_i Active transformer losses	No Load Active NLA = (FeA_1) + (FeA_2) + (FeA_3)	QI+QII+QIII+QIV
7	ΣL_i Active losses+	Total Losses Active, positive TLA+ = (OLA+) + (NLA+)	QI+QIV
8	ΣL_i Active losses–	Total Losses Active, negative TLA– = (OLA–) + (NLA–)	QII+QIII
9	ΣL_i Active losses	Total Losses Active TLA = OLA + NLA = TLA_1 + TLA_2 + TLA_3	QI+QII+QIII+QIV
10	ΣL_i Reactive line losses+	On Load Reactive, positive OLR+ = (CuR_1+) + (CuR_2+) + (CuR_3+)	QI+QII
11	ΣL_i Reactive line losses–	On Load Reactive, negative OLR– = (CuR_1–) + (CuR_2–) + (CuR_3–)	QIII+QIV

© Copyright 1997-2020 DLMS User Association

Value group E codes – Electricity – Transformer and line losses (A = 1, C = 83)			
E=	**Quantity**	**Formula**	**Quadrant / comment**
12	ΣL_i Reactive line losses	On Load Reactive OLR = (CuR_1) + (CuR_2) + (CuR_3)	QI+QII+QIII+QIV
13	ΣL_i Reactive transformer losses+	No Load reactive, positive NLR+ = (FeR_1+) + (FeR_2+) + (FeR_3+)	QI+QII
14	ΣL_i Reactive transformer losses–	No Load Reactive, negative NLR– = (FeR_1–) + (FeR_2–) + (FeR_3–)	QIII+QIV
15	ΣL_i Reactive transformer losses	No Load Reactive NLR = (FeR_1) + (FeR_2) + (FeR_3)	QI+QII+QIII+QIV
16	ΣL_i Reactive losses+	Total Losses Reactive, positive TLR+ = (OLR+) + (NLR+)	QI+QII
17	ΣL_i Reactive losses–	Total Losses Reactive, negative TLR– = (OLR–) + (NLR–)	QIII+QIV
18	ΣL_i Reactive losses	Total Losses Reactive TLR = OLR + NLR = TLR_1 + TLR_2 + TLR_3	QI+QII+QIII+QIV
19	Total transformer losses with normalized R_{Fe} = 1 MΩ	U^2h 1/R_{Fe} x (U^2h_{L1}+U^2h_{L2}+U^2h_{L3})	QI+QII+QIII+QIV
20	Total line losses with normalized R_{Cu} = 1 Ω	I^2h R_{Cu} x (I^2h_{L1} + I^2h_{L2} + I^2h_{L3})	QI+QII+QIII+QIV
21	Compensated active gross+	CA+ = (A+) + (TLA+)	QI+QIV; A+ is the quantity A = 1, C = 1
22	Compensated active net+	CA+ = (A+) - (TLA+)	QI+QIV
23	Compensated active gross–	CA– = (A–) + (TLA–)	QII+QIII, A– is the quantity A = 1, C = 2
24	Compensated active net–	CA – = (A–) - (TLA–)	QII+QIII
25	Compensated reactive gross+	CR+ = (R+) + (TLR+)	QI+QII; R+ is the quantity A = 1, C = 3
26	Compensated reactive net+	CR+ = (R+) - (TLR+)	QI+QII
27	Compensated reactive gross–	CR– = (R–) + (TLR–)	QIII+QIV;R– is the quantity A = 1, C = 4
28	Compensated reactive net–	CR– = (R–) - (TLR–)	QIII+QIV
29	Reserved		
30	Reserved		
31	L_1 Active line losses+	CuA_1+ = I^2h_{L1} x R_{Cu}	QI+QIV R_{Cu} is the serial resistive element of the line loss, OBIS code 1.x.0.10.2.VZ
32	L_1 Active line losses–	CuA_1– = I^2h_{L1} x R_{Cu}	QII+QIII
33	L_1 Active line losses	CuA_1 = I^2h_{L1} x R_{Cu}	QI+QII+QIII+QIV
34	L_1 Active transformer losses+	FeA_1+ = U^2h_{L1}/R_{Fe}	QI+QIV R_{Fe} is the parallel resistive element of the transformer loss, OBIS code 1.x.0.10.1.VZ
35	L_1 Active transformer losses–	FeA_1– = U^2h_{L1}/R_{Fe}	QII+QIII
36	L_1 Active transformer losses	FeA_1 = U^2h_{L1}/R_{Fe}	QI+QII+QIII+QIV
37	L_1 Active losses+	TLA_1+ = (CuA_1+) + (FeA_1+)	QI+QIV
38	L_1 Active losses–	TLA_1– = (CuA_1–) + (FeA_1–)	QII+QIII
39	L_1 Active losses	TLA_1 = CuA_1 + FeA_1	QI+QII+QIII+QIV
40	L_1 Reactive line losses+	CuR_1+ = I^2hL_1 x X_s	QI+QII X_s is the serial reactive element of

© Copyright 1997-2020 DLMS User Association

Value group E codes – Electricity – Transformer and line losses (A = 1, C = 83)			
E=	**Quantity**	**Formula**	**Quadrant / comment**
			the line loss, OBIS code 1.x.0.10.3.VZ
41	L_1 Reactive line losses–	$CuR_1 - = I^2 h_{L1} \times X_s$	QIII+QIV
42	L_1 Reactive line losses	$CuR_1 = I^2 h_{L1} \times X_s$	QI+QII+QIII+QIV
43	L_1 Reactive transformer losses+	$FeR_1 + = U^2 h_{L1} / X_m$	QI+QII X_m is the parallel reactive element of the transformer loss, OBIS code 1.x.0.10.0.VZ
44	L_1 Reactive transformer losses–	$FeR_1 - = U^2 h_{L1} / X_m$	QIII+QIV
45	L_1 Reactive transformer losses	$FeR_1 = U^2 h_{L1} / X_m$	QI+QII+QIII+QIV
46	L_1 Reactive losses+	$TLR_1 + = (CuR_1 +) + (FeR_1 +)$	QI+QII
47	L_1 Reactive losses–	$TLR_1 - = (CuR_1 -) + (FeR_1 -)$	QIII+QIV
48	L_1 Reactive losses	$TLR_1 = CuR_1 + FeR_1$	QI+QII+QIII+QIV
49	L_1 Ampere-squared hours	$A^2 h_{L1}$	QI+QII+QIII+QIV
50	L_1 Volt-squared hours	$V^2 h_{L1}$	QI+QII+QIII+QIV
51	L_2 Active line losses+	$CuA_2 + = I^2 h_{L2} \times R_{Cu}$	QI+QIV R_{Cu} is the serial resistive element of the line loss, OBIS code 1.x.0.10.2.VZ
52	L_2 Active line losses–	$CuA_2 - = I^2 h_{L2} \times R_{Cu}$	QII+QIII
53…70	L_2 quantities, (See 33…48)		
71	L_3 Active line losses +	$CuA_3 + = I^2 h_{L3} \times R_{Cu}$	QI+QIV R_{Cu} is the serial resistive element of the line loss, OBIS code 1.x.0.10.2.VZ
72	L_3 Active line losses -	$CuA_3 - = I^2 h_{L3} \times R_{Cu}$	QII+QIII
73…90	L_3 quantities (See 33…48)		
91… 255	Reserved		
NOTE In this table, no manufacturer specific range is available.			

7.5.3.6 UNIPEDE voltage dips

Table 71 shows the use of value group E for the identification of voltage dips according to the UNIPEDE classification.

© Copyright 1997-2020 DLMS User Association

Table 71 – Value group E codes – Electricity – UNIPEDE voltage dips

Value group E codes – Electricity – UNIPEDE voltage dips measurement (A = 1, C = 12, 32, 52, 72, 124…126 D = 32)							
Depth in % of U_n	Residual voltage U in % of U_n	Duration Δt s					
		0,01 < $\Delta t \leq$ 0,1	0,1 < $\Delta t \leq$ 0,5	0,5 < $\Delta t \leq$ 1	1 < $\Delta t \leq$ 3	3 < $\Delta t \leq$ 20	20 < $\Delta t \leq$ 60
10%…<15%	90 > $U \geq$ 85	00	01	02	03	04	05
15%…<30%	85 > $U \geq$ 70	10	11	12	13	14	15
30%…<60%	70 > $U \geq$ 40	20	21	22	23	24	25
60%…<90%	40 > $U \geq$ 10	30	31	32	33	34	35
90%…<100%	10 > $U \geq$ 0	40	41	42	43	44	45
NOTE These *dip classes* form a subset of the classes defined in IEC TR 61000-2-8, Table 2.							

7.5.3.7 Use of value group E for the identification of other objects

For identifiers of electricity related general purpose objects see 7.5.5.1.

7.5.4 Value group F codes – Electricity

7.5.4.1 Billing periods

Value group F specifies the allocation to different billing periods (sets of historical values) for the objects with following codes:

- value group A: 1;
- value group C: as defined in Table 65;
- value group D:
  - 0: Billing period average (since last reset);
  - 1, 2, 3, 6: (Cumulative) minimum / maximum 1;
  - 8, 9, 10: Time integral 1 / 2 / 3;
  - 11, 12, 13, 16: (Cumulative) minimum / maximum 2;
  - 21, 22, 23, 26: (Cumulative) minimum / maximum 3;

There are two billing period schemes available (for example to store weekly and monthly values). For each billing period scheme, the following general purpose objects are available:

- billing period counter;
- number of available billing periods;
- time stamp of most recent and historical billing periods;
- billing period length.

For OBIS codes see Table 72. For additional information, see 6.2.2 and 7.11.3.

© Copyright 1997-2020 DLMS User Association

7.5.4.2 Multiple thresholds

Value group F is also used to identify several thresholds for the same quantity, identified with the following codes:

- value group A = 1;
- value group C = 1…20, 21…40, 41…60, 61…80, 82, 84…89, 90… 92;
- value group D = 31, 35, 39 (under limit, over limit and missing thresholds);
- value group F = 0…99.

NOTE All quantities monitored are instantaneous values: D = 7 or D = 24.

When multiple thresholds are identified by value group F, then the Under limit / Over limit / Missing Occurrence counter / Duration / Magnitude quantities relative to a threshold are identified with the same value in value group F. In this case, value group F cannot be used to identify values relative to billing period. However, such values can be held by “Profile generic” objects.

Example:

- Over limit threshold #1 for current in any phase is identified with OBIS code 1-0:11.35.0*0;
- Over limit duration above threshold # 1 for current in any phase is identified with OBIS code 1-0:11.37.0*0.

To avoid ambiguity, value group F cannot be used to identify historical values of Under limit / Over limit / Missing Occurrence counter / Duration / Magnitude quantities. For historical values of these quantities “Profile generic” objects can be used and values related to previous billing periods can be accessed using selective access.

7.5.5 OBIS codes – Electricity

7.5.5.1 General and service entry objects – Electricity

Table 72 specifies the OBIS codes for electricity related general and service entry objects.

Table 72 – OBIS codes for general and service entry objects – Electricity

General and service entry objects – Electricity	OBIS code					
	A	B	C	D	E	F
Free ID-numbers for utilities						
Complete combined electricity ID	1	*b*	0	0		
Electricity ID 1	1	*b*	0	0	0	
…	…	…	…	…	…	
Electricity ID 10	1	*b*	0	0	9	
Billing period values/reset counter entries (First billing period scheme if there are more than one)						
Billing period counter (1) in a recent billing period VZ	1	*b*	0	1	0	VZ or 255
Billing period counter (1) in a recent billing period	1	*b*	0	1	0	101-125
Billing period counters (1) in unspecified number of recent billing periods	1	*b*	0	1	0	126
Number of available billing periods (1)	1	*b*	0	1	1	255
Time stamp of the most recent billing period (1)	1	*b*	0	1	2	255
Time stamp of the billing period (1) VZ (last reset)	1	*b*	0	1	2	VZ
Time stamp of the billing period (1) VZ_{-1}	1	*b*	0	1	2	VZ_{-1}
…	…	…	…	…	…	…
Time stamp of the billing period (1) VZ_{-n}	1	*b*	0	1	2	VZ_{-n}
Time stamp of the billing period (1) in a recent billing period	1	*b*	0	1	2	101-125

© Copyright 1997-2020 DLMS User Association

General and service entry objects – Electricity	OBIS code					
	A	B	C	D	E	F
Time stamp of the billing period (1) in unspecified number of recent billing periods	1	*b*	0	1	2	126
Billing period values/reset counter entries (Second billing period scheme)						
Billing period counter (2)	1	*b*	0	1	3	VZ or 255
Billing period counter (2) in a recent billing period	1	*b*	0	1	3	101-125
Billing period counters (2) in unspecified number of recent billing periods	1	*b*	0	1	3	126
Number of available billing periods (2)	1	*b*	0	1	4	255
Time stamp of the most recent billing period (2)	1	*b*	0	1	5	255
Time stamp of the billing period (2) VZ (last reset)	1	*b*	0	1	5	VZ
Time stamp of the billing period (2) VZ_{-1}	1	*b*	0	1	5	VZ_{-1}
…	…	…	…	…	…	…
Time stamp of the billing period (2) VZ_{-n}	1	*b*	0	1	5	VZ_{-n}
Time stamp of the billing period (2) in a recent billing period	1	*b*	0	1	5	101-125
Time stamp of the billing period (2) in unspecified number of recent billing periods	1	*b*	0	1	5	126
Program entries						
Active firmware identifier (Previously: Configuration program version number)	1	*b*	0	2	0	
Parameter record number	1	*b*	0	2	1	
Parameter record number, line 1	1	*b*	0	2	1	1
Reserved for future use	1	*b*	0	2	1	2… 127
Manufacturer specific	1	*b*	0	2	1	128… 254
Time switch program number	1	*b*	0	2	2	
RCR program number	1	*b*	0	2	3	
Meter connection diagram ID	1	*b*	0	2	4	
Passive calendar name	1	*b*	0	2	7	
Active firmware signature	1	*b*	0	2	8	
Output pulse values or constants NOTE For units, see 4.3.2.						
Active energy, metrological LED	1	*b*	0	3	0	
Reactive energy, metrological LED	1	*b*	0	3	1	
Apparent energy, metrological LED	1	*b*	0	3	2	
Active energy, output pulse	1	*b*	0	3	3	
Reactive energy, output pulse	1	*b*	0	3	4	
Apparent energy, output pulse	1	*b*	0	3	5	
Volt-squared hours, metrological LED	1	*b*	0	3	6	
Ampere-squared hours, metrological LED	1	*b*	0	3	7	
Volt-squared hours, output pulse	1	*b*	0	3	8	
Ampere-squared hours, output pulse	1	*b*	0	3	9	
Ratios						
Reading factor for power	1	*b*	0	4	0	
Reading factor for energy	1	*b*	0	4	1	
Transformer ratio – current (numerator) [a]	1	*b*	0	4	2	VZ
Transformer ratio – voltage (numerator) [a]	1	*b*	0	4	3	VZ
Overall transformer ratio (numerator) [a]	1	*b*	0	4	4	VZ

© Copyright 1997-2020 DLMS User Association

General and service entry objects – Electricity	OBIS code					
	A	B	C	D	E	F
Transformer ratio – current (denominator) [a]	1	*b*	0	4	5	VZ
Transformer ratio – voltage (denominator) [a]	1	*b*	0	4	6	VZ
Overall transformer ratio (denominator) [a]	1	*b*	0	4	7	VZ
Demand limits for excess consumption metering						
Reserved for Germany	1	*b*	0	5		
Nominal values						
Voltage	1	*b*	0	6	0	
Basic/nominal current	1	*b*	0	6	1	
Frequency	1	*b*	0	6	2	
Maximum current	1	*b*	0	6	3	
Reference voltage for power quality measurement	1	*b*	0	6	4	VZ
Reference voltage for aux. power supply	1	*b*	0	6	5	
Input pulse values or constants [b] NOTE For units, see 4.3.2.						
Active energy	1	*b*	0	7	0	
Reactive energy	1	*b*	0	7	1	
Apparent energy	1	*b*	0	7	2	
Volt-squared hours	1	*b*	0	7	3	
Ampere-squared hours	1	*b*	0	7	4	
Unitless quantities	1	*b*	0	7	5	
Active energy, export	1	*b*	0	7	10	
Reactive energy, export	1	*b*	0	7	11	
Apparent energy, export	1	*b*	0	7	12	
Measurement period- / recording interval- / billing period duration						
Measurement period 1, for averaging scheme 1	1	*b*	0	8	0	VZ
Measurement period 2, for averaging scheme 2	1	*b*	0	8	1	VZ
Measurement period 3, for instantaneous value	1	*b*	0	8	2	VZ
Measurement period 4, for test value	1	*b*	0	8	3	VZ
Recording interval 1, for load profile	1	*b*	0	8	4	VZ
Recording interval 2, for load profile	1	*b*	0	8	5	VZ
Billing period (Billing period 1 if there are two billing period schemes)	1	*b*	0	8	6	VZ
Billing period 2	1	*b*	0	8	7	VZ
Measurement period 4, for harmonics measurement	1	*b*	0	8	8	VZ
Time entries						
Time expired since last end of billing period (First billing period scheme if there are more than one)	1	*b*	0	9	0	
Local time	1	*b*	0	9	1	
Local date	1	*b*	0	9	2	
Reserved for Germany	1	*b*	0	9	3	
Reserved for Germany	1	*b*	0	9	4	
Week day (0…7)	1	*b*	0	9	5	
Time of last reset (First billing period scheme if there are more than one)	1	*b*	0	9	6	
Date of last reset (First billing period scheme if there are more than one)	1	*b*	0	9	7	
Output pulse duration	1	*b*	0	9	8	
Clock synchronization window	1	*b*	0	9	9	
Clock synchronization method	1	*b*	0	9	10	

© Copyright 1997-2020 DLMS User Association

General and service entry objects – Electricity	OBIS code					
	A	B	C	D	E	F
Clock time shift limit (default value: s)	1	*b*	0	9	11	
Billing period reset lockout time (First billing period scheme if there are more than one)	1	*b*	0	9	12	
Second billing period scheme						
Time expired since last end of billing period	1	*b*	0	9	13	
Time of last reset	1	*b*	0	9	14	
Date of last reset	1	*b*	0	9	15	
Billing period reset lockout time	1	*b*	0	9	16	
Coefficients						
Transformer magnetic losses, X_m	1	*b*	0	10	0	VZ
Transformer iron losses, R_{Fe}	1	*b*	0	10	1	VZ
Line resistance losses, R_{Cu}	1	*b*	0	10	2	VZ
Line reactance losses, X_s	1	*b*	0	10	3	VZ
Measurement methods						
Algorithm for active power measurement	1	*b*	0	11	1	
Algorithm for active energy measurement	1	*b*	0	11	2	
Algorithm for reactive power measurement	1	*b*	0	11	3	
Algorithm for reactive energy measurement	1	*b*	0	11	4	
Algorithm for apparent power measurement	1	*b*	0	11	5	
Algorithm for apparent energy measurement	1	*b*	0	11	6	
Algorithm for power factor calculation	1	*b*	0	11	7	
Metering point ID (electricity related)						
Metering point ID 1 (electricity related)	1	0	96	1	0	
.....						
Metering point ID 10 (electricity related)	1	0	96	1	9	
Internal operating status, electricity related						
Internal operating status, global [c]	1	*b*	96	5	0	
Internal operating status (status word 1)	1	*b*	96	5	1	
Internal operating status (status word 2)	1	*b*	96	5	2	
Internal operating status (status word 3)	1	*b*	96	5	3	
Internal operating status (status word 4)	1	*b*	96	5	4	
Meter started status flag	1	*b*	96	5	5	
Electricity related status data						
Status information missing voltage	1	0	96	10	0	
Status information missing current	1	0	96	10	1	
Status information current without voltage	1	0	96	10	2	
Status information auxiliary power supply	1	0	96	10	3	
Manufacturer specific [d]	1	*b*	96	50	*e*	*f*
.....	...	...	...	...	...	...
Manufacturer specific	1	*b*	96	99	*e*	*f*

a	If a transformer ratio is expressed as a fraction the ratio is numerator, divided by denominator. If the transformer ratio is expressed by an integer or real figure, only the numerator is used.
b	The codes for export active, reactive and apparent energy shall be used only if meters measuring import energy and meters measuring export energy are connected to the pulse inputs.
c	Global status words with E = 0 contain the individual status words E = 1...5. The contents of the status words are not defined In this Technical Report.
d	The range D = 50...99 is available for identifying objects, which are not represented by another defined code, but need representation on the display as well. If this is not required, the range D = 128...254 should be used.

© Copyright 1997-2020 DLMS User Association

It should be noted, that some of the codes above are normally used for display purposes only, as the related data items are attributes of objects having their own OBIS name. See Clause 4.

7.5.5.2 Error register objects – Electricity

Table 73 specifies the OBIS codes for electricity related error register objects.

Table 73 – OBIS codes for error register objects – Electricity

Error register objects – Electricity	OBIS code					
	A	B	C	D	E	F
Error register	1	*b*	97	97	*e*	
NOTE The information to be included in the error objects is not defined in this document.						

7.5.5.3 List objects – Electricity

Table 74 specifies the OBIS codes for electricity related list objects.

Table 74 – OBIS codes for list objects – Electricity

List objects – Electricity	OBIS code					
	A	B	C	D	E	F
Electricity related data of billing period (with billing period scheme 1 if there are two schemes available)	1	*b*	98	1	*e*	255 [a]
Electricity related data of billing period (with billing period scheme 2)	1	*b*	98	2	*e*	255 [a]
[a] F = 255 means a wildcard here. See 7.11.3.						

7.5.5.4 Data profile objects – Electricity

Electricity related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data. The OBIS codes are specified in Table 75.

Table 75 – OBIS codes for data profile objects – Electricity

Data profile objects – Electricity	OBIS code					
	A	B	C	D	E	F
Load profile with recording period 1	1	*b*	99	1	*e*	
Load profile with recording period 2	1	*b*	99	2	*e*	
Load profile during test	1	*b*	99	3	0	
Dips voltage profile	1	*b*	99	10	1	
Swells voltage profile	1	*b*	99	10	2	
Cuts voltage profile	1	*b*	99	10	3	
Voltage harmonic profile	1	*b*	99	11	n^{th}	
Current harmonic profile	1	*b*	99	12	n^{th}	
Voltage unbalance profile	1	*b*	99	13	0	
Power quality	1	*b*	99	14	0	
Power failure event log	1	*b*	99	97	*e*	
Event log	1	*b*	99	98	*e*	

© Copyright 1997-2020 DLMS User Association

Data profile objects – Electricity	OBIS code					
	A	B	C	D	E	F
Certification data log	1	*b*	99	99	*e*	

7.5.5.5 Register table objects – Electricity

Register tables - identified with a single OBIS code - are defined to hold a number of values of the same type. The OBIS codes are specified in Table 76.

Table 76 – OBIS codes for Register table objects – Electricity

Register table objects – Electricity	OBIS code					
	A	B	C	D	E	F
UNIPEDE voltage dips, any phase	1	*b*	12	32		
UNIPEDE voltage dips, L_1	1	*b*	32	32		
UNIPEDE voltage dips, L_2	1	*b*	52	32		
UNIPEDE voltage dips, L_3	1	*b*	72	32		
Extended angle measurement	1	*b*	81	7		
General use, electricity related	1	*b*	98	10	*e*	

© Copyright 1997-2020 DLMS User Association

7.6 Heat Cost Allocators (Value group A = 4)

7.6.1 General

NOTE The following introductory text is from EN 13757-1:2014, 11.3.2.1.

Heat Cost Allocators (HCAs) are mounted on radiators in the area to be monitored. The HCA should be mounted with in free air and radiators should not be enclosed. There will normally also be multiple HCAs, even for a single customer. This makes at, the present, direct connection to all HCAs using a two way connections an infeasible solution. It is nevertheless important, that data coming from a (number of) HCAs (via a concentrator) can be handled in the same way as data from other meters for remote reading.

This subclause 7.6 describes the naming of objects carrying HCA information in a COSEM environment. The words used in this clause are those used in EN 834:1994, the corresponding media standard.

The output from an HCA is "the temperature integral with respect to time", and it is only a relative sum. The main parameter from a HCA is this integral. Time series of this integral may be stored in the HCA for later readout. Other media related information available from a HCA are temperature and rating factors.

7.6.2 Value group C codes – HCA

The name of the different objects in the table for HCA objects corresponds to the name used in the relevant standard, EN 834:1994. The OBIS codes are specified in Table 77.

Table 77 – Value group C codes – HCA

Value group C codes – HCA (A = 4)	
0	General purpose objects [a]
1	Unrated integral [b]
2	Rated integral [c]
3	Radiator surface temperature [d]
4	Heating medium temperature, t_m
5	Flow (forward) temperature, t_V
6	Return temperature, t_R
7	Room temperature, t_L
93	Consortia specific identifiers, see Table 58.
94	Country specific identifiers, see Table 59.
96	General and service entry objects– HCA (See 7.6.4.1).
97	Error register objects – HCA (See 7.6.4.2).
98	List objects – HCA
99	Data profile objects – HCA (See 7.6.4.3)
128...199, 240	Manufacturer specific codes
All other	Reserved
[a]	Settings like time constant, thresholds etc. See the table of object codes in EN 13757-1:2014, 11.3.2.2.
[b]	Readout prior to compensation as specified in EN 834:1994.
[c]	Readout after compensation as specified in EN 834:1994.
[d]	Temperature measured prior to any rating

© Copyright 1997-2020 DLMS User Association

Value group C codes – HCA (A = 4)	
NOTE 1	The radiator surface (C = 3) temperature and the heating media (C=4) temperature are mutually exclusive.
NOTE 2	The forward flow (C = 5) and reverse flow (C = 6) temperatures are exclusive to the radiator surface (C = 3) temperature.
NOTE 3	The room temperature measurement (C = 7) is always be accompanied by either a radiator surface (C = 3) temperature, a heating media (C = 4) temperature or a pair of forward / return flow (C = 5 / C = 6) temperatures.

7.6.3 Value group D codes – HCA

This value group specifies the result of processing a *Quantity* according to a specific algorithm for Heat Cost Allocator related values. The OBIS codes are specified inTable 78.

Table 78 – Value group D codes – HCA

Value group D codes – HCA (A = 4, C <> 0, 96...99)	
0	Current value
1	Integral value over measurement periods (Periodical value) [a]
2	Integral value relative to billing periods: Set date value
3	Integral value relative to billing periods: Billing date value
4	Minimum of value
5	Maximum of value
6	Test value [b]
All other	Reserved
[a]	A set of values periodically stored (this may be once or twice a month)
[b]	A value specially processed for test purpose. This may be due to an increased precision of the data, or to a faster (but less precise) processing of data.

7.6.4 Value group E codes – HCA

Table 79 shows the use of value group E for identification of tariff rates typically used for energy (consumption) and demand quantities.

Table 79 – Value group E codes – HCA

Value group E codes – HCA	
0	Total
1	Rate 1
2	Rate 2
3	Rate 3
...	…
9	Rate 9
128...254	Manufacturer specific codes
All other	Reserved

© Copyright 1997-2020 DLMS User Association

7.6.5 OBIS codes – HCA

7.6.5.1 General and service entry objects – HCA

Table 80 specifies OBIS codes for heat cost allocator related general and service entry objects.

Table 80 – OBIS codes for general and service entry objects – HCA

General and service entry objects – HCA	OBIS code					
	A	B	C	D	E	F
Free ID-numbers for utilities						
Complete combined ID	4	*b*	0	0		
ID 1	4	*b*	0	0	0	
…			…	…	…	
ID 10	4	*b*	0	0	9	
Storage information						
Status (VZ) of the historical value counter	4	*b*	0	1	1	
Number of available historical values	4	*b*	0	1	2	
Set date (target date)	4	*b*	0	1	10	
Billing date	4	*b*	0	1	11	
Configuration						
Program version no.	4	*b*	0	2	0	
Firmware version no.	4	*b*	0	2	1	
Software version no.	4	*b*	0	2	2	
Device measuring principle						
Device measuring principle [a]	4	*b*	0	2	3	
Conversion factors						
Resulting rating factor, K	4	*b*	0	4	0	
Thermal output rating factor, K_Q	4	*b*	0	4	1	
Thermal coupling rating factor overall, K_C	4	*b*	0	4	2	
Thermal coupling rating factor room side, K_{CR}	4	*b*	0	4	3	
Thermal coupling rating factor heater side, K_{CH}	4	*b*	0	4	4	
Low temperature rating factor, K_T	4	*b*	0	4	5	
Display output scaling factor	4	*b*	0	4	6	
Threshold values						
Start temperature threshold	4	*b*	0	5	10	
Difference temperature threshold	4	*b*	0	5	11	
Period information						
Measuring period for average value	4	*b*	0	8	0	
Recording interval for consumption profile	4	*b*	0	8	4	
Billing period	4	*b*	0	8	6	
Time entries						
Local time	4	*b*	0	9	1	
Local date	4	*b*	0	9	2	
Time stamp (local time) of the most recent billing period [b]	4	*b*	0	9	3	
Manufacturer specific [c]	4	*b*	96	50	*e*	*f*

© Copyright 1997-2020 DLMS User Association

General and service entry objects – HCA	OBIS code					
	A	B	C	D	E	F
..... Manufacturer specific	4	*b*	96	99	*e*	*f*
[a] This is an object of the type 'Data' enumerated, (0) single sensor, (1) single sensor + start sensor, (2) dual sensor, (3) triple sensor.						
[b] In case of billing period schemes absence or event triggered, commonly calculated from local date and local time information.						
[c] The range D = 50…99 is available for identifying objects, which are not represented by another defined code, but need representation on the display as well. If this is not required, the range D = 128…254 should be used.						

7.6.5.2 Error register objects – HCA

Table 81 specifies OBIS codes for HCA related error register objects.

Table 81 – OBIS codes for error register objects – HCA

Error register objects – HCA	OBIS code					
	A	B	C	D	E	F
Error registers	4	*b*	97	97	*e*	

7.6.5.3 List objects - HCA

Table 82 specifies the OBIS codes for HCA related list objects.

Table 82 – OBIS codes for list objects – HCA

List objects – HCA	OBIS code					
	A	B	C	D	E	F
HCA related data of billing period (with billing period scheme 1 if there are two schemes available)	4	*b*	98	1	*e*	255 [a]
HCA related data of billing period (with billing period scheme 2)	4	*b*	98	2	*e*	255 [a]
[a] F = 255 means a wildcard here. See 7.11.3.						

7.6.5.4 Data profile objects – HCA

HCA related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data. The OBIS codes are specified in Table 83.

Table 83 – OBIS codes for data profile objects – HCA

Data profile objects – HCA	OBIS code					
	A	B	C	D	E	F
Data profile objects	4	*b*	99	1	*e*	

7.6.5.5 OBIS codes for HCA related objects (examples)

Table 84 specifies examples for OBIS codes of HCA related objects.

© Copyright 1997-2020 DLMS User Association

Table 84 – OBIS codes for HCA related objects (examples)

HCA related objects	OBIS code					
	A	B	C	D	E	F
Consumption						
Current unrated integral	4	*b*	1	0	0	
Current rated integral	4	*b*	2	0	0	
Rated integral, last set date	4	*b*	2	2	0	^{V}Z
Unrated integral, previous billing date	4	*b*	1	3	0	$^{V}Z-1$
Rated integral, two most recent periodical values	4	*b*	2	1	0	102
Monitoring values						
Radiator temperature, current value	4	*b*	3	0		
Radiator temperature, minimum value	4	*b*	3	4		
Radiator temperature, maximum value	4	*b*	3	5		
Flow temperature, test value	4	*b*	5	6		
Room temperature, current value	4	*b*	7	0		
Room temperature, minimum value	4	*b*	7	4		
Room temperature, maximum value	4	*b*	7	5		

© Copyright 1997-2020 DLMS User Association

7.7 Thermal energy (Value group A = 5 or A = 6)

7.7.1 General

This section describes the naming of objects carrying Thermal energy meter information in a COSEM environment. It covers the handling of heat, as well as cooling. The media specific terms used in this clause are those used in EN 1434-1:2015, EN 1434-2:2015 and parts of the corresponding media standard. The output from a Thermal energy meter is "the integral of power, i.e. the enthalpy difference times the mass flow-rate, with respect to time".

Thermal energy meters can be used for measurement in heating (A=6) or cooling (A=5) systems.

Value group A = 5 has been set aside for metering of cooling specific objects and value group A = 6 for the metering of heat specific objects. The other value groups are identical for heating and cooling.

7.7.2 Value group C codes – Thermal energy

The name of the different objects in the table for heat metering and cooling metering objects corresponds to the name used in EN 1434-1:2015. The OBIS codes are specified in Table 85.

Table 85 – Value group C codes – Thermal energy

Value group C codes – Thermal energy related objects (A = 5 or A = 6)	
0	General and service entry objects – Thermal energy (See 7.7.4.1)
1	Energy
2	Volume
3	Mass [b]
4	Inlet (Flow) volume [a]
5	Inlet (Flow) mass [a]
6	Outlet (Return) volume [a]
7	Outlet (Return) mass [a]
8	Power
9	Flow rate
10	Inlet (Flow) temperature [a]
11	Outlet (Return) temperature [a]
12	Temperature difference [c]
13	Pressure [d]
93	Consortia specific identifiers, see Table 58
94	Country specific identifiers, see Table 59
96	General and service entry objects – Thermal energy (See 7.7.4.1)
97	Error register objects – Thermal energy (See 7.7.4.2)
98	List objects – Thermal energy
99	Data profile objects – Thermal energy (See 7.7.4.3)
128...199, 240	Manufacturer specific codes
All other	Reserved
[a]	In a heating system the term “flow” is equivalent to “inlet” and the term “return” is equivalent to “outlet”

© Copyright 1997-2020 DLMS User Association

Value group C codes – Thermal energy related objects (A = 5 or A = 6)	
[b]	Used when metering steam.
[c]	Will often be available with a higher precision and accuracy than inlet (flow) and outlet (return) temperature.
[d]	Pressure of the media, if measured. Pressure can be retrieved as backup value from a general and service entry object (C=0), if incapable of measurement.

7.7.3 Value group D codes – Thermal energy

This value group specifies the result of processing a *Quantity* according to a specific algorithm for heat or cooling related values. See Table 86.

Table 86 – Value group D codes – Thermal energy

Value group D codes – Thermal energy (A = 5 or A = 6), (C <> 0, 96...99)	
0	Current value
1	Periodical value 1[a]
2	Set date value
3	Billing date value
4	Minimum of value 1
5	Maximum of value 1
6	Test value [b]
7	Instantaneous value [c]
8	Time integral 1 [d]
9	Time integral 2 [c]
10	Current average [f]
11	Last average [g]
12	Periodical value 2 [a]
13	Periodical value 3 [a]
14	Minimum of value 2
15	Maximum of value 2
20	Under limit occurrence counter
21	Under limit duration
22	Over limit occurrence counter
23	Over limit duration
24	Missing data occurrence counter [h]
25	Missing data duration [h]
All other	Reserved
[a]	A set of data that is collected periodically. Recording of data in this way is directly supported by 'profiles'.
[b]	A value specially processed for test purpose. This may be due to an increased precision of the data, or to a faster (but less precise) processing of data.
[c]	An immediate readout from the system, typically with a shorter measuring time than the current value.

© Copyright 1997-2020 DLMS User Association

Value group D codes – Thermal energy (A = 5 or A = 6), (C <> 0, 96...99)	
d	For a current billing period (F = 255): Time integral of the *quantity* calculated from the origin (first start of measurement) to the instantaneous time point. For a historical billing period (F = 0…99): Time integral of the *quantity* calculated from the origin to the end of the billing period given by the billing period code.
e	For a current billing period (F = 255): Time integral of the *quantity* calculated from the beginning of the current billing period to the instantaneous time point. For a historical billing period (F = 0…99): Time integral of the *quantity* calculated over the billing period given by the billing period code.
f	The value of a current demand register.
g	The value of a demand register at the end of the last measurement period.
h	Values considered as missing (for instance due to sensor failure).

7.7.4 Value group E codes – Thermal energy

Table 87 shows the use of value group E for identification of tariff rates typically used for energy (consumption) and demand quantities.

Table 87 – Value group E codes – Thermal Energy – Tariff rates

Value group E codes – Thermal Energy	
0	Total
1	Rate 1
2	Rate 2
3	Rate 3
…	…
9	Rate 9
128…254	Manufacturer specific codes
All other	Reserved

7.7.5 OBIS codes – Thermal energy

7.7.5.1 General and service entry objects – Thermal energy

Table 88 specifies OBIS codes for thermal energy related general and service entry objects.

Table 88 – OBIS codes for general and service entry objects – Thermal energy

General and service entry objects – Thermal energy	OBIS code					
	A	B	C	D	E	F
Free ID-numbers for utilities						
Complete combined ID	5/6	*b*	0	0		
ID 1	5/6	*b*	0	0	0	
…			…	…	…	
ID 10	5/6	*b*	0	0	9	
Storage information						
Status (VZ) of the historical /periodical value counter	5/6	*b*	0	1	1	f

© Copyright 1997-2020 DLMS User Association

General and service entry objects – Thermal energy	OBIS code					
	A	B	C	D	E	F
Status (VZ) of the periodical value counter, period 1	5/6	*b*	0	1	1	1 [f]
Number of available historical / periodical values	5/6	*b*	0	1	2	[f]
Number of available periodical values for period 2	5/6	*b*	0	1	2	2 [f]
Set date	5/6	*b*	0	1	10	
Billing date	5/6	*b*	0	1	11	
Configuration						
Program version	5/6	*b*	0	2	0	
Firmware version	5/6	*b*	0	2	1	
Software version	5/6	*b*	0	2	2	
Meter location (flow or return) [a]	5/6	*b*	0	2	3	
Device version	5/6	*b*	0	2	4	
Serial number of inlet (flow) temperature transducer	5/6	*b*	0	2	10	
Serial number of outlet (return) temperature transducer	5/6	*b*	0	2	11	
Serial number of forward flow transducer	5/6	*b*	0	2	12	
Serial number of return flow transducer	5/6	b	0	2	13	
Conversion factors						
Heat coefficient, k	5/6	*b*	0	4	1	
Pressure (backup value) [b]	5/6	*b*	0	4	2	
Enthalpy [c]	5/6	*b*	0	4	3	
Threshold values						
Threshold value limit for rate 1 [d]	5/6	*b*	0	5	1	
…			…	…	…	
Threshold value limit for rate 9 [d]	5/6	*b*	0	5	9	
Maximum contracted flow rate [e]	5/6	*b*	0	5	21	
Maximum contracted power [e]	5/6	*b*	0	5	22	
Maximum contracted $\Delta\theta$ [e]	5/6	*b*	0	5	23	
Minimum contracted return temperature [e]	5/6	*b*	0	5	24	
Timing information						
Averaging period for measurements, generic	5/6	*b*	0	8	0	
Averaging period for instantaneous measurements	5/6	*b*	0	8	1	
Averaging period for volume / flow measurements	5/6	*b*	0	8	2	
Averaging period for temperature measurements	5/6	*b*	0	8	3	
Averaging period for pressure measurements	5/6	*b*	0	8	4	
Averaging period, power	5/6	*b*	0	8	5	
Averaging period, flow rate	5/6	*b*	0	8	6	
Averaging period, test values	5/6	*b*	0	8	7	
Measurement period, peak values, period 1(short) [g]	5/6	*b*	0	8	11	
Measurement period, peak values, period 2 [g]	5/6	*b*	0	8	12	
Measurement period, peak values, period 3 [g]	5/6	*b*	0	8	13	
Measurement period, peak values, period 4 [g]	5/6	*b*	0	8	14	
Measurement period, periodical values, period 1(short) [g]	5/6	*b*	0	8	21	

© Copyright 1997-2020 DLMS User Association

General and service entry objects – Thermal energy	OBIS code					
	A	B	C	D	E	F
Measurement period, periodical values, period 2 [g]	5/6	*b*	0	8	22	
Measurement period, periodical values, period 3 [g]	5/6	*b*	0	8	23	
Measurement period, periodical values, period 4 [g]	5/6	*b*	0	8	24	
Measurement period, test values	5/6	*b*	0	8	25	
Recording interval 1 for profiles [h]	5/6	*b*	0	8	31	
Recording interval 2 for profiles [h]	5/6	*b*	0	8	32	
Recording interval 3 for profiles [h]	5/6	*b*	0	8	33	
Billing period	5/6	*b*	0	8	34	
Time entries						
Local time	5/6	*b*	0	9	1	
Local date	5/6	*b*	0	9	2	
Time stamp (local time) of the most recent billing period [i]	5/6	*b*	0	9	3	
Manufacturer specific [j]	5/6	*b*	96	50	*e*	*f*
.....						
Manufacturer specific	5/6	*b*	96	99	*e*	*f*

a	Information about where the (single) flow meter is inserted. A non-zero value is used when the flow meter is located in the flow path.
b	Defines the pressure of the media, if not measured. The default value is 16 bar according to EN 1434-2:2015.
c	The enthalpy of the thermal conveying liquid. This will be necessary when using media other than pure water. The enthalpy is a part of the calculations when converting from mass to power.
d	Part of the contract between the customer and the supplier. The threshold defines when to switch rate, and can be used for diagnostic purposes, or to control limiting valves as well.
e	Part of the contract between the customer and the supplier. The threshold may be used to set a 'flag', for diagnostic purposes, or to control limiting valves.
f	Value group 'F' may be left unused, if there is only one set of historical / periodical values in the meter.
g	The instantiation of periods in a meter shall always start at period 1.
h	If only one recording interval is implemented, then it shall be recording interval 1. If multiple recording intervals are implemented, the recording interval 1 shall be the interval with the shorter period.
i	In case of billing period schemes absence or event triggered, commonly calculated from local date and local time information
j	The range D = 50…99 is available for identifying objects, which are not represented by another defined code, but need representation on the display as well. If this is not required, the range D = 128…254 should be used.

7.7.5.2 Error register objects – Thermal energy

Table 89 specifies OBIS codes for thermal energy related error register objects.

Table 89 – OBIS codes for error register objects – Thermal energy

Error register objects – Thermal energy	OBIS code					
	A	B	C	D	E	F
Error registers	5/6	*b*	97	97	*e*	
NOTE The information to be included in the error objects is not defined in this document.						

© Copyright 1997-2020 DLMS User Association

7.7.5.3 List objects – Thermal Energy Meters

Table 90 specifies the OBIS codes for Thermal Energy Meters related list objects. .

Table 90 – OBIS codes for list objects – Thermal Energy Meters

List objects – Thermal Energy Meters	OBIS code					
	A	B	C	D	E	F
Thermal energy related data of billing period (with billing period scheme 1 if there are two schemes available)	5/6	*b*	98	1	*e*	255 [a]
Thermal energy related data of billing period (with billing period scheme 2)	5/6	*b*	98	2	*e*	255 [a]
[a] F = 255 means a wildcard here. See 7.11.3.						

7.7.5.4 Data profile objects – Thermal energy

Thermal energy related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data. The OBIS codes are specified in Table 91.

Table 91 – OBIS codes for data profile objects – Thermal energy

Data profile objects – Thermal energy	OBIS code					
	A	B	C	D	E	F
Consumption / load profile with recording interval 1	5/6	*b*	99	1	1	
Consumption / load profile with recording interval 2	5/6	*b*	99	1	2	
Consumption / load profile with recording interval 3	5/6	*b*	99	1	3	
Profile of maxima with recording interval 1	5/6	*b*	99	2	1	
Profile of maxima with recording interval 2	5/6	*b*	99	2	2	
Profile of maxima with recording interval 3	5/6	*b*	99	2	3	
Consumption / load profile during test	5/6	*b*	99	3	1	
Certification data log	5/6	*b*	99	99	*e*	

7.7.5.5 OBIS codes for Thermal energy related objects (examples)

Table 92 shows examples for OBIS codes of Thermal energy related objects.

Table 92 – OBIS codes for Thermal energy related objects (examples)

Thermal energy related objects (examples)	OBIS code					
	A	B	C	D	E	F
Consumption						
Energy, current value, total	5/6	*b*	1	0	0	
Energy, current value, rate 1	5/6	*b*	1	0	1	
Energy, periodical, total, the second last storage	5/6	*b*	1	1	0	102
Energy, billing date value, total, last storage, rate 1	5/6	*b*	1	3	1	V_Z

© Copyright 1997-2020 DLMS User Association

Thermal energy related objects (examples)	OBIS code					
	A	B	C	D	E	F
Monitoring values						
Energy, maximum value (current period)	5/6	*b*	1	5		
Flow rate, Period value 2, previous storage	5/6	*b*	9	12		V_{Z-1}
Power, Max value, previous period	5/6	*b*	8	5		V_{Z-1}
Energy, Missing duration c	5/6	*b*	1	25		
Differential temperature, Test value	5/6	*b*	12	6		
Flow path, temperature transducers serial no.	5/6	*b*	0	2	10	
Error handling						
Overall error status [a]	5/6	*b*	97	97	0	
Subsystem where error has occurred [b]	5/6	*b*	97	97	1	
Duration of error condition [c]	5/6	*b*	97	97	2	

[a]	This object is a 'mirror' of the object 0.x.97.97.0.
[b]	This is the time during which the meter has not been able to calculate energy.
[c]	A further subdivision of error information.

© Copyright 1997-2020 DLMS User Association

7.8 Gas (Value group A = 7)

7.8.1 General introduction to gas measurement

7.8.1.1 Overview

Measurement of the energy supplied in the form of gas to customers is a complex process. It has to take into account the characteristics of the measuring site, the gas measurement technology, the conditions and the properties of the gas and the characteristics of the billing process.

Energy measurement is generally a multi-step process:

- The first step is to measure either the amount of the volume or the mass of gas based on various measuring principles, like volume, flow, density or mass measurement. Accuracy can be improved by correcting the measurement error of the meter;
- In the case of volume measurement, the next step is to convert the volume measured at metering conditions to volume at base conditions;
- In the final step, the energy is calculated from the volume at base conditions or the mass, and the calorific value. The calorific value – either per volume unit or per mass unit – is determined using gas analysis techniques.

The measurement technology and the implementation of the volume conversion and energy calculation process depend on the application segment.

Conversion and calculation steps can take place at the measuring site by electronic devices, or in the IT system.

For measurement of larger volumes, there are several devices involved in the process, depending on installation and hazardous area requirements. Not only the final results, but also interim values in the conversion and calculation process are of interest for checking and controlling purposes.

7.8.1.2 Typical gas metering installations

7.8.1.2.1 Residential application

A typical residential gas metering installation is shown in Figure 36.

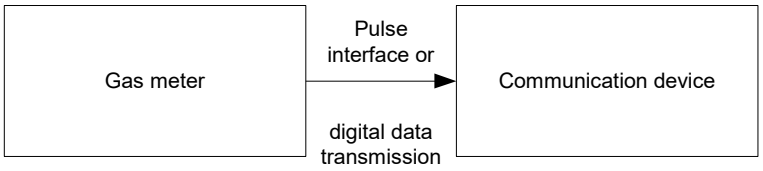


Figure 36 – Residential gas metering installation

The meter is typically a diaphragm (positive displacement) meter, which may perform mechanical temperature correction.

The information from the gas meter to the communication device may be transferred in the form of pulses. Alternatively, the meter may be equipped with a digital interface, e.g. an encoder turning the index reading to digital information.

Volume conversion and energy calculation takes place in the IT system.

7.8.1.2.2 Industrial application

A typical industrial gas metering installation is shown in Figure 37.

© Copyright 1997-2020 DLMS User Association

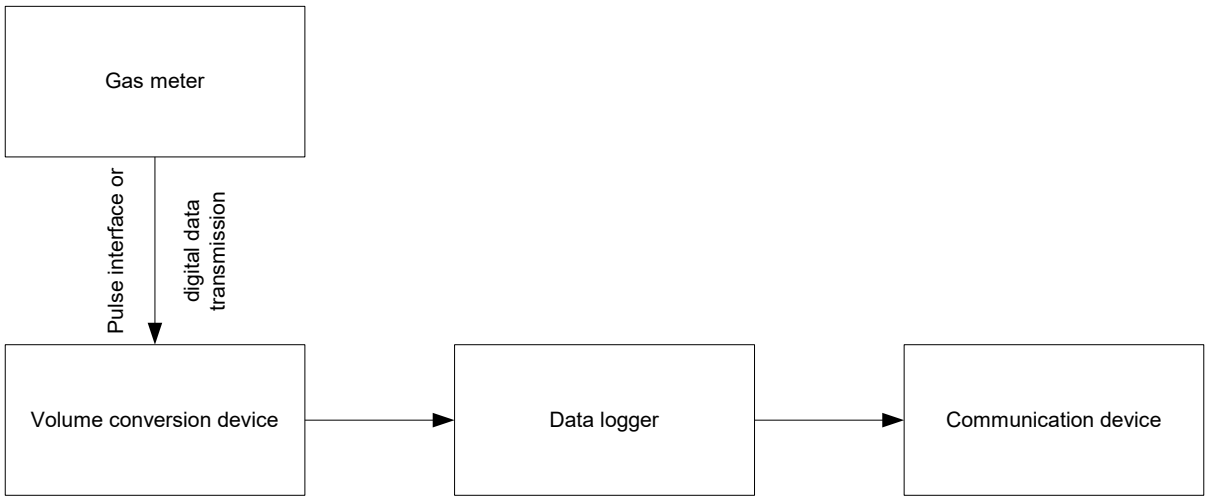


Figure 37 – Industrial gas metering installation (single stream)

In industrial applications, typically more functions are implemented at the measuring site than in residential applications. This may include the calculation of the volume at base conditions, and, if the calorific value is available (e.g. via remote communication), the calculation of the energy.

The data logger stores data relevant for billing, data validation and process control.

The functions may be integrated in fewer devices, depending on the hazardous zone restrictions and the level of integration of electronics.

7.8.1.2.3 Gas transport application

A typical gas transport metering “city gate” installation – also used for very large consumers – is shown in Figure 38.

Such gas stations are equipped with more than one pipe for the gas flow (multi stream).Typically, volume conversion devices are installed on each pipe, because the measurement is closely pipe related. Generally, there is one data logger and a device used to determine the calorific value (e.g. gas chromatograph).

All devices are connected via a bus system.

Depending on the design of these devices, selected functions may be implemented in a single cabinet or physical device.

© Copyright 1997-2020 DLMS User Association

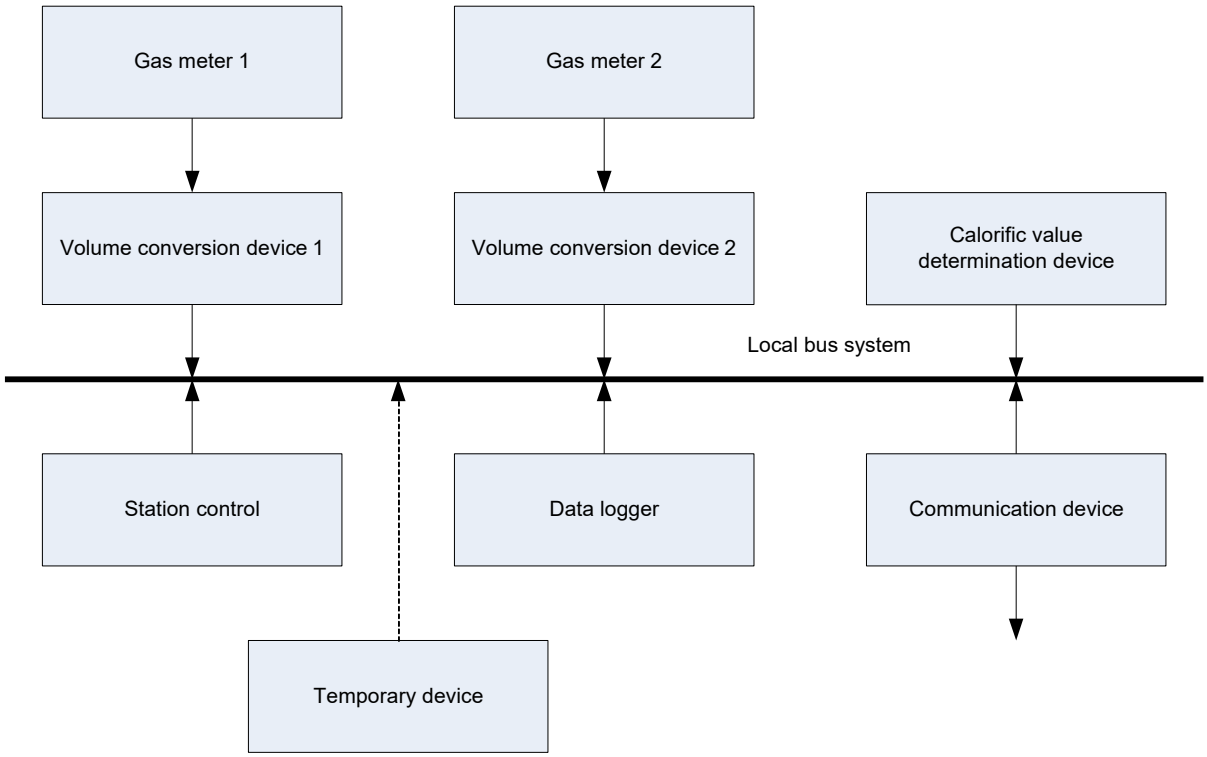


Figure 38 – City gate or border crossing installation (multi stream)

7.8.1.3 Gas volume conversion

7.8.1.3.1 General

The gas volume conversion process needs the following inputs:

- the volume information from a gas meter;
- the temperature of the gas measured;
- the pressure of the gas measured: this may be replaced by a constant;
- the compressibility, this may be replaced by a constant.

When the process is implemented in a gas conversion device, it is assumed to be capable of:

- performing error correction (optionally);
- measuring the temperature;
- measuring the pressure of the gas (optionally); and
- calculating the compressibility according to agreed algorithms, in function of temperature, pressure and gas composition (optionally).

The volume conversion device may handle bidirectional gas flows. The main direction of flow is *forward*.

It may be equipped with *disturbance registers* used when the value of temperature, pressure or compressibility is outside permissible metrological limits of plausibility, leading to an *alert condition*. When such alert condition occurs, the gas conversion process switches to store results in disturbance registers, until the alert conditions disappears.

7.8.1.3.2 Step 1: Error correction (optional)

The error curve of the gas meter is corrected by a correction factor:

$$V_c = C_f * V_m$$

© Copyright 1997-2020 DLMS User Association

where:

- V_c is the corrected volume;
- C_f is the correction factor given by an equation Cf = f(q) or Cf = f(Re); where q is the flow and Re is the Reynolds number;
- V_m is the volume at metering conditions.

The error correction method depends on station construction and operating conditions and its selection is made generally by manufacturer, utility or market specific.

7.8.1.3.3 Step 2: Volume conversion to base conditions

Volume at base conditions is calculated using the equation:

$$V_b = C \times V$$

Where:

- V_b is the volume at base conditions,
- V may be V_m or V_c (Volume at metering conditions or corrected volume);
- C is the conversion factor given by the relationship:

$$C = (P / P_b) \times (T_b / T) \times (Z_b / Z)$$

where Z is the compressibility factor allowing to take into account the difference in compressibility between the gas measured and the ideal gas. It is a function of the pressure and the temperature:

$$Z = f(P, T)$$

Settable gas properties and components are used for the compressibility calculation, combined into one of several existing calculation methods. If the compressibility factor is not calculated, it may be included as a fixed value in the calculation of the conversion factor. Below 1,5 bar, the value of Z is usually set to 1.

If the pressure is not measured, it may be included as a fixed value in the calculation of the conversion factor.

7.8.1.3.4 Step 3: Energy conversion

The final step is to calculate the energy, using the equation:

$$E = \text{CalValue} \times V_b$$

where CalValue is the calorific value, expressed in J/m^3. Typically, it is measured by calorimeter or gas chromatograph devices.

7.8.1.3.5 Model of data flow for volume conversion and energy calculation

The model of data flow for volume conversion and energy calculation is shown in Figure 39.

© Copyright 1997-2020 DLMS User Association

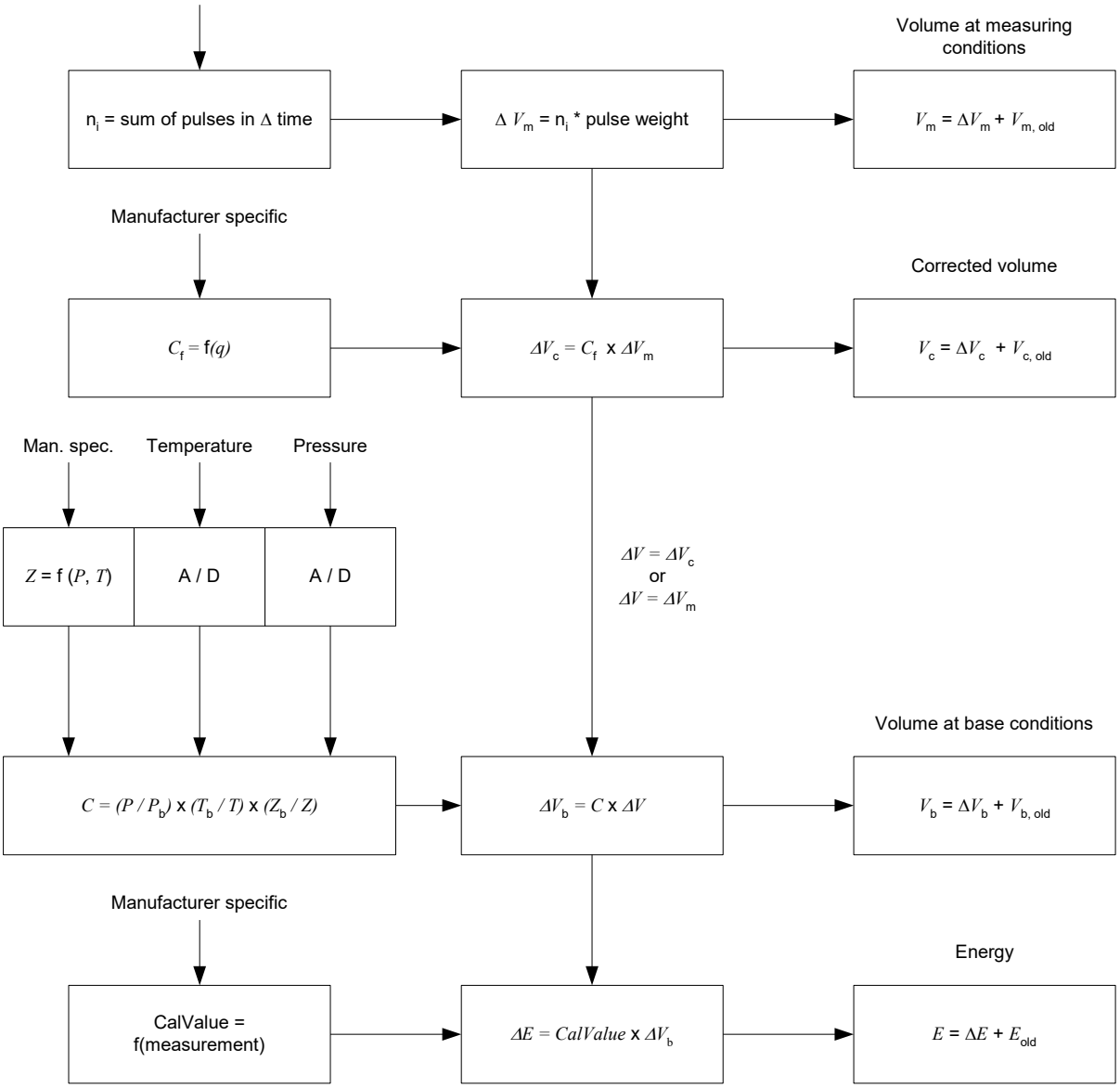


Figure 39 – Data flow of volume conversion and energy calculation

The OBIS codes of the main objects in the data flow are shown in Table 93, with the following assumptions:

- the conversion process passes through all four functions from metering to energy;
- the device has one single channel;
- the direction of the gas flow is forward;
- energy is the result of the conversion process from volume at base conditions to energy, by applying the calorific value as factor;
- the data of interest are current values of absolute indexes and the gas process data.

© Copyright 1997-2020 DLMS User Association

Table 93 – OBIS codes of the main objects in the gas conversion process data flow

Name	Symbol	OBIS code
Indexes		
Forward absolute meter volume, index, at metering conditions	V_m	7.0.3.0.0.255
Forward absolute converter volume, index, at metering conditions	V_m	7.0.13.0.0.255
Forward absolute converter volume, index, corrected value	V_c	7.0.13.1.0.255
Forward absolute converter volume, index, at base conditions	V_b	7.0.13.2.0.255
Forward absolute energy, index, at base conditions	E	7.0.33.2.0.255
Compressibility, correction and conversion values		
Correction factor [a]	C_f	7.0.51.0.0.255
Conversion factor [b]	C	7.0.52.0.0.255
Compressibility factor, current value at metering conditions [c]	Z	7.0.53.0.0.255
Compressibility factor, current value at base conditions [c]	Z_b	7.0.53.2.0.255
Compressibility factor, preset value [c]	Z_b	7.0.53.11.0.255
Compressibility factor, calculation method [c]		7.0.53.12.0.255
Superior calorific value [d]	CalVal	7.0.54.0.0.255
Metering site condition information		
Gas temperature (absolute), value at metering conditions [e]	T	7.0.41.0.0.255
Gas temperature (absolute), value at base conditions [e]	T_b	7.0.41.2.0.255
Gas temperature (absolute), backup value [e]	T	7.0.41.3.0.255
Gas pressure (absolute), value at metering conditions [f]	P	7.0.42.0.0.255
Gas pressure (absolute), value at base conditions [f]	P_b	7.0.42.2.0.255
Gas pressure (absolute), backup value [f]	P	7.0.42.3.0.255

a	A fixed value used to correct a scalar error on a meter: for example, if a meter under-registers volume by 0,5 %, then a correction factor value of 1,005 will compensate for the error.
b	See 7.8.1.3.3.
c	Compressibility, Z: effectively, the “difference” in compressibility between the gas being measured and “noble” gas. For example, EN 12405, SGERG-88, AGA 8 give full information on this, though below 1,5 Bar (a) this is usually set to 1.
d	The superior (or gross) calorific value can be seen as a conversion factor for converting volume to energy although it is also used for the conversion algorithm.
e	Temperature of the gas, expressed in Kelvin. Volume conversion depends on Kelvin temperature measurement. This may represent a measured value or a base condition, or a backup value, used if the temperature sensor fails, as identified by the value of value group D.
f	Pressure of the gas, expressed in a suitable unit, in absolute terms, for example Bar (a). This means that the value is referenced to a perfect vacuum, as opposed to “Gauge” pressure, which is referenced to current atmospheric conditions. This may represent a measured value or a base condition, or a backup value, used if the pressure sensor fails, as identified by the value of value group D.

7.8.1.4 Data logging

7.8.1.4.1 General

The data logging process captures, generates and makes available the data necessary for billing, as well as the data necessary for managing the measurement process and the gas grid.

© Copyright 1997-2020 DLMS User Association

7.8.1.4.2 Time bound processing

Quantities measured by the gas meter, calculated in the data logger or in the IT system may be:

- indexes, index differences and maxima of index differences; and
- average, minimum and maximum values

related to various intervals and periods. A distinction is made between:

- recording intervals for profiles;
- measurement periods for average values;
- process intervals;
- measurement periods for index differences;
- billing periods for indexes, index differences and maxima of index differences;
- averaging periods.

Some of these periods and intervals may have a default length, or otherwise their length can be held by specific objects. See 7.8.6.1, Table 104.

The processing methods depend on the kind of the quantity:

- indexes and index differences; see 7.8.3.2;
- flow rate, see 7.8.3.3;
- process values, see 7.8.3.4;
- conversion related factors and coefficients, see 7.8.3.5; and
- natural gas analysis, see 7.8.3.6.

7.8.1.4.3 Gas day

One specific element in gas metering is that the start of a gas day may be different from the start of a calendar day.

NOTE 1 For example the gas day starts at 6:00 in Germany.

NOTE 2 In some countries, the gas day start time retains its value when DST starts and ends, causing a 25 hour and 23 hour day in each year.

Therefore, taking the example above, a gas month lasts from 6:00 of the first day of a calendar month to 6:00 of the first day of the next calendar month. Similarly, a gas year starts at 6:00 on 1st of January and ends at 6:00 on 1st January of the next year.

7.8.1.4.4 Data profiles

COSEM “Profile generic” objects may capture one or several values – attributes of COSEM objects – in their buffer.

For gas metering, both *general purpose* and *dedicated* profiles are available:

- a general purpose “Profile generic” object captures one or several values. Such objects have a general OBIS code / logical name that do not provide specific information on the values captured. These profiles are also available with some fixed recording intervals;
- a dedicated “Profile generic” object captures only one value. The OBIS code / logical name of such a dedicated ”Profile generic” object is “self-explanatory”, i.e. it reflects the OBIS code of the object the value attribute of which is captured.

NOTE A time stamp and a status attribute may be captured in addition to the value(s) of interest.

In any case, the values captured are identified by the capture_objects attribute. See 7.8.6.4.

© Copyright 1997-2020 DLMS User Association

7.8.2 Value group C codes – Gas

The allocations in the value group C – see Table 94 – take into account the different combinations of measuring and calculating devices located at a metering point, to allow identifying the source where the data are generated.

For the purposes of volume / mass / energy measurement, value group C identifies:

- the location of the device in the measurement chain: meter (encoder), converter, logger;
- the direction of the gas flow: forward or reverse;
- the qualifier of the measurement: undisturbed, disturbed, or absolute, where absolute value is the sum of the values calculated under undisturbed and disturbed conditions.
- Value group C is also used for identifying process data.

For the purposes of gas analysis, a distinction is made between measured values generated by gas analysing systems (C = 70) and parameters used for calculation (C = 0, D = 12).

Table 94 – Value group C codes – Gas

Value group C codes – Gas (A = 7)	
0	General purpose objects
1	Forward undisturbed meter volume
2	Forward disturbed meter volume
3	Forward absolute meter volume
4	Reverse undisturbed meter volume
5	Reverse disturbed meter volume
6	Reverse absolute meter volume
7	Forward absolute meter volume (encoder)
8	Reverse absolute meter volume (encoder)
11	Forward undisturbed converter volume
12	Forward disturbed converter volume
13	Forward absolute converter volume
14	Reverse undisturbed converter volume
15	Reverse disturbed converter volume
16	Reverse absolute converter volume
21	Forward undisturbed logger volume
22	Forward disturbed logger volume
23	Forward absolute logger volume
24	Reverse undisturbed logger volume
25	Reverse disturbed logger volume
26	Reverse absolute logger volume
31	Forward undisturbed energy
32	Forward disturbed energy

© Copyright 1997-2020 DLMS User Association

Value group C codes – Gas (A = 7)	
33	Forward absolute energy
34	Reverse undisturbed energy
35	Reverse disturbed energy
36	Reverse absolute energy
41	Absolute temperature
42	Absolute pressure
43	Flow rate
44	Velocity of sound
45	Density (of gas)
46	Relative density
47	Gauge pressure
48	Differential pressure
49	Density of air
51	Correction factor
52	Conversion factor
53	Compressibility factor
54	Superior calorific value [a]
55	Gas law deviation coefficient (= compressibility factor ratio)
61	Forward undisturbed mass
62	Forward disturbed mass
63	Forward absolute mass
64	Reverse undisturbed mass
65	Reverse disturbed mass
66	Reverse absolute mass
70	Natural gas analysis
93	Consortia specific identifiers
94	Country specific identifiers
96	General and service entry objects – Gas (See 7.8.6.1)
97	Error register objects – Gas (See 7.8.6.2)
98	List objects – Gas (See 7.8.6.3)
99	Data profiles – Gas (See 7.8.6.4)
128...199, 240	Manufacturer specific codes
All other	Reserved
Notes	

[a] The superior (or gross) caloric value can be seen as a conversion factor for converting volume to energy although it is also used for the conversion algorithm.

© Copyright 1997-2020 DLMS User Association

7.8.3 Value group D codes – Gas

7.8.3.1 General

Allocations in value group D allow to further classify quantities identified by codes in value group A to C. The allocations depend on the kind of quantity:

- indexes and index differences; see 7.8.3.2;
- flow rate, see 7.8.3.3;
- process values, see 7.8.3.4;
- conversion related factors and coefficients, see 7.8.3.5; and
- natural gas analysis values, see 7.8.3.6.

7.8.3.2 Gas indexes and index differences

The allocations allow identifying the various volume, mass and energy quantities measured along the measuring chain and the gas volume conversion process, relative to various measurement and billing periods:

- indexes: current values and historical values relative to various billing periods;
- index differences: current and last values relative to measurement periods and billing periods;

NOTE Index difference over a certain measurement or billing period is also known as consumption. For consumption, thresholds may be defined, see Table 104.

- maximum of index differences over various measurement periods, relative to various billing periods;

A distinction is made between *value at metering conditions*, *corrected value* and *value at base conditions (converted value)*. The applicability of these qualifiers depends on the location in the measuring chain and in the gas volume conversion process.

Three measurement periods are available:

- measurement period 1: default value 15 min;
- measurement period 2: default value 1 hour;
- measurement period 3: no default value specified.

Four billing periods are available:

- billing period 1: default value 1 day;
- billing period 2: default value 1 month;
- billing period 3: default value 1 year;
- billing period 4: no default value specified.

The default values specified reflect the most common applications. If other values are used, they may be held by COSEM objects specified for this purpose; see 6.4.4 and Table 104.

In addition to the current values of the indexes, the following values are available:

For measurement periods 1 to 3:

- index differences for the current and the last measurement period (6 values each).

For billing periods 1, 3 and 4:

- historical indexes (3 values each);

© Copyright 1997-2020 DLMS User Association

- index differences for the current and the last billing period (6 values each);
- maximum of index differences over measurement periods 1, 2 and 3 (9 values each);
- in total, 18 values each.

For billing period 2:

- historical indexes (3 values);
- index differences for the current and the last billing period (6 values);
- maximum of index differences over measurement periods 1, 2 and 3, as well as over billing period 1 (12 values);
- in total, 21 values.

For all these values, tariffs may be applied. See 7.8.4.2.

Table 95 specifies the use of value group D to identify gas related indexes and index differences.

Table 95 – Value group D codes – Gas – Indexes and index differences

Value group D codes – Gas – Indexes and index differences (A= 7, C = 1…8, 11…16, 21…26, 31…36, 61…66)			
	Quantity	**Qualifier**	**Period**
0	Index	Value at metering conditions	Current [c]
1	Index	Corrected value [a]	Current [c]
2	Index	Value at base conditions / “Converted value”	Current [c]
3	Index	Current redundant value at metering conditions [b]	Current [c]
Values relative to measurement period 1 (default value = 15 minutes)			
6	Index difference	Value at metering conditions	Current
7	Index difference	Corrected value	Current
8	Index difference	Value at base conditions	Current
9	Index difference	Value at metering conditions	Last
10	Index difference	Corrected value	Last
11	Index difference	Value at base conditions	Last
Values relative to measurement period 2 (default value = 1 hour)			
12	Index difference	Value at metering conditions	Current
13	Index difference	Corrected value	Current
14	Index difference	Value at base conditions	Current
15	Index difference	Value at metering conditions	Last
16	Index difference	Corrected value	Last
17	Index difference	Value at base conditions	Last
Values relative to measurement period 3 (no default value)			
18	Index difference	Value at metering conditions	Current
19	Index difference	Corrected value	Current
20	Index difference	Value at base conditions	Current
21	Index difference	Value at metering conditions	Last
22	Index difference	Corrected value	Last

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Indexes and index differences (A= 7, C = 1…8, 11…16, 21…26, 31…36, 61…66)			
	Quantity	**Qualifier**	**Period**
23	Index difference	Value at base conditions	Last
Values relative to billing period 1 (default value = 1 day)			
24	Index	Value at metering conditions	Historical [c]
25	Index	Corrected value	Historical [c]
26	Index	Value at base conditions	Historical [c]
27	Index difference	Value at metering conditions	Current
28	Index difference	Corrected value	Current
29	Index difference	Value at base conditions	Current
30	Index difference	Value at metering conditions	Last
31	Index difference	Corrected value	Last
32	Index difference	Value at base conditions	Last
33	Maximum of Index differences over measurement period 1 [c]	Value at metering conditions	
34	Maximum of Index differences over measurement period 1 [c]	Corrected value	
35	Maximum of Index differences over measurement period 1 [c]	Value at base conditions	
36	Maximum of Index differences over measurement period 2 [c]	Value at metering conditions	
37	Maximum of Index differences over measurement period 2 [c]	Corrected value	
38	Maximum of Index differences over measurement period 2 [c]	Value at base conditions	
39	Maximum of Index differences over measurement period 3 [c]	Value at metering conditions	
40	Maximum of Index differences over measurement period 3 [c]	Corrected value	
41	Maximum of Index differences over measurement period 3 [c]	Value at base conditions	
Values relative to billing period 2 (default value = 1 month)			
42	Index	Value at metering conditions	Historical [c]
43	Index	Corrected value	Historical [c]
44	Index	Value at base conditions	Historical [c]
45	Index difference	Value at metering conditions	Current
46	Index difference	Corrected value	Current
47	Index difference	Value at base conditions	Current
48	Index difference	Value at metering conditions	Last
49	Index difference	Corrected value	Last
50	Index difference	Value at base conditions	Last
51	Maximum of Index differences over measurement period 1 [c]	Value at metering conditions	
52	Maximum of Index differences over measurement period 1 [c]	Corrected value	
53	Maximum of Index differences over measurement period 1 [c]	Value at base conditions	

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Indexes and index differences (A= 7, C = 1…8, 11…16, 21…26, 31…36, 61…66)			
	Quantity	**Qualifier**	**Period**
54	Maximum of Index differences over measurement period 2 [c]	Value at metering conditions	
55	Maximum of Index differences over measurement period 2 [c]	Corrected value	
56	Maximum of Index differences over measurement period 2 [c]	Value at base conditions	
57	Maximum of Index differences over measurement period 3 [c]	Value at metering conditions	
58	Maximum of Index differences over measurement period 3 [c]	Corrected value	
59	Maximum of Index differences over measurement period 3 [c]	Value at base conditions	
60	Maximum of Index differences over billing period 1 [c]	Value at metering conditions	
61	Maximum of Index differences over billing period 1 [c]	Corrected value	
62	Maximum of Index differences over billing period 1 [c]	Value at base conditions	
Values relative to billing period 3 (default value = 1 year)			
63	Index	Value at metering conditions	Historical [c]
64	Index	Corrected value	Historical [c]
65	Index	Value at base conditions	Historical [c]
66	Index difference	Value at metering conditions	Current
67	Index difference	Corrected value	Current
68	Index difference	Value at base conditions	Current
69	Index difference	Value at metering conditions	Last
70	Index difference	Corrected value	Last
71	Index difference	Value at base conditions	Last
72	Maximum of Index differences over measurement period 1 [c]	Value at metering conditions	
73	Maximum of Index differences over measurement period 1 [c]	Corrected value	
74	Maximum of Index differences over measurement period 1 [c]	Value at base conditions	
75	Maximum of Index differences over measurement period 2 [c]	Value at metering conditions	
76	Maximum of Index differences over measurement period 2 [c]	Corrected value	
77	Maximum of Index differences over measurement period 2 [c]	Value at base conditions	
78	Maximum of Index differences over measurement period 3 [c]	Value at metering conditions	
79	Maximum of Index differences over measurement period 3 [c]	Corrected value	
80	Maximum of Index differences over measurement period 3 [c]	Value at base conditions	
Values relative to billing period 4 (no default value)			
81	Index	Value at metering conditions	Historical [c]
82	Index	Corrected value	Historical [c]

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Indexes and index differences (A= 7, C = 1…8, 11…16, 21…26, 31…36, 61…66)			
	Quantity	**Qualifier**	**Period**
83	Index	Value at base conditions	Historical [c]
84	Index difference	Value at metering conditions	Current
85	Index difference	Corrected value	Current
86	Index difference	Value at base conditions	Current
87	Index difference	Value at metering conditions	Last
88	Index difference	Corrected value	Last
89	Index difference	Value at base conditions	Last
90	Maximum of Index differences over measurement period 1 [c]	Value at metering conditions	
91	Maximum of Index differences over measurement period 1 [c]	Corrected value	
92	Maximum of Index differences over measurement period 1 [c]	Value at base conditions	
93	Maximum of Index differences over measurement period 2 [c]	Value at metering conditions	
94	Maximum of Index differences over measurement period 2 [c]	Corrected value	
95	Maximum of Index differences over measurement period 2 [c]	Value at base conditions	
96	Maximum of Index differences over measurement period 3 [c]	Value at metering conditions	
97	Maximum of Index differences over measurement period 3 [c]	Corrected value	
98	Maximum of Index differences over measurement period 3 [c]	Value at base conditions	
All other	Reserved		
[a]	Error correction of meter curves can be allocated to meters (e.g. temperature compensation of a diaphragm gas meter) or subsequent connected devices (e.g. high pressure correction curve of a turbine meter implemented in an associated volume conversion device).		
[b]	From data logger (parallel recording) for use in case of a measurement device fails.		
[c]	Current value: F = 255 Historical values (F ≠ 255): - With F = 1…12, 0…99 value(s) of (a) previous billing period, relative to the billing period counter. - With F = 101…126 value(s) of (a) previous billing period(s) relative to the current billing period.		

7.8.3.3 Flow rate

The allocations allow identifying values associated with the flow rate of the gas. The flow rate is a process information. It is not linked to a physical device. No tariffication is applicable.

© Copyright 1997-2020 DLMS User Association

A distinction is made between:

- current average, last average, and maximum of last average values measured over various averaging periods, relative to various measurement and billing periods. Measurement period 2 and 3 shall be multiple of the averaging period of block demand / sliding demand measurement.
- values at metering conditions, corrected value, value at base conditions (converted value) and value at standard conditions;

NOTE Standard conditions refer to national regulations, which may differ from ISO standards reference values for base conditions.

EXAMPLE Gas reference temperature at standard conditions is 0 °C, gas reference temperature at base conditions is +15 °C.

For averaging period 2, block demand (default) or sliding demand is available. In the case of sliding demand, the averaging period is split to sub-periods. The number of sub-periods is carried by the object 7.b.0.8.35.255; see Table 104.

The last average values of the various flow rate quantities can be captured to load profiles, with self-explanatory OBIS codes, see 7.8.6.4.

Table 96 specifies the use of value group D to identify gas related flow rate values.

Table 96 – Value group D codes – Gas – Flow rate

Value group D codes – Gas – Flow rate (A = 7, C = 43)		
	Quantity	**Qualifier**
0	Instantaneous	Current value at metering conditions
1	Instantaneous	Corrected value
2	Instantaneous	Value at base conditions / “Converted value”
13	Instantaneous	Value at standard conditions
Averaging period 1, default value = 5 minutes		
15	Current average for averaging period 1	Value at metering conditions
16		Corrected value
17		Value at base conditions
18		Value at standard conditions
19	Last average for averaging period 1	Value at metering conditions
20		Corrected value
21		Value at base conditions
22		Value at standard conditions
23	Maximum of last averages for averaging period 1 relative to measurement period 2 (default value = 1 hour)	Value at metering conditions
24		Corrected value
25		Value at base conditions
26		Value at standard conditions
27	Maximum of last averages for averaging period 1 relative to measurement period 3 (no default value)	Value at metering conditions
28		Corrected value
29		Value at base conditions

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Flow rate (A = 7, C = 43)		
30		Value at standard conditions
31	Maximum of last averages for averaging period 1 relative to billing period 1 (default value = 1 day)	Value at metering conditions
32		Corrected value
33		Value at base conditions
34		Value at standard conditions
Averaging period 2, default value = 15 minutes (block demand or sliding demand)		
35	Current average for averaging period 2	Value at metering conditions
35		Corrected value
37		Value at base conditions
38		Value at standard conditions
39	Last average for averaging period 2	Value at metering conditions
40		Corrected value
41		Value at base conditions
42		Value at standard conditions
43	Maximum of last averages for averaging period 2 relative to measurement period 2 (default value = 1 hour)	Value at metering conditions
44		Corrected value
45		Value at base conditions
46		Value at standard conditions
47	Maximum of last averages for averaging period 2 relative to measurement period 3 (no default value)	Value at metering conditions
48		Corrected value
49		Value at base conditions
50		Value at standard conditions
51	Maximum of last averages for averaging period 2 relative to billing period 1 (default value = 1 day)	Value at metering conditions
52		Corrected value
53		Value at base conditions
54		Value at standard conditions
Averaging period 3, default value = 1 hour		
55	Current average for averaging period 3	Value at metering conditions
56		Corrected value
57		Value at base conditions
58		Value at standard conditions
59	Last average for averaging period 3	Value at metering conditions
60		Corrected value
61		Value at base conditions
62		Value at standard conditions

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Flow rate (A = 7, C = 43)		
Averaging period 4, (no default value)		
63	Current average for averaging period 4	Value at metering conditions
64		Corrected value
65		Value at base conditions
66		Value at standard conditions
67	Last average for averaging period 4	Value at metering conditions
68		Corrected value
69		Value at base conditions
70		Value at standard conditions
All other	Reserved	

7.8.3.4 Process values

For process values, a distinction is made between:

- instantaneous values;
- average, minimum and maximum values over various process intervals;
- value at metering conditions, value at base conditions; and value at standard conditions;

  NOTE Standard conditions refer to national regulations, which may differ from ISO standards reference values for base conditions.

  EXAMPLE Gas reference temperature at standard conditions is 0 °C, gas reference temperature at base conditions is +15 °C.

- for some quantities, backup, actual and preset values are available.

Table 97 specifies the use of value group D to identify gas related process values.

Table 97 – Value group D codes – Gas – Process values

Value group D codes – Gas – Process values (A = 7, C = 41, 42, 44...49)		
	Quantity	**Qualifier**
0	Instantaneous	Current value at metering conditions [a]
2	Instantaneous	Value at base conditions / “Converted value” [b]
3	Instantaneous	Backup value
10	Instantaneous	Actual value
11	Instantaneous	Preset value
13	Instantaneous	Value at standard conditions
Process interval 1 (default value = 15 minutes)		
15	Average, current interval, process interval 1	Value at metering conditions
16		Value at base conditions
17		Value at standard conditions
18	Minimum, current interval, process interval 1	Value at metering conditions

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Process values (A = 7, C = 41, 42, 44...49)		
19		Value at base conditions
20		Value at standard conditions
21	Maximum, current interval, process interval 1	Value at metering conditions
22		Value at base conditions
23		Value at standard conditions
24	Average, last interval, process interval 1	Value at metering conditions
25		Value at base conditions
26		Value at standard conditions
27	Minimum, last interval, process interval 1	Value at metering conditions
28		Value at base conditions
29		Value at standard conditions
30	Maximum, last interval, process interval 1	Value at metering conditions
31		Value at base conditions
32		Value at standard conditions
Process interval 2 (default value = 1 hour)		
33	Average, current interval, process interval 2	Value at metering conditions
34		Value at base conditions
35		Value at standard conditions
36	Minimum, current interval, process interval 2	Value at metering conditions
37		Value at base conditions
38		Value at standard conditions
39	Maximum, current interval, process interval 2	Value at metering conditions
40		Value at base conditions
41		Value at standard conditions
42	Average, last interval, process interval 2	Value at metering conditions
43		Value at base conditions
44		Value at standard conditions
45	Minimum, last interval, process interval 2	Value at metering conditions
46		Value at base conditions
47		Value at standard conditions
48	Maximum, last interval, process interval 2	Value at metering conditions
49		Value at base conditions
50		Value at standard conditions

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Process values (A = 7, C = 41, 42, 44...49)		
Process interval 3 (default value = 1 day)		
51	Average, current interval, process interval 3	Value at metering conditions
52		Value at base conditions
53		Value at standard conditions
54	Minimum, current interval, process interval 3	Value at metering conditions
55		Value at base conditions
56		Value at standard conditions
57	Maximum, current interval, process interval 3	Value at metering conditions
58		Value at base conditions
59		Value at standard conditions
60	Average, last interval, process interval 3	Value at metering conditions
61		Value at base conditions
62		Value at standard conditions
63	Minimum, last interval, process interval 3	Value at metering conditions
64		Value at base conditions
65		Value at standard conditions
66	Maximum, last interval, process interval 3	Value at metering conditions
67		Value at base conditions
68		Value at standard conditions
Process interval 4 (default value = 1 month)		
69	Average, current interval, process interval 4	Value at metering conditions
70		Value at base conditions
71		Value at standard conditions
72	Minimum, current interval, process interval 4	Value at metering conditions
73		Value at base conditions
74		Value at standard conditions
75	Maximum, current interval, process interval 4	Value at metering conditions
76		Value at base conditions
77		Value at standard conditions
78	Average, last interval, process interval 4	Value at metering conditions
79		Value at base conditions
80		Value at standard conditions
81	Minimum, last interval, process interval 4	Value at metering conditions
82		Value at base conditions
83		Value at standard conditions

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Process values (A = 7, C = 41, 42, 44…49)		
84	Maximum, last interval, process interval 4	Value at metering conditions
85		Value at base conditions
86		Value at standard conditions
Process interval 5, since last event		
87	Average, process interval 5, interval since last event	Value at metering conditions
88		Value at base conditions
89		Value at standard conditions
90	Average, process interval 6, interval between last two events	Value at metering conditions
91		Value at base conditions
92		Value at standard conditions
All other	Reserved	
[a]	To be used for e.g. velocity of sound.	
[b]	Value of the base conditions is associated with reference values for volume conversion: C = 41, 42.	

7.8.3.5 Conversion related factors and coefficients

For correction, conversion, compressibility, superior calorific value and gas law deviation coefficient values, various OBIS code allocations are made taking into consideration the specifics of the measuring process. See Table 98.

For these values, average values over various averaging periods are also defined; see 7.8.4.5.

Table 98 specifies the use of value group D to identify gas conversion related factors and coefficients values.

Table 98 – Value group D codes – Gas – Conversion related factors and coefficients

Value group D codes – Gas – Conversion related factors and coefficients (A = 7, C = 51…55)	
0	Current value at metering conditions
2	Current value at base conditions / “Converted Value”
3	Backup
10	Actual
11	Preset
12	Method
All other	Reserved

7.8.3.6 Natural gas analysis values

For natural gas analysis, allocations in value group D identify the key parameters and the components of the natural gas. For these values, average values over various averaging periods are also defined; see 7.8.4.6. Table 99 specifies the use of value group D to identify natural gas analysis values.

© Copyright 1997-2020 DLMS User Association

Table 99 – Value group D codes – Gas – Natural gas analysis values

Value group D codes – Gas – Natural gas analysis values (A = 7, C = 70)	
8	Reference pressure of gas analysis
9	Reference temperature of gas analysis
10	Superior [a] Wobbe index 0 °C
11	Inferior [b] Wobbe index 0 °C
12	Methane number
13	Total sulphur
14	Hydrogen sulphide H_2S
15	Mercaptans
16	Water dew point (DP H_2O)
17	Water (H_2O) dew point outlet / normalised
18	Hydrocarbon dew point (DP C_XH_Y)
19	Inferior [c] calorific value $H_{i,n}$
20	Water H_2O
60	Nitrogen N_2
61	Hydrogen H_2
62	Oxygen O_2
63	Helium He
64	Argon Ar
65	Carbon monoxide CO
66	Carbon dioxide CO_2
67	Methane CH_4
68	Ethene C_2H_4
69	Ethane C_2H_6
70	Propene C_3H_6
71	Propane C_3H_8
72	i-butane i-C_4H_{10}
73	n-butane n-C_4H_{10}
74	neo-pentane neo-C_5H_{12}
75	i-pentane i-C_5H_{12}
76	n-pentane n-C_5H_{12}
77	Hexane C_6H_{14}
78	Hexane share higher hydrocarbons C_6H_{14} %
79	Hexane+ C_6H_{14}+
80	Heptane C_7H_{16}

© Copyright 1997-2020 DLMS User Association

Value group D codes – Gas – Natural gas analysis values (A = 7, C = 70)	
81	Octane C_8H_{18}
82	Nonane C_9H_{20}
83	Decane $C_{10}H_{22}$
84	Tetrahydrothiophene C_4H_8S
All other	Reserved
a	Superior (gross) Wobbe index
b	Inferior (net) Wobbe index
c	Inferior (net) calorific value

7.8.4 Value group E codes – Gas

7.8.4.1 General

The following clauses define the use of value group E for identifying further classification or processing the measurement quantities defined by value groups A to D. The various classifications and processing methods are exclusive.

7.8.4.2 Indexes and index differences – Tariff rates

Table 100 shows the use of value group E for identification of tariff rates typically used for indexes and index differences of volume, mass and energy, specified in Table 95.

Table 100 – Value group E codes – Gas – Indexes and index differences – Tariff rates

Value group E codes – Gas – Indexes and index differences – Tariff rates (A = 7, C = 1…8, 11…16, 21…26, 31…36, 61…66, D = 0…3, 6…98)	
0	Total
1	Rate 1
…	
63	Rate 63
128…254	Manufacturer specific codes
All other	Reserved

7.8.4.3 Flow rate

No further classification in value group E are made. Therefore E shall be 0.

7.8.4.4 Process values

No further classification in value group E is made. Therefore, E shall be 0.

7.8.4.5 Conversion related factors and coefficients – Averages

Table 101 shows the use of value group E for the identification of average values of conversion related factors and coefficients – as specified in 7.8.3.5 – over various averaging periods.

© Copyright 1997-2020 DLMS User Association

Table 101 – Value group E codes – Gas – Conversion related factors and coefficients

	Value group E codes – Gas – Conversion related factors and coefficients– Averages (A = 7, C = 51…55, D = 0, 2, 3, 10, 11)
0	Process independent current value[a]
1	Weighted value (e.g. Superior calorific value)[b]
11	Average, current interval, averaging period 1 (default 5 minutes)
12	Average, last interval, averaging period 1 (default 5 minutes)
13	Average, current interval, averaging period 2 (default 15 minutes)
14	Average, last interval, averaging period 2 (default 15 minutes)
15	Average, current interval, averaging period 3 (default 1 hour)
16	Average, last interval, averaging period 3 (default 1 hour)
17	Average, current interval, averaging period 4 (no default value)
18	Average, last interval, averaging period 4 (no default value)
19	Average, current interval, averaging period 5 (default 1 day)
20	Average, last interval, averaging period 5 (default 1 day)
21	Average, current interval, averaging period 6 (default 1 month)
22	Average, last interval, averaging period 6 (default 1 month)
23	Average, current interval, averaging period 7 (default 1 year)
24	Average, last interval, averaging period 7 (default 1 year)
25	Average, current interval, averaging period 8 (no default value)
26	Average, last interval, averaging period 8 (no default value)
27	Average, averaging period 9, interval since last event
28	Average, averaging period 10, interval between last two events
All other	Reserved
[a]	Process independent current value is a gas analysis technology independent value, which is generated asynchronous to processing cycles, but used for further calculations.
[b]	Weighted value is the result of specific algorithms taking into account different values by weighting their influence on the algorithm result.

7.8.4.6 Calculation methods

Table 102 – Value group E codes – Gas – Calculation methods shows the use of value group E for the identification of calculation methods. See also 6.4.8.

© Copyright 1997-2020 DLMS User Association

Table 102 – Value group E codes – Gas – Calculation methods

	Value group E codes – Calculation methods (A = 7, C = 51…55, D = 12)
0	Calculation method in use
1	Calculation method 1 supported
…	…
20	Calculation method 20 supported
All other	Reserved

7.8.4.7 Natural gas analysis values – Averages

Table 103 shows the use of value group E for the identification of natural gas analysis values – as specified in 7.8.3.6 – over various averaging periods.

Table 103 – Value group E codes – Gas – Natural gas analysis values – Averages

Value group E codes – Gas – Natural gas analysis values – Averages (A = 7, C = 70, D = 8…20, 60…84)	
0	Process independent current value [a]
1	Weighted value (e.g. CO_2 in [GJ / t]) [b]
11	Average, current interval, averaging period 1 (default 5 minutes)
12	Average, last interval, averaging period 1 (default 5 minutes)
13	Average, current interval, averaging period 2 (default 15 minutes)
14	Average, last interval, averaging period 2 (default 15 minutes)
15	Average, current interval, averaging period 3 (default 1 hour)
16	Average, last interval, averaging period 3 (default 1 hour)
17	Average, current interval, averaging period 4 (no default value)
18	Average, last interval, averaging period 4 (no default value)
19	Average, current interval, averaging period 5 (default 1 day)
20	Average, last interval, averaging period 5 (default 1 day)
21	Average, current interval, averaging period 6 (default 1 month)
22	Average, last interval, averaging period 6 (default 1 month)
23	Average, current interval, averaging period 7 (default 1 year)
24	Average, last interval, averaging period 7 (default 1 year)
25	Average, current interval, averaging period 8 (no default value)
26	Average, last interval, averaging period 8 (no default value)
27	Average, averaging period 9, interval since last event
28	Average, averaging period 10, interval between last two events
All other	Reserved
a	Process independent current value is a gas analysis technology independent value, which is generated asynchronous to processing cycles, but used for further calculations.
b	Weighted value is the result of specific algorithms taking into account different values by weighting their influence on the algorithm result.

© Copyright 1997-2020 DLMS User Association

7.8.5 Value group F codes – Gas

Value group F identifies current (with F = 255) or historical values of quantities identified by value groups A to E, where appropriate.

There are four billing period schemes available (for example to store daily, monthly, yearly and weekly values). For each billing period scheme, the following general purpose objects are available:

- billing period counter;
- number of available billing periods;
- time stamp of most recent and historical billing periods;
- billing period length.

For OBIS codes see Table 104. For additional information, see 6.2.2 and 7.11.3.

7.8.6 OBIS codes – Gas

7.8.6.1 General and service entry objects – Gas

Table 104 specifies the OBIS codes for gas related general and service entry objects.

Table 104 – OBIS codes for general and service entry objects – Gas

General and shows service entry objects – Gas	OBIS code					
	A	B	C	D	E	F
Free ID-numbers for utilities						
Complete combined gas ID	7	*b*	0	0		
Gas ID 1	7	*b*	0	0	0	
…	…	…	…	…	…	
Gas ID 10	7	*b*	0	0	9	
Billing period values / reset counter entries (First billing period scheme if there are more than one)						
Billing period counter (1)	7	*b*	0	1	0	VZ or 255
Number of available billing periods (1)	7	*b*	0	1	1	
Time stamp of the most recent billing period (1)	7	*b*	0	1	2	
Time stamp of the billing period (1) VZ (last reset)	7	*b*	0	1	2	VZ
Time stamp of the billing period (1) VZ-1	7	*b*	0	1	2	VZ_{-1}
…			…	…	…	
Time stamp of the billing period (1) VZ-n	7	*b*	0	1	2	VZ_{-n}
Billing period values / reset counter entries (Second billing period scheme)						
Billing period counter (2)	7	*b*	0	1	3	VZ or 255
Number of available billing periods (2)	7	*b*	0	1	4	
Time stamp of the most recent billing period (2)	7	*b*	0	1	5	
Time stamp of the billing period (2) VZ (last reset)	7	*b*	0	1	5	VZ
Time stamp of the billing period (2) VZ-1	7	*b*	0	1	5	VZ_{-1}
…			…	…	…	
Time stamp of the billing period (2) VZ-n	7	*b*	0	1	5	VZ_{-n}

© Copyright 1997-2020 DLMS User Association

General and shows service entry objects – Gas	OBIS code					
	A	B	C	D	E	F
Billing period values / reset counter entries (Third billing period scheme)						
Billing period counter (3)	7	*b*	0	1	6	VZ or 255
Number of available billing periods (3)	7	*b*	0	1	7	
Time stamp of the most recent billing period (3)	7	*b*	0	1	8	
Time stamp of the billing period (3) VZ (last reset)	7	*b*	0	1	8	VZ
Time stamp of the billing period (3) VZ-1	7	*b*	0	1	8	VZ_{-1}
…			…	…	…	
Time stamp of the billing period (3) VZ-n	7	b	0	1	8	VZ_{-n}
Billing period values / reset counter entries (Fourth billing period scheme)						
Billing period counter (4)	7	*b*	0	1	9	VZ or 255
Number of available billing periods (4)	7	*b*	0	1	10	
Time stamp of the most recent billing period (4)	7	*b*	0	1	11	
Time stamp of the billing period (4) VZ (last reset)	7	*b*	0	1	11	VZ
Time stamp of the billing period (4) VZ-1	7	*b*	0	1	11	VZ_{-1}
…			…	…	…	
Time stamp of the billing period (4) VZ-n	7	*b*	0	1	11	VZ_{-n}
Configuration						
Program version	7	*b*	0	2	0	
Firmware version	7	*b*	0	2	1	
Software version	7	*b*	0	2	2	
Device version	7	*b*	0	2	3	
Active firmware signature	7	*b*	0	2	8	
Number of device channels	7	*b*	0	2	10	
Pressure sensor, serial no.	7	*b*	0	2	11	
Temperature sensor, serial no.	7	*b*	0	2	12	
Calculator, serial no.	7	*b*	0	2	13	
Volume sensor [a], serial no.	7	*b*	0	2	14	
Density sensor, serial no.	7	b	0	2	15	
Sensor (medium irrespective), serial no.	7	*b*	0	2	16	
Digital output configuration	7	*b*	0	2	17	
Analogue output configuration	7	*b*	0	2	18	
Output pulse constants converted / unconverted						
Volume forward at metering conditions	7	*b*	0	3	0	
Volume reverse at metering conditions	7	*b*	0	3	1	
Volume absolute [b] at metering conditions	7	*b*	0	3	2	
Volume forward at base conditions	7	*b*	0	3	3	
Volume reverse at base conditions	7	*b*	0	3	4	
Volume absolute [b] at base conditions	7	*b*	0	3	5	

© Copyright 1997-2020 DLMS User Association

General and shows service entry objects – Gas	OBIS code					
	A	B	C	D	E	F
Conversion factors						
	7	*b*	0	4	0	
	7	*b*	0	4	1	
{This area is to be used for polynomials, constants						
for conversion, and similar}	7	*b*	0	4	2	
…	7	*b*	0	4	3	
	7	*b*	0	4	4	
Threshold values						
Threshold power for over-consumption relative to measurement period 2 for indexes and index differences						
limit 1	7	*b*	0	5	1	1
…			…	…	…	…
limit 4	7	*b*	0	5	1	4
Threshold power for over-consumption relative to measurement period 3 for indexes and index differences						
limit 1	7	*b*	0	5	1	11
…			…	…	…	…
limit 4	7	*b*	0	5	1	14
Threshold limit for rate 1 for over-consumption relative to measurement period 2 for indexes and index differences	7	*b*	0	5	2	1
…			…	…	…	…
limit for rate 9	7	*b*	0	5	2	9
Threshold limit for rate 1 for over-consumption relative to measurement period 3 for indexes and index differences	7	*b*	0	5	2	11
…			…	…	…	…
limit for rate 9	7	*b*	0	5	2	19
Maximum contracted consumption for rec. interval 1	7	*b*	0	5	3	
Maximum contracted consumption for rec. interval 2	7	*b*	0	5	4	
Absolute temperature, minimum limit setting [c]	7	*b*	0	5	11	
Absolute temperature, maximum limit setting [c]	7	*b*	0	5	12	
Absolute pressure, minimum limit setting [c]	7	*b*	0	5	13	
Absolute pressure, maximum limit setting [c]	7	*b*	0	5	14	
Nominal values volume sensor						
Pressure	7	*b*	0	6	1	
Temperature	7	*b*	0	6	2	
Q_{min}	7	*b*	0	6	3	
Q_{max}	7	*b*	0	6	4	
Input pulse constants						
Volume forward at metering conditions	7	*b*	0	7	0	
Volume reverse metering conditions	7	*b*	0	7	1	
Volume absolute [b] at metering conditions	7	*b*	0	7	2	
Volume forward at base conditions	7	*b*	0	7	3	

© Copyright 1997-2020 DLMS User Association

General and shows service entry objects – Gas	OBIS code					
	A	B	C	D	E	F
Volume reverse at base conditions	7	*b*	0	7	4	
Volume absolute [b] at base conditions	7	*b*	0	7	5	
Intervals and periods						
Recording interval 1, for profile [d]	7	*b*	0	8	1	
Recording interval 2, for profile [d]	7	*b*	0	8	2	
Measurement period 1, for average value 1	7	*b*	0	8	3	
Measurement period 2, for average value 2	7	*b*	0	8	4	
Measurement period 3, for instantaneous value	7	*b*	0	8	5	
Measurement period 4, for test value	7	*b*	0	8	6	
Billing period	7	*b*	0	8	10	
NOTE Codes 7.b.0.8.11...35 are newly defined in Blue Book Edition 9.						
Process interval 1, default value 15 minutes	7	*b*	0	8	11	
Process interval 2, default value 1 hour	7	*b*	0	8	12	
Process interval 3, default value 1 day	7	*b*	0	8	13	
Process interval 4, default value 1 month	7	*b*	0	8	14	
Process interval 5, for process value, since last event	7	*b*	0	8	15	
Process interval 6, between last two events	7	*b*	0	8	16	
Measurement period 1, for indexes and index differences, default value 15 minutes	7	*b*	0	8	17	
Measurement period 2, for indexes and index differences, default value 1 hour	7	*b*	0	8	18	
Measurement period 3, for indexes and index differences, no default value	7	*b*	0	8	19	
Billing period 1, for indexes and index differences, default value 1 day	7	*b*	0	8	20	
Billing period 2, for indexes and index differences, default value 1 month	7	*b*	0	8	21	
Billing period 3, for indexes and index differences, default value 1 year,	7	*b*	0	8	22	
Billing period 4, for indexes and index differences, no default value	7	*b*	0	8	23	
Averaging period 1, default value 5 minutes	7	*b*	0	8	25	
Averaging period 2, default value 15 minutes	7	*b*	0	8	26	
Averaging period 3, default value 1 hour	7	*b*	0	8	27	
Averaging period 4, no default value	7	*b*	0	8	28	
Averaging period 5, default value 1 day	7	*b*	0	8	29	
Averaging period 6, default value 1 month	7	*b*	0	8	30	
Averaging period 7, default value 1 year	7	*b*	0	8	31	
Averaging period 8, no default value	7	*b*	0	8	32	
Averaging period 9, since last event	7	*b*	0	8	33	
Averaging period 10, between two last events	7	*b*	0	8	34	
Number of sub-periods for averaging period 2	7	*b*	0	8	35	
Time entries						

© Copyright 1997-2020 DLMS User Association

General and shows service entry objects – Gas	OBIS code					
	A	B	C	D	E	F
Number of days (time expired) since last reset (First billing period scheme if there are more than one)	7	*b*	0	9	0	
Local time	7	*b*	0	9	1	
Local date	7	*b*	0	9	2	
Start of conventional gas day	7	*b*	0	9	3	
Residual time shift [e]	7	*b*	0	9	4	
Time of last reset (First billing period scheme if there are more than one)	7	*b*	0	9	6	
Date of last reset (First billing period scheme if there are more than one)	7	*b*	0	9	7	
Clock time shift limit	7	*b*	0	9	11	
First billing period scheme						
Number of days (time expired) since last reset (end of billing period)	*See above.*					
Time of last reset	*See above.*					
Date of last reset	See above.					
Billing period reset lockout time (First billing period scheme if there are more than one)	7	*b*	0	9	12	
Second billing period scheme						
Number of days (time expired) since last end of billing period	7	*b*	0	9	13	
Time of last reset	7	*b*	0	9	14	
Date of last reset	7	*b*	0	9	15	
Billing period reset lockout time	7	*b*	0	9	16	
Third billing period scheme						
Number of days (Time expired) since last end of billing period	7	*b*	0	9	17	
Time of last reset	7	*b*	0	9	18	
Date of last reset	7	*b*	0	9	19	
Billing period reset lockout time	7	*b*	0	9	20	
Fourth billing period scheme						
Number of days (time expired) since last end of billing period	7	*b*	0	9	21	
Time of last reset	7	*b*	0	9	22	
Date of last reset	7	*b*	0	9	23	
Billing period reset lockout time	7	*b*	0	9	24	
Station management information objects						
Heating temperature [f], current value	7	*b*	0	10	0	
Heating temperature, average 15 minutes	7	*b*	0	10	1	
Heating temperature, average 60 minutes	7	*b*	0	10	11	
Heating temperature, average day	7	*b*	0	10	21	
Heating temperature, average month	7	*b*	0	10	31	
Ambient device temperature [g], current value	7	*b*	0	11	0	
Ambient device temperature, average 15 minutes	7	*b*	0	11	1	

© Copyright 1997-2020 DLMS User Association

General and shows service entry objects – Gas	OBIS code					
	A	B	C	D	E	F
Ambient device temperature, average 60 minutes	7	*b*	0	11	11	
Ambient device temperature, average day	7	*b*	0	11	21	
Ambient device temperature, average month	7	*b*	0	11	31	
Gas parameters for volume conversion, currently used in compressibility calculation						
Reference pressure of gas analysis	7	*b*	0	12	8	
Reference temperature of gas analysis	7	*b*	0	12	9	
Superior Wobbe number 0 °C	7	*b*	0	12	10	
Inferior Wobbe number 0 °C	7	*b*	0	12	11	
Methane number	7	*b*	0	12	12	
Total sulphur	7	*b*	0	12	13	
Hydrogen sulphide H_2S	7	*b*	0	12	14	
Mercaptans	7	*b*	0	12	15	
Water dew point (DP H_2O)	7	*b*	0	12	16	
Water (H_2O) dew point outlet / normalised	7	*b*	0	12	17	
Hydrocarbon dew point (DP C_XH_Y)	7	*b*	0	12	18	
Inferior calorific value $H_{i,n}$	7	*b*	0	12	19	
Water H_2O	7	*b*	0	12	20	
Density (of gas), base conditions	7	*b*	0	12	45	
Relative density	7	*b*	0	12	46	
Superior calorific value $H_{s,n}$	7	*b*	0	12	54	
Nitrogen N_2	7	*b*	0	12	60	
Hydrogen H_2	7	*b*	0	12	61	
Oxygen O_2	7	*b*	0	12	62	
Helium He	7	*b*	0	12	63	
Argon Ar	7	*b*	0	12	64	
Carbon monoxide CO	7	*b*	0	12	65	
Carbon dioxide CO_2	7	*b*	0	12	66	
Methane CH_4	7	*b*	0	12	67	
Ethene C_2H_4	7	*b*	0	12	68	
Ethane C_2H_6	7	*b*	0	12	69	
Propene C_3H_6	7	*b*	0	12	70	
Propane C_3H_8	7	*b*	0	12	71	
i-butane i-C_4H_{10}	7	*b*	0	12	72	
n-butane n-C_4H_{10}	7	*b*	0	12	73	
neo-pentane neo-C_5H_{12}	7	*b*	0	12	74	
i-pentane i-C_5H_{12}	7	*b*	0	12	75	
n-pentane n-C_5H_{12}	7	*b*	0	12	76	
Hexane C_6H_{14}	7	*b*	0	12	77	
Hexane share higher hydrocarbons C_6H_{14} %	7	*b*	0	12	78	
Hexane+ C_6 H_{14}+	7	*b*	0	12	79	
Heptane C_7H_{16}	7	*b*	0	12	80	

© Copyright 1997-2020 DLMS User Association

General and shows service entry objects – Gas	OBIS code					
	A	B	C	D	E	F
Octane C_8H_{18}	7	*b*	0	12	81	
Nonane C_9H_{20}	7	*b*	0	12	82	
Decane $C_{10}H_{22}$	7	*b*	0	12	83	
Tetrahydrothiophene	7	*b*	0	12	84	
Gas parameters for Venturi measurement						
Internal pipe diameter	7	*b*	0	13	1	
Orifice diameter	7	*b*	0	13	2	
Pressure type (orifice fitting)	7	*b*	0	13	3	
Flow coefficient (alfa)	7	*b*	0	13	4	
Expansion coefficient (epsilon)	7	*b*	0	13	5	
Reflux coefficient	7	*b*	0	13	6	
Isoentropic coefficient	7	*b*	0	13	7	
Dynamic viscosity	7	*b*	0	13	8	
Differential pressure dp for cut off	7	*b*	0	13	9	
Reynold number	7	*b*	0	13	10	
Gas parameters for density measurement						
K0 Densimeter Coefficient	7	*b*	0	14	1	
K2 Densimeter Coefficient	7	*b*	0	14	2	
Densimeter period for instantaneous measurement	7	*b*	0	14	10	
Densimeter period for measurement period 15 minutes	7	*b*	0	14	11	
Sensor manager						
Sensor manager objects	7	*b*	0	15	*e*	
Internal operating status, gas related						
Internal operating status, global [h]	7	*b*	96	5	0	
Internal operating status (status word 1) [h]	7	*b*	96	5	1	
Internal operating status (status word 2) [h]	7	*b*	96	5	2	
Internal operating status (status word 3) [h]	7	*b*	96	5	3	
Internal operating status (status word 4) [h]	7	*b*	96	5	4	
Internal operating status (status word 5) [h]	7	*b*	96	5	5	
Internal operating status (status word 6) [h]	7	*b*	96	5	6	
Internal operating status (status word 7) [h]	7	*b*	96	5	7	
Internal operating status (status word 8) [h]	7	*b*	96	5	8	
Internal operating status (status word 9) [h]	7	*b*	96	5	9	
Manufacturer specific [i)] ..... Manufacturer specific	7 7	*b* *b*	96 96	50 99	e e	

[a]	A volume sensor could be an external mechanical meter / encoder / electronic index.
[b]	Absolute in the sense that negative volume is summed as positive ABS().
[c]	An absolute temperature or absolute pressure outside these limits may affect the error status of the device.
[d]	If multiple recording intervals are implemented, then recording interval 1 shall be the shorter.

© Copyright 1997-2020 DLMS User Association

General and shows service entry objects – Gas		OBIS code					
		A	B	C	D	E	F
e	This value indicates the remaining time interval for soft time setting, where the clock is corrected in small steps (equivalent to Clock object method 6).						
f	Temperature heating is applied by stations with gas heating systems.						
g	Application for control of battery environment or volume conversion device environmental control.						
h	Status words referring to a status table with fix status words or to any status table bits using mapped status (class_id = 63).						
i	The range D = 50…99 is available for identifying objects, which are not represented by another defined code, but need representation on the display as well. If this is not required, the range D = 128…254 should be used.						

7.8.6.2 Error register objects – Gas

Table 105 – OBIS codes for error register specifies the OBIS codes for gas related error register objects.

Table 105 – OBIS codes for error register objects – Gas

Error register objects – Gas	OBIS code					
	A	B	C	D	E	F
Error registers	7	*b*	97	97	*e*	
NOTE The information to be included in the error objects is not defined in this document.						

7.8.6.3 List objects – Gas

Table 106 – OBIS codes for list objects – Gas specifies the OBIS codes for gas related list objects.

Table 106 – OBIS codes for list objects – Gas

List objects – Gas	OBIS code					
	A	B	C	D	E	F
Gas related data of billing period (with billing period scheme 1 if there are more than one schemes available)	7	*b*	98	1	*e*	255 [a]
Gas related data of billing period (with billing period scheme 2)	7	*b*	98	2	*e*	255 [a]
Gas related data of billing period (with billing period scheme 3)	7	*b*	98	3	*e*	255 [a]
Gas related data of billing period (with billing period scheme 4)	7	*b*	98	4	*e*	255 [a]
Gas related data of event triggered billing profile [b]	7	*b*	98	11	*e*	255 [a]

a	F = 255 means a wildcard here. See 7.11.3.
b	Event triggered means the termination of a billing period by events, e.g. by commands. (Therefore, the profile entries are not equidistant in time).

7.8.6.4 Data profile objects – Gas

Gas related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data. The OBIS codes are specified in Table 107.

© Copyright 1997-2020 DLMS User Association

Table 107 – OBIS codes for data profile objects – Gas

Data profile objects – Gas	OBIS code					
	A	B	C	D	E	F
Load profile with recording interval 1	7	*b*	99	1	4 [a]	
Load profile with recording interval 2	7	*b*	99	2	4 [a]	
Profile of maxima with recording interval 1	7	*b*	99	3	4 [a]	
Profile of maxima with recording interval 2	7	*b*	99	4	4 [a]	
Load profiles for indexes and index differences of volume, mass and energy [b]	7	*b*	99	*d* [c]	*e* [d]	
Load profiles for process values	7	*b*	99	*d* [e]	*e* [f]	
Load profiles for flow rate	7	*b*	99	43	*e* [g]	
Power failure event log	7	*b*	99	97	*e*	
Event log	7	*b*	99	98	*e*	
Certification data log	7	*b*	99	99	0	
Load profile with recording interval 15 minutes	7	*b*	99	99	1	
Load profile with recording interval 60 minutes	7	*b*	99	99	2	
Load profile with recording interval day	7	*b*	99	99	3	
Load profile with recording interval month	7	*b*	99	99	4	

[a]	The value in value group E has been changed from 0 to 4 to avoid overlaps with the self-descriptive profile OBIS codes. The use of the value 0 is deprecated.
[b]	Value group D and E identify the value captured in these profiles. Value group D and E of the OBIS code of the load profile is mapped to value group C and D of the OBIS code identifying the value captured. The value captured in the buffer is always attribute 2 (value) of the respective Register / Extended register object.
[c]	The possible values are 1…8, 11…16, 21…26, 31…36, 61…66. See Table 94.
[d]	The possible values are 0…3, 6…98. See Table 95. EXAMPLE A load profile with OBIS code 7.b.99.11.17.255 contains the logged values from a volume conversion device: Forward undisturbed converter volume, index difference, value at base conditions, relative to measurement period 2. The values are captured at the end of each measurement period (last values).
[e]	The possible values are 41, 42, 44…49. See Table 94.
[f]	The possible values are 0, 2, 13, 24…32, 42…50, 60…68, 78…86, 90…92. See Table 101. EXAMPLE A load profile with OBIS code 7.b.99.41.43.255 contains the logged values of absolute gas temperature, average, last interval, (relative to) process interval 2.
[g]	The possible values are 0, 1, 2, 13, 19…22, 39…42, 59…62, 67…70. See Table 96. EXAMPLE A load profile with OBIS code 7.b.99.43.19.255 contains the logged values of the flow rate, last average for averaging period 1, value at metering conditions.

7.9 Water (Value group A = 8 and A = 9)

7.9.1 General

This subclause 7.9 specifies the naming of objects carrying water meter information in a COSEM environment. It covers the handling of hot, as well as the handling of cold water.

7.9.2 Value group C codes – Water

Table 108 specifies the use of value group C for hot and cold water.

Table 108 – Value group C codes – Water

Value group C codes – Water (A=8 or A=9)	
0	General purpose objects
1	Accumulated volume
2	Flow rate

© Copyright 1997-2020 DLMS User Association

Value group C codes – Water (A=8 or A=9)	
3	Temperature
93	Consortia specific identifiers, see Table 54.
94	Country specific identifiers, see Table 55.
96	General and service entry objects – Water (See 7.9.4.1)
97	Error register objects – Water (See 7.9.4.2)
98	List objects – Water
99	Data profile objects – Water (See 7.9.4.3)
128...199, 240	Manufacturer specific codes
All other	Reserved

7.9.3 Value group D codes – Water

This value group specifies the result of processing a *Quantity* according to a specific algorithm for water related values. See Table 109.

Table 109 – Value group D codes – Water

Value group D codes – Water (A = 8 or A = 9, C <> 0, 96...99)	
0	Current value
1	Periodical value
2	Set date value
3	Billing date value
4	Minimum of value
5	Maximum of value
6	Test value
All other	Reserved

7.9.4 Value group E codes – Water

Table 110 shows the use of value group E for identification of tariff rates typically used for consumption and demand quantities.

Table 110 – Value group E codes – Water

Value group E codes – Water	
0	Total
1	Rate 1
2	Rate 2
3	Rate 3
...	…
9	Rate 9
128...254	Manufacturer specific codes
All other	Reserved

© Copyright 1997-2020 DLMS User Association

7.9.5 OBIS codes – Water

7.9.5.1 General and service entry objects – Water

Table 111 specifies the OBIS codes for water related general and service entry objects.

Table 111 – OBIS codes for general and service entry objects – Water

General and service entry objects – Water	OBIS code					
	A	B	C	D	E	F
Free ID-numbers for utilities						
Complete combined ID	8/9	*b*	0	0		
ID 1	8/9	*b*	0	0	0	
…			…	…	…	
ID 10	8/9	*b*	0	0	9	
Storage information						
Status (VZ) of the historical value counter	8/9	*b*	0	1	1	
Number of available historical values	8/9	*b*	0	1	2	
Due date	8/9	*b*	0	1	10	
Billing date	8/9	*b*	0	1	11	
Billing date period	8/9	*b*	0	1	12	
Program entries						
Program version no.	8/9	*b*	0	2	0	
Device version no.	8/9	*b*	0	2	3	
Threshold values						
Contracted maximum consumption	8/9	*b*	0	5	1	
Input pulse constants						
Volume forward	8/9	*b*	0	7	1	
Measurement / registration-period duration						
Recording interval for load profile	8/9	*b*	0	8	1	
Time integral, averaging period for actual flow rate value	8/9	*b*	0	8	6	
Time entries						
Local time	8/9	*b*	0	9	1	
Local date	8/9	*b*	0	9	2	
Time stamp (local time) of the most recent billing period [a]	8/9	*b*	0	9	3	
Manufacturer specific [b]	8/9	b	96	50	*e*	*f*
.....						
Manufacturer specific	8/9	*b*	96	99	*e*	*f*

a	In case of billing period schemes absence or event triggered, commonly calculated from local date and local time information.
b	The range D = 50…99 is available for identifying objects, which are not represented by another defined code, but need representation on the display as well. If this is not required, the range D = 128…254 should be used.

7.9.5.2 Error register objects – Water

Table 112 specifies the OBIS codes for water related error register objects.

© Copyright 1997-2020 DLMS User Association

Table 112 – OBIS codes for error register objects – Water

Error register objects – Water	OBIS code					
	A	B	C	D	E	F
Error registers	8/9	*b*	97	97	*e*	
NOTE The information to be included in the error objects is not defined in this document.						

7.9.5.3 List objects – water meters

Table 113 specifies the OBIS codes for Water meter related list objects.

Table 113 – OBIS codes for list objects – Water Meters

List objects – Water Meters	OBIS code					
	A	B	C	D	E	F
Water Meter related data of billing period (with billing period scheme 1 if there are two schemes available)	8/9	*b*	98	1	*e*	255 [a]
Water Meter related data of billing period (with billing period scheme 2)	8/9	*b*	98	2	*e*	255 [a]
[a] F = 255 means a wildcard.						

7.9.5.4 Data profile objects – Water

Water related data profiles – identified with one single OBIS code – are used to hold a series of measurement values of one or more similar quantities and/or to group various data. The OBIS codes are specified in Table 114.

Table 114 – OBIS codes for data profile objects – Water

Data profile objects – Water	OBIS code					
	A	B	C	D	E	F
Consumption/load profile	8/9	*b*	99	1	*e*	

1.1.1.1 OBIS codes for water related objects (examples)

Table 115 – OBIS codes for water related objects (examples) specifies examples for OBIS codes of water related objects.

Table 115 – OBIS codes for water related objects (examples)

Water related objects	OBIS code					
	A	B	C	D	E	F
Consumption						
Current index, total	8/9	*b*	1	0	0	
Current index, tariff 1	8/9	*b*	1	0	1	
Current index, periodical, total, the two last periods	8/9	*b*	1	1	0	102

© Copyright 1997-2020 DLMS User Association

Water related objects	OBIS code					
	A	B	C	D	E	F
Monitoring values						
Flow rate, maximum value, previous period	8/9	*b*	2	5	0	V_{Z-1}
Forward temperature, billing date value, last billing period	8/9	*b*	3	3	0	101

7.10 Other media (Value group A= 15)

7.10.1 General

This subclause 7.10 specifies naming of objects related to other media than what is defined with values A = 1, 4…9. Typical application is distributed energy generation using renewable energy sources.

NOTE The details of OBIS codes will be specified as application of DLMS/COSEM in this area grows.

7.10.2 Value group C codes – Other media

Table 116 specifies the use of value group C for other media.

Table 116 – Value group C codes – Other media

Value group C codes – Other media	
0	General purpose objects
1…10	Solar
11…20	Wind
128…254	Manufacturer specific codes
All other	Reserved

7.10.3 Value group E codes – Other media

To be specified later.

7.10.4 Value group D codes – Other media

To be specified later.

7.10.5 Value group F codes – Other media

To be specified later.

7.11 Code presentation

7.11.1 Reduced ID codes (e.g. for IEC 62056-21)

To comply with the syntax defined for protocol modes A to D of IEC 62056-21:2002, the range of ID codes is reduced to fulfil the limitations which usually apply to the number of digits and their ASCII representation. Values in all value groups are limited to a range of 0…99 and within that range, to the values specified in the clauses specifying the use of the value groups.

Some value groups may be suppressed, if they are not relevant to an application:

- optional value groups: A, B, E, F;
- mandatory value groups: C, D.

To allow the interpretation of shortened codes delimiters are inserted between all value groups, see Figure 40:

© Copyright 1997-2020 DLMS User Association



IEC 304/02

Figure 40 – Reduced ID code presentation

The delimiter between value groups E and F can be modified to carry some information about the source of a reset (& instead of * if the reset was performed manually).

The manufacturer shall ensure that the combination of the OBIS code and the class_id (see Clause 4) uniquely identifies each COSEM object.

7.11.2 Display

The usage of OBIS codes to display values is normally limited in a similar way as for data transfer, for example according to IEC 62056-21:2002.

Some codes in value group C and D may be replaced by letters to clearly indicate the differences from other data items; see Table 117.

Table 117 – Example of display code replacement

Value group C and D	
OBIS code	**Display code**
96	C
97	F
98	L
99	P
NOTE The letter codes may also be used in protocol modes A to D.	

7.11.3 Special handling of value group F

Unless otherwise specified, the value group F is used for the identification of values of billing periods.

The billing periods can be identified relative to the status of the billing period counter or relative to the current billing period.

For electricity, there are two billing period schemes available in Table 72, each scheme defined by the length of the billing period, the billing period counter, the number of available billing periods and the time stamps of the billing period. See also 6.2.2 and 7.5.4.1.

For gas, there are four billing period schemes available, see Table 104.

With $0 \leq F \leq 99$, a single billing period is identified relative to the value of the billing period counter, VZ. If the value of the value group of any OBIS code is equal to VZ, this identifies the most recent (youngest) billing period. VZ_{-1} identifies the second youngest, etc. The billing period counter may have different operating modes, for example modulo-12 or modulo-100. The value after reaching the limit of the billing period counter is 0 for the operating mode modulo-100 and 1 for other operating modes (for example modulo-12).

With $101 \leq F \leq 125$, a single billing period or a set of billing periods are identified relative to the current billing period. F=101 identifies the last billing period, F = 102 the second last / two last billing periods, etc., F = 125 identifies the 25th last / 25 last billing periods.

F = 126 identifies an unspecified number of last billing periods, therefore it can be used as a wildcard.

© Copyright 1997-2020 DLMS User Association

F=255 means that the value group F is not used, or identifies the current billing period value(s).

For use of ICs for representing values of historical billing periods, see 6.2.2 and Table 118.

Table 118 – Value group F – Billing periods

Value group F	
VZ	Most recent value
VZ_{-1}	Second most recent value
VZ_{-2}	Third most recent value
VZ_{-3}	Fourth most recent value
VZ_{-4}	…
etc.	
101	Last value
102	Second / two last value(s)
….	
125	25th/25 last value(s)
126	Unspecified number of last values

7.11.4 COSEM

The usage of OBIS codes in the COSEM environment shall be as defined in Clause 6.

© Copyright 1997-2020 DLMS User Association

Annex A
(Informative)
Additional information on Auto answer and Auto connect ICs

NOTE This information is related to the “Auto answer” (class_id = 28, version = 2, see 4.7.5) and “Auto connect” (class_id = 29, version = 2, see 4.7.6) interface classes.

Since the capabilities (e.g. connection time, number of parallel connections) of communication networks (e.g. GPRS) are limited, devices e.g. meters are not permanently connected to the communication network.

Devices may connect to the network in regular intervals or on special events either to send unsolicited data or just to become accessible.

If a DLMS/COSEM client e.g. a Head End System needs to access a server e.g. a meter that is not connected to the communication network a wake-up request can be sent. This may be a wake-up call or a wake-up message, e.g. an SMS message. After successfully processing the wake-up request the device connects to network.

Figure A. 1 below shows an example for a GSM/GPRS communication network. Please note that the dashed lines represent the network services, the solid lines refer to possible application layer services.

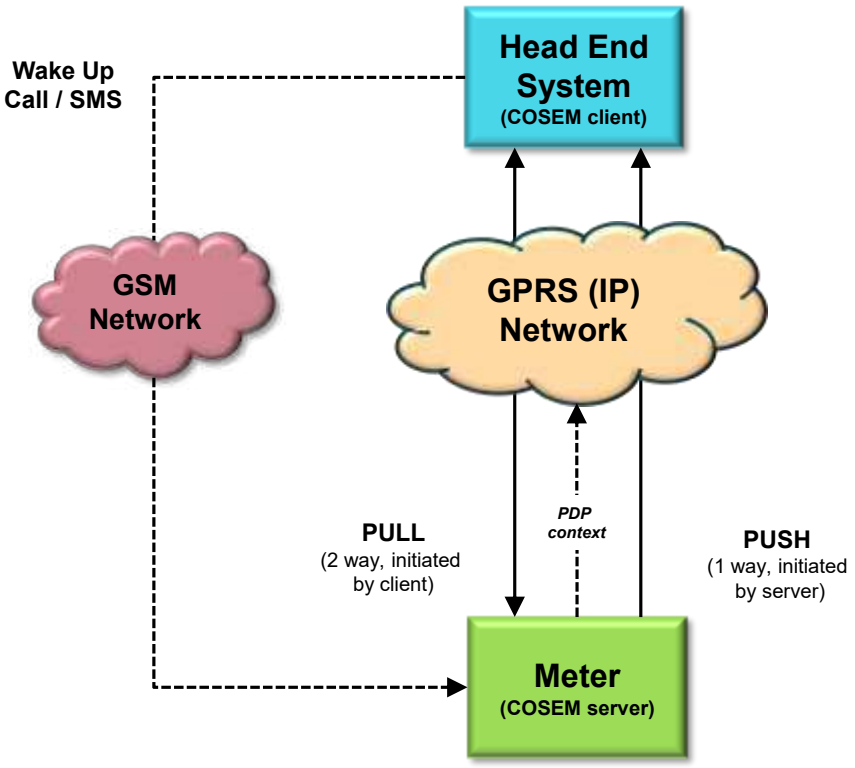


Figure A. 1 – Network connectivity example for a GSM/GPRS network

The basic network connectivity in the case of a mobile network (GPRS or equivalent service) is modelled by the “Auto connect” IC. Depending on the mode the connection can be 'always on', 'always on in a time window' or 'only on after a wake-up'. If the device is connected to the network it has the PDP context attached and it is accessible from by the HES via its IP address. If necessary the current IP address of the server (meter) can be sent to the client (HES) using the DataNotification xDLMS service.

© Copyright 1997-2020 DLMS User Association

The wake-up process is modelled using an instance of the “Auto answer” IC which provides additional security (check calling number) compared to today's solution.

Please also note that the “Auto answer” class is fully decoupled from the xDLMS application layer services. The main reason is to have a clear separation between the communication layers as well as to avoid creating an unsecured backdoor to execute application layer services with almost no protection. The execution of xDLMS services via SMS should be handled by sending ciphered xDLMS APDUs in a pre-established AA.

© Copyright 1997-2020 DLMS User Association

Annex B
(Informative)
Additional information to M-Bus client (class_id = 72, version 1)

State transitions of the *encryption_key_status* attribute for different use cases are shown in Figure B. 1.



Figure B. 1 – Encryption key status diagram

© Copyright 1997-2020 DLMS User Association

State transitions of the *encryption_key_status* attribute for different use cases are given below. At the time of installation of the slave four cases are possible:

c) Encryption key is preset in the slave and cannot be changed, see Table B. 1;

d) Encryption key is preset in the slave and new key is set after installation, see Table B. 2;

e) Encryption key is not preset in the slave, but can be set, see Table B. 3 and Table B. 4;

f) The slave is used without encryption. In this case, the *encryption_key_status* stays in state (0).

Table B. 1 – Encryption key is preset in the slave and cannot be changed

Step	State	Condition for state transition (event or successfully invoked method)
1	(2)	set_encryption_key
2	(3)	encrypted telegrams successfully exchanged
3	(4)	–

Table B. 2 – Encryption key is preset in the slave and new key is set after installation

Step	State	Condition for state transition (event or successfully invoked method)
1	(2)	set_encryption_key
2	(3)	transfer_key
3	(2)	set_encryption_key
4	(3)	encrypted telegrams successfully exchanged
5	(4)	–

Table B. 3 – Encryption key is not preset in the slave, but can be set, case a)

Step	State	Condition for state transition (event or successfully invoked method)
1a	(0)	set_encryption_key
2a	(1)	transfer_key
3a	(3)	encrypted telegrams successfully exchanged
4a	(4)	–

Table B. 4 – Encryption key is not preset in the slave, but can be set, case b)

Step	State	Condition for state transition (event or successfully invoked method)
1b	(0)	transfer_key
2b	(2)	set_encryption_key
3b	(3)	encrypted telegrams successfully exchanged
4b	(4)	–

© Copyright 1997-2020 DLMS User Association

Annex C
(Informative)
Additional information on IPv6 setup (class_id = 48, version = 0)

C.1 General

In most regards, IPv6 is a conservative extension of IPv4. Most transport and application-layer protocols need little or no change to operate over IPv6; exceptions are application protocols that embed internet-layer addresses, such as FTP or NTPv3.

IPv6 specifies a new packet format, designed to minimize packet-header processing. Since the headers of IPv4 packets and IPv6 packets are significantly different, the two protocols are not interoperable.

C.2 IPv6 addressing

The most important feature of IPv6 is a much larger address space than that of IPv4: addresses in IPv6 are 128 bits long, compared to 32-bit addresses in IPv4. Furthermore, compared to IPv4, IPv6 supports multi-addressing on one physical interface (global, unique or link local IPv6 addresses).

IPv6 addresses are typically composed of two logical parts: a 64-bit (sub-)network prefix used for routing, and a 64-bit host part used to identify a host within the network.

The formats allowed for an IPv6 address are shown in Figure C. 1 (see http://www.iana.org/assignments/ipv6-address-space/). Note that to facilitate the IPv6 address writing, a specific notation defined in RFC 4291 has been specified by IETF (e.g. FF00::/8).

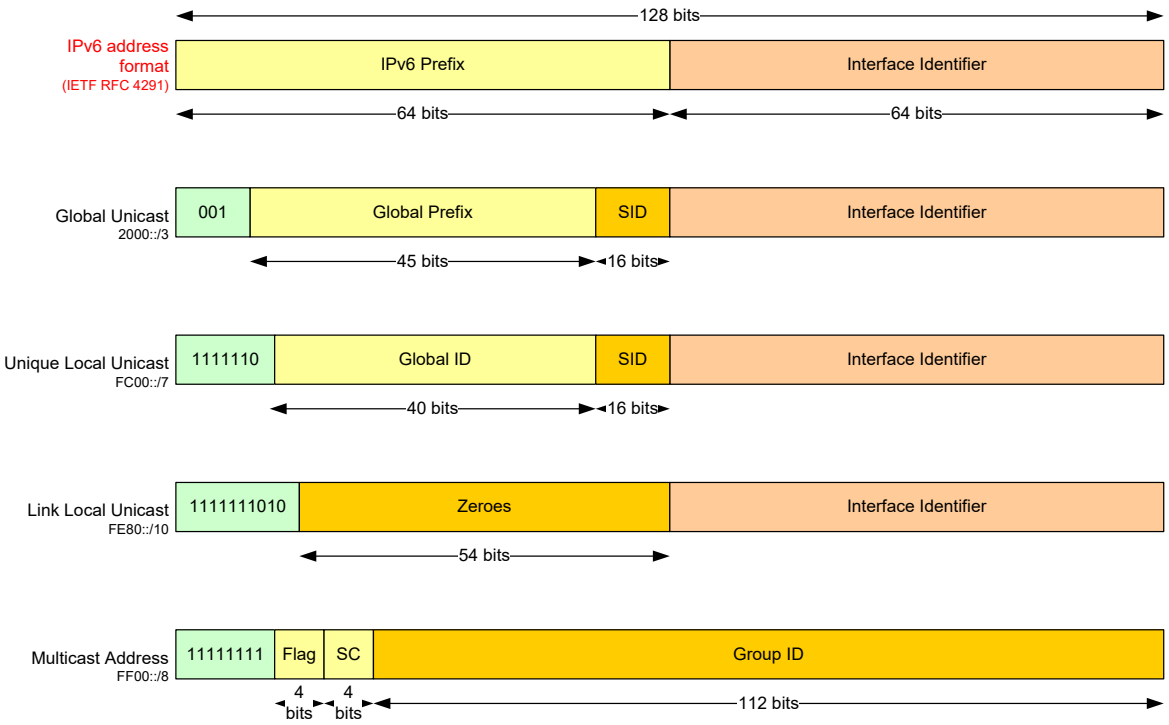


Figure C. 1 – IPv6 address formats

© Copyright 1997-2020 DLMS User Association

Where:

- *Global Unicast* is a routable address in the whole internet network and is composed as follows:
  - Global prefix assigned by IANA (see http://www.iana.org/assignments/ipv6-unicast-address-assignments/);
  - Subnet ID (SID) allocated by the network administrator; and
  - Interface Identifier either generated from the interface's MAC address (using modified EUI-64 format), or obtained from a DHCPv6 server, or assigned manually;
- *Unique Local Unicast* is an address only applicable to local network. This type of address is not routable outside the local network. The Global ID and the Subnet ID (SID) are allocated by the network administrator;
- *Link Local Unicast* is a unicast address allowed for a link local (without router). This type of address is not routable outside a local link;
- *Multicast* is an address assigned to different devices of the network. Following the scope (SC) of the address, the multicast group may be either Interface-local, Link-local, Admin-local, Site-local, Organization-local or global. For more information about Flag and SC (scope) parameters, see RFC 4291, 2.7.

It is important to note that there is no broadcast address defined in IPv6.

For more information concerning IPv6 addressing, see RFC 4291.

C.3 IPv6 header format

As defined in RFC 2460, Clause 3, the header of one IPv6 packet is composed as shown in Figure C. 2:

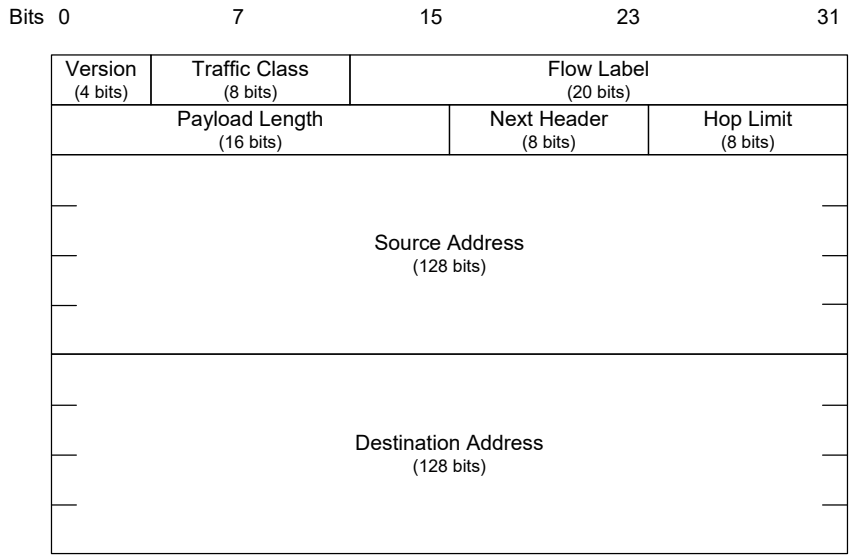


Figure C. 2 – IPv6 header format

Where:

- *Version* specifies the version of the protocol. For IPv6, the value is fixed and equals to 6;
- *Traffic class* is used by originating nodes and/or forwarding routers to identify and distinguish between different classes or priorities of IPv6 packets (see RFC 2474 Clause 3). Note that the traffic class value is only a notion of prioritization used to distribute the IPv6 frame and do not secure the transmission (the philosophy of the IP network is to do its best).

© Copyright 1997-2020 DLMS User Association

Figure C. 3 shows the content of the traffic class parameter:

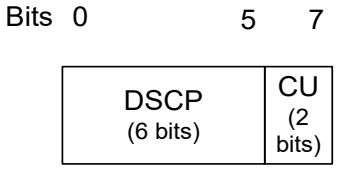


Figure C. 3 – Traffic class parameter format

Where:

- DSCP - Differentiated services code point contains the prioritization of the IPv6 packet in the network (see RFC 2474 for details);
- CU – Currently unused;
- *Flow label* may be used by a source to label sequences of packets for which it requests special handling by the IPv6 routers, such as non-default quality of service or "real-time" service. Currently, no fully definition of this field is given by IETF and may be seen as reserved for future used;
- *Payload length* indicates the size of the upper layer payload carried by the IPv6 frame;
- *Next header* identifies the type of header immediately following the IPv6 header of the current frame. It may indicate an upper applicative header (ICMP, UDP, TCP…) or extensions;
- *Hop limit* defines the time of life of the IPv6 packet in term of number of hop. The usage is similar to the one defined for IPv4 (decremented by 1 by each node that forwards the packet, the packet is discarded if Hop Limit is decremented to zero);
- *Source and destination addresses* indicate the originator of the IPv6 packet and the intended recipient.

Table C. 1 summarizes the IPv6 headers managed / not managed by the IPv6 objects.

Table C. 1 – IPv6 header vs. IPv6 IC

Parameters	IPv6 setup IC
Version	Not managed
Traffic class	Managed
Flow label	Not managed
Payload length	Not managed
Next header	Not managed
Hop limit	Not managed
Source and destination addresses	Not managed

C.4 IPv6 header extensions

C.4.1 Overview

Contrary to IPv4, IPv6 provides an extensible header by adding optional elements one by one. Currently, the following list of optional headers is defined by RFC 2460, see Table C. 1.

Table C. 2 summarizes the options managed / not managed by the IPv6 setup objects.

© Copyright 1997-2020 DLMS User Association

Table C. 2 – Optional IPv6 header extensions vs. IPv6 IC

Extensions	IPv6 setup IC	See subclause
Hop-by-Hop options	Not managed	C.4.2
Destination options	Not managed	C.4.3
Routing options	Not managed	C.4.4
Fragment options	Not managed	C.4.5
Security options (Authentication and Encapsulating Security Payload)	Not managed	C.4.6
NOTE The options are listed in the order as they appear in the packet.		

C.4.2 Hop-by-Hop options

The Hop-by-Hop options field must be examined by all devices on the path. Four elements currently defined may compose this header (see RFC 2460, 4.3):

- A padding form Pad1 which introduces one byte of padding (see RFC 2460, 4.2);
- A padding form PadN which introduces more than 2 bytes of padding (see RFC 2460, 4.2);
- A Jumbogram field to indicate the length of the IPv6 datagram in case of jumbo payload (higher than the maximum size of length field of IPv6 header) (see RFC 2460 and RFC 2675); and
- A router alert: The option indicates that the contents of the datagram may be interesting to the router (see RFC 2711).

These optional elements are directly managed by the IPv6 protocol stack. Therefore, they are not managed by the COSEM IPv6 setup class.

C.4.3 Destination options

The Destination options field must be examined only by the target device of the IP datagram. Four elements are currently defined by IETF (see RFC 2460, 4.6):

- A padding form Pad1 which introduces one byte of padding (see RFC 2460, 4.2);
- A padding form PadN which introduces more than 2 bytes of padding (see RFC 2460, 4.2);
- Mobility parameters (linked to new IP wireless networks – UMTS, LTE…) (see RFC 3775); and
- Tunnelling options which permit to communicate between two IPv6 networks separated by an IPv4 network (6To4 mechanism) (see RFC 2473).

NOTE The Mobility parameters and the Tunnelling options have been defined separately from RFC 2460 and are not mentioned in this document. See appropriate RFC's for more details.

These optional elements are directly managed by the IPv6 protocol stack. These optional fields are not managed by the COSEM IPv6 setup class.

C.4.4 Routing options

Routing options element is used to specify some routing parameters (see RFC 2460, 4.4). Currently, only one type is defined by IETF (type 2), which is used in case of mobility IPv6.

This notion is out of scope of the COSEM IPv6 setup class.

For information, due to an important security issue (see RFC 5095), the type 0 previously defined has been depreciated and is forbidden. Furthermore, the type 1 has been temporarily defined for experimental Nimrod and is obsolete since 1996.

© Copyright 1997-2020 DLMS User Association

C.4.5 Fragment options

Fragment options field is used in the case of fragmentation of the applicative payload if its size is higher than the MTU of the network (see RFC 2460, 4.5).

This element is directly managed by the IPv6 protocol stack. These optional fields are not managed by the COSEM IPv6 setup class.

C.4.6 Security options

Contrary to IPv4, IPv6 protocol layer includes natively the IPSec protocol. These extensions are fully defined in the RFC 4302 and RFC 4303.

The security options are composed of two extensible headers:

- Authentication Header (AH): Contains information used to verify the authenticity of most parts of the packet (see RFC 4302, Clause 2);
- Encapsulating Security Payload (ESP): Carries encrypted data for secure communication (see RFC 4303, Clause 2).

Due to the complexity of the IPsec protocol, the configuration of IPSec is out of scope of this document and must be treated separately.

© Copyright 1997-2020 DLMS User Association

Annex D
(Informative)
Overview of the narrow-band OFDM PLC technology for PRIME networks

For the specification of the PRIME narrow-band OFDM PLC setup classes, see 4.12.

NOTE This technology is supported by the PRIME Alliance, http://www.prime-alliance.org.

ITU-T G.9904:2012 specifies a physical layer, a medium access control layer and convergence layers for cost-effective narrowband (<200 kbps) data transmission over electrical power lines, intended for use in smart metering and smart grid applications. It is based on Orthogonal Frequency Division Multiplexing (OFDM).

The specification currently describes the following:

- a low-cost PHY capable of achieving rates of encoded 128 kbps;
- a Master-Slave MAC optimised for the power line environment;
- a convergence layer for the LLC layer specified in IEC 61334-4-32;
- a convergence layer for IPv4;
- a convergence layer for IPv6;

IEC 62056-8-4 specifies the DLMS/COSEM narrowband OFDM PLC profile for PRIME networks.

© Copyright 1997-2020 DLMS User Association

Annex E
(informative)
Overview of the narrow-band OFDM PLC technology for G3-PLC networks

For the specification of the G3 narrow-band OFDM PLC setup classes, see 4.13.

NOTE This specification is supported by the G3-PLC Alliance, http://www.G3-PLC.com.

ITU-T G.9903 specifies the physical, MAC and 6LoWPAN Adaptation layers of the G3-PLC technology while ITU-T G.9901 deals with frequency bandplan allocation and associated transmission level limitations.

Power line communication has been used for many decades, but a variety of new services and applications require more reliability and higher data rates. However, the power line channel is very hostile. Channel characteristics and parameters vary with frequency, location, time and the type of equipment connected to it. The lower frequency regions from 10 kHz to 200 kHz are especially susceptible to interference. Furthermore, the power line is a very frequency selective channel. Besides background noise, it is subject to impulsive noise often occurring at 50/60 Hz and group delays up to several hundred microseconds.

G3-PLC uses advanced modulation and channel coding techniques, which enables efficient use of the limited bandwidth of the CENELEC bands and facilitates communication over the power line channel. This combination enables a very robust communication in the presence of narrow-band interference, impulsive noise, and frequency selective attenuation. The specification addresses the following main objectives:

- provide robust communication on extremely harsh power line channels;
- provide a minimum of 20 kbps effective data rate in the normal mode of operation;
- ability to notch selected frequencies, to allow the cohabitation with other Narrow-band PLC communication technologies (e.g. IEC 61334-5-1:2001 S-FSK) or to be compliant with specific regulatory requirements;
- dynamic tone adaptation capability to select frequencies on the channel that do not have major interference, thereby ensuring a robust communication;
- access control, authentication, confidentiality and integrity to ensure high level of security.

To this end, the G3-PLC protocol stack aggregates several layers and sub-layers that form the G3-PLC profile:

- a robust high-performance PHY layer based on OFDM and adapted to the narrow-band PLC environment;
- a MAC layer of the IEEE 802.15.4 type (extended), well suited to low data rates;
- IPv6, the new generation of IP (Internet Protocol), which widely opens the range of potential applications and services; and
- to allow good IPv6 and MAC interoperability, an Adaptation sublayer taken from the Internet world (IETF) and called 6LoWPAN (RFC 4944 extended and RFC 6282). The adaptation sub layer also embeds the LOADng routing algorithm to allow multi-hop mesh connectivity.

For more information about the extensions of IEEE 802.15.4, RFC 4944 and RFC 6282 (AKA 6LoWPAN) standards, and LOADng, see ITU-T G.9903:2014.

IEC 62056-8-5 specifies the DLMS/COSEM narrowband OFDM PLC profile for G3-PLC networks.

© Copyright 1997-2020 DLMS User Association

Annex F
(informative)
Bibliography

IEC 61334-6	*Distribution automation using distribution line carrier systems – Part 6: A-XDR encoding rule*
IEC TR 62051:1999	*Electricity metering – Glossary of terms*
IEC TR 62051-1:2004	*Electricity metering – Data exchange for meter reading, tariff and load control – Glossary of terms – Part 1: Terms related to data exchange with metering equipment using DLMS/COSEM*
IEC 62056-46:2002 + AMD1:2006 Edition 1.1	*Electricity metering – Data exchange for meter reading, tariff and load control – Part 46: Data link layer using HDLC protocol*
IEC 62056-4-7	*Electricity metering – Data exchange for meter reading, tariff and load control – Part 47: COSEM transport layers for IPv4 networks*
IEC 62056-5-3	*Electricity metering data exchange – The DLMS/COSEM suite – Part 5-3: DLMS/COSEM application layer*
IEC 62056-6-1	*Electricity metering data exchange – The DLMS/COSEM suite – Part 6-1: Object identification system (OBIS)*
IEC 62056-6-2	*Electricity metering data exchange – The DLMS/COSEM suite Part 6-2: COSEM interface classes*
IEC 62056-6-9	*Electricity metering data exchange – The DLMS/COSEM suite Part 6-9: Mapping between the Common Information Model message profiles (IEC 61968-9) and DLMS/COSEM (IEC 62056) data models and protocols*
IEC 62056-7-3	*Electricity metering data exchange – The DLMS/COSEM suite – Part 7-3: Wired and wireless M-Bus communication profiles for local and neighbourhood networks*
IEC 62056-7-5	*Electricity metering data exchange – The DLMS/COSEM suite – Part 7-5 : Local data transmission profiles for Local Networks (LN)*
IEC 62056-7-6	*Electricity metering data exchange – The DLMS/COSEM suite – Part 7-6: The 3-layer, connection-oriented HDLC based communication profile*
IEC 62056-8-3	*Electricity metering data exchange – The DLMS/COSEM suite – Part 8 3: Communication profile for PLC S-FSK neighbourhood networks*
IEC 62056-8-4	*Electricity metering data exchange – The DLMS/COSEM suite – Part 8 4: Communication profiles for narrow-band OFDM PLC PRIME neighbourhood networks.*
IEC 62056-8-5	*Electricity metering data exchange – The DLMS/COSEM suite – Part 8-5: Narrow-band OFDM G3-PLC communication profile for neighbourhood networks*
IEC 62056-*8-20*	*Electricity metering data exchange – The DLMS/COSEM suite – Part 8-20: Mesh communication profile for neighbourhood networks*
IEC 62056-9-7	*Electricity metering data exchange – The DLMS/COSEM suite – Part 9-7: Communication profile for TCP-UDP/IP networks*
ISO 12213-3	*Natural gas – Calculation of compression factor – Part 3: Calculation using physical properties*
ISO/IEC 10646	*Information technology – Universal Coded Character Set (UCS)*
ISO/IEC 80000	*Quantities and Units*
EN 1434-3	*Heat meters - Part 3: Data exchange and interfaces*
EN 12405-1	*Gas meters - Conversion devices - Part 1: Volume conversion*
EN 12405-2	*Gas meters - Conversion devices - Part 2: Energy conversion*
ITU-T G.9901	*SERIES G: TRANSMISSION SYSTEMS AND MEDIA, DIGITAL SYSTEMS AND NETWORKS – Access Networks – In premises networks –* *Narrow-band orthogonal frequency division multiplexing power line communication transceivers – Power spectral density specification*

© Copyright 1997-2020 DLMS User Association

3GPP TS 36.214	*Technical Specification Group Radio Access Network; Evolved Universal Terrestrial Radio Access (E-UTRA); User Equipment (UE) procedures in idle mode.*
3GPP TS 36.214	*Technical Specification Group Radio Access Network; Evolved Universal Terrestrial Radio Access (E-UTRA); Physical layer; Measurements*
ITU Recommendation X.217	*Information technology – Open Systems Interconnection – Service definition for the association control service element*
ITU Recommendation X.227	*Information technology – Open Systems Interconnection Connection-oriented protocol for the association control service element: Protocol specification*
DIN 43863-3	*Electricity meters – Part 3: Tariff metering device as additional equipment for electricity meters – EDIS – Energy Data Identification System*
IETF STD 5	*Internet Protocol, 1981. (Also IETF RFC 0791, RFC 0792, RFC 0919, RFC 0922, RFC 0950, RFC 1112)*
AGA 8	*American Gas Association Report No 8. Compressiblity Factors of Natural Gas and other related Hydocarbon Gases*
Dictionary Gas 1997	*Multilingual Dictionary of the Gas Industry (International Gas Union IGU)*
RFC 793	*Transmission Control Protocol (Also IETF STD 0007), 1981, Updated by: RFC 3168. http://www.ietf.org/rfc/rfc793.txt*
RFC 940	*Toward an Internet Standard Scheme for Subnetting, 1985. http://www.ietf.org/rfc/rfc940.txt*
RFC 950	*Internet Standard Subnetting Procedure, 1985. http://www.ietf.org/rfc/rfc950.txt*
RFC 2460	*Internet Protocol, Version 6 (IPv6), 1998. http://www.ietf.org/rfc/rfc2460.txt*
RFC 2473	*Generic Packet Tunneling in IPv6, 1998. http://www.ietf.org/rfc/rfc2473.txt*
RFC 2675	*IPv6 Jumbograms, 1999. http://www.ietf.org/rfc/rfc2675.txt*
RFC 2711	*IPv6 Router Alert Option, 1999. http://www.ietf.org/rfc/rfc2711.txt*
RFC 3775	*Mobility Support in IPv6, 2004. http://www.ietf.org/rfc/rfc3775.txt*
RFC 4291	*IP Version 6 Addressing Architecture, 2006. http://www.ietf.org/rfc/rfc4291.txt*
RFC 4302	*IP Authentication Header, 2005. http://www.ietf.org/rfc/rfc4302.txt*
RFC 4303	*IP Encapsulating Security Payload (ESP), 2005. http://www.ietf.org/rfc/rfc4303.txt*
RFC 5095	*Deprecation of Type 0 Routing Headers in IPv6, 2007. http://www.ietf.org/rfc/rfc5095.txt*
ANSI/TIA-4957.200	*Layer 2 Standard Specification for the Smart Utility Network*

© Copyright 1997-2020 DLMS User Association

Index

“Function control 476
61334-4-32 LLC SSCS setup 291
Abstract object 494, 534, 535
ac_max_ctl_re_tx 293
Access 540
Access rights 50
Access selector 381
access_number 243
access_right 391
access_rights_list 91, 93, 383, 384
Account 30, 186, 189, 476, 483
account_activation_time 197
account_closure_time 197
account_mode_and_status 190
acknowledgement_time_t1 286
acknowledgement_timer 284
actions 167, 173, 182
activate_account 199
activate_passive_ 166
activate_passive_unit_charge 216
activated 265
activation_status 145
Active energy 558, 559, 560
Active initiator 25, 277
Active power 506, 546, 560
active_devices 335
active_initiator 275
active_mask 69
Activity calendar 162, 163, 476, 481
Actor, Arbitrator 181
add_authentication_key 266
add_function (data) 146
add_mask 69
add_object 100, 394, 399, 405
add_register 69
addr_state 241, 246
address_mask 246
adjacent_cells 433, 436
adjust_to_measuring_period 157
adjust_to_minute 157
adjust_to_preset_ time 157
adjust_to_quarter 157
adjustment_method 178
adp_broadcast_log_table 323, 473
adp_broadcast_log_table_entry_TTL 322, 471
adp_max_hops 319, 469
adp_max_join_wait_time 324, 473
adp_path_discovery_time 324, 473
adp_routing_table 322, 471
adp_weak_LQI_value 320, 469
Advanced power failure monitoring 498
AES-GCM-128 110
AES-GCM-256 110
aggregated_debt 194
Agreed_key 135
Alarm descriptor 117, 502, 543
Alarm filter 502, 543
Alarm monitor 117, 483
Alarm register 117, 502, 543
Alert condition 578
Algorithm 506, 507
All configured 274
All Physical 274
always repeater 273, 445
amount_to_clear 193
Ampere-squared hours . 546, 554, 558, 559
Angles 546
APN 232
Apparent energy 558, 559, 560
Apparent power 507, 546, 560
Application context 50
application_context_name 97, 392, 397, 402
aps_interframe_delay 334
aps_max_window_size 334
Arbitrator 34, 180, 496
Array manager 476
array_object_list 148
associated_partners_id 96, 391, 396, 402
Association 476, 492
Association LN 42, 89, 389, 394, 400
Association object 288
Association SN 41, 42, 89, 378, 380, 382, 385
Association view 50, 51
association_status 98, 393, 398, 404
Attribute 20, 40, 42, 43, 342, 346, 348, 352, 354, 358, 362, 363
Attributes 42
Authenticated request 110, 136
Authenticated response 110, 136
Authentication 379
Authentication key 112
Authentication mechanism 50
authentication_keys 266
authentication_mechanism_name 98, 393, 398, 403
Auto answer 226, 428
Auto connect 229, 430
Auto dial 429, 430
Auxiliary supply 541
avail_baud 241
available_credit 193
Average value 559, 592, 595
Averaging period 590, 604

© Copyright 1997-2020 DLMS User Association

Averaging scheme 550
A-XDR 627
backup_HAN 338
Base conditions, gas 576
Base node 26, 289
base_name 41, 90, 380, 383, 386
base_node_address 291
Basic/nominal current 559
Battery 498, 540
Beacon slot 26
Billing period 480, 504, 511, 515, 520, 527, 538, 539, 544, 550, 556, 557, 558, 559, 571, 582, 585, 587, 608, 614
Billing period counter ... 478, 539, 557, 615
Billing period for indexes and index differences 604
Billing period scheme, Gas 605
Billing period values / reset counter entries 601
Billing period, end of 482
Block demand 65, 68
Broadcast 480
Broadcast address 221, 277, 424
broadcast_frames_ 251, 280
buffer 71, 75, 372
bus_address 241
busy_state_timer 285
CAD 28
Calculation methods, Gas 525, 599
Calendar 163
Calendar day 582
Calendar month 582
calendar_name 164
Calibration 540
CALLING device address, ... 221
Calling line identification 228
calling_window 231, 430, 431
Calorific value 576, 577, 579
capture 74, 78, 373, 441
Capture period 495, 502, 504, 508, 511, 515, 520, 527
capture_definition 243, 439
capture_method 377
capture_objects 72, 372
capture_period 72, 372, 439
capture_time 64, 67, 251, 436
Captured object 504, 508, 511, 515, 520, 527
Cardinality 42
cell_info 433, 435
Certificate 111, 128
Certificate Signing Request 114
Certificate, export 114
Certificate, import 114
Certification data 562
Certification data log, Gas 609
Challenge 382, 394
change_HLS_secret 100, 379, 382, 394, 399, 405
change_LLS_secret 379, 382
change_secret 93, 385, 389
changed_parameter 175, 420
Channel 535
channel_Id 250
CHAP protocol 264
CHAP_algorithm 264
Charge 30, 187, 210, 476, 483
Charge collection history 544
charge_configuration 213, 215
charge_reference_list 194
charge_type 211
Christmas 163
CIASE 271, 444
class_id 42, 95, 390, 395, 400
clearance_threshold 194
Client user identification 89
client_key 266
client_SAP 96, 124, 391, 396, 402, 417
client_system_title 110, 406
Clock 52, 155, 372, 476, 479
Clock synchronization method 505
Clock time shift limit 560
clock_base 157
close_tunnel_timeout 341
Coefficient 560
Cold water 534
collect 30
comm_speed 221, 223, 225, 247, 423, 426
Commodity 30, 33
Communication channel 532
Communication port 542
Communication port log parameter 499
Communication tamper event 501
communication_window 123, 410, 417
Compact data 477, 495
compact_buffer 375
Compressibility 524, 578, 581, 595
Compressibility calculation 606
Compressibility factor 579
Compressibility, correction and conversion values – Averages 598
Configuration 244, 496, 602
Configuration data 42
Configuration program 540, 558
Conformance block 97, 392, 397, 403
connect_logical_device 101
Consortia specific 533, 535, 536, 546, 563, 569, 584, 610
Consumer message 500, 542
consumption_based_collection 187
consumption_based_credit 187
Context specific 535
Contracted value 504, 549
control_mode 170, 171
control_state 170

© Copyright 1997-2020 DLMS User Association

Convergence layer, PRIME NB OFDM PLC 625
Conversion 524
Conversion factor..512, 516, 521, 579, 603
Converted value.....585, 590
Converter..... 583
Corrected value 585, 590
Corrected volume..... 579
Correction..... 524
Correction factor 578
COSEM Conformance Test Process 20
COSEM Glossary of Terms..... 20
COSEM logical device name..... 50, 51
COSEM Object Identification System (OBIS)..... 532
COSEM objects, Electricity..... 504
COSEM objects, Gas 520
COSEM objects, HCA..... 511
COSEM physical device 267, 288
Country specific533, 535, 536, 547, 563, 569, 584, 610
cpas_address 327
cpas_ether_type 327
CRC_NOK_frames_ 251, 281
CRC_OK_frames_.....251, 281
create_HAN 340
Credit 186, 199, 204, 476, 483
credit mode..... 30
Credit states 200
credit token..... 33
Credit, emergency..... 30
Credit, enabled 200
Credit, exhausted..... 200
Credit, in use 200
Credit, priority 200
Credit, selectable 200
Credit, selected/invoked 200
credit_available_threshold..... 207
credit_charge_configuration..... 195
credit_configuration..... 205
credit_reference_list 194
credit_status 206
credit_type 204
cs_attachment.....433, 434
CT/VT ratio 506
Cumulative maximum 548
Cumulative minimum 548
Cumulative value 504
currency 197
Current 546
Current and last average values 504
Current association 51
Current average..... 548, 550, 590
current_average_value..... 64, 67
current_credit_amount..... 204
current_credit_in_use 191
current_credit_status..... 192
current_user..... 92, 99, 388, 404
Currently active tariff 500
Cuts 562
Data52, 57, 77, 166, 495, 498, 502, 505, 506, 509, 512, 516, 521, 528
Data access error 379
Data format 43, 45
Data logger 577
Data model..... 20
Data profile 582
Data profile objects – Abstract 544
Data profile objects – Electricity 561
Data profile objects – HCA... 514, 519, 567
Data profile objects – Water.....612
Data profile objects – water meter.....531
Data protection, COSEM127, 130, 476, 493
Data security, access..... 51
Data security, transport..... 51
Data type.....43, 44
data_index 72, 408, 411
data_send 442
Date and time format 45
day_profile_table 165
Daylight saving..... 155, 162
daylight_savings_ deviation.....157
daylight_savings_begin.....156
daylight_savings_end.....156
Default encryption key 442
default_baud..... 219, 240, 422
default_mode 219, 422
delete 161, 163
delete_authentication_key.....266
delete_mask..... 69
Delimiters.....614
delta_electrical_phase..... 270, 443
Demand register.....52, 64, 68, 69, 504
Demotion..... 28
Density measurement 607
destination_list..... 232, 432
Desynchronization 277
desynchronization-listing.....280
Device ID 495, 540
Device measuring principle 512
Device start-up 292
device_addr 220, 423
device_address..... 221, 246, 424
device_serial_number.....426
device_type..... 243, 249, 440
DHCP.....255
Digital signature, profile entry 500
Digitally signed request..... 110, 136
Digitally signed response 110, 136
Dips 561
Direction of the gas flow 583
Disconnect control..... 168, 480, 482, 496
Disconnected state 27
DiscoverReport request 282
Display.....614
Display code.....614

© Copyright 1997-2020 DLMS User Association

Disturbance registers 578
DL_reference 253, 256
DLMS version 98, 393, 398, 403
Domain Name Server 255
Duration..... 549
dynamic repeater 273, 445
ECDH P-384 110
ECDSA P-256 110
Electrical port..... 484
Electricity.....534, 548, 551
Electricity breaker 496
Electricity ID 557
Emergency activation time..... 171
Emergency credit 30
Emergency duration 171
Emergency profile 171
Emergency profile, Limiter 172
Emergency threshold..... 171
emergency_credit..... 187
emergency_profile 172
emergency_profile_ative 172
emergency_profile_group 172
enable/disable 161
enable_disable_ 335
Enabled 30
Encoder..... 583
Encrypted request.....110, 136
Encrypted response 110, 136
Encryption key100, 244, 382, 394, 399, 405, 438, 442
encryption_key_status 244
encryption_mode..... 326
End of billing period 559
Energy calculation.....576, 579
Energy conversion 579
Energy measurement 576
Enterprise Resource Planning 31
Entries, schedule 160
Entries, special days 163
entries_in_use..... 73, 373
Environment..... 541
Environmental related parameters 499
Ephemeral Unified Model 113
equipment_id 426
Error correction..... 578
Error register485, 501, 502, 514, 518, 530, 535, 543, 547, 561, 563, 567, 569, 573, 584, 610, 612
Error registers – Abstract 543
Error registers – Electricity 561
Error registers – Gas..... 608
Error registers – HCA.....514, 518, 567
Error registers – Water..... 612
Error registers –water..... 530
Event code.....499, 541
Event counter 500, 542
Event log 503, 545, 562
Event log, Gas 609
Excess consumption metering 559
execute 159
executed_script 167
execution_time 168
export_certificate..... 114, 115
Extended register63, 68, 69, 77, 166, 177, 486, 498, 504, 505, 509, 512, 513, 516, 521, 522, 528
extended_pan_ID 331
Fatal error register.....223
fatal_error 425
Firmware 478
Firmware identifier.....539
Firmware signature 539
Firmware version 539
Floating point format..... 47
Flow rate 524, 597
Flow rate, gas.....590
Forgotten system 277
Frame synchronization.....277
frequencies 271, 444
Frequency 546, 559
Frequency notching, G3-PLC 626
Friendly credit 31
Full Function Device 300, 459
Function control..... 145, 480, 482, 494
function_list.....145
Fundamental 507
Fundamental component.....506
G3-PLC 626
G3-PLC 6LoWPAN adaptation layer 490
G3-PLC 6LoWPAN adaptation layer setup 301, 319, 468
G3-PLC MAC layer counters300, 302, 460, 490
G3-PLC MAC setup301, 304, 460, 461, 490
Gas..... 534, 576
Gas analysis..... 576, 583
Gas day 582
Gas ID number 520, 527
Gas law deviation 524
Gas month 582
Gas station management 521
Gas transport 577
Gas valve.....496
Gas year 582
gateway_IP_address.....255
General and service entry objects 539
General and service entry objects – Electricity 557
General and service entry objects – Gas 601
General and service entry objects – HCA 566
General and service entry objects – Thermal energy 571
General and service entry objects – Water 611

© Copyright 1997-2020 DLMS User Association

General purpose object 546, 551
generate_certificate_request 114
generate_key_pair 114
get_attributes&methods 381, 382
get_attributes&services 379
get_buffer_by_index 374
get_buffer_by_range 374
get_HLS_challenge 379
get_protected_attributes..... 129, 136, 137
getlist_by_classid 378
getobj_by_logicalname 378
Global broadcast encryption key 112
Global Positioning System 157
Global unicast encryption key 112
global_broadcast_encryption_key 135
global_key_transfer 407
global_unicast_encryption_key 135
GPRS modem setup..... 232, 236, 476, 487
group_id 325
GSM diagnostic233, 432, 434, 476, 487, 488, 492
GSM diagnostic profile 544
GSM field strength 542
Harmonics 506, 507, 509, 551, 562
Hayes standard.....226, 427
HCA ID number..... 511
HDLC mode 221, 423
HDLC setup 476, 485
HDLC setup class 220, 423
Head End System 31
Heat / cooling related objects 574
Heat cost allocator 534, 563
High resolution clock 479
Historical index 586
Historical values..... 532
HLS secret.....100, 382, 394, 399, 405
Hot water 534
HS-PLC ISO/IEC 12139-1 neighbourhood network 325
HS-PLC ISO/IEC 12139-1 networks 476
I/O control signal..... 496
ID numbers, Electricity 504
Identification number..... 438
Identification system 531
identification_number 243, 249, 440
Identified_key 135
identify_device 339
IEC 61334-4-32 LLC setup 488
IEC local port setup..... 476
IEC twisted pair (1) Fatal Error register 485
IEC twisted pair (1) MAC address setup485
IEC twisted pair (1) setup 222, 424, 485
IEEE address..... 29
IHD..... 28
Image 24, 108
Image activation.....480, 482
Image transfer.....89, 101, 102, 476, 494
Image, activation..... 109
Image, completeness..... 108
Image, initiate transfer..... 108
image_activate..... 105
image_block_size 102
image_block_transfer 104
image_first_not_transferred_block_number..... 102
image_to_activate_infos..... 104
image_transfer_enabled 102
image_transfer_initiate..... 104
image_transfer_status..... 104
image_transferred_blocks_status 102
image_verify 105
ImageBlock 25
ImageBlockNumber 25
ImageBlocks, transfer..... 108
ImageBlockSize..... 25, 105
ImageSize 25
import_certificate 114
In Home Display 28
Inactive objects 503, 535
inactivity_time_out 221, 253, 424
Index difference, Gas523, 528, 529, 585, 597
Index reading 576
Index, Gas..... 523, 585, 597
Information security 51
Information security, COSEM object 51
Information, measured 40
Information, static 40
initial_encryption_key 326
initialization_string..... 225, 427
Initiator..... 25, 274
initiator_electrical_phase 270, 443
initiator_mac_address 274, 446
Input control signals 540
Input pulse constant 521, 559, 603
Input pulse values 559
Input/output control signals 540
insert..... 161, 163
insert_entry.....150
Install code..... 28
Instances 42
Instantaneous value504, 509, 548, 559, 592
Instantiation ... 40, 41, 43, 57, 59, 166, 371
Intelligent search initiator..... 271, 276, 444
inter_octet_time_out..... 221, 424
Interface class 40, 52, 54
Interface object..... 40
Internal control signals..... 497, 540
Internal operating status497, 508, 523, 540, 560, 607
Internet 487
Interval.....521
Invocation counter 493
invoke_credit 208
invoke_protected_method 129, 138

© Copyright 1997-2020 DLMS User Association

IP_address 254
ip_alive_time..... 327
ip_header_comp_type 327
IP_options 254
IP_reference 252
IPCP_options..... 262
IPv4 setup 253, 476, 487
IPv6 addressing 620
IPv6 header 621
IPv6 setup 476, 487, 492
ISO/IEC 8802-2 LLC layer setup..... 476
ISO/IEC 8802-2 LLC Type 1 setup 283, 489
ISO/IEC 8802-2 LLC Type 2 setup 283, 489
ISO/IEC 8802-2 LLC Type 3 setup 285, 489
Key agreement..... 110
Key information..... 139
key_agreement 113
key_info..... 133
key_transfer 112, 113
Last average..... 548, 550, 590
Last value..... 615
last_average_value 64, 67
last_collection_amount..... 215
last_collection_time 215
last_outcome..... 184
LCP_options..... 261
Letter codes..... 614
Limit 615
Limiter 171, 476, 483
Line loss 546
Line reactance losses..... 560
Line resistance losses 560
Link key 29
link_key 332
List objects – Abstract.....535, 544
List objects – Electricity..... 508, 530, 561
List objects – Gas 526, 608
List objects – General 503
List objects – HCA 514, 567
List objects – Thermal Energy 519
List objects – Thermal Energy Meters .. 574
list_of_ 228
listening_window 227, 249, 428
LLC layer.....281, 446
LLC sub-layer 267
LLS secret 93, 382, 385, 389
LLS_secret 393
Load limiting 31
Load profile..... 544, 550, 559, 561
Load profile control 480, 482
Local date.....539, 559
Local port setup 219, 421, 484
Local time.....539, 559
local_disconnect 169
local_reconnect..... 169
Logger..... 583
Logical device42, 49, 50, 51, 90, 94, 101, 380, 382, 385, 394, 400
Logical Device Name41, 267, 288, 476, 493
Logical name.....41, 43
Logical name referencing..... 94, 394, 400
logical_name 40, 57, 102
login_password 264
low_credit_threshold.....198
LTE monitoring 476, 545
LTE_quality_of_service 437
MAC address..... 29, 275, 425
MAC address setup 476, 487, 490
MAC sub-layer.....267
mac_address..... 259, 271, 444
mac_coord_short_address 451
mac_freq_notching.....451
mac_get_neighbour_table_entry 453
mac_group_addresses 273, 445
mac_max_frame_retries 451
mac_max_orphan_timer 451
mac_neighbour_table_ 451
mac_PAN_id.....451
mac_security_enabled.....451
mac_short_address 451
mac_TMR_TTL 451
Magnitude 549
Maintenance, interface classes 371
Managed payment mode..... 31
Management Logical Device 50, 51, 288
Mandatory 42, 43
manual_disconnect.....169
manual_reconnect 169
Manufacturer ID.....438
Manufacturer specific42, 532, 535, 536, 543, 547, 550, 551, 558, 560, 564, 566, 571, 573, 584, 608, 611, 612
Manufacturer specific codes552, 563, 569, 610
manufacturer_id..... 249, 426, 440
Manufacturing number 540
mask_list..... 69
Master key 112, 128
master_key.....135
master_station_cpas_address 327
master_station_id 328
Max credit limit 484
Max vend limit 484
max_frame_length 282, 447
max_info_length_ transmit 221
max_info_length_receive..... 221, 424
max_info_length_transmit 424
max_number_transmissions_n2.....284
max_number_transmissions_n4.....285
max_octets_acn_pdu_n3 285
max_octets_i_pdu_n1.....284
max_octets_ui_pdu.....283
max_pdu_size.....249
max_provision 198
max_receiving_gain..... 271, 444
max_transmitting_gain 271, 444

© Copyright 1997-2020 DLMS User Association

Maximum 373, 585, 592
Maximum and minimum values 504
Maximum current 559
Maximum of index differences 586
Maximum of last averages 590
Maximum Segment Size 252
maximum_incoming_transfer_size ... 341
maximum_outcoming_transfer_size . 341
M-Bus 476
M-Bus client438, 448, 449, 450, 454, 486, 618
M-Bus control log object 486
M-Bus diagnostics 486
M-Bus disconnect control object 486
M-Bus master port setup 246, 486
M-Bus port setup 438
M-Bus profile control 480, 482
M-Bus profile generic object 438, 486
M-Bus slave 438, 486
M-Bus slave port setup 240, 486
M-Bus value object 438
M-Bus_Data _Header_Type 248
mbus_port_ 242
M-Bus_port_communication_state ... 248
mbus_port_reference 439
M-Bus_profile_ 248
Measurement channel 532
Measurement methods 560
Measurement period505, 550, 559, 582, 585, 586, 590, 604
Measurement period for indexes and index differences 604
Measurement, absolute 583
Measurement, disturbed 583
Measurement, undisturbed 583
Medium access control layer 625
Messaging 20
Meter 583
Meter connection diagram 558
Meter open event 501
Meter reset 480
Meter tamper 542
Meter tamper event 501
Metering conditions 576
Metering point ID 495, 507
Metering point ID (abstract) 540
Metering point ID (electricity related) 560
Method 20, 40, 43
Metrological LED 558
metrological_ 178
Metrology tamper event 501
MIB variables 267
min_delta_credit 445
min_over_threshold_duration 172
min_under_threshold_duration 172
Minimum 548, 592
mode 223, 227, 231, 428, 430, 431
Modelling 20
Modem 225, 226, 427, 429, 479
Modem configuration 225, 476, 479
modem_profile 427
Modulo-100 615
Modulo-12 615
monitored_value 166, 171, 331, 335
Most recent value 615
most_recent_requests_table 184
MSS 252
multicast_IP_address 254
Natural gas analysis 525, 595
Natural gas analysis values – Averages600
NB OFDM PLC for G3-PLC networks....476
NB OFDM PLC for PRIME networks 476
NB OFDM PLC profile for PRIME networks 288
nb_of_sim_conn 253
neighbor_discovery_setup 257
Network Control Protocol 262
network_key 332
Neutral current 546
Neutral voltage 546
Never repeater 273, 445
NEW 274
New system 26
New system title 26
next_credit_available_threshold 198
next_period 68
NO-BODY 274
Node 27
Nominal value 521, 559
Nominal values volume sensor 603
Non configured state 277
Nonce 139
Normal mode 481
Normal threshold 171
Notation 41
NTP 265
NTP protocol 265
NTP setup 476, 487
number_of_calls 228, 429
number_of_periods 64, 68
number_of_retries 123, 410, 417
number_of_rings 228, 429
OBIS 41, 43, 531
OBIS code structure 532
OBIS codes, HCA related objects 567
OBIS codes, water related objects 613
OBIS, Reserved ranges 533
Object codes 538
Object Identification System 475
*object_list*51, 90, 93, 95, 96, 99, 100, 378, 380, 381, 383, 386, 391, 393, 395, 396, 399, 401, 404
Occurrence counter 504, 549
One-Pass Diffie-Hellman 139
Operating time 498, 541
Operator 432, 434

© Copyright 1997-2020 DLMS User Association

Optical port 484
Optical test output 481
Optional 42, 43
originator_system_title 133
Orthogonal Frequency Division Multiplexing 625
Other media 534
other_information 133
Output control 482
Output control signals 540
Output pulse 558
Output pulse constant 505, 521
Output pulse constants converted / unconverted 602
Output signals 481
output_state 170
output_type 178
Over limit 551
p_bit_timer 285
Packet switched network 230
PAN coordinator 300, 459
pan_ID 331
Parameter 540
Parameter changes 496
Parameter monitor 173, 420, 476, 483
Parameter monitor log 173, 420, 544
Parameter record 558
parameter_list 175, 421
parent_station_id 326
pass_p1 ..... 220, 423
pass_p2 ..... 220, 423
pass_w5 ..... 220, 423
Passive calendar 166
Password 93, 379, 382, 385, 389, 423
Payment metering 186, 483
payment_event_based_collection 187
Period 64, 67
Permission 181
permissions_table 183
Personal Area Network 329
Phase angle 552
phone_list 430
PHY_reference 259
Physical device 49, 101, 492
Physical layer, PRIME NB OFDM PLC 625
PIN_code 232
PLC OFDM Type 1 Application identification 490
PLC OFDM Type 1 MAC counters 489
PLC OFDM Type 1 MAC functional parameters 489
PLC OFDM Type 1 MAC network administration data 490
PLC OFDM Type 1 MAC setup 489
PLC OFDM Type 1 physical layer counters 489
port_reference 124, 417
Positive displacement 576
Post-payment 32
Power factor 507, 546, 560
Power factor, displacement 507
Power factor, true 507
Power failure 161, 162, 253, 498, 541
Power failure duration 498
Power failure event log 562
Power quality 559
PPP setup 259, 476, 487
PPP_authentication 264
Preferred readout 74
preset_adjusting_ time 157
preset_credit_amount 206
Pressure 578
Primary address 438
primary_ 248
primary_address 243, 440
primary_address_list 223, 425
primary_DNS_address 255, 257
PRIME NB OFDM PLC Applications identification 299
PRIME NB OFDM PLC MAC counters 296
PRIME NB OFDM PLC MAC functional parameters 294
PRIME NB OFDM PLC MAC setup 293
PRIME NB OFDM PLC Physical layer counters 292
priority 200, 205, 211
Process data 583
Process interval 582, 592, 604
Process value 42, 58, 524, 597
Process value, gas 592
processed_value 179
processed_value_actions 180
processed_value_thresholds 179
Profile 371
Profile entries 495, 502, 504, 508, 511, 515, 520, 527
Profile generic 52, 70, 478, 495, 502, 504, 508, 511, 515, 520, 527
profile_entries 73, 373
Program entries 505, 539, 558
Promotion 27
prop_baud 220, 422
proportion 215
Proprietary 40
Protection parameters 127
protection_buffer 128, 129, 131
protection_object_list 131
protection_parameters) 124, 419
protection_parameters_get 133
protection_parameters_set 136
protocol_address 341
protocol_version 331
ps_status 433, 435
PSTN modem configuration 426
Public client 51
Pulse constant 558

© Copyright 1997-2020 DLMS User Association

Pulse duration 560
Pulse value 558
Pulses 546
Push 83, 116, 480, 482
Push setup 117, 476, 488
Push window 408
push_object_list 119, 408, 415
Quadrant 546, 553
quality_of_service 232
random delay 407
Random delay 412
randomisation_start_interval 123, 410, 417
Rate 541, 551, 564, 571, 611
raw_value 179
raw_value_actions 179
raw_value_thresholds 179
RCR program number 558
Reactive energy 558, 559, 560
Reactive power 506, 546, 547, 560
read_by_logicalname 93, 378, 379, 381, 384, 389
Reading factor 506, 558
Readout 75, 476
receive_lifetime_var_t2 287
receive_window_size_rw 284
received_signal_strength 250
recipient_system_title 133
Recording interval 559, 582, 604
Recording period 544, 550, 561
Reduced Function Device 300, 459
Reduced ID codes 614
Reference voltage 559
Register 40, 41, 58, 63, 68, 69, 77, 166, 498, 504, 505, 509, 512, 513, 516, 521, 522, 528
Register activation 68, 476, 482
Register monitor 166, 177, 476, 483, 509, 510
Register table 76
Register table objects – Abstract 544
Register table objects – Electricity 562
register_assignment 68, 69
register_device 336
Registered system 26
Registration 27
reject_timer 285
rejoin_interval 333
rejoin_retry_interval 333
remote_disconnect 169, 170
remote_reconnect 169, 171
remove_certificate 115
remove_entries 151
remove_function (data) 146
remove_HAN 340
remove_mirror 339
remove_object 394
remove_object (data) 100, 400, 405
repayable 32
repayment mode 32
repeater 273, 445
repeater_status 273, 326, 445
Repetition phase 278, 281
repetition_delay 123, 231, 411, 417, 430, 431
repetitions 231, 430, 431
repetitions_counter 281
reply_status_list 447
reply_to_HLS_ 93, 100, 385, 389, 394, 399, 405
reply_to_HLS_authentication 382
reply_to_HLS_challenge 379
Reporting system 26
Reporting system list 282
required_protection 136
Reserved base_names 41
Reserved credit 32
reserved_credit 187
reset 59, 64, 68, 74, 78, 180, 373
Reset 559, 605
Reset counter 511, 515, 520, 527
reset_account 199
reset_alarm 441
reset_NEW_not_ 276
response_time 220, 422
restore_HAN 338
retrieve_entries 149, 152
retrieve_number_of_entries 149
Ripple control receiver program 540
Robust Header Compression 263
rts/cts 326
SAP 101
SAP assignment . 41, 51, 89, 100, 476, 492
SAP_assignment_list 101
scaler_unit 59, 78, 179
scan_attempts 333
Schedule 159, 162, 163, 476, 481
Script 157, 160, 161, 167
Script table 41, 157, 163, 166, 167, 476, 480
Script, null 158
scripts 158
sealing_method 179
season_profile 164
secondary_address 425
secondary_DNS_address 255, 257
secondary_group_id 325
Secret 93, 98, 100, 385, 389, 394, 398, 403
Secret, new 382
Security setup 89, 109, 476, 493
Security suite 127
Security switches 540
security_activate 112, 407
security_policy 110, 406
security_setup_reference 92, 98, 384, 387, 398, 404
security_suite 110, 406

© Copyright 1997-2020 DLMS User Association

Selectable 32
Selected/Invoked 32
Selective access 43, 72, 73, 75, 81, 90, 92, 96, 99, 131, 376, 380, 383, 384, 386, 387, 391, 393, 396, 399, 401, 404, 409, 414
Selective access parameters 43
send_ 409
send_destination_and_method ..122, 416
sender_address 265
Sensor manager 177, 522, 607
Sensor, pressure 180
Serial number, sensor 178
serial_number 178
Server model 49
server_address 265
server_port 264, 265
server_SAP 96, 391, 396, 402
server_system_title 111, 406
Service node 27
Service Specific Convergence Sublayer 291
service_node_address 291
set_amount_to_value 208
set_encryption_key 442
set_function_status (data) 146
set_protected_attributes 129, 137
set_total_amount_remaining 216
S-FSK Active initiator 488
S-FSK MAC counters 488
S-FSK MAC synchronization timeouts .. 488
S-FSK Phy&MAC setup 488
S-FSK Physical layer 267
S-FSK PLC setup 476
S-FSK Reporting system list 489
Shared line 227, 428
shift_time 157
Short name 41, 43
short_address 331
Single action schedule 167, 476, 482
slave_deinstall 244, 441
slave_install 244, 441
Sliding demand 64, 68
Smart Energy Profile 332
SMTP setup 264, 487, 492
Social credit 33
Solar 613
Sort method 495, 502, 504, 508, 511, 515, 520, 527
sort_method 72, 373
sort_object 72, 373
Source of reset 614
Special days 159
Special days table 162, 163, 476, 481
stack_profile 331
Standard object codes 534
Standard readout profile 484
start_time_current 67
start_up_control 331
Static Unified Model 139
Station management information objects 605
station_id 325
status 64, 67, 156, 227, 429, 434
Status information, Electricity 560
Status mapping 78
Status register 499, 541
Status value 58
status_word 79
Strong DC magnetic field event 501
subnet_mask 255
Subnetwork 27
Sub-slot 26
superior calorific value 524
Swells 562
Switch node 289
Switch state 27
SYNCHRO_FOUND 279
Synchronization 162, 271, 445
Synchronization method 560
Synchronization process 278
Synchronization window 560
synchronization_confirmation_timeout 277
synchronization_locked 274, 446
synchronization_register 280
synchronization-locked 279
Synchronization-locked state 274, 446
Synchronization-unlocked state 274, 446
synchronize 266
synchronize_clock 245, 441
System Title 275
tabi_list 223, 425
table_cell_definition 77
table_cell_values 77
table_ID 75
Tamper 501
Tariff rates 551, 597
Tariffication 68, 480, 482
TCP-UDP setup 252, 476, 487
TCP-UDP_port 252
Telephone number 542
Temperature 578
template_description 376
template_id 376
Temporary debt 33
Terminal cover open event 501
Terminal node 289
Terminal state 27
Test mode 481
Test time integral 550
Test value 559
Thermal energy 534, 569
Thermal Energy meter ID number 515
threshold 166
Threshold 550, 557
Threshold value 171, 509, 512, 513, 516, 521, 528, 603

© Copyright 1997-2020 DLMS User Association

Threshold, missing 509, 549
Threshold, over limit.....509, 549
Threshold, under limit.....509, 549
threshold_active..... 171
threshold_emergency..... 172
threshold_normal 172
Tilt event 501
time 156
Time changes 162
Time entries505, 512, 516, 521, 528, 539, 559, 566, 573, 605
Time integral.....504, 548, 550
Time of operation 541
Time setting backward 162
Time setting forward 162
Time stamp.....539, 557
Time switch program 540, 558
Time synchronization..... 162
time_based_collection 187
time_based_credit..... 187
time_between_scans..... 333
time_out_frame_not_ 278
time_zone..... 156
Timeslot.....26
token 217
Token 33
Token carrier 33
Token Carrier Interface 217
Token credit history..... 544
Token gateway.....217, 476, 483
Token transfer log 544
token_credit 187
token_delivery_method..... 217
token_description 217
token_gateway_configuration..... 196
token_time..... 217
Top-up.....33
Total..... 551, 564, 571, 611
Total Demand Distortion..... 551
Total Harmonic Distortion 551
total_amount_paid..... 211
total_amount_remaining 215
transaction_id 133
transfer_key.....246, 442
Transformer and line loss 552
Transformer loss 546
Transformer magnetic losses..... 560
Transformer ratio – current (numerator)559
Transformer ratio – voltage..... 559
Transformer thermal losses 560
Transmission phase 278, 281
transmissions_counter 251, 281
transmit_lifetime_var_t3 287
transmit_window_size_k 284
Transporting 20
trust_center_address, ZigBee ®..... 332
Twisted pair 222, 424
Twisted pair setup..... 476
Unbalance.....562
Unconfigured.....441
Unconfigured state 276
Under limit.....550
unicast_IPv6_addresses.....257
UNIPEDE 555
UNIPEDE voltage dips 562
Unit..... 59
unit_charge_activation_time 213
unit_charge_active 212
unit_charge_passive.....213
Unitless quantities 559
UNIX clock 479
unregister_all_devices 337
unregister_device 337
update_amount.....207
update_entry.....151
update_link_key.....340
update_network_key.....340
update_total_amount_remaining 216
update_unit_charge 216
use_DHCP_flag 255
use_insecure_join.....332
User group specific..... 42
user_list.....92, 99, 388, 404
user_name 264
UTC 155, 156
Utility specific..... 533, 535
Utility tables 75, 477, 494
V.44 compression 110
Value 41, 57, 58
Value at base conditions..... 585, 590, 592
Value at metering conditions 585, 590, 592
Value at standard conditions..... 590, 592
Value group A 534
Value group B 534
Value group C 476, 535, 614
Value group C, Electricity.....546
Value group C, Gas 583
Value group C, HCA 563
Value group C, Other media.....613
Value group C, Thermal energy.....569
Value group C, Water 610
Value group D 504, 536, 556, 564
Value group D, Electricity.....548
Value group D, Gas 585
Value group D, HCA 564
Value group D, Other media.....614
Value group D, Thermal energy.....570
Value group D, Water 610
Value group E538, 551, 552, 555, 598, 599, 600
Value group E, Electricity.....551
Value group E, Gas 597
Value group E, Other media.....613
Value group F..... 509, 538, 614
Value group F, Electricity.....556
Value group F, Gas.....601

© Copyright 1997-2020 DLMS User Association

Value group F, Other media 614
Value groups, mandatory 614
Value groups, optional 614
Value, default 43
Value, maximum 43
Value, minimum 43
Van Jacobson compression 263
Vend 33
Venturi measurement 607
Version 42, 95, 371, 390, 395, 400, 440, 531
Version, previous 371
Voltage 546, 559
Voltage dips 555
Volt-squared hours 546, 554, 558, 559
Volume 578
Volume at base conditions 577, 579
Volume at metering conditions 579
Volume conversion 576, 577, 579, 606
Volume measurement 576
Wake-up 616
Wake-up request 229
warning_threshold 205
Water 610, 611, 613
Week day 559
week_profile_table 165
Weighting 181
weighting_table 183
Wind 613
window_size_receive 221, 424
window_size_transmit 221, 424
Wireless Mode Q 476, 486
Wireless Mode Q channel 246
Wrapped_key 135
write 374
xDLMS context 50
xDLMS_context_info 97, 392, 397, 402
year_of_manufacture 426
ZigBee® 28, 29, 329, 333, 409, 476, 491
ZigBee® 053474 23
ZigBee® 2007 330
ZigBee® client 29
ZigBee® cluster 29
ZigBee® Communications hub 329
ZigBee® coordinator 29, 329
ZigBee® mirror 29, 339
ZigBee® network control 334, 491
ZigBee® PAN 335
ZigBee® PAN creation 329
ZigBee® Pro 29, 330
ZigBee® router 29
ZigBee® SAS APS fragmentation 333, 334, 491
ZigBee® SAS join 332, 491
ZigBee® SAS startup 330, 491
ZigBee® server 29, 329
ZigBee® Smart Energy Specification 339
ZigBee® Trust Center 30, 329, 332
ZigBee® tunnel setup 340, 491

© Copyright 1997-2020 DLMS User Association